

ZERO TO RESEARCH

KP Regression

A First-Principles Beginner Edition

Python · Mathematics · Statistics · Regression · Survival Analysis · Causal Inference

Complete chapters with integrated companion guides

Contents

1. Chapter 0 — From Nothing to Ready
 2. Chapter 0 Companion Guide: From Nothing to Ready
3. Chapter 0: Level 0 Orientation — Landing on Planet Regression
4. Chapter 1 — From a Question to Ordinary Least Squares
 5. Chapter 1 Companion Guide: From a Question to Ordinary Least Squares
6. Chapter 2 — From a Fitted Line to a Trustworthy Model
 7. Chapter 2 Companion Guide: From a Fitted Line to a Trustworthy Model
8. Chapter 3 — From a Trustworthy Baseline to a Research-Grade Regression Study
 9. Chapter 3 Companion Guide: From a Trustworthy Baseline to a Research-Grade Regression Study
10. Chapter 4 — From Probabilities to Accountable Decisions
 11. Chapter 4 Companion Guide: From Probabilities to Accountable Decisions
12. Interlude — From a Single Moment to a History Through Time
13. Chapter 5 — From Event Labels to Event Histories
 14. Chapter 5 Companion Guide: From Event Labels to Event Histories
15. Chapter 6 — From Prediction to Intervention
 16. Chapter 6 Companion Guide: From Prediction to Intervention

Chapter 0 — From Nothing to Ready

Level 0 Base Camp: five short days before the MHP Cost Estimator begins

Central promise. By the end of this chapter, a summation sign will not make you flinch, a NumPy shape error will feel like a solvable puzzle instead of a personal insult, and the sentence "take the derivative of the squared error with respect to the slope" will sound like an instruction you could actually follow. You will not yet know any machine learning. You will know enough Python, algebra, statistics, and calculus that Chapter 1 reads like a hard but fair climb, not a cliff.

If that sounds ambitious, good news: nobody is expected to arrive knowing the vocabulary. Treat this chapter as a workshop, not an exam. Type the small examples, laugh at the inevitable error messages, and repeat an exit check whenever it still feels foggy.

This chapter exists because Chapter 1 does not slow down for you. By its second day it is talking about vectors, matrices, and tensors. By its fifth day it is deriving a matrix gradient and proving a matrix is positive semidefinite. That is exactly as it should be — this book is aiming you at research-level regression, causal inference, and survival analysis by Chapter 6, and there is no version of that destination that skips the math.

So we are going to spend five days building the floor you'll stand on. Same rules as every other chapter in this book: no black boxes, code proves every claim, you build something small each day, you break it on purpose, and you don't move on until the exit check passes.

If you already know some of this — if `for` loops and `numpy` arrays are old friends — skim fast, but still run every code block. Muscle memory matters more than novelty here.

The pedagogical contract (same one Chapter 1 uses)

1. **One central concept per day.**
2. **No mathematical hand-waving.** Every symbol gets unpacked and connected to a number you can see.
3. **Code as proof.** If a rule is true, you will watch Python confirm it.
4. **Build, break, rebuild.** Each day adds one working piece to a tiny toolkit; each day you also break something on purpose.
5. **No copy-pasting.** Type it. Your fingers need to learn this as much as your eyes do.

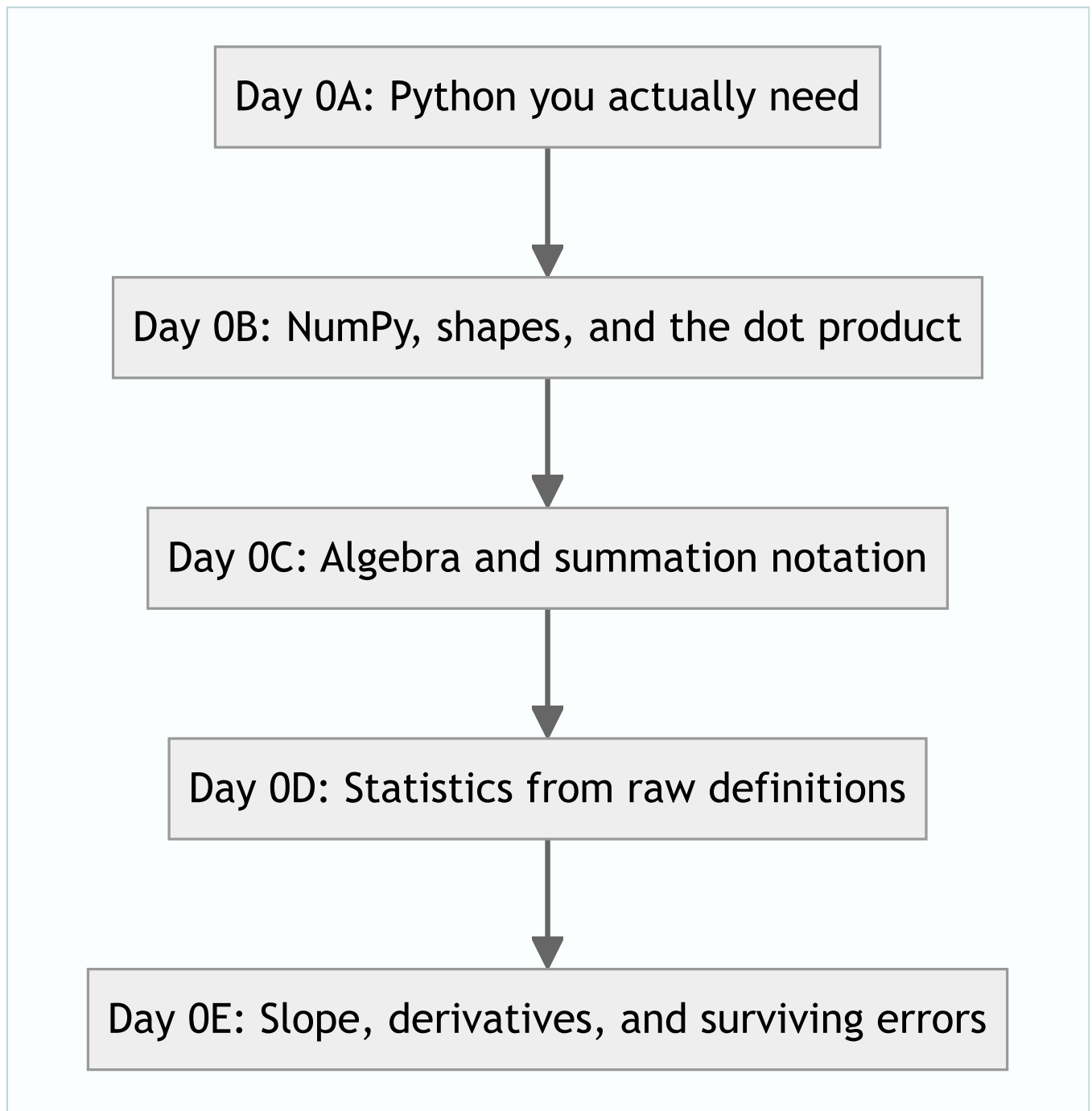
Level 0 learning outcomes

At the end of five days, you should be able to:

- write and reason about Python functions, loops, comprehensions, dictionaries, and a basic class, without looking up syntax for the basics;
- create, index, slice, reshape, and broadcast NumPy arrays, and explain *why* broadcasting did what it did instead of guessing;
- compute a dot product by hand, by loop, and with `@`, and know they are the same operation at three levels of abstraction;

- read and write summation notation ($\sum$), and translate between a formula and a line of code in either direction;
- compute a mean, variance, standard deviation, covariance, and correlation coefficient from raw definitions — not from a library you don't understand yet;
- compute a derivative and a partial derivative of a simple function, both by the limit definition and by rule, and explain what "gradient" means in one sentence;
- read a Python traceback methodically instead of panicking at it; and
- set up an isolated Python environment and generate the KP datasets this book runs on.

The five-day route



Minimal software setup

Same environment the rest of the book uses. If you already did this while reading the orientation chapter, skip ahead.


```
python3 -m venv .venv
source .venv/bin/activate      # Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install numpy matplotlib
```

Keep a scratch file — `level0_notes.py` or a notebook — and never delete a failed experiment. A wrong answer with a note explaining why it was wrong is worth more than a clean run you don't understand.

Day 0A — Python You Actually Need

Today's central idea: You don't need to be a "real programmer." You need six tools — variables, functions, loops, comprehensions, dictionaries, and one small class — used correctly and often.

0A.1 Variables and functions: naming a thought

Suppose your manager in Peshawar hands you the planned capacity of one microhydro power (MHP) site: 250 kilowatts. You want to convert that to megawatts.

```
capacity_kw = 250.0
capacity_mw = capacity_kw / 1000.0
print(capacity_mw)
# 0.25
```

That's a variable: a name you gave to a number so you don't have to keep re-typing `250.0` and hoping you remembered it correctly. Now suppose you need to do this conversion for every project that lands on your desk, forever. You don't write the same two lines a hundred times. You wrap the *behaviour* in a function, and give the *specific number* a name only when you call it.

```
def kw_to_mw(capacity_kw):
    """Convert kilowatts to megawatts."""
    return capacity_kw / 1000.0

print(kw_to_mw(250.0)) # 0.25
print(kw_to_mw(1800.0)) # 1.8
```

A function has a contract: it takes something in, and it promises something specific out. Every regression model you build in this book — starting with `MHPCostEstimator` in Chapter 1 — is, underneath, a pile of functions with contracts like this one, just with more numbers going in.

0A.2 Loops: doing the same thing many times, honestly

You have planned capacities for five projects and want each one in megawatts.

```
capacities_kw = [100.0, 250.0, 500.0, 1800.0, 60.0]

capacities_mw = []
for c in capacities_kw:
```

```
capacities_mw.append(kw_to_mw(c))

print(capacities_mw)
# [0.1, 0.25, 0.5, 1.8, 0.06]
```

Walk through this out loud (yes, actually say it): "For each value `c` in the list, convert it, and stick the result onto the end of a new list." That is the entire idea of a `for` loop. Nothing hides inside it.

0A.3 List comprehensions: the same loop, said more tersely

Python lets you write the loop above in one line. This is not a different idea — it is the *identical* operation, just written compactly enough that you'll see it constantly in other people's code (and in later chapters of this book).

```
capacities_mw = [kw_to_mw(c) for c in capacities_kw]
print(capacities_mw)
# [0.1, 0.25, 0.5, 1.8, 0.06]
```

Read it right to left inside the brackets: "for `c` in `capacities_kw`, compute `kw_to_mw(c)`, collect all of those." If a comprehension ever confuses you, the fix is always the same: mentally unroll it back into the `for` loop from §0A.2. They compute the same thing.

0A.4 Dictionaries: a record, not just a list

A list of numbers loses the *meaning* of each number. A single MHP project is not just `[15.0, 100.0, 3]` — it's cable length, capacity, and terrain difficulty, and you'll forget which number is which within a week. A dictionary keeps the label attached to the value.

```
project = {
    "site_id": "MHP-0042",
    "cable_length_km": 15.0,
    "planned_capacity_kw": 100.0,
    "terrain_index": 3,
}

print(project["planned_capacity_kw"])
# 100.0
```

This matters more than it looks like it should. Chapter 1, Day 2 introduces a "data dictionary" as part of the model itself — the discipline of never letting a raw number float around without its meaning attached starts here.

0A.5 A minimal class: bundling data and behaviour together

A function does one thing to whatever you hand it. A class is a way of saying: "here is a *thing* — it has some data that belongs to it, and some behaviour that belongs to it." You will build classes named `MHPCostEstimator` and `GaussianOLS` later in this book. Here is the smallest possible ancestor of those, so the syntax is not a surprise when it matters.

```
class UnitConverter:
    def __init__(self, factor):
        # This runs once, when the object is created.
        # It stores 'factor' so every method below can use it.
        self.factor = factor

    def convert(self, value):
        return value * self.factor
```

```
kw_to_mw_converter = UnitConverter(factor=0.001)
print(kw_to_mw_converter.convert(250.0))
# 0.25
```

`self` is just "this specific object." When you call `kw_to_mw_converter.convert(250.0)`, Python quietly hands the object itself in as `self`, so `self.factor` means "*this* converter's factor," not some factor floating in space. That's the whole trick. Everything else about classes in this book builds on exactly this.

0A.6 Build: a tiny project-cost sketch

Combine everything above into one small tool — not a real model yet, just proof you can chain these pieces together.

```
projects = [
    {"site_id": "MHP-0001", "cable_length_km": 12.0, "cost_per_km_million_pkr": 0.9},
    {"site_id": "MHP-0002", "cable_length_km": 30.0, "cost_per_km_million_pkr": 0.9},
    {"site_id": "MHP-0003", "cable_length_km": 5.0, "cost_per_km_million_pkr": 1.4},
]

def rough_cable_cost(project):
    return project["cable_length_km"] * project["cost_per_km_million_pkr"]

for p in projects:
    cost = rough_cable_cost(p)
    print(f"{p['site_id']}: approx {cost:.2f} million PKR of cable cost")
```

This is not a regression. It's a fixed formula, not a fitted one — the whole rest of the book is about the difference between "a formula I made up" and "a formula the data told me to use." Keep that distinction in your head; it will matter by Day 5.

0A.7 Break it deliberately

Run each of these, read the *last line* of the error only, and write one sentence translating it into English before you fix it.

```
# Break 1
def kw_to_mw(capacity_kw)  # missing colon
    return capacity_kw / 1000.0
```

```
# Break 2
project = {"cable_length_km": 15.0}
print(project["cost_per_km_million_pkr"]) # key that doesn't exist
```

```
# Break 3
capacities_kw = [100.0, 250.0, 500.0]
print(capacities_kw * 1000) # not what you think it does
```

The third one is the important one. `capacities_kw * 1000` does not multiply every number by 1000 — it repeats the *list* a thousand times, because `*` on a Python list means "repeat," not "scale." This single misunderstanding is the entire reason NumPy exists, and it's where Day 0B starts.

Day 0A exit check

You should be able to, without looking anything up:

- write a function with a docstring and a return value;
- write the same loop two ways (`for` and a comprehension);
- explain what `self` refers to inside a class method;
- explain, out loud, why `[1, 2, 3] * 2` is not `[2, 4, 6]`.

Day 0B — NumPy, Shapes, and the Dot Product

Today's central idea: A NumPy array is not "a faster list." It's a different kind of object with its own arithmetic rules, and almost every bug you'll hit in this book for the next six chapters is a shape mismatch.

0B.1 Why lists fail us

You saw it already: `capacities_kw * 1000` repeats a list instead of scaling it. NumPy arrays fix this by defining `*`, `+`, `-`, `/` to mean *elementwise* arithmetic.

```
import numpy as np

capacities_kw = np.array([100.0, 250.0, 500.0])
capacities_watts = capacities_kw * 1000.0
print(capacities_watts)
# [100000. 250000. 500000.]
```

0B.2 Shape is the first thing you check, always

Every array has a `.shape`. Get comfortable reading it before you do anything else with an array.

```
a = np.array([1.0, 2.0, 3.0])
print(a.shape)          # (3,) - a 1-D array of 3 numbers, a "vector"

b = np.array([[1.0, 2.0, 3.0]])
print(b.shape)          # (1, 3) - a 2-D array: 1 row, 3 columns

c = np.array([[1.0], [2.0], [3.0]])
print(c.shape)          # (3, 1) - a 2-D array: 3 rows, 1 column
```

`(3,)`, `(1, 3)`, and `(3, 1)` hold the same numbers and are **not interchangeable**. This looks pedantic until it silently ruins a calculation two chapters from now. Chapter 1's design matrices depend on you already having this reflex.

0B.3 Indexing and slicing

```
X = np.array([
    [15.0, 100.0, 3],    # cable_km, capacity_kw, terrain_index
    [30.0, 250.0, 4],
    [5.0, 500.0, 1],
```

```

])

print(X.shape)      # (3, 3) – 3 projects, 3 features
print(X[0])         # first row: array([ 15., 100.,   3.])
print(X[:, 0])      # first column (all rows): array([15., 30.,  5.])
print(X[0, 1])      # row 0, column 1: 100.0
print(X[:2, :2])    # first two rows, first two columns

```

The comma inside `[ ]` separates "which rows" from "which columns." `:` alone means "all of them." This exact slicing vocabulary is what Chapter 1 uses to pull a single feature column out of a design matrix — get it into your hands now, not while also learning what a design matrix is.

0B.4 Broadcasting: NumPy's rule for mismatched shapes

Broadcasting is *not* magic. It's one precise rule: NumPy compares shapes from the right, and a dimension of size 1 (or a missing dimension) is allowed to stretch to match. Watch it work, then watch it correctly refuse.

```

X = np.array([
    [15.0, 100.0],
    [30.0, 250.0],
    [5.0,  500.0],
]) # shape (3, 2): cable_km, capacity_kw

means = np.array([16.7, 283.3]) # shape (2,) – one mean per column

centred = X - means
print(centred)
# Each row of X has 'means' subtracted from it, column by column.

```

`(3, 2)` and `(2,)` are compatible because the trailing dimension `2` matches `2`. Now break it on purpose:

```

weights = np.array([1.5, 0.8, 2.1]) # shape (3,) – wrong length on purpose

result = X @ weights

```

```

ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0,
with gufunc signature (n?,k),(k?,m?)->(n?,m?) (size 3 is different from 2)

```

Translate the last line, not the jargon: *X has 2 columns; `weights` has 3 entries; matrix multiplication needs those to match.* Fix it by giving `weights` exactly 2 entries, one per column of `X`. This is the single most common error you will produce for the next six chapters, so get used to reading it calmly.

0B.5 The dot product, at three levels of honesty

A "weighted sum" is the single most important operation in this entire book — every regression coefficient you will ever fit multiplies a feature by a weight and adds the results up. Here it is proven three ways, so you know they're the same thing.

```

features = np.array([15.0, 100.0]) # cable_km, capacity_kw
weights  = np.array([0.05, 0.002]) # million PKR per unit

# Level 1: by hand, term by term
manual = features[0] * weights[0] + features[1] * weights[1]

# Level 2: by an explicit loop (works for any length)
loop_total = 0.0

```

```

for f, w in zip(features, weights):
    loop_total += f * w

# Level 3: NumPy's dot product
dot_total = features @ weights # equivalently: np.dot(features, weights)

print(manual, loop_total, dot_total)
# 0.95 0.95 0.95

```

All three give `0.95` — 0.75 (cable contribution) + 0.2 (capacity contribution) million PKR. When Chapter 1 writes `X @ beta`, this is exactly what it means: do this weighted sum, once per row of `X`, all at once.

0B.6 Build: a manual prediction, matrix style

```

X = np.array([
    [15.0, 100.0],
    [30.0, 250.0],
    [5.0, 500.0],
]) # 3 projects, 2 features each

beta = np.array([0.05, 0.002]) # made-up weights, not fitted – just practising the mechanics

predictions = X @ beta
print(predictions)
# one predicted cost per project, in the same units as beta implies

```

0B.7 Break it deliberately

```

row_vector = np.array([[1.0, 2.0, 3.0]]) # shape (1, 3)
col_vector = np.array([[1.0], [2.0], [3.0]]) # shape (3, 1)

print(row_vector + col_vector)

```

This does **not** error — it broadcasts to a `(3, 3)` result, which surprises almost everyone the first time. Before running it, predict the shape of the output. Then run it and see if you were right. If you were wrong, walk through §0B.4's rule again until the result stops being a surprise.

Day 0B exit check

You should be able to, without looking anything up:

- state the shape of any array you just created, without running `.shape`, and then confirm it;
 - explain broadcasting as one sentence about matching dimensions from the right;
 - compute a dot product three ways and get the same number each time;
 - read a `matmul` shape-mismatch error and say, in plain English, which two numbers disagree.
-

Day 0C — Algebra and Summation Notation

Today's central idea: $\sum$ is not a foreign symbol. It's a `for` loop that adds things up, written in a more compressed alphabet.

0C.1 A function is a recipe with a name

You already know this from Python — `def kw_to_mw(capacity_kw): return capacity_kw / 1000.0` is a recipe. Algebra just writes recipes with single letters instead of English words:

$$f(x) = \frac{x}{1000}$$

says exactly what the Python function said: give me a number x , I'll divide it by 1000 and call the result $f(x)$. Every formula in this book is a recipe like this one — the discipline is learning to read the recipe before panicking about the symbols.

0C.2 The equation of a line, and why it matters here

$$y = mx + c$$

m is the slope: how much y changes when x increases by exactly 1. c is the intercept: the value of y when x is 0. This is the entire idea behind linear regression with one feature — Chapter 1 spends its first three days building up to exactly this equation, generalised to many features. If you can already sketch $y = 2x + 5$ on paper and say what happens as x grows, you have the geometric intuition; the rest is notation.

0C.3 Summation notation, built from a loop you already wrote

Recall Day 0A's loop:

```
loop_total = 0.0
for f, w in zip(features, weights):
    loop_total += f * w
```

In algebra, if there are p features indexed $i = 1, 2, \dots, p$, this exact loop is written:

$$\sum_{i=1}^p f_i w_i$$

Read it left to right, the same way you'd read the `for` loop: "start a running total at 0; for each i from 1 to p , add f_i times w_i ; stop." The little number under $\sum$ is where the loop starts, the number on top is where it ends, and everything to the right of $\sum$ is what gets added at each step.

Code proof — the formula and the loop must agree:

```
import numpy as np

f = np.array([15.0, 100.0, 3.0])
w = np.array([0.05, 0.002, -1.5])
```

```
# The formula, as a loop
total_loop = 0.0
for i in range(len(f)):
    total_loop += f[i] * w[i]

# The formula, as NumPy
total_numpy = f @ w

print(total_loop, total_numpy)
# 0.55 0.55
```

$\sum_{i=1}^p f_i w_i$, the `for` loop, and `f @ w` are three spellings of one idea. When Chapter 1, Day 4 writes the sum of squared residuals as

$$\text{SSR} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

you should now be able to read it as: "loop over every project i ; take the actual cost minus the predicted cost; square it; add it to a running total." Try writing that as Python *before* Chapter 1 does it for you.

0C.4 Exponents and why we square errors (a preview)

x^2 means x multiplied by itself. Two properties matter for everything downstream:

- squaring always produces a non-negative number, so a positive error and an equally-sized negative error contribute the same amount once squared;
- squaring punishes large errors much more than small ones — an error of 10 contributes 100, an error of 2 contributes only 4.

Chapter 1, Day 4 spends real time justifying *why* regression squares errors instead of just adding them up. You don't need the full argument yet — just the two facts above, so the argument doesn't start from zero.

0C.5 Build: SSR from the formula, three ways

```
y_actual = np.array([12.0, 30.0, 8.0])
y_predicted = np.array([11.2, 31.5, 9.0])

# Level 1: by hand
errors = y_actual - y_predicted
squared_errors = errors ** 2
ssr_manual = squared_errors.sum()

# Level 2: as a literal loop mirroring the sigma
ssr_loop = 0.0
for a, p in zip(y_actual, y_predicted):
    ssr_loop += (a - p) ** 2

# Level 3: one line
ssr_online = np.sum((y_actual - y_predicted) ** 2)

print(ssr_manual, ssr_loop, ssr_online)
```

All three must match. If they don't, you have a bug — and finding it is exactly the kind of debugging Chapter 1 will ask of you constantly.

0C.6 Break it deliberately

Predict the output on paper before running:

```
values = np.array([2.0, -3.0, 5.0])
print(np.sum(values ** 2))    # sum of squares
print(np.sum(values) ** 2)    # square of the sum
```

These are **not equal**, and the difference between "sum of squares" and "square of a sum" is a mistake that will cost you real debugging time in later chapters (variance, in particular, depends on getting this order right). Say out loud why they differ before moving on.

Day 0C exit check

You should be able to:

- translate a $\sum$ formula into a `for` loop and into one line of NumPy, and get matching numbers all three ways;
- explain why $y = mx + c$ is the same shape of idea as Chapter 1's regression equation;
- explain the difference between $\sum x_i^2$ and $(\sum x_i)^2$ using a concrete 3-number example.

Day 0D — Statistics From Raw Definitions

Today's central idea: Mean, variance, and correlation are not library functions you call. They are formulas you can build from Day 0C's summation notation in about four lines each — and Chapter 2's probability content assumes you already know what they mean.

0D.1 The mean: the balancing point

Suppose five MHP projects had these actual costs, in million PKR: `[12.0, 30.0, 8.0, 45.0, 15.0]`.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

"Add everything up, divide by how many there are."

```
costs = np.array([12.0, 30.0, 8.0, 45.0, 15.0])

n = len(costs)
mean_manual = np.sum(costs) / n
mean_numpy = costs.mean()

print(mean_manual, mean_numpy)
# 22.0 22.0
```

0D.2 Variance: how far, on average, things sit from the mean

You can't just average the raw deviations from the mean — they always sum to zero (that's *what* "mean" means). So we square them first, borrowing exactly the trick from Day 0C.

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

```
deviations = costs - costs.mean()
print(deviations)          # [-10.  8. -14. 23. -7.]
print(deviations.sum())    # 0.0 — always, by construction

variance_manual = np.sum(deviations ** 2) / n
variance_numpy = costs.var()    # NumPy's default matches this population formula

print(variance_manual, variance_numpy)
```

A warning that will save you real confusion later: you'll sometimes see variance computed by dividing by $n - 1$ instead of n (the "sample" variance, `ddof=1` in NumPy). Chapter 2, Day 6 explains exactly why that correction exists. For now, just notice that `costs.var()` and `costs.var(ddof=1)` disagree, and don't assume it's a bug when you see it — it's a deliberate choice depending on what you're trying to estimate.

0D.3 Standard deviation: getting the units back

Variance is in squared units (million PKR², which means nothing to anyone). Standard deviation undoes the squaring:

$$\sigma = \sqrt{\sigma^2}$$

```
std_manual = np.sqrt(variance_manual)
std_numpy = costs.std()
print(std_manual, std_numpy)
```

0D.4 Covariance: do two things move together?

Now bring in a second variable — cable length per project, in km: `[12.0, 30.0, 5.0, 40.0, 15.0]`. Covariance asks: when one variable is above its mean, is the other one usually above its mean too?

$$\text{cov}(x, y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

```
cable_km = np.array([12.0, 30.0, 5.0, 40.0, 15.0])

x_dev = costs - costs.mean()
y_dev = cable_km - cable_km.mean()

cov_manual = np.sum(x_dev * y_dev) / n
cov_numpy = np.cov(costs, cable_km, ddof=0)[0, 1]

print(cov_manual, cov_numpy)
```

If both deviations tend to have the same sign (both above their means, or both below), each product is positive and covariance comes out positive: as cable length rises, cost tends to rise too. That's not a coincidence for this dataset — it's the whole reason regression will later find cable length a useful feature.

0D.5 Correlation: covariance with the units washed out

Covariance's size depends on the units you happened to measure in, which makes two covariances hard to compare. Correlation fixes that by dividing out each variable's own spread:

$$r = \frac{\text{cov}(x, y)}{\sigma_x \sigma_y}$$

```
r_manual = cov_manual / (costs.std() * cable_km.std())
r_numpy = np.corrcoef(costs, cable_km)[0, 1]

print(r_manual, r_numpy)
```

r always sits between -1 and 1 . Close to 1 : strong positive relationship. Close to -1 : strong negative. Close to 0 : little linear relationship. This single number is going to matter enormously in Chapter 1, Day 2, where you'll learn that two *nearly identical* correlated features (like literacy rate and school enrollment in the orientation chapter's development dataset) can quietly break a regression — that's called multicollinearity, and it's just "correlation close to 1" between features, wearing a bigger hat.

0D.6 Build: a tiny stats report

```
def describe(values, label):
    print(f"{label}: mean={values.mean():.2f}, std={values.std():.2f}, "
          f"min={values.min():.2f}, max={values.max():.2f}")

describe(costs, "cost (million PKR)")
describe(cable_km, "cable length (km)")
print(f"correlation(cost, cable_km) = {np.corrcoef(costs, cable_km)[0,1]:.3f}")
```

This four-line report is, in essence, what Exercise 0.1 in the orientation chapter asked you to build for the full KP datasets. Now you know what every number in it actually means, instead of trusting `.describe()` blindly.

0D.7 Break it deliberately

```
constant = np.array([5.0, 5.0, 5.0, 5.0, 5.0])
print(np.corrcoef(constant, costs))
```

This produces `nan`, not an error — correlation divides by `constant.std()`, which is `0.0`, and dividing by zero silently poisons the whole calculation with `nan` instead of crashing loudly. This is a real trap: a feature with zero variance (every project has the same terrain grade, say) will not throw an exception, it will quietly corrupt anything downstream that touches it. Get used to checking `.std()` for zero *before* you trust a correlation number.

Day 0D exit check

You should be able to:

- compute mean, variance, standard deviation, covariance, and correlation from the raw summation formulas, and match NumPy's built-ins;

- explain in one sentence why variance squares the deviations instead of just averaging them;
- explain what a correlation near **1** between two features will eventually mean for a regression model;
- explain why a zero-variance feature produces `nan` instead of an error, and why that's more dangerous.

Day 0E — Slope, Derivatives, and Surviving Errors

Today's central idea: A derivative is just the slope of a curve at one exact point, and every model you train in this book is, underneath, a search for the point where that slope hits zero.

0E.1 From a straight slope to a curved one

You already know $y = mx + c$ has one constant slope, m , everywhere. Now consider a curve: $y = x^2$. Its steepness is different at every point — flat near $x = 0$, steep far from it. The **derivative** is a formula that tells you the exact slope at any single point you pick.

For $y = x^2$, the derivative is:

$$\frac{dy}{dx} = 2x$$

- At $x = 0$: slope is $2(0) = 0$ — flat, the bottom of the bowl.
- At $x = 3$: slope is $2(3) = 6$ — climbing steeply.
- At $x = -3$: slope is $2(-3) = -6$ — descending steeply, mirror image.

0E.2 Where that formula actually comes from (the limit definition)

You don't have to memorise derivative rules as magic — they fall out of one idea: *look at the average slope between two points that are getting closer and closer together.*

$$\frac{dy}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Code proof — approximate this numerically and watch it converge to $2x$:

```
def f(x):
    return x ** 2

def numerical_slope(f, x, h):
    return (f(x + h) - f(x)) / h

x = 3.0
for h in [1.0, 0.1, 0.01, 0.0001, 0.000001]:
    print(f"h={h:<10} slope estimate={numerical_slope(f, x, h):.6f}")
# As h shrinks, the estimate creeps toward exactly 6.0 = 2 * 3
```

This is not a party trick. Chapter 1, Day 5 checks its hand-derived matrix gradient against exactly this kind of numerical approximation ("finite differences"), and Chapter 2, Day 9 builds gradient descent directly on top of it. You just ran a miniature version of both.

0E.3 Two derivative rules you'll actually use

You don't need a full calculus course — this book only leans on a handful of rules, repeatedly:

- **Power rule:** the derivative of x^n is nx^{n-1} . (This is where $x^2 \rightarrow 2x$ came from.)
- **Constant multiple:** the derivative of $a \cdot f(x)$ is a times the derivative of $f(x)$.
- **Sum rule:** the derivative of $f(x) + g(x)$ is just the derivative of $f(x)$ plus the derivative of $g(x)$ — you can differentiate term by term.

```
# Example: derivative of y = 3x^2 + 5x
# Power rule on 3x^2 -> 6x
# Power rule on 5x -> 5 (since x^1's derivative is 1*x^0 = 1)
# Sum rule combines them -> 6x + 5

def y(x):
    return 3 * x**2 + 5 * x

def dy_dx_by_rule(x):
    return 6 * x + 5

x = 2.0
print(dy_dx_by_rule(x))                # 17.0
print(numerical_slope(y, x, h=0.000001)) # ~17.0, confirms the rule
```

0E.4 Partial derivatives: slope when there's more than one dial to turn

Regression rarely has one adjustable number — it has one per feature. A **partial derivative** asks: "if I nudge *just this one* number and freeze everything else, how does the output change?" Notation-wise, ∂ replaces d to signal "there are other variables here, and I'm holding them still."

Take a toy error surface: $E(m, c) = (10 - (m \cdot 2 + c))^2$ — the squared error of predicting $y = 10$ at $x = 2$ using slope m and intercept c .

```
def error(m, c):
    prediction = m * 2 + c
    return (10 - prediction) ** 2

def partial_wrt_m(error_fn, m, c, h=1e-6):
    return (error_fn(m + h, c) - error_fn(m, c)) / h

def partial_wrt_c(error_fn, m, c, h=1e-6):
    return (error_fn(m, c + h) - error_fn(m, c)) / h

m, c = 1.0, 1.0
print("slope if we nudge m:", partial_wrt_m(error, m, c))
print("slope if we nudge c:", partial_wrt_c(error, m, c))
```

Both numbers together form the **gradient** — a small arrow made of "how much does the error change per dial, holding the others still." Chapter 1, Day 5 derives this gradient by hand for the full OLS objective, with one partial derivative per feature instead of two. You just did the two-dial version.

0E.5 Why any of this matters: descending toward zero error

The core trick behind fitting almost every model in this book: start somewhere, compute the gradient, take a small step in the direction that *decreases* error, and repeat until the gradient is (close to) zero.

```
m, c = 0.0, 0.0      # start with a bad guess
learning_rate = 0.01

for step in range(50):
    grad_m = partial_wrt_m(error, m, c)
    grad_c = partial_wrt_c(error, m, c)
    m -= learning_rate * grad_m
    c -= learning_rate * grad_c

print(f"m={m:.3f}, c={c:.3f}, error={error(m, c):.6f}")
```

Run it and watch the error shrink toward zero as `m` and `c` settle into values that make `m * 2 + c` land near `10`. This is a hand-built, miniature version of gradient descent — the real one arrives in Chapter 2, Day 9, with real data instead of one toy point.

0E.5a Symbolic bridge: differentiate squared error by hand

Numerical slopes tell us *what* a derivative is. Later chapters also manipulate symbols to obtain a reusable rule. Start with one observation, treating x , y , and c as fixed numbers while m is the adjustable slope:

$$E(m) = (y - mx - c)^2.$$

First expand the square. Using $(a - b)^2 = a^2 - 2ab + b^2$ with $a = y - c$ and $b = mx$:

$$E(m) = (y - c)^2 - 2(y - c)mx + m^2x^2.$$

Now differentiate term by term with respect to m :

$$\frac{dE}{dm} = 0 - 2(y - c)x + 2mx^2.$$

Factor the result to recover the compact chain-rule form:

$$\frac{dE}{dm} = -2x(y - mx - c).$$

For $x = 2$, $y = 10$, $c = 1$, and $m = 1$, the expanded polynomial is $E(m) = 81 - 36m + 4m^2$, so its derivative is $-36 + 8m = -28$ at $m = 1$.

```
def symbolic_gradient_m(x, y, m, c):
    return -2 * x * (y - m * x - c)

x, y, m, c = 2.0, 10.0, 1.0, 1.0
by_algebra = symbolic_gradient_m(x, y, m, c)
by_finite_difference = partial_wrt_m(error, m, c, h=1e-6)

print(by_algebra, by_finite_difference)
assert by_algebra == -28.0
assert abs(by_algebra - by_finite_difference) < 1e-4
```

The letters are placeholders, not mysterious objects. Expanding, differentiating term by term, refactoring, and checking numerically is the same workflow used for the matrix gradient in Chapter 1 and the regularisation derivatives in Chapter 3.

0E.6 Surviving the traceback

You will see red error text constantly for the rest of this book. A practitioner reads an error the way an investigator reads a scene: bottom line first, ignore the jargon, translate to English.

Shape mismatch (you already met this on Day 0B):

```
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0...  
(size 4 is different from 2)
```

→ "Two things that need matching lengths don't have matching lengths."

Missing key:

```
project = {"cable_length_km": 15.0}  
project["terrain_index"]
```

```
KeyError: 'terrain_index'
```

→ "You asked the dictionary for a label it doesn't have. Check spelling, check whether the data actually contains that field."

Wrong type:

```
"cost: " + 15.0
```

```
TypeError: can only concatenate str (not "float") to str
```

→ "Python won't silently guess how to combine a string and a number. Convert one of them explicitly: `"cost: " + str(15.0)`."

The instinct to panic and immediately paste the whole traceback into a search bar is understandable but skips the useful step: read the last line first, translate it into a plain sentence, *then* decide if you need help.

0E.7 The rule of the rubber duck

Put a mug, a duck, or an unimpressed cat on your desk. When something breaks, explain your code to it out loud, line by line, as if it has never seen Python. "Okay, here I create an array with three rows. Then I multiply it by a vector with... wait, how many entries does that vector have?" In a large fraction of cases you will catch the bug mid-sentence, before the duck says a word.

0E.8 Break it deliberately

```
def error(m, c):  
    prediction = m * 2 + c  
    return (10 - prediction) ** 2  
  
# Break: learning rate far too large  
m, c = 0.0, 0.0  
learning_rate = 5.0 # was 0.01  
  
for step in range(10):
```

```
grad_m = partial_wrt_m(error, m, c)
grad_c = partial_wrt_c(error, m, c)
m -= learning_rate * grad_m
c -= learning_rate * grad_c
print(step, m, c, error(m, c))
```

Watch the error *grow* instead of shrink, possibly exploding toward infinity. You just reproduced, in miniature, the single most common failure in Chapter 2, Day 9 — a learning rate that's too large overshoots the bottom of the valley on every step instead of settling into it. File that feeling away; you'll need to recognise it fast later.

Day 0E exit check

You should be able to:

- compute a derivative numerically (finite differences) and confirm it against a rule-based derivative;
- explain a partial derivative in one sentence to someone who's never seen the word;
- explain, without notes, what gradient descent is doing and why a too-large learning rate breaks it;
- read a `KeyError`, `TypeError`, and shape-mismatch `ValueError` and translate each into plain English before trying to fix it.

Level 0 Capstone — Prove You're Ready

Do this without peeking at earlier sections. This mirrors the format every later chapter uses for its own capstone, so it's worth taking seriously now.

Given:

```
projects = np.array([
    [12.0, 15.0], # cable_km, terrain_index
    [30.0, 25.0],
    [5.0, 8.0],
    [40.0, 45.0],
    [15.0, 12.0],
])
costs = np.array([12.0, 30.0, 8.0, 45.0, 15.0]) # million PKR
```

1. Report the shape of `projects` and explain what each dimension means, in a sentence.
2. Compute the mean and standard deviation of each column of `projects`, without using `.mean(axis=...)` — write the loop or comprehension yourself first, then confirm with NumPy.
3. Compute the correlation between `costs` and column 0 of `projects` from the raw covariance/std formulas, then check it with `np.corrcoef`.
4. Pick any made-up weight vector `beta` of length 2 and compute `predictions = projects @ beta` by hand-loop and with `@`, and confirm they match.
5. Write a function `sse(y_actual, y_predicted)` that returns $\sum (y_i - \hat{y}_i)^2$, and test it against your `predictions` from step 4.
6. Using finite differences, numerically estimate how `sse` changes if you nudge just the first entry of `beta` by a small amount. (You now have one entry of the gradient Chapter 1, Day 5 will derive properly.)
7. Deliberately break something — pass `beta` with the wrong length into step 4 — and write, in one sentence, what the resulting error is telling you.

If you can do all seven without reopening this chapter, you are ready for Chapter 1.

Glossary (symbols you'll now recognise on sight)

Symbol	Name	Meaning
x_i	indexed variable	the i -th value in a collection
$\sum_{i=1}^n$	summation	add up a formula for every i from 1 to n
$\bar{x}$	mean	average value
σ^2	variance	average squared distance from the mean
σ	standard deviation	square root of variance; same units as the data
r	correlation	covariance rescaled to sit between -1 and 1
$\frac{dy}{dx}, f'(x)$	derivative	exact slope of $y = f(x)$ at a point
$\frac{\partial E}{\partial m}$	partial derivative	slope of E with respect to m alone, holding other variables fixed
gradient	—	the collection of all partial derivatives, one per adjustable number

Instructor and self-study notes

- **Suggested timebox:** one focused day per section (0A–0E), or two half-days if math is genuinely new to you. Do not compress this into a single afternoon — the exit checks exist because rushing here is exactly what makes Chapter 1 feel impossible later.
- **Most common failure point:** learners skim Day 0B's broadcasting section because arrays "look like lists." Don't. Every day from Chapter 1 onward assumes shape-checking is reflexive, not effortful.
- **If the capstone is a struggle:** repeat Day 0C and 0D before moving on. Chapter 1 will not re-teach summation notation or variance; it will simply use them.

Where Chapter 1 begins

Chapter 1 opens with a question you're now equipped to actually think about: *what is a regression for* — *prediction, explanation, or causation*? Everything you built this week — functions, shapes, sums, means, slopes — is about to get one name each: features, design matrix, objective function, gradient. Turn the page.

Chapter 0 Companion Guide: From Nothing to Ready

A First-Principles Mathematical & Computational Reference Manual for Beginners

This companion guide is designed as an exhaustive, first-principles reference manual to accompany **Chapter 0: From Nothing to Ready**. It provides step-by-step mathematical derivations, intuitive mental models, line-by-line code breakdowns, and explicit structural mappings to ensure you never encounter a "black box" symbol or code snippet.

All mathematical notation is strictly formatted using standard LaTeX delimiters (`$\inline$` and ```

display

```), making it immediately compatible with Markdown renderers, MathJax, and KaTeX.

---

## Table of Contents

1. Pedagogical Philosophy & Operational Contract
  2. Day 0A — Python Foundations Refresher
  3. Day 0B — NumPy, Array Shapes, and Matrix Algebra
  4. Day 0C — Algebra, Summation Notation, and Loss Functions
  5. Day 0D — First-Principles Statistics
  6. Day 0E — Calculus, Gradients, Optimization, and Debugging
  7. Level 0 Capstone: Complete Step-by-Step Reference Solution
  8. Master Rosetta Stone & Traceback Cheat Sheet
- 
- 

## 1. Pedagogical Philosophy & Operational Contract

The primary goal of Chapter 0 is to construct the mathematical, computational, and statistical intuition required before encountering matrix regression, multivariate calculus, and statistical inference in Chapter 1.

### The Five Rules of the Pedagogical Contract

1. **One Central Idea Per Day:** Structured, incremental progression over five foundational days without skipping steps.
2. **No Mathematical Hand-Waving:** Every algebraic symbol, subscript, and operator is unpacked into concrete arithmetic and visible Python outputs.
3. **Code as Proof:** If a mathematical property or equation is claimed to be true, clean Python code will prove it numerically.

4. **Build, Break, Rebuild:** To truly understand how systems work, you must deliberately break them (e.g., shape mismatches, missing keys, learning rate explosions) and read the resulting errors calmly.
  5. **Type Everything:** Muscle memory in writing Python syntax and NumPy indexing is essential for effortless problem-solving.
- 

## 2. Day 0A — Python Foundations Refresher

### 2.1 Variables and Functions: State vs. Behavior

A **variable** assigns a symbolic label to a location in memory holding a specific value. A **function** encapsulates a reusable sequence of computations (a behavior) that accepts inputs, executes a contract, and explicitly returns an output.

```
Variable assignment: Storing a scalar value
capacity_kw = 250.0

Function definition: Defining an operational contract
def kw_to_mw(capacity_kw: float) -> float:
 """Convert electrical capacity from kilowatts (kW) to megawatts (MW)."""
 return capacity_kw / 1000.0

Calling the function
result_mw = kw_to_mw(capacity_kw)
print(result_mw) # Output: 0.25
```

#### Line-by-Line Breakdown:

- `def kw_to_mw(capacity_kw: float) -> float:`
  - `def` tells Python we are defining a new function named `kw_to_mw`.
  - `capacity_kw` is the **parameter** (the input variable expected by the function contract).
  - `: float` and `-> float` are optional type hints indicating that the expected input and output are floating-point numbers.
  - `"""Convert electrical capacity..."""`: The **docstring**, providing documentation accessible via `help(kw_to_mw)`.
  - `return capacity_kw / 1000.0`: The `return` statement explicitly sends the calculated result back to the caller. If `return` is omitted, Python returns `None` by default.
- 

### 2.2 Loops and List Iteration

A `for` loop iterates over a sequential collection (such as a list), binding each element to a loop variable one step at a time.

```
capacities_kw = [100.0, 250.0, 500.0, 1800.0, 60.0]

capacities_mw = []
for c in capacities_kw:
 capacities_mw.append(kw_to_mw(c))

print(capacities_mw) # Output: [0.1, 0.25, 0.5, 1.8, 0.06]
```

## Execution Mechanics:

1. `capacities_mw = []` allocates an empty list in memory.
  2. In the first iteration, `c` becomes `100.0`. `kw_to_mw(100.0)` evaluates to `0.1`, which is appended to `capacities_mw`.
  3. This process repeats sequentially for every element in `capacities_kw`.
- 

## 2.3 List Comprehensions: Syntactic Unrolling

A **list comprehension** is a concise way to create lists. It performs the exact same operation as a standard `for` loop with `.append()`, but in a single line.

```
List comprehension syntax: [expression for item in iterable]
capacities_mw = [kw_to_mw(c) for c in capacities_kw]
```

## Translation Mapping:

- **Expression:** `kw_to_mw(c)` (what gets placed into the new list).
- **Loop target:** `c`.
- **Iterable:** `capacities_kw`.

If a list comprehension ever feels confusing, unroll it back into a multi-line `for` loop with an explicit empty list and `.append()`.

---

## 2.4 Dictionaries: Label-Value Mapping

A standard list stores items by numerical index (`list[0]`, `list[1]`), which can lead to index-confusion errors when managing tabular data. A **dictionary** (`dict`) stores data using key-value pairs, maintaining explicit labels for every value.

```
project = {
 "site_id": "MHP-0042",
 "cable_length_km": 15.0,
 "planned_capacity_kw": 100.0,
 "terrain_index": 3,
}

Accessing a value by key
print(project["planned_capacity_kw"]) # Output: 100.0
```

## 2.5 Object-Oriented Encapsulation: The `self` Keyword

A **class** is a blueprint that combines state (data attributes) and behavior (methods) into a single object.

```
class UnitConverter:
 def __init__(self, factor: float):
 # __init__ is the constructor method.
 # It runs automatically when a new object instance is created.
 self.factor = factor # Store factor inside this specific instance.

 def convert(self, value: float) -> float:
```

```
Access the instance's stored factor using self.factor
return value * self.factor

Instantiate the class with a specific conversion factor (1/1000 = 0.001)
kw_to_mw_converter = UnitConverter(factor=0.001)

Call the method on the instantiated object
print(kw_to_mw_converter.convert(250.0)) # Output: 0.25
```

## What is self?

- `self` refers to the **current instance** of the object.
- When you execute `kw_to_mw_converter.convert(250.0)`, Python automatically translates it under the hood to `UnitConverter.convert(kw_to_mw_converter, 250.0)`.
- `self.factor` ensures that the method reads the specific factor stored inside *that* converter instance.

## 2.6 The Standard List Multiplication Trap

```
Standard Python List
capacities_kw = [100.0, 250.0, 500.0]

Unexpected Behavior:
print(capacities_kw * 2)
Output: [100.0, 250.0, 500.0, 100.0, 250.0, 500.0]
```

**Why does this happen?** In standard Python, the `*` operator applied to a list means **sequence repetition**, not mathematical scaling. To perform mathematical scaling on every element without writing explicit loops, we must use vectorized array operations provided by **NumPy**.

## 3. Day 0B — NumPy, Array Shapes, and Matrix Algebra

### 3.1 Why NumPy? Contiguous Memory and Vectorization

Standard Python lists are arrays of pointers to scattered memory locations. A **NumPy array** (`np.ndarray`) stores homogeneous numerical data in a **contiguous block of memory**. This allows NumPy to perform arithmetic operations using low-level, vectorized CPU instructions.

```
import numpy as np

Creating a 1D NumPy array
capacities_kw = np.array([100.0, 250.0, 500.0])

True vectorized elementwise scaling
capacities_watts = capacities_kw * 1000.0
print(capacities_watts)
Output: [100000. 250000. 500000.]
```

### 3.2 Array Dimensions and the .shape Attribute

Every NumPy array has a `.shape` tuple that defines its dimensions. It is vital to distinguish between a 1D vector, a 2D row matrix, and a 2D column matrix.

```
import numpy as np

1D Array (Vector of 3 elements)
a = np.array([1.0, 2.0, 3.0])
print(a.shape) # Output: (3,)
```

```
2D Array with 1 Row and 3 Columns (Row Matrix)
b = np.array([[1.0, 2.0, 3.0]])
print(b.shape) # Output: (1, 3)
```

```
2D Array with 3 Rows and 1 Column (Column Matrix)
c = np.array([[1.0], [2.0], [3.0]])
print(c.shape) # Output: (3, 1)
```

Structural Differences Table:

| Array Representation      | .shape | Number of Dimensions (.ndim) | Conceptual Meaning           |
|---------------------------|--------|------------------------------|------------------------------|
| np.array([1, 2, 3])       | (3,)   | 1                            | Flat 1D collection           |
| np.array([[1, 2, 3]])     | (1, 3) | 2                            | Matrix with 1 row, 3 columns |
| np.array([[1], [2], [3]]) | (3, 1) | 2                            | Matrix with 3 rows, 1 column |

These shapes are **not interchangeable** in linear algebra operations. A `(3,)` vector behaves differently than a `(3, 1)` matrix during matrix multiplication and broadcasting.

### 3.3 Matrix Slicing and Subscript Indexing

For a 2D array (matrix ***X***), indexing uses the syntax `X[row_slice, column_slice]`.

```
X = np.array([
 [15.0, 100.0, 3.0], # Row 0: Site MHP-001
 [30.0, 250.0, 4.0], # Row 1: Site MHP-002
 [5.0, 500.0, 1.0], # Row 2: Site MHP-003
])

print(X.shape) # Output: (3, 3) -> 3 rows (projects), 3 columns (features)
print(X[0, :]) # Output: array([15., 100., 3.]) -> Entire Row 0
print(X[:, 0]) # Output: array([15., 30., 5.]) -> Entire Column 0
print(X[0, 1]) # Output: 100.0 -> Element at Row 0, Column 1
print(X[:2, :2]) # Output: First 2 rows and first 2 columns (2x2 submatrix)
```

### 3.4 The Mechanics of Broadcasting

**Broadcasting** is NumPy's set of rules for performing elementwise operations on arrays of different shapes without copying data unnecessarily.

## The Official Broadcasting Rule:

NumPy compares the shapes of two arrays **from right to left** (trailing dimensions forward). Two dimensions are compatible if:

1. They are **equal**, or
2. One of them is **1**.

If a dimension is **1**, it is stretched along that dimension to match the size of the other array.

## Concrete Example: Mean Centering Matrix $X$

Suppose  $X$  has shape **(3, 2)** (3 project observations, 2 features) and **means** has shape **(2,)**.

```
X = np.array([
 [15.0, 100.0],
 [30.0, 250.0],
 [5.0, 500.0]
]) # Shape: (3, 2)

means = np.array([16.7, 283.3]) # Shape: (2,)

Subtract means from X
centred = X - means
```

## Step-by-Step Shape Alignment Check:

1. Shape of  $X$ : **(3, 2)**
2. Shape of **means**: **(2,)** → Right-aligned: **(1, 2)**
3. Compare trailing dimension: **2 == 2** (Match!)
4. Compare leading dimension: **3** vs **1**. The dimension of size **1** stretches from **1** → **3**.
5. Final broadcasted subtraction subtracts **16.7** from Column 0 and **283.3** from Column 1 for all rows.

## 3.5 The Dot Product ( $\cdot$ ) at Three Abstraction Levels

The **dot product** of two equal-length 1D vectors **a** and **b** of length **p** is defined mathematically as:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^p a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_p b_p$$

Here are three ways to compute the exact same result in Python:

```
features = np.array([15.0, 100.0]) # Cable length (km), Capacity (kW)
weights = np.array([0.05, 0.002]) # Cost weights per unit

Level 1: Manual Scalar Term-by-Term
manual = features[0] * weights[0] + features[1] * weights[1]

Level 2: Explicit Python Loop with zip
loop_total = 0.0
for f, w in zip(features, weights):
 loop_total += f * w

Level 3: Vectorized NumPy Operator (@)
dot_total = features @ weights # Equivalently: np.dot(features, weights)

print(manual, loop_total, dot_total)
Output: 0.95 0.95 0.95
```

---

### 3.6 Matrix Prediction ( $X\beta$ )

When we multiply a design matrix  $X$  of shape  $(N \times P)$  by a parameter vector  $\beta$  of shape  $(P,)$  using the `@` operator, NumPy performs a dot product between **each row of  $X$**  and the weight vector  $\beta$ .

```
X = np.array([
 [15.0, 100.0], # Project 0
 [30.0, 250.0], # Project 1
 [5.0, 500.0], # Project 2
]) # Shape: (3, 2)

beta = np.array([0.05, 0.002]) # Shape: (2,)

Matrix-vector multiplication
predictions = X @ beta
print(predictions)
Output: [0.95 2.0 1.25]
```

#### Matrix Dimension Rule:

$$(N \times P) \times (P \times 1) \longrightarrow (N \times 1)$$

The inner dimensions ( $P$ ) must match exactly, and the resulting prediction vector has length  $N$  (one predicted output per observation row).

---

---

## 4. Day 0C — Algebra, Summation Notation, and Loss Functions

### 4.1 Functions and Equations of Lines

A linear relationship between an input variable  $x$  and an output variable  $y$  is expressed as:

$$y = mx + c$$

- **$m$ : Slope** — The change in  $y$  given a **1**-unit increase in  $x$  ( $\frac{\Delta y}{\Delta x}$ ).
- **$c$ : Intercept** — The value of  $y$  when  $x = 0$ .

In multiple regression, we extend this equation to  $P$  features:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

---

### 4.2 Deconstructing Summation Notation ( $\sum$ )

The summation symbol  $\sum$  represents an accumulator loop.

$$\sum_{i=1}^n x_i$$

- **$i = 1$ : Lower limit** (starting index of the loop).
- **$n$ : Upper limit** (stopping index of the loop).
- **$x_i$ : Expression** evaluated at step  $i$  and added to the total.



## Translating Summation to Python Code

Consider the **Sum of Squared Residuals (SSR)** formula:

$$SSR = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where  $y_i$  is the actual target value, and  $\hat{y}_i$  is the predicted target value.

```
import numpy as np

y_actual = np.array([12.0, 30.0, 8.0])
y_predicted = np.array([11.2, 31.5, 9.0])

1. Formula expressed directly as a loop
ssr_loop = 0.0
for i in range(len(y_actual)):
 ssr_loop += (y_actual[i] - y_predicted[i]) ** 2

2. Formula expressed as a single vectorized NumPy line
ssr_numpy = np.sum((y_actual - y_predicted) ** 2)

print(ssr_loop, ssr_numpy) # Output: 2.89 2.89
```

## 4.3 Why We Square Errors in Loss Functions

When evaluating prediction errors  $e_i = y_i - \hat{y}_i$ :

- Cancellation Problem:** Unsquared errors can be positive or negative. A model with errors  $+10$  and  $-10$  would sum to zero error ( $\sum e_i = 0$ ), falsely appearing perfect.
- Squaring Eliminates Signs:**  $(+10)^2 = 100$  and  $(-10)^2 = 100$ , ensuring all deviations contribute positively to total loss.
- Non-Linear Penalty for Large Outliers:** Squaring penalizes large errors much more severely than small ones:
  - \* Error = 2  $\implies$  Penalty =  $2^2 = 4$
  - \* Error = 10  $\implies$  Penalty =  $10^2 = 100$  (a 5x increase in error results in a 25x increase in penalty).

## 4.4 Critical Distinction: Sum of Squares vs. Square of Sum

$$\sum_{i=1}^n x_i^2 \neq \left( \sum_{i=1}^n x_i \right)^2$$

Let  $x = [2.0, -3.0, 5.0]$ :

1. **Sum of Squares** ( $\sum x_i^2$ ):

$$\sum x_i^2 = (2.0)^2 + (-3.0)^2 + (5.0)^2 = 4.0 + 9.0 + 25.0 = 38.0$$

2. **Square of Sum** ( $(\sum x_i)^2$ ):

$$\left( \sum x_i \right)^2 = (2.0 + (-3.0) + 5.0)^2 = (4.0)^2 = 16.0$$

```
x = np.array([2.0, -3.0, 5.0])

sum_of_squares = np.sum(x ** 2) # Output: 38.0
square_of_sum = np.sum(x) ** 2 # Output: 16.0
```

## 5. Day 0D — First-Principles Statistics

### 5.1 Sample Mean ( $\bar{x}$ )

The mean represents the arithmetic average (or balancing point) of a distribution.

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

```
costs = np.array([12.0, 30.0, 8.0, 45.0, 15.0])
mean_manual = np.sum(costs) / len(costs) # Output: 22.0
```

### 5.2 Variance ( $\sigma^2$ ) and Standard Deviation ( $\sigma$ )

**Variance** measures the average squared distance of data points from their mean:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

#### Why can't we just average raw deviations ( $x_i - \bar{x}$ )?

The sum of raw deviations around the mean is **always zero** ( $\sum_{i=1}^n (x_i - \bar{x}) = 0$ ) because positive and negative deviations cancel out.

**Standard Deviation** ( $\sigma$ ) takes the square root of variance to return the spread metric to the original units of the dataset:

$$\sigma = \sqrt{\sigma^2}$$

```
First principles computation
deviations = costs - costs.mean()
variance_manual = np.sum(deviations ** 2) / len(costs)
std_manual = np.sqrt(variance_manual)

print(variance_manual, std_manual)
Output: 182.8 13.520355024924517
```

#### Population Variance vs. Sample Variance ( $n$ vs $n - 1$ ):

- **Population Variance (ddof=0)**: Divides by  $n$ . Used when analyzing an entire dataset.
- **Sample Variance (ddof=1)**: Divides by  $n - 1$  (Bessel's correction). Used when estimating population variance from a sample to eliminate downward bias.

---

## 5.3 Covariance ( $\text{cov}(x, y)$ )

Covariance measures whether two continuous variables tend to deviate from their respective means in the same direction at the same time.

$$\text{cov}(x, y) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

```
cable_km = np.array([12.0, 30.0, 5.0, 40.0, 15.0])

x_dev = costs - costs.mean()
y_dev = cable_km - cable_km.mean()

cov_manual = np.sum(x_dev * y_dev) / len(costs)
Output: 162.8
```

- **Positive Covariance:** When  $x$  is above its mean,  $y$  tends to be above its mean.
- **Negative Covariance:** When  $x$  is above its mean,  $y$  tends to be below its mean.
- **Zero Covariance:** No linear co-movement between  $x$  and  $y$ .

---

## 5.4 Pearson Correlation Coefficient ( $r$ )

Because covariance depends on the units of measurement, it cannot be used to compare strength across different features. **Pearson Correlation** ( $r$ ) normalizes covariance by dividing it by the product of the standard deviations of both variables:

$$r = \frac{\text{cov}(x, y)}{\sigma_x \sigma_y}$$

```
r_manual = cov_manual / (costs.std() * cable_km.std())
print(r_manual) # Output: 0.9413
```

### Properties of $r$ :

- $r \in [-1, +1]$ .
- $r = +1$ : Perfect positive linear relationship.
- $r = -1$ : Perfect negative linear relationship.
- $r = 0$ : No linear relationship.

---

## 5.5 The Zero Variance Trap and NaN Poisoning

If a feature vector has zero variation (e.g.,  $x = [5.0, 5.0, 5.0, 5.0]$ ), its standard deviation is zero ( $\sigma_x = 0.0$ ).

```
constant_feature = np.array([5.0, 5.0, 5.0, 5.0, 5.0])
r_result = np.corrcoef(constant_feature, costs)[0, 1]
print(r_result) # Output: nan
```

## Why does this happen?

Calculating  $\sigma_{\mathbf{x}}$  requires dividing by  $\sigma_{\mathbf{x}}$ . Division by zero produces NaN (Not a Number). In machine learning pipelines, a single NaN will silently propagate through matrix multiplications, corrupting downstream calculations.

---

## 6. Day 0E — Calculus, Gradients, Optimization, and Debugging

### 6.1 The Definition of a Derivative

A **derivative** measures the instantaneous rate of change (the slope of the tangent line) of a function  $f(x)$  at a specific point  $x$ .

$$\frac{dy}{dx} = f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

#### Code Proof: Finite Difference Approximation

We can approximate derivatives numerically by choosing an extremely small step size  $h$  (e.g.,  $h = 10^{-6}$ ).

```
def f(x):
 return x ** 2

def numerical_slope(f, x, h=1e-6):
 return (f(x + h) - f(x)) / h

x_val = 3.0
print(numerical_slope(f, x_val))
Output: 6.000000999922336 (Converges to analytical derivative 2*x = 6.0)
```

### 6.2 Essential Analytical Derivative Rules

#### 1. Power Rule:

$$\frac{d}{dx}[x^n] = nx^{n-1}$$

- Example:  $\frac{d}{dx}[x^2] = 2x$

#### 2. Constant Multiple Rule:

$$\frac{d}{dx}[c \cdot f(x)] = c \cdot \frac{d}{dx}[f(x)]$$

#### 3. Sum Rule:

$$\frac{d}{dx}[f(x) + g(x)] = \frac{d}{dx}[f(x)] + \frac{d}{dx}[g(x)]$$

#### Combined Example:

For  $y = 3x^2 + 5x$ :

$$\frac{dy}{dx} = \frac{d}{dx}[3x^2] + \frac{d}{dx}[5x] = 3(2x) + 5(1) = 6x + 5$$

---

## 6.3 Partial Derivatives ( $\partial$ ) and the Gradient ( $\nabla$ )

When an error function depends on multiple parameters—such as slope  $m$  and intercept  $c$ —a **partial derivative** measures how the output changes when we adjust *one parameter* while keeping all others constant. We use the symbol  $\partial$  (curly "d") instead of  $d$ .

$$\text{Gradient } \nabla E(m, c) = \begin{bmatrix} \frac{\partial E}{\partial m} \\ \frac{\partial E}{\partial c} \end{bmatrix}$$

```
def error(m, c):
 # Loss for predicting y=10.0 when x=2.0
 prediction = m * 2.0 + c
 return (10.0 - prediction) ** 2

def partial_wrt_m(m, c, h=1e-6):
 return (error(m + h, c) - error(m, c)) / h

def partial_wrt_c(m, c, h=1e-6):
 return (error(m, c + h) - error(m, c)) / h

m, c = 1.0, 1.0
print(partial_wrt_m(m, c)) # Output: -27.9999
print(partial_wrt_c(m, c)) # Output: -13.9999
```

---

## 6.4 Gradient Descent Mechanics

**Gradient Descent** is an iterative optimization algorithm that minimizes a loss function by taking steps in the direction opposite to the gradient.

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla E(\theta)$$

Where  $\alpha$  is the **learning rate** (step size hyperparameter).

```
m, c = 0.0, 0.0 # Initial bad guess
learning_rate = 0.01 # Step size alpha

for step in range(50):
 grad_m = partial_wrt_m(m, c)
 grad_c = partial_wrt_c(m, c)

 # Step opposite to the gradient
 m -= learning_rate * grad_m
 c -= learning_rate * grad_c

print(f"Optimized parameters: m = {m:.3f}, c = {c:.3f}")
Output: m and c settle into values where m*2 + c ≈ 10.0
```

### What happens if the learning rate $\alpha$ is too large?

If  $\alpha$  is set too high (e.g.,  $\alpha = 5.0$ ), parameter updates will overshoot the minimum, causing error to explode toward infinity.

---

---

## 7. Level 0 Capstone: Complete Step-by-Step Reference Solution

Here is the complete, execution-verified Python script solving all 7 steps of the Level 0 Capstone.

```
import numpy as np

Capstone Input Data
projects = np.array([
 [12.0, 15.0], # Row 0: [cable_km, terrain_index]
 [30.0, 25.0], # Row 1
 [5.0, 8.0], # Row 2
 [40.0, 45.0], # Row 3
 [15.0, 12.0], # Row 4
])
costs = np.array([12.0, 30.0, 8.0, 45.0, 15.0]) # Actual costs (million PKR)

STEP 1: Shape Inspection

print("STEP 1: Shape of projects =", projects.shape)
Explanation: Shape is (5, 2), meaning 5 project observations (rows)
and 2 features per project (columns: cable length and terrain index).

STEP 2: Custom Loop Means and Standard Deviations vs NumPy

num_rows, num_cols = projects.shape
custom_means = []
custom_stds = []

for j in range(num_cols):
 col_data = projects[:, j]
 col_mean = sum(col_data) / num_rows
 col_var = sum((x - col_mean) ** 2 for x in col_data) / num_rows
 custom_means.append(col_mean)
 custom_stds.append(np.sqrt(col_var))

print(f"STEP 2: Custom Means = {custom_means} | NumPy Means = {projects.mean(axis=0)}")
print(f" Custom Stds = {custom_stds} | NumPy Stds = {projects.std(axis=0)}")

STEP 3: Pearson Correlation from First Principles (costs vs projects[:, 0])

x = projects[:, 0]
mean_x = custom_means[0]
mean_y = sum(costs) / len(costs)

cov_xy = sum((x[i] - mean_x) * (costs[i] - mean_y) for i in range(len(costs))) / len(costs)
std_y = np.sqrt(sum((y - mean_y) ** 2 for y in costs) / len(costs))

r_custom = cov_xy / (custom_stds[0] * std_y)
r_numpy = np.corrcoef(costs, x)[0, 1]

print(f"STEP 3: Custom Correlation r = {r_custom:.4f} | NumPy r = {r_numpy:.4f}")

STEP 4: Matrix Predictions using Custom Loop and Vectorized @

beta = np.array([0.5, 0.2]) # Made-up weights

pred_loop = np.zeros(num_rows)
for i in range(num_rows):
 pred_loop[i] = projects[i, 0] * beta[0] + projects[i, 1] * beta[1]

pred_matrix = projects @ beta

print("STEP 4: Loop Predictions =", pred_loop)
print(" Matrix Predictions =", pred_matrix)
print(" Match check:", np.allclose(pred_loop, pred_matrix))
```

```

STEP 5: Sum of Squared Errors (SSE) Function

def sse(y_actual, y_predicted):
 return np.sum((y_actual - y_predicted) ** 2)

current_sse = sse(costs, pred_matrix)
print(f"STEP 5: Current SSE = {current_sse:.4f}")

STEP 6: Finite Difference Numerical Partial Derivative wrt beta[0]

h = 1e-5
beta_nudged = beta.copy()
beta_nudged[0] += h

nudged_predictions = projects @ beta_nudged
nudged_sse = sse(costs, nudged_predictions)

partial_beta_0 = (nudged_sse - current_sse) / h
print(f"STEP 6: Numerical d(SSE)/d(beta_0) = {partial_beta_0:.4f}")

STEP 7: Deliberate Shape Mismatch Error

print("STEP 7: Intentional Error Triggering:")
try:
 bad_beta = np.array([0.5, 0.2, 0.9]) # Length 3 instead of 2
 invalid_pred = projects @ bad_beta
except ValueError as e:
 print(" CAUGHT EXPECTED ERROR:", e)
 print(" DIAGNOSTIC: projects has shape (5, 2); bad_beta has shape (3,).")
 print(" Matrix multiplication requires the inner dimension of projects (2)")
 print(" to match the dimension of bad_beta (3).")
```

## 8. Master Rosetta Stone & Traceback Cheat Sheet

### 8.1 Mathematical Symbol → Python Loop → NumPy Expression

| Mathematical Concept  | LaTeX Symbol                                      | Raw Python Loop Expression                     | Vectorized NumPy Expression |
|-----------------------|---------------------------------------------------|------------------------------------------------|-----------------------------|
| Vector Dot Product    | $\sum_{i=1}^p a_i b_i$                            | sum(a[i] * b[i] for i in range(p))             | a @ b                       |
| Sample Mean           | $\bar{x} = \frac{1}{n} \sum x_i$                  | sum(x) / len(x)                                | x.mean()                    |
| Mean Centering        | $x_i - \bar{x}$                                   | [val - mean_x for val in x]                    | x - x.mean()                |
| Sum of Squared Errors | $\sum (y_i - \hat{y}_i)^2$                        | sum((y[i] - y_hat[i])**2 for i in range(n))    | np.sum((y - y_hat)**2)      |
| Population Variance   | $\frac{1}{n} \sum (x_i - \bar{x})^2$              | sum((x[i] - mean_x)**2 for i in range(n)) / n  | x.var()                     |
| Covariance            | $\frac{1}{n} \sum (x_i - \bar{x})(y_i - \bar{y})$ | sum((x[i]-mx)*(y[i]-my) for i in range(n)) / n | np.cov(x, y, ddof=0)[0,1]   |
| Derivative Limit      | $\lim_{h \rightarrow 0} \frac{f(x+h)-f(x)}{h}$    | (f(x + h) - f(x)) / h                          | Numerical finite difference |

## 8.2 Traceback Diagnostic Reference

### 1. Shape Mismatch Error (ValueError)

```
ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0,
with gufunc signature (n?,k),(k?,m?)->(n?,m?) (size 3 is different from 2)
```

- **Translation:** You attempted matrix multiplication (`@`) between two arrays whose inner dimensions do not match.
- **Fix:** Check `array1.shape` and `array2.shape`. Ensure that the column count of the first array equals the row count of the second array.

### 2. Non-existent Key Error (KeyError)

```
KeyError: 'cost_per_km'
```

- **Translation:** You requested a key from a dictionary that does not exist.
- **Fix:** Check for typos in string keys or print `dictionary.keys()` to verify available labels.

### 3. Data Type Mismatch Error (TypeError)

```
TypeError: can only concatenate str (not "float") to str
```

- **Translation:** Python refused to automatically combine text strings with numeric floating-point values.
- **Fix:** Explicitly convert numbers to strings using `str(val)` or use f-strings (`f"Cost: {val}"`).



# Chapter 0: Level 0 Orientation — Landing on Planet Regression

---

Hey there! Welcome. If you've ever looked at a messy spreadsheet and thought, *"I wonder if we can predict one column using the others without losing my mind,"* you're in the right place.

This guide is designed to take you from a complete beginner—someone who doesn't know their features from their targets—to someone who can confidently sit down with PhD researchers, discuss causal estimation, and not break a sweat. All in about a week of focused work.

But we have a strict rule here: **no passive reading**. You don't learn swimming by reading a manual on hydrodynamics; you learn it by jumping into the pool and swallowing a bit of water. In our case, the pool is full of data, and the water is Python code.

Let's get oriented.

---

## 1. How to Use This Guide

This is a self-paced, project-based curriculum. Instead of using toy datasets like Boston Housing (which has been beaten to death and has some pretty sketchy history anyway) or Iris flowers, we are going to work with three custom-built datasets based on Khyber Pakhtunkhwa (KP), Pakistan.

For every single chapter, you should follow this rhythm:

1. **Read the theory** (informal, conceptual, but mathematically honest).
2. **Explain it back** (tell a rubber duck or your cat what the concept means).
3. **Run the experiments** (write the code yourself—no copy-pasting allowed).
4. **Break it on purpose** (what happens if you scale the target? what if you pass ID columns? find out!).
5. **Solve the exercises** (thoughtful, structured problems designed to build muscle memory).

### Your Toolkit

Make sure you have a working Python installation. We'll be using:

- `pandas` & `numpy` (the bread and butter of data manipulation)
  - `matplotlib` & `seaborn` (for making plots that actually look good)
  - `scikit-learn` (for machine learning)
  - `statsmodels` (for statistical tests and classical econometrics)
  - Advanced tools (which we will introduce as we go: `XGBoost`, `Optuna`, `SHAP`, `MAPIE`, etc.)
- 

## 2. Setup and Data Generation

Suppose someone collected data across the beautiful valleys and plains of KP. Before we can do any math or fit any lines, we need to generate this data.

### Step 2.1: The Command Line Ritual

Open your terminal and run the following commands to set up a clean, isolated environment. Don't install packages globally; that's how you break other projects and end up crying over dependency conflicts at 2 AM.

```
Clone or navigate to your workspace
cd /var/www/documentation/regression-tasks

Create a virtual environment
python3 -m venv .venv

Activate the virtual environment
source .venv/bin/activate

Install the essentials
pip install --upgrade pip
pip install pandas numpy matplotlib seaborn scikit-learn statsmodels
```

## Step 2.2: Generate the Synthetic KP Datasets

The project directory contains a script called `generate_data.py`. This script simulates a mini-world with realistic statistical dependencies. Let's run it:

```
python generate_data.py
```

If successful, you will see output like this:

```
Generating development dataset: data/kp_subdistrict_development_index.csv with 100000 records...
Finished generating data/kp_subdistrict_development_index.csv.
Generating agricultural dataset: data/kp_agricultural_yields.csv with 100000 records...
Finished generating data/kp_agricultural_yields.csv.
Generating infrastructure dataset: data/kp_infrastructure_projects.csv with 100000 records...
Finished generating data/kp_infrastructure_projects.csv.
All datasets generated successfully in the 'data' directory!
```

### [!WARNING]

These datasets are **synthetic**. They are mathematically designed to teach you regression. They reflect realistic terrain constraints, crop scales, and budget overruns of Pakistan, but they are **not** real survey data. Do not present these as actual policy evidence to the KP Planning and Development Department unless you want to get laughed out of the room.

## 3. The Three KP Regression Datasets

Let's meet our three main characters. Each represents a distinct style of regression problem.

### 3.1 The Development Index Dataset

- **File:** `data/kp_subdistrict_development_index.csv`
- **Target (What we want to find):** `development_score` (a bounded index from 0 to 100).
- **Concept:** *Suppose someone collected survey data from 100,000 communities in KP. They have indicators like literacy rate, school enrollment, average household income, distance to health facilities, and public funding. Now, they want to predict the overall community development score. What do we do?*
- **The Reality Inside:** This dataset is highly linear with some terrain-based penalties (e.g., mountainous regions face structural challenges). However, it contains severe **multicollinearity** (literacy rate and school enrollment are almost copies of each other).

## 3.2 The Agricultural Yields Dataset

- **File:** `data/kp_agricultural_yields.csv`
- **Target:** `crop_yield_tons_per_acre` (a positive continuous value).
- **Concept:** Suppose a researcher collected farm-level data including crop types (wheat, maize, sugarcane, apples, peaches), soil pH, rainfall, elevation, fertilizer application, and whether the farm is organic. They want to predict the yield per acre to advise farmers. What do we do?
- **The Reality Inside:** This dataset is highly **nonlinear** and filled with **interactions**. Soil pH has an optimal point (curves downwards if pH is too acidic or basic). Apples and peaches die in high temperatures, while sugarcane thrives. Organic farms have strictly zero fertilizer and pesticides. A simple straight line will fail miserably here.

## 3.3 The Infrastructure Projects Dataset

- **File:** `data/kp_infrastructure_projects.csv`
- **Targets:** `actual_cost_million_pkr` and `actual_duration_months` (two continuous values!).
- **Concept:** Suppose the government of KP tracked 100,000 public works projects (roads, schools, water supply). They have the approved budget, estimated duration, contractor experience, and year of initiation. They want to predict the actual final cost and duration before building starts to prevent budget overruns. What do we do?
- **The Reality Inside:** The targets are heavily **right-skewed** (most projects are small, but a few mega-projects cost billions). It is a **multi-output** problem where cost and duration are intimately related, but actual duration is unknown when planning.

---

## Chapter 0 Exercises: A Dress Rehearsal, Not a Preview

Here are your first hands-on challenges. Write python scripts (save them under `scratch/` or run them in an interactive environment) to solve them.

**What this rehearsal is for.** These three datasets are large, realistic, and deliberately disconnected from the running case the rest of the book uses. That's on purpose. Chapter 1 onward follows a single narrower thread — the MHP Cost Estimator — built from small, hand-inspectable datasets that grow chapter by chapter, not from these CSVs. You will not see `kp_subdistrict_development_index.csv`, `kp_agricultural_yields.csv`, or `kp_infrastructure_projects.csv` again after this chapter, and that's fine. The point here is to build the reflex — load it, check its shape, find the ID columns, read the generating code — on data with real scale and real messiness, *before* the stakes go up and that reflex has to be automatic. Think of it as a practice scrimmage before the season starts, not the season itself.

### Exercise 0.1: The Data Handshake

Write a Python script `verify_setup.py` to load all three datasets and print their shapes, columns, and target variable ranges (minimum and maximum values).

#### Helpful Comments:

- \* Use `pd.read_csv()` to load.
- \* Use `.shape`, `.columns`, and `.describe()` to extract statistics.
- \* *Comic Relief:* If your script throws a `FileNotFoundError`, verify you ran `generate_data.py` first. Data does not materialize out of thin air, no matter how hard you stare at the screen.

## Exercise 0.2: Spotting the Imposters (Identifiers)

Look closely at the columns of all three datasets. Identify the columns that represent unique IDs (identifiers).

1. Why must we drop these columns before feeding them to a regression model?
2. What happens if a model "memorizes" `community_id` to predict `development_score`?

### Helpful Comments:

- \* Unique identifiers have high cardinality (almost every row has a unique value).
- \* If you train a model with ID columns, it might find a spurious pattern (e.g., ID `COM-00042` has a high score just by luck) and fail when predicting on a new community `COM-99999`. This is the ultimate form of cheating and overfitting.

## Exercise 0.3: Tracing the Data-Generating Code

Open `generate_data.py` and search for the function `generate_development_dataset`. Look at how `raw_score` is calculated.

Write down the exact formula used to build `raw_score`.

What weights are given to `literacy_rate` vs `public_funding_allocated_million_pkr`?

---

## Handing Off to Chapter 1

One last thing before you turn the page. Everything in this chapter — the three CSVs, `generate_data.py`, the exercises above — belongs to this chapter alone. Chapter 1 opens a new, separate running case: the **MHP Cost Estimator**, predicting the cost of microhydro power projects from a handful of hand-built rows (eight, to start). It does not load anything you generated here, and it doesn't need to. Different dataset, same rules: read the theory, run the code yourself, break it on purpose. If Chapter 1 feels like a fresh start rather than a continuation, that's expected — you're not missing a step.

---

## References

1. **Python Virtual Environments:** Python venv Docs - Read this if you still don't know why we use virtual environments.
2. **pandas Basics:** pandas Getting Started Guide - Essential reading for data loading.
3. **Pakistan Development Context:** Planning & Development Department, Khyber Pakhtunkhwa KP P&D Website - For background on how development funding and infrastructure projects are managed in KP.

# Chapter 1 — From a Question to Ordinary Least Squares

---

## Level 1 Data Explorer: Week 1 of the MHP Cost Estimator

**Central promise.** By the end of this chapter, you will not merely be able to call a regression function. You will be able to state the question a regression can answer, represent the data correctly, derive ordinary least squares (OLS) in scalar and matrix form, implement it without a machine-learning library, diagnose when it fails, and explain the result to a decision-maker responsible for public infrastructure.

This chapter uses a running case: estimating the cost of microhydro power (MHP) projects in Khyber Pakhtunkhwa (KP). The projects are fictional but realistic. Their geography, procurement problems, transport constraints, and management decisions are grounded in the kinds of conditions infrastructure teams encounter in Chitral, Dir, Swat, and other mountainous districts.

**A note if you just finished Chapter 0.** This is a new dataset, not a continuation of the development, agricultural, or infrastructure CSVs you generated there. Those built the reflex; this is where it gets used. The MHP case starts small — eight projects below — and grows with each chapter.

The case is deliberately high-stakes. A regression coefficient is not just a number on a screen. If misunderstood, it can lead to an unrealistic budget, an unfair judgement about a field team, or a decision to withdraw a service from a remote community.

---

## The pedagogical contract

This book asks the learner and the author to keep the same contract.

1. **One central concept per day.** Each lesson has a single intellectual centre. Supporting ideas are introduced only when that centre requires them.
2. **No mathematical hand-waving.** A displayed equation is unpacked symbol by symbol and connected to numbers.
3. **Code as proof.** Important equations are verified with NumPy and, where useful, with plain loops. We do not use `scikit-learn` in Week 1.
4. **Geometry before memorisation.** Vectors, projections, residuals, and error surfaces are drawn or plotted.
5. **Build, break, rebuild.** Every day adds to the `MHPCostEstimator`, then deliberately creates a failure that the learner must diagnose.
6. **Interpretation in context.** Every mathematical result is translated back into a budget, engineering, or management statement.
7. **Cognitive load is managed, not intelligence underestimated.** New notation is introduced gradually, but no essential reasoning is hidden.

## What “from scratch” means here

We will use Python, NumPy, and Matplotlib as basic tools. We will not ask a library to fit a regression model for us. NumPy stores arrays and performs arithmetic; the learner still constructs the design matrix, objective function, normal equations, parameter estimates, predictions, residuals, and diagnostics.

Later chapters can remove even more scaffolding—for example, by implementing matrix decomposition manually—but doing that now would bury the statistical idea under numerical bookkeeping.

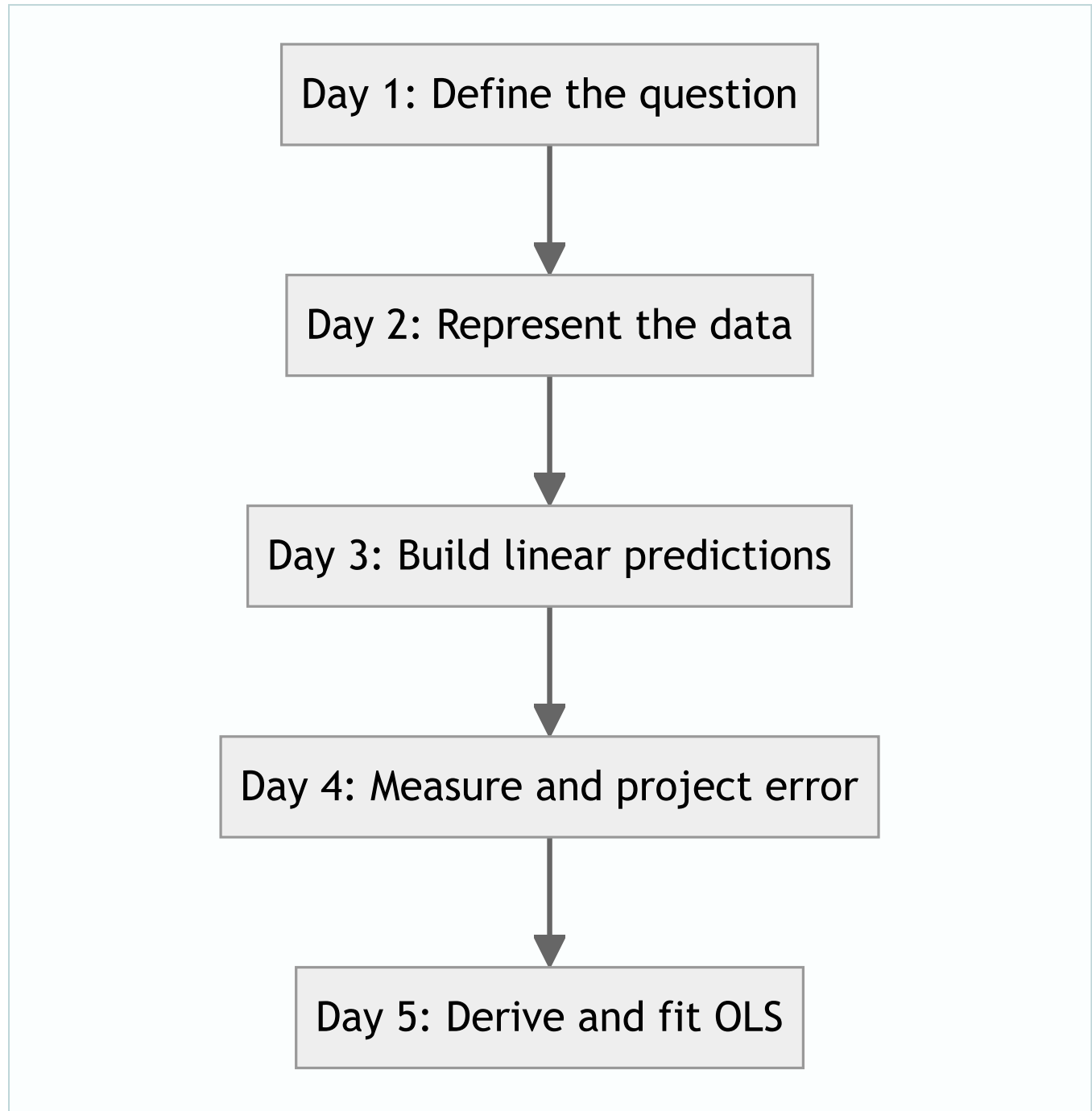
---

## Week 1 learning outcomes

At the end of five days, you should be able to:

- distinguish prediction, explanation, and causal inference;
- identify a target, features, observations, parameters, and hyperparameters;
- read and verify the shapes of vectors and matrices;
- explain why a column of ones creates an intercept;
- calculate residuals, SSR, MSE, RMSE, and MAE by hand and in code;
- describe OLS as both error minimisation and orthogonal projection;
- derive the scalar and matrix OLS solutions;
- explain when the OLS solution is unique;
- detect perfect multicollinearity and other common failures;
- fit and evaluate a small regression model without a machine-learning library; and
- separate a mathematically correct calculation from a defensible policy conclusion.

## The five-day route



Each day follows the same rhythm:

1. **Orient:** connect the concept to a decision;
  2. **Construct:** develop the mathematics carefully;
  3. **Prove:** test the mathematics with raw code;
  4. **See:** create a diagram or figure;
  5. **Build:** add one working part to the application;
  6. **Break:** produce a predictable failure;
  7. **Reflect:** decide what the result does and does not justify.
-

## Running case and units

The MHP Cost Estimator will eventually use multiple project features. During the first four days, we also use very small datasets so that every number can be inspected.

| Field               | Symbol | Unit        | Meaning                                            |
|---------------------|--------|-------------|----------------------------------------------------|
| Project count       | $n$    | projects    | Number of observations                             |
| Feature count       | $p$    | features    | Number of explanatory columns before the intercept |
| Cable length        | $x_1$  | km          | Length of distribution cable                       |
| Hydraulic head      | $x_2$  | m           | Vertical fall available to drive the turbine       |
| Road distance       | $x_3$  | km          | Distance from an all-weather road to the site      |
| Terrain difficulty  | $x_4$  | index 1–5   | Ordered field assessment of access difficulty      |
| Planned capacity    | $x_5$  | kW          | Intended generating capacity                       |
| Actual project cost | $y$    | million PKR | Target to be estimated                             |

**Unit discipline:** A coefficient is meaningless without its units. If cost is recorded in million PKR and cable length in kilometres, the cable coefficient has units of “million PKR per kilometre,” holding the other included features constant.

The data in this chapter are **synthetic**. They are for learning, not for judging a real district, contractor, community, or organisation.

### Minimal software setup

The code assumes Python 3.11 or later. In a terminal, create an isolated environment and install the two numerical packages used in this chapter:

```
python -m venv .venv
source .venv/bin/activate # Windows PowerShell: .venv\Scripts\Activate.ps1
python -m pip install numpy matplotlib
```

Use a plain `.py` file, a Jupyter notebook, or an editor with an interactive Python terminal. Keep every day's experiments. A failed calculation with a short note about why it failed is part of the learning record, not clutter to erase.

## Day 1 — What Question Are We Asking?

**Today's central idea:** Prediction, explanation, and causation are different jobs. A regression calculation does not decide which job you meant.



## 1.1 Regression in one sentence

Regression constructs a numerical relationship between a target variable  $y$  and one or more observed features  $X$ .

That sentence describes the mechanism but not the purpose. The same regression equation can be used for three very different purposes.

| Job              | Question                                                                            | What success looks like                             | Main danger                                                      |
|------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------|
| Prediction       | "How accurately can we estimate the cost of a new project?"                         | Low error on genuinely new projects                 | A model may fail when conditions change                          |
| Explanation      | "How is road distance associated with cost after accounting for included features?" | Stable, interpretable coefficients with uncertainty | Omitted variables and correlated features distort interpretation |
| Causal inference | "How much would cost change if we improved road access?"                            | Credible estimate of an intervention                | Association is mistaken for the effect of an action              |

The distinction is not philosophical decoration. It changes the data you need, the assumptions you must defend, and the way the result may be used.

## 1.2 Prediction: estimating an unknown outcome

For prediction, we want a function  $f$  that converts the observed features of project  $i$  into an estimate:

$$\hat{y}_i = f(x_{i1}, x_{i2}, \dots, x_{ip}).$$

Read the symbols slowly:

- $i$  identifies one project;
- $x_{ij}$  is feature  $j$  for project  $i$ ;
- $f$  is the fitted prediction rule;
- $y_i$  is the actual cost; and
- $\hat{y}_i$ , pronounced "y-hat," is the estimated cost.

A predictor may be useful even when a feature is not causal. For instance, the number of difficult river crossings may predict transport cost because it summarises remoteness. A planning unit can use that signal to improve a contingency estimate without claiming that a river crossing, by itself, causes every extra rupee.

Prediction must be evaluated on data not used to fit the model. A rule that memorises completed projects can appear perfect and still fail on the next valley.

## 1.3 Explanation: describing conditional association

For explanation, attention shifts from the prediction alone to the model's parameters. Consider:

$$\hat{y}_i = \beta_0 + \beta_1 \text{road\_distance}_i + \beta_2 \text{capacity}_i.$$

Here  $\beta_1$  describes the estimated difference in cost associated with one additional kilometre of road distance **among projects with the same included capacity**.

The phrase in bold is essential. In multiple regression, a coefficient is conditional on the other included features. It does not automatically control for variables that are absent, poorly measured, or structurally different across districts.

An explanatory analysis also needs uncertainty. A coefficient of 0.8 million PKR per kilometre is not equally persuasive if it comes from 6 projects or 600 projects. Standard errors and confidence intervals arrive later; Week 1 first establishes what is being estimated.

## 1.4 Causation: estimating an intervention

A causal question asks about a change we could make:

If an all-weather access road were constructed before the MHP works began, how would the final project cost differ from what it would otherwise have been?

This question compares two potential outcomes for the same project:

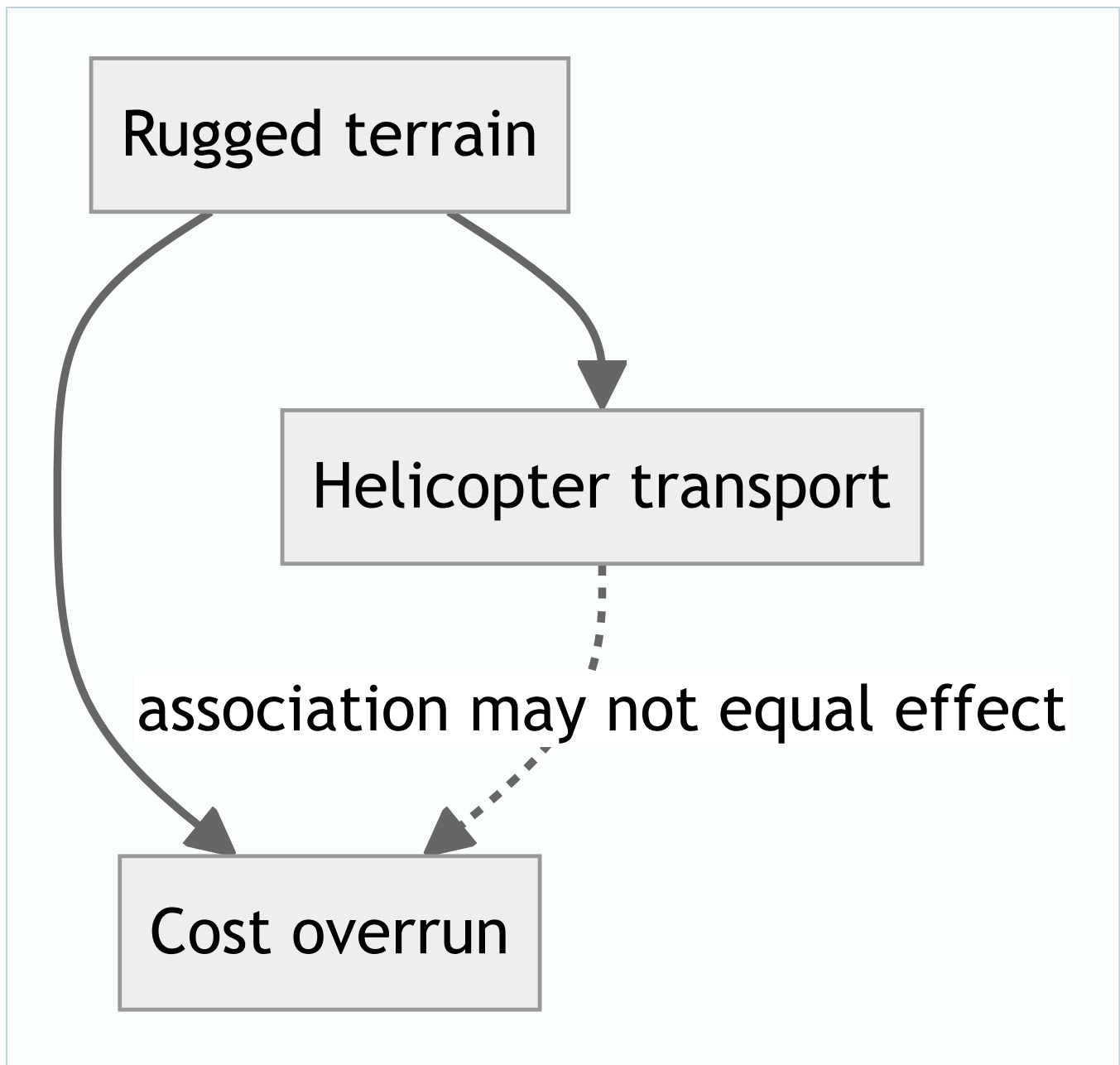
$$\text{causal effect}_i = Y_i(\text{road improved}) - Y_i(\text{road not improved}).$$

We can observe only one of those outcomes for a given project. The unobserved alternative is the **counterfactual**. Causal inference is therefore a problem of constructing a credible comparison, not merely fitting a line.

Ordinary regression can be part of a causal design, but the regression output is causal only when the research design and assumptions make it so. Random assignment, natural experiments, careful matching, panel designs, and defensible causal graphs can help. A high  $R^2$  cannot turn an associational design into a causal one.

## 1.5 Confounding: the hidden common cause

Suppose projects in rugged terrain are more likely to require helicopter transport, and rugged terrain independently increases excavation delays and labour costs.



**Figure 1.1 — A confounding structure.** Terrain is a common cause of the selected transport method and the cost overrun. The dotted arrow is the relationship we might estimate naively.

If terrain is ignored, projects using helicopters will look more expensive even if helicopter transport reduced cost relative to the feasible alternative—weeks of manual carriage or an unfinished project.

A variable is a confounder when it is causally related to both the exposure or decision and the outcome, and is not itself a consequence of that exposure. The precise definition depends on the full causal structure; “control for everything available” is not a safe rule because controlling for mediators or colliders can introduce new bias.

## 1.6 Simpson’s paradox: when aggregation reverses a pattern

Imagine eight projects, four in relatively accessible terrain and four in remote mountain terrain. Within each terrain group, longer cable raises cost. Yet remote projects happen to use shorter cable and start from a much higher cost base.

| Terrain group | Cable length (km) | Cost (million PKR) |
|---------------|-------------------|--------------------|
| Accessible    | 4                 | 18                 |
| Accessible    | 5                 | 20                 |
| Accessible    | 6                 | 22                 |
| Accessible    | 7                 | 24                 |
| Remote        | 1                 | 30                 |
| Remote        | 2                 | 33                 |
| Remote        | 3                 | 36                 |
| Remote        | 4                 | 39                 |

Within accessible projects, one more kilometre is associated with about 2 million PKR more cost. Within remote projects, it is associated with about 3 million PKR more cost. But in the combined data, low cable lengths are concentrated among costly remote projects, so the overall line slopes downward.

For one feature, the OLS slope can be written as:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}.$$

The numerator measures how  $x$  and  $y$  move together. The denominator measures how much  $x$  varies. Day 5 derives this formula; today we use it as a diagnostic.

## Code proof: calculate all three slopes

```
import numpy as np

def slope_from_sums(x, y):
 """One-feature OLS slope, written directly from the equation."""
 x = np.asarray(x, dtype=float)
 y = np.asarray(y, dtype=float)
 x_centered = x - x.mean()
 y_centered = y - y.mean()
 numerator = np.sum(x_centered * y_centered)
 denominator = np.sum(x_centered ** 2)
 return numerator / denominator

x_accessible = np.array([4, 5, 6, 7], dtype=float)
y_accessible = np.array([18, 20, 22, 24], dtype=float)

x_remote = np.array([1, 2, 3, 4], dtype=float)
y_remote = np.array([30, 33, 36, 39], dtype=float)

x_all = np.concatenate([x_accessible, x_remote])
y_all = np.concatenate([y_accessible, y_remote])

print("Accessible slope:", slope_from_sums(x_accessible, y_accessible))
print("Remote slope: ", slope_from_sums(x_remote, y_remote))
print("Combined slope: ", slope_from_sums(x_all, y_all))

Expected, approximately:
Accessible slope: 2.0
Remote slope: 3.0
Combined slope: -2.0
```

The code does not “prove” which relationship is causal. It proves the numerical reversal and forces us to explain it using geography and the data-generating process.

## Figure lab: make the reversal visible

```
import matplotlib.pyplot as plt

def line_for(x, y, grid):
 b1 = slope_from_sums(x, y)
 b0 = y.mean() - b1 * x.mean()
 return b0 + b1 * grid

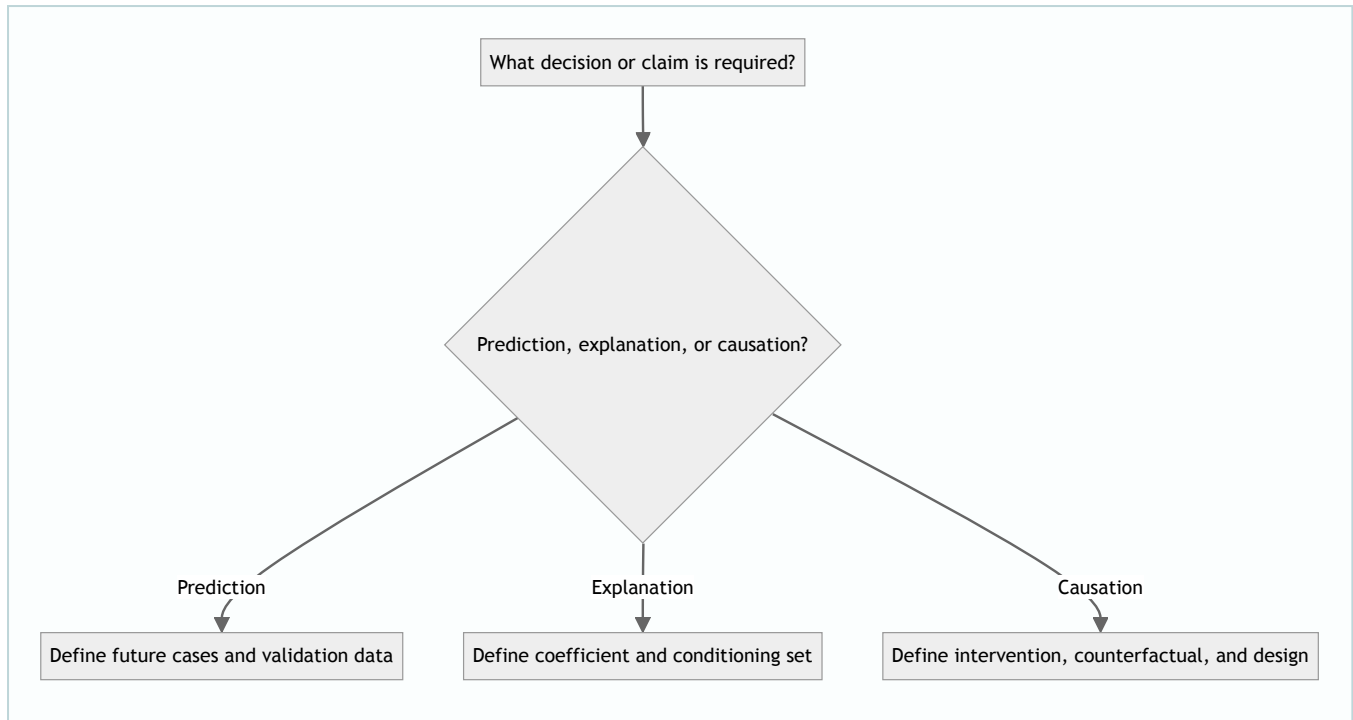
grid = np.linspace(0.5, 7.5, 100)

plt.figure(figsize=(8, 5))
plt.scatter(x_accessible, y_accessible, label="Accessible terrain", s=70)
plt.scatter(x_remote, y_remote, label="Remote terrain", s=70)
plt.plot(grid, line_for(x_accessible, y_accessible, grid), linewidth=2)
plt.plot(grid, line_for(x_remote, y_remote, grid), linewidth=2)
plt.plot(
 grid,
 line_for(x_all, y_all, grid),
 color="black",
 linestyle="--",
 label="Misleading combined line",
)
plt.xlabel("Cable length (km)")
plt.ylabel("Actual cost (million PKR)")
plt.title("Simpson's paradox in synthetic MHP projects")
plt.legend()
plt.tight_layout()
plt.show()
```

**Figure 1.2 — Group-specific and aggregated regression lines.** The dashed combined line answers a different—and substantively misleading—question.

## 1.7 A decision protocol before touching code

Use this sequence whenever someone asks for “a regression.”



Before analysis, write a one-sentence **estimand**—the exact quantity to be estimated. Examples:

- Prediction: “The expected final cost, in million PKR, of an approved project at the design stage.”
- Explanation: “The adjusted difference in final cost associated with one additional kilometre from an all-weather road, conditional on planned capacity and measured terrain difficulty.”
- Causation: “The average change in final MHP cost caused by completing an access road before civil works, among currently approved remote projects.”

If the sentence is vague, the model will not rescue it.

## 1.8 Build: start the application with a question contract

Create `mhp_estimator.py`:

```
from dataclasses import dataclass
from typing import Literal

import numpy as np

AnalysisPurpose = Literal["prediction", "explanation", "causation"]

@dataclass(frozen=True)
class AnalysisContract:
 purpose: AnalysisPurpose
 target_name: str
 target_unit: str
 unit_of_observation: str
 intended_use: str

 def __post_init__(self):
 if self.purpose not in {"prediction", "explanation", "causation"}:
 raise ValueError("purpose must be prediction, explanation, or causation")
 for name, value in vars(self).items():
 if not str(value).strip():
 raise ValueError(f"{name} cannot be empty")
```

```

class MHPCostEstimator:
 """OLS estimator built progressively during Week 1."""

 def __init__(self, contract: AnalysisContract):
 self.contract = contract
 self.parameters_ = None
 self.feature_names_ = None
 self.rank_ = None
 self.is_fitted_ = False

 def __repr__(self):
 status = "fitted" if self.is_fitted_ else "unfitted"
 return f"MHPCostEstimator(purpose={self.contract.purpose!r}, {status})"

if __name__ == "__main__":
 contract = AnalysisContract(
 purpose="prediction",
 target_name="actual_project_cost",
 target_unit="million PKR",
 unit_of_observation="one completed MHP project",
 intended_use="early budget review for approved projects",
)
 print(MHPCostEstimator(contract))

```

`@dataclass` asks Python to generate routine methods such as `__init__` and `__repr__` from the listed fields. `frozen=True` prevents those fields from being reassigned after creation, which protects the analysis contract from accidental mid-analysis changes. `Literal[...]` is a type hint saying that readers and type-checking tools should expect exactly one of the three listed strings; the runtime check in `__post_init__` still performs the actual enforcement.

The class begins with a contract because a technically correct model can still be unfit for its intended use.

## 1.9 Break it deliberately

Change `purpose="prediction"` to `purpose="forecasting"`. The program should reject the unrecognised purpose. Then leave the purpose valid but make `intended_use=""`. It should reject the empty field.

The lesson is small but important: silent ambiguity is a data-quality problem.

## 1.10 Day 1 practice

1. A finance unit wants next year's total contingency allocation. Is its first task prediction, explanation, or causation?
2. A director asks whether contractor type "causes" delay because its regression coefficient is positive. What additional question must you ask?
3. Draw a DAG in which flood damage affects both the decision to redesign a headrace channel and final cost.
4. In the Simpson dataset, add 10 million PKR to every accessible project. Recalculate the three slopes. Which slopes change, and why?
5. Write an estimand for a real infrastructure decision you know.

## Day 1 exit check

You are ready to continue if you can complete this sentence without hesitation:

"A regression coefficient is a causal effect only when \_\_\_\_."

A defensible completion is: “the design and assumptions make the comparison equivalent to the relevant intervention and counterfactual.”

# Day 2 — Regression Vocabulary and the Shape of Data

**Today’s central idea:** Data have roles, shapes, units, and provenance. Most early modeling errors are representation errors before they are statistical errors.

## 2.1 The vocabulary of one project table

Suppose the rows of a table are completed MHP projects.

| Term           | Meaning                                          | MHP example                        |
|----------------|--------------------------------------------------|------------------------------------|
| Observation    | One unit represented by one row                  | One completed MHP project          |
| Feature        | An input measured before the target is known     | Planned capacity in kW             |
| Target         | The outcome to estimate                          | Actual project cost                |
| Parameter      | A value learned from data                        | Cost coefficient for road distance |
| Hyperparameter | A setting chosen outside the fitting calculation | Whether to include an intercept    |
| Prediction     | Model output for an observation                  | Estimated cost of 42 million PKR   |
| Residual       | Actual minus predicted target                    | $45 - 42 = 3$ million PKR          |
| Loss           | A rule for penalising errors during fitting      | Squared residual                   |
| Metric         | A summary used to evaluate performance           | RMSE in million PKR                |
| Estimator      | The procedure that learns parameters             | OLS                                |
| Fitted model   | Estimator plus learned parameter values          | Intercept and slopes after fitting |

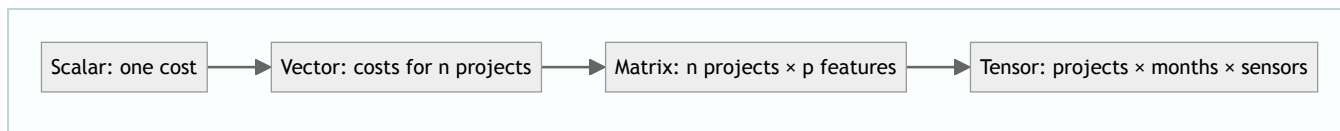
**Parameter** and **hyperparameter** should not be explained through a loose physical analogy. The reliable distinction is procedural: the fitting algorithm learns parameters from the training data; the analyst sets hyperparameters or design choices.

## 2.2 Scalars, vectors, matrices, and tensors

A **scalar** is one number. A **vector** is an ordered one-dimensional array. A **matrix** is a two-dimensional rectangular array. A **tensor** is the general term for an array with any number of axes.

The number of tensor axes is sometimes called its order or tensor rank. Do not confuse that usage with **matrix rank**, introduced on Day 5, which counts independent directions in a matrix.





**Figure 2.1 — The dimensional ladder.** Regression in this chapter mainly uses vectors and matrices; “tensor” is useful as the general family name.

The mathematical objects are:

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} \in \mathbb{R}^n, \quad \boldsymbol{\beta} \in \mathbb{R}^p.$$

Read them as:

- $\mathbf{X}$  is a real-valued matrix with  $n$  rows and  $p$  feature columns;
- $\mathbf{y}$  is a real-valued target vector with  $n$  entries; and
- $\boldsymbol{\beta}$  is a parameter vector with one entry per feature.

Once an intercept column is added,  $\mathbf{X}$  has  $p + 1$  columns and  $\boldsymbol{\beta}$  has  $p + 1$  entries.

## A concrete shape example

Three projects with two features—road distance and capacity—can be represented as:

$$\mathbf{X} = \begin{bmatrix} 12 & 100 \\ 25 & 150 \\ 8 & 80 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 28 \\ 44 \\ 22 \end{bmatrix}.$$

Then  $\mathbf{X}$  has shape  $3 \times 2$  and  $\mathbf{y}$  has length 3. Row 2 means that the second project is 25 km from an all-weather road, has planned capacity 150 kW, and actually cost 44 million PKR.

## 2.3 Shape is part of meaning

These arrays contain the same three numbers but do not have the same shape:

```
import numpy as np

a = np.array([28.0, 44.0, 22.0]) # shape (3,)
b = np.array([[28.0], [44.0], [22.0]]) # shape (3, 1)
c = np.array([28.0, 44.0, 22.0]) # shape (1, 3)

print(a.shape, b.shape, c.shape)
(3,) (3, 1) (1, 3)
```

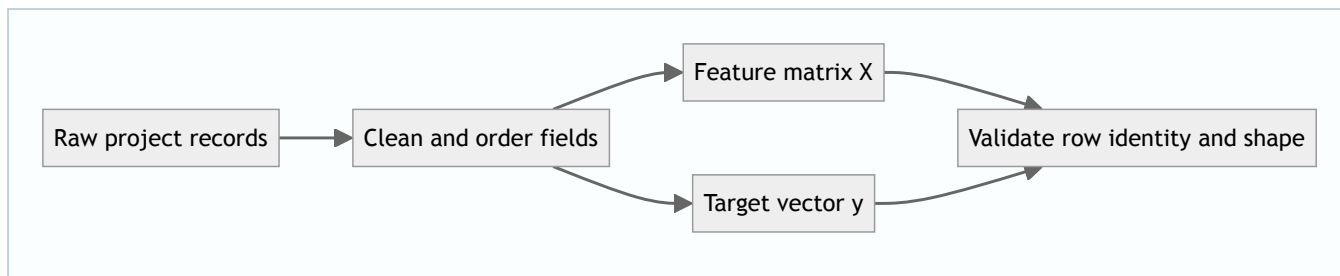
In NumPy, `(3,)` is a one-dimensional vector. `(3, 1)` is a two-dimensional column matrix. `(1, 3)` is a two-dimensional row matrix. They can behave differently under multiplication and broadcasting.

Our application will accept a one-dimensional target vector `y`. `y.shape == (n,)`. It will convert a single feature from `(n,)` to a two-dimensional matrix `(n, 1)`.

## 2.4 The design matrix and row alignment

The feature matrix used in a regression is called the **design matrix**. “Design” here refers to the arrangement of predictor columns; it does not mean the projects were experimentally assigned.

The  $i$ th row of  $\mathbf{X}$  and the  $i$ th entry of  $\mathbf{y}$  must refer to the same project.



**Figure 2.2 — Data assembly.** Matching row counts are necessary but not sufficient: row identities must also match.

If `X` is accidentally sorted by district while `y` remains sorted by project ID, the shapes still match. The code may run and the model may be nonsense. Preserve a project identifier during preparation, even if it is not used as a numerical feature.

## 2.5 A data dictionary is part of the model

A usable dataset needs more than column names.

| Field               | Data type | Unit / categories               | Timing           | Missing-value rule                  |
|---------------------|-----------|---------------------------------|------------------|-------------------------------------|
| project_id          | string    | unique ID                       | approval         | never missing                       |
| road_distance_km    | float     | km                              | design survey    | verify from GIS if missing          |
| planned_capacity_kw | float     | kW                              | approved design  | never impute casually               |
| terrain_index       | integer   | 1, 2, 3, 4, 5                   | pre-construction | document assessor                   |
| actual_cost_m_pkr   | float     | million PKR, stated price basis | completion       | target; exclude incomplete projects |

Timing prevents **target leakage**. A feature recorded after cost escalation—for example, “number of contract amendments”—might predict final cost extremely well. It is unavailable when the original budget is prepared and may partly be a consequence of the overrun. It should not be used in an early-stage prediction model without redefining the intended use.

## 2.6 Parsing one nested project record

```

import json
import numpy as np

raw_json = """
{
 "project_id": "MHP-CH-003",
 "design": {
 "road_distance_km": 18.5,
 "planned_capacity_kw": 120.0,
 "terrain_index": 4
 },
 "completion": {
 "actual_cost_m_pkr": 39.2
 }
}
"""

record = json.loads(raw_json)

X_one = np.array([[# two brackets: one row, three columns
 record["design"]["road_distance_km"],
 record["design"]["planned_capacity_kw"],

```

```

 record["design"]["terrain_index"],
], dtype=float)

y_one = np.array([
 record["completion"]["actual_cost_m_pkr"]
], dtype=float)

print("X:", X_one, "shape:", X_one.shape)
print("y:", y_one, "shape:", y_one.shape)

assert X_one.shape == (1, 3)
assert y_one.shape == (1,)

```

The `assert` statements are executable claims. If the parser stops satisfying the intended mathematical shape, the program fails at the boundary instead of producing a later mystery.

## 2.7 Code proof: matrix–vector multiplication by loops

The linear prediction without an intercept is:

$$\hat{y}_i = \sum_{j=1}^p x_{ij} \beta_j.$$

For every project  $i$ , multiply each feature  $x_{ij}$  by its corresponding parameter  $\beta_j$ , then add the products.

```

X = np.array([
 [12.0, 100.0],
 [25.0, 150.0],
 [8.0, 80.0],
])
beta = np.array([0.6, 0.20])

Proof using explicit loops
y_hat_loop = np.zeros(X.shape[0])
for i in range(X.shape[0]):
 total = 0.0
 for j in range(X.shape[1]):
 total += X[i, j] * beta[j]
 y_hat_loop[i] = total

The same operation in matrix notation
y_hat_matrix = X @ beta

print(y_hat_loop)
print(y_hat_matrix)
assert np.allclose(y_hat_loop, y_hat_matrix)

```

The `@` operator is compact, but it is not magic. The loops show exactly what it does.

## 2.8 Build: robust input validation

Add these methods inside `MHPCostEstimator`:

```

@staticmethod
def _validate_inputs(X, y=None):
 """Return finite float arrays with predictable regression shapes."""
 X = np.asarray(X, dtype=float)

 if X.ndim == 1:
 X = X.reshape(-1, 1)

```

```

if X.ndim != 2:
 raise ValueError(f"X must be 2D after conversion; got shape {X.shape}")
if X.shape[0] == 0 or X.shape[1] == 0:
 raise ValueError("X must contain at least one row and one feature")
if not np.isfinite(X).all():
 raise ValueError("X contains NaN or infinite values")

if y is None:
 return X

y = np.asarray(y, dtype=float)
if y.ndim == 2 and y.shape[1] == 1:
 y = y.reshape(-1)
if y.ndim != 1:
 raise ValueError(f"y must be 1D; got shape {y.shape}")
if X.shape[0] != y.shape[0]:
 raise ValueError(
 f"row mismatch: X has {X.shape[0]} rows but y has {y.shape[0]}"
)
if not np.isfinite(y).all():
 raise ValueError("y contains NaN or infinite values")

return X, y

```

## Why each check exists

- `dtype=float` prevents accidental string arithmetic.
- A 1D `X` is interpreted as one feature observed for many projects.
- Regression requires a 2D design matrix even when  $p = 1$ .
- Empty arrays have mathematically valid-looking shapes but no estimable relationship.
- `NaN` and infinity propagate through calculations and can silently destroy results.
- A one-column target matrix is flattened to the application's chosen convention.
- Matching row counts are enforced, though project-ID alignment must be checked earlier.

## 2.9 Break it deliberately

Run each case and predict the error message before reading it.

```

contract = AnalysisContract(
 purpose="prediction",
 target_name="actual_cost_m_pkr",
 target_unit="million PKR",
 unit_of_observation="completed MHP project",
 intended_use="design-stage budget review",
)
model = MHPCostEstimator(contract)

bad_cases = [
 (np.ones((10, 3)), np.ones(8)), # row mismatch
 (np.array([[1.0, np.nan]]), [2.0]), # missing feature
 (np.empty((0, 2)), np.empty(0)), # no observations
 (np.ones((2, 2, 2)), np.ones(2)), # three-dimensional X
]

for X_bad, y_bad in bad_cases:
 try:
 model._validate_inputs(X_bad, y_bad)
 except ValueError as error:
 print(type(error).__name__, "->", error)

```

Then construct the more dangerous failure: create `X` and `y` with the same number of rows but place `y` in reverse project order. Validation cannot detect the mistake unless project identifiers are checked during data assembly.

## 2.10 Day 2 practice

1. State the shape of a dataset containing 70 projects and 5 features.
2. After adding an intercept, what are the shapes of  $\mathbf{X}$  and  $\beta$ ?
3. Why is `project_id` important even when it is not a numerical feature?
4. Give one example of target leakage in an MHP cost model.
5. Extend `_validate_inputs` to reject a constant target vector, but explain why a constant target is not mathematically invalid in every context.

### Day 2 exit check

Given `X.shape == (40, 6)` and `beta.shape == (5,)`, you should immediately say: "Multiplication is impossible because the inner dimensions, 6 and 5, do not match."

## Day 3 — Linear Algebra as a System of Predictions

**Today's central idea:** A linear model forms each prediction by adding weighted features. Matrix notation performs that same operation for every project at once.

### 3.0 Matplotlib in five lines

Matplotlib turns arrays into figures. `plt.figure()` starts a canvas, `scatter` draws points, `plot` joins coordinates with a line, `legend` labels the layers, and `show` displays the finished result.

```
import matplotlib.pyplot as plt

x_demo = [0, 1, 2]
y_demo = [1, 3, 5]
plt.figure(figsize=(5, 3))
plt.scatter(x_demo, y_demo, label="observations")
plt.plot(x_demo, y_demo, color="darkorange", label="joined points")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```

Every later figure repeats this pattern with different drawing commands. Run this tiny example first; if no window or notebook figure appears, fix the plotting setup before continuing.

### 3.1 From a verbal rule to a linear equation

Suppose a deliberately simple budget rule uses only cable length:

Begin with a fixed base cost of 8 million PKR, then add 4 million PKR for each kilometre of cable.

For project  $i$ :

$$\hat{y}_i = 8 + 4x_i.$$

The general one-feature linear model is:

$$\hat{y}_i = \beta_0 + \beta_1 x_i.$$

- $\beta_0$  is the **intercept**: the model's predicted target when  $x_i = 0$ .
- $\beta_1$  is the **slope**: the predicted change in  $y$  associated with a one-unit increase in  $x$ .

If  $y$  is million PKR and  $x$  is km, the slope's units are million PKR per km.

The intercept may be necessary for good predictions without having a useful physical interpretation. "A project with zero cable" may lie outside the scope of the data. An intercept is an algebraic baseline, not automatically the price of a literal zero-feature project.

## 3.2 Multiple features are still a weighted sum

With road distance, capacity, and terrain difficulty:

$$\hat{y}_i = \beta_0 + \beta_1 \text{road\_distance}_i + \beta_2 \text{capacity}_i + \beta_3 \text{terrain}_i.$$

For one project, this is a dot product between a row of feature values and a vector of parameters. For every project, it becomes matrix–vector multiplication:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}.$$

The apparent simplicity of the matrix equation is earned by carefully constructing  $\mathbf{X}$ .

## 3.3 Why a column of ones creates an intercept

For three projects and one measured feature, start with:

$$\mathbf{X}_{\text{raw}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}.$$

Prepend a column of ones:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}.$$

Now multiply row by column:

$$\mathbf{X}\boldsymbol{\beta} = \begin{bmatrix} 1\beta_0 + x_1\beta_1 \\ 1\beta_0 + x_2\beta_1 \\ 1\beta_0 + x_3\beta_1 \end{bmatrix}.$$

The same  $\beta_0$  is added to every project because every first-column entry is 1.

Without the column of ones, the one-feature model is  $\hat{y} = \beta_1 x$ . At  $x = 0$ , it must pass through the origin. With the ones column, the model is **affine**: a linear transformation plus a translation. In everyday statistics it is still called a linear regression because it is linear in the parameters.

### 3.4 Code proof: expand $X @ \text{beta}$ by hand

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
X = np.column_stack([np.ones(x.size), x])
beta = np.array([8.0, 4.0])

y_hat_matrix = X @ beta
y_hat_manual = np.array([
 1.0 * 8.0 + 1.0 * 4.0,
 1.0 * 8.0 + 2.0 * 4.0,
 1.0 * 8.0 + 3.0 * 4.0,
])

print(X)
print(y_hat_matrix)
assert np.array_equal(y_hat_matrix, y_hat_manual)
assert np.array_equal(y_hat_matrix, np.array([12.0, 16.0, 20.0]))
```

Every multiplication in the displayed equation appears explicitly in the code.

### 3.5 Shape algebra before numerical algebra

If:

$$X \in \mathbb{R}^{n \times (p+1)} \quad \text{and} \quad \beta \in \mathbb{R}^{p+1},$$

then:

$$X\beta \in \mathbb{R}^n.$$

The inner dimensions must agree:

$$(n \times (p+1)) (p+1 \times 1) = n \times 1.$$

This is not a software convention. It follows from the dot product: each row of  $X$  needs exactly one parameter for each column.

#### Symbol-to-Python map

| Mathematics  | Meaning               | NumPy                                           |
|--------------|-----------------------|-------------------------------------------------|
| $X\beta$     | Matrix–vector product | <code>X @ beta</code>                           |
| $X^T$        | Transpose             | <code>X.T</code>                                |
| $x^T y$      | Dot product           | <code>x @ y</code> or <code>np.dot(x, y)</code> |
| $I$          | Identity matrix       | <code>np.eye(p)</code>                          |
| $\mathbf{1}$ | Vector of ones        | <code>np.ones(n)</code>                         |
| $\ v\ _2$    | Euclidean length      | <code>np.linalg.norm(v)</code>                  |

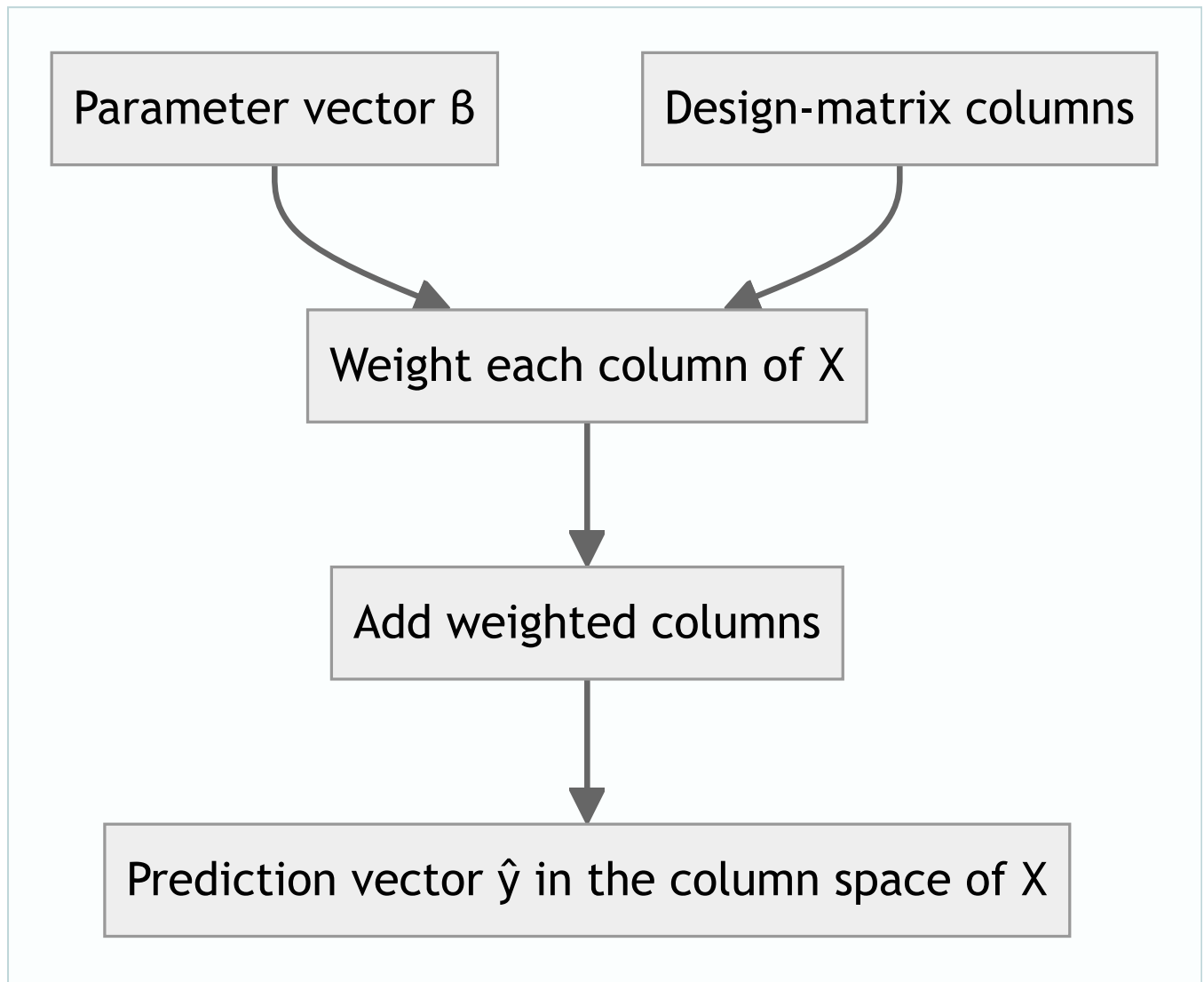
### 3.6 Matrices as transformations—and the limit of the analogy

A matrix can transform geometric coordinates by stretching, shrinking, rotating, reflecting, or shearing vectors. That view is useful for understanding linear algebra. In regression, however, the most immediate interpretation of  $\mathbf{X}\boldsymbol{\beta}$  is slightly different: the columns of  $\mathbf{X}$  are building blocks, and  $\boldsymbol{\beta}$  chooses how much of each column to combine.

If  $\mathbf{X}$  has columns  $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_p$ , then:

$$\mathbf{X}\boldsymbol{\beta} = \beta_0\mathbf{x}_0 + \beta_1\mathbf{x}_1 + \dots + \beta_p\mathbf{x}_p.$$

Because  $\mathbf{x}_0$  is the column of ones,  $\beta_0\mathbf{x}_0$  adds the same amount to every prediction.



**Figure 3.1 — Column-combination view of regression.** The prediction vector is not an arbitrary vector; it must be built from the columns available in  $\mathbf{X}$ .

### 3.7 Figure lab: with and without an intercept

Use four synthetic projects:

```
import matplotlib.pyplot as plt
import numpy as np

x_cable_km = np.array([1.0, 2.0, 3.0, 4.0])
```



```

y_cost_m_pkr = np.array([12.0, 19.0, 29.0, 38.0])

OLS through the origin: minimise ||y - xb||^2
b_origin = (x_cable_km @ y_cost_m_pkr) / (x_cable_km @ x_cable_km)

OLS with an intercept: solve the normal equations
X = np.column_stack([np.ones(x_cable_km.size), x_cable_km])
beta = np.linalg.solve(X.T @ X, X.T @ y_cost_m_pkr)

grid = np.linspace(0, 5, 100)

plt.figure(figsize=(8, 5))
plt.scatter(x_cable_km, y_cost_m_pkr, color="navy", s=70, label="Projects")
plt.plot(grid, b_origin * grid, "--", color="firebrick", label="Forced through origin")
plt.plot(grid, beta[0] + beta[1] * grid, color="darkgreen", label="With intercept")
plt.axhline(0, color="grey", linewidth=0.6)
plt.axvline(0, color="grey", linewidth=0.6)
plt.xlabel("Cable length (km)")
plt.ylabel("Actual cost (million PKR)")
plt.title("The intercept translates the fitted line")
plt.legend()
plt.tight_layout()
plt.show()

print("Through-origin slope:", b_origin)
print("Intercept and slope:", beta)

```

**Figure 3.2 — The effect of the intercept.** The through-origin model is required to predict zero cost at zero cable length. The intercept model is allowed to fit the observed baseline.

Notice that the code uses `np.linalg.solve(A, b)` for  $A\beta = b$ , not `np.linalg.inv(A) @ b`. The inverse formula is useful for derivation, but directly solving the linear system is clearer and usually more numerically stable.

## 3.8 Scaling and units

Suppose cable is changed from kilometres to metres:

$$x_{\text{metres}} = 1000x_{\text{km}}.$$

To preserve the same prediction:

$$\beta_{\text{metres}} = \frac{\beta_{\text{km}}}{1000}.$$

Then:

$$\beta_{\text{metres}}x_{\text{metres}} = \frac{\beta_{\text{km}}}{1000}(1000x_{\text{km}}) = \beta_{\text{km}}x_{\text{km}}.$$

### Code proof: changing units changes the coefficient, not the prediction

```

x_km = np.array([1.0, 2.0, 3.0])
x_m = 1000.0 * x_km

beta_km = 4.0 # million PKR per km
beta_m = beta_km / 1000.0

assert np.allclose(beta_km * x_km, beta_m * x_m)

```

This is why comparing coefficient magnitudes across differently scaled features is misleading. A smaller numerical coefficient need not represent a less important variable.

## 3.9 Build: intercept construction and prediction

Add these methods inside `MHPCostEstimator`:

```
@staticmethod
def _add_intercept_column(X):
 """Prepend one constant column; X must already be validated and 2D."""
 ones = np.ones((X.shape[0], 1), dtype=float)
 return np.hstack([ones, X])

def predict(self, X):
 """Return predictions for new feature rows."""
 if not self.is_fitted_:
 raise RuntimeError("fit the estimator before calling predict")

 X = np.asarray(X, dtype=float)
 if X.ndim == 1:
 X = X.reshape(1, -1) # one new project with p features
 X = self._validate_inputs(X)
 if X.shape[1] != len(self.feature_names_):
 raise ValueError(
 f"expected {len(self.feature_names_)} features, got {X.shape[1]}"
)

 X_design = self._add_intercept_column(X)
 return X_design @ self.parameters_
```

`predict` cannot yet succeed because `fit` has not been built. This is intentional. We have defined the behaviour of a fitted application before learning how to estimate the parameters.

The context resolves a genuine shape ambiguity: during fitting, a one-dimensional `X` means one feature observed across many projects; during prediction, a one-dimensional `X` means one new project containing several features. Passing an explicit two-dimensional array remains safest.

## 3.10 Break it deliberately: broadcasting is not matrix multiplication

```
X = np.array([
 [1.0, 12.0, 100.0],
 [1.0, 25.0, 150.0],
])
beta = np.array([5.0, 0.6, 0.2])

correct = X @ beta
print("correct shape:", correct.shape, "values:", correct)

elementwise = X * beta
print("elementwise shape:", elementwise.shape)
print(elementwise)
```

`X * beta` broadcasts the parameter vector across rows and multiplies element by element. It returns a matrix of products, not a prediction vector. The expression runs without an error, which makes it more dangerous than a crash.

Also try multiplying the same `X` by `np.array([5.0, 0.6])`. Matrix multiplication should fail because the number of parameters does not equal the number of columns.

## 3.11 Day 3 practice

1. Expand one row of  $X\beta$  for an intercept plus three features.

2. Explain why a column of twos would not estimate the conventional intercept. What would its coefficient represent?
3. If cost changes from million PKR to PKR, how do the coefficients change?
4. Write a `matvec_with_loops(X, beta)` function that rejects incompatible dimensions.
5. Plot a dataset for which the intercept has a strong effect on fit and another for which it has little effect.

## Day 3 exit check

You should be able to explain both statements:

- “ $X\beta$  is one dot product per project.”
- “ $X\beta$  is a weighted combination of the columns of  $X$ .”

They are two views of the same operation.

# Day 4 — Residuals, Loss, and the Geometry of OLS

**Today's central idea:** OLS chooses the attainable prediction vector closest to the observed target vector in squared Euclidean distance.

## 4.1 A prediction becomes testable through its residual

For project  $i$ :

$$e_i = y_i - \hat{y}_i.$$

- $e_i > 0$ : actual cost exceeded predicted cost; the model underpredicted.
- $e_i < 0$ : actual cost was below predicted cost; the model overpredicted.
- $e_i = 0$ : prediction matched the observed target.

The sign convention matters. This chapter uses **actual minus predicted**.

For all projects:

$$e = y - \hat{y} = y - X\beta.$$

## Code proof: the vector equation and the elementwise equation agree

```
import numpy as np

y = np.array([12.0, 19.0, 29.0])
y_hat = np.array([10.5, 20.0, 27.5])

e_vector = y - y_hat
e_manual = np.array([
 12.0 - 10.5,
 19.0 - 20.0,
 29.0 - 27.5,
```

```

])

assert np.allclose(e_vector, e_manual)
print(e_vector) # [1.5 -1. 1.5]

```

Residuals are observed after fitting. They are not the same as the unobservable population errors in a statistical model, though introductory discussions often use the words loosely.

## 4.2 Why OLS squares residuals

The **sum of squared residuals** (SSR), also called the residual sum of squares (RSS) or sum of squared errors (SSE), is:

$$SSR(\beta) = \sum_{i=1}^n e_i^2 = \sum_{i=1}^n (y_i - x_i^T \beta)^2.$$

In vector notation:

$$SSR(\beta) = e^T e = (y - X\beta)^T (y - X\beta).$$

Squaring has four important consequences:

1. Positive and negative residuals cannot cancel.
2. Large residuals receive disproportionate weight.
3. The objective is smooth and differentiable everywhere.
4. The objective has a geometric interpretation as squared Euclidean distance.

The second consequence is not automatically desirable. A data-entry error of 390 million instead of 39 million can dominate an OLS fit. High stakes do not, by themselves, prove that squared loss is the right institutional loss function. The analyst must inspect outliers and ask whether the objective reflects the decision.

### Code proof: $e^T e$ equals the sum of elementwise squares

```

e = np.array([1.5, -1.0, 1.5])

ssr_loop = 0.0
for value in e:
 ssr_loop += value ** 2

ssr_sum = np.sum(e ** 2)
ssr_dot = e.T @ e

print(ssr_loop, ssr_sum, ssr_dot)
assert np.isclose(ssr_loop, ssr_sum)
assert np.isclose(ssr_sum, ssr_dot)

```

## 4.3 SSR, MSE, RMSE, and MAE answer different reporting needs

$$MSE = \frac{1}{n} \sum_{i=1}^n e_i^2,$$

$$RMSE = \sqrt{MSE},$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |e_i|.$$

| Metric | Units                | Sensitivity to large residuals | Typical use                                           |
|--------|----------------------|--------------------------------|-------------------------------------------------------|
| SSR    | squared target units | high                           | Fitting and comparing models on the same observations |
| MSE    | squared target units | high                           | Average squared loss                                  |
| RMSE   | target units         | high                           | Error summary interpretable in cost units             |
| MAE    | target units         | lower                          | Typical absolute miss, more robust than RMSE          |

With targets in million PKR, RMSE and MAE are also in million PKR. MSE is in squared million PKR, which is harder to communicate.

## Code proof: compute every metric from definitions

```
def regression_metrics(y, y_hat):
 y = np.asarray(y, dtype=float)
 y_hat = np.asarray(y_hat, dtype=float)
 if y.shape != y_hat.shape:
 raise ValueError("y and y_hat must have the same shape")

 residuals = y - y_hat
 ssr = np.sum(residuals ** 2)
 mse = ssr / residuals.size
 rmse = np.sqrt(mse)
 mae = np.sum(np.abs(residuals)) / residuals.size
 return {"ssr": ssr, "mse": mse, "rmse": rmse, "mae": mae}

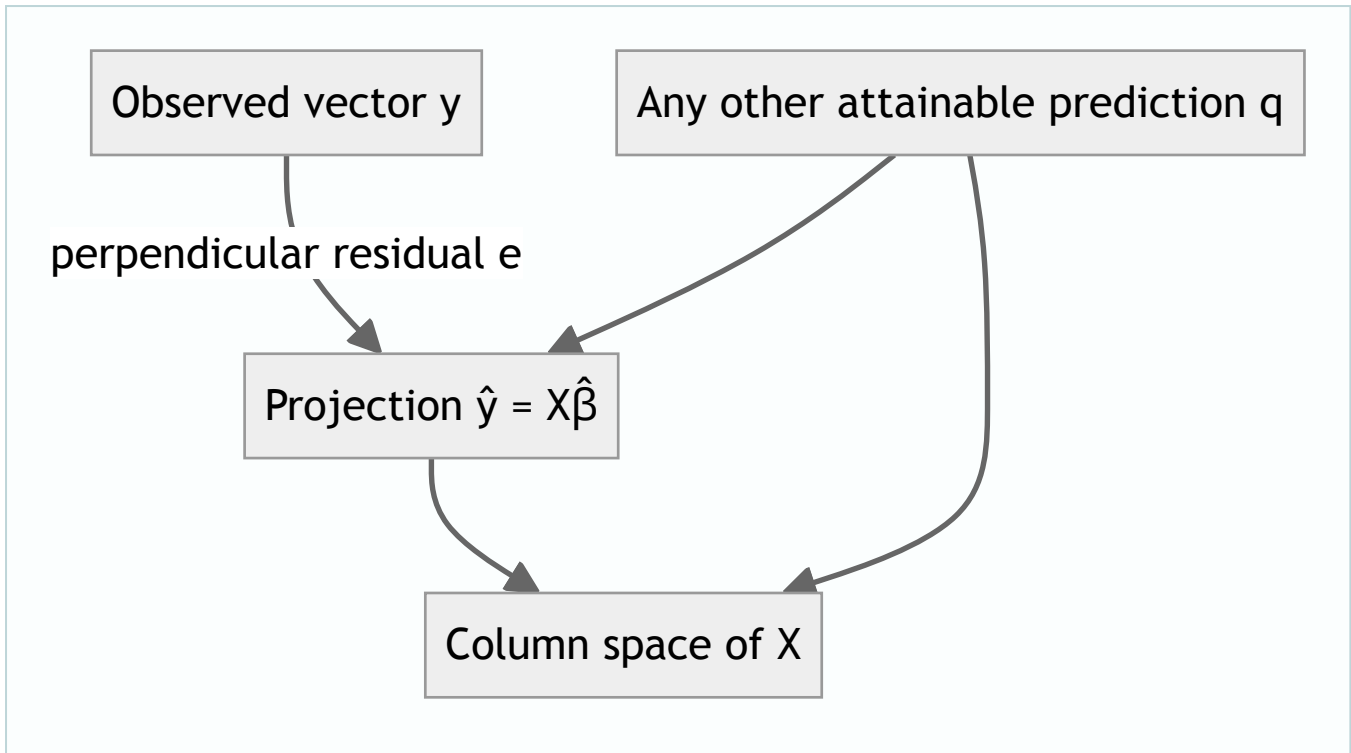
metrics = regression_metrics(
 y=np.array([12.0, 19.0, 29.0]),
 y_hat=np.array([10.5, 20.0, 27.5]),
)
print(metrics)
```

For training data, some texts divide SSR by  $n - p - 1$  to estimate error variance. That is a different quantity from predictive MSE. Always name the denominator.

## 4.4 OLS as a constrained search in observation space

The target vector  $\mathbf{y}$  contains one coordinate for each project, so it lives in  $\mathbb{R}^n$ . Every possible  $\mathbf{X}\boldsymbol{\beta}$  is a weighted combination of the columns of  $\mathbf{X}$ . The set of all such combinations is the **column space** of  $\mathbf{X}$ .

OLS cannot choose any point in  $\mathbb{R}^n$ . It must choose a point inside that column space. It selects the point closest to  $\mathbf{y}$ .



**Figure 4.1 — Projection view of OLS.** The diagram is conceptual:  $\hat{y}$  and  $q$  lie in the column space, while the residual from  $\hat{y}$  to  $y$  is perpendicular to it.

## 4.5 The Pythagorean proof of minimum distance

Let:

- $\hat{y}$  be the perpendicular projection of  $y$  onto the column space of  $X$ ;
- $q$  be any other prediction in the column space;
- $e = y - \hat{y}$  be the OLS residual; and
- $d = \hat{y} - q$  be the displacement within the column space.

Then:

$$y - q = (y - \hat{y}) + (\hat{y} - q) = e + d.$$

Because  $e$  is perpendicular to the entire column space and  $d$  lies inside it:

$$e^T d = 0.$$

Now expand the squared distance:

$$\begin{aligned} \|y - q\|_2^2 &= (e + d)^T (e + d) \\ &= e^T e + e^T d + d^T e + d^T d \\ &= \|e\|_2^2 + \|d\|_2^2 \\ &\geq \|e\|_2^2. \end{aligned}$$

The last step holds because a squared length cannot be negative. Therefore no other attainable prediction  $q$  can be closer to  $y$  than  $\hat{y}$ .

## 4.6 Why orthogonality produces the normal equations

Every column of  $\mathbf{X}$  lies in the column space. If the residual is perpendicular to the whole column space, it is perpendicular to every column:

$$\mathbf{X}^T \mathbf{e} = 0.$$

Substitute  $\mathbf{e} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$ :

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = 0.$$

Distribute  $\mathbf{X}^T$ :

$$\mathbf{X}^T \mathbf{y} - \mathbf{X}^T \mathbf{X} \hat{\boldsymbol{\beta}} = 0.$$

Rearrange:

$$\mathbf{X}^T \mathbf{X} \hat{\boldsymbol{\beta}} = \mathbf{X}^T \mathbf{y}.$$

These are the **normal equations**. “Normal” refers to perpendicularity, not to the normal probability distribution.

If an intercept column is present, one of the orthogonality conditions is:

$$\mathbf{1}^T \mathbf{e} = \sum_{i=1}^n e_i = 0.$$

Therefore OLS residuals from the training data sum to zero when the model includes an intercept, up to floating-point error.

## 4.7 Code proof: orthogonality and Pythagoras

```
import numpy as np

x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([12.0, 19.0, 29.0, 38.0])
X = np.column_stack([np.ones(x.size), x])

beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
y_hat = X @ beta_hat
e = y - y_hat

Orthogonality: one near-zero dot product per design column
print("X.T @ e =", X.T @ e)
assert np.allclose(X.T @ e, np.zeros(X.shape[1]), atol=1e-10)

Intercept implication: residuals sum to zero
assert np.isclose(e.sum(), 0.0, atol=1e-10)

Choose another attainable prediction q = X @ beta_other
beta_other = beta_hat + np.array([2.0, -1.0])
q = X @ beta_other
d = y_hat - q

left = np.linalg.norm(y - q) ** 2
right = np.linalg.norm(e) ** 2 + np.linalg.norm(d) ** 2

print("Pythagorean sides:", left, right)
assert np.isclose(e @ d, 0.0, atol=1e-10)
assert np.isclose(left, right, atol=1e-10)
assert left >= np.linalg.norm(e) ** 2
```

This block verifies three equations: the normal-equation orthogonality condition, the zero-sum residual property, and the Pythagorean decomposition.

## 4.8 Figure lab: draw the projection in three dimensions

**Optional visual extension.** This 3D plot is useful intuition but uses advanced plotting APIs that are not required for the algebra, code, or exit check. Skip it on a first pass if `quiver`, `plot_surface`, or 3D axes distract from the projection idea.

Three observations allow the target and prediction vectors to be drawn in  $\mathbb{R}^3$ .

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
X = np.column_stack([np.ones(3), x])
y = np.array([1.0, 3.0, 2.0])

beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
y_hat = X @ beta_hat
e = y - y_hat

fig = plt.figure(figsize=(8, 7))
ax = fig.add_subplot(111, projection="3d")
origin = np.zeros(3)

ax.quiver(*origin, *y, color="navy", label="Observed y", arrow_length_ratio=0.08)
ax.quiver(*origin, *y_hat, color="darkgreen", label="Projection $\hat{y}$ ", arrow_length_ratio=0.08)
ax.quiver(*y_hat, *e, color="firebrick", label="Residual e", arrow_length_ratio=0.15)

Draw a patch of the column space span{ones, x}
a = np.linspace(-0.5, 2.5, 10)
b = np.linspace(-1.0, 1.0, 10)
A, B = np.meshgrid(a, b)
plane = A[..., None] * X[:, 0] + B[..., None] * X[:, 1]
ax.plot_surface(plane[..., 0], plane[..., 1], plane[..., 2], alpha=0.2)

ax.set_xlabel("Project 1 coordinate")
ax.set_ylabel("Project 2 coordinate")
ax.set_zlabel("Project 3 coordinate")
ax.set_title("OLS projects y onto the column space of X")
ax.legend()
plt.tight_layout()
plt.show()
```

Here `[..., None]` keeps every existing axis and adds one new length-one axis at the end. That makes NumPy broadcast each grid coefficient across the three coordinates of the vector it multiplies.

**Figure 4.2 — OLS geometry in observation space.** The residual begins at  $\hat{y}$  and ends at  $y$ ; it is perpendicular to the surface of attainable predictions.

## 4.9 Build: metrics as methods

Add these methods to `MHPCostEstimator`:

```
@staticmethod
def _metrics_from_predictions(y, y_hat):
```



```

y = np.asarray(y, dtype=float).reshape(-1)
y_hat = np.asarray(y_hat, dtype=float).reshape(-1)
if y.shape != y_hat.shape:
 raise ValueError("y and y_hat must have identical shapes")

residuals = y - y_hat
ssr = float(residuals @ residuals)
mse = ssr / residuals.size
return {
 "ssr": ssr,
 "mse": mse,
 "rmse": float(np.sqrt(mse)),
 "mae": float(np.mean(np.abs(residuals))),
}

def evaluate(self, X, y):
 """Evaluate a fitted model on supplied observations."""
 X, y = self._validate_inputs(X, y)
 y_hat = self.predict(X)
 return self._metrics_from_predictions(y, y_hat)

```

The method does not call these “test metrics” because the caller may supply training, validation, or test data. The application should not pretend to know the provenance of an array.

## 4.10 Break it deliberately: one outlier

```

y = np.array([20.0, 22.0, 24.0, 26.0])
y_hat = np.array([21.0, 21.0, 25.0, 25.0])

ordinary = regression_metrics(y, y_hat)

y_with_error = y.copy()
y_with_error[-1] = 260.0 # possible extra zero in data entry
contaminated = regression_metrics(y_with_error, y_hat)

print("Ordinary: ", ordinary)
print("Contaminated:", contaminated)

```

Compare the change in RMSE and MAE. Do not conclude merely that MAE is “better.” First determine whether 260 is a genuine extreme project, a different type of project, or an error. Robustness cannot replace data investigation.

## 4.11 Day 4 practice

1. Calculate the residuals and SSR for  $\mathbf{y} = [3, 5]$  and  $\hat{\mathbf{y}} = [4, 3]$ .
2. Prove in code that RMSE has the same units as the target under a change from million PKR to PKR.
3. Why does including an intercept make training residuals sum to zero?
4. Does a zero sum of residuals imply a good model? Construct a counterexample.
5. Modify Figure 4.2 by choosing a new  $\mathbf{y}$ . Verify that  $\mathbf{X.T} @ \mathbf{e}$  remains near zero.

## Day 4 exit check

You should be able to explain OLS without calculus:

“Among all prediction vectors that can be constructed from the columns of  $\mathbf{X}$ , OLS chooses the one with the shortest squared distance to  $\mathbf{y}$ .”

# Day 5 — Deriving, Fitting, and Stress-Testing OLS

**Today's central idea:** OLS is the parameter value that makes the squared-error surface as small as possible. The normal equation is a consequence, not a formula to memorise.

Day 4 reached the normal equations through geometry:

$$X^T X \hat{\beta} = X^T y.$$

Today we reach the same point through calculus, first with one feature and then with matrices. The two derivations are complementary:

- geometry explains **what** OLS is doing; and
- calculus explains **how** the optimum is located.

## 5.1 The scalar error bowl

Begin with a model that has one parameter and no intercept:

$$\hat{y}_i = \beta x_i.$$

The squared-error objective is:

$$S(\beta) = \sum_{i=1}^n (y_i - \beta x_i)^2.$$

Expand one squared term:

$$(y_i - \beta x_i)^2 = y_i^2 - 2\beta x_i y_i + \beta^2 x_i^2.$$

Therefore:

$$S(\beta) = \sum_i y_i^2 - 2\beta \sum_i x_i y_i + \beta^2 \sum_i x_i^2.$$

Differentiate with respect to  $\beta$ :

$$\frac{dS}{d\beta} = -2 \sum_i x_i y_i + 2\beta \sum_i x_i^2.$$

At a stationary point, the derivative is zero:

$$-2 \sum_i x_i y_i + 2\hat{\beta} \sum_i x_i^2 = 0.$$

Rearrange:

$$\hat{\beta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}.$$

The second derivative is:

$$\frac{d^2 S}{d\beta^2} = 2 \sum_i x_i^2.$$

If at least one  $x_i$  is nonzero, the second derivative is positive, so the stationary point is the unique minimum.

## Code proof: derivative, grid search, and analytic solution

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 7.0])

def ssr_through_origin(beta, x, y):
 return np.sum((y - beta * x) ** 2)

def derivative_through_origin(beta, x, y):
 return -2.0 * np.sum(x * y) + 2.0 * beta * np.sum(x ** 2)

beta_hat = np.sum(x * y) / np.sum(x ** 2)
grid = np.linspace(0.0, 4.0, 4001)
grid_best = grid[np.argmin([ssr_through_origin(b, x, y) for b in grid])]

print("Analytic minimum:", beta_hat)
print("Grid minimum: ", grid_best)
print("Derivative there:", derivative_through_origin(beta_hat, x, y))

assert np.isclose(derivative_through_origin(beta_hat, x, y), 0.0)
assert np.isclose(beta_hat, grid_best, atol=0.001)
assert 2.0 * np.sum(x ** 2) > 0.0
```

The grid is not how we want to fit the model; it is an independent numerical check that the analytic solution sits at the bottom of the bowl.

## 5.2 One feature with an intercept

Now use:

$$\hat{y}_i = \beta_0 + \beta_1 x_i.$$

The objective is:

$$S(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

There are two unknown parameters, so there are two partial derivatives.

For the intercept:

$$\frac{\partial S}{\partial \beta_0} = -2 \sum_i (y_i - \beta_0 - \beta_1 x_i).$$

Set it to zero:

$$\sum_i y_i - n\hat{\beta}_0 - \hat{\beta}_1 \sum_i x_i = 0.$$

Divide by  $n$ :

$$\bar{y} - \hat{\beta}_0 - \hat{\beta}_1 \bar{x} = 0,$$

so:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

This proves that the fitted OLS line with an intercept passes through  $(\bar{x}, \bar{y})$ .

For the slope:

$$\frac{\partial S}{\partial \beta_1} = -2 \sum_i x_i (y_i - \beta_0 - \beta_1 x_i).$$

Substituting the intercept relationship and simplifying gives:

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}.$$

The numerator is the unnormalised co-movement of  $\mathbf{x}$  and  $\mathbf{y}$ ; the denominator is the unnormalised variation in  $\mathbf{x}$ . If every  $\mathbf{x}_i$  is identical, the denominator is zero and the slope cannot be identified.

## Code proof: centred formula, intercept, and mean point

```
x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([12.0, 19.0, 29.0, 38.0])

x_centered = x - x.mean()
y_centered = y - y.mean()

beta_1 = np.sum(x_centered * y_centered) / np.sum(x_centered ** 2)
beta_0 = y.mean() - beta_1 * x.mean()

print("beta_0:", beta_0)
print("beta_1:", beta_1)

The fitted line passes through (x̄, ȳ)
assert np.isclose(beta_0 + beta_1 * x.mean(), y.mean())

Both partial derivatives are zero
residuals = y - (beta_0 + beta_1 * x)
partial_b0 = -2.0 * np.sum(residuals)
partial_b1 = -2.0 * np.sum(x * residuals)
assert np.isclose(partial_b0, 0.0, atol=1e-10)
assert np.isclose(partial_b1, 0.0, atol=1e-10)
```

## 5.3 The matrix objective

For multiple features, collect all parameters in  $\beta$  and write:

$$S(\beta) = (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta).$$

Expand carefully:

$$\begin{aligned} S(\beta) &= (\mathbf{y}^T - \beta^T \mathbf{X}^T)(\mathbf{y} - \mathbf{X}\beta) \\ &= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\beta - \beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X}\beta. \end{aligned}$$

The middle two terms are scalars and transposes of one another, so they are equal:

$$\mathbf{y}^T \mathbf{X}\beta = (\mathbf{y}^T \mathbf{X}\beta)^T = \beta^T \mathbf{X}^T \mathbf{y}.$$

Therefore:

$$S(\beta) = \mathbf{y}^T \mathbf{y} - 2\beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X}\beta.$$

## 5.4 The matrix derivative without hand-waving

### A numeric $2 \times 2$ quadratic before index notation

Let

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix}.$$

Writing all four double-sum terms gives

$$\beta^T \mathbf{A}\beta = 2\beta_1^2 + \beta_1\beta_2 + 3\beta_2\beta_1 + 4\beta_2^2 = 2\beta_1^2 + 4\beta_1\beta_2 + 4\beta_2^2.$$

Differentiate that ordinary polynomial one coordinate at a time:

$$\frac{\partial}{\partial \beta_1} = 4\beta_1 + 4\beta_2, \quad \frac{\partial}{\partial \beta_2} = 4\beta_1 + 8\beta_2.$$

At  $\beta = (5, 6)^T$ , the gradient is  $(44, 68)^T$ . Now verify that the matrix rule produces the same result:

```
import numpy as np

A = np.array([[2.0, 1.0], [3.0, 4.0]])
beta = np.array([5.0, 6.0])
gradient_by_expansion = np.array([
 4 * beta[0] + 4 * beta[1],
 4 * beta[0] + 8 * beta[1],
])
gradient_by_matrix_rule = (A + A.T) @ beta

print(gradient_by_expansion)
assert np.allclose(gradient_by_expansion, gradient_by_matrix_rule)
```

The general argument below says exactly the same thing for any matrix size.

Three derivative facts are needed.

First,  $\mathbf{y}^T \mathbf{y}$  contains no  $\beta$ , so:

$$\nabla_{\beta}(\mathbf{y}^T \mathbf{y}) = 0.$$

Second, for a constant vector  $\mathbf{c}$ :

$$\nabla_{\beta}(\beta^T \mathbf{c}) = \mathbf{c}.$$

Thus:

$$\nabla_{\beta}(-2\beta^T X^T y) = -2X^T y.$$

Third, let  $A = X^T X$ . Expand the quadratic form by coordinates:

$$\beta^T A \beta = \sum_j \sum_k \beta_j A_{jk} \beta_k.$$

The derivative with respect to coordinate  $\beta_r$  collects terms where  $j = r$  and where  $k = r$ :

$$\frac{\partial}{\partial \beta_r}(\beta^T A \beta) = \sum_k A_{rk} \beta_k + \sum_j \beta_j A_{jr}.$$

In vector form:

$$\nabla_{\beta}(\beta^T A \beta) = (A + A^T)\beta.$$

Because  $A = X^T X$  is symmetric,  $A = A^T$ :

$$\nabla_{\beta}(\beta^T X^T X \beta) = 2X^T X \beta.$$

Combine the three derivatives:

$$\nabla_{\beta} S(\beta) = -2X^T y + 2X^T X \beta.$$

At the minimum:

$$-2X^T y + 2X^T X \hat{\beta} = 0,$$

which reduces again to:

$$X^T X \hat{\beta} = X^T y.$$

## Code proof: compare the analytic gradient with finite differences

A finite-difference gradient estimates each partial derivative by nudging one parameter a very small distance:

$$\frac{\partial S}{\partial \beta_j} \approx \frac{S(\beta + h u_j) - S(\beta - h u_j)}{2h},$$

where  $u_j$  is a vector with 1 in position  $j$  and 0 elsewhere.

```
import numpy as np

X = np.array([
 [1.0, 1.0],
 [1.0, 2.0],
 [1.0, 3.0],
 [1.0, 4.0],
])
y = np.array([12.0, 19.0, 29.0, 38.0])
beta = np.array([4.0, 7.0])

def ssr(beta_vector):
 residuals = y - X @ beta_vector
 return residuals @ residuals

analytic_gradient = -2.0 * X.T @ (y - X @ beta)

h = 1e-6
finite_difference_gradient = np.zeros_like(beta)
for j in range(beta.size):
```

```

unit = np.zeros_like(beta)
unit[j] = 1.0
finite_difference_gradient[j] = (
 ssr(beta + h * unit) - ssr(beta - h * unit)
) / (2.0 * h)

print("Analytic:", analytic_gradient)
print("Finite difference:", finite_difference_gradient)
assert np.allclose(analytic_gradient, finite_difference_gradient, rtol=1e-6)

```

The finite-difference calculation is an independent numerical audit of the matrix derivative. A poor choice of  $h$  can introduce approximation or floating-point error; its role is verification, not fitting.

## 5.5 Isolating $\hat{\beta}$ and stating the missing condition

If  $X^T X$  is invertible:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

This famous closed form is valid only when the design matrix has full column rank. In plain language, no design column can be constructed exactly from the others, and there must be enough independent information to estimate every parameter.

The Hessian—the matrix of second derivatives—is:

$$\nabla_{\beta}^2 S(\beta) = 2X^T X.$$

For any vector  $z$ :

$$z^T X^T X z = (Xz)^T (Xz) = \|Xz\|_2^2 \geq 0.$$

Thus  $X^T X$  is positive semidefinite and the objective is convex. If  $X$  has full column rank,  $Xz \neq \mathbf{0}$  for every nonzero  $z$ , so the quantity is strictly positive and the objective is strictly convex. The minimiser is then unique.

### Code proof: positive semidefiniteness

```

rng = np.random.default_rng(7)
X = rng.normal(size=(20, 4))
z = rng.normal(size=4)

left = z.T @ X.T @ X @ z
right = np.linalg.norm(X @ z) ** 2

print(left, right)
assert np.isclose(left, right)
assert left >= -1e-12 # tolerance for floating-point roundoff

```

## 5.6 Why we do not compute the inverse in application code

The derivation uses  $(X^T X)^{-1}$  to express the mathematical solution. A program should usually solve:

$$(X^T X)\hat{\beta} = X^T y$$

with:

```
beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
```

Better still for OLS, solve the least-squares problem directly:

```
beta_hat, residual_sums, rank, singular_values = np.linalg.lstsq(X, y, rcond=None)
```

Why?

- Forming  $X^T X$  squares the condition number and can magnify numerical instability.
- Explicit inversion performs unnecessary work.
- `lstsq` uses a decomposition designed for least-squares problems and also reports rank and singular values.

This is not a conceptual shortcut. We have already built and derived the objective. It is a numerical implementation choice that respects the same mathematics.

## Research note: fitting assumptions are not inference assumptions

The OLS coefficient calculation does **not** require the residuals to be normally distributed. To compute a least-squares projection, we need a defined numerical problem; for a unique coefficient vector, we additionally need full column rank.

Stronger claims require stronger assumptions. Unbiased causal or explanatory coefficients require a defensible relationship between included features and the unobserved error. Classical small-sample confidence intervals add assumptions about variance and, in some treatments, normality. Consistency, robust standard errors, clustered data, and sampling design introduce further conditions.

Keep three layers separate:

1. **Algebra:** Did the program minimise SSR correctly?
2. **Statistical generalisation:** Does this sample support claims about new projects or a wider population?
3. **Causal interpretation:** Does the design identify an intervention effect?

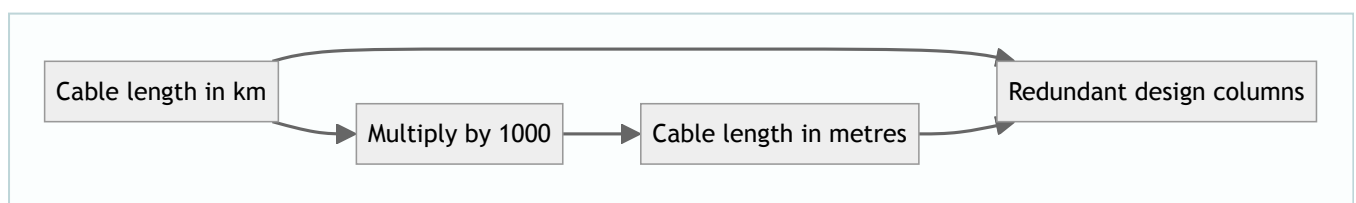
A perfect answer at the first layer does not settle the next two.

## 5.7 Rank failure: two columns carrying the same information

Suppose the design matrix includes both cable length in kilometres and the same cable length in metres:

$$x_{\text{metres}} = 1000x_{\text{kilometres}}.$$

One column is an exact multiple of another. The model cannot decide how much coefficient to assign to each, because infinitely many pairs produce the same combined prediction.



**Figure 5.1 — Perfect multicollinearity.** More columns do not necessarily provide more information.



## Break it deliberately

```
x_km = np.array([1.0, 2.0, 3.0, 4.0])
x_m = 1000.0 * x_km
y = np.array([12.0, 19.0, 29.0, 38.0])

X_bad = np.column_stack([np.ones(x_km.size), x_km, x_m])

print("columns:", X_bad.shape[1])
print("rank: ", np.linalg.matrix_rank(X_bad))

try:
 beta_bad = np.linalg.solve(X_bad.T @ X_bad, X_bad.T @ y)
except np.linalg.LinAlgError as error:
 print("Normal-equation solve failed:", error)

beta_lstsq, _, rank, singular_values = np.linalg.lstsq(X_bad, y, rcond=None)
print("One least-squares solution:", beta_lstsq)
print("Reported rank:", rank)
print("Singular values:", singular_values)
```

`lstsq` can return a prediction-minimising solution, but the individual coefficients on kilometres and metres are not separately meaningful. A numerical answer does not repair an unidentified interpretation.

The correct response is usually to remove the redundant feature or redesign the representation—not to celebrate that the program returned numbers.

## 5.8 Figure lab: see the error valley

For a model with an intercept and one slope, SSR is a surface over  $(\beta_0, \beta_1)$ .

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([12.0, 19.0, 29.0, 38.0])

b0_grid = np.linspace(-5.0, 20.0, 200)
b1_grid = np.linspace(2.0, 12.0, 200)
B0, B1 = np.meshgrid(b0_grid, b1_grid)

SSR = np.zeros_like(B0)
for i in range(x.size):
 SSR += (y[i] - (B0 + B1 * x[i])) ** 2

X = np.column_stack([np.ones(x.size), x])
beta_hat = np.linalg.lstsq(X, y, rcond=None)[0]

plt.figure(figsize=(8, 6))
contours = plt.contour(B0, B1, SSR, levels=25)
plt.clabel(contours, inline=True, fontsize=7)
plt.scatter(beta_hat[0], beta_hat[1], color="red", s=80, label="OLS minimum")
plt.xlabel("Intercept β_0 ")
plt.ylabel("Slope β_1 ")
plt.title("Contours of the OLS error surface")
plt.legend()
plt.tight_layout()
plt.show()
```

**Figure 5.2 — The OLS error valley.** Each contour joins parameter pairs with equal SSR. The centre is the minimum. With highly correlated features, the contours become long and narrow; with perfect collinearity, the unique centre becomes a flat valley.

## 5.9 Build: the complete Week 1 estimator

The complete application below consolidates the daily increments. Replace the partial `mhp_estimator.py` with this version.

```
from dataclasses import dataclass
from typing import Literal, Sequence

import numpy as np

AnalysisPurpose = Literal["prediction", "explanation", "causation"]

@dataclass(frozen=True)
class AnalysisContract:
 purpose: AnalysisPurpose
 target_name: str
 target_unit: str
 unit_of_observation: str
 intended_use: str

 def __post_init__(self):
 allowed = {"prediction", "explanation", "causation"}
 if self.purpose not in allowed:
 raise ValueError(f"purpose must be one of {sorted(allowed)}")
 for name, value in vars(self).items():
 if not str(value).strip():
 raise ValueError(f"{name} cannot be empty")

class MHPCostEstimator:
 """Small OLS estimator for learning, diagnostics, and explicit reasoning."""

 def __init__(self, contract: AnalysisContract):
 self.contract = contract
 self.parameters_ = None
 self.feature_names_ = None
 self.rank_ = None
 self.singular_values_ = None
 self.training_metrics_ = None
 self.is_fitted_ = False

 def __repr__(self):
 status = "fitted" if self.is_fitted_ else "unfitted"
 return f"MHPCostEstimator(purpose={self.contract.purpose!r}, {status})"

 @staticmethod
 def _validate_inputs(X, y=None):
 X = np.asarray(X, dtype=float)
 if X.ndim == 1:
 X = X.reshape(-1, 1)
 if X.ndim != 2:
 raise ValueError(f"X must be 2D after conversion; got shape {X.shape}")
 if X.shape[0] == 0 or X.shape[1] == 0:
 raise ValueError("X must contain at least one row and one feature")
 if not np.isfinite(X).all():
 raise ValueError("X contains NaN or infinite values")

 if y is None:
 return X

 y = np.asarray(y, dtype=float)
 if y.ndim == 2 and y.shape[1] == 1:
 y = y.reshape(-1)
 if y.ndim != 1:
 raise ValueError(f"y must be 1D; got shape {y.shape}")
 if X.shape[0] != y.shape[0]:
```

```

 raise ValueError(
 f"row mismatch: X has {X.shape[0]} rows but y has {y.shape[0]}"
)
 if not np.isfinite(y).all():
 raise ValueError("y contains NaN or infinite values")
 return X, y

@staticmethod
def _add_intercept_column(X):
 return np.column_stack([np.ones(X.shape[0]), X])

@staticmethod
def _metrics_from_predictions(y, y_hat):
 y = np.asarray(y, dtype=float).reshape(-1)
 y_hat = np.asarray(y_hat, dtype=float).reshape(-1)
 if y.shape != y_hat.shape:
 raise ValueError("y and y_hat must have identical shapes")

 residuals = y - y_hat
 ssr = float(residuals @ residuals)
 mse = ssr / residuals.size
 return {
 "ssr": ssr,
 "mse": mse,
 "rmse": float(np.sqrt(mse)),
 "mae": float(np.mean(np.abs(residuals))),
 }

def fit(self, X, y, feature_names: Sequence[str]):
 X, y = self._validate_inputs(X, y)
 feature_names = [str(name).strip() for name in feature_names]

 if len(feature_names) != X.shape[1]:
 raise ValueError(
 f"received {len(feature_names)} names for {X.shape[1]} features"
)
 if any(not name for name in feature_names):
 raise ValueError("feature names cannot be empty")
 if len(set(feature_names)) != len(feature_names):
 raise ValueError("feature names must be unique")

 X_design = self._add_intercept_column(X)
 parameters, _, rank, singular_values = np.linalg.lstsq(
 X_design, y, rcond=None
)

 required_rank = X_design.shape[1]
 if rank < required_rank:
 raise ValueError(
 "design matrix is rank deficient: "
 f"rank {rank}, but {required_rank} independent columns are required"
)

 self.parameters_ = parameters
 self.feature_names_ = feature_names
 self.rank_ = int(rank)
 self.singular_values_ = singular_values
 self.is_fitted_ = True

 training_predictions = X_design @ self.parameters_
 self.training_metrics_ = self._metrics_from_predictions(
 y, training_predictions
)
 return self

def predict(self, X):
 if not self.is_fitted_:
 raise RuntimeError("fit the estimator before calling predict")

 X = np.asarray(X, dtype=float)

```

```

 if X.ndim == 1:
 X = X.reshape(1, -1)
 X = self._validate_inputs(X)
 if X.shape[1] != len(self.feature_names_):
 raise ValueError(
 f"expected {len(self.feature_names_)} features, got {X.shape[1]}"
)

 return self._add_intercept_column(X) @ self.parameters_

def evaluate(self, X, y):
 X, y = self._validate_inputs(X, y)
 return self._metrics_from_predictions(y, self.predict(X))

def coefficients(self):
 if not self.is_fitted_:
 raise RuntimeError("fit the estimator before requesting coefficients")
 names = ["intercept", *self.feature_names_]
 return dict(zip(names, self.parameters_))

```

## 5.10 End-to-end use on synthetic KP projects

The small dataset below has three pre-construction features. Keeping it small makes the entire calculation inspectable; it is not enough for a real deployment.

```

import numpy as np

contract = AnalysisContract(
 purpose="prediction",
 target_name="actual_cost_m_pkr",
 target_unit="million PKR",
 unit_of_observation="one completed MHP project",
 intended_use="design-stage budget review for approved projects",
)

Columns: road distance (km), planned capacity (kW), terrain index (1-5)
X_train = np.array([
 [5.0, 80.0, 2.0],
 [8.0, 100.0, 2.0],
 [12.0, 120.0, 3.0],
 [15.0, 150.0, 3.0],
 [18.0, 160.0, 4.0],
 [22.0, 180.0, 4.0],
 [25.0, 200.0, 5.0],
 [30.0, 240.0, 5.0],
])

y_train = np.array([19.0, 23.0, 30.0, 35.0, 42.0, 47.0, 55.0, 63.0])

model = MHPCostEstimator(contract).fit(
 X_train,
 y_train,
 feature_names=["road_distance_km", "planned_capacity_kw", "terrain_index"],
)

print(model)
print("Coefficients:", model.coefficients())
print("Training metrics:", model.training_metrics_)

X_new = np.array([
 [20.0, 170.0, 4.0],
 [10.0, 110.0, 3.0],
])
print("New predictions:", model.predict(X_new))

```

## Interpretation discipline

If the fitted road-distance coefficient is positive, the prediction reading is:

“Holding the other included numerical features fixed, the fitted rule adds this many million PKR for each additional kilometre of recorded road distance.”

It is **not** automatically:

“Building one kilometre of road will change MHP cost by this amount.”

The second statement is causal and may even reverse the intervention: the feature is distance from a road, while the action is constructing a road.

Also inspect whether “holding capacity and terrain fixed” describes plausible comparisons in the dataset. Regression can calculate contrasts that the observed projects barely support.

## 5.11 Verify the class against the equations

```
X_design = np.column_stack([np.ones(X_train.shape[0]), X_train])
beta_from_normal_equations = np.linalg.solve(
 X_design.T @ X_design,
 X_design.T @ y_train,
)

assert np.allclose(model.parameters_, beta_from_normal_equations)

residuals = y_train - model.predict(X_train)
assert np.allclose(X_design.T @ residuals, 0.0, atol=1e-9)
assert np.isclose(residuals.sum(), 0.0, atol=1e-9)

metrics = model.evaluate(X_train, y_train)
assert np.isclose(metrics["ssr"], residuals @ residuals)
assert np.isclose(metrics["rmse"], np.sqrt(np.mean(residuals ** 2)))
```

These are not decorative tests. Each one ties an application output to a mathematical result developed earlier in the chapter.

## 5.12 What the completed application still cannot do

The Week 1 estimator is a learning instrument, not a deployment-ready public investment system. It does not yet:

- split data into training, validation, and test sets;
- handle categorical districts, project types, or contractors;
- standardise features or examine conditioning;
- quantify coefficient uncertainty;
- diagnose heteroskedasticity, nonlinearity, or influential observations;
- model time trends, inflation, or district-level clustering;
- protect against data leakage across project phases;
- establish causal effects;
- document data lineage, permissions, or revision history; or

- monitor performance when construction conditions change.

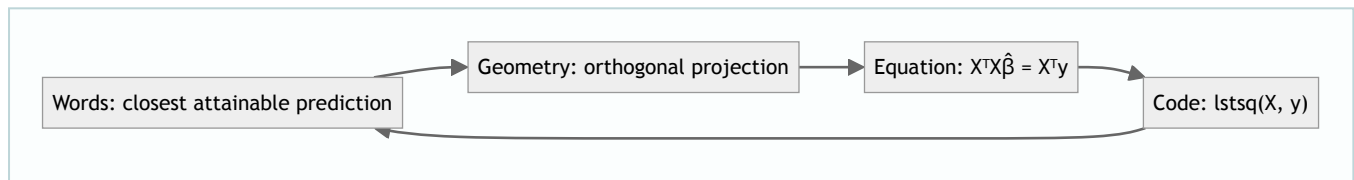
Recognising these limits is part of mastery.

## 5.13 Day 5 practice

1. Derive the through-origin slope without looking at the text.
2. Explain, by coordinates, why the derivative of  $\beta^T A \beta$  is  $(A + A^T)\beta$ .
3. Construct a full-rank matrix with four rows and three columns. Confirm its rank in NumPy.
4. Make one column an exact combination of the other columns. Confirm the rank loss.
5. Change one target in the end-to-end dataset by 20 million PKR. Compare coefficient changes, RMSE, and MAE.
6. Fit the same data after converting road distance from kilometres to metres. Verify that predictions are unchanged and explain the coefficient change.
7. Write a one-paragraph briefing for a senior manager that reports a prediction, its unit, its intended use, and two limitations without using mathematical jargon.

## Day 5 exit check

You are ready to leave Week 1 when you can move in both directions:



# Week 1 Capstone — Build It, Break It, Rebuild It

## Capstone brief

You are given 40 synthetic completed MHP projects and asked to create an early-stage cost estimator. Complete the work in four passes.

### Pass 1: define

Write the `AnalysisContract`. State:

- the unit of observation;
- the target and price basis;
- the point in the project cycle when prediction occurs;
- the decision supported; and
- whether the work is prediction, explanation, or causation.

### Pass 2: build

Use at least three features known at the stated prediction time. Fit the model, report coefficients with units, and calculate training metrics.

## Pass 3: break

Create and explain each failure:

1. a row-alignment error;
2. a missing or infinite value;
3. a redundant feature;
4. a feature recorded after construction begins; and
5. one extreme target value.

## Pass 4: rebuild

Correct the failures. Add a held-out set of projects. Compare training and held-out RMSE and MAE. Write a recommendation that separates what the model can support from what it cannot.

## Assessment rubric

| Dimension     | Emerging         | Competent                     | Mastery                                                      |
|---------------|------------------|-------------------------------|--------------------------------------------------------------|
| Question      | Purpose is vague | Purpose and target are stated | Estimand, timing, and intended use are precise               |
| Mathematics   | Formula copied   | Main steps reproduced         | Derivation explained and verified independently              |
| Code          | Runs once        | Validates common failures     | Tests equations, shapes, rank, and interpretation boundaries |
| Geometry      | Terms repeated   | Projection described          | Column space, orthogonality, and minimum distance connected  |
| Context       | Generic example  | KP units and features used    | Institutional consequences and data limitations addressed    |
| Communication | Numbers listed   | Metrics interpreted           | Decision, uncertainty, and limits clearly separated          |

## Formula Sheet

| Concept           | Equation                                                                 | Meaning                                       |
|-------------------|--------------------------------------------------------------------------|-----------------------------------------------|
| Linear prediction | $\hat{y} = X\beta$                                                       | Weighted combination of design columns        |
| Residual          | $e = y - \hat{y}$                                                        | Actual minus predicted                        |
| SSR               | $e^T e = \sum_i e_i^2$                                                   | Total squared training error                  |
| MSE               | $SSR/n$                                                                  | Mean squared error                            |
| RMSE              | $\sqrt{SSR/n}$                                                           | Squared-error summary in target units         |
| MAE               | $\frac{1}{n} \sum_i  e_i $                                               | $e_i$                                         |
| Orthogonality     | $X^T e = 0$                                                              | Residual perpendicular to every design column |
| Normal equations  | $X^T X \hat{\beta} = X^T y$                                              | First-order condition for OLS                 |
| Closed form       | $(X^T X)^{-1} X^T y$                                                     | Unique solution when $X$ has full column rank |
| Scalar slope      | $\frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$ | One-feature slope with intercept              |

| Concept          | Equation                          | Meaning                               |
|------------------|-----------------------------------|---------------------------------------|
| Scalar intercept | $\bar{y} - \hat{\beta}_1 \bar{x}$ | Makes the line pass through the means |
| Hessian          | $2X^T X$                          | Curvature of the SSR surface          |

# Glossary

**Association:** A numerical relationship in observed data; it need not be causal.

**Column space:** All vectors that can be formed as weighted combinations of a matrix's columns.

**Confounder:** A common cause that can distort an exposure–outcome association.

**Counterfactual:** The outcome that would have occurred under an alternative action or condition.

**Design matrix:** The rectangular matrix of features used to construct model predictions.

**Estimator:** A rule or algorithm that converts data into parameter estimates.

**Feature:** An input variable used by a model.

**Full column rank:** A condition in which no design column is an exact linear combination of the others.

**Gradient:** A vector of partial derivatives; it points in the direction of steepest local increase.

**Hessian:** A matrix of second derivatives describing local curvature.

**Hyperparameter:** A modeling setting chosen rather than estimated by the fitting calculation.

**Intercept:** The constant term added to every linear prediction.

**Loss function:** A numerical rule that penalises prediction error during fitting.

**Multicollinearity:** Strong linear dependence among features; perfect multicollinearity causes rank deficiency.

**Observation:** One unit represented by one row of the dataset.

**OLS:** Ordinary least squares, the estimator that minimises the sum of squared residuals.

**Parameter:** A numerical value learned from data by an estimator.

**Projection:** The closest point in a subspace to a given vector under Euclidean distance.

**Rank:** The number of linearly independent directions represented by a matrix.

**Residual:** The observed target minus its fitted prediction.

**Target:** The outcome the model estimates.

**Target leakage:** Use of information that would not legitimately be available at prediction time.



# Instructor and Self-Study Notes

---

## Suggested daily timebox

| Activity                            | Minutes    |
|-------------------------------------|------------|
| Orient and retrieve prior knowledge | 15         |
| Work through the central derivation | 35         |
| Run and alter the proof code        | 35         |
| Create or inspect the figure        | 20         |
| Extend the application              | 35         |
| Break-and-repair exercise           | 25         |
| Exit check and written reflection   | 15         |
| <b>Total</b>                        | <b>180</b> |

Three hours is a guide, not a race. If a learner cannot explain yesterday's exit check, begin there rather than adding more notation.

## Retrieval prompts for the next morning

- After Day 1: "Name the three regression jobs and one risk of each."
- After Day 2: "Draw the shapes of  $\mathbf{X}$ ,  $\mathbf{y}$ , and  $\beta$ ."
- After Day 3: "Expand one row of  $\mathbf{X}\beta$  and explain the ones column."
- After Day 4: "Why is the OLS residual perpendicular to  $\mathbf{X}$ ?"
- After Day 5: "State the normal equations and the condition for a unique coefficient vector."

## Guidance on the KP context

Context should discipline the mathematics, not decorate it. Ask repeatedly:

- At what project stage is this field known?
- Who recorded it and for what administrative purpose?
- Does one row mean the same thing across districts and implementing partners?
- Are costs expressed in the same price year?
- Does a terrain index of 4 mean the same thing to every assessor?
- Could the model penalise remote projects for structural conditions outside field teams' control?
- Would an apparent "efficiency" relationship survive a fair comparison of project types?

These questions connect modeling capability to institutional responsibility.

---

# Where Week 2 Begins

---

Week 1 ends with a correct small OLS implementation and an honest account of its limits. Week 2 should extend the same application in this order:

1. numerical conditioning and feature scaling;
2. QR and singular-value views of least squares;
3. probability models and maximum likelihood;
4. gradient descent built from the OLS gradient;
5. train/validation/test separation;
6. coefficient and prediction uncertainty; and
7. diagnostic plots and model revision.

The next week should not treat these as disconnected techniques. Each one answers a problem exposed by the model built here.

---

## Final reflection

The intellectual achievement of Week 1 is not the formula  $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ . It is the chain of reasoning around it:

A precise infrastructure question



A defensible data representation



A transparent mathematical model



Independent code and geometric verification



A bounded, responsible judgement

You now know enough to be dangerous if you report coefficients carelessly—and enough to begin becoming useful if you preserve the chain from question to judgement.

# Chapter 1 Companion Guide: From a Question to Ordinary Least Squares

---

## A beginner-first guide to the Python, mathematics, statistics, and judgement behind Chapter 1

This guide accompanies `chapter1.md`. It assumes that you have never programmed in Python, have not studied statistics, and may be uncomfortable with algebra. Nothing important is treated as “obvious.”

The chapter's running example is a cost estimator for fictional microhydro power (MHP) projects in Khyber Pakhtunkhwa (KP). The goal is not merely to produce a number. It is to understand the complete chain:

1. ask a precise question;
2. represent the data correctly;
3. form linear predictions;
4. measure prediction errors;
5. find the ordinary least-squares solution;
6. check when that solution is meaningful; and
7. communicate what the result can and cannot support.

You do not need to memorise every formula on the first reading. Aim to translate each formula into a sentence and each line of code into a small operation you understand.

---

## How to use this guide

For each day in Chapter 1:

1. Read the corresponding section here first.
2. Type the small examples yourself. Copying and pasting hides useful mistakes.
3. Read the main chapter and run its longer code blocks.
4. Change one number and predict what will happen before rerunning the code.
5. Complete the “Check your understanding” questions without looking back.
6. Keep a notebook containing new words, errors you encountered, and how you fixed them.

When an equation feels difficult, use this three-part translation:

- **objects:** What are the numbers, vectors, or matrices?
  - **operation:** What arithmetic is performed?
  - **meaning:** What does the result say about an MHP project?
-

# 0. Foundations you need before Day 1

## 0.1 The running case and its units

One **observation** is one completed MHP project. Possible input fields include:

| Field               | Symbol | Unit        | Plain-language meaning                               |
|---------------------|--------|-------------|------------------------------------------------------|
| Number of projects  | $n$    | projects    | Number of rows in the dataset                        |
| Number of features  | $p$    | features    | Number of input columns before an intercept is added |
| Cable length        | $x_1$  | km          | Length of distribution cable                         |
| Hydraulic head      | $x_2$  | m           | Vertical fall used to drive the turbine              |
| Road distance       | $x_3$  | km          | Distance from an all-weather road                    |
| Terrain difficulty  | $x_4$  | index 1–5   | An ordered assessment of access difficulty           |
| Planned capacity    | $x_5$  | kW          | Intended electricity-generating capacity             |
| Actual project cost | $y$    | million PKR | Outcome that the model will estimate                 |

Units are part of the meaning. If cost is measured in million PKR and road distance in km, a road-distance coefficient is measured in **million PKR per km**. A coefficient without units is incomplete.

The chapter's projects are synthetic. They are suitable for learning but not for evaluating a real district, contractor, organisation, or community.

## 0.2 Installing and running Python

Python executes instructions from top to bottom. Chapter 1 uses Python 3.11 or later, NumPy for numerical arrays, and Matplotlib for plots.

In a terminal:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install numpy matplotlib
```

On Windows PowerShell, activation is usually:

```
.venv\Scripts\Activate.ps1
```

The virtual environment keeps this project's packages separate from other Python projects. You can run code in a `.py` file, a Jupyter notebook, or an editor's interactive Python window.

## 0.3 A small Python survival kit

### Values, names, and arithmetic

```
project_name = "MHP-CH-003" # text, called a string
road_distance = 18.5 # decimal number, called a float
terrain_index = 4 # whole number, called an integer
is_complete = True # Boolean: True or False
```

```
base_cost = 8.0
cost_per_km = 4.0
prediction = base_cost + cost_per_km * road_distance
print(prediction)
```

= assigns a value to a name. It does not mean “is mathematically equal forever.” # begins a comment that Python ignores. Common arithmetic operators are +, -, \*, /, and \*\* for powers.

## Lists and NumPy arrays

```
import numpy as np

cost_list = [19.0, 23.0, 30.0]
cost_array = np.array(cost_list, dtype=float)

print(cost_array[0]) # first value; Python counts from zero
print(cost_array.shape) # (3,)
print(cost_array.mean()) # arithmetic mean
```

A Python list is general-purpose. A NumPy array is designed for numerical work and has a fixed shape. dtype=float asks NumPy to store floating-point numbers.

## Functions

```
def predict_cost(distance_km, base=8.0, rate=4.0):
 """Return a simple cost prediction in million PKR."""
 return base + rate * distance_km

estimate = predict_cost(3.0)
```

def creates a reusable function. Indentation defines the function body. Inputs are called arguments or parameters in Python terminology; do not confuse these with fitted statistical parameters. return sends a result back to the caller.

## Conditions and errors

```
def require_positive(value):
 if value <= 0:
 raise ValueError("value must be positive")
 return value
```

An exception stops an invalid calculation near its source. A clear early error is better than a plausible but wrong result later.

## Loops

```
total = 0.0
for value in [2.0, 3.0, 4.0]:
 total += value
print(total) # 9.0
```

The loop visits each item. total += value is shorthand for total = total + value.

## Classes, objects, and dataclasses

The chapter builds an MHPCostEstimator class. A **class** is a blueprint; an **object** is one created instance. self means “this particular object.” A method is a function attached to a class.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class AnalysisContract:
 purpose: str
 target_name: str

contract = AnalysisContract(
 purpose="prediction",
 target_name="actual_cost_m_pkr",
)

```

The type annotations describe expected values but do not by themselves validate everything. `frozen=True` prevents accidental changes after creation. Names ending in `_`, such as `parameters_`, conventionally mean values learned during fitting.

## Assertions and floating-point comparisons

```

result = 0.1 + 0.2
assert np.isclose(result, 0.3)

```

An `assert` is an executable claim. Decimal arithmetic on a computer has tiny rounding errors, so use `np.isclose` or `np.allclose` for calculated floating-point results instead of exact `==`.

## 0.4 The mathematical language used in the chapter

### Symbols and subscripts

- $x$  is usually an input feature.
- $y$  is the observed target.
- $\hat{y}$ , read “y-hat,” is a prediction.
- $\beta$ , read “beta,” is a parameter or vector of parameters.
- $i$  identifies a row or project.
- $j$  identifies a feature column.
- $x_{ij}$  means feature  $j$  for project  $i$ .

A hat means “estimated from data.” Thus  $\hat{\beta}$  is the fitted parameter value, not the unknown population value  $\beta$ .

### Sums, means, powers, roots, and absolute values

The symbol  $\sum$  means “add a sequence of values.” For example:

$$\sum_{i=1}^3 x_i = x_1 + x_2 + x_3.$$

The arithmetic mean is:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

$x^2$  means  $x$  multiplied by itself.  $\sqrt{x}$  is the nonnegative square root.  $|x|$  is absolute value, the distance from zero:  $|-3| = 3$ .

## Scalars, vectors, matrices, and tensors

- A **scalar** is one number, such as one project's cost.
- A **vector** is an ordered one-dimensional collection, such as all project costs.
- A **matrix** is a rectangular two-dimensional collection, such as projects by features.
- A **tensor** is the general term for an array with any number of axes.

Do not confuse the number of tensor axes with **matrix rank**, which counts independent directions and is introduced on Day 5.

## Dot products and matrix multiplication

For vectors  $\mathbf{a} = [a_1, a_2]$  and  $\mathbf{b} = [b_1, b_2]$ :

$$\mathbf{a}^T \mathbf{b} = a_1 b_1 + a_2 b_2.$$

NumPy writes this as `a @ b`. A matrix-vector product performs one dot product for every matrix row.

## Transpose and length

$\mathbf{X}^T$  is the transpose of  $\mathbf{X}$ : rows become columns. In NumPy it is `X.T`.

The Euclidean length of a vector is:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

NumPy computes it with `np.linalg.norm(v)`.

## Functions, derivatives, gradients, and Hessians

A mathematical function maps an input to an output. A derivative measures the local rate at which an output changes when one input changes. At the bottom of a smooth bowl, the slope is zero.

When a function has several inputs, a **partial derivative** changes one input while holding the others fixed. A **gradient** collects all partial derivatives into a vector. A **Hessian** collects all second partial derivatives into a matrix and describes curvature.

This intuition is enough to begin Day 5; the full derivation is developed there.

---

# Day 1 — Decide what question regression should answer

---

## 1.1 Regression is a mechanism, not a purpose

Regression constructs a numerical relationship between a target  $\mathbf{y}$  and observed features  $\mathbf{X}$ . The same fitted equation can be used for three different jobs:

| Job        | Core question                                    | Evidence of success                 | Main danger                                           |
|------------|--------------------------------------------------|-------------------------------------|-------------------------------------------------------|
| Prediction | What outcome should we expect for a new project? | Low error on genuinely new projects | Memorising old data or failing when conditions change |



| Job              | Core question                                                              | Evidence of success                                    | Main danger                                            |
|------------------|----------------------------------------------------------------------------|--------------------------------------------------------|--------------------------------------------------------|
| Explanation      | How is one feature associated with cost, conditional on included features? | Stable, interpretable estimates with uncertainty       | Omitted or correlated variables distort interpretation |
| Causal inference | What would change if we intervened?                                        | A credible comparison with the relevant counterfactual | Treating association as the effect of an action        |

The calculation does not decide which question you meant. The purpose determines what data, design, validation, and wording are defensible.

## 1.2 Prediction

A prediction rule can be written as:

$$\hat{y}_i = f(x_{i1}, x_{i2}, \dots, x_{ip}).$$

For project  $i$ , the function  $f$  converts its feature values into an estimated cost  $\hat{y}_i$ .

A useful predictor does not need every feature to be causal. River crossings may help predict transport difficulty even if “removing one crossing” is not a meaningful intervention. Prediction should be judged on projects that were not used to fit the model. Training accuracy alone may reward memorisation.

## 1.3 Explanation and conditional association

Consider:

$$\hat{y}_i = \beta_0 + \beta_1 \text{road distance}_i + \beta_2 \text{capacity}_i.$$

$\beta_1$  describes the fitted cost difference associated with one additional kilometre of road distance **while the included capacity value is held fixed**. This is a conditional association.

“Holding included features fixed” does not control for omitted features, measurement errors, or structural differences between districts. Explanatory analysis also needs uncertainty. The same numerical coefficient is less persuasive when estimated from 6 projects than from 600 comparable projects.

## 1.4 Causation, interventions, and counterfactuals

A causal question compares outcomes under different actions. Suppose the action is completing an access road before construction. For one project, the causal effect would compare:

$$Y_i(\text{road completed}) - Y_i(\text{road not completed}).$$

Only one of these outcomes can usually be observed for the same project. The unobserved alternative is the **counterfactual**. Causal inference needs a study design and assumptions that make another comparison stand in for this missing outcome.

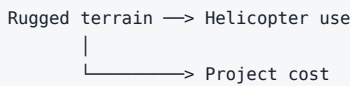
A regression coefficient is therefore not automatically a causal effect. A causal statement must define:

- the intervention;
- the affected population;
- the outcome and time horizon;
- the counterfactual alternative; and
- why the comparison identifies that effect.

## 1.5 Confounding, mediators, and colliders

A **confounder** is a common cause of both an exposure or decision and an outcome. Rugged terrain may make helicopter transport more likely and independently make a project more expensive. Ignoring terrain can make helicopters appear responsible for high cost even if they reduce cost relative to weeks of manual carriage.

In a causal diagram:



Not every available variable should be controlled:

- A **mediator** lies on the pathway from action to outcome. Controlling it may remove part of the effect being studied.
- A **collider** is caused by two other variables. Conditioning on it can create a misleading association.

The beginner-safe rule is not “control for everything.” It is “state the causal story and justify the adjustment set.”

## 1.6 Simpson's paradox

Simpson's paradox occurs when an aggregated trend differs from, or reverses, trends within meaningful groups.

| Terrain    | Cable km | Cost, million PKR |
|------------|----------|-------------------|
| Accessible | 4        | 18                |
| Accessible | 5        | 20                |
| Accessible | 6        | 22                |
| Accessible | 7        | 24                |
| Remote     | 1        | 30                |
| Remote     | 2        | 33                |
| Remote     | 3        | 36                |
| Remote     | 4        | 39                |

Within accessible projects, cost rises by about 2 million PKR per km. Within remote projects, it rises by about 3 million PKR per km. Yet the combined slope is about -2 because costly remote projects mostly have shorter cables.

For a line with one feature and an intercept:

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}.$$

The numerator measures whether  $x$  and  $y$  move together. The denominator measures variation in  $x$ . The numerical reversal does not by itself identify a causal relationship; subject-matter knowledge explains why aggregation is misleading.

## 1.7 Write an estimand before code

An **estimand** is the precise quantity an analysis intends to estimate.

- Prediction: “The expected final cost, in million PKR at the stated price basis, of an approved project using design-stage information.”
- Explanation: “The adjusted difference in final cost associated with one additional kilometre from an all-weather road, conditional on planned capacity and measured terrain.”
- Causation: “The average change in final cost caused by completing an access road before civil works among approved remote projects.”

Also state the unit of observation, intended use, prediction time, target units, and decision-maker. If these are vague, a more complicated model will not fix the question.

## 1.8 The AnalysisContract

The chapter begins the application with an immutable dataclass:

```
from dataclasses import dataclass
from typing import Literal

AnalysisPurpose = Literal["prediction", "explanation", "causation"]

@dataclass(frozen=True)
class AnalysisContract:
 purpose: AnalysisPurpose
 target_name: str
 target_unit: str
 unit_of_observation: str
 intended_use: str

 def __post_init__(self):
 if self.purpose not in {"prediction", "explanation", "causation"}:
 raise ValueError("invalid purpose")
 for name, value in vars(self).items():
 if not str(value).strip():
 raise ValueError(f"{name} cannot be empty")
```

`Literal` documents the allowed choices. `__post_init__` runs after creation and validates them. `vars(self)` provides the object's fields as name-value pairs. `.strip()` removes surrounding whitespace, so a string containing only spaces is rejected.

Deliberately try `purpose="forecasting"` and an empty intended use. Predict the errors first. This demonstrates a general engineering principle: make invalid states fail clearly and early.

## Day 1 check

You should now be able to answer:

1. Why can a feature help prediction without being causal?
2. What does “holding the other included features fixed” mean?
3. What is the missing counterfactual in an access-road study?
4. How can rugged terrain confound a transport-cost comparison?
5. Why can an aggregated slope have the opposite sign from group-specific slopes?

Key idea: a coefficient is a causal effect only when the design and assumptions make the comparison equivalent to the stated intervention and counterfactual.

---

# Day 2 — Give data the correct roles, shapes, units, and provenance

## 2.1 Regression vocabulary

| Term                         | Meaning                                    | MHP example                          |
|------------------------------|--------------------------------------------|--------------------------------------|
| Observation                  | One unit represented by one row            | One completed project                |
| Feature                      | An input used to form a prediction         | Planned capacity                     |
| Target                       | The outcome to estimate                    | Actual project cost                  |
| Parameter                    | A number learned during fitting            | Road-distance coefficient            |
| Hyperparameter/design choice | A setting chosen outside fitting           | Whether to include an intercept      |
| Prediction                   | Model output for one observation           | 42 million PKR                       |
| Residual                     | Actual target minus prediction             | $45 - 42 = 3$ million PKR            |
| Loss                         | Rule used to penalise error during fitting | Squared residual                     |
| Metric                       | Summary used to report performance         | RMSE                                 |
| Estimator                    | Procedure that learns parameters           | OLS                                  |
| Fitted model                 | Estimator plus learned values              | OLS plus fitted intercept and slopes |

The distinction between a parameter and hyperparameter is procedural: fitting learns a parameter from data; the analyst chooses a hyperparameter or modeling design.

## 2.2 Arrays and shape notation

The central objects are:

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} \in \mathbb{R}^n, \quad \boldsymbol{\beta} \in \mathbb{R}^p.$$

$\mathbb{R}$  means real-valued numbers.  $\mathbf{X}$  has  $n$  project rows and  $p$  feature columns.  $\mathbf{y}$  has one target per row.  $\boldsymbol{\beta}$  has one weight per feature. After adding an intercept,  $\mathbf{X}$  and  $\boldsymbol{\beta}$  each gain one column or entry.

For three projects and two features:

$$\mathbf{X} = \begin{bmatrix} 12 & 100 \\ 25 & 150 \\ 8 & 80 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 28 \\ 44 \\ 22 \end{bmatrix}.$$

Row 2 means that the second project has road distance 25 km, planned capacity 150 kW, and actual cost 44 million PKR.

## 2.3 NumPy shapes are not decoration

```
a = np.array([28.0, 44.0, 22.0]) # (3,)
b = np.array([[28.0], [44.0], [22.0]]) # (3, 1)
```

```
c = np.array([[28.0, 44.0, 22.0]]) # (1, 3)
```

- `(3,)` is a one-dimensional array.
- `(3, 1)` is a two-dimensional column matrix.
- `(1, 3)` is a two-dimensional row matrix.

They contain the same numbers but behave differently during multiplication and broadcasting. In this application, `y` uses shape `(n,)` and `X` always becomes 2D.

Useful shape tools:

```
x = np.array([1.0, 2.0, 3.0])
X_one_feature = x.reshape(-1, 1) # infer rows, make one column
X_two_features = np.column_stack([x, x ** 2])
flat_again = X_one_feature.reshape(-1)
```

## 2.4 The design matrix and row alignment

The feature matrix supplied to regression is the **design matrix**. The word “design” means the arrangement of predictor columns; it does not imply an experiment.

The *i*th row of **X** and the *i*th value of **y** must describe the same project. Equal row counts are necessary but insufficient. If **X** is sorted by district and **y** by project ID, the program can run successfully while learning nonsense.

Preserve `project_id` while assembling and checking data, even if it is not used as a numerical feature. Join or sort **X** and **y** together, then assert identifiers align.

## 2.5 A data dictionary belongs to the model

A data dictionary should record:

- field name and plain-language definition;
- data type;
- unit or allowed categories;
- when the value becomes available;
- who measured it and how;
- missing-value rule;
- valid range; and
- price year or price basis for monetary targets.

This information determines whether two rows mean the same thing and whether a feature is legitimate at prediction time.

### Target leakage

**Target leakage** occurs when training uses information that would not legitimately be available when a real prediction is made. “Number of contract amendments” may predict final cost well but is unavailable during initial budgeting and may be caused by the overrun itself.

Leakage creates impressive-looking training or test results if the split does not reproduce the real timeline. Prevent it by stating the prediction time first, then checking every feature's availability and provenance.

## 2.6 Parsing nested JSON

JSON represents nested records with objects and lists. Python's `json.loads` converts JSON text into dictionaries.

```
import json

record = json.loads(raw_json)
distance = record["design"]["road_distance_km"]

X_one = np.array([[
 record["design"]["road_distance_km"],
 record["design"]["planned_capacity_kw"],
 record["design"]["terrain_index"],
]], dtype=float)

y_one = np.array([
 record["completion"]["actual_cost_m_pkr"]
], dtype=float)

assert X_one.shape == (1, 3)
assert y_one.shape == (1,)
```

The double opening bracket means one row containing three columns. Assertions make the intended mathematical shapes executable.

## 2.7 Matrix-vector multiplication is repeated weighted addition

Without an intercept:

$$\hat{y}_i = \sum_{j=1}^p x_{ij}\beta_j.$$

For each project, multiply every feature by its matching coefficient, then add the products.

```
y_hat_loop = np.zeros(X.shape[0])
for i in range(X.shape[0]):
 total = 0.0
 for j in range(X.shape[1]):
 total += X[i, j] * beta[j]
 y_hat_loop[i] = total

y_hat_matrix = X @ beta
assert np.allclose(y_hat_loop, y_hat_matrix)
```

`X.shape[0]` is the number of rows and `X.shape[1]` the number of columns. The compact `@` expression performs the same nested-loop calculation.

## 2.8 Input validation, line by line

The chapter's `_validate_inputs` method:

1. converts input to floating-point NumPy arrays;
2. turns 1D training `X` into one feature column;
3. rejects arrays that are not 2D;
4. rejects empty rows or columns;
5. rejects `NaN` and infinity;
6. flattens a one-column target to `(n,)`;

7. rejects other target shapes; and
8. checks equal row counts.

Important expressions:

```
X = np.asarray(X, dtype=float)
X = X.reshape(-1, 1)
np.isfinite(X).all()
```

`np.asarray` accepts lists or arrays. `-1` asks NumPy to infer the needed dimension. `np.isfinite` is false for NaN, positive infinity, and negative infinity; `.all()` requires every entry to pass.

Validation cannot detect correctly shaped but misaligned projects. That requires identifiers and data-provenance checks before numerical arrays discard labels.

## Day 2 failure laboratory

Predict the result of each case:

- 10 feature rows and 8 targets: row mismatch;
- a feature containing `np.nan`: non-finite input;
- shape `(0, 2)`: no observations;
- shape `(2, 2, 2)`: too many dimensions;
- equal-length `X` and reversed `y`: no shape error, but corrupted meaning.

For 70 projects and 5 raw features, `X.shape == (70, 5)` and `y.shape == (70,)`. After adding an intercept, the design matrix is `(70, 6)` and the parameter vector is `(6,)`.

---

## Day 3 — Build linear predictions

### 3.1 From words to an equation

“Start with 8 million PKR and add 4 million PKR per kilometre of cable” becomes:

$$\hat{y}_i = 8 + 4x_i.$$

The general one-feature model is:

$$\hat{y}_i = \beta_0 + \beta_1 x_i.$$

- $\beta_0$  is the **intercept**, the prediction when  $x = 0$ .
- $\beta_1$  is the **slope**, the predicted change for one unit more of  $x$ .

If cost is million PKR and cable is km, the slope is million PKR per km. The intercept can improve predictions without describing a physically possible zero-feature project. It is an algebraic baseline, and interpreting it literally may require extrapolating beyond observed data.

### 3.2 Several features are still one weighted sum

$$\hat{y}_i = \beta_0 + \beta_1 \text{road distance}_i + \beta_2 \text{capacity}_i + \beta_3 \text{terrain}_i.$$

For one project, this is a dot product. For all projects simultaneously:

$$\hat{y} = X\beta.$$

“Linear” means linear in the parameters: predictions are sums of parameter-weighted columns. A model may still include transformed features such as  $x^2$ , provided their coefficients enter linearly.

### 3.3 Why a column of ones creates an intercept

For three projects and one feature:

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}.$$

Multiplication gives:

$$X\beta = \begin{bmatrix} \beta_0 + x_1\beta_1 \\ \beta_0 + x_2\beta_1 \\ \beta_0 + x_3\beta_1 \end{bmatrix}.$$

Every prediction receives the same  $\beta_0$  because every first-column value is 1. Without that column, the fitted line is forced through  $(0, 0)$ . With it, the model is technically affine—a linear combination plus translation—but is conventionally called linear regression.

In NumPy:

```
X = np.column_stack([np.ones(x.size), x])
y_hat = X @ beta
```

A column of twos would estimate  $2\beta_0$  as the common offset, so its coefficient would be half the conventional intercept. Ones give the coefficient itself the natural intercept meaning.

### 3.4 Shape algebra

If an intercept has been added:

$$X \in \mathbb{R}^{n \times (p+1)}, \quad \beta \in \mathbb{R}^{p+1}, \quad X\beta \in \mathbb{R}^n.$$

The inner dimensions must match:

$$(n \times (p+1))(p+1 \times 1) = n \times 1.$$

Always check shapes before values. If `X.shape == (40, 6)` and `beta.shape == (5,)`, multiplication is impossible because six design columns need six parameters.

### Mathematics-to-NumPy map

| Mathematics | Meaning               | NumPy                 |
|-------------|-----------------------|-----------------------|
| $X\beta$    | Matrix-vector product | <code>X @ beta</code> |
| $X^T$       | Transpose             | <code>X.T</code>      |
| $x^T y$     | Dot product           | <code>x @ y</code>    |



| Mathematics  | Meaning          | NumPy                          |
|--------------|------------------|--------------------------------|
| $I$          | Identity matrix  | <code>np.eye(p)</code>         |
| $\mathbf{1}$ | Ones vector      | <code>np.ones(n)</code>        |
| $\ v\ _2$    | Euclidean length | <code>np.linalg.norm(v)</code> |

### 3.5 Two views of matrix multiplication

**Row view:** each row of  $X$  is one project, and its dot product with  $\beta$  makes one prediction.

**Column view:** if columns are  $x_0, x_1, \dots, x_p$ , then:

$$X\beta = \beta_0 x_0 + \beta_1 x_1 + \dots + \beta_p x_p.$$

The columns are building blocks; the coefficients decide how much of each to combine. All possible combinations form the **column space** of  $X$ . This column view prepares the geometry of Day 4.

Matrices can also represent rotations, stretches, reflections, and shears. That transformation viewpoint is useful, but in regression the immediate meaning of  $X\beta$  is a weighted construction of a prediction vector.

### 3.6 With and without an intercept

For a one-feature fit through the origin:

$$\hat{\beta} = \frac{x^T y}{x^T x}.$$

This model must predict zero when  $x = 0$ . A model with an intercept can translate vertically and will often fit a nonzero baseline better. Plotting both lines reveals whether forcing the origin creates systematic errors.

The chapter uses:

```
beta = np.linalg.solve(X.T @ X, X.T @ y)
```

This solves the linear system directly. Day 5 explains why application code should avoid `np.linalg.inv(X.T @ X) @ X.T @ y`.

### 3.7 Scaling and unit conversions

If kilometres become metres:

$$x_m = 1000x_{km}.$$

To preserve predictions:

$$\beta_m = \frac{\beta_{km}}{1000}.$$

Then  $\beta_m x_m = \beta_{km} x_{km}$ . Changing feature units changes the coefficient inversely, not the prediction. Changing the target from million PKR to PKR multiplies the intercept, slopes, residuals, RMSE, and MAE by one million; MSE and SSR multiply by one million squared.

Coefficient magnitude alone does not measure importance when features have different units or scales.

## 3.8 Prediction code and shape ambiguity

The chapter's `predict` method must:

1. reject prediction before fitting;
2. convert a single new project from `(p,)` to `(1, p)`;
3. validate finite values and a 2D shape;
4. require the same number and order of features used in fitting;
5. add the intercept; and
6. return `X_design @ parameters_`.

There is a context-dependent ambiguity:

- during fitting, a 1D `X` means many observations of one feature;
- during prediction, a 1D `X` means one observation with many features.

Explicit 2D input is safest: `[[20.0, 170.0, 4.0]]` clearly means one project with three features.

## 3.9 Broadcasting is not matrix multiplication

```
correct = X @ beta
elementwise = X * beta
```

`@` returns one dot product per row, so its result is a prediction vector. `*` performs elementwise multiplication. NumPy may **broadcast** `beta` across every row, returning a matrix of unadded products. It can run without an error while answering the wrong question.

This is a crucial debugging lesson: code that runs is not necessarily mathematically correct.

## Day 3 check

Explain both statements in your own words:

- `X @ beta` is one dot product per project.
- `X @ beta` is a weighted combination of the columns of `X`.

Then state the shapes of the raw features, design matrix, coefficient vector, and predictions for 40 projects and 5 features.

---

# Day 4 — Residuals, loss, metrics, and OLS geometry

---

## 4.1 Residuals

For project  $i$ :

$$e_i = y_i - \hat{y}_i.$$

- $e_i > 0$ : the actual cost is higher; the model underpredicted.

- $e_i < 0$ : the actual cost is lower; the model overpredicted.
- $e_i = 0$ : prediction equals the observed target.

For all projects:

$$e = y - \hat{y} = y - X\beta.$$

This guide follows **actual minus predicted**. Some software uses the opposite sign, so always check the convention.

A residual is calculated after fitting from observed data. It is not exactly the same as an unobservable population error term, even though introductory writing sometimes uses the words interchangeably.

## 4.2 Why OLS squares residuals

The **sum of squared residuals** (SSR), also called RSS or SSE, is:

$$SSR(\beta) = \sum_{i=1}^n (y_i - x_i^T \beta)^2 = (y - X\beta)^T (y - X\beta) = e^T e.$$

Squaring:

1. stops positive and negative residuals cancelling;
2. penalises large misses much more heavily;
3. produces a smooth differentiable objective; and
4. turns fitting into a squared Euclidean-distance problem.

These are properties, not proof that squared loss is always institutionally correct. One data-entry error can dominate the result.

## 4.3 Work the metrics by hand

Suppose:

$$y = [3, 5], \quad \hat{y} = [4, 3].$$

Residuals are  $e = [-1, 2]$ .

$$SSR = (-1)^2 + 2^2 = 5,$$

$$MSE = \frac{5}{2} = 2.5,$$

$$RMSE = \sqrt{2.5} \approx 1.581,$$

$$MAE = \frac{|-1| + |2|}{2} = 1.5.$$

| Metric | Formula      | Units                | What it communicates                       |
|--------|--------------|----------------------|--------------------------------------------|
| SSR    | $\sum e_i^2$ | squared target units | Total squared fitting objective            |
| MSE    | $SSR/n$      | squared target units | Average squared loss                       |
| RMSE   | $\sqrt{MSE}$ | target units         | Squared-loss summary in communicable units |
| MAE    | $\sum e_i$   | e_i                  | /n\$                                       |

RMSE reacts more strongly than MAE to large residuals. Neither is automatically “better.” Select and interpret metrics according to the decision and inspect unusual observations.

Training MSE here divides by  $n$ . An estimate of residual variance may divide by  $n - p - 1$  when an intercept and  $p$  raw features are fitted. These are different quantities; always state the denominator.

## 4.4 OLS searches inside the column space

The target vector  $\mathbf{y}$  has one coordinate per project, so it lies in  $\mathbb{R}^n$ . Every possible prediction  $\mathbf{X}\boldsymbol{\beta}$  is a weighted combination of design columns and therefore lies in the column space of  $\mathbf{X}$ .

OLS cannot choose any vector in  $\mathbb{R}^n$ . It chooses the attainable vector  $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$  closest to  $\mathbf{y}$  under squared Euclidean distance. This is an **orthogonal projection**.

The geometric picture is:

- column space: a line, plane, or higher-dimensional flat surface of attainable predictions;
- $\mathbf{y}$ : the observed target vector, usually outside that surface;
- $\hat{\mathbf{y}}$ : the closest point on the surface;
- $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ : the perpendicular arrow from prediction to observation.

## 4.5 The Pythagorean minimum proof

Let  $\mathbf{q}$  be any other attainable prediction. Define:

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}, \quad \mathbf{d} = \hat{\mathbf{y}} - \mathbf{q}.$$

Then:

$$\mathbf{y} - \mathbf{q} = \mathbf{e} + \mathbf{d}.$$

$\mathbf{e}$  is perpendicular to the column space, while  $\mathbf{d}$  lies inside it, so  $\mathbf{e}^T \mathbf{d} = 0$ . Expanding:

$$\begin{aligned} \|\mathbf{y} - \mathbf{q}\|_2^2 &= (\mathbf{e} + \mathbf{d})^T (\mathbf{e} + \mathbf{d}) \\ &= \mathbf{e}^T \mathbf{e} + \mathbf{e}^T \mathbf{d} + \mathbf{d}^T \mathbf{e} + \mathbf{d}^T \mathbf{d} \\ &= \|\mathbf{e}\|_2^2 + \|\mathbf{d}\|_2^2 \\ &\geq \|\mathbf{e}\|_2^2. \end{aligned}$$

The other prediction's squared distance equals the OLS distance plus another nonnegative squared length. Therefore it cannot be smaller.

## 4.6 Orthogonality and the normal equations

Perpendicular to every design column means:

$$\mathbf{X}^T \mathbf{e} = 0.$$

Substitute  $\mathbf{e} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$ :

$$\mathbf{X}^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = 0,$$

so:

$$X^T X \hat{\beta} = X^T y.$$

These are the **normal equations**. “Normal” means perpendicular here; it has nothing to do with the normal probability distribution.

If the design includes a ones column, one orthogonality equation is:

$$\mathbf{1}^T e = \sum_i e_i = 0.$$

Thus training residuals sum to approximately zero when an intercept is included. This does not imply a good fit: residuals of `[-100, 100]` sum to zero but are enormous.

## 4.7 Verify geometry in code

```
beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
y_hat = X @ beta_hat
e = y - y_hat

assert np.allclose(X.T @ e, 0.0, atol=1e-10)
assert np.isclose(e.sum(), 0.0, atol=1e-10)

beta_other = beta_hat + np.array([2.0, -1.0])
q = X @ beta_other
d = y_hat - q

assert np.isclose(e @ d, 0.0, atol=1e-10)
assert np.isclose(
 np.linalg.norm(y - q) ** 2,
 np.linalg.norm(e) ** 2 + np.linalg.norm(d) ** 2,
)
```

`atol` is an absolute tolerance that allows tiny floating-point error. The chapter's three-dimensional plot makes the same relationships visible when there are three observations and two independent design columns.

## 4.8 Metrics in the estimator

`_metrics_from_predictions` should flatten both arrays, require equal shapes, calculate `residuals = y - y_hat`, and return Python floats for SSR, MSE, RMSE, and MAE.

`evaluate(X, y)` validates supplied data, calls `predict`, then computes metrics. The method does not call them “test metrics” because an array's provenance—training, validation, or test—is the caller's responsibility.

## 4.9 Outliers and data investigation

Changing 26 to 260 creates a residual hundreds of times larger than the others. Its square can dominate SSR and RMSE and pull fitted coefficients toward it. MAE changes less because it does not square the miss.

Do not immediately delete an outlier or declare MAE superior. Ask:

- Is it a data-entry error?
- Is it a genuine extreme project?
- Does it belong to a different project type?
- Is the chosen loss aligned with the real consequences of large budget misses?
- How do results change with and without it, and why?

---

# Day 5 — Derive, fit, and stress-test ordinary least squares

---

## 5.1 A one-parameter error bowl

Start with no intercept:

$$\hat{y}_i = \beta x_i.$$

The objective is:

$$S(\beta) = \sum_i (y_i - \beta x_i)^2.$$

Expand the square:

$$(y_i - \beta x_i)^2 = y_i^2 - 2\beta x_i y_i + \beta^2 x_i^2.$$

After summing and differentiating:

$$\frac{dS}{d\beta} = -2 \sum_i x_i y_i + 2\beta \sum_i x_i^2.$$

At the bottom of a smooth bowl the derivative is zero. Solving gives:

$$\hat{\beta} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}.$$

The second derivative is:

$$\frac{d^2 S}{d\beta^2} = 2 \sum_i x_i^2.$$

If at least one  $x_i$  is nonzero, this is positive, proving a unique minimum rather than a maximum. A grid search in the chapter independently checks that the formula lands at the bottom of the error bowl.

## 5.2 One feature with an intercept

Now:

$$S(\beta_0, \beta_1) = \sum_i (y_i - \beta_0 - \beta_1 x_i)^2.$$

There are two unknowns, so set two partial derivatives to zero. The intercept equation gives:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

Therefore the fitted line passes through  $(\bar{x}, \bar{y})$ . Substitution into the slope equation gives:

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}.$$

If all  $x_i$  values are identical, the denominator is zero. The data contain no horizontal variation, so they cannot identify a slope.

The centred numerator resembles covariance and the denominator resembles variance. Different textbook definitions divide these sums by  $n$  or  $n - 1$ , but that common factor cancels in the slope ratio.

## 5.3 Expand the matrix objective

For multiple features:

$$S(\beta) = (y - X\beta)^T(y - X\beta).$$

Because  $(y - X\beta)^T = y^T - \beta^T X^T$ :

$$S(\beta) = y^T y - y^T X\beta - \beta^T X^T y + \beta^T X^T X\beta.$$

The middle expressions are both one-number scalars and are transposes of one another, so they are equal:

$$S(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X\beta.$$

Check shapes as you read:  $y^T y$  is one number,  $\beta^T X^T y$  is one number, and  $\beta^T X^T X\beta$  is one number. An objective function must return one number for each candidate parameter vector.

## 5.4 The matrix gradient

Use three derivative facts:

1.  $y^T y$  contains no  $\beta$ , so its gradient is zero.
2. For constant vector  $c$ ,  $\nabla_{\beta}(\beta^T c) = c$ .
3.  $\nabla_{\beta}(\beta^T A\beta) = (A + A^T)\beta$ .

Here  $A = X^T X$ , which is symmetric because  $(X^T X)^T = X^T X$ . Therefore:

$$\nabla_{\beta} S(\beta) = -2X^T y + 2X^T X\beta.$$

At a minimum the gradient is zero:

$$-2X^T y + 2X^T X\hat{\beta} = 0,$$

which produces the same normal equations found geometrically:

$$X^T X\hat{\beta} = X^T y.$$

### Why the quadratic derivative has two terms

Writing  $\beta^T A\beta = \sum_j \sum_k \beta_j A_{jk} \beta_k$ , a derivative with respect to  $\beta_r$  finds occurrences on the left ( $j = r$ ) and the right ( $k = r$ ). Those contributions form  $A\beta$  and  $A^T \beta$ . Symmetry makes them identical, producing  $2A\beta$ .

### Finite-difference audit

A centred finite difference estimates one gradient component:

$$\frac{\partial S}{\partial \beta_j} \approx \frac{S(\beta + hu_j) - S(\beta - hu_j)}{2h}.$$

$u_j$  has 1 in position  $j$  and 0 elsewhere. Comparing this numerical estimate with `-2 * X.T @ (y - X @ beta)` helps catch derivative mistakes. Very large  $h$  gives approximation error; extremely small  $h$  suffers floating-point cancellation.

## 5.5 Existence, uniqueness, convexity, and rank

If  $X^T X$  is invertible:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

This requires **full column rank**: no design column can be made exactly from the others, and the data contain enough independent directions to estimate every coefficient.

The Hessian is:

$$\nabla_{\beta}^2 \mathcal{S}(\beta) = 2X^T X.$$

For any vector  $z$ :

$$z^T X^T X z = (Xz)^T (Xz) = \|Xz\|_2^2 \geq 0.$$

Thus  $X^T X$  is **positive semidefinite** and SSR is convex: it has no deceptive local minima. With full column rank,  $Xz \neq 0$  for every nonzero  $z$ , so the surface is strictly convex and the parameter minimiser is unique.

Predictions can still have a unique closest projection when coefficients are non-unique. Rank deficiency mainly destroys the unique interpretation of individual coefficients.

## 5.6 Why code should use `lstsq`

Three related implementations are:

```
Useful as mathematical notation, usually avoid in application code
beta_hat = np.linalg.inv(X.T @ X) @ X.T @ y

Solve the normal-equation system without explicit inversion
beta_hat = np.linalg.solve(X.T @ X, X.T @ y)

Preferred here: solve least squares directly
beta_hat, residual_sums, rank, singular_values = np.linalg.lstsq(
 X, y, rcond=None
)
```

Forming  $X^T X$  squares the condition number and can magnify numerical instability. Explicit inversion also does unnecessary work. `lstsq` uses a decomposition designed for least squares and reports rank and singular values. It solves the same objective; it is not replacing the mathematics with magic.

`residual_sums` may be empty in exact-fit or rank-deficient cases, so the estimator computes training metrics explicitly from predictions.

## 5.7 Fitting assumptions are not inference assumptions

Keep three layers separate:

1. **Algebra**: Did the program find a least-squares minimum?
2. **Generalisation**: Will the relationship work on future or wider-population projects?
3. **Causation**: Does the design identify an intervention effect?



Normally distributed residuals are not required merely to calculate OLS. Full column rank is required for a unique coefficient vector. Claims about unbiasedness, confidence intervals, heteroskedasticity, clustering, sampling, or causal effects require additional assumptions and methods.

A perfect training calculation proves only the first layer.

## 5.8 Perfect multicollinearity and rank failure

If both kilometres and metres are columns:

$$x_m = 1000x_{km}.$$

One column contains no new information. Infinitely many coefficient pairs can produce the same combined contribution, so the separate coefficients are unidentified.

```
X_bad = np.column_stack([np.ones(x_km.size), x_km, 1000 * x_km])
print(np.linalg.matrix_rank(X_bad))
print(X_bad.shape[1])
```

If rank is less than the number of columns, the design is rank deficient. `np.linalg.solve` may fail because  $X^T X$  is singular. `np.linalg.lstsq` can return one minimum-norm solution, but returned numbers do not make redundant coefficients meaningful.

Usually remove or redesign the redundant feature. More columns do not necessarily mean more information.

Near-collinearity is not exact rank failure but can create unstable coefficients and long, narrow SSR contours. Small data changes may then cause large coefficient changes while predictions move much less.

## 5.9 Read the error-surface figure

With an intercept and one slope, every point  $(\beta_0, \beta_1)$  is a candidate model. A contour joins candidate pairs with equal SSR. The centre marks the minimum.

- Well-separated information produces compact elliptical contours.
- Highly correlated design columns produce a long, narrow valley.
- Perfect collinearity produces a flat direction with no unique centre.

The contour plot is the two-parameter version of the one-parameter error bowl.

## 5.10 The complete estimator, method by method

The final `MHPCostEstimator` in the chapter has these responsibilities:

| Component                              | Responsibility                                                                             |
|----------------------------------------|--------------------------------------------------------------------------------------------|
| <code>AnalysisContract</code>          | Record purpose, target, unit of observation, and intended use                              |
| <code>__init__</code>                  | Create empty learned attributes and fitted state                                           |
| <code>_validate_inputs</code>          | Enforce numerical shapes and finite values                                                 |
| <code>_add_intercept_column</code>     | Prepend exactly one ones column                                                            |
| <code>_metrics_from_predictions</code> | Calculate residual-based metrics                                                           |
| <code>fit</code>                       | Check names, build design, call <code>lstsq</code> , reject rank deficiency, store results |

| Component    | Responsibility                                          |
|--------------|---------------------------------------------------------|
| predict      | Require fitted state and compatible new features        |
| evaluate     | Predict and calculate metrics for supplied observations |
| coefficients | Pair intercept and feature names with learned values    |

## Important safeguards in fit

- Feature-name count must equal raw feature count.
- Feature names cannot be empty or duplicated.
- The intercept is added internally; callers should not add another one.
- Required rank equals the number of design columns.
- Learned state is stored only after rank passes.
- Training predictions and metrics are calculated from the fitted design.
- Returning `self` permits `model = MHPCostEstimator(contract).fit(...)`.

## Important safeguards in predict

- Calling before `fit` raises `RuntimeError`.
- New rows must have the same feature count and order as training data.
- Inputs must be finite.
- A single project becomes shape `(1, p)`.

The implementation checks feature counts, not semantic column order. In production, labeled tables and schema checks should prevent accidentally swapping capacity and road-distance columns.

## 5.11 Interpret the synthetic KP fit carefully

The chapter fits eight projects using road distance, planned capacity, and terrain index. This tiny synthetic set is inspectable, not deployment-ready.

If the road-distance coefficient is positive, a prediction-oriented reading is:

Holding the other included numerical features fixed, the fitted rule adds the coefficient's number of million PKR for each additional kilometre of recorded road distance.

It does not establish that building a kilometre of road changes cost by that amount. “Distance from a road” and “constructing a road” are different variables and the latter is an intervention.

Also ask whether fixed-capacity, fixed-terrain comparisons at different distances actually occur in the data. Regression can calculate contrasts in weakly supported regions.

## 5.12 Tests that connect code to mathematics

The chapter verifies:

```
assert np.allclose(model.parameters_, beta_from_normal_equations)
assert np.allclose(X_design.T @ residuals, 0.0, atol=1e-9)
assert np.isclose(residuals.sum(), 0.0, atol=1e-9)
```

```
assert np.isclose(metrics["ssr"], residuals @ residuals)
assert np.isclose(metrics["rmse"], np.sqrt(np.mean(residuals ** 2)))
```

Each test corresponds to a claim:

- two solution methods agree for this full-rank example;
- residuals are perpendicular to every design column;
- the intercept makes training residuals sum to zero;
- code matches the SSR definition; and
- code matches the RMSE definition.

## 5.13 What this estimator still cannot do

It does not yet:

- create training, validation, and test splits;
- handle categorical districts, contractors, or project types;
- standardise features or thoroughly diagnose numerical conditioning;
- quantify coefficient or prediction uncertainty;
- diagnose nonlinearity, heteroskedasticity, or influential observations;
- model inflation, time trends, repeated districts, or clustering;
- enforce data lineage, access permissions, and revision history;
- protect automatically against project-phase leakage;
- establish causal effects; or
- monitor drift when future construction conditions change.

Acknowledging these limitations is part of a correct analysis, not an apology added after it.

---

# Week 1 capstone: a beginner's execution plan

---

The capstone asks you to build, break, and rebuild an early-stage estimator for 40 synthetic projects.

## Pass 1: define

Write an `AnalysisContract` and a short data note answering:

- What exactly is one row?
- What is the target, unit, and price basis?
- At what project stage is the prediction made?
- Which decision will it support?
- Is the job prediction, explanation, or causation?
- What is the precise estimand?

## Pass 2: build

1. Select at least three features genuinely known at prediction time.
2. Preserve project IDs while aligning rows.
3. Validate shapes, finite values, units, and names.

4. Fit the estimator.
5. Report coefficients with units and conditional wording.
6. Report training SSR, MSE, RMSE, and MAE.
7. Verify orthogonality and the metric equations.

## Pass 3: break

Create one controlled example of each failure and explain why it matters:

1. reverse target row order while leaving features unchanged;
2. insert NaN or infinity;
3. add a redundant metres version of a kilometre feature;
4. add information recorded after construction begins;
5. add one extreme target value.

Record whether validation catches the failure. Row misalignment and leakage are especially dangerous because plain array-shape validation may not detect them.

## Pass 4: rebuild

1. restore identifier alignment;
2. investigate and document missing/extreme values;
3. remove redundant information;
4. remove leaked features or redefine prediction time;
5. hold out projects not used for fitting;
6. compare training and held-out RMSE and MAE; and
7. write a recommendation separating supported prediction claims from unsupported explanatory or causal claims.

High held-out error relative to training error is a warning about overfitting, dataset differences, or both. One small held-out set also has uncertainty; do not treat it as a permanent truth.

## Self-assessment rubric

| Dimension     | Ready when you can...                                                 |
|---------------|-----------------------------------------------------------------------|
| Question      | state purpose, estimand, timing, target, and intended use precisely   |
| Mathematics   | reproduce key derivations and explain every symbol                    |
| Code          | validate shapes and rank and tie tests to equations                   |
| Geometry      | connect column space, projection, orthogonality, and minimum distance |
| Context       | discuss units, provenance, fairness, and institutional consequences   |
| Communication | report results, uncertainty, and limits in plain language             |

# Formula sheet with reading instructions

| Concept               | Formula                                                                  | Read it as...                                    |
|-----------------------|--------------------------------------------------------------------------|--------------------------------------------------|
| Linear prediction     | $\hat{y} = X\beta$                                                       | Build predictions from weighted design columns   |
| Residual              | $e = y - \hat{y}$                                                        | Actual minus predicted                           |
| SSR                   | $e^T e = \sum_i e_i^2$                                                   | Total squared training error                     |
| MSE                   | $SSR/n$                                                                  | Average squared error                            |
| RMSE                  | $\sqrt{SSR/n}$                                                           | Squared-error summary in target units            |
| MAE                   | $\frac{1}{n} \sum_i  e_i $                                               | Average absolute error                           |
| Orthogonality         | $X^T e = 0$                                                              | Residual is perpendicular to every design column |
| Normal equations      | $X^T X \hat{\beta} = X^T y$                                              | Zero-gradient/perpendicularity condition         |
| Closed form           | $(X^T X)^{-1} X^T y$                                                     | Unique coefficients when full column rank holds  |
| Through-origin slope  | $\frac{\sum_i x_i y_i}{\sum_i x_i^2}$                                    | One-feature slope when no intercept is allowed   |
| Intercept-model slope | $\frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$ | Co-movement divided by feature variation         |
| Intercept             | $\bar{y} - \hat{\beta}_1 \bar{x}$                                        | Make the fitted line pass through the mean point |
| Gradient              | $-2X^T y + 2X^T X \beta$                                                 | Direction and rate of local SSR increase         |
| Hessian               | $2X^T X$                                                                 | Curvature of the SSR surface                     |

## Shape sheet

For  $n$  observations,  $p$  raw features, and an intercept:

| Object        | Shape            |
|---------------|------------------|
| Raw features  | $(n, p)$         |
| Ones column   | $(n, 1)$         |
| Design matrix | $(n, p + 1)$     |
| Target        | $(n, 1)$         |
| Parameters    | $(p + 1, 1)$     |
| Predictions   | $(n, 1)$         |
| Residuals     | $(n, 1)$         |
| $X^T X$       | $(p + 1, p + 1)$ |
| $X^T y$       | $(p + 1, 1)$     |
| $X^T e$       | $(p + 1, 1)$     |

# Glossary in beginner language

---

**Affine model:** A linear combination plus a constant translation; regression with an intercept is affine in its inputs but conventionally called linear regression.

**Association:** A numerical relationship observed between variables. It need not be causal.

**Broadcasting:** NumPy's rules for expanding compatible shapes during elementwise operations. Convenient, but not a substitute for matrix multiplication.

**Causal effect:** Difference between outcomes under two interventions or conditions for a defined population and time horizon.

**Column space:** Every vector obtainable as a weighted combination of a matrix's columns.

**Condition number:** A measure of sensitivity to small numerical changes. A large value warns that coefficient calculations may be unstable.

**Confounder:** A common cause of an exposure or decision and an outcome.

**Convex objective:** A bowl-shaped objective with no deceptive local minima.

**Counterfactual:** The outcome that would have occurred under an alternative action or condition.

**Data dictionary:** Documentation of fields, meanings, units, timing, provenance, valid values, and missing-value rules.

**Design matrix:** The rectangular matrix whose columns are the building blocks of predictions.

**Estimator:** A procedure that converts data into parameter estimates; OLS is an estimator.

**Estimand:** The exact quantity an analysis intends to estimate.

**Feature:** An input variable used to form model predictions.

**Finite difference:** A numerical approximation to a derivative made by slightly changing an input.

**Full column rank:** No design column is an exact linear combination of the others.

**Gradient:** Vector containing all first partial derivatives of a scalar function.

**Hessian:** Matrix containing second partial derivatives and describing curvature.

**Hyperparameter:** A modeling setting chosen outside the fitting calculation.

**Intercept:** Constant amount added to every linear prediction.

**Loss function:** Numerical rule that penalises errors during model fitting.

**Matrix rank:** Number of independent directions represented by a matrix.

**Metric:** Numerical summary used to evaluate performance.

**Multicollinearity:** Linear dependence among features. Perfect multicollinearity causes rank deficiency; near-multicollinearity can make coefficients unstable.

**Normal equations:**  $X^T X \hat{\beta} = X^T y$ , the OLS condition obtained from geometry or calculus.

**Observation:** One unit represented by one dataset row.

**OLS:** Ordinary least squares, which chooses parameters that minimise the sum of squared residuals.

**Orthogonal:** Perpendicular; two vectors have dot product zero.

**Parameter:** Numerical value learned by fitting, such as an intercept or slope.

**Positive semidefinite:** A matrix  $A$  for which  $z^T A z \geq 0$  for every  $z$ .

**Prediction:** Estimated target value produced by the fitted model.

**Projection:** Closest point in a subspace to a given vector under Euclidean distance.

**Provenance:** Where, when, why, and by whom data were created or changed.

**Rank deficiency:** Fewer independent design directions than columns, so coefficients are not uniquely identified.

**Residual:** Observed target minus fitted prediction.

**Scalar:** One number.

**Simpson's paradox:** Aggregated and subgroup relationships differ or reverse because group composition matters.

**Singular value:** A quantity reported by matrix decompositions that helps describe independent directions and numerical conditioning.

**Target:** Outcome the model estimates.

**Target leakage:** Use of information unavailable or illegitimate at the real prediction time.

**Tensor:** An array with any number of axes; do not confuse tensor order with matrix rank.

**Training data:** Observations used to learn model parameters.

**Validation/test data:** Observations withheld from fitting and used to assess generalisation.

**Vector:** Ordered one-dimensional collection of numbers.

---

## Suggested five-day study schedule

---

| Activity each day               | Approximate minutes |
|---------------------------------|---------------------|
| Recall yesterday's main idea    | 15                  |
| Work through today's derivation | 35                  |
| Type, run, and alter proof code | 35                  |
| Create or inspect the figure    | 20                  |
| Extend the estimator            | 35                  |
| Break and repair one failure    | 25                  |
| Complete exit check in writing  | 15                  |
| <b>Total</b>                    | <b>180</b>          |

Three hours is a guide, not a race. If you cannot explain yesterday's exit check, retrieve that idea before adding new notation.

Morning retrieval prompts:

- After Day 1: name the three regression jobs and one risk of each.
- After Day 2: draw the shapes of  $\mathbf{X}$ ,  $\mathbf{y}$ , and  $\beta$ .
- After Day 3: expand one row of  $\mathbf{X}\beta$  and explain the ones column.
- After Day 4: explain why the OLS residual is perpendicular to  $\mathbf{X}$ .
- After Day 5: state the normal equations and the condition for unique coefficients.

---

## Responsible interpretation in the KP context

---

Ask these questions throughout the project:

- At what project stage is each field known?
- Who recorded it, using which definition and instrument?
- Does one row mean the same thing across districts and implementing partners?
- Are all costs expressed in the same currency and price year?
- Does a terrain index of 4 mean the same thing to every assessor?
- Does missingness reflect geography, staffing, or project difficulty?
- Could the estimator systematically disadvantage remote communities?
- Could a coefficient be used to unfairly judge a field team for structural conditions?
- Are comparisons supported by similar observed projects, or are they extrapolations?
- Has the analysis confused a predictive signal with an intervention effect?

Technical correctness is necessary but not sufficient. Public-infrastructure analysis also requires traceable data, appropriate comparisons, careful language, and awareness of who bears the consequences of error.

---

## Where Week 2 begins

---

Week 1 gives you a transparent OLS calculation and an honest statement of its limits. The next topics naturally address weaknesses exposed here:

1. feature scaling and numerical conditioning;
2. QR and singular-value decompositions;
3. probability models and maximum likelihood;
4. gradient descent built from the OLS gradient;
5. training, validation, and test separation;
6. coefficient and prediction uncertainty; and
7. residual diagnostics and model revision.

The enduring chain is:

```
precise question
-> defensible data representation
-> transparent mathematical model
```



- > independent code and geometric checks
- > bounded, responsible judgement

If you can move both ways through that chain—from a decision to equations and code, and from output back to a careful decision statement—you have achieved the central goal of Chapter 1.

# Chapter 2 — From a Fitted Line to a Trustworthy Model

---

## Level 2 Model Builder: the next seven days of the MHP Cost Estimator

**Central promise.** Chapter 1 taught you to derive and fit ordinary least squares (OLS). By the end of this chapter, you will be able to explain when that calculation is numerically stable, connect least squares to probability, fit it by gradient descent, measure performance on genuinely unseen projects, compare it with honest baselines and a shallow decision tree, quantify several kinds of uncertainty, diagnose failure patterns, and defend an evaluation design before a technical reviewer.

The learner is still assumed to be a beginner. Nothing important is hidden behind the phrase “the library does it.” We first construct each idea with small arrays and equations. Only after the mechanism is visible do we use `scikit-learn` to assemble a reliable workflow.

The long-term destination is research-level practice. That does **not** mean introducing every advanced term immediately. It means building habits that remain valid when the datasets, models, and papers become difficult:

- define the intended use before choosing an evaluation;
- separate mathematical computation from statistical evidence;
- distinguish information available at prediction time from information created later;
- treat model selection as part of the learning procedure;
- report uncertainty and subgroup failures, not only an average score; and
- connect claims to reproducible evidence.

---

## Why the original three-day draft needed a larger bridge

The earlier draft moved directly from OLS to three ready-made models, a data split, and three metrics. Those topics belong here, but an absolute beginner would have been asked to use several ideas before understanding them.

Chapter 1 explicitly identified the unresolved problems. The fitted line could be unstable when features have different scales; the normal-equation formula did not explain QR or singular values; OLS had not yet been connected to a probability model; the learner had derived a gradient but had not used it to learn; training error was not evidence about new projects; and uncertainty and diagnostic revision were still missing.

This chapter therefore follows the order promised at the end of Chapter 1.

## Learning outcomes

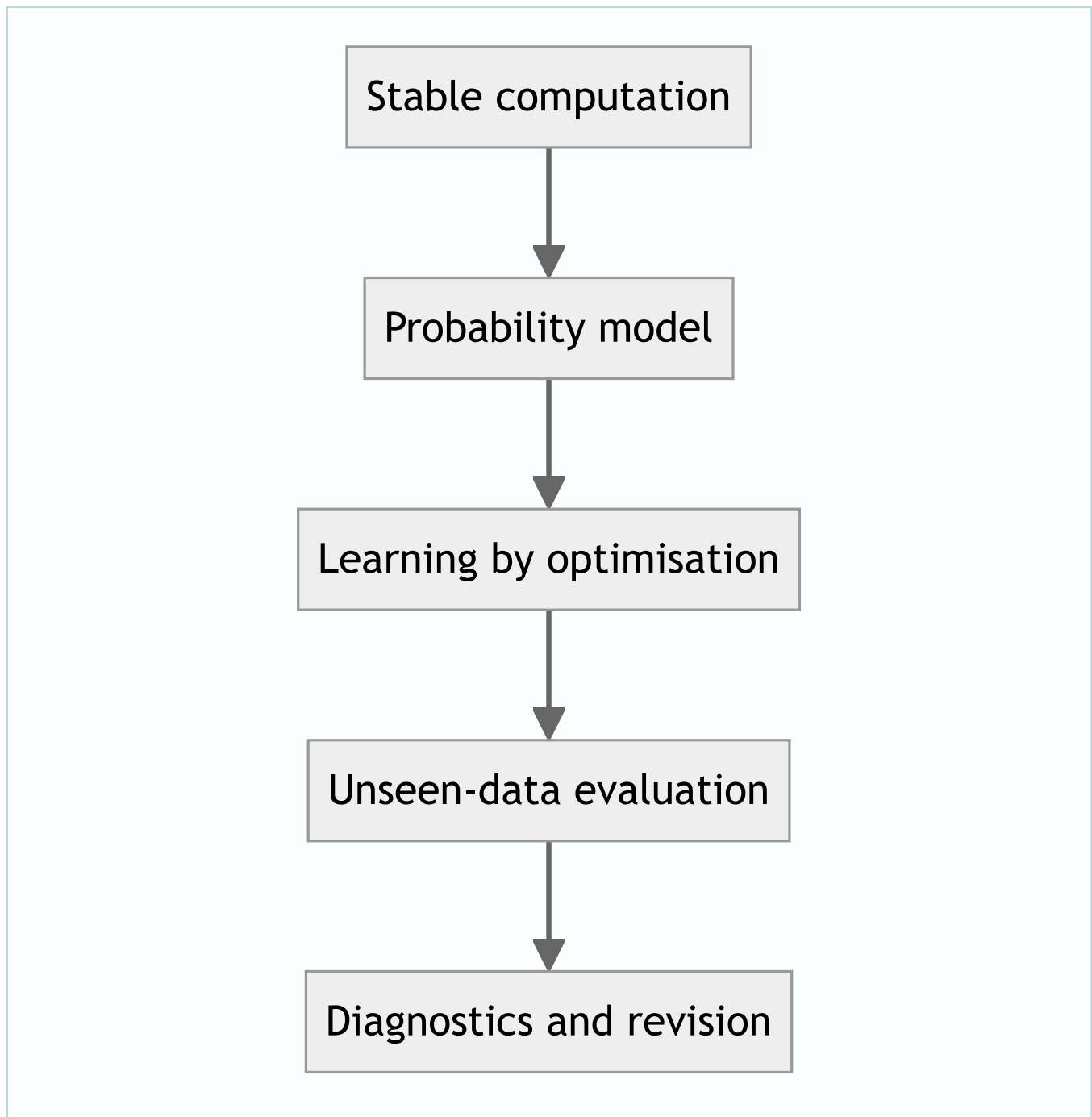
At the end of Chapter 2, you should be able to:

- explain floating-point approximation and numerical conditioning in plain language;
- standardise a feature using statistics calculated from the training data only;
- show why scaling changes coefficient units but need not change OLS predictions;
- calculate a condition number from singular values;
- solve least squares with QR decomposition and with the singular value decomposition (SVD);

- distinguish an error term from an observed residual;
- derive OLS as maximum likelihood under a Gaussian noise model;
- estimate residual variance, coefficient standard errors, confidence intervals, and prediction intervals under stated assumptions;
- derive and implement batch gradient descent for squared error;
- explain parameters, hyperparameters, fitting, selection, and evaluation as different operations;
- derive why the mean is the squared-error constant baseline and the median is the absolute-error constant baseline;
- recognise underfitting, overfitting, and distribution shift;
- choose among random, group, and temporal validation designs by matching the split to deployment;
- identify target, temporal, group, preprocessing, duplicate, and test-set leakage;
- explain ordinary, grouped, temporal, and nested cross-validation;
- calculate and interpret MAE, RMSE, median absolute error, and test-set  $R^2$ ;
- explain why MAPE can be misleading for small or zero targets;
- inspect residuals by fitted value, time, district, and project scale;
- place uncertainty around a held-out performance estimate using a transparent bootstrap;
- state why fold-to-fold variation is not automatically a confidence interval; and
- read a research paper by separating its question, design, result, limitation, and implication.

## The seven-day route

| Day    | Central idea                         | Problem it resolves                                          |
|--------|--------------------------------------|--------------------------------------------------------------|
| Day 6  | Scaling and conditioning             | Correct algebra can still produce fragile computation        |
| Day 7  | QR, SVD, rank, pseudoinverse         | The inverse formula is not the safest or most general solver |
| Day 8  | Probability, likelihood, uncertainty | A fitted line alone does not quantify uncertainty            |
| Day 9  | Gradient descent                     | A gradient becomes a learning algorithm                      |
| Day 10 | Generalisation and baselines         | Low training error can be meaningless                        |
| Day 11 | Splits, leakage, cross-validation    | Model selection can contaminate evaluation                   |
| Day 12 | Metrics, diagnostics, revision       | One average score can conceal operational failure            |



Each day uses Chapter 1's rhythm: orient, construct, prove, see, build, break, and reflect.

---

## Running case: when is an MHP cost estimate made?

We continue with fictional microhydro power (MHP) projects in Khyber Pakhtunkhwa. The target is final project cost expressed in **constant 2025 million PKR**. Using constant prices removes general inflation from the target so that the model can focus on project differences. A later forecasting chapter can model nominal prices and inflation explicitly.

The prediction is made at technical appraisal, before procurement and construction. This timing rule decides which features are legitimate.

| Field                  | Available at appraisal? | Use as a feature?                     | Reason                                                                 |
|------------------------|-------------------------|---------------------------------------|------------------------------------------------------------------------|
| Planned capacity       | Yes                     | Yes                                   | Known from design                                                      |
| Estimated cable length | Yes                     | Yes                                   | Known approximately from survey                                        |
| Road distance          | Yes                     | Yes                                   | Known from access assessment                                           |
| Terrain index          | Yes                     | Yes, cautiously                       | Known, but measurement consistency must be checked                     |
| District               | Yes                     | Depends on deployment                 | Useful for known districts; problematic for unseen-district deployment |
| Start year             | Yes                     | Yes, if the deployment design permits | May represent technical or institutional change                        |
| Final material bill    | No                      | No                                    | It is created during construction and leaks the outcome                |
| Actual completion time | No                      | No                                    | It is a later outcome, not an appraisal input                          |

**Prediction-time test:** Imagine the appraisal officer sitting at a desk on the day the estimate must be issued. If the value is not legitimately available then, it cannot enter the feature matrix—even if it is present in the completed-project database.

## Chapter data generator

The following function creates a reproducible learning dataset. It deliberately contains nonlinearity, district structure, changing conditions over time, larger noise for remote projects, and one tempting post-construction field. These are not defects in the exercise; they create the problems an honest workflow must discover.

Save this as `chapter2_data.py`.

```
import numpy as np
import pandas as pd

def make_mhp_projects(n=360, seed=2026):
 """Create fictional MHP projects for Chapter 2.

 The relationships are educational, not estimates of real KP projects.
 The target is final cost in constant 2025 million PKR.
 """
 rng = np.random.default_rng(seed)

 districts = np.array(["Chitral", "Dir", "Swat", "Shangla", "Kohistan"])
 district = rng.choice(districts, size=n, p=[0.22, 0.20, 0.24, 0.17, 0.17])
 start_year = rng.integers(2016, 2026, size=n)

 planned_capacity_kw = rng.uniform(80.0, 850.0, size=n)
 road_distance_km = np.clip(rng.gamma(shape=2.0, scale=6.0, size=n), 0.2, 42.0)
 terrain_index = rng.integers(1, 6, size=n)
 estimated_cable_km = np.clip(
 0.8 + 0.006 * planned_capacity_kw + rng.normal(0.0, 0.9, size=n),
 0.5,
 None,
)
```

```

district_effect = {
 "Chitral": 4.0,
 "Dir": 1.0,
 "Swat": 0.0,
 "Shangla": 2.5,
 "Kohistan": 5.0,
}
district_cost = np.array([district_effect[d] for d in district])

remote_surcharge = 7.0 * (road_distance_km > 18.0)
capacity_curve = 0.000025 * planned_capacity_kw**2
time_change = 0.7 * (start_year - 2016)
noise_sd = 2.5 + 0.12 * road_distance_km
noise = rng.normal(0.0, noise_sd)

actual_cost = (
 9.0
 + 0.030 * planned_capacity_kw
 + capacity_curve
 + 1.15 * estimated_cable_km
 + 0.85 * road_distance_km
 + 2.4 * terrain_index
 + remote_surcharge
 + district_cost
 + time_change
 + noise
)

This is known only after procurement/construction. It is deliberately leaky.
final_material_bill = 0.62 * actual_cost + rng.normal(0.0, 1.2, size=n)

return pd.DataFrame(
 {
 "project_id": [f"MHP-{i:04d}" for i in range(n)],
 "district": district,
 "start_year": start_year,
 "planned_capacity_kw": planned_capacity_kw,
 "estimated_cable_km": estimated_cable_km,
 "road_distance_km": road_distance_km,
 "terrain_index": terrain_index,
 "final_material_bill_million_pkr": final_material_bill,
 "actual_cost_2025_million_pkr": actual_cost,
 }
).sort_values(["start_year", "project_id"]).reset_index(drop=True)

if __name__ == "__main__":
 projects = make_mhp_projects()
 projects.to_csv("mhp_projects_chapter2.csv", index=False)
 print(projects.head())
 print(projects.shape)

```

## Minimal software setup

Chapter 1 used NumPy and Matplotlib. This chapter adds pandas, SciPy, and scikit-learn:

```
python -m pip install numpy pandas matplotlib scipy scikit-learn
```

Record the environment used for an experiment:

```

import numpy as np
import pandas as pd
import scipy
import sklearn

```

```
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)
print("SciPy:", scipy.__version__)
print("scikit-learn:", sklearn.__version__)
```

A paper or report is not computationally reproducible if the data, code, random seeds, and software environment cannot be identified.

## Day 6 — Scaling and Numerical Conditioning

**Today's central idea:** Algebra asks whether a solution exists. Numerical analysis asks whether a computer can calculate that solution reliably with finite precision.

### 6.1 Computers approximate most real numbers

A computer stores floating-point numbers with a limited number of binary digits. Many ordinary decimal values cannot be represented exactly. This is why:

```
print(0.1 + 0.2)
print((0.1 + 0.2) == 0.3)
```

prints a value close to 0.3 but the equality test is false.

This does not mean numerical computing is unreliable. It means calculations must be designed so that tiny representation errors are not unnecessarily magnified.

### 6.2 A well-posed problem can still be ill-conditioned

Suppose two datasets differ by a tiny amount. If their fitted coefficients also differ only slightly, the problem is well-conditioned. If a tiny data change causes a large coefficient change, it is ill-conditioned.

Conditioning belongs to the **problem and its representation**. Numerical stability belongs to the **algorithm** used to solve it. A stable algorithm cannot create information that nearly redundant features do not contain, but it can avoid making the situation worse.

For a full-column-rank matrix, the two-norm condition number is:

$$\kappa_2(X) = \frac{\sigma_{\max}(X)}{\sigma_{\min}(X)},$$

where:

- $\sigma_{\max}$  is the largest singular value;
- $\sigma_{\min}$  is the smallest singular value; and
- a larger ratio means that some directions in parameter space are much less informed than others.

A condition number near 1 is favourable. There is no universal number at which a model suddenly becomes invalid. Interpretation depends on floating-point precision, units, noise, and the decision. The condition number is a warning instrument, not a courtroom verdict.

## 6.3 Different units can create a numerical imbalance

In the same design matrix we might store:

- planned capacity around hundreds of kW;
- road distance around tens of km; and
- terrain on a 1–5 scale.

The columns then occupy very different numerical ranges. OLS predictions are not automatically wrong, but optimisation can become harder and coefficient comparison can become misleading.

Standardisation transforms feature  $j$  using:

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

where  $\mu_j$  and  $s_j$  must be calculated from the **training data**.

After transformation:

- a value of  $z = 0$  is at the training mean;
- $z = 1$  is one training standard deviation above the mean; and
- $z = -2$  is two training standard deviations below it.

## 6.4 Construct a scaler from scratch

```
import numpy as np

class StandardScalerFromScratch:
 def fit(self, X):
 X = np.asarray(X, dtype=float)
 if X.ndim != 2:
 raise ValueError("X must be a two-dimensional matrix")

 self.mean_ = X.mean(axis=0)
 # ddof=0 matches the population-style convention used by StandardScaler.
 self.scale_ = X.std(axis=0, ddof=0)

 if np.any(self.scale_ == 0.0):
 bad = np.where(self.scale_ == 0.0)[0].tolist()
 raise ValueError(f"Constant feature columns cannot be scaled: {bad}")
 return self

 def transform(self, X):
 if not hasattr(self, "mean_"):
 raise RuntimeError("Call fit before transform")
 X = np.asarray(X, dtype=float)
 return (X - self.mean_) / self.scale_

 def inverse_transform(self, Z):
 Z = np.asarray(Z, dtype=float)
 return Z * self.scale_ + self.mean_

 def fit_transform(self, X):
 return self.fit(X).transform(X)
```



```

X_train = np.array(
 [
 [100.0, 2.0, 1.0],
 [250.0, 8.0, 2.0],
 [500.0, 14.0, 4.0],
 [800.0, 25.0, 5.0],
]
)

scaler = StandardScalerFromScratch()
Z_train = scaler.fit_transform(X_train)

print("Training means:", scaler.mean_)
print("Training scales:", scaler.scale_)
print("Scaled means:", Z_train.mean(axis=0))
print("Scaled standard deviations:", Z_train.std(axis=0, ddof=0))

assert np.allclose(Z_train.mean(axis=0), 0.0)
assert np.allclose(Z_train.std(axis=0, ddof=0), 1.0)
assert np.allclose(scaler.inverse_transform(Z_train), X_train)

```

## Population standard deviation versus sample standard deviation

Two common formulas differ in the denominator:

$$s_{\text{population}} = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2},$$

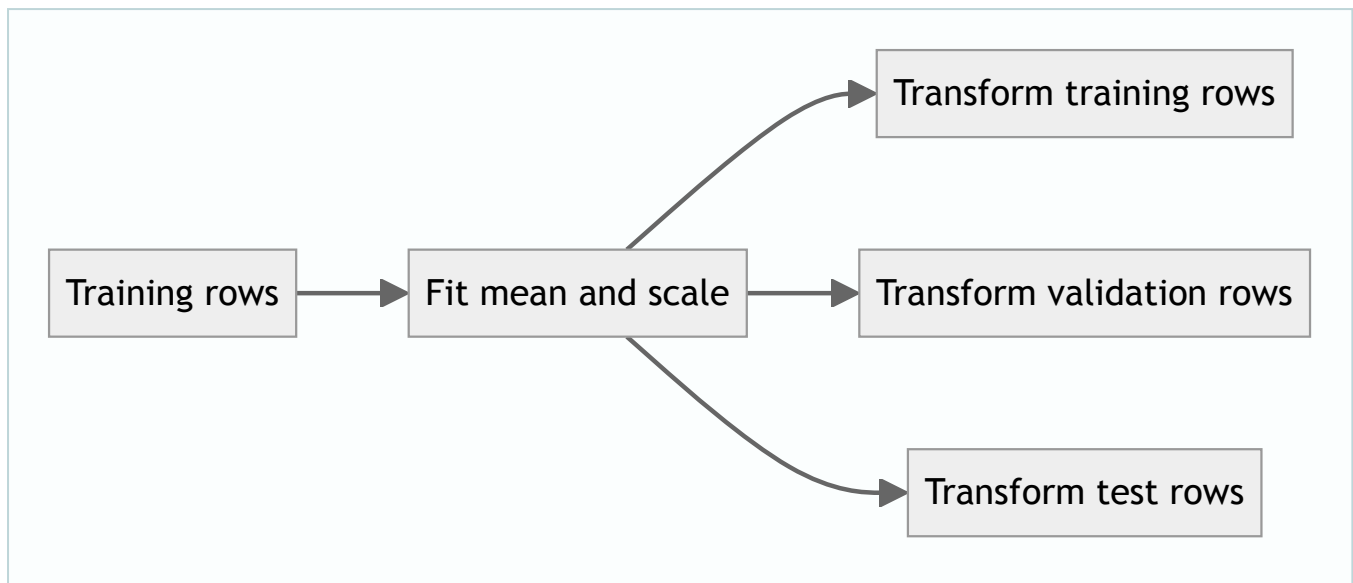
$$s_{\text{sample}} = \sqrt{\frac{1}{n-1} \sum_i (x_i - \bar{x})^2}.$$

The second is used in classical estimation of a population variance. A feature scaler is performing a deterministic transformation of the observed training column, so many software libraries use the first convention. The important point is not to memorise one denominator for every purpose; it is to know which quantity is being calculated and why.

## 6.5 Why the test set must not teach the scaler

Assume older projects have a mean capacity of 300 kW and future projects have a mean of 600 kW. If the future test projects help calculate the mean and standard deviation, the transformation has already learned something about the future distribution.

The correct sequence is:



The validation and test sets receive the training transformation. They do not modify it.

```

X_train = np.array([[100.0], [200.0], [300.0]])
X_test = np.array([[900.0], [1000.0]])

scaler = StandardScalerFromScratch().fit(X_train)
print("Correct test values:", scaler.transform(X_test).ravel())

Wrong: this lets the test distribution influence preprocessing.
leaky_scaler = StandardScalerFromScratch().fit(np.vstack([X_train, X_test]))
print("Leaky test values: ", leaky_scaler.transform(X_test).ravel())

```

The second result may look numerically more moderate. That is precisely the problem: it uses information the deployed model would not have had when it was trained.

## 6.6 Scaling changes coefficient units

Suppose the scaled model is:

$$\hat{y} = \gamma_0 + \sum_{j=1}^p \gamma_j z_j,$$

and  $z_j = (x_j - \mu_j)/s_j$ . Substitute:

$$\hat{y} = \gamma_0 + \sum_j \gamma_j \frac{x_j - \mu_j}{s_j}.$$

Rearrange:

$$\hat{y} = \left( \gamma_0 - \sum_j \frac{\gamma_j \mu_j}{s_j} \right) + \sum_j \frac{\gamma_j}{s_j} x_j.$$

Therefore the original-unit coefficients are:

$$\beta_j = \frac{\gamma_j}{s_j},$$

$$\beta_0 = \gamma_0 - \sum_j \frac{\gamma_j \mu_j}{s_j}.$$

The fitted predictions can remain the same while the coefficient numbers and their units change.

## Code proof: raw and scaled OLS give the same predictions

```
import numpy as np

rng = np.random.default_rng(7)
X = rng.normal(size=(80, 3)) * np.array([500.0, 10.0, 2.0])
y = 12.0 + X @ np.array([0.03, 0.8, 2.2]) + rng.normal(0.0, 1.0, size=80)

X_raw_design = np.column_stack([np.ones(X.shape[0]), X])
beta_raw = np.linalg.lstsq(X_raw_design, y, rcond=None)[0]
pred_raw = X_raw_design @ beta_raw

scaler = StandardScalerFromScratch()
Z = scaler.fit_transform(X)
Z_design = np.column_stack([np.ones(Z.shape[0]), Z])
gamma = np.linalg.lstsq(Z_design, y, rcond=None)[0]
pred_scaled = Z_design @ gamma

beta_recovered = np.empty_like(gamma)
beta_recovered[1:] = gamma[1:] / scaler.scale_
beta_recovered[0] = gamma[0] - np.sum(gamma[1:] * scaler.mean_ / scaler.scale_)

print("Raw coefficients: ", beta_raw)
print("Recovered coefficients:", beta_recovered)
print("Maximum prediction gap:", np.max(np.abs(pred_raw - pred_scaled)))

assert np.allclose(pred_raw, pred_scaled)
assert np.allclose(beta_raw, beta_recovered)
```

## 6.7 See the condition number improve

```
import numpy as np

rng = np.random.default_rng(11)
capacity = rng.uniform(80.0, 850.0, size=200)
road = rng.uniform(0.5, 35.0, size=200)
terrain = rng.integers(1, 6, size=200)
X = np.column_stack([capacity, road, terrain])

raw_design = np.column_stack([np.ones(X.shape[0]), X])

scaler = StandardScalerFromScratch()
Z = scaler.fit_transform(X)
scaled_design = np.column_stack([np.ones(Z.shape[0]), Z])

print("Condition number before scaling:", np.linalg.cond(raw_design))
print("Condition number after scaling: ", np.linalg.cond(scaled_design))
```

Scaling often reduces the imbalance caused by units. It cannot repair true redundancy. If one column is almost a copy of another, their smallest singular value can remain tiny after scaling.

## 6.8 Break it: near-duplicate information

```
rng = np.random.default_rng(12)
road_km = rng.uniform(1.0, 30.0, size=100)
```

```
Almost the same information, with a tiny measurement perturbation.
road_minutes_proxy = 4.0 * road_km + rng.normal(0.0, 0.001, size=100)
X_near_duplicate = np.column_stack([road_km, road_minutes_proxy])

print("Rank:", np.linalg.matrix_rank(X_near_duplicate))
print("Condition number:", np.linalg.cond(X_near_duplicate))
```

The matrix may technically have full rank, yet its coefficients can be unstable because the two columns provide almost the same direction. Prediction may remain acceptable while individual coefficient interpretations become fragile. This distinction will matter throughout the book.

## 6.9 When should features be scaled?

| Method                               | Usually benefits from scaling? | Main reason                                           |
|--------------------------------------|--------------------------------|-------------------------------------------------------|
| OLS solved by a stable decomposition | Often                          | Conditioning and coefficient comparability            |
| Gradient descent                     | Yes                            | Features otherwise create very different curvature    |
| Ridge or lasso regression            | Yes                            | Penalties act on coefficient magnitudes               |
| k-nearest neighbours                 | Yes                            | Distance is sensitive to units                        |
| Support vector machines              | Usually                        | Margins and kernels depend on scale                   |
| Decision trees                       | Usually not required           | Threshold splits depend on order, not Euclidean scale |

This is not a licence to scale before splitting. Any learned transformation belongs inside the training procedure.

## 6.10 Day 6 build, break, and reflect

### Build

1. Load the synthetic project data.
2. Select the four numeric appraisal features.
3. Fit the from-scratch scaler on projects through 2021.
4. transform later projects without refitting.
5. Compare condition numbers before and after scaling.

### Break

1. Add capacity in watts as well as capacity in kilowatts.
2. Add a constant feature.
3. Fit the scaler on all years.
4. Write the specific failure caused by each action.

### Reflect

Explain why “all my columns are now between roughly -3 and 3” is not evidence that the model will generalise.

## Day 6 exit check

You are ready for Day 7 when you can explain all four statements:

1. Full rank and good conditioning are not the same condition.
2. Scaling can improve computation without adding information.

3. A scaler has learned parameters: its training means and scales.
4. Test data must never influence those parameters.

## Day 7 — QR, SVD, Rank, and the Pseudoinverse

**Today's central idea:** Least squares is one mathematical problem with several computational routes. QR and SVD expose the geometry and rank more safely than explicitly forming  $(X^T X)^{-1}$ .

### 7.1 Return to the least-squares problem

Chapter 1 minimised:

$$\|y - X\beta\|_2^2.$$

The normal equations are:

$$X^T X \hat{\beta} = X^T y.$$

They are mathematically correct. But forming  $X^T X$  roughly squares the condition number:

$$\kappa_2(X^T X) = \kappa_2(X)^2$$

when  $X$  has full column rank. A problem with condition number  $10^6$  can therefore produce a normal-equation matrix with condition number about  $10^{12}$ .

This motivates decompositions that operate on  $X$  directly.

### 7.2 QR decomposition: rotate, then solve a triangular system

For an  $n \times p$  full-column-rank design matrix with  $n \geq p$ , the reduced QR decomposition is:

$$X = QR,$$

where:

- $Q$  is  $n \times p$ ;
- the columns of  $Q$  are orthonormal, so  $Q^T Q = I_p$ ; and
- $R$  is a  $p \times p$  upper-triangular matrix.

Insert  $X = QR$  into the objective:

$$\|y - QR\beta\|_2^2.$$

The fitted component of  $y$  in the column space is  $QQ^T y$ . Therefore:

$$R\hat{\beta} = Q^T y.$$

Because  $R$  is triangular, solve it by back substitution rather than inversion.



## 7.3 Code proof: QR and lstsq agree

```

import numpy as np

rng = np.random.default_rng(21)
X_features = rng.normal(size=(60, 3))
X = np.column_stack([np.ones(X_features.shape[0]), X_features])
beta_true = np.array([10.0, 2.0, -1.5, 0.7])
y = X @ beta_true + rng.normal(0.0, 0.5, size=X.shape[0])

Q, R = np.linalg.qr(X, mode="reduced")
beta_qr = np.linalg.solve(R, Q.T @ y)
beta_lstsq = np.linalg.lstsq(X, y, rcond=None)[0]

print("Q shape:", Q.shape)
print("R shape:", R.shape)
print("Orthonormality error:", np.linalg.norm(Q.T @ Q - np.eye(Q.shape[1])))
print("Coefficient gap:", np.max(np.abs(beta_qr - beta_lstsq)))

assert np.allclose(Q.T @ Q, np.eye(Q.shape[1]))
assert np.allclose(beta_qr, beta_lstsq)

```

## 7.4 What “orthonormal” means

For two different columns  $q_j$  and  $q_k$  of  $Q$ :

$$q_j^T q_k = 0.$$

Each column also has length 1:

$$q_j^T q_j = 1.$$

Thus  $Q$  describes perpendicular unit directions spanning the same column space as  $X$ . Multiplying by  $Q^T$  measures how much of  $y$  lies along each direction.

## 7.5 SVD: reveal every informed direction

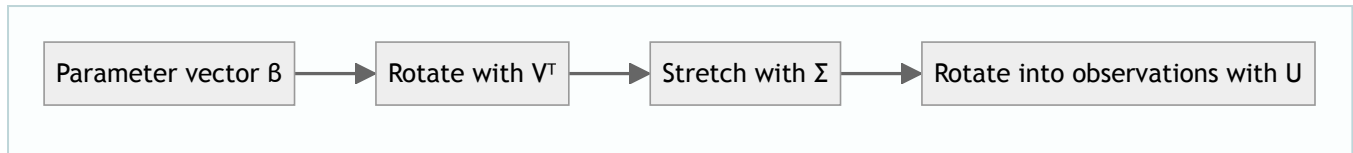
The singular value decomposition writes:

$$X = U \Sigma V^T.$$

For the reduced SVD of an  $n \times p$  matrix:

- $U$  contains orthonormal directions in observation space;
- $V$  contains orthonormal directions in parameter space; and
- $\Sigma$  is diagonal, with nonnegative singular values  $\sigma_1 \geq \sigma_2 \geq \dots$ .

The transformation can be read in three stages:



A very small singular value identifies a parameter direction that changes predictions only slightly. The data contain little information for distinguishing coefficients along that direction.

## 7.6 The pseudoinverse

If every singular value is positive, invert the diagonal entries:

$$X^+ = V\Sigma^{-1}U^T.$$

For a rank-deficient matrix, use the Moore–Penrose pseudoinverse. Singular values treated as zero are not inverted:

$$\hat{\beta} = X^+y = V\Sigma^+U^Ty.$$

When multiple coefficient vectors achieve the same minimum residual norm, the pseudoinverse returns the minimum-Euclidean-norm solution. That is a precise selection rule, not proof that the individual coefficients have become identifiable.

## 7.7 Construct an SVD least-squares solver

```

import numpy as np

def svd_least_squares(X, y, rcond=None):
 X = np.asarray(X, dtype=float)
 y = np.asarray(y, dtype=float)

 U, singular_values, Vt = np.linalg.svd(X, full_matrices=False)

 if rcond is None:
 rcond = np.finfo(float).eps * max(X.shape)

 tolerance = rcond * singular_values[0]
 keep = singular_values > tolerance

 inverse_singular_values = np.zeros_like(singular_values)
 inverse_singular_values[keep] = 1.0 / singular_values[keep]

 beta = Vt.T @ (inverse_singular_values * (U.T @ y))
 rank = int(np.sum(keep))

 return beta, rank, singular_values, tolerance

X = np.array(
 [
 [1.0, 1.0, 1000.0],
 [1.0, 2.0, 2000.0],
 [1.0, 3.0, 3000.0],
 [1.0, 4.0, 4000.0],
]
)
y = np.array([12.0, 19.0, 29.0, 38.0])

beta, rank, singular_values, tolerance = svd_least_squares(X, y)
beta_numpy = np.linalg.lstsq(X, y, rcond=None)[0]

```

```

print("Rank:", rank)
print("Singular values:", singular_values)
print("Tolerance:", tolerance)
print("SVD coefficients:", beta)
print("NumPy coefficients:", beta_numpy)

assert rank == 2
assert np.allclose(X @ beta, X @ beta_numpy)

```

The kilometre and metre columns are redundant. The coefficient vector returned by SVD is computable, but it should not be interpreted as two separately learned physical effects.

## 7.8 Rank is tolerance-dependent in numerical work

In exact mathematics, a singular value is either zero or nonzero. In floating-point computation, a value such as  $10^{-15}$  may be numerical residue rather than meaningful information.

Numerical rank therefore asks:

Which singular values are large enough to treat as genuine directions at the working precision and scale?

Changing the tolerance can change the reported rank. A research report should record the software, scaling, tolerance rule, and singular values when rank is substantively important.

## 7.9 Compare the three computational routes

| Route                        | Core operation                         | Strength                                         | Main caution                                           |
|------------------------------|----------------------------------------|--------------------------------------------------|--------------------------------------------------------|
| Normal equations             | Solve $X^T X \beta = X^T y$            | Connects directly to derivation                  | Squares the condition number                           |
| QR                           | Solve $R \beta = Q^T y$                | Stable and efficient for full-rank least squares | Rank deficiency needs pivoting or another strategy     |
| SVD                          | Apply $V \Sigma^+ U^T y$               | Reveals rank and weak directions                 | More computational work                                |
| <code>np.linalg.lstsq</code> | Library-selected least-squares routine | Appropriate default in application code          | Still requires interpretation of rank and conditioning |

“Use `lstsq`” is now earned advice: the learner understands the objective, normal equations, QR route, SVD route, and rank issue beneath the function call.

## 7.10 Figure lab: singular values on a log scale

```

import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(24)
x1 = rng.normal(size=120)
x2 = 2.0 * x1 + rng.normal(0.0, 1e-4, size=120)
x3 = rng.normal(size=120)
X = np.column_stack([np.ones(120), x1, x2, x3])

singular_values = np.linalg.svd(X, compute_uv=False)

fig, ax = plt.subplots(figsize=(7, 4))

```



```

ax.plot(range(1, len(singular_values) + 1), singular_values, marker="o")
ax.set_yscale("log")
ax.set_xlabel("Singular-value index")
ax.set_ylabel("Singular value (log scale)")
ax.set_title("A near-duplicate feature creates a weak direction")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

The plot is a spectrum: it shows how strongly the design matrix represents different directions. A sharp fall to a tiny final singular value is a warning about coefficient stability.

## 7.11 Build: a stable deterministic estimator

```

import numpy as np

class StableOLS:
 def __init__(self, fit_intercept=True):
 self.fit_intercept = fit_intercept

 def _design(self, X):
 X = np.asarray(X, dtype=float)
 if X.ndim != 2:
 raise ValueError("X must be two-dimensional")
 if self.fit_intercept:
 return np.column_stack([np.ones(X.shape[0]), X])
 return X

 def fit(self, X, y):
 design = self._design(X)
 y = np.asarray(y, dtype=float)

 if y.ndim != 1 or y.shape[0] != design.shape[0]:
 raise ValueError("y must be one-dimensional and aligned with X")
 if not np.isfinite(design).all() or not np.isfinite(y).all():
 raise ValueError("X and y must contain only finite values")

 result = np.linalg.lstsq(design, y, rcond=None)
 self.parameters_, self.ssr_array_, self.rank_, self.singular_values_ = result
 self.n_parameters_ = design.shape[1]
 self.condition_number_ = np.linalg.cond(design)
 self.is_full_rank_ = self.rank_ == self.n_parameters_
 return self

 def predict(self, X):
 if not hasattr(self, "parameters_"):
 raise RuntimeError("Fit the model before prediction")
 return self._design(X) @ self.parameters_

```

This class makes two diagnostics first-class outputs: rank and condition number. It does not silently turn a computational answer into an interpretive claim.

## 7.12 Day 7 build, break, and reflect

### Build

1. Fit the same training data using normal equations, QR, SVD, and `lstsq`.
2. Compare predictions, coefficients, rank, singular values, and condition number.
3. Explain why predictions can agree even when rank-deficient coefficients are not unique.

## Break

1. Add capacity in both kW and watts.
2. Add a near-copy of road distance.
3. Perturb one road-distance value by  $10^{-6}$ .
4. Compare coefficient changes with prediction changes.

## Reflect

Write two separate conclusions:

- one about predictive stability; and
- one about coefficient interpretability.

## Day 7 exit check

Without looking back, complete the chain:

$$X = QR \Rightarrow \underline{\hspace{2cm}}$$

and:

$$X = U\Sigma V^T \Rightarrow \hat{\beta} = \underline{\hspace{2cm}}.$$

Then explain what a tiny singular value means in one sentence a nontechnical manager could understand.

# Day 8 — Probability, Likelihood, and Uncertainty

**Today's central idea:** OLS can be fitted without a probability model. To make classical probability statements about coefficients and future outcomes, we must add assumptions and state them openly.

**Day 8 notation checkpoint.** A probability density  $f(y)$  is a curve whose *area* represents probability. The integral  $\int_a^b f(y) dy$  means “add infinitely thin strips of area from  $a$  to  $b$ ”; it is the continuous counterpart of the sums used since Chapter 0. An expectation  $\mathbb{E}[Z]$  is a probability-weighted average of the possible values of  $Z$ . A subscript such as  $\mathbb{E}_{(X,Y) \sim P_{\text{deployment}}}[L(Y, f(X))]$  says both what is averaged—the loss  $L$ —and which population distribution supplies the cases—the deployment population. You need to read these symbols, not calculate difficult integrals by hand.

## 8.1 Deterministic fit versus stochastic model

So far,  $y$  and  $X$  have been arrays and OLS has been a geometric optimisation:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

No probability distribution was required to calculate that projection.

A statistical model adds a claim about how outcomes vary:

$$y = X\beta + \varepsilon.$$

Here:

- $\beta$  is an unknown population parameter vector;
- $X\beta$  is the systematic component specified by the model; and
- $\varepsilon$  is a random error vector representing influences not captured by  $X\beta$ .

The word *error* does not mean a data-entry mistake. It means the difference between the outcome and the model's conditional mean in the assumed data-generating process.

## 8.2 Error terms are not residuals

The unobserved error for project  $i$  is:

$$\varepsilon_i = y_i - x_i^T \beta.$$

The observed residual after fitting is:

$$e_i = y_i - x_i^T \hat{\beta}.$$

We never observe the true  $\beta$ , so we do not observe the true errors. Residuals are estimates shaped by the fitted model. For OLS with an intercept, they also obey constraints such as summing to zero. Treating residuals as independent raw observations is therefore unsafe.

## 8.3 A probability distribution describes possible values

For a continuous random variable  $Y$ , a probability density  $f(y)$  describes relative density around possible values. Probability over an interval is area:

$$P(a \leq Y \leq b) = \int_a^b f(y) dy.$$

For a Gaussian—or normal—random variable with mean  $\mu$  and variance  $\sigma^2$ :

$$f(y | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[ -\frac{(y - \mu)^2}{2\sigma^2} \right].$$

Read the equation in parts:

- the density is centred at  $\mu$ ;
- $\sigma$  controls the horizontal spread;
- deviations are squared, so equal positive and negative deviations have equal density; and
- the exponential makes large deviations progressively less likely under the model.

## 8.4 The classical Gaussian linear model

One common model assumes:

$$Y_i | X_i = x_i \sim \mathcal{N}(x_i^T \beta, \sigma^2).$$

Equivalently:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

This compact statement contains several assumptions:

1. the conditional mean is linear in the chosen design columns;
2. errors have conditional mean zero;
3. errors have a common conditional variance  $\sigma^2$ ;
4. errors are independent under the sampling model; and
5. errors are Gaussian for the exact finite-sample distributional results developed below.

These assumptions are not automatically true because `LinearRegression()` ran successfully.

## 8.5 Likelihood reverses the question

A probability model asks:

If  $\beta$  and  $\sigma^2$  were known, how plausible would different datasets be?

Likelihood asks:

Given the dataset we observed, which parameter values make it most plausible under the model?

For independent observations, multiply their Gaussian densities:

$$L(\beta, \sigma^2 \mid X, y) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[ -\frac{(y_i - x_i^T \beta)^2}{2\sigma^2} \right].$$

The likelihood is a function of the parameters with the observed data held fixed. It is not the probability that a fixed parameter is true.

## 8.6 Why we use the log-likelihood

Products of many small densities can underflow numerically. The logarithm turns products into sums and preserves the location of the maximum because the logarithm is strictly increasing.

Take logs:

$$\ell(\beta, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - x_i^T \beta)^2.$$

For a fixed  $\sigma^2$ , the first two terms do not depend on  $\beta$ . Maximising the log-likelihood therefore means minimising:

$$\sum_i (y_i - x_i^T \beta)^2.$$

Thus, under the Gaussian equal-variance model:

**maximum likelihood for  $\beta$  = ordinary least squares.**

This is a bridge between optimisation and probability—not a claim that every least-squares dataset is Gaussian.

## 8.7 Code proof: the likelihood and SSR choose the same slope

```
import numpy as np

x = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([11.0, 18.0, 31.0, 39.0])
sigma = 2.0

def ssr(beta_0, beta_1):
 residuals = y - (beta_0 + beta_1 * x)
 return np.sum(residuals**2)

def gaussian_log_likelihood(beta_0, beta_1, sigma):
 residuals = y - (beta_0 + beta_1 * x)
 n = y.size
 return (
 -0.5 * n * np.log(2.0 * np.pi)
 - n * np.log(sigma)
 - np.sum(residuals**2) / (2.0 * sigma**2)
)

beta_0_grid = np.linspace(-5.0, 15.0, 401)
beta_1_grid = np.linspace(5.0, 15.0, 401)

best_ssr = (np.inf, None)
best_likelihood = (-np.inf, None)

for beta_0 in beta_0_grid:
 for beta_1 in beta_1_grid:
 current_ssr = ssr(beta_0, beta_1)
 current_log_likelihood = gaussian_log_likelihood(beta_0, beta_1, sigma)

 if current_ssr < best_ssr[0]:
 best_ssr = (current_ssr, (beta_0, beta_1))
 if current_log_likelihood > best_likelihood[0]:
 best_likelihood = (current_log_likelihood, (beta_0, beta_1))

print("SSR minimum: ", best_ssr[1])
print("Likelihood maximum: ", best_likelihood[1])
assert best_ssr[1] == best_likelihood[1]
```

The grid is intentionally inefficient. Its purpose is to verify the equivalence by two independently calculated criteria.

## 8.8 Estimating the unexplained variance

After fitting  $k$  parameters, including the intercept, calculate:

$$SSR = e^T e.$$

Under the classical linear model, an unbiased estimator of  $\sigma^2$  is:

$$s^2 = \frac{SSR}{n - k}.$$

Why  $n - k$  rather than  $n$ ? Fitting  $k$  independent parameters imposes  $k$  constraints and uses  $k$  degrees of freedom. Only  $n - k$  residual degrees of freedom remain for estimating noise.

The maximum-likelihood estimator of  $\sigma^2$  under the Gaussian model uses  $SSR/n$ . The two formulas answer slightly different estimation criteria. This is another example of why denominators must be connected to purpose.

## 8.9 Sampling variability of the OLS coefficients

Under the classical assumptions and treating  $X$  as fixed:

$$\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(X^T X)^{-1}).$$

Replace unknown  $\sigma^2$  with  $s^2$ :

$$\widehat{\text{Var}}(\hat{\beta}) = s^2(X^T X)^{-1}.$$

The standard error of coefficient  $j$  is the square root of diagonal element  $j$ :

$$SE(\hat{\beta}_j) = \sqrt{[s^2(X^T X)^{-1}]_{jj}}.$$

A large standard error can arise from substantial outcome noise, limited sample size, narrow feature variation, or multicollinearity. It is not simply a sign that the fitting code failed.

## 8.10 Confidence interval for a coefficient

A two-sided  $100(1 - \alpha)\%$  interval is:

$$\hat{\beta}_j \pm t_{1-\alpha/2, n-k} SE(\hat{\beta}_j).$$

In a frequentist interpretation, the parameter is fixed and the interval is random before sampling. If the entire sampling-and-interval procedure were repeated under its assumptions, a proportion  $1 - \alpha$  of those intervals would contain the true parameter. It is not strictly correct to say there is a 95% probability that this one fixed-parameter interval contains  $\beta_j$ .

## 8.11 Confidence interval versus prediction interval

For a new feature row  $x_0$ , the estimated conditional mean is:

$$\hat{y}_0 = x_0^T \hat{\beta}.$$

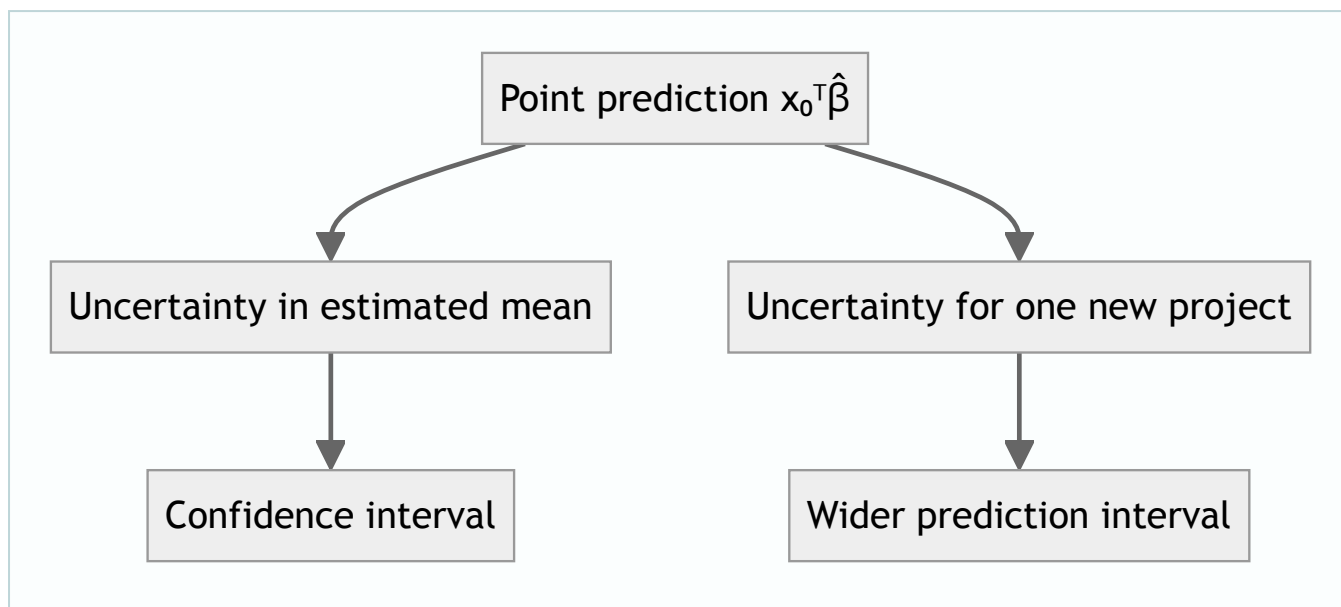
The standard error for the **mean response** is:

$$SE_{\text{mean}}(x_0) = s \sqrt{x_0^T (X^T X)^{-1} x_0}.$$

The standard error for one **new project outcome** is:

$$SE_{\text{prediction}}(x_0) = s \sqrt{1 + x_0^T (X^T X)^{-1} x_0}.$$

The extra 1 represents irreducible project-to-project noise. Therefore a prediction interval for an individual new project is wider than a confidence interval for the mean cost of many comparable projects.



## 8.12 Build: GaussianOLS

`GaussianOLS` is a **subclass** of the `StableOLS` class built earlier. Class inheritance means “start with the parent class's behaviour, then add or replace selected behaviour.” Inside `fit`, `super().fit(X, y)` calls the parent's already-tested fitting method on this same object before the subclass calculates uncertainty quantities; it avoids copying the original fitting code.

In the code below, `t.ppf(q, df)` uses the Student-*t* **percent-point function**: it returns the value below which probability *q* falls, so it is the inverse of the cumulative distribution function. For a two-sided 95% interval, `q=0.975` finds the upper critical value.

```
import numpy as np
from scipy.stats import t

class GaussianOLS(StableOLS):
 def fit(self, X, y):
 super().fit(X, y)
 design = self._design(X)
 y = np.asarray(y, dtype=float)

 if not self.is_full_rank_:
 raise ValueError(
 "Classical coefficient intervals require an identified full-rank design"
)

 self.residuals_ = y - design @ self.parameters_
 self.ssr_ = self.residuals_ @ self.residuals_
 self.degrees_of_freedom_ = design.shape[0] - design.shape[1]

 if self.degrees_of_freedom_ <= 0:
 raise ValueError("Not enough residual degrees of freedom")

 self.residual_variance_ = self.ssr_ / self.degrees_of_freedom_
 xtx_inverse = np.linalg.inv(design.T @ design)
 self.covariance_ = self.residual_variance_ * xtx_inverse
 self.standard_errors_ = np.sqrt(np.diag(self.covariance_))
 self.xtx_inverse_ = xtx_inverse
 return self

 def coefficient_intervals(self, confidence=0.95):
 alpha = 1.0 - confidence
 critical = t.ppf(1.0 - alpha / 2.0, self.degrees_of_freedom_)
```

```

 margin = critical * self.standard_errors_
 return np.column_stack(
 [self.parameters_ - margin, self.parameters_ + margin]
)

 def intervals_for(self, X_new, confidence=0.95):
 design_new = self._design(X_new)
 predictions = design_new @ self.parameters_

 leverage_new = np.einsum(
 "ij,jk,ik->i", design_new, self.txt_inverse_, design_new
)

 alpha = 1.0 - confidence
 critical = t.ppf(1.0 - alpha / 2.0, self.degrees_of_freedom_)

 mean_margin = critical * np.sqrt(
 self.residual_variance_ * leverage_new
)
 prediction_margin = critical * np.sqrt(
 self.residual_variance_ * (1.0 + leverage_new)
)

 return {
 "prediction": predictions,
 "mean_lower": predictions - mean_margin,
 "mean_upper": predictions + mean_margin,
 "prediction_lower": predictions - prediction_margin,
 "prediction_upper": predictions + prediction_margin,
 }

```

`np.einsum("ij,jk,ik->i", X, A, X)` calculates  $\mathbf{x}_i^T \mathbf{A} \mathbf{x}_i$  separately for every row  $i$ , without writing a Python loop or constructing a large intermediate array. Read the index string as a bookkeeping rule:  $i$  remains in the output, while repeated  $j$  and  $k$  indices are multiplied and summed. Chapter 3 reuses this exact row-wise quadratic-form pattern for leverage.

This class uses  $(\mathbf{X}^T \mathbf{X})^{-1}$  for the covariance formula after checking full rank. In more advanced work, use numerically stable decomposition-based routines and consider heteroskedasticity-robust or cluster-robust covariance estimators when their assumptions match the data structure.

## 8.13 Code laboratory: intervals widen away from the data centre

```

import numpy as np

rng = np.random.default_rng(81)
x = rng.uniform(2.0, 20.0, size=80)
y = 10.0 + 1.8 * x + rng.normal(0.0, 3.0, size=80)

model = GaussianOLS().fit(x.reshape(-1, 1), y)
new_x = np.array([[2.0], [11.0], [20.0], [35.0]])
intervals = model.intervals_for(new_x)

for i, value in enumerate(new_x.ravel()):
 width = intervals["prediction_upper"][i] - intervals["prediction_lower"][i]
 print(f"x={value:5.1f}, prediction-interval width={width:6.2f}")

```

The interval is usually narrower near the centre of well-observed feature values and wider for extrapolation. The model is less certain about a mean where the design provides less information.



## 8.14 Assumption map: which claim needs which support?

| Claim                                                 | Key requirements                                                                        |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------|
| lstsq found a least-squares solution                  | Valid finite arrays and a correct computation                                           |
| The coefficient vector is unique                      | Full column rank                                                                        |
| The fitted relationship predicts similar new projects | Representative evaluation under the deployment distribution                             |
| Classical standard errors are valid                   | Correct mean structure plus the variance and dependence assumptions used by the formula |
| A coefficient is causal                               | A defensible causal design, not merely Gaussian residuals                               |

Normal-looking residuals do not solve confounding. Conversely, non-normal residuals do not prove that the fitted least-squares projection was calculated incorrectly.

## 8.15 Break it: heteroskedastic project costs

The synthetic generator intentionally makes noise increase with road distance. This is heteroskedasticity: conditional variance changes across observations.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(82)
road = rng.uniform(0.0, 35.0, size=250)
noise_sd = 1.0 + 0.25 * road
cost = 12.0 + 1.4 * road + rng.normal(0.0, noise_sd)

model = StableOLS().fit(road.reshape(-1, 1), cost)
pred = model.predict(road.reshape(-1, 1))
residual = cost - pred

fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(pred, residual, alpha=0.65)
ax.axhline(0.0, color="black", linewidth=1)
ax.set_xlabel("Fitted cost")
ax.set_ylabel("Residual")
ax.set_title("Fan-shaped residuals signal changing variance")
plt.tight_layout()
plt.show()
```

The OLS line may still estimate a useful conditional mean under suitable conditions, but the equal-variance standard-error formula and a constant-width uncertainty story are questionable.

## 8.16 Day 8 build, break, and reflect

### Build

1. Fit `GaussianOLS` to a small full-rank subset of the MHP features.
2. Report coefficient estimates, standard errors, and intervals with units.
3. Construct a confidence interval for mean cost and a prediction interval for an individual project.

### Break

1. Add a redundant column and observe why the classical interval method refuses to proceed.

2. Create variance that grows with road distance.
3. Fit a straight line to a curved relationship.
4. State which assumptions each break threatens.

### Reflect

Complete this sentence precisely:

“Assuming the linear mean, independence, equal variance, Gaussian finite-sample model, and the stated sampling process are adequate, the interval means ...”

### Day 8 exit check

You are ready for Day 9 when you can distinguish:

- parameter, estimate, error, and residual;
- probability and likelihood;
- residual standard deviation and coefficient standard error; and
- confidence interval for a mean from prediction interval for one new project.

## Day 9 — Gradient Descent from the OLS Gradient

**Today's central idea:** Gradient descent repeatedly moves parameters downhill. The OLS gradient derived in Chapter 1 tells us both the direction and the size of the local slope.

### 9.1 Why learn an iterative method when OLS has direct solvers?

For a moderate linear regression, QR or SVD is usually preferable. Gradient descent is introduced because the same optimisation logic extends to models that have no convenient closed-form solution and to datasets too large for some direct operations.

The learner should not conclude that iterative means better. The method must match the problem.

### 9.2 Choose the objective carefully

Use mean squared error as the training objective:

$$J(\beta) = \frac{1}{n} \|y - X\beta\|_2^2.$$

Chapter 1 derived the gradient of SSR. Dividing by  $n$  gives:

$$\nabla_{\beta} J(\beta) = \frac{2}{n} X^T (X\beta - y).$$

Shapes:

| Quantity          | Shape        |
|-------------------|--------------|
| $X$               | $n \times k$ |
| $\beta$           | $k$          |
| $X\beta - y$      | $n$          |
| $X^T(X\beta - y)$ | $k$          |

The gradient has one component per parameter.

## 9.3 The update rule

Starting from  $\beta^{(0)}$ , repeat:

$$\beta^{(t+1)} = \beta^{(t)} - \eta \nabla J(\beta^{(t)}),$$

where  $\eta > 0$  is the learning rate.

- The minus sign moves downhill.
- A very small  $\eta$  makes slow progress.
- A very large  $\eta$  can overshoot, oscillate, or diverge.
- Stopping after a fixed number of steps is simple but may stop too early or waste work.

## 9.4 A one-parameter numerical walk

For a through-origin model  $\hat{y} = \beta x$ :

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 4.0, 7.0])
beta = 0.0
learning_rate = 0.05

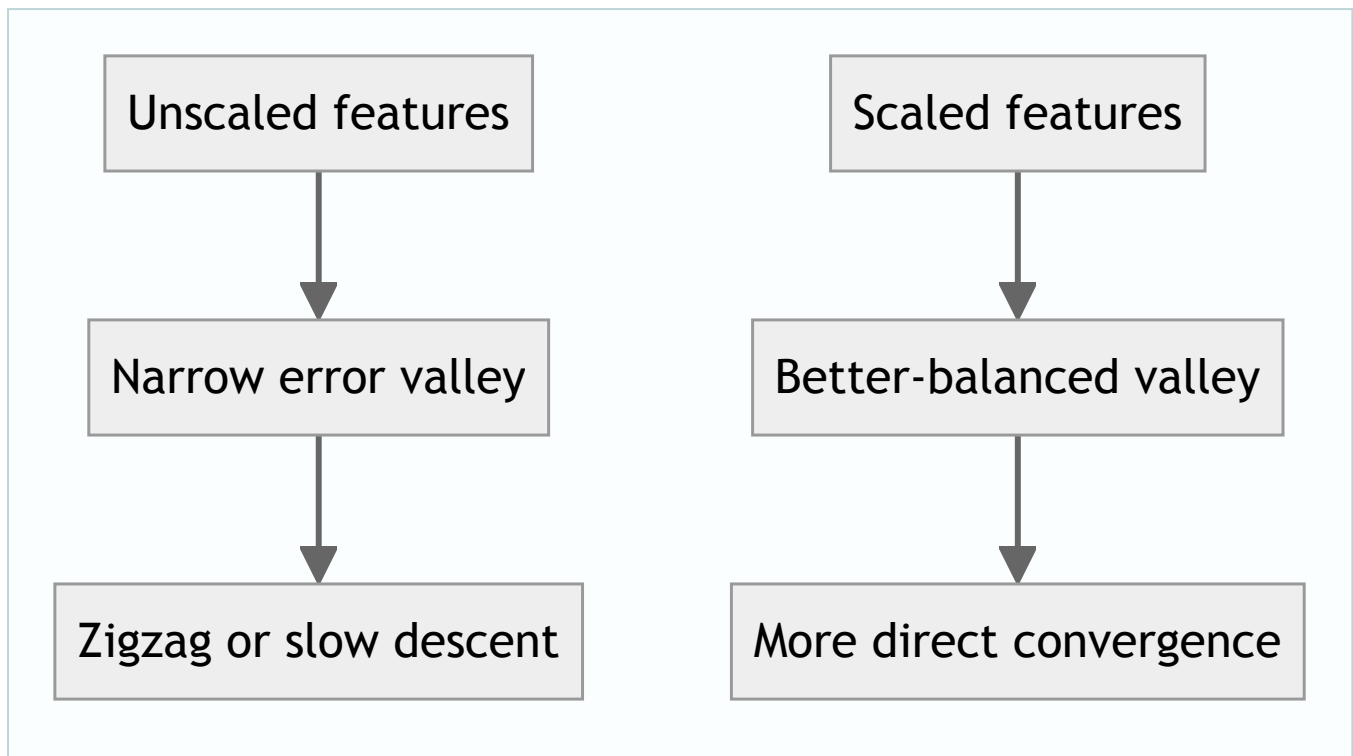
for step in range(20):
 predictions = beta * x
 gradient = (2.0 / x.size) * np.sum(x * (predictions - y))
 loss = np.mean((y - predictions) ** 2)
 print(f"step={step:02d} beta={beta:8.4f} loss={loss:8.4f}")
 beta = beta - learning_rate * gradient
```

Do not merely run it. Change the learning rate to `0.0001`, `0.2`, and `1.0`. Record the behaviour.

## 9.5 Why scaling changes the optimisation landscape

With two features, MSE forms a bowl in parameter space. If one feature is numerically much larger than another, the bowl can be long and narrow. A single learning rate then causes the algorithm to zigzag across the steep direction while crawling along the shallow direction.

Standardisation makes the curvature more balanced. It does not guarantee perfect convergence, but it often makes a useful learning rate easier to find.



## 9.6 Build batch gradient descent from scratch

```
import numpy as np

class GradientDescentOLS:
 def __init__(
 self,
 learning_rate=0.05,
 max_iter=20_000,
 tolerance=1e-10,
 fit_intercept=True,
):
 self.learning_rate = learning_rate
 self.max_iter = max_iter
 self.tolerance = tolerance
 self.fit_intercept = fit_intercept

 def _design(self, X):
 X = np.asarray(X, dtype=float)
 if X.ndim != 2:
 raise ValueError("X must be two-dimensional")
 if self.fit_intercept:
 return np.column_stack([np.ones(X.shape[0]), X])
 return X

 def fit(self, X, y):
 design = self._design(X)
 y = np.asarray(y, dtype=float)

 if y.ndim != 1 or y.size != design.shape[0]:
 raise ValueError("y must be one-dimensional and aligned with X")
 if not np.isfinite(design).all() or not np.isfinite(y).all():
 raise ValueError("X and y must contain finite values")

 n, k = design.shape
 beta = np.zeros(k)
 self.loss_history_ = []

 for iteration in range(self.max_iter):
```

```

 residual_direction = design @ beta - y
 loss = np.mean(residual_direction**2)
 gradient = (2.0 / n) * design.T @ residual_direction

 if not np.isfinite(loss) or not np.isfinite(gradient).all():
 raise FloatingPointError(
 "Optimisation diverged; inspect scaling and learning rate"
)

 self.loss_history_.append(loss)
 new_beta = beta - self.learning_rate * gradient

 if np.linalg.norm(new_beta - beta) < self.tolerance:
 beta = new_beta
 self.n_iter_ = iteration + 1
 break

 beta = new_beta
 else:
 self.n_iter_ = self.max_iter

 self.parameters_ = beta
 return self

def predict(self, X):
 if not hasattr(self, "parameters_"):
 raise RuntimeError("Fit the model before prediction")
 return self._design(X) @ self.parameters_

```

## 9.7 Code proof: gradient descent approaches the SVD solution

```

import numpy as np

rng = np.random.default_rng(91)
X_raw = rng.normal(size=(200, 3)) * np.array([500.0, 12.0, 2.0])
y = 15.0 + X_raw @ np.array([0.025, 0.9, 2.5]) + rng.normal(0.0, 1.0, size=200)

scaler = StandardScalerFromScratch()
X_scaled = scaler.fit_transform(X_raw)

gd = GradientDescentOLS(learning_rate=0.08, max_iter=50_000).fit(X_scaled, y)
svd = StableOLS().fit(X_scaled, y)

print("Iterations:", gd.n_iter_)
print("Gradient-descent parameters:", gd.parameters_)
print("SVD/lstsq parameters: ", svd.parameters_)
print("Final loss:", gd.loss_history_[-1])

assert np.allclose(gd.parameters_, svd.parameters_, atol=1e-5)

```

The two methods minimise the same convex objective. Their agreement is an important implementation test.

## 9.8 A finite-difference gradient check

A bug in the gradient can produce smooth-looking but wrong learning. Check the analytic gradient against small numerical perturbations:

```

import numpy as np

rng = np.random.default_rng(92)
X = np.column_stack([np.ones(30), rng.normal(size=(30, 2))])
y = rng.normal(size=30)

```

```

beta = np.array([0.4, -0.7, 1.1])

def mse(beta_vector):
 return np.mean((y - X @ beta_vector) ** 2)

analytic = (2.0 / X.shape[0]) * X.T @ (X @ beta - y)
numeric = np.zeros_like(beta)
h = 1e-6

for j in range(beta.size):
 unit = np.zeros_like(beta)
 unit[j] = 1.0
 numeric[j] = (mse(beta + h * unit) - mse(beta - h * unit)) / (2.0 * h)

print("Analytic gradient:", analytic)
print("Numerical gradient:", numeric)
assert np.allclose(analytic, numeric, rtol=1e-6, atol=1e-8)

```

## 9.9 Convergence diagnostics

Do not print only the final coefficients. Inspect the learning process:

```

import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(gd.loss_history_)
ax.set_xlabel("Iteration")
ax.set_ylabel("Training MSE")
ax.set_title("Gradient-descent learning curve")
ax.set_yscale("log")
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

Typical patterns:

| Pattern                                       | Possible meaning                        | Response                                                  |
|-----------------------------------------------|-----------------------------------------|-----------------------------------------------------------|
| Smooth decrease to a plateau                  | Convergence                             | Check gradient norm and agreement with <code>lstsq</code> |
| Very slow decrease                            | Learning rate too small or poor scaling | Scale features; adjust learning rate                      |
| Oscillation                                   | Learning rate too large                 | Reduce learning rate                                      |
| Exploding or non-finite loss                  | Divergence                              | Stop; inspect scale, sign, and gradient formula           |
| Training loss falls but validation loss rises | Overfitting or distribution mismatch    | Evaluation issue, not an optimiser victory                |

## 9.10 Batch, stochastic, and mini-batch gradient descent

The implementation above uses every training row for each gradient. This is **batch gradient descent**.

- **Stochastic gradient descent (SGD):** update using one randomly selected observation at a time.
- **Mini-batch gradient descent:** update using a small subset of rows.

Stochastic updates are noisy but cheap and can scale to large datasets. Their behaviour depends on data order, learning-rate schedules, batching, and randomness. Later chapters will study these topics in depth. For this chapter, the full-batch method keeps the equation and code directly aligned.

## 9.11 Parameters versus hyperparameters

This distinction becomes concrete here.

| Type           | Example                            | How obtained                        |
|----------------|------------------------------------|-------------------------------------|
| Parameter      | Intercept and feature coefficients | Learned by minimising training loss |
| Hyperparameter | Learning rate                      | Chosen outside the gradient update  |
| Hyperparameter | Maximum iterations                 | Chosen outside fitting              |
| Hyperparameter | Decision-tree depth                | Chosen during model selection       |

If the validation set guides the learning rate, it has influenced model selection. That is legitimate—provided the final test set remains separate.

## 9.12 Break it deliberately

Run the same problem under four conditions:

1. raw features with learning rate 0.08;
2. scaled features with learning rate 0.08;
3. scaled features with learning rate 2.0; and
4. scaled features with the gradient sign accidentally reversed.

For each case, record:

- first loss;
- final loss;
- number of iterations;
- whether values remained finite; and
- maximum coefficient difference from `np.linalg.lstsq`.

The aim is to make failure recognisable before it occurs in a more complicated model.

## 9.13 Day 9 build, break, and reflect

### Build

1. Fit scaled MHP numeric features with `GradientDescentOLS`.
2. Verify the gradient by finite differences.
3. Verify final coefficients and predictions against `StableOLS`.
4. Plot loss by iteration.

### Break

1. Remove scaling.
2. Increase the learning rate until loss diverges.
3. Stop after five iterations.
4. Explain why each final coefficient vector is or is not trustworthy.

### Reflect

Answer: if two algorithms reach the same minimum training MSE, have they demonstrated the same performance on new projects? Explain why the answer prepares the next day.

## Day 9 exit check

Write the MSE gradient and update rule from memory. Then identify which quantities are learned parameters and which are chosen hyperparameters.

# Day 10 — Generalisation, Baselines, and Model Complexity

**Today's central idea:** Machine learning is not the production of a small training error. It is the construction of a rule that performs usefully on new cases drawn from the conditions for which it is intended.

## 10.1 Training is an information boundary

Let a learning procedure receive training data  $D_{\text{train}}$  and return a fitted prediction function:

$$\hat{f} = \mathcal{A}(D_{\text{train}}).$$

Here  $\mathcal{A}$  includes more than the named algorithm. It may include:

- feature definitions;
- missing-value handling;
- scaling and encoding;
- model family;
- hyperparameter search;
- random seeds; and
- model-selection rules.

The whole procedure learns from data. An honest evaluation must reproduce that procedure using only the permitted training information.

## 10.2 Empirical risk and generalisation risk

For a loss function  $L$ , training—or empirical—risk is:

$$\hat{R}_{\text{train}}(f) = \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} L(y_i, f(x_i)).$$

The population generalisation risk is:

$$R(f) = \mathbb{E}_{(X,Y) \sim P_{\text{deployment}}} [L(Y, f(X))].$$

The expectation is over the deployment distribution—the projects the model will actually face. That distribution is not visible in full. A held-out sample is useful only insofar as its construction represents the deployment question.



## 10.3 The constant baseline is a real model

A baseline is not an insult to the data. It defines the minimum comparison a feature-using model must beat.

For a constant squared-error predictor  $c$ , minimise:

$$S(c) = \sum_{i=1}^n (y_i - c)^2.$$

Differentiate:

$$\frac{dS}{dc} = -2 \sum_i (y_i - c).$$

Set the derivative to zero:

$$\sum_i y_i - nc = 0 \quad \Rightarrow \quad \hat{c} = \bar{y}.$$

Therefore the training mean is the best constant prediction under squared error.

For absolute error:

$$A(c) = \sum_i |y_i - c|,$$

any sample median minimises the objective. Intuitively, moving  $c$  right reduces the error for points to its right but increases the error for points to its left; a median balances those counts.

## 10.4 Code proof: search for the best constant

```
import numpy as np

y = np.array([10.0, 12.0, 13.0, 14.0, 60.0])
candidates = np.linspace(0.0, 70.0, 70_001)

squared_losses = np.array([np.mean((y - c) ** 2) for c in candidates])
absolute_losses = np.array([np.mean(np.abs(y - c)) for c in candidates])

best_squared = candidates[np.argmin(squared_losses)]
best_absolute = candidates[np.argmin(absolute_losses)]

print("Mean:", y.mean(), "grid squared-error optimum:", best_squared)
print("Median:", np.median(y), "grid absolute-error optimum:", best_absolute)

assert np.isclose(best_squared, y.mean(), atol=0.001)
assert np.isclose(best_absolute, np.median(y), atol=0.001)
```

The outlying value 60 pulls the mean much more than the median. Baseline choice and evaluation loss should match the decision problem.

## 10.5 What “better than baseline” means

Fit the baseline on the training target only:

$$\hat{y}_i^{\text{baseline}} = \bar{y}_{\text{train}}.$$

Then evaluate both baseline and candidate model on the same held-out rows with the same metric.

A candidate that fails to beat the baseline may have:

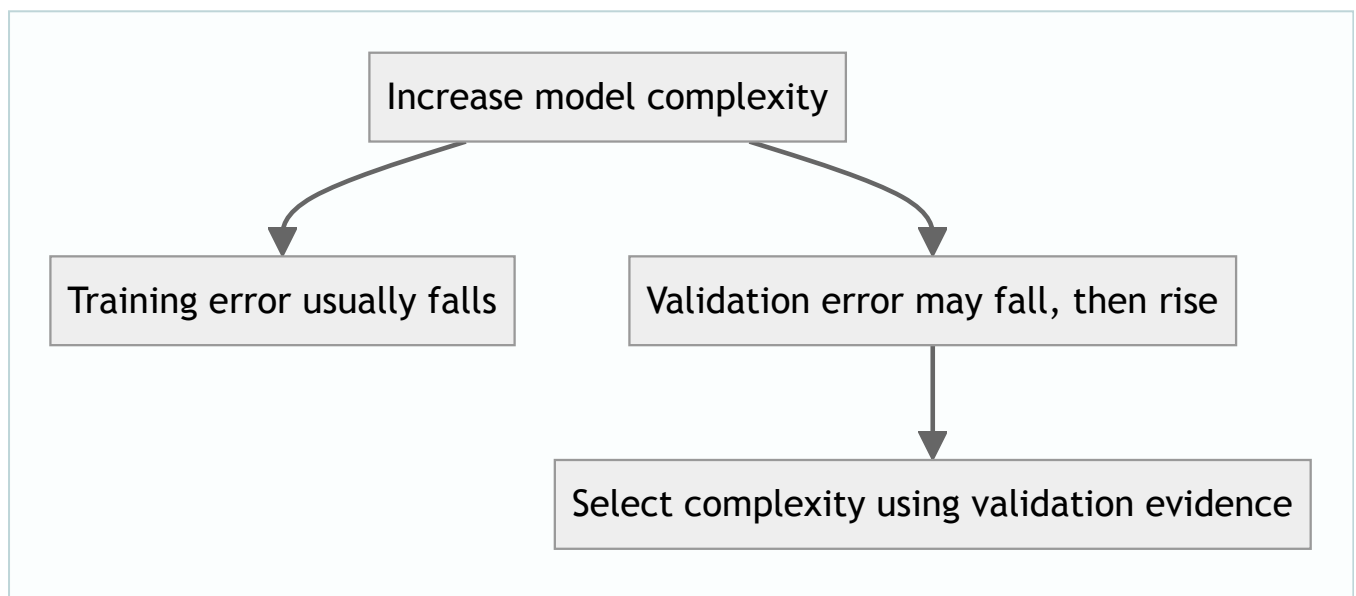
- weak or irrelevant features;
- incorrect data alignment;
- distribution shift;
- excessive complexity;
- insufficient training data; or
- leakage in an earlier, misleading experiment.

Failure to beat a baseline is a diagnostic result, not permission to hide the baseline.

## 10.6 Underfitting and overfitting

**Underfitting** occurs when the learned rule is too restricted to capture useful structure. Both training and validation errors may be high.

**Overfitting** occurs when a flexible procedure adapts to idiosyncrasies of the training sample that do not repeat. Training error can be low while validation error is high.



The model with the smallest training error is almost never a defensible automatic choice.

## 10.7 Bias–variance decomposition: a first research-level view

Assume squared-error prediction at a fixed input  $\mathbf{x}_0$  and a data-generating relationship:

$$Y = f(\mathbf{x}_0) + \varepsilon, \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2.$$

Imagine repeatedly drawing new training datasets and fitting  $\hat{f}$ . The expected test error at  $\mathbf{x}_0$  decomposes as:

$$\mathbb{E} \left[ (Y - \hat{f}(\mathbf{x}_0))^2 \right] = \underbrace{\left( \mathbb{E}[\hat{f}(\mathbf{x}_0)] - f(\mathbf{x}_0) \right)^2}_{\text{squared bias}} + \underbrace{\text{Var}(\hat{f}(\mathbf{x}_0))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}.$$

Step-by-step:

1. Add and subtract  $\mathbb{E}[\hat{f}(\mathbf{x}_0)]$  inside the error.
2. Expand the square.

3. Cross-terms vanish under the stated expectations.
4. The remaining terms describe systematic miss, training-sample sensitivity, and outcome noise.

This is an expectation over repeated datasets, not something fully observed from one train/test split. “High bias” and “high variance” should not be assigned casually from a single plot.

## 10.8 A shallow decision tree as a nonlinear comparator

A regression tree divides feature space into regions. At a node, it considers a feature  $j$  and threshold  $s$ :

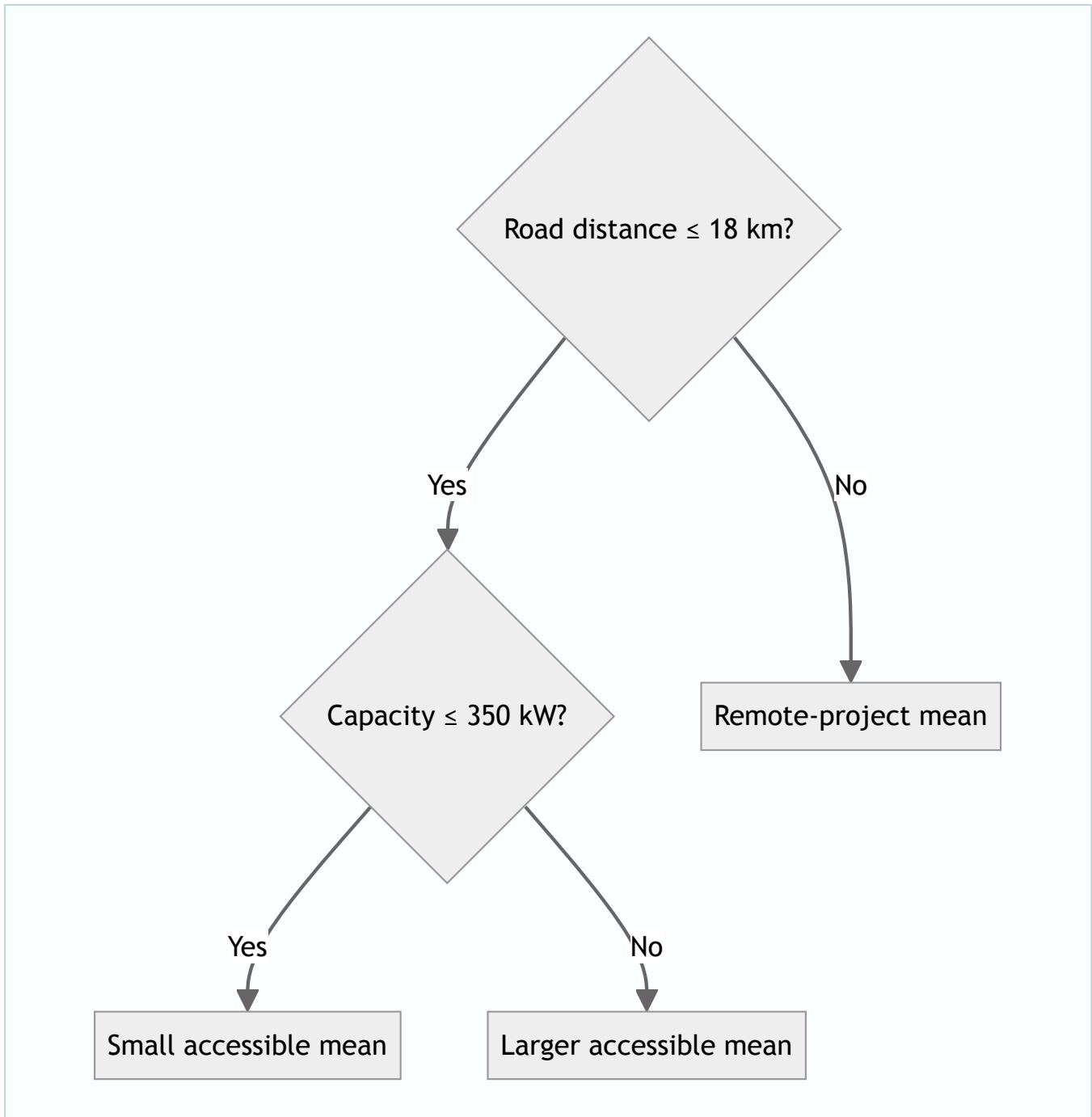
$$R_{\text{left}}(j, s) = \{x : x_j \leq s\},$$

$$R_{\text{right}}(j, s) = \{x : x_j > s\}.$$

For squared error, each leaf predicts the mean target among its training observations. A candidate split is chosen to minimise the combined within-child squared error:

$$\sum_{i: x_i \in R_{\text{left}}} (y_i - \bar{y}_{\text{left}})^2 + \sum_{i: x_i \in R_{\text{right}}} (y_i - \bar{y}_{\text{right}})^2.$$

A tree is a piecewise-constant model. It can represent thresholds and interactions without manually adding polynomial or interaction columns.



The displayed thresholds are illustrative. A fitted tree learns thresholds from its training data.

## 10.9 Why a shallow tree is a useful baseline

`max_depth` limits the number of successive splits along a path.

- Depth 0 is effectively one constant leaf.
- Small depth captures broad nonlinear structure.
- Large depth can create tiny leaves and memorise training observations.

Unlike linear regression, ordinary trees do not extrapolate a slope beyond the observed range. A leaf predicts a constant learned from its training region. This can be safer in some settings and badly limiting in others.

## 10.10 First library comparison

The following demonstration uses a single split only to make the API visible. Day 11 will decide which split is defensible.

```
import numpy as np
import pandas as pd
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

from chapter2_data import make_mhp_projects

df = make_mhp_projects()
features = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_km",
 "terrain_index",
]
target = "actual_cost_2025_million_pkr"

X_train, X_holdout, y_train, y_holdout = train_test_split(
 df[features],
 df[target],
 test_size=0.25,
 random_state=42,
)

models = {
 "training mean": DummyRegressor(strategy="mean"),
 "linear regression": LinearRegression(),
 "tree depth 3": DecisionTreeRegressor(max_depth=3, random_state=42),
 "unrestricted tree": DecisionTreeRegressor(random_state=42),
}

for name, model in models.items():
 model.fit(X_train, y_train)
 train_mae = mean_absolute_error(y_train, model.predict(X_train))
 holdout_mae = mean_absolute_error(y_holdout, model.predict(X_holdout))
 print(f"{name:20s} train MAE={train_mae:6.2f} holdout MAE={holdout_mae:6.2f}")
```

Expect the unrestricted tree to have extremely low training error. Whether it has the lowest holdout error is a separate empirical question.

## 10.11 Learning curves: is more data likely to help?

A learning curve plots performance as training size grows. It can reveal:

- a persistent high training and validation error: model/features may underfit;
- a large gap that narrows with more data: variance may be important;
- a gap that remains wide: the procedure may be too flexible or groups may be leaking; and
- unstable curves: sample size or split structure may be inadequate.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

Use a fixed chronological holdout for this illustration.
ordered = df.sort_values("start_year")
train_pool = ordered[ordered["start_year"] <= 2022]
```

```

future_holdout = ordered[ordered["start_year"] >= 2023]

fractions = np.linspace(0.2, 1.0, 8)
train_scores = []
future_scores = []

for fraction in fractions:
 subset = train_pool.sample(frac=fraction, random_state=42)
 model = LinearRegression().fit(subset[features], subset[target])
 train_scores.append(
 mean_absolute_error(subset[target], model.predict(subset[features]))
)
 future_scores.append(
 mean_absolute_error(
 future_holdout[target], model.predict(future_holdout[features])
)
)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(fractions * len(train_pool), train_scores, marker="o", label="training")
ax.plot(fractions * len(train_pool), future_scores, marker="o", label="future holdout")
ax.set_xlabel("Training projects")
ax.set_ylabel("MAE (million PKR)")
ax.set_title("Learning curve under a future-project holdout")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

Because each point uses only one sampled subset, the curve itself is noisy. A fuller analysis repeats the sampling or uses cross-validation appropriate to the deployment structure.

## 10.12 Research paper discussion 1: Breiman’s “Two Cultures”

**Paper:** Leo Breiman (2001), “Statistical Modeling: The Two Cultures”, *Statistical Science* 16(3), 199–231.

### The question

Breiman argued that data analysis contained two broad traditions:

- a **data-modeling culture**, which specifies a stochastic model and studies its parameters; and
- an **algorithmic modeling culture**, which treats the data-generating mechanism as largely unknown and prioritises predictive performance.

### The argument

He criticised excessive reliance on convenient stochastic models when their adequacy was weakly checked, and pressed for stronger use of predictive evaluation and algorithmic methods.

### What a beginner should learn

1. A transparent equation is not automatically true because it is interpretable.
2. High predictive accuracy does not automatically explain a mechanism or identify a cause.
3. Model checking must include out-of-sample evidence when the purpose is prediction.
4. The unit of comparison is a full modeling procedure, not mathematical elegance alone.

### Application to the MHP case

A planning unit may need both cultures:

- a linear or probabilistic model to communicate cost relationships and uncertainty; and
- an algorithmic comparator, such as a shallow tree, to test whether important nonlinear structure is being missed.

Neither purpose authorises a causal claim that improving a particular feature will reduce cost. That still requires a causal design.

## Limitation and debate

The two-cultures framing is intentionally provocative. Modern practice often combines prediction, structured probability models, domain knowledge, and causal reasoning. Read the paper and its published discussion as an argument that reshaped a debate, not as a command to choose one permanent camp.

## Reproduction prompt

Compare OLS and trees over increasing depth using a fixed, defensible validation design. Plot training and validation MAE. Identify where flexibility improves generalisation and where it begins fitting sample-specific noise.

## 10.13 Day 10 build, break, and reflect

### Build

1. Fit the mean and median baselines.
2. Fit OLS, gradient-descent OLS, a depth-3 tree, and an unrestricted tree.
3. Report training and held-out errors separately.
4. Draw a learning curve.

### Break

1. Select the model with the smallest training MAE.
2. Add the final material bill as a feature.
3. Increase tree depth until training error is nearly zero.
4. Explain why each action can create false confidence.

### Reflect

Write one paragraph answering:

Is the purpose of this MHP model to explain cost formation, predict new costs, or both? What evidence would each purpose require?

## Day 10 exit check

You should now be able to derive the mean baseline, explain why the median baseline differs, and identify a situation in which the best training model is the worst responsible choice.

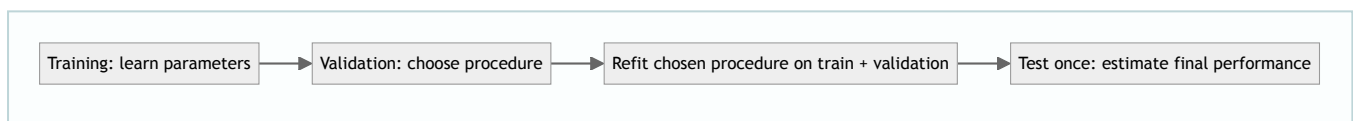
---

# Day 11 — Honest Splitting, Leakage, and Cross-Validation

**Today's central idea:** A split is a simulation of deployment. Choose it by asking what will be new when the model is used: a new project, a new district, a future period, or some combination.

## 11.1 Three roles for data

| Partition  | Permitted use                                      | Not permitted                                         |
|------------|----------------------------------------------------|-------------------------------------------------------|
| Training   | Fit preprocessing and model parameters             | Final performance claim                               |
| Validation | Choose features, model family, and hyperparameters | Repeated use followed by calling it an untouched test |
| Test       | One final estimate after the procedure is fixed    | Model tuning, threshold choice, feature revision      |



The exact percentages are not universal. A 60/20/20 split is a teaching example, not a law. Sample size, group structure, time, and the cost of uncertainty determine a defensible design.

## 11.2 Random split: new exchangeable projects

A random split asks whether the fitted procedure generalises to new observations assumed to arise from roughly the same distribution as the training observations.

It may be reasonable when:

- observations are independent enough for the purpose;
- there are no repeated entities spanning both partitions;
- time ordering is irrelevant; and
- deployment covers the same mixture of districts and conditions.

It is not the default answer to every dataset.

## 11.3 Group split: new independent groups

Projects within one district may share roads, contractors, administrative practices, survey teams, and shocks. A random split can place highly related projects on both sides. Performance then partly measures recognition of shared group structure.

If deployment is to a district absent from training, hold out entire districts:

$$G_{\text{train}} \cap G_{\text{test}} = \emptyset.$$

The group used for splitting need not be an input feature. It identifies dependence or the unit of intended transfer.



## 11.4 Temporal split: future conditions

When predicting future projects, preserve time order:

$$\max(t_{\text{train}}) < \min(t_{\text{test}}).$$

An expanding-window evaluation is often useful:

Fold 1: train 2016-2018; validate 2019

Fold 2: train 2016-2019; validate 2020

Fold 3: train 2016-2020; validate 2021

Fold 4: train 2016-2021; validate 2022

Future prediction can fail because relationships, measurement, procurement, technology, or the mixture of projects changes. This is distribution shift, not merely random sampling noise.

## 11.5 A deployment-question table

| Intended deployment                                                   | Evaluation design                                       |
|-----------------------------------------------------------------------|---------------------------------------------------------|
| New projects in already represented districts under stable conditions | Random split may be adequate                            |
| New projects in a district not represented in training                | Group holdout by district                               |
| Projects initiated in later years                                     | Temporal split or forward-chaining validation           |
| Future projects in unseen districts                                   | Combined time-and-group design; expect high uncertainty |
| Repeated measurements of the same project                             | Keep all rows from a project in one partition           |

When the desired deployment has little or no analogue in the data, no clever splitter can manufacture evidence. The correct conclusion may be that external validation data are needed.

## 11.6 Leakage: information crosses the boundary

Leakage occurs when the training or selection process receives information it would not legitimately possess at deployment or final evaluation.

### Target leakage

A feature contains the target or a post-outcome proxy. The final material bill is an obvious MHP example.

### Preprocessing leakage

Means, standard deviations, imputation values, categories, feature selection, or transformations are fitted using validation/test rows.

### Temporal leakage

Future information helps predict the past, or a feature is recorded after the prediction date.

### Group leakage

Related observations from the same district, household, patient, contractor, or project appear in both train and evaluation sets when the intended deployment requires new groups.

## Duplicate leakage

Exact or near-duplicate records cross partitions.

## Test-set leakage

The analyst repeatedly examines test performance and changes the procedure. The test set becomes an informal validation set.

## Label leakage through data cleaning

Even “cleaning” can leak. For example, deleting unusual feature rows only after observing that their targets are hard to predict uses outcome information to define the sample.

## 11.7 Break it: a leaky feature creates a spectacular score

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split

from chapter2_data import make_mhp_projects

df = make_mhp_projects()
target = "actual_cost_2025_million_pkr"
legitimate = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_km",
 "terrain_index",
]
leaky = legitimate + ["final_material_bill_million_pkr"]

train_idx, test_idx = train_test_split(
 df.index, test_size=0.25, random_state=42
)

for label, columns in [("legitimate", legitimate), ("leaky", leaky)]:
 model = LinearRegression().fit(df.loc[train_idx, columns], df.loc[train_idx, target])
 predictions = model.predict(df.loc[test_idx, columns])
 mae = mean_absolute_error(df.loc[test_idx, target], predictions)
 print(label, "MAE:", round(mae, 3))
```

The leaky score is not a better appraisal model. It answers a different and largely useless question: how well can final cost be reconstructed after much of the cost is already known?

## 11.8 Pipelines protect the fitting boundary

A pipeline joins learned preprocessing and the model so that each cross-validation training fold fits its own transformations.

```
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = [
 "planned_capacity_kw",
```

```

 "estimated_cable_km",
 "road_distance_km",
 "terrain_index",
]
categorical_features = ["district"]

numeric_pipeline = Pipeline(
 steps=[
 ("impute", SimpleImputer(strategy="median")),
 ("scale", StandardScaler()),
]
)

categorical_pipeline = Pipeline(
 steps=[
 ("impute", SimpleImputer(strategy="most_frequent")),
 (
 "one_hot",
 OneHotEncoder(handle_unknown="ignore", drop="first"),
),
]
)

preprocessor = ColumnTransformer(
 transformers=[
 ("numeric", numeric_pipeline, numeric_features),
 ("categorical", categorical_pipeline, categorical_features),
]
)

linear_pipeline = Pipeline(
 steps=[
 ("preprocess", preprocessor),
 ("model", LinearRegression()),
]
)

```

With an intercept, retaining every one-hot column would make one column an exact sum of the others. `drop="first"` removes that exact linear dependence—the same rank-deficiency problem derived in §7.8—while treating the dropped category as the reference level.

The order is `fit` preprocessing on the current training fold, `transform` that fold, fit the model, and transform the corresponding validation fold using the learned training-fold values.

## 11.9 K-fold cross-validation

In ***K***-fold cross-validation:

1. divide the development data into ***K*** folds;
2. hold out fold 1 and train on the other ***K* − 1** folds;
3. evaluate on fold 1;
4. repeat so every fold is held out once; and
5. aggregate the held-out losses.

For fold losses  $L_1, \dots, L_K$ :

$$\hat{L}_{CV} = \frac{1}{K} \sum_{k=1}^K L_k.$$

Cross-validation reuses observations for efficient development, but each prediction is generated by a model that did not train on that observation.

## 11.10 Cross-validation is a family, not one method

| Splitter        | Preserves                                      | Appropriate question                        |
|-----------------|------------------------------------------------|---------------------------------------------|
| KFold           | Fold separation                                | New approximately independent observations  |
| GroupKFold      | Group separation                               | New groups                                  |
| TimeSeriesSplit | Time order                                     | Future observations under expanding history |
| Nested CV       | Separation between tuning and outer evaluation | Performance of a selected procedure         |

Stratification is mainly discussed in classification. For regression, analysts sometimes bin the target for approximate balance, but target-informed splitting must be designed cautiously and documented. It does not replace group or temporal structure.

## 11.11 Grouped cross-validation in code

```
import numpy as np
from sklearn.model_selection import GroupKFold, cross_validate

df = make_mhp_projects()
X = df[numeric_features + categorical_features]
y = df["actual_cost_2025_million_pkr"]
groups = df["district"]

group_cv = GroupKFold(n_splits=df["district"].nunique())

scores = cross_validate(
 linear_pipeline,
 X,
 y,
 groups=groups,
 cv=group_cv,
 scoring={
 "mae": "neg_mean_absolute_error",
 "mse": "neg_mean_squared_error",
 "r2": "r2",
 },
 return_train_score=True,
)

mae = -scores["test_mae"]
rmse = np.sqrt(-scores["test_mse"])

print("Held-out district MAE values:", mae)
print("Mean MAE:", mae.mean())
print("Held-out district RMSE values:", rmse)
print("Held-out district R2 values:", scores["test_r2"])
```

Do not report only the mean. Each fold corresponds to a district and may reveal an operationally important failure.

## 11.12 Temporal validation in code

```
import numpy as np
from sklearn.base import clone
from sklearn.metrics import mean_absolute_error

df = make_mhp_projects().sort_values("start_year")
feature_columns = numeric_features + categorical_features
validation_years = [2019, 2020, 2021, 2022]
```

```

forward_mae = []

for year in validation_years:
 train = df[df["start_year"] < year]
 validation = df[df["start_year"] == year]

 model = clone(linear_pipeline)
 model.fit(train[feature_columns], train["actual_cost_2025_million_pkr"])
 predictions = model.predict(validation[feature_columns])
 score = mean_absolute_error(
 validation["actual_cost_2025_million_pkr"], predictions
)
 forward_mae.append(score)
 print(year, "MAE:", round(score, 3), "validation n:", len(validation))

print("Mean forward-validation MAE:", np.mean(forward_mae))

```

The folds have different training sizes and time periods. Their losses are not identically distributed replications. Inspecting the sequence can be more informative than compressing it to one average.

## 11.13 Hyperparameter selection and the validation set

For a decision tree, consider depths **1, 2, 3, 4, 6, 8**. Selecting the depth with the best validation score is a data-dependent operation:

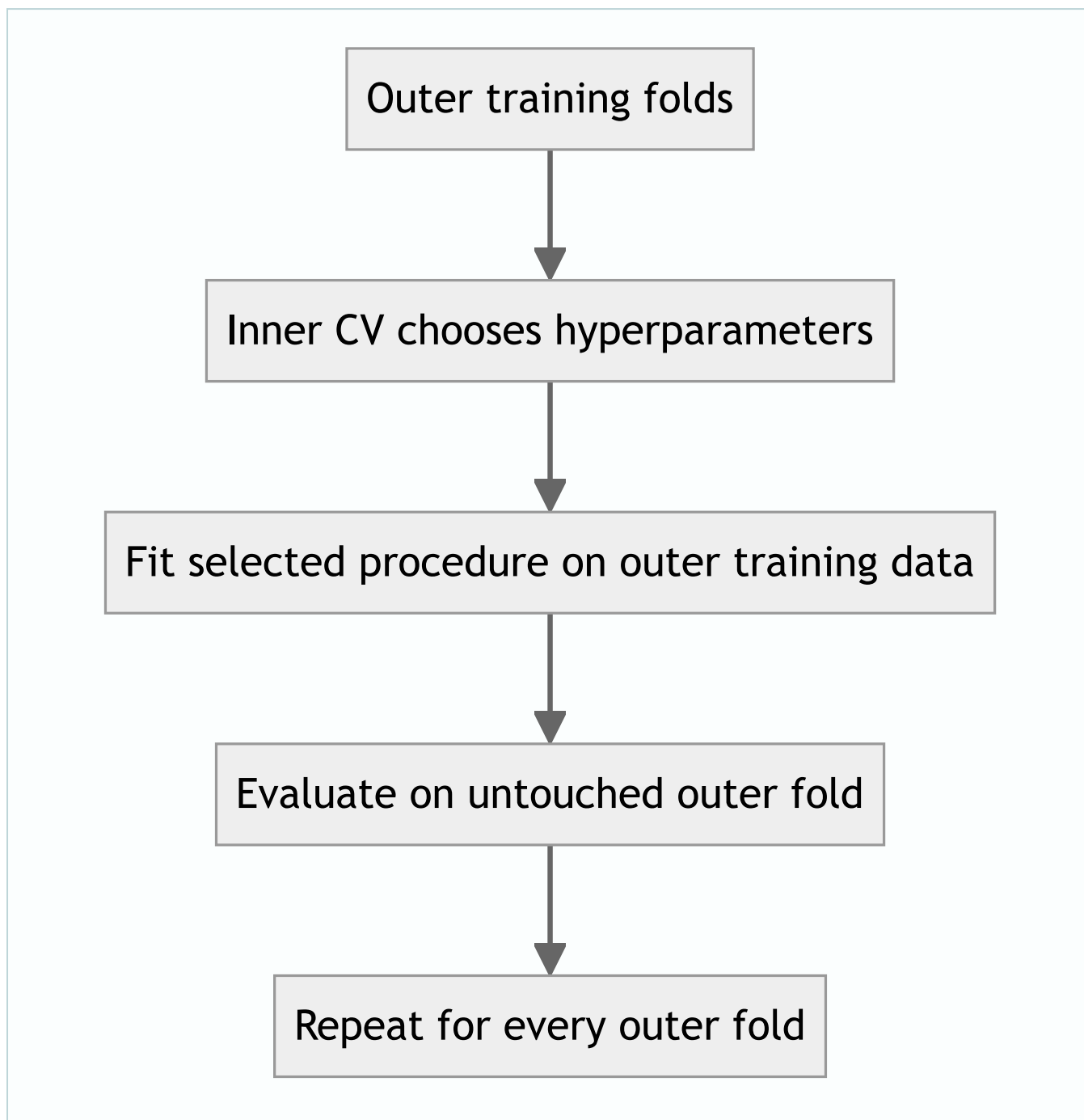
$$\hat{d} = \arg \min_{d \in \mathcal{D}} \hat{L}_{\text{validation}}(d).$$

If the same validation data are used to report final performance, the estimate is optimistically biased because the best-looking setting partly benefited from validation noise.

## 11.14 Nested cross-validation

Nested CV separates two loops:

- the **inner loop** selects hyperparameters;
- the **outer loop** estimates the performance of the entire selection procedure.



The object being evaluated is not “a depth-3 tree” chosen in advance. It is “the procedure that searches specified depths using specified inner folds and then refits the winner.”

## 11.15 Manual nested group cross-validation

This explicit version keeps the mechanics visible.

Before combining pipelines, grouped folds, cloning, and nesting, isolate what `GridSearchCV` does with one plain estimator:

```
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.tree import DecisionTreeRegressor

small_search = GridSearchCV(
 estimator=DecisionTreeRegressor(random_state=42),
```

```

 param_grid={"max_depth": [1, 2, 3, 4]},
 scoring="neg_mean_absolute_error",
 cv=KFold(n_splits=5, shuffle=True, random_state=42),
 refit=True,
)
small_search.fit(X[numeric_features], y)
print(small_search.best_params_)
print(-small_search.best_score_)

```

This tries four depths, scores each through five validation folds, and refits the winning depth on all supplied development rows. The full example below keeps that same search operation but places it inside an outer evaluation loop, swaps ordinary folds for district-preserving folds, and searches a complete preprocessing-plus-model pipeline.

```

import numpy as np
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import GroupKFold, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeRegressor

df = make_mhp_projects()
feature_columns = numeric_features + categorical_features
X = df[feature_columns]
y = df["actual_cost_2025_million_pkr"]
groups = df["district"]

tree_pipeline = Pipeline(
 steps=[
 ("preprocess", clone(preprocessor)),
 ("model", DecisionTreeRegressor(random_state=42)),
]
)

outer_cv = GroupKFold(n_splits=5)
outer_results = []

for outer_fold, (outer_train_idx, outer_test_idx) in enumerate(
 outer_cv.split(X, y, groups), start=1
):
 X_outer_train = X.iloc[outer_train_idx]
 y_outer_train = y.iloc[outer_train_idx]
 groups_outer_train = groups.iloc[outer_train_idx]
 X_outer_test = X.iloc[outer_test_idx]
 y_outer_test = y.iloc[outer_test_idx]

 # With four districts in the outer training portion, use four inner folds.
 inner_cv = GroupKFold(n_splits=groups_outer_train.nunique())
 search = GridSearchCV(
 estimator=clone(tree_pipeline),
 param_grid={
 "model__max_depth": [1, 2, 3, 4, 6],
 "model__min_samples_leaf": [5, 10, 20],
 },
 scoring="neg_mean_absolute_error",
 cv=inner_cv,
 refit=True,
)
 search.fit(
 X_outer_train,
 y_outer_train,
 groups=groups_outer_train,
)

 outer_predictions = search.predict(X_outer_test)
 outer_mae = mean_absolute_error(y_outer_test, outer_predictions)

```

```

held_out_district = groups.iloc[outer_test_idx].unique().tolist()

outer_results.append(outer_mae)
print(
 f"outer fold={outer_fold}",
 f"held out={held_out_district}",
 f"best={search.best_params_}",
 f"MAE={outer_mae:.3f}",
)

print("Nested group-CV mean MAE:", np.mean(outer_results))

```

This is computationally more expensive because selection is repeated inside each outer training set. That repetition is the protection, not wasted work.

## 11.16 Research paper discussion 2: Cawley and Talbot on selection bias

**Paper:** Gavin C. Cawley and Nicola L. C. Talbot (2010), “On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation”, *Journal of Machine Learning Research* 11, 2079–2107.

### The question

Can the model-selection criterion itself be overfit, and can this produce biased performance comparisons?

### The key argument

When many candidate settings are compared, random variation in the selection criterion becomes an opportunity. The selected candidate can look best partly because it benefited from noise. The paper shows that this selection overfitting can be comparable in magnitude to reported differences between algorithms.

### The important change in the unit of comparison

The paper argues that empirical comparisons should evaluate the combination:

**learning algorithm + model-selection procedure,**

not the named algorithm in isolation.

For the MHP example, “decision tree” is incomplete. A reproducible procedure includes:

- candidate depths and leaf sizes;
- split type;
- inner scoring metric;
- preprocessing;
- refitting rule; and
- random-state policy.

### Practical implication

Use an untouched test set after tuning or use nested cross-validation when data are too limited for a single validation holdout. Keep the entire preprocessing and selection process inside the appropriate loop.



## Limitation

Nested CV addresses selection bias under the sampled evaluation design. It does not fix a wrong deployment simulation, unmeasured shift, leakage in feature definitions, or a sample that excludes the hardest projects.

## Reproduction prompt

On repeated synthetic datasets, search increasingly large collections of irrelevant hyperparameters. Compare the best inner-CV score with outer held-out performance. Plot the optimism as the search space grows.

## 11.17 A locked-test protocol

Before opening the final test result, write and timestamp an evaluation contract containing:

1. target and units;
2. prediction time;
3. eligible features;
4. exclusion rules;
5. split design;
6. candidate procedures;
7. selection metric;
8. subgroup reports;
9. uncertainty method; and
10. conditions that would block deployment.

If the test result triggers model revision, that is allowed—but the old test set has become development information. A new independent test is then needed for a fresh final claim.

## 11.18 Day 11 build, break, and reflect

### Build

1. Write the deployment question before splitting.
2. Compare random, district-grouped, and temporal validation.
3. Put all learned preprocessing inside a pipeline.
4. Tune a shallow tree with an inner loop.
5. Preserve projects from 2023 onward as a locked final test.

### Break

1. Scale before cross-validation.
2. include the final material bill.
3. let the same district appear in both sides of an unseen-district evaluation.
4. choose tree depth using the test set.
5. explain exactly how information crosses the boundary in each case.

### Reflect

For each proposed deployment below, name the split and one remaining limitation:

- a new project in Swat next month;
- a new district never represented in the database; and
- MHP projects five years after a procurement-policy change.

## Day 11 exit check

You are ready for Day 12 when you can draw the inner and outer loops of nested CV, state what each loop is allowed to influence, and explain why a pipeline is part of statistical validity rather than code tidiness.

# Day 12 — Metrics, Diagnostics, and Responsible Revision

**Today's central idea:** A metric compresses errors according to a value judgement. Responsible evaluation keeps the individual errors available, measures uncertainty, looks for structured failure, and revises the model without contaminating the final test.

## 12.1 Begin with signed errors

For a held-out observation:

$$e_i = y_i - \hat{y}_i.$$

With this convention:

- $e_i > 0$ : the actual cost exceeded the prediction, so the model **underpredicted**;
- $e_i < 0$ : the prediction exceeded the actual cost, so the model **overpredicted**.

The mean signed error is:

$$ME = \frac{1}{n} \sum_i e_i.$$

Positive and negative misses cancel, so ME is useful for directional bias but not as a total accuracy measure.

## 12.2 Mean absolute error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

If MAE is 4.2 million PKR, the average magnitude of the held-out miss is 4.2 million PKR. MAE weights an error of 10 twice as much as an error of 5.

MAE is not immune to extreme values; every extreme error still enters. It is simply less dominated by them than a squared-error metric.

## 12.3 Mean squared error and root mean squared error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

$$RMSE = \sqrt{MSE}.$$

RMSE returns to the target's units. Because errors are squared before averaging, large misses receive disproportionate weight.

Do not describe RMSE as “the average error” without qualification. It is the square root of the mean squared error, not the arithmetic mean of absolute misses.

## 12.4 Median absolute error

$$MedAE = \text{median}(|y_i - \hat{y}_i|).$$

MedAE describes a typical central miss and can remain small even when a minority of projects fail badly. Reporting it beside MAE and a high error quantile makes that contrast visible.

For example:

$$Q_{0.90}(|e|)$$

is the 90th percentile of absolute error: 90% of held-out projects have an absolute miss at or below that value.

## 12.5 $R^2$ on held-out data

For evaluation outcomes  $y_1, \dots, y_n$ :

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{\text{evaluation}})^2}.$$

Interpretation on that evaluation set:

- $R^2 = 1$ : perfect predictions;
- $R^2 = 0$ : the candidate's squared error equals that of predicting the **evaluation-set mean** for every evaluation row;
- $R^2 < 0$ : the candidate is worse, in squared-error terms, than that evaluation-set-mean reference.

This contains an important correction to a common simplification. The standard held-out  $R^2$  denominator uses the mean of the evaluation targets, whereas a deployable dummy model predicts a value learned from the training targets. Under distribution shift, those are not the same. Always score an explicit training-fitted baseline rather than assuming  $R^2 = 0$  represents its exact held-out performance.

$R^2$  is dimensionless and sensitive to the range of outcomes. The same absolute errors can yield different  $R^2$  values in two districts with different target variation.

## 12.6 Calculate every metric from raw definitions

```
import numpy as np

y_true = np.array([20.0, 24.0, 31.0, 45.0, 80.0])
y_pred = np.array([22.0, 20.0, 34.0, 37.0, 95.0])

errors = y_true - y_pred
absolute_errors = np.abs(errors)
squared_errors = errors**2

me = np.mean(errors)
mae = np.mean(absolute_errors)
```

```

mse = np.mean(squared_errors)
rmse = np.sqrt(mse)
medae = np.median(absolute_errors)
r2 = 1.0 - np.sum(squared_errors) / np.sum((y_true - y_true.mean()) ** 2)

print("Errors:", errors)
print("ME:", me)
print("MAE:", mae)
print("RMSE:", rmse)
print("Median AE:", medae)
print("R²:", r2)

```

Verify the same values with library functions, but keep the definition code. It makes sign, units, and denominators auditable.

## 12.7 Why MAPE is dangerous for infrastructure costs

Mean absolute percentage error is:

$$MAPE = \frac{100}{n} \sum_i \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

Problems:

1. it is undefined when  $y_i = 0$ ;
2. it becomes extremely large when  $y_i$  is near zero;
3. it weights the same absolute miss more heavily for small projects; and
4. it can encourage asymmetric behaviour because over- and underprediction interact differently with the denominator.

Percentage error may be relevant if proportional miss is truly the decision cost, but it should not be used automatically because stakeholders like percentages.

## 12.8 Scale-aware alternatives

No single alternative is universally best.

- Report MAE separately by project scale.
- Divide aggregate absolute error by aggregate actual cost when that portfolio interpretation is intended:

$$WAPE = \frac{\sum_i |y_i - \hat{y}_i|}{\sum_i |y_i|}.$$

- Model log cost when multiplicative structure is substantively plausible, while handling retransformation carefully.
- Use a scaled error relative to an explicit baseline when comparing series.

The metric must reflect the decision, not merely avoid a mathematical inconvenience.

## 12.9 Asymmetric decision loss

Suppose underbudgeting is more damaging than overbudgeting. MAE and RMSE are symmetric: errors  $+a$  and  $-a$  receive the same penalty.

One simple asymmetric absolute loss is:

$$L(e) = \begin{cases} c_{\text{under}}|e|, & e > 0, \\ c_{\text{over}}|e|, & e \leq 0, \end{cases}$$

with  $c_{\text{under}} > c_{\text{over}}$ .

Quantile—or pinball—loss for quantile level  $\tau$  is:

$$L_{\tau}(y, \hat{q}) = \begin{cases} \tau(y - \hat{q}), & y \geq \hat{q}, \\ (1 - \tau)(\hat{q} - y), & y < \hat{q}. \end{cases}$$

Predicting a high conditional quantile, such as  $\tau = 0.8$ , can support contingency planning. It should be labelled as an 80th-quantile estimate, not confused with a mean forecast plus an arbitrary percentage.

## 12.10 Metric choice is separate from training loss

The model may minimise squared training loss but be selected by validation MAE and reported with several test metrics. These are different roles:

- **training loss** determines fitted parameters;
- **selection metric** chooses among procedures;
- **reporting metrics** describe consequences to readers.

Changing a reporting metric after seeing which one favours a preferred model is another form of researcher flexibility. Choose primary and secondary metrics before final evaluation.

## 12.11 Always retain a prediction table

Create one row per held-out prediction:

| Field                      | Purpose                         |
|----------------------------|---------------------------------|
| Project ID                 | Trace the source record         |
| Actual target              | Verify outcome and unit         |
| Prediction                 | Inspect the estimate            |
| Signed error               | Detect under- or overprediction |
| Absolute error             | Rank large misses               |
| Squared error              | Understand RMSE contribution    |
| District                   | Examine spatial performance     |
| Start year                 | Examine temporal drift          |
| Project scale / remoteness | Examine operational subgroups   |

Aggregate metrics can always be recomputed from this table. They cannot reconstruct the table after detail has been discarded.

## 12.12 Residual diagnostics on held-out predictions

Training residuals diagnose fit to the training sample. Held-out errors diagnose prediction under the evaluation design. Both matter, but they answer different questions.

## Error versus prediction

Look for curvature, a fan shape, and systematic offset.

## Error over time

Look for drift after procurement, technology, policy, or price-basis changes.

## Error by district

Look for groups with poor representation or different relationships.

## Error by project scale

A low overall MAE may be dominated by numerous small projects while a few large projects carry most budget risk.

## Distribution of signed errors

Look for asymmetry and extreme tails. A normal-looking histogram is neither necessary nor sufficient for generalisation.

## 12.13 A four-panel diagnostic figure

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def plot_prediction_diagnostics(results):
 """Plot held-out diagnostics from a prediction-results DataFrame."""
 required = {
 "prediction",
 "error",
 "absolute_error",
 "district",
 "start_year",
 }
 missing = required.difference(results.columns)
 if missing:
 raise ValueError(f"Missing diagnostic columns: {sorted(missing)}")

 fig, axes = plt.subplots(2, 2, figsize=(12, 8))

 axes[0, 0].scatter(results["prediction"], results["error"], alpha=0.7)
 axes[0, 0].axhline(0.0, color="black", linewidth=1)
 axes[0, 0].set_xlabel("Predicted cost")
 axes[0, 0].set_ylabel("Actual - predicted")
 axes[0, 0].set_title("Signed error versus prediction")

 by_year = results.groupby("start_year")["error"].mean()
 axes[0, 1].plot(by_year.index, by_year.values, marker="o")
 axes[0, 1].axhline(0.0, color="black", linewidth=1)
 axes[0, 1].set_xlabel("Start year")
 axes[0, 1].set_ylabel("Mean signed error")
 axes[0, 1].set_title("Directional error over time")

 district_order = (
 results.groupby("district")["absolute_error"].median().sort_values().index
)
 district_data = [
 results.loc[results["district"] == district, "absolute_error"]
 for district in district_order
]
```

```

axes[1, 0].boxplot(district_data, tick_labels=district_order, vert=True)
axes[1, 0].tick_params(axis="x", rotation=30)
axes[1, 0].set_ylabel("Absolute error")
axes[1, 0].set_title("Error distribution by district")

axes[1, 1].hist(results["error"], bins=15, edgecolor="black")
axes[1, 1].axvline(0.0, color="black", linewidth=1)
axes[1, 1].set_xlabel("Signed error")
axes[1, 1].set_ylabel("Projects")
axes[1, 1].set_title("Held-out error distribution")

fig.tight_layout()
return fig

```

Plots are question generators. A visible pattern suggests investigation; it does not identify the cause by itself.

## 12.14 Subgroup evaluation without false certainty

For each operationally relevant group, report:

- sample count;
- MAE;
- mean signed error;
- median absolute error; and
- a high absolute-error quantile.

Small subgroups have noisy estimates. Do not rank districts as if a difference based on five projects were a stable performance league table. Use intervals, domain knowledge, and replication.

```

def subgroup_report(results, group_column):
 return (
 results.groupby(group_column)
 .agg(
 n=("error", "size"),
 mean_error=("error", "mean"),
 mae=("absolute_error", "mean"),
 median_ae=("absolute_error", "median"),
 q90_ae=("absolute_error", lambda x: x.quantile(0.90)),
)
 .sort_values("mae", ascending=False)
)

```

## 12.15 Bootstrap uncertainty for held-out MAE

One fixed test set gives one observed MAE. A nonparametric bootstrap approximates its sampling variability by resampling the held-out prediction rows with replacement.

```

import numpy as np

def bootstrap_mae_interval(y_true, y_pred, confidence=0.95, repeats=5000, seed=42):
 y_true = np.asarray(y_true, dtype=float)
 y_pred = np.asarray(y_pred, dtype=float)

 if y_true.ndim != 1 or y_true.shape != y_pred.shape:
 raise ValueError("y_true and y_pred must be aligned one-dimensional arrays")
 if y_true.size < 2:
 raise ValueError("At least two held-out observations are required")

 rng = np.random.default_rng(seed)

```

```

n = y_true.size
bootstrap_scores = np.empty(repeats)

for b in range(repeats):
 indices = rng.integers(0, n, size=n)
 bootstrap_scores[b] = np.mean(
 np.abs(y_true[indices] - y_pred[indices])
)

alpha = 1.0 - confidence
lower, upper = np.quantile(
 bootstrap_scores,
 [alpha / 2.0, 1.0 - alpha / 2.0],
)

return {
 "estimate": np.mean(np.abs(y_true - y_pred)),
 "lower": lower,
 "upper": upper,
 "bootstrap_scores": bootstrap_scores,
}

```

This percentile interval treats held-out rows as the resampling units. If projects are clustered within districts or repeated within contractors, row-wise resampling can understate dependence. A cluster bootstrap resamples independent groups instead. The resampling unit must match the data-generating and deployment structure.

## 12.16 Why “mean $\pm$ fold standard deviation” is not a confidence interval

Cross-validation training sets overlap, so fold scores are dependent. They may also correspond to different districts or time periods. Their standard deviation is a descriptive measure of variation across those folds; it is not automatically the standard error of the mean and not automatically a 95% interval.

Even dividing fold standard deviation by  $\sqrt{K}$  does not repair the dependence by assumption.

## 12.17 Research paper discussion 3: Bengio and Grandvalet on CV variance

**Paper:** Yoshua Bengio and Yves Grandvalet (2004), “No Unbiased Estimator of the Variance of K-Fold Cross-Validation”, *Journal of Machine Learning Research* 5, 1089–1105.

### The question

Can one construct a universally unbiased estimator of the variance of the  $K$ -fold cross-validation error using the usual cross-validation results?

### Main result

The paper proves that no universal unbiased estimator exists under the broad conditions it studies. Overlap among training sets creates covariance terms that cannot generally be recovered from the observed fold results alone.

### What the result does not say

It does **not** say that cross-validation is useless or that uncertainty can never be studied. It says a very convenient, universally unbiased variance formula is unavailable.



## Practical implication

- Do not label fold standard deviation as a confidence interval.
- Preserve fold-level results and describe the split structure.
- Use repeated or nested designs, independent test data, bootstrap methods, or model-based uncertainty only with their assumptions stated.
- Treat small reported differences between procedures cautiously.

## Application to the MHP case

Five group folds may be the five districts. Variation across those folds is partly real geographic heterogeneity, not five interchangeable measurements of one constant error. Report each district's result and ask whether the deployment population weights districts equally, by project count, or by budget exposure.

## Reproduction prompt

Repeat  $K$ -fold CV on many independently generated synthetic datasets. Compare variation across folds within one dataset with variation of the mean CV estimate across datasets. They are not the same quantity.

## 12.18 Diagnostics do not authorise test-set tuning

There is a tension:

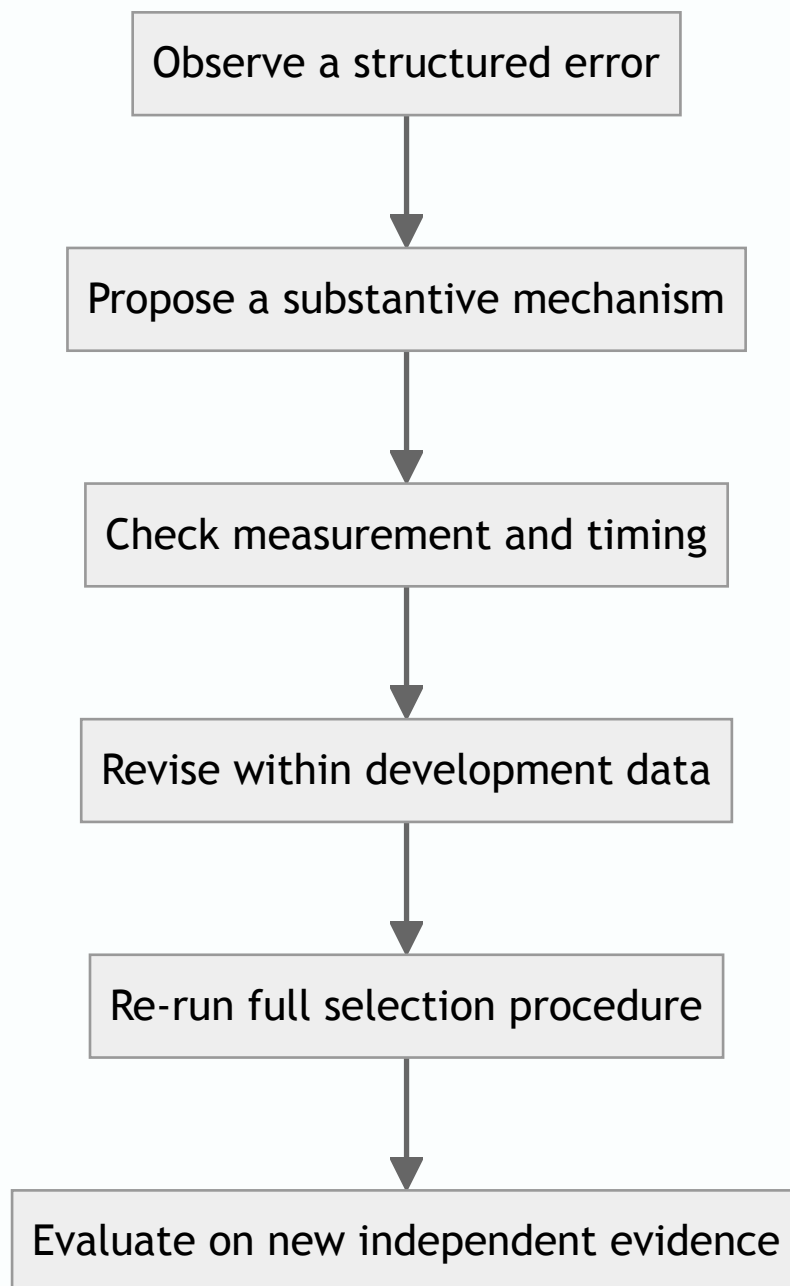
- final test diagnostics are valuable for learning where a procedure fails;
- changing the procedure in response means the test has influenced development.

The correct workflow is:

1. report the pre-specified final result;
2. use diagnostics to generate a revised hypothesis;
3. label the revision as post-test development; and
4. obtain new independent evaluation data or wait for prospective deployment evidence.

Do not quietly overwrite the original result and reuse the same test set as if it remained untouched.

## 12.19 A responsible model-revision protocol



Examples:

- Curvature with capacity may justify a pre-specified squared-capacity feature or nonlinear model.
- Larger variance for remote sites may justify conditional quantile modeling or variance modeling.
- Persistent error after a policy change may justify a time indicator, but only if that indicator will be known and meaningful at deployment.
- One district's poor performance may reveal measurement inconsistency rather than a need to add district identity.

## 12.20 Day 12 build, break, and reflect

**Build**

1. Generate a held-out prediction table.
2. Calculate ME, MAE, RMSE, MedAE,  $R^2$ , and the 90th percentile absolute error from definitions.
3. Score the explicit training-fitted dummy baseline.
4. Produce the four diagnostic panels.
5. report metrics by district, remoteness, and project scale.
6. bootstrap an MAE interval using the appropriate resampling unit.

### Break

1. Report MAPE after adding a near-zero target.
2. Evaluate only the numerous small projects and claim portfolio-wide success.
3. call fold standard deviation a 95% confidence interval.
4. change the model after viewing test diagnostics and reuse the same score as final.

### Reflect

Write a one-page evaluation note that begins with the decision and ends with a deployment recommendation. A table of metrics without that reasoning is not sufficient.

## Day 12 exit check

You are ready for the capstone when you can answer:

1. Which direction of signed error represents underprediction?
2. What reference does held-out  $R^2$  use?
3. Why can low overall MAE coexist with unacceptable remote-project performance?
4. Why is fold standard deviation not automatically a confidence interval?
5. What must happen after test-driven model revision?

---

# Chapter 2 Capstone — An Auditable MHP Evaluation Pipeline

---

## Capstone brief

The planning unit wants an appraisal-stage estimate of final cost in constant 2025 million PKR for projects initiated from 2023 onward. The initial dataset includes projects from 2016 through 2025.

Your job is not merely to return the smallest error. You must design and document the entire learning procedure.

## Pass 1: write the evaluation contract

Before running models, state:

- unit of observation;
- target and price basis;
- prediction time;
- eligible and prohibited features;
- deployment population;
- primary metric and why it matches the decision;

- validation design;
- locked test period;
- subgroup reports; and
- deployment-blocking conditions.

## Pass 2: audit the data boundary

Confirm:

- project IDs are not features;
- final material bills are excluded;
- preprocessing is learned only inside training partitions;
- years are ordered correctly;
- duplicates do not cross partitions; and
- every categorical value is handled without inspecting test outcomes.

## Pass 3: compare complete procedures

Compare:

1. training-mean baseline;
2. linear regression with imputation, scaling, and one-hot encoding;
3. a shallow decision tree with depth selected on 2022 validation data.

Use MAE as the primary selection metric. Retain RMSE, median absolute error,  $R^2$ , and mean signed error as secondary reports.

## Pass 4: lock, select, refit, test once

- Training: 2016–2021.
- Validation: 2022.
- Final test: 2023–2025.
- Select the procedure using validation MAE only.
- Refit the selected procedure on training plus validation data.
- Evaluate once on the final test.

## Pass 5: diagnose without rewriting history

Report:

- overall metrics and bootstrap MAE interval;
- performance by district;
- performance for road distance above and below 18 km;
- the ten largest absolute errors; and
- any evidence of temporal drift.

If diagnostics motivate revision, label it as the start of the next development cycle.

## Complete runnable capstone

Save as `mhp_chapter2_capstone.py`. It generates its own fictional data, so the chapter remains reproducible without a private dataset.

```
import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
 mean_absolute_error,
 mean_squared_error,
 median_absolute_error,
 r2_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeRegressor

from chapter2_data import make_mhp_projects

TARGET = "actual_cost_2025_million_pkr"
NUMERIC_FEATURES = [
 "start_year",
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_km",
 "terrain_index",
]
CATEGORICAL_FEATURES = ["district"]
FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES

def make_preprocessor():
 numeric_pipeline = Pipeline(
 steps=[
 ("impute", SimpleImputer(strategy="median")),
 ("scale", StandardScaler()),
]
)
 categorical_pipeline = Pipeline(
 steps=[
 ("impute", SimpleImputer(strategy="most_frequent")),
 (
 "one_hot",
 OneHotEncoder(handle_unknown="ignore", drop="first"),
),
]
)

 # Dropping one category prevents exact dependence among the intercept and
 # one-hot columns; see the rank-deficiency argument in §7.8.
 return ColumnTransformer(
 transformers=[
 ("numeric", numeric_pipeline, NUMERIC_FEATURES),
 ("categorical", categorical_pipeline, CATEGORICAL_FEATURES),
]
)

def make_pipeline(regressor):
 return Pipeline(
 steps=[
 ("preprocess", make_preprocessor()),
```

```

 ("model", regressor),
]
)

def regression_metrics(y_true, y_pred):
 y_true = np.asarray(y_true, dtype=float)
 y_pred = np.asarray(y_pred, dtype=float)
 error = y_true - y_pred
 return {
 "ME": np.mean(error),
 "MAE": mean_absolute_error(y_true, y_pred),
 "RMSE": np.sqrt(mean_squared_error(y_true, y_pred)),
 "MedAE": median_absolute_error(y_true, y_pred),
 "R2": r2_score(y_true, y_pred),
 "Q90_AE": np.quantile(np.abs(error), 0.90),
 }

def bootstrap_mae_interval(
 y_true,
 y_pred,
 confidence=0.95,
 repeats=5000,
 seed=42,
):
 y_true = np.asarray(y_true, dtype=float)
 y_pred = np.asarray(y_pred, dtype=float)
 rng = np.random.default_rng(seed)
 scores = np.empty(repeats)

 for b in range(repeats):
 indices = rng.integers(0, y_true.size, size=y_true.size)
 scores[b] = np.mean(np.abs(y_true[indices] - y_pred[indices]))

 alpha = 1.0 - confidence
 return np.quantile(scores, [alpha / 2.0, 1.0 - alpha / 2.0])

def prediction_table(frame, predictions):
 result = frame[
 [
 "project_id",
 "district",
 "start_year",
 "road_distance_km",
 TARGET,
]
].copy()
 result["prediction"] = predictions
 result["error"] = result[TARGET] - result["prediction"]
 result["absolute_error"] = result["error"].abs()
 result["squared_error"] = result["error"] ** 2
 result["remote"] = np.where(
 result["road_distance_km"] > 18.0,
 "above 18 km",
 "18 km or less",
)
 return result

def subgroup_report(results, group_column):
 return (
 results.groupby(group_column)
 .agg(
 n=("error", "size"),
 mean_error=("error", "mean"),
 mae=("absolute_error", "mean"),
 median_ae=("absolute_error", "median"),
 q90_ae=("absolute_error", lambda values: values.quantile(0.90)),
)
)

```

```

)
 .sort_values("mae", ascending=False)
)

def main():
 df = make_mhp_projects()

 # Prediction-time audit.
 prohibited = {
 "project_id",
 "final_material_bill_million_pkr",
 TARGET,
 }
 assert prohibited.isdisjoint(FEATURES)
 assert not df["project_id"].duplicated().any()

 train = df[df["start_year"] <= 2021].copy()
 validation = df[df["start_year"] == 2022].copy()
 test = df[df["start_year"] >= 2023].copy()

 assert train["start_year"].max() < validation["start_year"].min()
 assert validation["start_year"].max() < test["start_year"].min()

 print(
 "Split sizes:",
 {"train": len(train), "validation": len(validation), "test": len(test)},
)

 candidates = {
 "training mean": make_pipeline(DummyRegressor(strategy="mean")),
 "linear regression": make_pipeline(LinearRegression()),
 }

 for depth in [1, 2, 3, 4, 6]:
 candidates[f"tree depth {depth}"] = make_pipeline(
 DecisionTreeRegressor(
 max_depth=depth,
 min_samples_leaf=10,
 random_state=42,
)
)

 validation_rows = []
 fitted_candidates = {}

 for name, procedure in candidates.items():
 fitted = clone(procedure).fit(train[FEATURES], train[TARGET])
 predictions = fitted.predict(validation[FEATURES])
 metrics = regression_metrics(validation[TARGET], predictions)
 validation_rows.append({"procedure": name, **metrics})
 fitted_candidates[name] = fitted

 validation_report = pd.DataFrame(validation_rows).sort_values("MAE")
 print("\nValidation results")
 print(validation_report.to_string(index=False, float_format=lambda x: f"{x:.3f}"))

 selected_name = validation_report.iloc[0]["procedure"]
 print("\nSelected by validation MAE:", selected_name)

 # The procedure is now fixed. Combine development data and refit it.
 development = pd.concat([train, validation], ignore_index=True)
 selected_procedure = clone(candidates[selected_name])
 selected_procedure.fit(development[FEATURES], development[TARGET])

 # Open the final test once.
 test_predictions = selected_procedure.predict(test[FEATURES])
 test_metrics = regression_metrics(test[TARGET], test_predictions)
 print("\nFinal test metrics")
 for metric, value in test_metrics.items():

```

```

print(f"{metric:7s}: {value:8.3f}")

lower, upper = bootstrap_mae_interval(test[TARGET], test_predictions)
print(f"95% row-bootstrap interval for test MAE: [{lower:.3f}, {upper:.3f}]")

results = prediction_table(test, test_predictions)

print("\nBy district")
print(subgroup_report(results, "district").to_string(float_format=lambda x: f"{x:.3f}"))

print("\nBy remoteness")
print(subgroup_report(results, "remote").to_string(float_format=lambda x: f"{x:.3f}"))

print("\nTen largest absolute errors")
columns = [
 "project_id",
 "district",
 "start_year",
 TARGET,
 "prediction",
 "error",
 "absolute_error",
]
print(
 results.nlargest(10, "absolute_error")[columns].to_string(
 index=False,
 float_format=lambda x: f"{x:.3f}",
)
)

results.to_csv("mhp_chapter2_test_predictions.csv", index=False)

if __name__ == "__main__":
 main()

```

## Capstone interpretation questions

1. Did the selected procedure beat the training-mean baseline on validation and test data?
2. Was the validation ranking preserved on the test period? Why might it change?
3. Is the test MAE small relative to actual planning tolerances, not merely smaller than another model?
4. Which district and remoteness group had the largest errors? Are sample sizes sufficient for a strong conclusion?
5. Is mean signed error positive, suggesting underprediction, or negative, suggesting overprediction?
6. Are later test years systematically underpredicted?
7. Would adding the final material bill improve the numerical score? Why is it prohibited?
8. If the tree wins, does that make its feature thresholds causal or stable outside the observed period?
9. If linear regression wins, does that prove the true cost mechanism is linear?
10. What new independent evidence would be required after revising the model in response to test diagnostics?

## Capstone assessment rubric

| Dimension           | Emerging         | Competent                               | Research-ready habit                                                   |
|---------------------|------------------|-----------------------------------------|------------------------------------------------------------------------|
| Prediction contract | Target named     | Timing, units, and use stated           | Feature availability and deployment population audited                 |
| Numerical work      | Model runs       | Scaling, rank, and conditioning checked | Solver and tolerance choices justified                                 |
| Probability         | Interval printed | Assumptions and interval type stated    | Dependence, heteroskedasticity, and extrapolation limitations examined |



| Dimension       | Emerging         | Competent                                    | Research-ready habit                                     |
|-----------------|------------------|----------------------------------------------|----------------------------------------------------------|
| Optimisation    | Loss falls       | Gradient verified against finite differences | Convergence compared with decomposition solution         |
| Validation      | One random split | Split matches deployment                     | Selection procedure evaluated without leakage            |
| Metrics         | One score        | Baseline and multiple pre-specified metrics  | Decision loss, uncertainty, and subgroup risk integrated |
| Diagnostics     | Plot produced    | Patterns interpreted cautiously              | Revision hypotheses separated from confirmatory evidence |
| Reproducibility | Code shown       | Seed, versions, and data recipe recorded     | Full procedure can be rerun and claims traced to outputs |
| Communication   | Winner named     | Magnitude and limitations explained          | Deployment recommendation includes blocking conditions   |

## Formula Sheet

| Concept                  | Equation                                                | Meaning                                                   |
|--------------------------|---------------------------------------------------------|-----------------------------------------------------------|
| Standardisation          | $z_{ij} = (x_{ij} - \mu_j)/s_j$                         | Centre and scale using training statistics                |
| Condition number         | $\kappa_2(X) = \sigma_{\max}/\sigma_{\min}$             | Sensitivity associated with unequal singular directions   |
| QR least squares         | $R\hat{\beta} = Q^T y$                                  | Solve after $X = QR$                                      |
| SVD                      | $X = U\Sigma V^T$                                       | Rotations plus singular-value stretching                  |
| Pseudoinverse solution   | $\hat{\beta} = V\Sigma^+ U^T y$                         | Minimum-norm least-squares solution under rank deficiency |
| Linear probability model | $y = X\beta + \epsilon$                                 | Systematic mean plus unobserved error                     |
| Gaussian errors          | $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$              | Classical equal-variance independent normal model         |
| Gaussian log-likelihood  | $C - \frac{n}{2} \log \sigma^2 - \frac{SSR}{2\sigma^2}$ | Maximising over $\beta$ equals minimising SSR             |
| Residual variance        | $s^2 = SSR/(n - k)$                                     | Unbiased classical estimator under assumptions            |
| Coefficient covariance   | $s^2(X^T X)^{-1}$                                       | Classical estimated sampling covariance                   |
| Coefficient interval     | $\hat{\beta}_j \pm t^* SE(\hat{\beta}_j)$               | Repeated-sampling confidence procedure                    |
| Mean-response SE         | $s\sqrt{x_0^T (X^T X)^{-1} x_0}$                        | Uncertainty in estimated conditional mean                 |
| Prediction SE            | $s\sqrt{1 + x_0^T (X^T X)^{-1} x_0}$                    | Adds new-project outcome noise                            |
| MSE objective            | $J(\beta) = \ y - X\beta\ ^2/n$                         | Average squared training loss                             |
| MSE gradient             | $\nabla J = 2X^T(X\beta - y)/n$                         | Direction of steepest increase                            |
| Gradient update          | $\beta^{(t+1)} = \beta^{(t)} - \eta \nabla J$           | Iterative downhill step                                   |
| Squared-error baseline   | $\hat{c} = \bar{y}_{train}$                             | Optimal training constant under squared loss              |
| Absolute-error baseline  | $\hat{c} = \text{median}(y_{train})$                    | Optimal training constant under absolute loss             |

| Concept      | Equation                                   | Meaning                                           |
|--------------|--------------------------------------------|---------------------------------------------------|
| CV loss      | $\hat{L}_{CV} = K^{-1} \sum_k L_k$         | Mean held-out fold loss                           |
| Mean error   | $n^{-1} \sum_i (y_i - \hat{y}_i)$          | Directional error                                 |
| MAE          | $n^{-1} \sum_i  y_i - \hat{y}_i $          | Linear penalty in target units                    |
| RMSE         | $\sqrt{n^{-1} \sum_i (y_i - \hat{y}_i)^2}$ | Quadratically weighted error in target units      |
| $R^2$        | $1 - SSR/TSS$                              | Squared-error comparison with evaluation-set mean |
| Pinball loss | Piecewise loss weighted by $\tau$          | Trains or scores a conditional quantile           |

## Glossary

**Algorithm:** A computational procedure. A full learning procedure also includes preprocessing, tuning, and selection.

**Baseline:** A simple reference predictor that a candidate procedure should be compared against.

**Bias–variance decomposition:** Under squared loss, a repeated-sampling decomposition of expected prediction error into squared bias, estimator variance, and irreducible noise.

**Bootstrap:** A resampling method that approximates sampling variability by repeatedly sampling observed units with replacement.

**Condition number:** A measure of how strongly perturbations can be amplified in a numerical problem.

**Confidence interval:** An interval produced by a repeated-sampling procedure designed to cover a fixed parameter at a stated long-run rate under assumptions.

**Cross-validation:** A resampling design in which observations or groups take turns serving as held-out development data.

**Data leakage:** Illegitimate movement of information across prediction-time, training, selection, or evaluation boundaries.

**Degrees of freedom:** The number of independent pieces of information remaining after fitted constraints are accounted for in a stated model.

**Distribution shift:** A difference between training and deployment distributions.

**Empirical risk:** Average loss on an observed dataset.

**Error term:** Unobserved deviation from the model's population conditional mean.

**Extrapolation:** Prediction beyond feature regions well represented in the training data.

**Generalisation:** Performance of a learned procedure on genuinely new cases from its intended deployment conditions.

**Heteroskedasticity:** Conditional error variance that changes across observations or feature values.

**Hyperparameter:** A setting chosen outside the parameter-fitting calculation.

**Ill-conditioning:** Sensitivity in which small changes in data or arithmetic can cause large changes in a solution.

**Likelihood:** A function of parameter values with observed data held fixed, derived from a probability model.

**Maximum likelihood:** Parameter selection that maximises the likelihood under a specified model.

**Nested cross-validation:** An outer evaluation loop surrounding an inner model-selection loop.

**Numerical rank:** The number of singular directions treated as nonzero at a stated scale and tolerance.

**Overfitting:** Adaptation to sample-specific patterns that do not generalise.

**Parameter:** A value learned inside a fitted model, such as a regression coefficient.

**Pipeline:** A single procedure that fits transformations and a model within the correct data boundary.

**Prediction interval:** An interval intended to cover an individual future outcome under a stated model; it includes outcome noise as well as mean-estimation uncertainty.

**Pseudoinverse:** A generalised inverse used to express least-squares solutions, including rank-deficient cases.

**QR decomposition:** A factorisation of a matrix into an orthonormal-column matrix  $Q$  and upper-triangular matrix  $R$ .

**Residual:** Observed target minus fitted prediction.

**Selection bias in evaluation:** Optimism created when data used to select a procedure also influence its reported performance.

**Singular value:** A nonnegative scale factor in the SVD that measures the strength of a matrix direction.

**Test set:** Data reserved for final evaluation after the learning and selection procedure has been fixed.

**Training set:** Data permitted to fit learned transformations and model parameters.

**Underfitting:** Failure of a procedure to represent useful structure.

**Validation set:** Development data used to compare procedures and choose hyperparameters.

---

## Research Reading Guide

---

The three paper discussions in this chapter are not decorative citations. Use this six-question template whenever reading them.

1. **Question:** What exact problem is the paper trying to resolve?
2. **Object:** Is it studying a parameter, predictor, algorithm, selection procedure, or evaluation estimate?
3. **Design:** Is the evidence a theorem, simulation, benchmark, observational dataset, or case study?
4. **Result:** What did the authors actually establish?
5. **Boundary:** Under what assumptions or settings might the result not transfer?
6. **Consequence:** What should change in our MHP workflow if we accept the result?

## Paper comparison table

| Paper                      | Main object                              | Central warning                                                                              | Chapter action                                                      |
|----------------------------|------------------------------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Breiman (2001)             | Modeling culture and predictive practice | Interpretable stochastic models can be weakly checked; prediction deserves direct evaluation | Compare transparent and algorithmic models while preserving purpose |
| Cawley & Talbot (2010)     | Model-selection procedure                | Selection itself can overfit and bias performance reports                                    | Use untouched tests or nested CV; compare complete procedures       |
| Bengio & Grandvalet (2004) | Variance of $K$ -fold CV                 | No universal unbiased variance estimator follows from usual folds                            | Do not turn fold SD into an automatic confidence interval           |

## A small replication portfolio

To move from textbook reader toward researcher, complete three reproducible notebooks:

1. **Complexity notebook:** regenerate 100 datasets, fit trees of increasing depth, and estimate training/validation optimism.
2. **Selection notebook:** increase the number of tried hyperparameter combinations and measure inner-to-outer performance gaps.
3. **CV uncertainty notebook:** compare within-dataset fold variation with across-dataset variation in average CV performance.

Each notebook must contain:

- a pre-written question;
- a data-generating process;
- seeds and software versions;
- an expected result stated before execution;
- figures with labelled axes and units;
- a result that distinguishes observation from interpretation; and
- one limitation that could change the conclusion.

---

## Instructor and Self-Study Notes

---

### Suggested daily timebox

| Activity                            | Minutes |
|-------------------------------------|---------|
| Retrieve Chapter 1 knowledge        | 15      |
| Work through the central derivation | 40      |
| Run raw proof code                  | 35      |
| Create or inspect the figure        | 25      |
| Extend the estimator or evaluation  | 40      |
| Break and diagnose                  | 30      |
| Exit check and written reflection   | 15      |

| Activity | Minutes |
|----------|---------|
| Total    | 200     |

Day 11 may require two sessions because nested validation is conceptually dense. Splitting it across sessions is better than treating the inner and outer loops as incantations.

## Retrieval prompts for the next morning

- After Day 6: “What does a condition number measure, and why must a scaler be fit on training data?”
- After Day 7: “How do QR and SVD solve least squares without explicitly inverting  $\mathbf{X}^T \mathbf{X}$ ?”
- After Day 8: “Why does Gaussian maximum likelihood produce OLS, and which interval is wider?”
- After Day 9: “Write the MSE gradient and explain the learning rate.”
- After Day 10: “Derive the mean baseline and define generalisation risk.”
- After Day 11: “Which loop selects and which loop evaluates in nested CV?”
- After Day 12: “Why is one average metric insufficient for deployment?”

## Common beginner confusions to diagnose

| Statement                                                    | Correction                                                                              |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| “Scaling adds information.”                                  | It changes representation, not information content.                                     |
| “Full rank means stable coefficients.”                       | A full-rank matrix can be severely ill-conditioned.                                     |
| “SVD fixes multicollinearity.”                               | It diagnoses weak directions and selects a solution; it does not create identification. |
| “OLS assumes normal data.”                                   | OLS fitting does not; particular inferential results add distributional assumptions.    |
| “A residual is the true error.”                              | A residual uses estimated parameters and is constrained by fitting.                     |
| “Gradient descent is more advanced, so it is more accurate.” | It is another optimiser; for OLS it should converge to the same minimum.                |
| “The lowest training error wins.”                            | Training error rewards flexibility and cannot establish generalisation.                 |
| “Random splitting is always objective.”                      | It can violate group and time deployment structure.                                     |
| “A pipeline is software convenience.”                        | It protects the statistical fitting boundary.                                           |
| “Cross-validation eliminates the need for a test.”           | It supports development; repeated selection can still create optimism.                  |
| “ $R^2 = 0$ equals the training dummy on test data.”         | Standard test $R^2$ uses the evaluation-set mean reference.                             |
| “Fold SD is a 95% interval.”                                 | Fold scores are dependent and may represent heterogeneous conditions.                   |

## Guidance on the KP infrastructure context

Keep asking:

- Is a field known at appraisal or only after procurement?
- Are costs in the same base year and do they include community contribution consistently?
- Does “road distance” measure distance to any road or an all-weather road?
- Does the terrain index mean the same thing across survey teams?
- Are multiple projects packaged under one contract or budget line?

- Did policy or procurement rules change during the dataset period?
- Are remote projects missing from the database because they were delayed or never completed?
- Will model errors shift funds away from precisely the places with the greatest structural disadvantage?

Technical validity and institutional responsibility meet at these questions.

---

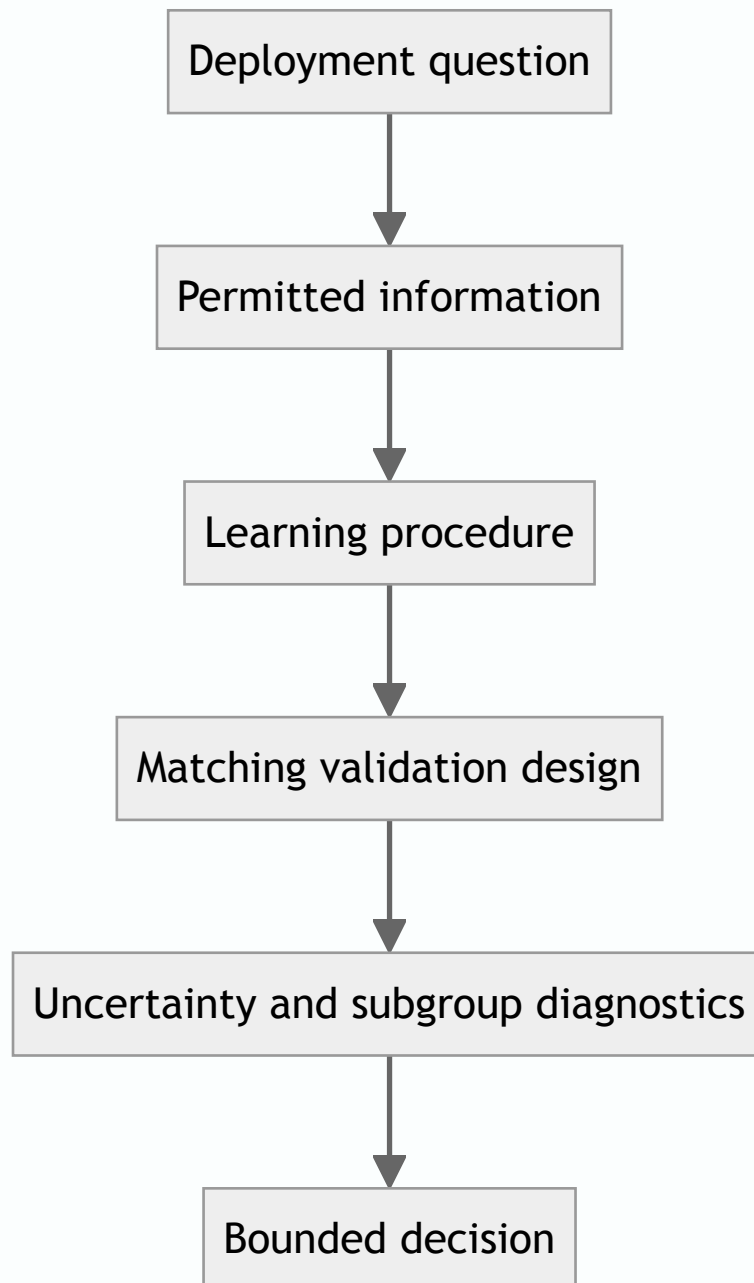
## Where Chapter 3 Begins

---

Chapter 2 has created an honest evaluation framework around OLS and one nonlinear comparator. The next chapter can deepen statistical learning without abandoning this framework:

1. polynomial and interaction features;
2. regularisation through ridge and lasso;
3. the geometry and Bayesian interpretation of penalties;
4. robust and quantile regression;
5. heteroskedasticity-robust and cluster-aware inference;
6. influence, leverage, and Cook's distance;
7. repeated and nested model comparison; and
8. a pre-registered benchmark study.

Every new model must pass through the same chain:



## Final reflection

A model becomes trustworthy neither because its derivation is elegant nor because its test score is impressive. Trust is built by an auditable chain: stable computation, explicit assumptions, legitimate information, a deployment-matched evaluation, uncertainty about the result, investigation of failures, and restraint in the conclusion.

That chain is the real subject of Chapter 2.

---

# References and Further Reading

---

## Research papers discussed

1. Bengio, Y., & Grandvalet, Y. (2004). No Unbiased Estimator of the Variance of K-Fold Cross-Validation. *Journal of Machine Learning Research*, 5, 1089–1105.
2. Breiman, L. (2001). Statistical Modeling: The Two Cultures. *Statistical Science*, 16(3), 199–231.
3. Cawley, G. C., & Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11, 2079–2107.

## Additional conceptual reading

4. Shmueli, G. (2010). To Explain or to Predict?. *Statistical Science*, 25(3), 289–310.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer. Chapters 3 and 7.
6. James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning: with Applications in Python*. Springer.

## Official software documentation

7. scikit-learn. Getting Started.
8. scikit-learn. Preprocessing Data.
9. scikit-learn. Pipelines and Composite Estimators.
10. scikit-learn. Cross-Validation: Evaluating Estimator Performance.
11. scikit-learn. Decision Trees.
12. scikit-learn. Regression Metrics.



# Chapter 2 Companion Guide: From a Fitted Line to a Trustworthy Model

## A beginner-first guide to numerical stability, probability, optimisation, evaluation, and responsible model revision

This guide accompanies `chapter2.md`. It assumes only the material developed in Chapter 1, and it explains the new Python, mathematics, probability, statistics, and machine-learning tools from first principles.

Chapter 1 answered: “How do we construct and fit an ordinary least-squares line?” Chapter 2 asks the harder question:

When should anyone trust that fitted line for a real decision?

Trust is not created by a low training error or a library call. It requires a chain of evidence:

1. the numerical calculation is stable;
2. probability claims are tied to explicit assumptions;
3. optimisation has genuinely converged;
4. evaluation data represent the intended deployment;
5. preprocessing and model selection do not leak information;
6. the model beats useful baselines on unseen cases;
7. average metrics do not conceal systematic failure; and
8. revisions are evaluated on new evidence.

The running case remains the fictional microhydro power (MHP) cost estimator for Khyber Pakhtunkhwa (KP). The target is final project cost in **constant 2025 million PKR**, predicted at technical appraisal before procurement and construction.

## How to use this guide

For each of Days 6–12:

1. Read the corresponding section here before the main chapter.
2. Type the smallest code example yourself.
3. Predict the output before running it.
4. Run the main chapter’s longer proof and build code.
5. deliberately create the listed failure.
6. Explain the result in plain language and in the MHP context.
7. Complete the exit check without looking back.

Use four questions whenever you meet a new method:

- **What is learned?** Coefficients, means, scales, categories, or hyperparameters?
- **From which rows?** Training, validation, a cross-validation fold, or test?

- **What claim follows?** Computation, prediction, uncertainty, or causation?
- **What can break it?** Poor conditioning, wrong assumptions, leakage, shift, or small samples?

## 0. Foundations for an absolute beginner

### 0.1 What Chapter 1 already gave us

Ordinary least squares (OLS) chooses coefficients that minimise squared residuals:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

You should recall:

- $X$  is the design matrix;
- $y$  is the target vector;
- $\beta$  is a coefficient vector;
- $\hat{y} = X\hat{\beta}$  contains fitted predictions;
- $e = y - \hat{y}$  contains residuals; and
- full column rank gives a unique coefficient vector.

Chapter 2 does not discard this mathematics. It surrounds it with the computational and evidential practices required for responsible use.

### 0.2 The prediction-time rule

Imagine an appraisal officer on the day a cost estimate must be issued. A feature is eligible only if it is legitimately available then.

| Field                  | Available at appraisal? | Eligible?  | Reason                                                      |
|------------------------|-------------------------|------------|-------------------------------------------------------------|
| Planned capacity       | Yes                     | Yes        | Comes from the proposed design                              |
| Estimated cable length | Yes                     | Yes        | Comes from the survey                                       |
| Road distance          | Yes                     | Yes        | Comes from the access assessment                            |
| Terrain index          | Yes                     | Cautiously | Assessor consistency must be checked                        |
| District               | Yes                     | Depends    | Use depends on whether deployment includes unseen districts |
| Start year             | Yes                     | Depends    | May represent time change if used correctly                 |
| Final material bill    | No                      | No         | Created during construction; leaks the outcome              |
| Actual completion time | No                      | No         | It is a later outcome                                       |

Constant-price cost removes general inflation from the target. It does not remove changes in procurement, technology, project mix, or data collection.

## 0.3 Installing the software

Chapter 2 adds pandas, SciPy, and scikit-learn:

```
python -m pip install numpy pandas matplotlib scipy scikit-learn
```

Record versions when an analysis must be reproducible:

```
import numpy as np
import pandas as pd
import scipy
import sklearn

print(np.__version__)
print(pd.__version__)
print(scipy.__version__)
print(sklearn.__version__)
```

A reproducible result identifies its data recipe, code, random seeds, package versions, and evaluation procedure.

## 0.4 NumPy ideas used repeatedly

### Rows, columns, and axis

For an array with shape `(n, p)`:

```
X.mean(axis=0) # one mean for each column
X.mean(axis=1) # one mean for each row
```

`axis=0` collapses the row direction and leaves one result per feature. Feature scaling therefore uses `axis=0`.

### Boolean masks

```
later = start_year >= 2023
X_test = X[later]
```

The comparison creates `True` or `False` for every row. Indexing with it retains the `True` rows.

### Random-number generators and seeds

```
rng = np.random.default_rng(2026)
noise = rng.normal(0.0, 2.0, size=100)
```

A seed makes a pseudo-random sequence reproducible. It does not make one random split scientifically correct or remove sampling uncertainty.

### Useful linear-algebra functions

```
np.linalg.cond(X) # condition number
np.linalg.matrix_rank(X) # numerical rank
np.linalg.qr(X, mode="reduced") # QR decomposition
```

```
np.linalg.svd(X, full_matrices=False) # singular value decomposition
np.linalg.lstsq(X, y, rcond=None) # least-squares solution
```

## 0.5 A pandas survival kit

A pandas `DataFrame` is a labeled table. A `Series` is one labeled column.

```
df = make_mhp_projects()

print(df.head())
print(df.shape)

features = ["planned_capacity_kw", "road_distance_km"]
X = df[features] # DataFrame, two columns
y = df["actual_cost_2025_million_pkr"] # Series, one column

train = df[df["start_year"] <= 2021].copy()
```

Important operations:

- `df[columns]` selects named columns;
- `df.loc[row_labels, columns]` selects by labels;
- `df.iloc[row_positions]` selects by integer positions;
- `sort_values` orders rows;
- `groupby` calculates summaries within groups;
- `pd.concat` combines tables; and
- `.copy()` makes an explicit independent table for modification.

Pandas labels help preserve meaning. Converting too early to raw arrays can hide a swapped-column or row-alignment error.

## 0.6 Understanding the chapter's data generator

`make_mhp_projects(n=360, seed=2026)` creates a reproducible teaching dataset rather than loading confidential project records. Its structure deliberately gives every later diagnostic something meaningful to discover.

The generator:

1. samples districts and start years;
2. creates appraisal-stage capacity, road distance, terrain, and cable features;
3. adds district-specific cost differences;
4. adds a threshold surcharge when road distance exceeds 18 km;
5. adds a curved capacity contribution using capacity squared;
6. adds a time trend;
7. makes noise larger for more remote projects;
8. constructs actual constant-price cost; and
9. creates a tempting final material bill from actual cost after construction.

These choices introduce:

- **nonlinearity**: one straight line cannot exactly represent the squared-capacity term or road threshold;
- **group structure**: districts have shared differences;
- **temporal change**: later years have systematically changed cost;
- **heteroskedasticity**: outcome noise grows with road distance; and
- **target leakage**: final material bill contains later information strongly related to cost.

Important generator expressions:

```
rng = np.random.default_rng(seed)
road_distance_km = np.clip(values, 0.2, 42.0)
remote_surcharge = 7.0 * (road_distance_km > 18.0)
capacity_curve = 0.000025 * planned_capacity_kw ** 2
noise = rng.normal(0.0, noise_sd)
```

`np.clip` limits values to a stated range. Multiplying a Boolean array by `7.0` turns `False` into 0 and `True` into 7. A vector `noise_sd` gives each project its own Gaussian standard deviation.

The final `sort_values(["start_year", "project_id"])` makes chronological analysis convenient. `reset_index(drop=True)` replaces the old row labels after sorting. Saving CSV with `index=False` avoids writing a meaningless extra index column.

Knowing the data-generating recipe helps interpret this exercise. In real analysis, that recipe is unknown; diagnostics and domain evidence are used to learn where a proposed model is inadequate.

## 0.7 The scikit-learn pattern

Most scikit-learn objects follow one interface:

```
model.fit(X_train, y_train)
predictions = model.predict(X_new)
```

Transformers use:

```
transformer.fit(X_train)
X_train_changed = transformer.transform(X_train)
X_new_changed = transformer.transform(X_new)
```

`fit` learns from data. `transform` applies what was learned. `fit_transform` combines them for training data. Calling `fit` on test data would let the test set teach the procedure.

Other tools introduced in the chapter:

- `Pipeline`: chains transformations and a model within one data boundary;
- `ColumnTransformer`: applies different transformations to different columns;
- `SimpleImputer`: learns replacement values for missing data;
- `OneHotEncoder`: turns categories into indicator columns;
- `StandardScaler`: learns means and scales;
- `clone`: makes a fresh unfitted copy of a procedure;
- `GridSearchCV`: searches hyperparameters using cross-validation; and
- `DummyRegressor`: supplies a simple baseline.

## 0.8 Mathematical and probability language

### Logarithms and exponentials

$\exp(a) = e^a$  is an exponential.  $\log$  is its inverse. Useful rules are:

$$\log(ab) = \log a + \log b, \quad \log(a^b) = b \log a.$$

These rules turn a product of many probability densities into a sum of log densities.

## Random variables, expectation, and variance

A random variable represents a numerical outcome before it is observed. Uppercase  $Y$  often denotes the random variable and lowercase  $y$  its realised value.

Expectation  $\mathbb{E}[Y]$  is a probability-weighted long-run mean. Variance measures spread around that mean:

$$\text{Var}(Y) = \mathbb{E}[(Y - \mathbb{E}[Y])^2].$$

Standard deviation is the square root of variance and returns to the original units.

## Distributions and density

A continuous probability density describes relative concentration. Probability over an interval is area under the density:

$$P(a \leq Y \leq b) = \int_a^b f(y) dy.$$

The integral symbol  $\int$  is continuous addition. You do not need to calculate these integrals manually in this chapter.

## Samples and sampling variability

A dataset is one sample from a wider process. If we could repeatedly collect new datasets and refit the model, estimates would change. This repeated-sample variation is the basis of standard errors and frequentist confidence intervals.

## Quantiles

The  $q$  quantile is a value at or below which proportion  $q$  of observations lies. The median is the 0.5 quantile. A 90th percentile absolute error is a threshold at or below which 90% of observed absolute errors fall.

---

# Day 6 — Scaling and numerical conditioning

---

## 6.1 Floating-point numbers are approximations

Computers store a finite number of binary digits. Many decimal fractions have no finite binary representation:

```
print(0.1 + 0.2)
print((0.1 + 0.2) == 0.3)
```

The first result is extremely close to 0.3; the exact comparison is false. Use `np.isclose` or `np.allclose` for computed floats.

Finite precision is manageable when algorithms avoid magnifying tiny errors unnecessarily.

## 6.2 Conditioning and numerical stability are different

An **ill-conditioned problem** turns a tiny input change into a large solution change. **Numerical stability** describes whether an algorithm adds avoidable error while solving the problem.

- Conditioning belongs to the problem and its representation.
- Stability belongs to the computational method.
- A stable solver cannot invent information missing from nearly duplicate features.

For a full-column-rank matrix:

$$\kappa_2(X) = \frac{\sigma_{\max}(X)}{\sigma_{\min}(X)}.$$

$\sigma_{\max}$  and  $\sigma_{\min}$  are the largest and smallest singular values. A ratio near 1 is favourable. A large ratio means some coefficient directions affect predictions far less than others, so noise or rounding can create large coefficient changes.

There is no universal pass/fail threshold. Precision, scale, noise, intended claim, and consequences matter.

### Full rank is not good conditioning

A matrix can have no exactly redundant columns yet contain two almost identical columns. It is full rank in exact or tolerance-based arithmetic but can be badly conditioned. Rank answers “how many directions?” Conditioning asks “how unevenly are those directions represented?”

## 6.3 Why differing units cause imbalance

Capacity may be hundreds of kW, road distance tens of km, and terrain only 1–5. Their columns live on different numerical scales.

Standardisation transforms entry  $i, j$ :

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j},$$

where  $\mu_j$  and  $s_j$  come from the training column.

- $z = 0$ : at the training mean;
- $z = 1$ : one training standard deviation above it;
- $z = -2$ : two training standard deviations below it.

Scaling changes representation, not information. It does not guarantee generalisation and cannot repair redundancy.

## 6.4 Build a scaler from scratch

The chapter's `StandardScalerFromScratch` teaches four distinct operations:

1. `fit` validates a 2D matrix and learns one mean and scale per column;
2. `transform` subtracts stored means and divides by stored scales;
3. `inverse_transform` reconstructs original units; and
4. `fit_transform` fits and then transforms the training rows.

```
self.mean_ = X.mean(axis=0)
self.scale_ = X.std(axis=0, ddof=0)
```

```
Z = (X - self.mean_) / self.scale_
```

NumPy broadcasts the `(p, )` means and scales across all `n` rows. A constant feature has zero standard deviation, making division impossible. The scaler rejects it rather than producing infinities or `NaN`.

## Population versus sample standard deviation

$$s_{\text{population}} = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x})^2},$$
$$s_{\text{sample}} = \sqrt{\frac{1}{n-1} \sum_i (x_i - \bar{x})^2}.$$

`ddof=0` uses  $n$  and matches scikit-learn's `StandardScaler`. `ddof=1` uses  $n - 1$ , common when estimating a population variance. The correct denominator depends on purpose; it is not one rule for all statistics.

## 6.5 Scaling must respect the information boundary

Correct procedure:

```
scaler.fit(X_train)
Z_train = scaler.transform(X_train)
Z_validation = scaler.transform(X_validation)
Z_test = scaler.transform(X_test)
```

Incorrect procedure:

```
scaler.fit(np.vstack([X_train, X_test]))
```

The incorrect version learns the future test distribution. Test values may look less extreme only because the test set has already influenced preprocessing. Means, scales, imputations, categories, and feature-selection decisions are learned parameters and must be fitted only on permitted training rows.

## 6.6 Scaling changes coefficient units, not fitted OLS predictions

For scaled coefficients  $\gamma$ :

$$\hat{y} = \gamma_0 + \sum_j \gamma_j \frac{x_j - \mu_j}{s_j}.$$

Rearrangement gives raw-unit coefficients:

$$\beta_j = \frac{\gamma_j}{s_j},$$
$$\beta_0 = \gamma_0 - \sum_j \frac{\gamma_j \mu_j}{s_j}.$$

With an intercept and the same unpenalised OLS feature space, raw and standardised fits can produce the same predictions. Coefficient numbers and units change because one scaled unit means one training standard deviation.



This equivalence may not hold unchanged for penalised models, constrained optimisation, finite gradient-descent iterations, or preprocessing that changes information.

## 6.7 Condition number before and after scaling

Compare the whole design including the intercept:

```
raw_design = np.column_stack([np.ones(X.shape[0]), X])
scaled_design = np.column_stack([np.ones(Z.shape[0]), Z])

print(np.linalg.cond(raw_design))
print(np.linalg.cond(scaled_design))
```

Scaling often improves unit-driven imbalance. If a feature is an almost exact copy of another, the smallest singular value may remain tiny.

## 6.8 Failure laboratory

- Add capacity in watts and kW: exact redundancy and rank loss.
- Add an almost identical road-time proxy: poor conditioning despite possible full rank.
- Add a constant feature: zero scaling denominator and redundancy with the intercept.
- Fit the scaler on all years: preprocessing leakage.

For each failure, distinguish computational symptoms, information content, and invalid evaluation claims.

## 6.9 Which methods benefit from scaling?

| Method                        | Usually scale?      | Reason                                         |
|-------------------------------|---------------------|------------------------------------------------|
| OLS with stable decomposition | Often               | Conditioning and coefficient comparison        |
| Gradient descent              | Yes                 | Balances curvature and learning-rate behaviour |
| Ridge/lasso                   | Yes                 | Penalties act on coefficient magnitude         |
| k-nearest neighbours          | Yes                 | Distance depends on units                      |
| Support vector machines       | Usually             | Margins and kernels depend on scale            |
| Decision trees                | Usually unnecessary | Splits depend on order and thresholds          |

Any learned transformation must still be fitted inside the training partition or cross-validation fold.

## Day 6 exit check

You are ready when you can explain:

1. why full rank does not guarantee stable coefficients;
  2. why scaling improves representation without adding information;
  3. which quantities a scaler learns; and
  4. why test data cannot influence them.
-

# Day 7 — QR, SVD, rank, and the pseudoinverse

## 7.1 Why not form the normal equations?

The normal equations are correct:

$$X^T X \hat{\beta} = X^T y.$$

But, for full-column-rank  $X$ :

$$\kappa_2(X^T X) = \kappa_2(X)^2.$$

A condition number of  $10^6$  becomes roughly  $10^{12}$ . QR and SVD work with  $X$  directly and avoid this unnecessary magnification.

## 7.2 QR decomposition

For  $n \geq p$  and full column rank, reduced QR writes:

$$X = QR,$$

where:

- $Q$  has shape  $n \times p$ ;
- $Q^T Q = I_p$ ;
- $R$  has shape  $p \times p$ ; and
- $R$  is upper triangular, meaning entries below its diagonal are zero.

Substitute into least squares. The fitted projection is built from  $Q$ , and coefficients satisfy:

$$R\hat{\beta} = Q^T y.$$

Solve this triangular system by back substitution:

```
Q, R = np.linalg.qr(X, mode="reduced")
beta_qr = np.linalg.solve(R, Q.T @ y)
```

Do not invert  $R$  explicitly.

## 7.3 Orthonormal means perpendicular and unit length

For distinct columns  $q_j, q_k$ :

$$q_j^T q_k = 0.$$

For each column:

$$q_j^T q_j = 1.$$

Thus the columns of  $Q$  are clean perpendicular unit directions spanning the same column space as  $X$ .  $Q^T y$  measures how much of  $y$  lies along each direction.

Check numerically:

```
assert np.allclose(Q.T @ Q, np.eye(Q.shape[1]))
```

Basic QR without pivoting is most straightforward for full-rank problems. Rank-revealing QR uses additional column pivoting.

## 7.4 Singular value decomposition

The reduced SVD is:

$$X = U\Sigma V^T.$$

- $V^T$  rotates parameter coordinates;
- diagonal  $\Sigma$  stretches each direction by a singular value; and
- $U$  rotates the result into observation space.

$U$  and  $V$  contain orthonormal directions. Singular values are nonnegative and usually ordered largest to smallest.

A tiny singular value means movement along one parameter direction barely changes predictions. The data provide little information for distinguishing coefficients in that direction.

## 7.5 The Moore–Penrose pseudoinverse

For positive singular values:

$$X^+ = V\Sigma^+U^T,$$

where  $\Sigma^+$  replaces retained singular values  $\sigma_j$  with  $1/\sigma_j$  and leaves discarded directions at zero.

Then:

$$\hat{\beta} = X^+y = V\Sigma^+U^Ty.$$

If multiple coefficient vectors minimise residual length, the pseudoinverse returns the one with minimum Euclidean coefficient norm. This precise convention does not make redundant coefficients separately identifiable.

## 7.6 Read the from-scratch SVD solver

The chapter's solver:

1. computes `U`, `singular_values`, `Vt`;
2. chooses a tolerance;
3. marks singular values above tolerance as retained;
4. safely inverts only retained values;
5. calculates  $V\Sigma^+U^Ty$ ; and
6. reports numerical rank and the full singular-value spectrum.

Default tolerance:

```
rcond = np.finfo(float).eps * max(X.shape)
tolerance = rcond * singular_values[0]
keep = singular_values > tolerance
```

`np.finfo(float).eps` is machine epsilon, a scale for floating-point resolution near 1. Multiplying by matrix size and the largest singular value creates a scale-aware numerical rule.

## 7.7 Numerical rank depends on tolerance

Exact mathematics distinguishes zero from nonzero. Floating-point computation must decide whether a tiny value represents information or numerical residue.

Changing tolerance can change reported rank and the pseudoinverse solution. When rank matters, record:

- feature scaling;
- singular values;
- tolerance or `rcond` rule;
- software version; and
- sensitivity of predictions and coefficients.

## 7.8 Compare computational routes

| Route              | Calculation                                                                  | Strength                           | Caution                                  |
|--------------------|------------------------------------------------------------------------------|------------------------------------|------------------------------------------|
| Normal equations   | Solve $\mathbf{X}^T \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^T \mathbf{y}$ | Direct link to derivation          | Squares condition number                 |
| QR                 | Solve $\mathbf{R} \boldsymbol{\beta} = \mathbf{Q}^T \mathbf{y}$              | Stable and efficient for full rank | Rank deficiency needs care               |
| SVD                | Use $\mathbf{V} \boldsymbol{\Sigma}^+ \mathbf{U}^T \mathbf{y}$               | Reveals rank and weak directions   | More computational work                  |
| <code>lstsq</code> | Library least-squares routine                                                | Appropriate default                | Diagnostics still require interpretation |

Agreement of predictions across routes is a useful code check. Coefficients can disagree in a rank-deficient problem because multiple solutions produce the same projection.

## 7.9 Singular-value spectrum

Plot singular values on a logarithmic vertical axis. A large drop to a tiny final value reveals a weak direction that might be invisible on a linear scale.

The spectrum is diagnostic, not an automatic deletion rule. Investigate whether weakness arises from duplicated units, feature construction, narrow sampling, or a meaningful but poorly represented comparison.

## 7.10 StableOLS

The chapter's deterministic estimator stores:

- fitted parameters;
- the residual-sum array returned by `lstsq`;
- numerical rank;
- singular values;
- number of parameters;
- condition number; and
- whether full column rank holds.

This separates two outputs:

A solver returned a prediction-minimising vector.

from:

Every individual coefficient is uniquely identified and stable enough to interpret.

The first can be true while the second is false.

## Day 7 exit check

Complete:

$$\begin{aligned}X &= QR \Rightarrow R\hat{\beta} = Q^T y, \\X &= U\Sigma V^T \Rightarrow \hat{\beta} = V\Sigma^+ U^T y.\end{aligned}$$

Manager-friendly explanation of a tiny singular value: “The data barely distinguish one combination of effects, so small data changes can produce large changes in the reported coefficients.”

# Day 8 — Probability, likelihood, and uncertainty

## 8.1 Deterministic OLS versus a stochastic model

OLS projection needs no probability distribution:

$$\hat{\beta} = \arg \min_{\beta} \|y - X\beta\|_2^2.$$

A statistical model adds claims about outcome variation:

$$y = X\beta + \varepsilon.$$

- $\beta$  is an unknown population parameter;
- $X\beta$  is the model's systematic conditional mean; and
- $\varepsilon$  represents unmodeled random influences.

“Error” here does not mean data-entry error.

## 8.2 Errors are not residuals

True error:

$$\varepsilon_i = y_i - x_i^T \beta.$$

Observed residual:

$$e_i = y_i - x_i^T \hat{\beta}.$$

We do not know true  $\beta$ , so we do not observe true errors. Residuals depend on the fitted sample and obey constraints such as summing to zero with an intercept. They are diagnostic proxies, not independent observations of the error distribution.

## 8.3 The Gaussian distribution

For mean  $\mu$  and variance  $\sigma^2$ :

$$f(y | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[ -\frac{(y - \mu)^2}{2\sigma^2} \right].$$

- $\mu$  locates the centre;
- $\sigma$  controls spread;
- equal positive and negative deviations have equal density; and
- large deviations receive rapidly declining density.

A density value is not itself the probability of one exact continuous outcome. Probability is area over an interval.

## 8.4 The classical Gaussian linear model

$$Y_i | X_i = x_i \sim \mathcal{N}(x_i^T \beta, \sigma^2),$$

equivalently:

$$\varepsilon \sim \mathcal{N}(0, \sigma^2 I).$$

This compact statement assumes:

1. the conditional mean is linear in chosen design columns;
2. conditional errors have mean zero;
3. conditional variance is constant;
4. errors are independent under the sampling design; and
5. Gaussian errors support the exact finite-sample results used here.

Running `LinearRegression` does not verify these assumptions.

## 8.5 Probability and likelihood reverse what is held fixed

Probability asks how data would vary for fixed parameter values. Likelihood holds the observed data fixed and compares parameter values:

$$L(\beta, \sigma^2 | X, y) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left[ -\frac{(y_i - x_i^T \beta)^2}{2\sigma^2} \right].$$

Likelihood is not the probability that a fixed parameter value is true.

## 8.6 Log-likelihood connects Gaussian MLE to OLS

Products of many small densities can underflow. Logs turn products into sums without changing which positive likelihood is largest:

$$\ell(\beta, \sigma^2) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2} \sum_i (y_i - x_i^T \beta)^2.$$

For fixed  $\sigma^2$ , only the final term depends on  $\beta$ . Maximising log-likelihood therefore minimises SSR.

**Gaussian maximum likelihood for  $\beta = \text{OLS}$ .**

This equivalence does not establish that the real errors are Gaussian.

## 8.7 Residual variance and degrees of freedom

After fitting  $k$  parameters including the intercept:

$$s^2 = \frac{SSR}{n - k}.$$

Fitting  $k$  independent parameters imposes constraints and leaves  $n - k$  residual degrees of freedom. Under the classical model, this is an unbiased estimator of  $\sigma^2$ . Gaussian maximum likelihood uses  $SSR/n$ ; the criteria differ.

Residual standard deviation is  $s = \sqrt{s^2}$  and has target units. It is not the same as a coefficient's standard error.

## 8.8 Coefficient sampling variability

Under classical assumptions and fixed  $X$ :

$$\hat{\beta} \sim \mathcal{N}(\beta, \sigma^2(X^T X)^{-1}).$$

Estimate covariance with:

$$\widehat{\text{Var}}(\hat{\beta}) = s^2(X^T X)^{-1}.$$

Coefficient  $j$ 's standard error is the square root of covariance diagonal entry  $j$ .

Large standard errors can result from high outcome noise, small samples, narrow feature ranges, or multicollinearity. They measure sampling uncertainty under assumptions, not whether code syntax succeeded.

## 8.9 Coefficient confidence intervals

$$\hat{\beta}_j \pm t_{1-\alpha/2, n-k} SE(\hat{\beta}_j).$$

Frequentist interpretation: if the full sampling and interval procedure were repeated under its assumptions, proportion  $1 - \alpha$  of intervals would cover the fixed true parameter. It is not strictly a 95% posterior probability that this one fixed parameter lies in this one interval.

An interval's units match its coefficient. It does not make an association causal.

## 8.10 Mean-response confidence interval versus prediction interval

For design row  $x_0$ :

$$\hat{y}_0 = x_0^T \hat{\beta}.$$

Mean-response standard error:

$$SE_{mean}(x_0) = s\sqrt{x_0^T(X^T X)^{-1}x_0}.$$

Individual prediction standard error:

$$SE_{prediction}(x_0) = s\sqrt{1 + x_0^T(X^T X)^{-1}x_0}.$$

The extra 1 represents individual project noise, so the prediction interval is wider. Both usually widen far from the centre of observed feature values because extrapolated means are less precisely estimated.

## 8.11 Read GaussianOLS

The chapter extends `StableOLS` to:

1. require full rank;
2. calculate residuals and SSR;
3. require positive residual degrees of freedom;
4. estimate residual variance;
5. calculate the classical covariance matrix and standard errors;
6. use Student's  $t$  critical values; and
7. return coefficient, mean-response, and individual-prediction intervals.

`np.einsum("ij,jk,ik->i", ...)` efficiently calculates one quadratic form  $x_i^T A x_i$  per new row. It supplies each row's leverage-like uncertainty factor.

The class uses an inverse after full-rank checking to show the textbook covariance formula. Production inference should use mature decomposition-based methods and covariance estimators appropriate to heteroskedastic or clustered data.

## 8.12 Assumptions map

| Claim                                    | Required support                                                   |
|------------------------------------------|--------------------------------------------------------------------|
| A least-squares solution was computed    | Valid finite arrays and correct numerical method                   |
| Coefficients are unique                  | Full column rank                                                   |
| Predictions generalise                   | Evaluation representing deployment                                 |
| Classical intervals have stated coverage | Mean, variance, dependence, distribution, and sampling assumptions |
| A coefficient is causal                  | A causal identification design                                     |

Normal-looking residuals do not fix confounding. Heteroskedastic residuals do not mean the least-squares projection was computed incorrectly.

## 8.13 Heteroskedasticity

**Heteroskedasticity** means conditional variance changes across observations. In the synthetic data, remote projects have larger noise. A residual-versus-fitted plot may show a fan shape.

OLS may still estimate a useful conditional mean under suitable conditions, but equal-variance standard errors and constant-width uncertainty become questionable. Later work introduces heteroskedasticity-robust and cluster-aware inference.



## Day 8 exit check

Distinguish:

- parameter versus estimate;
- unobserved error versus fitted residual;
- probability versus likelihood;
- residual standard deviation versus coefficient standard error; and
- confidence interval for a mean versus prediction interval for one project.

## Day 9 — Gradient descent from the OLS gradient

### 9.1 Why learn an iterative method?

QR or SVD is generally preferable for moderate OLS. Gradient descent matters because the same optimisation pattern works for models without convenient direct solutions and for very large datasets.

“Iterative” does not mean “more accurate.” For convex OLS, a correct converged implementation should approach the same minimum as `lstsq`.

### 9.2 MSE objective and gradient

$$J(\beta) = \frac{1}{n} \|y - X\beta\|_2^2,$$
$$\nabla_{\beta} J(\beta) = \frac{2}{n} X^T (X\beta - y).$$

Shape check:

| Quantity     | Shape        |
|--------------|--------------|
| $X$          | $n \times k$ |
| $\beta$      | $k$          |
| $X\beta - y$ | $n$          |
| gradient     | $k$          |

The gradient has one component per parameter and points in the direction of steepest local increase.

### 9.3 Update rule

$$\beta^{(t+1)} = \beta^{(t)} - \eta \nabla J(\beta^{(t)}).$$

- Minus means move downhill.
- $\eta$  is the learning rate.
- Too small means slow progress.

- Too large causes overshooting, oscillation, or divergence.
- A stopping rule decides when further changes are negligible.

In a one-parameter walk, print both coefficient and loss each step. Try learning rates `0.0001`, `0.05`, `0.2`, and `1.0`; recognising patterns matters more than seeing one successful run.

## 9.4 Why scaling helps gradient descent

Very different feature scales create a long narrow MSE valley. One learning rate zigzags across steep directions while moving slowly along shallow ones. Standardisation makes curvature more balanced and a useful learning rate easier to find.

Scaling does not fix a wrong gradient sign, leakage, nonlinearity, or an unrepresentative validation design.

## 9.5 Read GradientDescentOLS

The chapter's batch optimiser:

1. creates a design matrix;
2. starts all coefficients at zero;
3. calculates predictions, MSE, and the full-data gradient;
4. rejects non-finite loss or gradient;
5. stores loss history;
6. updates coefficients;
7. stops when coefficient movement is below tolerance; and
8. otherwise stops at `max_iter`.

The `for ... else` construct runs `else` only if the loop did not reach `break`. It records that maximum iterations were exhausted.

Stopping because parameter movement is small is not automatically proof of the correct optimum. Also inspect gradient size, loss history, and agreement with a trusted direct solver.

## 9.6 Verify against SVD and finite differences

Two independent checks are essential:

- fitted coefficients and predictions approach `StableOLS/lstsq` on scaled data;
- analytic gradient agrees with a centred finite-difference approximation.

For component  $j$ :

$$\frac{\partial J}{\partial \beta_j} \approx \frac{J(\beta + hu_j) - J(\beta - hu_j)}{2h}.$$

A smooth falling loss is not sufficient—a wrong gradient can still look orderly.

## 9.7 Convergence patterns

| Loss history              | Likely issue | Response                                     |
|---------------------------|--------------|----------------------------------------------|
| Smooth decline to plateau | Convergence  | Compare with direct solver and gradient norm |

| Loss history                          | Likely issue                 | Response                                  |
|---------------------------------------|------------------------------|-------------------------------------------|
| Very slow decline                     | Small rate or poor scaling   | Scale and tune rate                       |
| Oscillation                           | Rate too large               | Reduce rate                               |
| Explosion or non-finite values        | Divergence or sign/scale bug | Stop and audit                            |
| Training improves, validation worsens | Overfit or shift             | Evaluation problem, not optimiser success |

A logarithmic y-axis helps show changes across many orders of magnitude.

## 9.8 Batch, stochastic, and mini-batch descent

- **Batch:** uses every training row for every update.
- **Stochastic:** uses one randomly chosen row.
- **Mini-batch:** uses a small subset.

Stochastic methods are cheaper per update but noisy. They require decisions about shuffling, batch size, learning-rate schedules, seeds, and stopping. Chapter 2 uses batch descent so code stays directly aligned with the equation.

## 9.9 Parameters and hyperparameters

| Quantity                | Type                     | How obtained                        |
|-------------------------|--------------------------|-------------------------------------|
| Intercept and slopes    | Parameters               | Learned by minimising training loss |
| Scaler means and scales | Preprocessing parameters | Learned from training features      |
| Learning rate           | Hyperparameter           | Chosen outside coefficient updates  |
| Maximum iterations      | Hyperparameter           | Chosen before fitting               |
| Tree depth              | Hyperparameter           | Selected using development evidence |

When validation results guide a learning rate, validation has participated in model selection. The final test must remain untouched.

## Day 9 exit check

Write the gradient and update rule from memory. Then explain why equal minimum training MSE does not demonstrate equal future-project performance.

# Day 10 — Generalisation, baselines, and model complexity

## 10.1 The full learning procedure

$$\hat{f} = \mathcal{A}(D_{train}).$$

$\mathcal{A}$  includes feature definitions, cleaning, imputation, scaling, encoding, model family, hyperparameter search, random seeds, and selection rules. Honest evaluation must reproduce the entire procedure using only permitted information.

The named algorithm alone is not the unit of comparison.

## 10.2 Training risk versus deployment risk

Empirical training risk:

$$\hat{R}_{train}(f) = \frac{1}{n_{train}} \sum_{i \in train} L(y_i, f(x_i)).$$

Generalisation risk:

$$R(f) = \mathbb{E}_{(X,Y) \sim P_{deployment}} [L(Y, f(X))].$$

The deployment distribution is only partially observed. A held-out set estimates relevant performance only when its construction represents the cases, groups, and time periods of intended use.

## 10.3 Constant baselines are genuine models

For squared error:

$$S(c) = \sum_i (y_i - c)^2.$$

Setting the derivative to zero proves:

$$\hat{c} = \bar{y}_{train}.$$

The training mean is the best constant under squared loss. Under absolute loss, any training median minimises total absolute error because it balances observations on either side.

Fit the baseline on training targets only, then score it and candidate models on exactly the same held-out rows with exactly the same metric.

The mean is pulled more strongly by extreme values; the median is more resistant. Baseline and metric should match the decision loss.

## 10.4 What failure to beat baseline teaches

Possible causes include:

- weak features;
- incorrect row or column alignment;
- distribution shift;
- excessive flexibility;
- insufficient data; or
- an earlier leaky experiment that set unrealistic expectations.

The response is investigation, not hiding the baseline.

## 10.5 Underfitting and overfitting

- **Underfitting:** procedure is too restricted; training and validation errors are both high.
- **Overfitting:** procedure adapts to training-specific noise; training error is low but validation error is high.

Training error normally decreases as flexibility increases. Validation error may decrease and then rise. Choose complexity using validation evidence, not training victory.

## 10.6 Bias–variance decomposition

At fixed input  $x_0$ , assume:

$$Y = f(x_0) + \varepsilon, \quad \mathbb{E}[\varepsilon] = 0, \quad \text{Var}(\varepsilon) = \sigma^2.$$

Across repeated training datasets:

$$\mathbb{E}[(Y - \hat{f}(x_0))^2] = \left( \mathbb{E}[\hat{f}(x_0)] - f(x_0) \right)^2 + \text{Var}(\hat{f}(x_0)) + \sigma^2.$$

The terms are squared bias, procedure variance, and irreducible noise. This is a repeated-sampling identity under squared loss, not three quantities fully visible in one split. Do not label one model “high variance” solely from one graph.

## 10.7 A shallow decision tree

A regression tree repeatedly splits feature space. For feature  $j$  and threshold  $s$ :

$$R_L = \{x : x_j \leq s\}, \quad R_R = \{x : x_j > s\}.$$

Under squared loss, each leaf predicts its training-target mean. A split minimises combined within-child SSR.

Trees can represent thresholds and interactions without manually specifying them. They are piecewise constant and do not linearly extrapolate beyond observed ranges.

- Depth 0: one constant leaf.
- Small depth: broad nonlinear structure.
- Large depth: small leaves and possible memorisation.

A shallow tree is a useful nonlinear comparator, not automatically the correct deployment model.

## 10.8 Read the first library comparison

The main chapter compares:

- `DummyRegressor(strategy="mean")`;
- `LinearRegression`;
- depth-3 `DecisionTreeRegressor`; and
- an unrestricted tree.

`random_state` makes the split or tree behaviour reproducible. The unrestricted tree may nearly eliminate training error. Only held-out error addresses generalisation.

The first random split demonstrates the API; Day 11 determines whether that split matches deployment.

## 10.9 Learning curves

A learning curve plots training and held-out performance as training size grows.

| Pattern               | Possible lesson                          |
|-----------------------|------------------------------------------|
| Both errors stay high | Model or features may underfit           |
| Large gap narrows     | More data may reduce variance            |
| Gap remains wide      | Excess flexibility or structural leakage |
| Curves unstable       | Too little data or unsuitable split      |

One sampled subset per size makes a noisy curve. Repeat sampling or use deployment-appropriate cross-validation for stronger evidence.

## 10.10 Research discussion: Breiman's “Two Cultures”

Breiman contrasted:

- **data modeling:** specify a stochastic model and study its parameters;
- **algorithmic modeling:** treat the mechanism as partly unknown and prioritise predictive performance.

Beginner lessons:

1. An interpretable equation is not automatically an adequate description of reality.
2. Predictive accuracy does not automatically explain a mechanism or identify causation.
3. Prediction claims need out-of-sample evaluation.
4. Complete procedures, not mathematical elegance alone, should be compared.

For MHP work, a probabilistic linear model can communicate conditional relationships and uncertainty, while a shallow tree checks whether broad nonlinearity matters. Modern practice can combine these traditions; the paper's framing is a provocative argument, not a command to choose one camp forever.

Replication exercise: over many synthetic datasets, compare OLS and increasing tree depths under a fixed validation design. Plot training and validation MAE and identify where flexibility stops helping new cases.

## Day 10 exit check

Derive the mean baseline, explain why the median minimises absolute error, and give an example where the lowest-training-error model is the least responsible choice.

---

# Day 11 — Honest splitting, leakage, and cross-validation

## 11.1 Training, validation, and test roles

| Partition  | Allowed                                            | Not allowed                       |
|------------|----------------------------------------------------|-----------------------------------|
| Training   | Fit preprocessing and model parameters             | Final performance claim           |
| Validation | Select features, model family, and hyperparameters | Later call it untouched test data |
| Test       | One final evaluation after procedure is fixed      | Tuning or revising the procedure  |

Typical workflow:

```
train parameters
-> validate/select procedure
-> refit selected procedure on train + validation
-> test once
```

Percentages such as 60/20/20 are examples, not laws. Time, groups, sample size, and deployment decide the split.

## 11.2 A split simulates deployment

### Random split

Appropriate for new, approximately independent and exchangeable observations under the same mixture of conditions. It is unsafe when related entities, time order, or deployment shifts matter.

### Group split

Keep related observations together. For unseen-district deployment:

$$G_{train} \cap G_{test} = \emptyset.$$

The group need not be a model feature; it identifies dependence and intended transfer.

### Temporal split

For future prediction:

$$\max(t_{train}) < \min(t_{test}).$$

Forward-chaining trains on earlier history and validates on the next period. It exposes changing procurement, technology, measurement, or project mix.

### Combined new-time/new-group deployment

Future projects in unseen districts require both boundaries. If the dataset contains no analogue, a splitter cannot create evidence; external validation is needed.

## 11.3 Leakage catalogue

**Target leakage:** use target or post-outcome proxy, such as final material bill.

**Preprocessing leakage:** learn imputation, scaling, categories, transformations, or feature selection from validation/test rows.

**Temporal leakage:** allow future information to predict the past or use values recorded after prediction time.

**Group leakage:** related projects appear on both sides of an evaluation intended for new groups.

**Duplicate leakage:** exact or near-duplicate records cross partitions.

**Test-set leakage:** repeatedly inspect test scores and modify the procedure.

**Label leakage through cleaning:** use difficult target outcomes to decide which rows or feature values to remove.

Leakage is about illegitimate information flow, not merely whether a target column is visibly present.

## 11.4 Why the leaky feature looks spectacular

The synthetic final material bill is strongly based on actual cost and is known only later. Adding it dramatically lowers random-holdout MAE but answers a useless appraisal question: reconstruct final cost after much of it is already known.

High accuracy is not evidence of valid timing. Always perform the desk-at-prediction-time test.

## 11.5 Pipelines protect statistical validity

The numeric branch:

```
numeric columns -> median imputation -> standardisation
```

The categorical branch:

```
district -> most-frequent imputation -> one-hot encoding
```

`ColumnTransformer` combines them, then `Pipeline` fits the model. In every cross-validation split, the pipeline:

1. fits preprocessing only on the current training fold;
2. transforms that training fold;
3. fits the model;
4. transforms the validation fold using learned training values; and
5. predicts the validation fold.

`OneHotEncoder(handle_unknown="ignore")` prevents an unseen category from crashing transformation. It does not guarantee good predictions for that group. `drop="first"` removes one redundant category indicator when an intercept is present.

A pipeline is not just code tidiness. It defines the correctly bounded learning procedure.



## 11.6 K-fold cross-validation

Divide development data into  $K$  folds. Each fold is held out once while the others train the procedure. Average held-out losses:

$$\hat{L}_{CV} = \frac{1}{K} \sum_{k=1}^K L_k.$$

Every cross-validated prediction comes from a fitted procedure that did not train on that row. However, cross-validation estimates only the deployment question represented by its splitter.

Scikit-learn scoring names such as `neg_mean_absolute_error` return negative losses because its selection API treats larger scores as better. Negate them to report positive MAE.

## 11.7 Cross-validation is a family

| Splitter                        | Preserves                    | Question                             |
|---------------------------------|------------------------------|--------------------------------------|
| KFold                           | Row separation               | New independent-like observations    |
| GroupKFold                      | Group separation             | New groups                           |
| TimeSeriesSplit or forward loop | Time order                   | Future observations                  |
| Nested CV                       | Tuning/evaluation separation | Performance of a selection procedure |

Regression target binning for stratification does not replace time or group structure and must be documented carefully.

## 11.8 Grouped and temporal validation

With five district folds, report every district's MAE, RMSE, and  $R^2$ , not only their mean. One fold may reveal operational failure.

Temporal fold losses have different training sizes and represent different periods. They are not identical repeated measurements. The trend over years may be more informative than one average.

Use `clone(pipeline)` inside manual loops to ensure a fresh unfitted procedure for every fold.

## 11.9 Hyperparameter selection creates optimism

For depths in set  $\mathcal{D}$ :

$$\hat{d} = \arg \min_{d \in \mathcal{D}} \hat{L}_{\text{validation}}(d).$$

The selected depth looks best partly because of genuine performance and partly because of validation noise. Reporting the same validation score as final is optimistically biased.

The more irrelevant choices tried, the more opportunities noise has to create an apparent winner.

## 11.10 Nested cross-validation

- **Inner loop:** selects hyperparameters using only the outer-training data.
- **Outer loop:** evaluates the entire inner-selection-and-refit procedure on untouched outer data.

For every outer split:

1. create outer train and outer test;
2. run an inner search entirely inside outer train;
3. refit the inner winner on all outer train;
4. predict outer test once; and
5. store outer performance.

The evaluated object is not a fixed depth chosen in advance. It is the complete procedure that searches a specified grid with a specified splitter, score, preprocessing pipeline, seed policy, and refitting rule.

Nested CV is expensive because selection is repeated. That repetition is the protection.

## 11.11 Research discussion: Cawley and Talbot

The paper asks whether a model-selection criterion itself can be overfit. Its key lesson is that selection noise can be comparable with reported differences between algorithms.

The unit of comparison should be:

**learning algorithm + model-selection procedure.**

“Decision tree” is incomplete. Specify candidate depths and leaf sizes, splitter, metric, preprocessing, search procedure, refit rule, and randomness.

Nested CV or a final untouched test addresses selection optimism under the chosen sampling design. It cannot fix a wrong deployment simulation, feature leakage, unmeasured distribution shift, or missing hard projects.

Replication exercise: as the number of irrelevant hyperparameter choices grows, compare the best inner score with outer performance across many synthetic datasets.

## 11.12 Locked-test protocol

Before opening test results, write and timestamp:

1. target and units;
2. prediction time;
3. eligible and prohibited features;
4. exclusion rules;
5. split design;
6. candidate procedures;
7. selection metric;
8. subgroup reports;
9. uncertainty method; and
10. deployment-blocking conditions.

If test results motivate revision, revision is legitimate but the test set has become development data. A fresh final claim needs new independent evidence.

## Day 11 exit check

Draw inner and outer nested-CV loops, state what each may influence, and explain why a pipeline is part of statistical validity.

---

# Day 12 — Metrics, diagnostics, uncertainty, and revision

---

## 12.1 Begin with signed error

$$e_i = y_i - \hat{y}_i.$$

- positive: actual cost is larger, so model underpredicted;
- negative: prediction is larger, so model overpredicted.

Mean error:

$$ME = \frac{1}{n} \sum_i e_i.$$

Opposite signs cancel. ME detects directional bias but cannot measure total accuracy.

## 12.2 MAE, MSE, RMSE, and median absolute error

$$MAE = \frac{1}{n} \sum_i |e_i|,$$

$$MSE = \frac{1}{n} \sum_i e_i^2, \quad RMSE = \sqrt{MSE},$$

$$MedAE = \text{median}(|e_i|).$$

| Metric | Units  | Sensitivity                                | Plain reading                    |
|--------|--------|--------------------------------------------|----------------------------------|
| ME     | target | signs cancel                               | average direction of miss        |
| MAE    | target | linear penalty                             | average absolute miss            |
| RMSE   | target | emphasises large errors                    | square-root average squared miss |
| MedAE  | target | highly resistant to a minority of extremes | central typical miss             |

MAE is not immune to outliers; it is less dominated than RMSE. MedAE can look reassuring while a minority of projects fail catastrophically. Also report a high absolute-error quantile such as  $Q_{0.90}(|e|)$ .

## 12.3 Held-out $R^2$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y}_{\text{evaluation}})^2}.$$

- 1: perfect predictions;
- 0: same SSR as predicting the evaluation-set mean;
- negative: worse than that evaluation-set-mean reference.

Standard held-out  $R^2$  uses the evaluation targets' mean in its denominator. A deployable dummy predicts a value learned from training targets. Under shift these are not equal. Always score an explicit training-fitted baseline.

$R^2$  is dimensionless and depends on target variation. Identical absolute errors can yield different  $R^2$  across districts with different cost ranges.

## 12.4 Why MAPE can be dangerous

$$MAPE = \frac{100}{n} \sum_i \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

It is undefined at zero, explodes near zero, gives the same absolute miss more weight for small projects, and introduces asymmetries. Use it only when proportional error genuinely matches the decision.

Possible scale-aware alternatives:

- report metrics by project scale;
- use weighted absolute percentage error for a justified portfolio meaning:

$$WAPE = \frac{\sum_i |y_i - \hat{y}_i|}{\sum_i |y_i|};$$

- model log cost when multiplicative structure is plausible; or
- compare scaled error with a defined baseline.

No substitute is universal.

## 12.5 Asymmetric decision loss

If underbudgeting is more costly:

$$L(e) = \begin{cases} c_{under}|e|, & e > 0, \\ c_{over}|e|, & e \leq 0, \end{cases} \quad c_{under} > c_{over}.$$

Quantile or pinball loss for level  $\tau$  is:

$$L_\tau(y, \hat{q}) = \begin{cases} \tau(y - \hat{q}), & y \geq \hat{q}, \\ (1 - \tau)(\hat{q} - y), & y < \hat{q}. \end{cases}$$

An estimated 80th conditional quantile can support contingency planning. Label it as a quantile, not as a mean plus an arbitrary buffer.

## 12.6 Training loss, selection metric, and reporting metrics

- Training loss determines fitted parameters.
- Selection metric chooses among procedures.
- Reporting metrics communicate operational consequences.

A model can minimise training MSE, be selected by validation MAE, and be reported with test MAE, RMSE, ME, MedAE,  $R^2$ , and subgroup errors. Pre-specify primary and secondary metrics rather than choosing after seeing which favours a preferred model.

## 12.7 Keep a row-level prediction table

Retain:

- project ID;
- actual target and unit;
- prediction;
- signed, absolute, and squared errors;
- district and year;
- project scale and remoteness; and
- any evaluation-fold identifier.

Metrics can be regenerated from rows. Aggregate scores cannot reconstruct lost individual failures.

## 12.8 Held-out diagnostic views

**Error versus prediction:** curvature, fan shape, and offset.

**Error over time:** policy, procurement, technology, measurement, or drift.

**Error by district:** representation, group relationships, or measurement differences.

**Error by project scale/remoteness:** whether overall averages hide high-risk projects.

**Signed-error distribution:** asymmetry and extreme tails.

The four-panel function in the chapter validates required columns, plots these views, and returns the Matplotlib figure. Plots generate hypotheses; they do not identify causes by themselves.

## 12.9 Subgroup evaluation without false certainty

For each meaningful group report:

- count;
- mean signed error;
- MAE;
- MedAE; and
- 90th percentile absolute error.

Small subgroup estimates are noisy. Do not rank five-project districts as a stable league table. Consider intervals, domain knowledge, replication, and whether groups are weighted by project count, community importance, or budget exposure.

## 12.10 Bootstrap interval for held-out MAE

A row bootstrap:

1. retains fixed held-out actual/prediction pairs;
2. samples  $n$  row indices with replacement;
3. calculates MAE on each resample;
4. repeats thousands of times; and
5. takes lower and upper empirical quantiles.

“With replacement” means a project can appear multiple times or not at all in one resample. The percentile interval approximates sampling variability under the assumption that held-out rows are suitable resampling units.

If projects share districts or contractors, row resampling may understate dependence. A cluster bootstrap resamples independent groups. The resampling unit must match the data-generating structure.

The seed makes the simulation reproducible; the number of repeats controls Monte Carlo smoothness, not the underlying amount of evidence.

## 12.11 Fold standard deviation is not a confidence interval

Cross-validation training sets overlap, so fold scores are dependent. Folds may also represent substantively different districts or years. Fold standard deviation describes variation across those folds; it is not automatically a standard error or 95% interval. Dividing by  $\sqrt{K}$  does not make dependence disappear.

## 12.12 Research discussion: Bengio and Grandvalet

The paper proves that, under its broad setting, no universal unbiased estimator of  $K$ -fold cross-validation variance can be recovered from ordinary fold results. Overlapping training sets create covariance terms that are not generally identifiable.

It does not say cross-validation is useless or uncertainty impossible. It warns against a convenient universal variance formula.

Practical lessons:

- never relabel fold SD as a confidence interval;
- preserve individual fold results and their identities;
- use repeated/nested designs, independent tests, bootstrap, or model-based methods only with assumptions stated; and
- treat small performance differences cautiously.

For district folds, variation may represent real geographic heterogeneity rather than interchangeable noise.

Replication exercise: across many independently generated datasets, compare variation among folds within one dataset with variation of mean CV performance across datasets.

## 12.13 Diagnostics do not authorise test tuning

Correct workflow after finding a test failure:

1. report the original pre-specified result;
2. use diagnostics to propose a substantive mechanism;
3. label changes as post-test development;
4. revise using development data; and
5. obtain new independent or prospective evidence.

Do not overwrite history and reuse the same test set as untouched.

## 12.14 Responsible revision protocol

```
structured error
-> proposed mechanism
```

```
-> measurement/timing audit
-> development-data revision
-> rerun full selection procedure
-> new independent evaluation
```

Curvature may motivate a capacity-squared feature; changing spread may motivate quantile or variance modeling; drift may motivate a known-at-deployment time feature; district failure may reveal measurement inconsistency rather than a need to add district identity.

## Day 12 exit check

You should be able to state:

1. positive signed error means underprediction;
2. held-out  $R^2$  uses the evaluation-set mean reference;
3. low overall MAE can hide remote-project failure;
4. fold SD is not automatically an interval; and
5. test-driven revision requires new evaluation evidence.

---

# Chapter 2 capstone — An auditable MHP evaluation pipeline

---

## Capstone objective

Estimate final constant-2025 cost at appraisal for projects beginning in 2023 onward. The deliverable is the documented learning and evaluation procedure, not only the winning score.

## Pass 1: evaluation contract

State before modeling:

- unit of observation;
- target, unit, and price basis;
- prediction time;
- eligible and prohibited features;
- deployment population;
- primary metric and decision rationale;
- validation design;
- locked test period;
- subgroup reports; and
- conditions that block deployment.

## Pass 2: boundary audit

Confirm:

- project ID is an identifier, not a feature;

- final material bill is prohibited;
- transformations fit only inside training partitions;
- time ordering is correct;
- duplicates do not cross partitions; and
- test outcomes do not guide category handling or cleaning.

## Pass 3: compare complete procedures

Compare:

1. training-mean baseline;
2. linear regression with imputation, scaling, and one-hot encoding; and
3. trees whose depth is selected on validation data.

Use validation MAE as the primary selection criterion. Retain RMSE, MedAE,  $R^2$ , and ME as secondary reports.

## Pass 4: select, refit, and test once

- Train: 2016–2021.
- Validate: 2022.
- Test: 2023–2025.
- Select procedure using validation MAE only.
- Refit the chosen procedure on train plus validation.
- Open final test once.

The capstone uses `clone` so every candidate begins unfitted. All candidates share one preprocessing recipe, which makes comparison procedural and prevents leakage.

## Pass 5: diagnose without rewriting history

Report:

- overall metrics and a bootstrap MAE interval;
- district and remoteness results with counts;
- the ten largest absolute errors;
- temporal drift evidence; and
- whether errors meet actual planning tolerance.

If diagnostics motivate a revision, mark the start of a new development cycle.

## Read the runnable capstone by function

| Function/component                  | Role                                                     |
|-------------------------------------|----------------------------------------------------------|
| <code>make_preprocessor</code>      | Defines numeric and categorical training transformations |
| <code>make_pipeline</code>          | Joins preprocessing to one model                         |
| <code>regression_metrics</code>     | Computes pre-specified test summaries                    |
| <code>bootstrap_mae_interval</code> | Quantifies row-resampling uncertainty                    |
| <code>prediction_table</code>       | Preserves project-level evidence                         |



| Function/component       | Role                                               |
|--------------------------|----------------------------------------------------|
| subgroup_report          | Summarises operational groups                      |
| feature audit assertions | Enforce timing and identifier rules                |
| chronological masks      | Create train, validation, and test periods         |
| candidate loop           | Fits each procedure on train and scores validation |
| selected refit           | Learns from all development data after selection   |
| final test block         | Produces the one locked evaluation                 |

Run it, then inspect the exported prediction CSV rather than reading only console averages.

## Capstone interpretation checklist

1. Did the selected procedure beat the training-fitted baseline on validation and test?
2. Did procedure ranking change between validation and test?
3. Is error small relative to planning tolerance, not merely relative to a competitor?
4. Which groups fail, and how many observations support each estimate?
5. Does ME show systematic underbudgeting?
6. Do later years show drift?
7. Why is final material bill prohibited despite its predictive score?
8. Are tree thresholds predictive patterns rather than causal effects?
9. Does an OLS win prove the true mechanism is linear? No.
10. What fresh evidence is needed after revision?

## Self-assessment rubric

| Area            | Research-ready beginner habit                                               |
|-----------------|-----------------------------------------------------------------------------|
| Contract        | Timing, units, permitted information, deployment, and blockers are explicit |
| Numerics        | Scaling, rank, conditioning, solver, and tolerance are examined             |
| Probability     | Interval type and assumptions are stated                                    |
| Optimisation    | Gradient is checked and convergence compared with a direct solver           |
| Validation      | Split matches deployment and selection remains inside its boundary          |
| Metrics         | Baseline, decision loss, uncertainty, and subgroup risk are integrated      |
| Diagnostics     | Patterns generate hypotheses without reusing test as confirmation           |
| Reproducibility | Data recipe, seed, code, versions, and outputs are traceable                |
| Communication   | Recommendation includes magnitude, limitations, and blocking conditions     |

# Formula sheet with plain-language readings

| Concept                | Formula                                                 | Read it as...                                       |
|------------------------|---------------------------------------------------------|-----------------------------------------------------|
| Standardisation        | $z_{ij} = (x_{ij} - \mu_j) / s_j$                       | Centre and scale using training statistics          |
| Condition number       | $\kappa_2(X) = \sigma_{\max} / \sigma_{\min}$           | Imbalance between strongest and weakest directions  |
| QR                     | $X = QR$                                                | Orthonormal directions times triangular coordinates |
| QR least squares       | $R\hat{\beta} = Q^T y$                                  | Project, then solve a triangular system             |
| SVD                    | $X = U\Sigma V^T$                                       | Rotate, stretch, rotate                             |
| Pseudoinverse          | $X^+ = V\Sigma^+ U^T$                                   | Invert retained singular directions                 |
| SVD solution           | $\hat{\beta} = V\Sigma^+ U^T y$                         | Minimum-norm least-squares solution                 |
| Statistical model      | $y = X\beta + \varepsilon$                              | Conditional mean plus unobserved error              |
| Gaussian errors        | $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$           | Independent equal-variance normal errors            |
| Log-likelihood         | $C - \frac{n}{2} \log \sigma^2 - \frac{SSR}{2\sigma^2}$ | Gaussian fit rewards smaller SSR                    |
| Residual variance      | $s^2 = SSR / (n - k)$                                   | Noise estimate using residual degrees of freedom    |
| Coefficient covariance | $s^2 (X^T X)^{-1}$                                      | Classical sampling uncertainty matrix               |
| Coefficient CI         | $\hat{\beta}_j \pm t^* SE(\hat{\beta}_j)$               | Repeated-sampling interval procedure                |
| Mean SE                | $s \sqrt{x_0^T (X^T X)^{-1} x_0}$                       | Uncertainty in estimated conditional mean           |
| Prediction SE          | $s \sqrt{1 + x_0^T (X^T X)^{-1} x_0}$                   | Mean uncertainty plus individual noise              |
| MSE objective          | $J = \ y - X\beta\ ^2 / n$                              | Average squared training loss                       |
| MSE gradient           | $\nabla J = 2X^T(X\beta - y) / n$                       | Local uphill direction                              |
| GD update              | $\beta^{t+1} = \beta^t - \eta \nabla J$                 | Take a learning-rate-sized downhill step            |
| Mean baseline          | $\hat{c} = \bar{y}_{train}$                             | Best constant under squared loss                    |
| Median baseline        | $\hat{c} = \text{median}(y_{train})$                    | Best constant under absolute loss                   |
| Training risk          | $n^{-1} \sum_i L(y_i, f(x_i))$                          | Average observed training loss                      |
| Generalisation risk    | $\mathbb{E}_{deployment}[L(Y, f(X))]$                   | Expected loss where model will be used              |
| CV loss                | $K^{-1} \sum_k L_k$                                     | Average held-out fold loss                          |
| ME                     | $n^{-1} \sum_i e_i$                                     | Directional error                                   |
| MAE                    | $n^{-1} \sum_i  e_i $                                   | Average absolute error                              |
| RMSE                   | $\sqrt{n^{-1} \sum_i e_i^2}$                            | Large-error-sensitive summary in target units       |
| MedAE                  | $\text{median}( e_i )$                                  | Median absolute error                               |
| $R^2$                  | $1 - SSR / TSS$                                         | Squared-error comparison with evaluation mean       |
| WAPE                   | $n^{-1} \sum_i  e_i  / \bar{y}$                         | Average absolute error relative to mean             |

| Concept      | Formula                         | Read it as...                                    |
|--------------|---------------------------------|--------------------------------------------------|
| Pinball loss | Piecewise $\tau$ -weighted loss | Asymmetric loss targeting a conditional quantile |

## Shape sheet

For  $n$  observations and  $k$  design columns:

| Object                       | Shape                                         |
|------------------------------|-----------------------------------------------|
| $X$                          | $(n, k)$                                      |
| $y$ , predictions, residuals | $(n, )$                                       |
| $\beta$ , gradient           | $(k, )$                                       |
| Reduced $Q$                  | $(n, k)$                                      |
| $R$                          | $(k, k)$                                      |
| Reduced $U$                  | $(n, r)$ or $(n, \min(n, k))$ computationally |
| singular values              | $(\min(n, k), )$                              |
| $V^T$                        | $(\min(n, k), k)$                             |
| coefficient covariance       | $(k, k)$                                      |

## Expanded glossary

**Algorithm:** Computational procedure. A learning procedure also includes preprocessing, tuning, and selection.

**Baseline:** Simple reference model a candidate should beat under the same evaluation.

**Batch gradient descent:** Uses all training observations for each update.

**Bias–variance decomposition:** Repeated-sampling squared-error decomposition into squared bias, estimator variance, and irreducible noise.

**Bootstrap:** Repeatedly resamples observed units with replacement to approximate sampling variability.

**Condition number:** Measure of how strongly perturbations can be amplified.

**Confidence interval:** Repeated-sampling procedure designed for stated long-run coverage under assumptions.

**Constant prices:** Monetary values converted to a common price-year basis.

**Cross-validation:** Development design where rows or groups take turns as held-out data.

**Data leakage:** Illegitimate information flow across timing, training, selection, or evaluation boundaries.

**Degrees of freedom:** Independent information remaining after fitted constraints in a specified model.

**Density:** Relative concentration of a continuous probability distribution.

**Distribution shift:** Difference between training and deployment distributions.

**Empirical risk:** Average loss on an observed dataset.

**Error term:** Unobserved deviation from a population conditional mean.

**Exchangeable:** Observations whose ordering does not change their joint probabilistic role under a model; informally, similarly generated units for a random split.

**Extrapolation:** Prediction outside feature regions well represented in training.

**Generalisation:** Performance on genuinely new intended-deployment cases.

**Gradient descent:** Iterative optimisation that moves parameters opposite the gradient.

**Heteroskedasticity:** Conditional error variance changes with observations or features.

**Hyperparameter:** Setting chosen outside parameter fitting, often using validation evidence.

**Ill-conditioning:** Small input changes can cause large solution changes.

**Imputation:** Replacement of missing values using a stated learned rule.

**Irreducible noise:** Outcome variability not eliminated by knowing the modeled relationship.

**Learning curve:** Training and evaluation performance plotted against training-set size.

**Likelihood:** Function comparing parameter values with observed data fixed.

**Log-likelihood:** Logarithm of likelihood, converting products to sums.

**Maximum likelihood:** Chooses parameters maximising likelihood under a specified model.

**Mini-batch gradient descent:** Uses a subset of training rows for each update.

**Nested cross-validation:** Outer evaluation loop surrounding inner model selection.

**Numerical rank:** Singular directions retained as nonzero at a stated tolerance and scale.

**Numerical stability:** Extent to which an algorithm avoids magnifying rounding error unnecessarily.

**One-hot encoding:** Represents a category with indicator columns.

**Orthonormal:** Mutually perpendicular vectors, each of length one.

**Overfitting:** Learning sample-specific patterns that do not generalise.

**Parameter:** Value learned inside a fitted model or transformer.

**Pipeline:** One bounded procedure joining learned transformations and a model.

**Prediction interval:** Interval for an individual future outcome, including outcome noise.

**Pseudoinverse:** Generalised inverse expressing least-squares solutions in full- or deficient-rank cases.

**QR decomposition:** Factorisation into orthonormal-column  $Q$  and upper-triangular  $R$ .

**Quantile:** Value below which a specified proportion lies.

**Residual:** Observed target minus fitted prediction.

**Selection bias in evaluation:** Optimism when data used for selection also influence reported performance.

**Singular value:** Nonnegative SVD scale describing strength of a matrix direction.

**Stochastic gradient descent:** Uses one observation per parameter update.

**Test set:** Data reserved for final evaluation after procedure is fixed.

**Training set:** Data permitted to fit transformations and model parameters.

**Underfitting:** Failure to capture useful structure because procedure is too restricted.

**Validation set:** Development data used to compare procedures and choose hyperparameters.

---

## Research reading guide

---

For each paper, answer:

1. What exact question is asked?
2. Is the object a parameter, predictor, algorithm, selection procedure, or evaluation estimate?
3. Is the evidence a theorem, simulation, benchmark, observational analysis, or case study?
4. What did the authors actually establish?
5. Which assumptions or settings limit transfer?
6. What should change in the MHP workflow?

| Paper                      | Main object         | Warning                                                   | Workflow consequence                                                    |
|----------------------------|---------------------|-----------------------------------------------------------|-------------------------------------------------------------------------|
| Breiman (2001)             | Modeling cultures   | Elegant stochastic models may be weakly checked           | Compare transparent and algorithmic models on purpose-relevant evidence |
| Cawley & Talbot (2010)     | Selection procedure | Model selection itself can overfit                        | Evaluate complete selection procedures with nested CV or locked test    |
| Bengio & Grandvalet (2004) | CV variance         | No universal unbiased variance estimator from usual folds | Do not turn fold SD into an automatic interval                          |

Small replication portfolio:

1. Generate many datasets and compare tree-depth training/validation optimism.
2. Increase the number of hyperparameters tried and measure inner-to-outer gaps.
3. Compare within-dataset fold variation with across-dataset variation of mean CV performance.

For every notebook, pre-write the question and expected result, save seeds and versions, label axes and units, separate observation from interpretation, and state one limitation.

---

## Suggested seven-day study schedule

---

| Activity                        | Minutes |
|---------------------------------|---------|
| Retrieve Chapter 1 knowledge    | 15      |
| Work through central derivation | 40      |

| Activity                    | Minutes    |
|-----------------------------|------------|
| Run and alter proof code    | 35         |
| Inspect or create figure    | 25         |
| Extend estimator/evaluation | 40         |
| Break and diagnose          | 30         |
| Exit check and reflection   | 15         |
| <b>Total</b>                | <b>200</b> |

Day 11 may require two sessions. Nested validation is conceptually dense; drawing every inner and outer boundary is more valuable than memorising an API call.

Morning retrieval prompts:

- After Day 6: What does condition number measure, and where does a scaler fit?
- After Day 7: How do QR and SVD solve least squares?
- After Day 8: Why does Gaussian MLE produce OLS, and which interval is wider?
- After Day 9: Write the gradient and explain learning rate.
- After Day 10: Derive the mean baseline and define deployment risk.
- After Day 11: Which nested-CV loop selects and which evaluates?
- After Day 12: Why is one average score insufficient?

## Common beginner confusions

| Confusion                                       | Correction                                                                           |
|-------------------------------------------------|--------------------------------------------------------------------------------------|
| Scaling adds information                        | It changes representation only                                                       |
| Full rank means stable coefficients             | Full-rank matrices can be ill-conditioned                                            |
| SVD fixes multicollinearity                     | It diagnoses directions and selects a solution; it cannot identify redundant effects |
| OLS requires normal raw data                    | OLS fitting does not; particular inference formulas add assumptions                  |
| Residual equals true error                      | Residual uses estimated parameters and fitting constraints                           |
| Gradient descent is automatically more accurate | It is another optimiser and should reach the same OLS minimum                        |
| Lowest training error wins                      | Training error rewards flexibility and cannot establish generalisation               |
| Random split is always objective                | It can violate group and time structure                                              |
| Pipeline is code convenience                    | It protects the learning boundary                                                    |
| CV removes need for final evaluation            | Repeated selection can overfit development evidence                                  |
| Test $R^2 = 0$ equals training dummy            | Standard denominator uses evaluation-set mean                                        |
| Fold SD is a 95% interval                       | Fold scores are dependent and possibly heterogeneous                                 |

# Responsible interpretation in the KP context

---

Keep asking:

- Was each field genuinely known at appraisal?
- Are costs in the same base year and do they treat community contributions consistently?
- Does road distance always mean distance to an all-weather road?
- Is terrain scored consistently across survey teams?
- Do projects share contracts, contractors, valleys, or administrative processes?
- Did procurement policy change during the data period?
- Are delayed or abandoned remote projects absent from completed-project data?
- Does a random split place near-identical local conditions on both sides?
- Would systematic underprediction deny remote communities adequate budgets?
- Are subgroup differences supported by enough projects?

Numerical correctness, statistical validity, and institutional responsibility meet at these questions.

---

## Where Chapter 3 begins

---

Chapter 2 builds an honest framework around OLS and a nonlinear comparator. Chapter 3 can now add:

1. polynomial and interaction features;
2. ridge and lasso regularisation;
3. geometric and Bayesian views of penalties;
4. robust and quantile regression;
5. heteroskedasticity-robust and cluster-aware inference;
6. influence, leverage, and Cook's distance;
7. repeated and nested model comparison; and
8. a pre-registered benchmark study.

Every future model must travel through the same chain:

```
deployment question
-> permitted information
-> complete learning procedure
-> matching validation design
-> uncertainty and subgroup diagnostics
-> bounded decision
```

A trustworthy model is not certified by elegant mathematics or an impressive score alone. Trust grows from an auditable chain of stable computation, explicit assumptions, legitimate information, deployment-matched evaluation, quantified uncertainty, investigated failures, and restraint in the final claim.

# Chapter 3 — From a Trustworthy Baseline to a Research-Grade Regression Study

---

## Level 3 Practitioner: eight days of model design, diagnosis, and disciplined comparison

**Central promise.** Chapter 2 taught you to build a stable model and evaluate it honestly. By the end of Chapter 3, you will be able to design missing-data and categorical-data preprocessing without leakage, express nonlinear and conditional relationships through engineered features, derive ridge and lasso regression, diagnose multicollinearity and influential projects, distinguish robust estimation from robust inference, estimate conditional cost quantiles, and run a pre-specified benchmark study whose conclusions another researcher can audit.

The learner is still protected from unnecessary cognitive overload. Every new technique begins with a question, a small numerical example, and an equation developed from familiar ideas. Library calls come only after the mechanism is visible.

The chapter also marks an important change of attitude. A practitioner does not ask only:

“Which model produced the smallest number?”

A practitioner asks:

- What exact relationship did the feature representation permit?
- Which observations determined the answer?
- Which assumptions support the uncertainty statement?
- Did model selection occur inside the evaluation boundary?
- Is an apparent improvement large enough and stable enough to matter?
- Which communities or project types bear the cost of failure?

---

## Why the original draft required reconstruction

The earlier Chapter 3 contained valuable headings but compressed preprocessing, feature engineering, multicollinearity, diagnostics, ridge, lasso, elastic net, robust covariance, and subgroup debugging into fewer than 2,000 words. It also jumped from section 15 to section 25 and repeated several shortcuts that would mislead a beginner.

This reconstruction corrects those problems.

1. **Scaling is stated accurately.** An unpenalised linear model does not automatically treat a numerically large feature as more important. Scaling matters for conditioning, optimisation, distance-based models, and penalties whose meaning depends on coefficient units.
2. **Feature engineering becomes model specification.** A negative quadratic coefficient does not, by itself, prove a meaningful optimum. Interactions and transformations must be interpreted through derivatives, ranges, and deployment timing.



3. **VIF becomes a diagnostic, not a ritual cutoff.** Values such as 5 or 10 are conventions, not universal laws. The consequence depends on prediction, explanation, design, sample size, and the variables' roles.
4. **Diagnostics are separated by purpose.** A residual plot can reveal a pattern but cannot diagnose causation. A robust covariance estimator changes uncertainty calculations; it does not repair an incorrect mean function or endogeneity.
5. **Regularisation is derived.** Ridge, lasso, and elastic net are not presented as three buttons. Their objectives, geometry, scaling requirements, algorithms, and inferential limits are developed explicitly.
6. **Research papers become working material.** Each paper study identifies its question, design, result, limitation, and a replication task.

## Prerequisite checkpoint

Before beginning, retrieve these ideas from Chapters 1 and 2 without notes:

- $\hat{y} = X\hat{\beta}$  and  $e = y - \hat{y}$ ;
- the OLS objective and gradient;
- rank, singular values, and condition number;
- why learned preprocessing belongs inside a pipeline;
- train, validation, test, grouped CV, temporal CV, and nested CV;
- confidence interval versus prediction interval; and
- MAE, RMSE, held-out  $R^2$ , signed error, and subgroup evaluation.

If these are not yet explainable, revisit the relevant exit checks. Practitioner-level work rests on them.

## Learning outcomes

At the end of Chapter 3, you should be able to:

- represent missingness with an indicator and explain MCAR, MAR, and MNAR without claiming they can always be diagnosed from observed data;
- explain why simple imputation changes a distribution and why multiple imputation serves a different inferential purpose;
- compare one-hot, ordinal, frequency, and target encoding by the information and assumptions they introduce;
- implement cross-fitted target encoding and explain why ordinary target means leak;
- distinguish a transformation selected from prior knowledge from one selected after examining outcomes;
- construct centred polynomial, interaction, log, ratio, piecewise, and spline features;
- interpret a quadratic model using its derivative and observed range;
- interpret a continuous-by-binary and continuous-by-continuous interaction;
- apply the hierarchy principle to polynomial and interaction models;
- derive the ridge solution and its singular-value shrinkage factors;
- explain the constrained geometry and Gaussian-prior MAP interpretation of ridge;
- derive the lasso soft-thresholding solution for an orthonormal design;
- explain why lasso can set coefficients to zero and why zero does not establish scientific irrelevance;
- describe elastic net's treatment of correlated predictors;
- calculate leverage, studentised residuals, Cook's distance, and VIF from definitions;
- distinguish an outlying target, a high-leverage design point, and an influential observation;
- construct HC0 and explain HC1–HC3 heteroskedasticity-consistent covariance estimators;
- construct the cluster "sandwich" and state why five districts are too few for casual asymptotic confidence;
- distinguish robust standard errors, robust regression, and quantile regression;
- derive Huber loss and pinball loss;
- fit and evaluate conditional median and upper-quantile models;
- verify empirical quantile coverage on held-out data;

- design a locked-test benchmark containing all preprocessing, feature generation, and tuning; and
- report a result whose uncertainty, subgroup limitations, and post-selection status are explicit.

## The eight-day route

| Day    | Central idea                               | Problem resolved                                                   |
|--------|--------------------------------------------|--------------------------------------------------------------------|
| Day 13 | Missingness and categorical representation | “Cleaning” can alter the question and leak the answer              |
| Day 14 | Nonlinearity and interactions              | A straight additive surface may omit known mechanisms              |
| Day 15 | $L_2$ shrinkage                            | Correlated, numerous features create unstable estimates            |
| Day 16 | $L_1$ sparsity and mixed penalties         | Prediction may need shrinkage and a smaller active set             |
| Day 17 | Diagnostic geometry                        | A few design points or weak directions can dominate coefficients   |
| Day 18 | Robust and clustered covariance            | Classical standard errors can use the wrong variance structure     |
| Day 19 | Alternative conditional targets and losses | The conditional mean and squared loss are not every decision       |
| Day 20 | Research workflow                          | Flexible analysis can turn noise into a publishable-looking result |

Defensible data representation



Explicit feature hypothesis



Regularised candidate models



Assumption and influence audit



Pre-specified benchmark

---

## Running case and prediction contract

We continue the fictional microhydro power (MHP) appraisal case. One row is one project. The target is final cost in constant 2025 million PKR. Prediction occurs at technical appraisal, before procurement and construction.

The Chapter 2 generator produced:

- project ID;
- district;
- start year;
- planned capacity;
- estimated cable length;
- road distance;

- terrain index;
- a prohibited final material bill; and
- actual project cost.

Chapter 3 adds realistic data-quality problems and candidate features:

- some road-distance surveys are missing;
- contractor experience is recorded as an ordered count;
- access mode is categorical;
- capacity and cable length interact;
- the cost–capacity relationship is curved;
- remote projects have larger conditional variance; and
- a few extreme but legitimate projects test robustness.

## Practitioner data extension

Save the Chapter 2 generator as `chapter2_data.py`, then save the following as `chapter3_data.py`.

```
import numpy as np
import pandas as pd

from chapter2_data import make_mhp_projects

def make_mhp_practitioner_data(n=520, seed=3030):
 """Extend Chapter 2's fictional projects for Chapter 3 laboratories."""
 df = make_mhp_projects(n=n, seed=seed)
 rng = np.random.default_rng(seed + 1)

 # Appraisal-time variables.
 df["contractor_experience_projects"] = rng.poisson(lam=5.0, size=n)
 df["access_mode"] = np.select(
 [
 df["road_distance_km"] <= 5.0,
 df["road_distance_km"] <= 18.0,
],
 ["road", "mixed"],
 default="porter_or_air",
)

 # A noisy duplicate creates interpretable multicollinearity.
 df["surveyed_route_km"] = (
 df["road_distance_km"]
 + 0.35 * df["estimated_cable_km"]
 + rng.normal(0.0, 0.35, size=n)
)

 # Missingness is more likely in difficult access modes and early years.
 missing_probability = (
 0.03
 + 0.15 * (df["access_mode"] == "porter_or_air").astype(float)
 + 0.07 * (df["start_year"] <= 2018).astype(float)
)
 missing_road = rng.random(n) < missing_probability
 df["road_distance_observed_km"] = df["road_distance_km"].mask(missing_road)

 # A handful of extreme but genuine projects.
 candidate_extremes = df.index[df["road_distance_km"] > 20.0].to_numpy()
 extreme_count = min(8, candidate_extremes.size)
 extreme_index = rng.choice(candidate_extremes, size=extreme_count, replace=False)
 df["extreme_logistics_event"] = 0
 df.loc[extreme_index, "extreme_logistics_event"] = 1
 df.loc[extreme_index, "actual_cost_2025_million_pkr"] += rng.uniform(
 18.0, 35.0, size=extreme_count
)
)
```

```

return df

if __name__ == "__main__":
 projects = make_mhp_practitioner_data()
 projects.to_csv("mhp_projects_chapter3.csv", index=False)
 print(projects.head())
 print(projects.isna().mean().sort_values(ascending=False).head())

```

The generator deliberately retains the complete `road_distance_km` so that the learner can verify how missingness was created. In an actual incomplete dataset that hidden truth would be unavailable. It must never be used as a feature when evaluating the imputation workflow.

## Software setup

```
python -m pip install numpy pandas scipy matplotlib scikit-learn statsmodels
```

The main predictive capstone uses NumPy, pandas, Matplotlib, SciPy, and scikit-learn. Statsmodels is used for inferential and influence demonstrations. Record exact versions in every benchmark report.

# Day 13 — Preprocessing Is Part of the Model

**Today's central idea:** Missing-value handling and categorical encoding are learned representations. They change what information the model receives and therefore belong inside the fitted and evaluated procedure.

## 13.1 Begin with a missingness indicator

For a variable  $X_j$ , define:

$$R_{ij} = \begin{cases} 1, & X_{ij} \text{ is observed,} \\ 0, & X_{ij} \text{ is missing.} \end{cases}$$

The missingness mechanism describes how  $R$  relates to observed and unobserved data. The terms below concern conditional relationships, not moral judgements about data quality.

## 13.2 MCAR, MAR, and MNAR

Let  $X_{obs}$  denote observed values and  $X_{mis}$  missing values.

### Missing completely at random (MCAR)

$$P(R \mid X_{obs}, X_{mis}) = P(R).$$

Missingness is unrelated to observed or missing data. A random scanner failure affecting forms independently of project characteristics might approximate MCAR.

## Missing at random (MAR)

$$P(R \mid X_{obs}, X_{mis}) = P(R \mid X_{obs}).$$

After conditioning on observed variables, missingness no longer depends on the unseen value. If road distance is more often missing in early years and particular access modes, and those variables are observed and adequately modeled, MAR may be defensible.

## Missing not at random (MNAR)

Even after conditioning on observed information, missingness depends on the missing value or another unobserved quantity. For example, extremely remote surveys may be selectively omitted precisely because the true access distance was difficult to establish.

These mechanisms cannot generally be proven from the observed dataset alone. MNAR sensitivity analysis requires explicit alternative assumptions.

## 13.3 What complete-case analysis changes

Complete-case analysis deletes any row with a missing required field. It is simple but can:

- reduce sample size;
- change the mixture of districts and project types;
- exclude precisely the remote projects of interest; and
- produce biased estimates unless the missingness and analysis conditions justify deletion.

Always compare the retained and excluded rows on observed variables. This cannot prove that deletion is safe, but it can reveal obvious selection.

```
from chapter3_data import make_mhp_practitioner_data

df = make_mhp_practitioner_data()
observed = df[df["road_distance_observed_km"].notna()]
missing = df[df["road_distance_observed_km"].isna()]

comparison_columns = [
 "terrain_index",
 "planned_capacity_kw",
 "actual_cost_2025_million_pkr",
]

print("Observed road distance, n =", len(observed))
print(observed[comparison_columns].mean())
print("\nMissing road distance, n =", len(missing))
print(missing[comparison_columns].mean())
print("\nMissingness by access mode")
print(df.groupby("access_mode")["road_distance_observed_km"].apply(lambda x: x.isna().mean()))
```

The target is shown here for a retrospective missingness audit. It must not be used by an appraisal-time imputer.

## 13.4 Simple imputation is a learned rule

Median imputation replaces missing training values with the training median:

$$x_{ij}^{imp} = \begin{cases} x_{ij}, & R_{ij} = 1, \\ \text{median}(X_{j,train}), & R_{ij} = 0. \end{cases}$$

This preserves sample size and is often a useful predictive baseline. It does not recreate the missing values or preserve the original distribution:

- many observations pile up at one value;
- variance is reduced;
- relationships with other variables are weakened; and
- ordinary inferential formulas that pretend imputed values were observed can understate uncertainty.

## 13.5 Missing indicators

Add a binary feature:

$$M_{ij} = 1 - R_{ij}.$$

The model can then learn a separate offset for observations whose value was imputed. This can improve prediction if missingness carries stable signal. It does not make an MNAR mechanism ignorable or turn “missing” into a causal explanation.

```
import numpy as np
from sklearn.impute import SimpleImputer

x_train = np.array([[2.0], [4.0], [np.nan], [10.0], [np.nan]])
x_future = np.array([[np.nan], [30.0]])

imputer = SimpleImputer(strategy="median", add_indicator=True)
train_transformed = imputer.fit_transform(x_train)
future_transformed = imputer.transform(x_future)

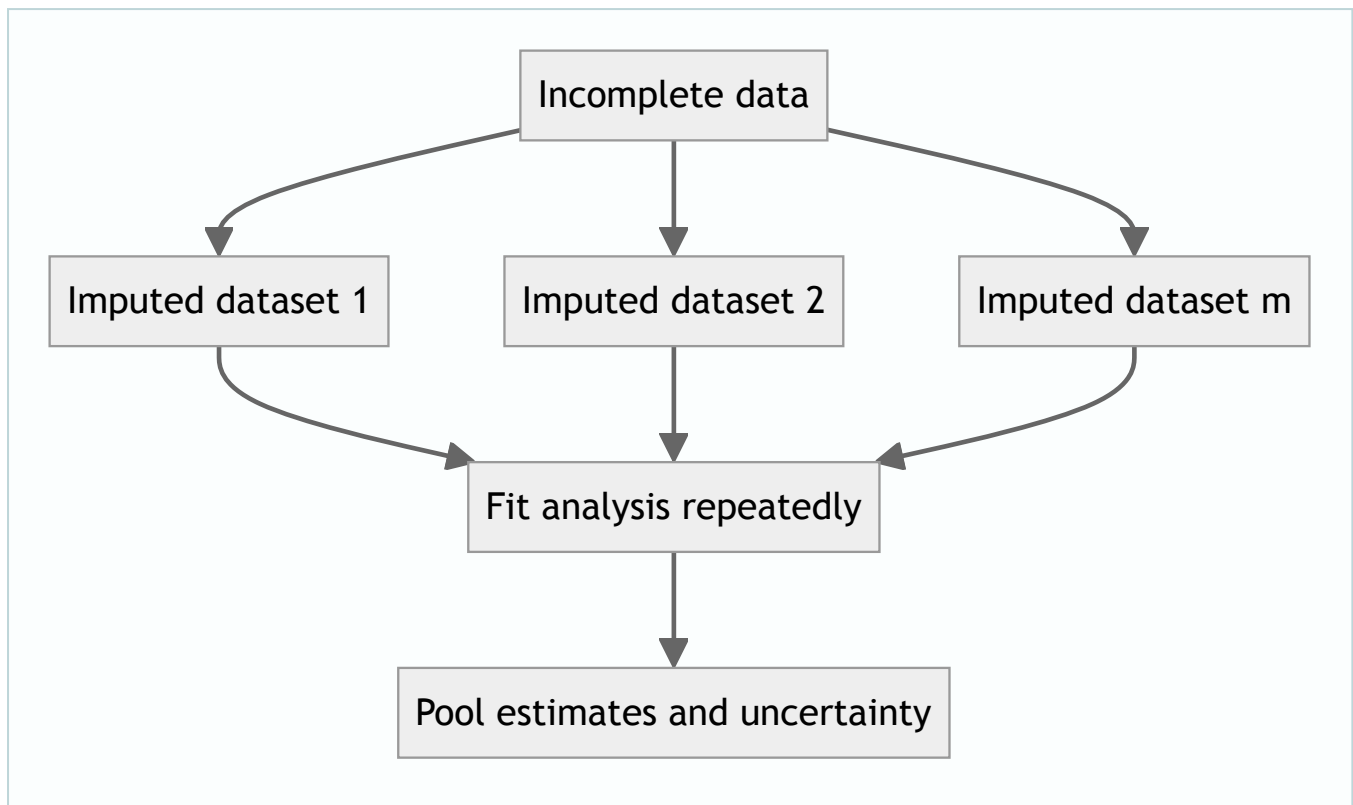
print("Learned median:", imputer.statistics_)
print("Training output:\n", train_transformed)
print("Future output:\n", future_transformed)
```

The future missing value receives the training median and an indicator of 1. The value 30 remains 30 and has indicator 0.

## 13.6 Single imputation versus multiple imputation

Single imputation creates one completed dataset. Multiple imputation creates  $m$  plausible completed datasets under an imputation model, fits the analysis to each, and combines estimates and uncertainties.

Conceptually:



Multiple imputation is particularly important for parameter inference because it carries missing-value uncertainty into the result. It is not automatically superior for every prediction pipeline, and its imputation model must respect training boundaries and deployment availability.

## Rubin's pooling rules

Suppose completed dataset  $l$  produces estimate  $\hat{Q}_l$  and estimated within-imputation variance  $U_l$ , for  $l = 1, \dots, m$ . Pool the point estimates:

$$\bar{Q} = \frac{1}{m} \sum_{l=1}^m \hat{Q}_l.$$

Average their within-imputation variances:

$$\bar{U} = \frac{1}{m} \sum_{l=1}^m U_l.$$

Measure between-imputation variation:

$$B = \frac{1}{m-1} \sum_{l=1}^m (\hat{Q}_l - \bar{Q})^2.$$

The pooled variance is

$$T = \bar{U} + \left(1 + \frac{1}{m}\right) B.$$

The formulas are short enough to implement directly:

```
import numpy as np
```



```
def rubins_pooling(estimates, within_variances):
 estimates = np.asarray(estimates, dtype=float)
 within_variances = np.asarray(within_variances, dtype=float)
 if estimates.ndim != 1 or estimates.shape != within_variances.shape:
 raise ValueError("supply one within-variance for every estimate")
 if len(estimates) < 2:
 raise ValueError("pooling requires at least two imputations")

 m = len(estimates)
 q_bar = estimates.mean()
 u_bar = within_variances.mean()
 between = estimates.var(ddof=1)
 total = u_bar + (1.0 + 1.0 / m) * between
 return {"estimate": q_bar, "within": u_bar,
 "between": between, "variance": total,
 "standard_error": np.sqrt(total)}

pooled = rubins_pooling(
 estimates=[2.1, 2.4, 2.2, 2.3, 2.0],
 within_variances=[0.16, 0.18, 0.15, 0.17, 0.16],
)
print(pooled)
assert np.isclose(pooled["estimate"], 2.2)
assert pooled["variance"] > pooled["within"]
```

The final assertion proves that differing plausible imputations add uncertainty beyond the ordinary within-dataset variance.

$\bar{U}$  reflects ordinary estimation uncertainty if the completed values were known.  $B$  reflects sensitivity to which plausible missing values were supplied. The finite- $m$  multiplier accounts for using only a limited number of imputations.

These rules are not valid merely because several datasets were generated. The imputation and analysis models must be compatible enough for the intended estimand, all variables governing missingness and the analysis should be considered, and pooling degrees of freedom require appropriate finite-sample formulas. For predictive assessment, imputation must occur separately inside each training fold, and performance is evaluated on the observed held-out outcomes—not by pooling a leaky completion of the entire dataset.

## 13.7 Categorical variables are hypotheses about similarity

Encoding does more than convert strings to numbers. It tells the model which category differences can share information.

### One-hot encoding

For categories A, B, and C, create indicator columns. With an intercept, use a reference coding or an equivalent constrained parameterisation for unique coefficients.

If A is the reference:

$$\hat{y} = \beta_0 + \beta_B I(B) + \beta_C I(C).$$

- $\beta_0$  is the fitted value for A at the other features' reference values;
- $\beta_B$  is the conditional difference B minus A; and
- $\beta_C$  is the conditional difference C minus A.

Changing the reference changes the coefficient labels, not fitted values.

## Ordinal encoding

Mapping categories to 0, 1, 2 imposes order. A linear model further treats adjacent gaps as equal unless transformed. Use it only when that structure is intended.

## Frequency encoding

Replacing a category by its training frequency says commonness is predictive. Two categories with the same frequency become indistinguishable.

## Target encoding

Replacing category  $c$  with its target mean uses outcome information:

$$TE(c) = \frac{1}{n_c} \sum_{i:C_i=c} y_i.$$

Naively applying this to the same rows used to calculate the mean leaks each row's outcome into its own feature, especially for rare categories.

## 13.8 Smoothed target encoding

A smoothed estimate blends the category mean with the global training mean:

$$\widetilde{TE}(c) = \frac{n_c \bar{y}_c + m \bar{y}}{n_c + m},$$

where  $m > 0$  controls shrinkage. Rare categories are pulled more strongly toward the global mean.

Smoothing reduces variance but does not solve same-row leakage. Training encodings still require cross-fitting.

## 13.9 Cross-fitted target encoding from scratch

For each training fold:

1. calculate category statistics on the other folds;
2. encode the held-out fold with those statistics;
3. combine the out-of-fold encodings; and
4. after training the downstream model, fit one encoder on all development rows for future transformation.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold

def smoothed_mapping(category, target, smoothing=10.0):
 frame = pd.DataFrame({"category": category, "target": target})
 global_mean = frame["target"].mean()
 stats = frame.groupby("category")["target"].agg(["count", "mean"])
 encoded = (
 stats["count"] * stats["mean"] + smoothing * global_mean
) / (stats["count"] + smoothing)
 return encoded.to_dict(), global_mean

def cross_fitted_target_encode(category, target, folds=5, smoothing=10.0, seed=42):
 category = pd.Series(category).reset_index(drop=True)
 target = pd.Series(target, dtype=float).reset_index(drop=True)
```

```

encoded = np.empty(len(category), dtype=float)

splitter = KFold(n_splits=folds, shuffle=True, random_state=seed)
for fit_index, held_index in splitter.split(category):
 mapping, fallback = smoothed_mapping(
 category.iloc[fit_index],
 target.iloc[fit_index],
 smoothing=smoothing,
)
 encoded[held_index] = (
 category.iloc[held_index].map(mapping).fillna(fallback).to_numpy()
)

final_mapping, final_fallback = smoothed_mapping(
 category, target, smoothing=smoothing
)
return encoded, final_mapping, final_fallback

category = pd.Series(["A", "A", "B", "B", "C", "rare"])
target = pd.Series([10.0, 12.0, 20.0, 23.0, 31.0, 100.0])

encoded, mapping, fallback = cross_fitted_target_encode(
 category, target, folds=3, smoothing=5.0
)
print("Cross-fitted training values:", encoded)
print("Final deployment mapping:", mapping)
print("Unseen-category fallback:", fallback)

```

For grouped or temporal data, the inner cross-fitting splitter must also respect groups or time. Random K-fold target encoding can leak future or district information even though it avoids same-row leakage.

## 13.10 A leakproof mixed-type pipeline

```

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = [
 "start_year",
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 "terrain_index",
 "contractor_experience_projects",
]
categorical_features = ["district", "access_mode"]

numeric_pipeline = Pipeline(
 steps=[
 ("impute", SimpleImputer(strategy="median", add_indicator=True)),
 ("scale", StandardScaler()),
]
)

categorical_pipeline = Pipeline(
 steps=[
 ("impute", SimpleImputer(strategy="most_frequent")),
 ("one_hot", OneHotEncoder(handle_unknown="ignore", drop="first")),
]
)

preprocessor = ColumnTransformer(
 transformers=[

```

```

 ("numeric", numeric_pipeline, numeric_features),
 ("categorical", categorical_pipeline, categorical_features),
]
)

ridge_pipeline = Pipeline(
 steps=[
 ("preprocess", preprocessor),
 ("model", Ridge(alpha=1.0)),
]
)

```

The chapter has not yet justified `alpha=1.0`; Day 15 will derive and tune it. Today's point is the information boundary.

## 13.11 Reading guide and discussion prompts: Rubin on missing data

**Paper:** Donald B. Rubin (1976), "Inference and Missing Data", *Biometrika* 63(3), 581–592.

### The question

Under what conditions can an analysis ignore the process that caused data to be missing?

### The contribution

Rubin formalised relationships among the data model, missingness process, observed values, and missing values. The paper established conditions under which likelihood or sampling-based analyses may ignore the missingness mechanism, with important distinctions among inferential frameworks.

### What the beginner should retain

1. "Missing at random" is conditional on observed information; it does not mean values disappeared for no reason.
2. Whether missingness can be ignored depends on both the mechanism and the analysis.
3. Observed data alone generally cannot rule out MNAR.
4. Filling blanks is not the same as accounting for missing-data uncertainty.

### MHP application

If road distance is missing more often for remote sites, median imputation may support a predictive baseline but can hide a systematic access problem. A defensible report should compare missingness by observed terrain, year, district, and access mode, then conduct sensitivity analysis for plausible unobserved distances.

### Limitation

The paper provides a general inferential framework, not an automatic recipe for a particular MHP database. The analyst must still argue which variables govern recording, whether they were observed, and whether the imputation model is adequate.

### Replication prompt

Simulate the same complete data under MCAR, MAR, and MNAR deletion. Compare complete-case estimates, median-imputed predictions, and a model that includes a missing indicator. Separate predictive error from coefficient bias.

## 13.12 Day 13 build, break, and reflect

### Build

1. Produce a missingness table by year, district, access mode, and terrain.
2. Compare complete-case and median-plus-indicator pipelines under forward validation.
3. One-hot encode district and explain the reference category.
4. Cross-fit a target encoding for a high-cardinality fictional contractor ID.

### Break

1. Fit the imputer on the whole dataset.
2. Calculate target means on the same rows they encode.
3. assign arbitrary ordinal numbers to districts and fit a linear slope.
4. Delete incomplete projects and call the remaining sample representative without checking.

### Reflect

Write an “information receipt” for every generated column: what raw fields it used, whether it used **y**, which rows fitted it, and when it would be available at deployment.

### Day 13 exit check

You are ready for Day 14 when you can explain why MAR is not “missing for no reason,” why a missing indicator does not solve MNAR, and why target encoding needs two levels of protection: cross-fitting inside development data and separation from the final test.

---

## Day 14 — Feature Engineering as Model Specification

---

**Today’s central idea:** Creating a feature changes the family of relationships a model can express. A feature is a mathematical hypothesis about the world, not a free source of accuracy.

**Symbolic-calculus bridge.** Day 0E.5a expanded  $(y - mx - c)^2$ , differentiated it term by term, and checked the result numerically; Chapter 1 Day 5 repeated that workflow for a matrix objective. The symbols  $d/dx$ ,  $\partial/\partial x$ , and  $\nabla$  below mean ordinary slope, one-coordinate slope while other inputs are fixed, and the vector of all coordinate slopes, respectively. Revisit those two worked derivations before continuing if this notation is not yet readable.

### 14.1 Linear regression is linear in parameters

Consider:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2.$$

The prediction is curved in **x** but linear in the parameters  $\beta_0, \beta_1, \beta_2$ . OLS can fit it because the design matrix contains columns **1, x, x<sup>2</sup>**:

$$X_{poly} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix}.$$

Feature engineering alters the columns. The fitting machinery remains linear least squares or its regularised variant.

## 14.2 Centre before forming powers

Let:

$$x_c = x - c,$$

where  $c$  is a meaningful reference, often the training mean or a policy-relevant value. Fit:

$$\hat{y} = \gamma_0 + \gamma_1 x_c + \gamma_2 x_c^2.$$

Benefits:

- $\gamma_0$  is the fitted value at  $x = c$ ;
- $\gamma_1$  is the slope at  $x = c$ ;
- correlation between  $x_c$  and  $x_c^2$  is often reduced; and
- coefficient magnitudes can be easier to compute and explain.

The centre must be learned from training data when it is data-derived.

## 14.3 Interpret a quadratic through its derivative

For:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2,$$

the instantaneous slope is:

$$\frac{df(x)}{dx} = \beta_1 + 2\beta_2 x.$$

A stationary point occurs at:

$$x^* = -\frac{\beta_1}{2\beta_2},$$

provided  $\beta_2 \neq 0$ .

- If  $\beta_2 < 0$ , the curve is concave and the stationary point is a maximum.
- If  $\beta_2 > 0$ , it is convex and the stationary point is a minimum.

But a negative  $\beta_2$  does not “confirm an optimum.” The stationary point must lie in a credible feature range, the specification must be adequate, uncertainty must be considered, and the relationship remains associational unless causally identified.

## 14.4 Code proof: recover a turning point

```
import numpy as np

rng = np.random.default_rng(141)
capacity = rng.uniform(80.0, 850.0, size=300)
capacity_center = capacity.mean()
x = capacity - capacity_center

cost = 50.0 + 0.08 * x + 0.00008 * x**2 + rng.normal(0.0, 4.0, size=x.size)
X = np.column_stack([np.ones(x.size), x, x**2])
beta = np.linalg.lstsq(X, cost, rcond=None)[0]

turning_centered = -beta[1] / (2.0 * beta[2])
turning_original = turning_centered + capacity_center

print("Coefficients:", beta)
print("Stationary point in original capacity units:", turning_original)
print("Observed capacity range:", capacity.min(), capacity.max())
```

The true simulated curve is convex, so the stationary point is a minimum. Whether it lies in the observed range must be checked before interpretation.

## 14.5 Continuous-by-continuous interactions

Fit:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2.$$

The effect of  $x_1$  depends on  $x_2$ :

$$\frac{\partial \hat{y}}{\partial x_1} = \beta_1 + \beta_3 x_2.$$

Similarly:

$$\frac{\partial \hat{y}}{\partial x_2} = \beta_2 + \beta_3 x_1.$$

Once an interaction is present,  $\beta_1$  is not “the effect of  $x_1$  everywhere.” It is the slope of  $x_1$  when  $x_2 = 0$ . Centring can make that reference meaningful.

For MHP projects, cable-length cost may rise more steeply when road access is poor:

$$\widehat{cost} = \dots + \beta_1 cable + \beta_2 road + \beta_3 (cable \times road).$$

## 14.6 Continuous-by-binary interactions

Let  $D = 1$  for `porter_or_air` access and 0 for the reference access category:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 D + \beta_3 x D.$$

For  $D = 0$ :

$$\hat{y} = \beta_0 + \beta_1 x.$$

For  $D = 1$ :

$$\hat{y} = (\beta_0 + \beta_2) + (\beta_1 + \beta_3)x.$$

$\beta_2$  changes the intercept and  $\beta_3$  changes the slope.

## 14.7 The hierarchy principle

If a model includes  $x_1x_2$ , usually retain  $x_1$  and  $x_2$ . If it includes  $x^2$ , retain  $x$ . This is the hierarchy—or marginality—principle.

Reasons:

- the interaction's meaning depends on the lower-order terms;
- excluding them imposes special constraints that are rarely intended; and
- predictions can change under harmless shifts of the feature origin.

Hierarchy is a default modeling discipline, not an inviolable theorem. Departures require a substantive constraint that can be defended.

## 14.8 Ratios are strong assumptions

The ratio:

$$r = \frac{x_1}{x_2}$$

assumes that proportional comparison is meaningful. Risks include:

- instability when  $x_2$  is near zero;
- measurement error in both numerator and denominator;
- loss of separate size information; and
- accidental leakage if the denominator is measured after prediction time.

If “cost per kW” is used as a target while planned capacity is also a feature, converting predictions back to total cost requires careful weighting and error interpretation. A low error in ratios need not mean a low portfolio budget error.

## 14.9 Log transformations

For positive  $x$ :

$$z = \log x.$$

For nonnegative  $x$ , analysts often use:

$$z = \log(1 + x).$$

These transformations compress large values and can express multiplicative relationships. They do not “make data normal” by guarantee, and normal feature distributions are not an OLS requirement.

Interpretation examples:

- **Level-log:**  $y = \beta_0 + \beta_1 \log x$ . A 1% change in  $x$  is associated approximately with  $0.01\beta_1$  units of  $y$  for small changes.
- **Log-level:**  $\log y = \beta_0 + \beta_1 x$ . A one-unit change in  $x$  is associated approximately with a  $100\beta_1$  percent change in  $y$  when  $\beta_1$  is small.
- **Log-log:**  $\log y = \beta_0 + \beta_1 \log x$ .  $\beta_1$  is an elasticity under the model.



When predicting on a log target,  $\widehat{\exp(\log y)}$  is generally a conditional median under a log-error model, not automatically the conditional mean. Retransformation bias needs explicit treatment.

## 14.10 Piecewise linear features

For a knot  $k$ , define the hinge:

$$(x - k)_+ = \max(0, x - k).$$

Fit:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 (x - k)_+.$$

- Below  $k$ , the slope is  $\beta_1$ .
- Above  $k$ , the slope is  $\beta_1 + \beta_2$ .

This expresses a slope change without forcing a global parabola. The knot must be pre-specified or selected inside validation.

## 14.11 Splines: flexible curves built from controlled pieces

A spline represents a curve as a weighted combination of basis functions:

$$f(x) = \sum_{m=1}^M \theta_m B_m(x).$$

The basis functions are fixed once degree and knots are defined; the coefficients  $\theta_m$  are learned. Splines can fit local curvature more gently than high-degree global polynomials.

The number and location of knots control flexibility. They are hyperparameters and belong inside the selection procedure.

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import SplineTransformer, StandardScaler

spline_model = Pipeline(
 steps=[
 (
 "spline",
 SplineTransformer(
 n_knots=5,
 degree=3,
 include_bias=False,
),
),
 ("scale", StandardScaler()),
 ("model", Ridge(alpha=1.0)),
]
)
```

The spline shown is for one numeric feature. A full mixed-type model uses a `ColumnTransformer` so only selected columns receive spline expansion.

## 14.12 Feature explosion

With  $p$  original features, degree-2 expansion can create:

- $p$  first-order terms;
- $p$  squares; and
- $p(p - 1)/2$  pairwise interactions.

The number grows quadratically. High-degree expansion grows faster. This increases computation, multicollinearity, model-selection opportunity, and the risk of fitting noise. Domain knowledge and regularisation are therefore partners, not rivals.

## 14.13 Build an explicit engineering transformer

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin

class MHPFeatureEngineer(BaseEstimator, TransformerMixin):
 """Create appraisal-time features with training-learned centres."""

 def fit(self, X, y=None):
 X = pd.DataFrame(X).copy()
 required = {
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 }
 missing = required.difference(X.columns)
 if missing:
 raise ValueError(f"Missing required columns: {sorted(missing)}")

 self.capacity_center_ = X["planned_capacity_kw"].median()
 self.cable_center_ = X["estimated_cable_km"].median()
 self.road_center_ = X["road_distance_observed_km"].median()
 return self

 def transform(self, X):
 X = pd.DataFrame(X).copy()
 capacity_c = X["planned_capacity_kw"] - self.capacity_center_
 cable_c = X["estimated_cable_km"] - self.cable_center_
 road_c = X["road_distance_observed_km"] - self.road_center_

 X["capacity_centered"] = capacity_c
 X["capacity_centered_sq"] = capacity_c**2
 X["cable_road_interaction"] = cable_c * road_c
 X["road_above_18_km"] = np.maximum(
 X["road_distance_observed_km"] - 18.0,
 0.0,
)
 X["log1p_contractor_experience"] = np.log1p(
 X["contractor_experience_projects"]
)
 return X
```

Because missing road distance has not yet been imputed, this transformer must be placed after an appropriate imputation step or written to handle missing values deliberately. Pipeline order is part of the model.

## 14.14 Visualise before and after extrapolation

```
import numpy as np
import matplotlib.pyplot as plt

observed_x = np.linspace(80.0, 850.0, 150)
future_x = np.linspace(20.0, 1300.0, 300)

Illustrative fitted functions, not fitted project results.
quadratic = 20.0 + 0.03 * future_x + 0.00004 * future_x**2
hinge = 20.0 + 0.05 * future_x + 0.06 * np.maximum(future_x - 700.0, 0.0)

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(future_x, quadratic, label="quadratic")
ax.plot(future_x, hinge, label="piecewise linear")
ax.axvspan(observed_x.min(), observed_x.max(), alpha=0.15, label="observed range")
ax.set_xlabel("Planned capacity (kW)")
ax.set_ylabel("Illustrative fitted cost")
ax.set_title("Feature choice controls extrapolation")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Two models can fit the observed range similarly and diverge dramatically outside it. Validation cannot certify extrapolation where no comparable cases exist.

## 14.15 Day 14 build, break, and reflect

### Build

1. Centre capacity using the training median.
2. Add capacity squared, cable-by-road interaction, an 18 km hinge, and log contractor experience.
3. Compare additive, quadratic, and spline models with forward validation.
4. Plot partial fitted curves across the observed capacity range.

### Break

1. Choose a polynomial degree after examining final test error.
2. include an interaction but remove its main effects.
3. create a ratio with a near-zero denominator.
4. claim that a negative square coefficient proves a causal optimum.
5. extrapolate a fifth-degree polynomial far beyond observed capacity.

### Reflect

For every engineered feature, write the mechanism it represents, its units, its valid range, and the evidence that it will be available at appraisal.

### Day 14 exit check

You are ready for regularisation when you can interpret a quadratic through its derivative, explain an interaction using a conditional slope, and state why feature generation must be repeated inside each validation training fold.

---

# Day 15 — Ridge Regression: Shrinkage and Stability

---

## 15.1 The question for today

Suppose two predictors contain almost the same information. OLS can fit their combined contribution well while assigning a large positive coefficient to one and a large negative coefficient to the other. A small change in the sample may reverse those assignments.

Can we accept a little bias in order to obtain a more stable model with better predictions?

Ridge regression answers yes. It prefers coefficients that fit the observations while remaining collectively small.

## 15.2 From OLS to a penalised objective

Assume for the moment that:

- every numeric column has been centred and standardised using training data;
- the target has been centred; and
- the intercept has therefore been separated from the penalised coefficients.

OLS minimises residual sum of squares:

$$J_{\text{OLS}}(\beta) = \|y - X\beta\|_2^2.$$

Ridge adds the squared Euclidean length of the coefficient vector:

$$J_{\text{ridge}}(\beta) = \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad \lambda \geq 0.$$

The first term asks for fit. The second asks for restraint. The tuning parameter  $\lambda$  determines their trade-off.

- $\lambda = 0$  recovers OLS when the OLS solution is unique.
- A small positive  $\lambda$  introduces mild shrinkage.
- A very large  $\lambda$  pushes the penalised coefficients toward zero.
- The intercept is normally not penalised.

This chapter uses the objective above for derivations. Software packages sometimes divide the loss by  $n$ ,  $2$ , or  $2n$ . Their parameter called `alpha` or `lambda` is therefore not always numerically interchangeable. Always read the implementation's stated objective before copying a tuned value between packages.

## 15.3 Deriving the ridge solution

Expand the objective:

$$J(\beta) = (y - X\beta)^\top (y - X\beta) + \lambda \beta^\top \beta.$$

Its gradient is

$$\nabla_\beta J = -2X^\top y + 2X^\top X\beta + 2\lambda\beta.$$

Set the gradient to zero:

$$(X^T X + \lambda I) \hat{\beta}_{\text{ridge}} = X^T y.$$

For  $\lambda > 0$ ,

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

when every displayed coefficient is to be penalised. If a column of ones is retained in  $X$ , replace  $I$  with a penalty matrix whose intercept entry is zero.

## Why the inverse now exists

For any nonzero vector  $v$ ,

$$v^T (X^T X + \lambda I) v = \|Xv\|_2^2 + \lambda \|v\|_2^2 > 0$$

when  $\lambda > 0$ . The matrix is positive definite even if  $X^T X$  is singular. Ridge therefore produces a unique coefficient vector in settings where OLS coefficients are not uniquely identified.

That algebraic fact does **not** create information that the design never contained. If two columns are identical, ridge can stabilise their combined predictive effect, but it cannot tell a scientist which identical measurement is the true cause.

## 15.4 Why standardisation is part of the model

Consider two equivalent measurements:

- distance in kilometres; and
- the same distance in metres.

A one-unit coefficient measured per metre is 1,000 times smaller than the corresponding coefficient measured per kilometre. The ridge penalty acts on coefficient magnitude, so the unit choice changes the penalty unless features are placed on comparable scales.

For training mean  $\bar{x}_j$  and training standard deviation  $s_j$ ,

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

Standardisation makes a one-unit movement correspond to one training standard deviation. It does not make the features equally important, normally distributed, causal, or free of outliers.

The scaler must be fitted inside each training fold. Fitting it once on all observations lets validation values affect the representation used during training.

## 15.5 Ridge through singular-value decomposition

Let the centred design have singular-value decomposition

$$X = U \Sigma V^T,$$

where the nonzero singular values are  $\sigma_1, \dots, \sigma_r$ . Substitute this factorisation into the ridge solution:

$$\hat{\beta}_{\text{ridge}} = V \text{diag} \left( \frac{\sigma_j}{\sigma_j^2 + \lambda} \right) U^T y.$$

For comparison, the minimum-norm OLS solution uses  $1/\sigma_j$  in directions with nonzero singular values. A small  $\sigma_j$  can therefore amplify noise. Ridge replaces it with

$$\frac{\sigma_j}{\sigma_j^2 + \lambda}.$$

The fitted values can be written as

$$\hat{\mathbf{y}}_{\text{ridge}} = \mathbf{U} \text{diag} \left( \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \right) \mathbf{U}^\top \mathbf{y}.$$

Each data direction has a shrinkage factor

$$s_j(\lambda) = \frac{\sigma_j^2}{\sigma_j^2 + \lambda}.$$

Well-supported directions with large  $\sigma_j$  retain more of their signal. Weak directions with small  $\sigma_j$  are suppressed more strongly. This is the central stability mechanism.

## 15.6 Effective degrees of freedom

Ridge fitted values are linear in  $\mathbf{y}$ :

$$\hat{\mathbf{y}} = \mathbf{H}_\lambda \mathbf{y}, \quad \mathbf{H}_\lambda = \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top.$$

A useful measure of flexibility is

$$\text{df}_{\text{eff}}(\lambda) = \text{tr}(\mathbf{H}_\lambda) = \sum_{j=1}^r \frac{\sigma_j^2}{\sigma_j^2 + \lambda}.$$

This is not generally an integer.

- At  $\lambda = 0$ , it equals the rank of the centred design.
- As  $\lambda$  grows, it approaches zero for the penalised component.

Ridge can therefore use every feature while behaving like a less flexible fit.

## 15.7 Geometry: a circle meets an ellipse

The penalised form is equivalent, for a corresponding value of  $t$ , to the constrained problem

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_2^2 \leq t.$$

For two coefficients, the constraint is a circle. Residual-sum-of-squares contours are ellipses around the OLS solution. The ridge solution is the first point at which the smallest attainable ellipse touches the circle.

Unstable OLS direction



Add an L2 penalty



Shrink weak singular directions



Trade some bias for lower variance

Unlike the lasso constraint introduced tomorrow, the ridge circle has no corners. A smooth ellipse usually touches it away from an axis. Ridge shrinks coefficients but ordinarily does not make them exactly zero.

## 15.8 A ridge implementation from first principles

The following class deliberately exposes centring, scaling, and the linear solve. It is educational, not a replacement for a production estimator.

```
import numpy as np

class RidgeFromScratch:
 def __init__(self, alpha=1.0):
 if alpha < 0:
 raise ValueError("alpha must be non-negative")
 self.alpha = float(alpha)
```

```

def fit(self, X, y):
 X = np.asarray(X, dtype=float)
 y = np.asarray(y, dtype=float)

 self.x_mean_ = X.mean(axis=0)
 self.x_scale_ = X.std(axis=0, ddof=0)
 self.x_scale_[self.x_scale_ == 0] = 1.0
 self.y_mean_ = y.mean()

 Z = (X - self.x_mean_) / self.x_scale_
 y_centered = y - self.y_mean_

 system = Z.T @ Z + self.alpha * np.eye(Z.shape[1])
 self.coef_standardized_ = np.linalg.solve(system, Z.T @ y_centered)
 self.coef_ = self.coef_standardized_ / self.x_scale_
 self.intercept_ = self.y_mean_ - self.x_mean_ @ self.coef_
 return self

def predict(self, X):
 X = np.asarray(X, dtype=float)
 return self.intercept_ + X @ self.coef_

```

Verify it against `scikit-learn`. Both use the objective

$\|y - X\beta\|_2^2 + \alpha\|\beta\|_2^2$  for the penalised part.

```

import numpy as np
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(15)
x1 = rng.normal(size=120)
x2 = x1 + rng.normal(0.0, 0.03, size=120)
X = np.column_stack([x1, x2])
y = 4.0 * x1 + rng.normal(0.0, 0.8, size=120)

manual = RidgeFromScratch(alpha=5.0).fit(X, y)
library = make_pipeline(StandardScaler(), Ridge(alpha=5.0)).fit(X, y)

assert np.allclose(manual.predict(X), library.predict(X), atol=1e-10)
print(manual.coef_, manual.intercept_)

```

Use `np.linalg.solve`, not an explicitly computed matrix inverse. Solving the system is more accurate and efficient.

## 15.9 Seeing the coefficient path

A coefficient path shows estimates across candidate penalty strengths.

```

import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler

Z = StandardScaler().fit_transform(X)
alphas = np.logspace(-4, 4, 120)
path = np.vstack([
 Ridge(alpha=alpha).fit(Z, y).coef_
 for alpha in alphas
])

fig, ax = plt.subplots(figsize=(8, 5))
ax.semilogx(alphas, path[:, 0], label="x1")
ax.semilogx(alphas, path[:, 1], label="x2")

```



```
ax.set(xlabel="alpha", ylabel="standardised coefficient",
 title="Ridge coefficient path")
ax.legend()
plt.tight_layout()
plt.show()
```

The path is a diagnostic, not a licence to select an attractive coefficient after viewing final-test results. Candidate values must be tuned within the training and validation process.

## 15.10 The bias–variance trade-off

For an estimator  $\hat{f}(x)$  trained on varying samples, expected squared prediction error at  $x$  decomposes conceptually as

$$\mathbb{E}[(Y - \hat{f}(x))^2] = \underbrace{\sigma^2}_{\text{irreducible noise}} + \underbrace{\text{Bias}[\hat{f}(x)]^2}_{\text{systematic error}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{sample sensitivity}}.$$

Ridge intentionally increases bias relative to OLS. If it reduces variance by more, expected test error falls. It is not universally better: when the sample is large, predictors are few and well-conditioned, and the linear model is appropriate, OLS may already be excellent.

## 15.11 A Bayesian MAP interpretation

A **posterior distribution** describes what the assumed model says about plausible parameter values after combining the prior assumptions with the observed data through the likelihood. Its mode—the highest point—is called the maximum a posteriori (MAP) estimate.

Suppose

$$y \mid X, \beta \sim \mathcal{N}(X\beta, \sigma^2 I)$$

and place independent Gaussian priors on the standardised slopes:

$$\beta \sim \mathcal{N}(0, \tau^2 I).$$

Ignoring constants, the negative log posterior is

$$\frac{1}{2\sigma^2} \|y - X\beta\|_2^2 + \frac{1}{2\tau^2} \|\beta\|_2^2.$$

Multiplying by  $2\sigma^2$  gives the ridge objective with

$$\lambda = \frac{\sigma^2}{\tau^2}.$$

Thus the ridge solution is a maximum a posteriori estimate under this model. A tighter prior—smaller  $\tau^2$ —creates stronger shrinkage. This is an interpretation, not proof that the independent zero-centred Gaussian prior is scientifically correct.

## 15.12 What ridge does not provide automatically

Ridge does not by itself provide:

- a causally interpretable coefficient;
- a valid classical OLS  $p$ -value after tuning;
- immunity to leakage or distribution shift;

- robustness to extreme target values;
- a principled missing-data analysis; or
- selection of a small set of variables.

If the penalty is selected after comparing validation performance, the final estimator includes a model-selection step. Conventional fixed-model uncertainty formulas do not automatically account for that selection.

## 15.13 Reading guide and discussion prompts: Hoerl and Kennard (1970)

Arthur Hoerl and Robert Kennard's paper, *Ridge Regression: Biased Estimation for Nonorthogonal Problems*, made a provocative proposal: when predictors are nonorthogonal, a biased estimator can have lower mean squared error than least squares.

Read it with five questions.

1. **Problem.** What instability arises when the information matrix is ill-conditioned?
2. **Intervention.** How is a positive constant added to the diagonal of the normal equations?
3. **Evidence.** Which results concern coefficient mean squared error rather than test prediction error as used today?
4. **Judgment.** How was the ridge constant to be chosen, and how does this differ from nested or temporal validation?
5. **Limitation.** Which claims depend on the linear model and on the scale of predictors?

### Replication task

Simulate  $\mathbf{x}_2 = \mathbf{x}_1 + \epsilon$  for decreasing noise levels. Across 1,000 training samples, compare OLS and ridge on:

- coefficient variance;
- coefficient bias;
- coefficient mean squared error; and
- test prediction error.

Do not search for one sample that supports the claim. Average across repeated samples and plot the distributions.

## 15.14 Day 15 build, break, and reflect

### Build

1. Standardise the MHP numeric features inside a pipeline.
2. Plot a ridge path for `road_distance_observed_km` and `surveyed_route_km`.
3. Tune `alpha` using only forward validation years.
4. Report both validation MAE and coefficient stability across folds.

### Break

1. Penalise unscaled kilometres and metres as if their coefficient magnitudes were comparable.
2. tune `alpha` on the locked test set.
3. copy an `alpha` from software with a differently normalised objective.
4. describe a shrunken coefficient as unbiased.
5. treat a stable predictive coefficient as a causal estimate.

### Reflect

If ridge improves validation MAE by 0.05 million PKR but greatly stabilises coefficients, is it operationally preferable? State the decision criterion before looking at the answer.

## Day 15 exit check

You are ready for lasso when you can derive  $(X^T X + \lambda I)^{-1} X^T y$ , explain the singular-direction shrinkage factor, and say why standardisation changes the meaning of the penalty.

---

# Day 16 — Lasso, Elastic Net, and Sparse Models

---

## 16.1 The question for today

Ridge retains every feature. What if the practitioner also wants a model whose fitted coefficient vector contains exact zeros?

The lasso replaces the squared  $L_2$  penalty with an  $L_1$  penalty. Elastic net combines both.

Sparsity can simplify a deployed model, but it must not be confused with scientific discovery. A variable can receive a zero coefficient because its information is redundant, its scale was mishandled, the penalty is strong, or the sample happened to favour a correlated alternative.

## 16.2 The lasso objective

For centred  $y$  and standardised  $X$ , define

$$J_{\text{lasso}}(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1,$$

where

$$\|\beta\|_1 = \sum_{j=1}^p |\beta_j|.$$

This is the convention used by `sklearn.linear_model.Lasso`. The intercept is fitted separately and not penalised.

Lasso has no ordinary closed-form matrix solution because  $|\beta_j|$  is not differentiable at zero. That kink is precisely what permits exact zeros.

## 16.3 Subgradients at the kink

For a nonzero scalar  $b$ ,

$$\frac{d|b|}{db} = \text{sign}(b).$$

At zero, the subgradient is the interval

$$\partial|b|_{b=0} = [-1, 1].$$

The lasso optimum satisfies

$$-\frac{1}{n}X^\top(y - X\hat{\beta}) + \alpha z = 0,$$

where

$$z_j = \begin{cases} +1, & \hat{\beta}_j > 0, \\ -1, & \hat{\beta}_j < 0, \\ [-1, 1], & \hat{\beta}_j = 0. \end{cases}$$

A coefficient can remain zero whenever the feature's residual correlation lies inside the interval  $[-\alpha, \alpha]$ .

## 16.4 Deriving soft thresholding

The mechanism is easiest to see when the standardised columns are orthonormal in the convention

$$\frac{1}{n}X^\top X = I.$$

Let

$$z_j = \frac{1}{n}x_j^\top y.$$

Ignoring terms that do not depend on  $\beta_j$ , the objective for one coordinate becomes

$$\frac{1}{2}(\beta_j - z_j)^2 + \alpha|\beta_j|.$$

Consider three cases.

1. If  $z_j > \alpha$ , the positive solution is  $\hat{\beta}_j = z_j - \alpha$ .
2. If  $z_j < -\alpha$ , the negative solution is  $\hat{\beta}_j = z_j + \alpha$ .
3. If  $|z_j| \leq \alpha$ , the optimum is exactly zero.

Together,

$$\hat{\beta}_j = S(z_j, \alpha) = \text{sign}(z_j) \max(|z_j| - \alpha, 0)$$

This is the soft-thresholding operator. It both shrinks and selects.

```
import numpy as np

def soft_threshold(value, threshold):
 return np.sign(value) * max(abs(value) - threshold, 0.0)

for value in [-2.0, -0.4, 0.2, 1.6]:
 print(value, soft_threshold(value, threshold=0.5))
```

## 16.5 Geometry: why lasso reaches axes

The constrained form is

$$\min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_1 \leq t.$$

With two coefficients, the feasible region is a diamond. Its corners lie on the axes. An expanding elliptical loss contour often first touches a corner, making one coordinate zero.

Ridge uses a circular  $L_2$  boundary and typically shrinks both correlated features. Lasso uses a cornered  $L_1$  boundary and may retain one while dropping another.

## 16.6 Coordinate descent from scratch

For a general correlated design, one practical algorithm repeatedly updates one coefficient while holding the others fixed.

Let the partial residual excluding feature  $j$  be

$$r^{(j)} = y - \sum_{k \neq j} x_k \beta_k.$$

Holding every other coefficient fixed turns the full lasso objective into the one-variable problem

$$\frac{1}{2n} \|r^{(j)} - x_j \beta_j\|_2^2 + \alpha |\beta_j|.$$

For  $\beta_j \neq 0$ , its subgradient condition is

$$0 = -\frac{1}{n} x_j^T (r^{(j)} - x_j \beta_j) + \alpha \text{sign}(\beta_j).$$

Rearranging gives a positive or negative shrunken solution; when the unpenalised correlation lies inside  $[-\alpha, \alpha]$ , zero satisfies the subgradient condition. Combining those three cases with the soft-threshold operator  $S$  yields the update below.

The coordinate update under the `scikit-learn` objective is

$$\beta_j \leftarrow \frac{S\left(\frac{1}{n} x_j^T r^{(j)}, \alpha\right)}{\frac{1}{n} x_j^T x_j}.$$

```
import numpy as np

class LassoCoordinateDescent:
 def __init__(self, alpha=0.1, max_iter=10_000, tol=1e-8):
 if alpha < 0:
 raise ValueError("alpha must be non-negative")
 self.alpha = float(alpha)
 self.max_iter = int(max_iter)
 self.tol = float(tol)

 @staticmethod
 def _soft_threshold(value, threshold):
 return np.sign(value) * max(abs(value) - threshold, 0.0)

 def fit(self, X, y):
 X = np.asarray(X, dtype=float)
 y = np.asarray(y, dtype=float)
 n, p = X.shape

 self.x_mean_ = X.mean(axis=0)
 self.x_scale_ = X.std(axis=0, ddof=0)
 self.x_scale_[self.x_scale_ == 0] = 1.0
```

```

self.y_mean_ = y.mean()

Z = (X - self.x_mean_) / self.x_scale_
yc = y - self.y_mean_
beta = np.zeros(p)
column_energy = np.sum(Z * Z, axis=0) / n

for iteration in range(self.max_iter):
 old = beta.copy()
 for j in range(p):
 partial_residual = yc - Z @ beta + Z[:, j] * beta[j]
 correlation = Z[:, j] @ partial_residual / n
 beta[j] = self._soft_threshold(
 correlation, self.alpha
) / column_energy[j]

 if np.max(np.abs(beta - old)) < self.tol:
 break

self.n_iter_ = iteration + 1
self.coef_standardized_ = beta
self.coef_ = beta / self.x_scale_
self.intercept_ = self.y_mean_ - self.x_mean_ @ self.coef_
return self

def predict(self, X):
 X = np.asarray(X, dtype=float)
 return self.intercept_ + X @ self.coef_

```

Verify the implementation on a small problem:

```

from sklearn.linear_model import Lasso
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

manual = LassoCoordinateDescent(alpha=0.08).fit(X, y)
library = make_pipeline(
 StandardScaler(),
 Lasso(alpha=0.08, max_iter=100_000, tol=1e-10),
).fit(X, y)

assert np.allclose(manual.predict(X), library.predict(X), atol=1e-5)
print("iterations:", manual.n_iter_)
print("coefficients:", manual.coef_)

```

Production solvers also monitor an optimality condition such as the duality gap. A small change in coefficients is a useful teaching criterion but not the only possible convergence check.

## 16.7 Correlated predictors and unstable selection

Suppose `road_distance_observed_km` and `surveyed_route_km` are strongly correlated. Both explain access difficulty. Lasso may:

- select the first in one resample;
- select the second in another;
- select both under a weaker penalty; or
- select neither after another correlated access feature enters.

The prediction may remain stable while the selected set changes. Therefore:

A zero lasso coefficient means “not used by this penalised fit at this tuning value and representation.” It does not mean “has no relationship” or “is scientifically irrelevant.”

A simple selection-stability study repeats the entire fit on resampled training data and reports each feature's selection frequency. Those frequencies are descriptive unless a formal stability-selection procedure and its assumptions are specified.

```
from sklearn.base import clone
from sklearn.utils import resample
import numpy as np

def selection_frequency(pipeline, X, y, feature_names, repeats=200, seed=16):
 rng = np.random.default_rng(seed)
 selected = np.zeros(len(feature_names), dtype=float)

 for _ in range(repeats):
 index = rng.integers(0, len(X), size=len(X))
 fitted = clone(pipeline).fit(X.iloc[index], y.iloc[index])
 coefficients = fitted.named_steps["model"].coef_
 selected += np.abs(coefficients) > 1e-10

 return dict(zip(feature_names, selected / repeats))
```

The helper assumes the transformed feature names already match `feature_names`. In a real pipeline, obtain them from the fitted `ColumnTransformer` rather than guessing their order.

## 16.8 Elastic net

Elastic net combines  $L_1$  and squared  $L_2$  penalties:

$$J_{\text{EN}}(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \rho \|\beta\|_1 + \frac{\alpha(1 - \rho)}{2} \|\beta\|_2^2,$$

where `scikit-learn` calls  $\rho$  `l1_ratio`.

- $\rho = 1$  gives lasso.
- $\rho = 0$  gives a ridge-like objective, although `Ridge` is a more suitable solver for pure  $L_2$ .
- $0 < \rho < 1$  combines sparsity with ridge stabilisation.

The  $L_2$  component can encourage correlated features to enter or leave more smoothly as a group, while the  $L_1$  component can still produce zeros.

Two hyperparameters now require tuning. Every tried pair  $(\alpha, \rho)$  belongs to the model-selection process and must remain inside the validation boundary.

## 16.9 A Bayesian view of lasso

As in §15.11, the posterior combines the likelihood supplied by the observed outcomes with a prior distribution for the coefficients. The MAP estimate is the coefficient vector at the posterior's highest point; it is a point summary, not the entire posterior uncertainty distribution.

Under Gaussian outcome errors, place independent Laplace priors on standardised slopes:

$$p(\beta_j | b) = \frac{1}{2b} \exp\left(-\frac{|\beta_j|}{b}\right).$$

The negative log prior is proportional to  $\sum_j |\beta_j|$ , so the posterior mode has a lasso form after matching the likelihood and penalty normalisations. This parallels ridge's Gaussian-prior interpretation.

The posterior mode being exactly zero does not imply that the posterior probability of  $\beta_j = 0$  is positive under a continuous Laplace prior. Nor does the MAP alone convey posterior uncertainty. A sparse point estimate and a probabilistic claim about exact absence are different things.

## 16.10 Penalty paths and the one-standard-error rule

A validation curve often has a broad flat minimum. Choosing the exact lowest estimated error can favour a needlessly complex or weakly penalised model because validation scores are noisy.

The **one-standard-error rule** chooses the most regularised candidate whose estimated validation error is within one estimated standard error of the minimum. It is a preference for simplicity or stability under near-ties, not a theorem that the chosen model is optimal.

For temporal validation folds with different years, ordinary independent-fold standard-error calculations are only approximate. Report the fold scores and the rule you used; do not conceal the variability behind one average.

## 16.11 Post-selection inference warning

Suppose lasso chooses three variables from forty, and OLS is then refitted on those three using the same data. Ordinary OLS intervals act as if the three-variable model had been fixed before the data were seen. They ignore the search.

This can make intervals too narrow and  $p$ -values too optimistic. Safer choices include:

- treat the analysis as predictive and report held-out performance;
- separate exploratory selection from a new confirmatory sample;
- use a method designed for selective or debiased inference and state its assumptions; or
- pre-specify a low-dimensional explanatory model based on subject knowledge.

Regularisation solves a prediction and stability problem. It does not automatically solve inference after selection.

## 16.12 Reading guide and discussion prompts: Tibshirani (1996)

Robert Tibshirani's *Regression Shrinkage and Selection via the Lasso* proposed minimising residual sum of squares under a bound on the sum of absolute coefficients. The paper connects the stability benefits of ridge with subset-like sparsity.

Read it using the same research template.

1. **Question.** What disadvantages of subset selection and ridge motivated a new estimator?
2. **Method.** How does the  $L_1$  constraint differ geometrically from an  $L_2$  constraint?
3. **Evidence.** Which simulation settings and real datasets were used?
4. **Computation.** How was the constrained problem solved in 1996, before the modern software ecosystem?
5. **Boundary.** Does good prediction or a sparse fit validate causal or inferential claims about selected variables?



## Companion paper: least-angle regression

Efron, Hastie, Johnstone, and Tibshirani (2004) later introduced least-angle regression (LARS), an efficient algorithm that can produce the entire lasso path under suitable conditions. Read its geometric description after you can explain coordinate descent. The two algorithms illuminate different aspects of the same path.

### Replication task

Create ten features arranged as five highly correlated pairs. Give one member of each of two pairs a nonzero data-generating coefficient. Across repeated samples, compare lasso and elastic net on:

- validation MAE;
- number of nonzero coefficients;
- selection frequency for each feature; and
- prediction correlation between repeated fits.

The purpose is to observe that prediction stability and variable-selection stability are different quantities.

## 16.13 Day 16 build, break, and reflect

### Build

1. Fit a lasso path to standardised MHP features.
2. record the order in which features become active as the penalty weakens.
3. compare lasso and elastic net across forward validation years.
4. bootstrap the development set and report descriptive selection frequencies.

### Break

1. interpret every zero as evidence of no relationship.
2. report the best pair of hyperparameters after evaluating them on the final test period.
3. standardise once before splitting.
4. refit OLS on selected features and call ordinary intervals confirmatory.
5. compare  $\alpha$  values across differently normalised objectives without conversion.

### Reflect

If two access variables alternate across lasso resamples while predictions remain almost unchanged, which conclusion is supported: stable prediction, stable mechanism, both, or neither?

### Day 16 exit check

You are ready for influence diagnostics when you can derive soft thresholding, explain why the  $L_1$  geometry produces zeros, and state why lasso selection is unstable among correlated predictors.

---

# Day 17 — Multicollinearity, Leverage, and Influence

## 17.1 Four different questions

Diagnostics are useful only when the practitioner knows what each one asks.

| Diagnostic idea   | Question                                                              |
|-------------------|-----------------------------------------------------------------------|
| Multicollinearity | Are some predictor directions weakly distinguished by this design?    |
| Target outlier    | Is an observed outcome surprising under the fitted model?             |
| Leverage          | Is a row's predictor combination unusual relative to the design?      |
| Influence         | Would the fitted result change substantially if this row were absent? |

A row can have high leverage and a small residual, or a large residual and ordinary leverage. Influence usually requires some combination of discrepancy and leverage.

These are sample- and model-dependent descriptions. A project does not possess “high leverage” in isolation; it has leverage relative to a particular design matrix.

## 17.2 Multicollinearity revisited

Exact collinearity means one design column is an exact linear combination of others. Then  $\mathbf{X}$  is rank deficient and OLS coefficients are not unique.

Near collinearity means a combination is almost redundant. OLS may still exist, but coefficient estimates can vary greatly under small sample changes.

Consequences differ by goal:

- **Prediction inside a well-covered region:** fitted values may remain stable even when individual coefficients are not.
- **Interpretation:** conditional slopes can be very uncertain because the data contain little independent movement in one predictor while others are held fixed.
- **Extrapolation:** cancellation between large coefficients can fail dramatically outside the observed joint support.
- **Computation:** ill-conditioning magnifies numerical error.

Collinearity does not bias OLS under the classical model. It inflates variance and weakens the separation of effects.

## 17.3 Variance inflation factor

To diagnose predictor  $x_j$ , regress it on all the other predictors and calculate the auxiliary  $R_j^2$ . Define

$$\text{VIF}_j = \frac{1}{1 - R_j^2}.$$

Why this form? In a model with an intercept, the variance of the OLS slope can be expressed as

$$\text{Var}(\hat{\beta}_j | X) = \frac{\sigma^2}{\sum_i (x_{ij} - \bar{x}_j)^2 (1 - R_j^2)}.$$

Compared with an otherwise equivalent orthogonal design, the variance is inflated by VIF and the standard error by

$$\sqrt{\text{VIF}_j}.$$

For example, VIF 9 corresponds to a threefold standard-error inflation relative to that reference design. It does not mean the coefficient is “90% wrong.”

## No universal cutoff

Rules such as “VIF above 5 is bad” or “above 10 must be removed” are conventions, not laws. A high VIF may be expected when:

- a polynomial is represented by  $x$  and  $x^2$ ;
- an interaction is accompanied by its main effects;
- two measurements intentionally capture related mechanisms; or
- a control variable must remain for design reasons.

Respond to the consequence, not the number. Centre structural polynomial and interaction terms, collect more informative data, combine redundant measurements when scientifically justified, or use shrinkage for prediction. Never remove a necessary confounder merely to lower VIF.

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

columns = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 "surveyed_route_km",
 "terrain_index",
]
complete = df[columns].dropna()
design = sm.add_constant(complete, has_constant="add")

vif = pd.Series(
 [variance_inflation_factor(design.to_numpy(), j)
 for j in range(1, design.shape[1])],
 index=columns,
 name="VIF",
)
print(vif.sort_values(ascending=False))
```

VIF diagnoses one feature at a time. A condition index or singular-value analysis diagnoses weak directions involving several features. Both should be calculated on the actual transformed design, not only on the raw columns.

## 17.4 The hat matrix

For a full-rank OLS design that includes an intercept,

$$\hat{y} = X\hat{\beta} = X(X^\top X)^{-1}X^\top y = Hy,$$

where

$$H = X(X^\top X)^{-1}X^\top$$

is the **hat matrix** because it puts the “hat” on  $\mathbf{y}$ .

The diagonal element

$$h_{ii} = \mathbf{x}_i^\top (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{x}_i$$

is the leverage of row  $i$ . It measures how far that row's predictor vector lies in the geometry of the design.

For an ordinary full-rank design with  $k$  columns including the intercept:

$$0 \leq h_{ii} \leq 1, \quad \sum_i h_{ii} = \text{tr}(\mathbf{H}) = k,$$

so average leverage is  $k/n$ .

Values such as  $2k/n$  or  $3k/n$  are screening conventions. A balanced designed experiment may have nearly equal leverage. A rare but important project type may legitimately have high leverage. Context determines the response.

## 17.5 Residual variance and studentisation

Under the homoskedastic linear model,

$$\mathbf{e} = (\mathbf{I} - \mathbf{H})\mathbf{y} = (\mathbf{I} - \mathbf{H})\boldsymbol{\varepsilon}.$$

Because  $\mathbf{H}$  is symmetric and idempotent,

$$\text{Var}(\mathbf{e} \mid \mathbf{X}) = \sigma^2(\mathbf{I} - \mathbf{H}).$$

Therefore

$$\text{Var}(e_i \mid \mathbf{X}) = \sigma^2(1 - h_{ii}).$$

Raw residuals do not all have the same variance, even when the original errors do. An internally studentised residual is

$$r_i = \frac{e_i}{s\sqrt{1 - h_{ii}}}, \quad s^2 = \frac{\sum_i e_i^2}{n - k}.$$

An externally studentised residual uses an error-variance estimate calculated with row  $i$  deleted. It is especially useful for formal outlier procedures, which must account for multiple testing if many rows are screened.

## 17.6 Cook's distance

Influence concerns change in the fitted model, not merely a large residual. Cook's distance combines residual size and leverage:

$$D_i = \frac{e_i^2}{ks^2} \frac{h_{ii}}{(1 - h_{ii})^2}.$$

It is proportional to the aggregate change in fitted values when row  $i$  is deleted:

$$D_i = \frac{\sum_{m=1}^n (\hat{y}_m - \hat{y}_{m(i)})^2}{ks^2},$$

where  $\hat{y}_{m(i)}$  denotes the fit obtained without row  $i$ .

Rules such as  $D_i > 4/n$  are flags for investigation, not automatic deletion rules.

## 17.7 Calculate influence from definitions

```
import numpy as np
import pandas as pd

def ols_influence_table(X, y):
 """Educational OLS diagnostics for a full-rank design with intercept."""
 X = np.asarray(X, dtype=float)
 y = np.asarray(y, dtype=float)
 n, k = X.shape

 xtx_inverse = np.linalg.inv(X.T @ X)
 beta = xtx_inverse @ X.T @ y
 fitted = X @ beta
 residual = y - fitted
 # Chapter 2's einsum pattern: keep row i; sum repeated j and k.
 # This computes $x_i^T (X^T X)^{-1} x_i$ for every row without a Python loop.
 leverage = np.einsum("ij,jk,ik->i", X, xtx_inverse, X)

 mse = residual @ residual / (n - k)
 studentised = residual / np.sqrt(mse * (1.0 - leverage))
 cooks_d = (
 residual**2 / (k * mse)
 * leverage / (1.0 - leverage) ** 2
)

 return beta, pd.DataFrame({
 "fitted": fitted,
 "residual": residual,
 "leverage": leverage,
 "studentised_residual": studentised,
 "cooks_distance": cooks_d,
 })
```

This function uses an inverse to mirror the formula. A production implementation should use stable factorisations and a tested diagnostics library.

```
import statsmodels.api as sm

X_design = sm.add_constant(complete, has_constant="add")
y_complete = df.loc[complete.index, "actual_cost_2025_million_pkr"]
result = sm.OLS(y_complete, X_design).fit()
influence = result.get_influence()
summary = influence.summary_frame()

review = pd.concat(
 [df.loc[complete.index, ["project_id", "district", "start_year"]],
 summary[["hat_diag", "student_resid", "cooks_d"]]],
 axis=1,
).sort_values("cooks_d", ascending=False)
print(review.head(10))
```

## 17.8 The diagnostic quartet

At minimum, inspect these views on development data.

1. **Residuals versus fitted values:** look for curvature, changing spread, and clusters.
2. **Normal Q–Q plot:** assess whether residual tails strongly depart from the normal reference when normal-theory inference matters.
3. **Scale–location plot:** plot  $\sqrt{|r_i|}$  against fitted values to reveal changing variance.
4. **Residuals versus leverage with Cook contours:** locate potentially influential combinations.

```

import matplotlib.pyplot as plt
import statsmodels.api as sm

fig, axes = plt.subplots(2, 2, figsize=(11, 8))

axes[0, 0].scatter(result.fittedvalues, result.resid, alpha=0.65)
axes[0, 0].axhline(0, color="black", linewidth=1)
axes[0, 0].set(title="Residuals versus fitted",
 xlabel="fitted", ylabel="residual")

sm.qqplot(result.get_influence().resid_studentized_internal,
 line="45", ax=axes[0, 1])
axes[0, 1].set_title("Normal Q-Q")

studentised = result.get_influence().resid_studentized_internal
axes[1, 0].scatter(result.fittedvalues,
 np.sqrt(np.abs(studentised)), alpha=0.65)
axes[1, 0].set(title="Scale-location", xlabel="fitted",
 ylabel="sqrt(|studentised residual|)")

sm.graphics.influence_plot(result, criterion="cooks", ax=axes[1, 1])
plt.tight_layout()
plt.show()

```

A plot suggests a failure mode; it does not identify a remedy by itself. A funnel could arise from multiplicative noise, omitted groups, an incorrect mean transformation, or data errors.

## 17.9 An influence investigation protocol

When a row is flagged:

1. **Verify provenance.** Check source records, units, duplicates, and transcription.
2. **Check eligibility.** Confirm that the row belongs to the target population and prediction time.
3. **Understand the mechanism.** Ask why this combination is rare or its residual large.
4. **Compare specifications.** Examine whether a pre-justified transformation or group effect resolves a systematic omission.
5. **Run a sensitivity analysis.** Report how coefficients, validation scores, and substantive conclusions change with and without the row.
6. **Preserve the primary analysis.** If the observation is valid and in scope, do not silently remove it because it is inconvenient.

If deletion is justified by a documented data error, record the rule so that it can be applied without inspecting the desired result.

## 17.10 A crucial regularisation distinction

The ordinary hat matrix above applies directly to unpenalised OLS. Ridge has the smoother matrix  $H_\lambda$  from Day 15, whose diagonal values can be interpreted as regularised leverages and whose trace gives effective degrees of freedom. Lasso is nonlinear in  $y$  across changes of active set, so copying OLS influence formulas without qualification is unsafe.

Run classical influence diagnostics on the intended OLS model, and use resampling or case-deletion refits when assessing the stability of a complex selected pipeline.

## 17.11 Day 17 build, break, and reflect

**Build**

1. Calculate VIF on the transformed MHP design.
2. identify high-leverage and large-residual projects separately.
3. inspect the ten largest Cook distances and verify their records.
4. compare OLS conclusions with and without the most influential valid project, while retaining it in the primary analysis.

### Break

1. remove every feature whose VIF exceeds 5.
2. call a large residual high leverage.
3. delete every row above  $4/n$  without checking provenance.
4. calculate diagnostics using a different feature representation from the fitted model.
5. apply OLS Cook formulas mechanically to a tuned lasso pipeline.

### Reflect

Which is more concerning for deployment: a valid rare project with high leverage and a small residual, or a common project with a large residual? The answer depends on whether the rare region will occur in the deployment population.

## Day 17 exit check

You are ready for robust covariance when you can distinguish residual size, leverage, and influence; derive average leverage; and explain VIF as variance inflation rather than as a deletion command.

---

# Day 18 — Heteroskedasticity and Dependent Data

---

## 18.1 The question for today

OLS coefficients can remain useful when error variance changes across projects, but the usual standard errors can become wrong. Projects within the same district may also share unobserved conditions, making errors dependent.

How can the covariance calculation acknowledge those patterns without pretending that it repairs every model defect?

## 18.2 Point estimates and uncertainty are different layers

The OLS coefficient estimate is

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

If the conditional mean is correctly linear and

$$\mathbb{E}[\varepsilon \mid X] = 0,$$

OLS is unbiased conditional on  $X$  even when conditional error variance varies.

Under homoskedastic, uncorrelated errors,

$$\text{Var}(\varepsilon \mid X) = \sigma^2 I,$$

and

$$\text{Var}(\hat{\beta} | X) = \sigma^2 (X^\top X)^{-1}.$$

If error variances differ, this covariance formula is generally incorrect. The OLS coefficients do not change when a heteroskedasticity-consistent covariance estimator is requested; their estimated covariance, standard errors, tests, and intervals do.

## 18.3 The sandwich form

For a general error covariance matrix  $\Omega$ ,

$$\text{Var}(\hat{\beta} | X) = (X^\top X)^{-1} X^\top \Omega X (X^\top X)^{-1}.$$

The two outer matrices are the “bread.” The middle matrix is the “meat.” Together they form a sandwich estimator.

When observations are independent but have variances  $\sigma_i^2$ ,  $\Omega$  is diagonal. Because the true errors are unobserved, residuals estimate their scale.

## 18.4 HC0 through HC3

Let  $x_i^\top$  denote row  $i$  of  $X$  and  $e_i$  its OLS residual. HC0 uses

$$\widehat{\text{Var}}_{\text{HC0}}(\hat{\beta}) = (X^\top X)^{-1} \left( \sum_{i=1}^n e_i^2 x_i x_i^\top \right) (X^\top X)^{-1}.$$

The common variants modify the residual-square contribution:

| Estimator | Replace $e_i^2$ by           | Motivation                                                   |
|-----------|------------------------------|--------------------------------------------------------------|
| HC0       | $e_i^2$                      | basic large-sample sandwich                                  |
| HC1       | $\frac{n}{n-k} e_i^2$        | degrees-of-freedom scaling                                   |
| HC2       | $\frac{e_i^2}{1-h_{ii}}$     | compensate for leverage-related residual shrinkage           |
| HC3       | $\frac{e_i^2}{(1-h_{ii})^2}$ | stronger leverage correction, related to leave-one-out ideas |

HC3 is often a defensible default in modest samples, but no label guarantees accurate inference in every design. Very small samples, extreme leverage, weak identification, or misspecified dependence still require care.

## 18.5 Build HC0 and HC3 from scratch

```
import numpy as np

def ols_with_hc_covariance(X, y, kind="HC3"):
 """Return OLS coefficients and HC0/HC1/HC2/HC3 covariance."""
 X = np.asarray(X, dtype=float)
 y = np.asarray(y, dtype=float)
 n, k = X.shape

 bread = np.linalg.inv(X.T @ X)
 beta = bread @ X.T @ y
 residual = y - X @ beta
```



```

leverage = np.einsum("ij,jk,ik->i", X, bread, X)

if kind == "HC0":
 omega = residual**2
elif kind == "HC1":
 omega = residual**2 * n / (n - k)
elif kind == "HC2":
 omega = residual**2 / (1.0 - leverage)
elif kind == "HC3":
 omega = residual**2 / (1.0 - leverage) ** 2
else:
 raise ValueError("kind must be HC0, HC1, HC2, or HC3")

meat = X.T @ (omega[:, None] * X)
covariance = bread @ meat @ bread
return beta, covariance

```

Validate the result against a tested package before relying on educational code.

```

import numpy as np
import statsmodels.api as sm

X_demo = sm.add_constant(complete, has_constant="add").to_numpy()
y_demo = y_complete.to_numpy()

beta, covariance = ols_with_hc_covariance(X_demo, y_demo, kind="HC3")
reference = sm.OLS(y_demo, X_demo).fit(cov_type="HC3")

assert np.allclose(beta, reference.params)
assert np.allclose(covariance, reference.cov_params())
print(reference.summary())

```

## 18.6 What robust standard errors do not fix

Heteroskedasticity-consistent standard errors do **not** repair:

- an incorrect conditional mean, such as missing curvature;
- omitted-variable bias or endogeneity;
- target or feature leakage;
- errors-in-variables bias;
- lack of support for a causal contrast;
- dependence across rows if an independent-row sandwich is used; or
- poor out-of-sample prediction.

The phrase “robust standard error” means robust to a stated variance misspecification under other assumptions. It does not mean the research conclusion is universally robust.

## 18.7 Heteroskedasticity tests

The Breusch–Pagan test relates squared residuals to predictors. White's test permits a richer auxiliary variance model. Both can flag evidence against constant variance, but:

- a small sample may have low power;
- a large sample may detect operationally minor deviations;
- rejection does not reveal whether the mean is misspecified; and
- non-rejection does not prove homoskedasticity.

Use tests together with residual plots and design knowledge. If heteroskedasticity is plausible by construction—as when large remote projects naturally have wider cost variation—pre-specifying HC3 can be more coherent than using a preliminary test to decide whether robust inference is “allowed.”

## 18.8 Dependence within clusters

MHP projects in the same district may share geology, administrative practice, contractor markets, or measurement processes. If district-level shocks induce correlated errors, treating every project as independent overstates the effective information.

Let cluster  $g$  have design matrix  $X_g$  and residual vector  $e_g$ . A cluster-robust sandwich has meat

$$\sum_{g=1}^G (X_g^\top e_g)(X_g^\top e_g)^\top.$$

Thus

$$\widehat{\text{Var}}_{\text{cluster}}(\hat{\beta}) = (X^\top X)^{-1} \left[ \sum_{g=1}^G (X_g^\top e_g)(X_g^\top e_g)^\top \right] (X^\top X)^{-1},$$

usually with a finite-cluster correction.

Clustering permits arbitrary error covariance within a cluster while relying on approximate independence across clusters. The number of clusters, not merely the number of rows, drives the asymptotic justification.

## 18.9 Five districts are not a comfortable asymptotic regime

The fictional dataset has only five districts. Ordinary cluster-robust standard errors with five clusters can be severely unreliable. A software command can produce a number even when the reference distribution behind that number is weak.

Possible responses include:

- redesigning the study to include many more independent districts;
- modelling district effects under explicit hierarchical assumptions;
- using small-cluster methods such as an appropriate wild cluster bootstrap;
- conducting randomisation-based inference when the design permits it; or
- limiting the claim and reporting district-level sensitivity descriptively.

No method manufactures five independent district shocks into fifty. This is a design limitation, not a syntax problem.

```
import statsmodels.api as sm

clustered = sm.OLS(y_complete, X_design).fit(
 cov_type="cluster",
 cov_kwds={"groups": df.loc[complete.index, "district"]},
)
print(clustered.summary())
```

For this five-district teaching dataset, treat that output as a demonstration of the estimator, not as automatically reliable confirmatory inference.

## 18.10 How to choose a dependence unit

Cluster at a level that makes errors plausibly independent across clusters after conditioning on the model—not at whichever level produces desirable significance.

- If projects share district-level shocks, cluster by district.
- If the same project is observed repeatedly, cluster by project.
- If assignment occurred at village level, the assignment unit constrains the inferential design.
- If dependence runs across district and year, one-way district clustering may be insufficient.

Time series may require heteroskedasticity-and-autocorrelation-consistent covariance estimators. Spatial data may require spatial covariance models. These methods have their own bandwidth, distance, and asymptotic assumptions.

## 18.11 Prediction intervals under changing variance

HC covariance improves uncertainty for estimated mean coefficients. It does not automatically create a project-level prediction interval whose width changes correctly with remoteness.

A prediction distribution needs a model for conditional outcome variability. Options include:

- modelling residual scale as a function of appraisal features;
- fitting conditional quantiles, as in Day 19;
- conformal procedures under an explicit exchangeability or covariate-shift framework; or
- a probabilistic hierarchical model.

Do not add a single global residual standard deviation to every project if the conditional spread plainly changes and operational decisions depend on that spread.

## 18.12 Reading guide and discussion prompts: White (1980)

Halbert White's *A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity* developed an estimator of the OLS covariance that remains consistent under unknown heteroskedasticity, subject to regularity conditions.

Read with five questions.

1. **Target.** Which parameter and asymptotic distribution are being estimated?
2. **Failure.** Why is the familiar homoskedastic covariance estimator inconsistent under heteroskedasticity?
3. **Construction.** How do squared residuals estimate the unknown diagonal variance contributions?
4. **Scale.** Which results are asymptotic, and what does that imply for small or high-leverage samples?
5. **Boundary.** Which assumptions beyond homoskedasticity remain necessary?

### Replication task

Simulate

$$y = 1 + 2x + \varepsilon, \quad \varepsilon \mid x \sim \mathcal{N}(0, (0.5 + |x|)^2).$$

Across many samples, compare 95% interval coverage for the slope using classical and HC3 standard errors. Repeat under an omitted quadratic mean term. Observe that HC3 addresses the variance form but does not repair the omitted mean structure.

## 18.13 Day 18 build, break, and reflect

### Build

1. Derive HC0 from the general sandwich covariance.
2. compare classical and HC3 standard errors for the MHP OLS model.
3. plot residual scale across fitted cost and access mode.
4. state the plausible dependence unit before computing any clustered result.

### Break

1. claim that HC3 fixes omitted-variable bias.
2. select the covariance estimator that makes a coefficient significant.
3. cluster five districts and invoke large- $G$  confidence without qualification.
4. use project-level HC3 when errors share district shocks.
5. turn robust coefficient intervals into heteroskedastic prediction intervals without modelling outcome spread.

### Reflect

Write one sentence beginning “The coefficient estimate relies on...” and another beginning “The HC3 uncertainty statement additionally relies on...”. Separating these layers prevents the word *robust* from doing too much rhetorical work.

### Day 18 exit check

You are ready for robust and quantile regression when you can construct the sandwich estimator, explain HC3's leverage adjustment, and state why a small number of clusters remains a design problem.

---

## Day 19 — Robust and Quantile Regression

---

### 19.1 Three uses of the word “robust”

These methods answer different questions.

| Method                  | Coefficient fit                          | Uncertainty calculation       | Main purpose                                                                        |
|-------------------------|------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------|
| OLS with HC3 covariance | ordinary squared loss                    | heteroskedasticity-consistent | protect large-sample coefficient inference against unknown independent-row variance |
| Huber regression        | quadratic near zero, linear in the tails | estimator-specific            | reduce the effect of large residuals on the fitted conditional location             |
| Quantile regression     | asymmetric pinball loss                  | quantile-specific             | estimate a chosen conditional quantile rather than the conditional mean             |

One cannot substitute for another merely because all are sometimes called robust.

### 19.2 Why squared loss reacts strongly to large residuals

OLS minimises

$$\sum_i r_i^2, \quad r_i = y_i - x_i^\top \beta.$$

The derivative of  $\frac{1}{2}r^2$  with respect to  $r$  is  $r$ . A residual of magnitude 20 therefore has twenty times the local influence on the loss gradient of a residual of magnitude 1.

Large residuals can arise from:

- data errors;
- omitted mechanisms;
- a heavy-tailed outcome distribution;
- a mixture of project regimes;
- a legitimate rare logistics shock; or
- an observation outside the intended population.

Changing the loss can reduce sensitivity, but it does not identify which explanation is correct.

## 19.3 Huber loss

Huber loss is quadratic for small residuals and linear for large residuals:

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta), & |r| > \delta. \end{cases}$$

It is continuous and differentiable at  $\pm\delta$ . Its derivative, often called the score or  $\psi$  function, is

$$\psi_\delta(r) = \begin{cases} r, & |r| \leq \delta, \\ \delta \text{sign}(r), & |r| > \delta. \end{cases}$$

Large-residual gradients are clipped at magnitude  $\delta$ . Huber regression is therefore less sensitive to vertical outliers than OLS.

```
import numpy as np

def huber_loss(residual, delta=1.35):
 residual = np.asarray(residual, dtype=float)
 absolute = np.abs(residual)
 return np.where(
 absolute <= delta,
 0.5 * residual**2,
 delta * (absolute - 0.5 * delta),
)

grid = np.array([-4.0, -1.0, 0.0, 1.0, 4.0])
print(huber_loss(grid, delta=1.5))
```

## 19.4 Iteratively reweighted least squares intuition

For  $r \neq 0$ , define a weight

$$w(r) = \frac{\psi_\delta(r)}{r} = \begin{cases} 1, & |r| \leq \delta, \\ \delta/|r|, & |r| > \delta. \end{cases}$$

At a current coefficient estimate:

1. calculate residuals;
2. assign smaller weights to residuals beyond the threshold;
3. solve a weighted least-squares problem; and
4. repeat until the objective and coefficients stabilise.

This is the intuition behind iteratively reweighted least squares for Huber-type M-estimation. Production implementations also estimate or account for residual scale, because a threshold of 1.35 has no unit-free meaning if one outcome is measured in rupees and another in millions of rupees.

## 19.5 Robust to vertical residuals is not robust to every leverage point

A high-leverage observation can pull the fitted surface toward itself and thereby acquire a modest residual. Residual clipping alone may not protect against unusual predictor configurations.

Always combine robust regression with:

- predictor-space inspection;
- leverage or case-deletion analysis;
- provenance checks; and
- explicit definition of the deployment population.

Robust fitting is not permission to ignore influential design points.

## 19.6 Huber regression in a pipeline

`sklearn`'s `HuberRegressor` jointly handles a scale parameter, uses `epsilon` to define which scaled residuals enter the linear-loss region, and can apply an  $L_2$  penalty controlled by `alpha`.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import HuberRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_columns = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 "surveyed_route_km",
 "terrain_index",
 "contractor_experience_projects",
]
categorical_columns = ["district", "access_mode"]

numeric_pipe = Pipeline([
 ("impute", SimpleImputer(strategy="median", add_indicator=True)),
 ("scale", StandardScaler()),
])
categorical_pipe = Pipeline([
 ("impute", SimpleImputer(strategy="most_frequent")),
 ("one_hot", OneHotEncoder(handle_unknown="ignore", drop="first")),
])

preprocess = ColumnTransformer([
 ("numeric", numeric_pipe, numeric_columns),
 ("categorical", categorical_pipe, categorical_columns),
])

huber_pipeline = Pipeline([
 ("preprocess", preprocess),
```

```
("model", HuberRegressor(epsilon=1.35, alpha=0.01, max_iter=1_000)),
1)
```

Tune `epsilon` and `alpha` inside validation. A smaller `epsilon` treats more observations as outlying; it is not automatically more robust in a useful decision sense. Excessive clipping can discard important structure.

## 19.7 From the mean to conditional quantiles

OLS targets the conditional mean under squared loss:

$$m(x) = \mathbb{E}[Y \mid X = x].$$

The conditional  $\tau$ -quantile is

$$Q_Y(\tau \mid x) = \inf\{q : \Pr(Y \leq q \mid X = x) \geq \tau\}, \quad 0 < \tau < 1.$$

Examples:

- $\tau = 0.50$  targets the conditional median;
- $\tau = 0.80$  targets a value at or above roughly 80% of outcomes with comparable features, under a calibrated model; and
- $\tau = 0.95$  targets a more conservative upper conditional quantile.

The quantile is a property of a conditional distribution, not a confidence bound for a mean coefficient.

## 19.8 Deriving pinball loss

Define residual  $u = y - q$  for a proposed quantile prediction  $q$ . Pinball loss is

$$\rho_\tau(u) = u(\tau - \mathbb{1}\{u < 0\}) = \begin{cases} \tau u, & u \geq 0, \\ (\tau - 1)u, & u < 0. \end{cases}$$

For  $\tau = 0.5$ , over- and underprediction receive equal absolute penalties up to a factor of one half. Minimising expected pinball loss gives a median.

For  $\tau = 0.8$ :

- underprediction has slope **0.8**;
- overprediction has slope **0.2**.

Underprediction is therefore penalised four times as strongly per unit. The asymmetric loss moves the optimum upward.

### Why its minimiser is a quantile

Let  $R(q) = \mathbb{E}[\rho_\tau(Y - q)]$ . Away from point masses, differentiating with respect to  $q$  gives

$$R'(q) = F_Y(q) - \tau.$$

At an optimum,  $F_Y(q) = \tau$ . With discontinuities, the subgradient condition gives the usual set-valued definition of a quantile.

```
import numpy as np

def pinball_loss(y_true, y_pred, quantile):
 error = np.asarray(y_true) - np.asarray(y_pred)
```

```

return np.maximum(quantile * error, (quantile - 1.0) * error)

y_small = np.array([2.0, 3.0, 10.0])
for proposal in [3.0, 6.0, 9.0]:
 print(proposal, pinball_loss(y_small, proposal, 0.8).mean())

```

## 19.9 Linear quantile regression

Linear quantile regression assumes that a chosen conditional quantile has the form

$$Q_Y(\tau \mid X = x) = x^\top \beta_\tau.$$

It solves

$$\hat{\beta}_\tau = \arg \min_{\beta} \sum_{i=1}^n \rho_\tau(y_i - x_i^\top \beta),$$

possibly with a regularisation term.

The coefficient  $\beta_{\tau,j}$  describes a conditional change in the  $\tau$ -quantile under the specified linear model. It need not equal the corresponding mean-regression coefficient, and coefficients can vary across quantiles when the conditional distribution changes shape or scale.

```

from sklearn.linear_model import QuantileRegressor

median_pipeline = Pipeline([
 ("preprocess", preprocess),
 ("model", QuantileRegressor(quantile=0.50, alpha=0.01,
 solver="highs")),
])

upper_pipeline = Pipeline([
 ("preprocess", preprocess),
 ("model", QuantileRegressor(quantile=0.80, alpha=0.01,
 solver="highs")),
])

```

In this implementation, `alpha` controls  $L_1$  regularisation. Standardisation and validation remain necessary.

## 19.10 Evaluating a quantile model

Use the loss that corresponds to the stated target.

```

from sklearn.metrics import mean_pinball_loss

upper_pipeline.fit(X_train, y_train)
q80 = upper_pipeline.predict(X_valid)

loss = mean_pinball_loss(y_valid, q80, alpha=0.80)
coverage = (y_valid <= q80).mean()
print({"pinball_loss": loss, "empirical_coverage": coverage})

```

On held-out observations, the overall coverage

$$\frac{1}{m} \sum_{i=1}^m \mathbb{1}\{y_i \leq \hat{q}_\tau(x_i)\}$$



should be near  $\tau$  for a well-calibrated conditional-quantile model in a stable population. But global coverage can hide subgroup failures. A model could over-cover road-access projects and under-cover remote projects while averaging to 80%.

Report:

- mean pinball loss at the target  $\tau$ ;
- overall empirical coverage;
- coverage by important pre-specified subgroup;
- coverage across predicted-quantile bins; and
- sample sizes and uncertainty around these estimates.

Coverage is not the only objective. A trivially enormous upper prediction could cover almost everything while being operationally useless. Pinball loss rewards both calibration and sharpness in a target-aligned way.

## 19.11 Quantile crossing

If separate models are fitted for  $\tau = 0.5$  and  $\tau = 0.8$ , they can produce

$$\hat{Q}(0.5 | x) > \hat{Q}(0.8 | x)$$

for some  $x$ . This violates quantile ordering and is called **quantile crossing**.

Check crossing on validation and deployment-like ranges. Remedies include:

- jointly fitting quantiles with non-crossing constraints;
- using a model for the full conditional distribution;
- rearrangement or other principled post-processing; and
- simplifying an unstable feature representation.

Replacing the two predictions by their sorted values after seeing them may be a useful descriptive patch, but it changes the model and must be validated as part of the procedure.

## 19.12 A decision example: cost planning

Suppose a planner must reserve a contingency budget. The consequences of underbudgeting and overbudgeting differ.

If the per-unit cost of underprediction is  $c_u$  and of overprediction is  $c_o$ , minimising expected asymmetric absolute cost leads to the quantile

$$\tau = \frac{c_u}{c_u + c_o}.$$

If underbudgeting costs four times as much as overbudgeting, then  $\tau = 4/(4 + 1) = 0.8$ .

This provides a decision-based reason for an 80th percentile. Choosing 0.8 because it looked favourable after examining outcomes is not the same justification.

## 19.13 Reading guide and discussion prompts: Huber (1964)

Peter Huber's *Robust Estimation of a Location Parameter* formalised an estimator that balances performance near a reference distribution with protection against contamination.

Ask:

1. What contamination neighbourhood motivates the minimax problem?
2. Why does the optimal score behave linearly near zero and clip in the tails?
3. What is the trade-off between normal-model efficiency and contamination resistance?
4. How does a one-dimensional location problem differ from regression with leverage points?
5. Which tuning choice is theoretical, and which choices must a practitioner validate?

## Replication task

Estimate the centre of samples from a standard normal distribution, then replace 5% of observations with values from a much wider distribution. Across repeats, compare the sample mean, median, and Huber estimate on bias, variance, and mean squared error.

## 19.14 Reading guide and discussion prompts: Koenker and Bassett (1978)

Roger Koenker and Gilbert Bassett's *Regression Quantiles* extended the sample-quantile idea to linear regression, allowing conditional effects to be studied at different points of the outcome distribution.

Ask:

1. How does minimising an asymmetric sum of absolute residuals generalise the sample quantile?
2. Why can regression-quantile coefficients reveal changes in spread or shape hidden by a mean model?
3. Which computational representation connects the estimator to linear programming?
4. What sampling assumptions support the paper's asymptotic results?
5. Why is a fitted 80th conditional quantile not automatically an 80% finite-sample prediction guarantee?

## Replication task

Simulate

$$Y = 2 + 3X + (0.5 + X)\epsilon, \quad X \sim \text{Uniform}(0, 2),$$

with standard normal  $\epsilon$ . Fit quantile regressions at  $\tau = 0.1, 0.5, 0.9$ . Compare estimated slopes with the theoretical slopes implied by the data-generating process, then check held-out coverage.

## 19.15 Day 19 build, break, and reflect

### Build

1. Fit OLS, Huber, median, and 80th-quantile pipelines on the same development period.
2. evaluate each with the loss corresponding to its target.
3. inspect which projects Huber downweights or flags as outlying.
4. calculate held-out 80% coverage overall and by access mode.

### Break

1. call HC3, Huber, and quantile regression interchangeable.
2. choose the upper quantile after seeing which one covers the test data.
3. interpret a quantile curve as a confidence interval for the mean.
4. declare 80% calibration from global coverage while one access group has 55%.
5. remove genuine rare projects merely because Huber labels their residuals large.

### Reflect

Which target matches each question: expected cost, typical cost, or a contingency budget exceeded only about 20% of the time? Model choice begins with that distinction.

Day 19 exit check

You are ready for the benchmark when you can derive Huber and pinball loss, explain the target of a conditional quantile, and evaluate upper-quantile calibration on held-out subgroups.

# Day 20 — A Pre-Specified Regression Benchmark

## 20.1 The question for today

The chapter has introduced many reasonable choices. That flexibility creates a research risk: if the analyst keeps trying representations, penalties, subgroups, years, metrics, and exclusions until an attractive result appears, ordinary validation language can hide extensive adaptation.

How can a flexible modelling study remain auditable?

The answer is to define the prediction contract, candidate procedures, tuning boundary, primary metric, test period, and reporting rules before opening the locked result.

## 20.2 Exploratory and confirmatory work

Exploration is valuable. It generates hypotheses, reveals data problems, and helps design later studies. The problem is not exploration; the problem is presenting an exploration-dependent result as if it had been specified in advance.

Use explicit labels.

- **Exploratory:** choices may adapt to observed results; findings generate hypotheses.
- **Confirmatory:** the primary question and analysis are fixed in advance; deviations are recorded.
- **Predictive benchmark:** candidate procedures and selection rule are fixed; a locked test estimates performance of the selected procedure.

A benchmark can contain model selection and remain honest because the selection occurs entirely inside the development period. The final test evaluates the whole selection procedure's result.

## 20.3 The registered prediction contract

For the MHP capstone, freeze the following statement before fitting candidates.

| Component          | Pre-specified choice                    |
|--------------------|-----------------------------------------|
| Unit               | one eligible MHP project                |
| Prediction time    | technical appraisal, before procurement |
| Target             | final cost in constant 2025 million PKR |
| Development period | projects starting through 2022          |

| Component               | Pre-specified choice                                    |
|-------------------------|---------------------------------------------------------|
| Locked test             | projects starting in 2023–2025                          |
| Validation              | expanding windows with validation years 2019–2022       |
| Primary point decision  | one appraisal-stage cost estimate                       |
| Primary point metric    | MAE, averaged equally across validation years           |
| Secondary point metrics | RMSE, signed error, fold-to-fold spread                 |
| Planning target         | conditional 80th percentile                             |
| Planning metric         | 0.80 pinball loss; empirical coverage is diagnostic     |
| Subgroups               | access mode and district, reported with sample size     |
| Exclusions              | only documented eligibility failures or source errors   |
| Test use                | one evaluation after model and hyperparameter selection |

Equal weighting across years asks for typical year-level performance. Weighting every project equally would answer a different question and let high-volume years dominate. Either can be reasonable; pre-specify it.

## 20.4 Candidate procedures, not isolated estimators

A candidate includes every operation learned from data:

1. missing-data imputation;
2. missingness indicators;
3. categorical encoding;
4. centring and feature construction;
5. standardisation;
6. estimator and penalty; and
7. hyperparameter values.

The benchmark below pre-specifies:

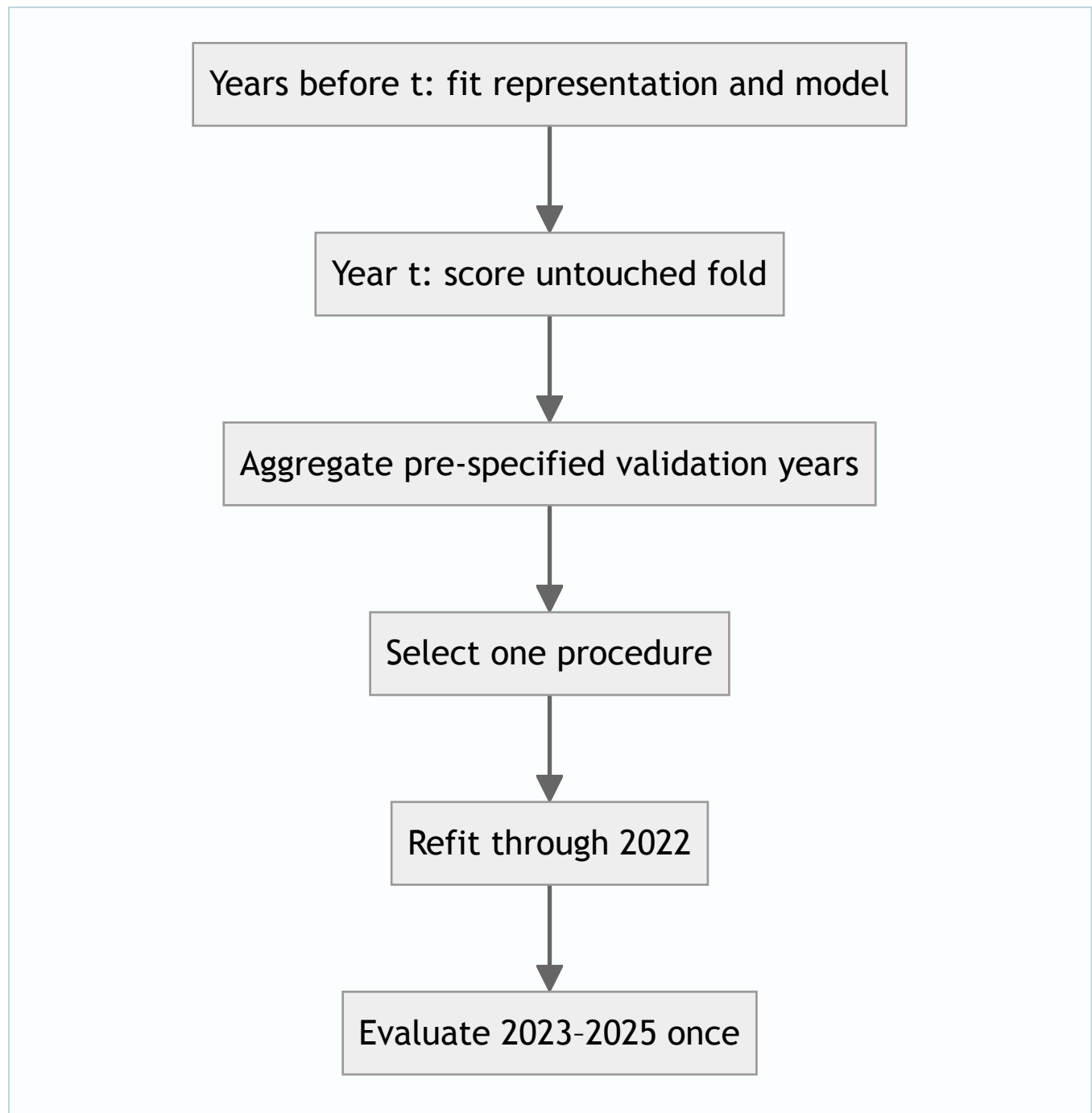
- historical training mean;
- raw-feature OLS;
- engineered OLS;
- engineered ridge at five penalty values;
- engineered elastic net at six  $(\alpha, \rho)$  pairs;
- engineered Huber regression at four settings; and
- regularised median and 80th-quantile models at three penalty values each.

This is a modest, theory-guided search space. A larger search is not forbidden, but its computational budget and selection rule should also be fixed.

MAE makes this an operational **point-prediction** comparison. Its population-optimal unrestricted prediction is a conditional median, whereas squared loss targets a conditional mean and Huber loss targets a robust conditional location. The candidates therefore need not estimate identical scientific parameters even though their point forecasts can be compared under one decision loss. If the estimand itself is expected cost—for example, for unbiased portfolio totals—pre-specify squared loss or another expectation-aligned criterion and do not silently reinterpret an MAE-selected robust location as a conditional mean.

## 20.5 The temporal boundary

For validation year  $t$ , fit every learned step using only years earlier than  $t$ .



The locked test remains unavailable during feature choice, tuning, influence decisions, and metric selection.

## 20.6 End-to-end capstone code

The code is deliberately complete. It assumes the earlier generators were saved as `chapter2_data.py` and `chapter3_data.py`.

```
from pathlib import Path
import json
import platform
```

```

import numpy as np
import pandas as pd
import scipy
import sklearn
from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import (
 ElasticNet,
 HuberRegressor,
 LinearRegression,
 QuantileRegressor,
 Ridge,
)
from sklearn.metrics import (
 mean_absolute_error,
 mean_pinball_loss,
 mean_squared_error,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from chapter3_data import make_mhp_practitioner_data

TARGET = "actual_cost_2025_million_pkr"
VALIDATION_YEARS = [2019, 2020, 2021, 2022]
TEST_START_YEAR = 2023
RANDOM_SEED = 2020

class MHPFeatureEngineer(BaseEstimator, TransformerMixin):
 """Create appraisal-time features using training-fold centres."""

 def fit(self, X, y=None):
 X = X.copy()
 self.capacity_center_ = float(X["planned_capacity_kw"].median())
 self.year_center_ = float(X["start_year"].median())
 return self

 def transform(self, X):
 X = X.copy()
 capacity_c = X["planned_capacity_kw"] - self.capacity_center_
 year_c = X["start_year"] - self.year_center_

 X["capacity_centered"] = capacity_c
 X["capacity_centered_squared"] = capacity_c**2
 X["year_centered"] = year_c
 X["road_hinge_above_18_km"] = (
 X["road_distance_observed_km"] - 18.0
).clip(lower=0.0)
 X["cable_by_road"] = (
 X["estimated_cable_km"]
 * X["road_distance_observed_km"]
)
 X["log_contractor_experience"] = np.log1p(
 X["contractor_experience_projects"]
)
 return X

RAW_NUMERIC = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 "surveyed_route_km",
 "terrain_index",
 "contractor_experience_projects",
 "start_year",

```

```

]
ENGINEERED_NUMERIC = [
 "planned_capacity_kw",
 "capacity_centered",
 "capacity_centered_squared",
 "estimated_cable_km",
 "road_distance_observed_km",
 "road_hinge_above_18_km",
 "cable_by_road",
 "surveyed_route_km",
 "terrain_index",
 "log_contractor_experience",
 "year_centered",
]
CATEGORICAL = ["district", "access_mode"]

def make_preprocessor(numeric_columns):
 numeric = Pipeline([
 ("impute", SimpleImputer(strategy="median", add_indicator=True)),
 ("scale", StandardScaler()),
])
 categorical = Pipeline([
 ("impute", SimpleImputer(strategy="most_frequent")),
 ("one_hot", OneHotEncoder(
 handle_unknown="ignore",
 drop="first",
 sparse_output=False,
)),
])
 return ColumnTransformer([
 ("numeric", numeric, numeric_columns),
 ("categorical", categorical, CATEGORICAL),
])

def raw_pipeline(model):
 return Pipeline([
 ("preprocess", make_preprocessor(RAW_NUMERIC)),
 ("model", model),
])

def engineered_pipeline(model):
 return Pipeline([
 ("features", MHPFeatureEngineer()),
 ("preprocess", make_preprocessor(ENGINEERED_NUMERIC)),
 ("model", model),
])

def make_point_candidates():
 candidates = {
 "historical_mean": Pipeline([
 ("model", DummyRegressor(strategy="mean")),
]),
 "ols_raw": raw_pipeline(LinearRegression()),
 "ols_engineered": engineered_pipeline(LinearRegression()),
 }

 for alpha in [0.1, 1.0, 10.0, 100.0, 1_000.0]:
 candidates[f"ridge_alpha_{alpha:g}"] = engineered_pipeline(
 Ridge(alpha=alpha)
)

 for alpha in [0.01, 0.05, 0.20]:
 for ll_ratio in [0.25, 0.75]:
 name = f"elastic_alpha_{alpha:g}_ll_{ll_ratio:g}"
 candidates[name] = engineered_pipeline(
 ElasticNet(

```

```

 alpha=alpha,
 ll_ratio=ll_ratio,
 max_iter=50_000,
 tol=1e-7,
 random_state=RANDOM_SEED,
)
)

for epsilon, alpha in [
 (1.20, 0.0001),
 (1.35, 0.0001),
 (1.35, 0.01),
 (1.70, 0.0001),
]:
 name = f"huber_eps_{epsilon:g}_alpha_{alpha:g}"
 candidates[name] = engineered_pipeline(
 HuberRegressor(
 epsilon=epsilon,
 alpha=alpha,
 max_iter=2_000,
)
)
return candidates

def make_quantile_candidates(quantile):
 return {
 f"q{quantile:g}_alpha_{alpha:g}": engineered_pipeline(
 QuantileRegressor(
 quantile=quantile,
 alpha=alpha,
 solver="highs",
)
)
 for alpha in [0.0, 0.01, 0.05]
 }

def expanding_window_scores(data, candidates, years, score_function):
 """Fit every operation afresh inside each temporal training fold."""
 rows = []
 feature_frame = data.drop(columns=[TARGET])
 target = data[TARGET]

 for year in years:
 train_mask = data["start_year"] < year
 validation_mask = data["start_year"] == year
 if not validation_mask.any():
 raise ValueError(f"No validation rows for {year}")

 X_train = feature_frame.loc[train_mask]
 y_train = target.loc[train_mask]
 X_validation = feature_frame.loc[validation_mask]
 y_validation = target.loc[validation_mask]

 for name, procedure in candidates.items():
 fitted = clone(procedure).fit(X_train, y_train)
 prediction = fitted.predict(X_validation)
 rows.append({
 "candidate": name,
 "validation_year": year,
 "n_validation": int(validation_mask.sum()),
 "score": float(score_function(y_validation, prediction)),
 })

 return pd.DataFrame(rows)

def select_by_equal_year_average(scores):
 summary = (

```



```

 scores.groupby("candidate")["score"]
 .agg(mean="mean", sd="std", worst="max")
 .sort_values(["mean", "worst"])
)
 return summary.index[0], summary

def regression_metrics(y_true, prediction):
 error = np.asarray(prediction) - np.asarray(y_true)
 return {
 "mae": float(mean_absolute_error(y_true, prediction)),
 "rmse": float(mean_squared_error(y_true, prediction) ** 0.5),
 "signed_error": float(error.mean()),
 }

def paired_mae_bootstrap(y_true, prediction_a, prediction_b,
 repeats=5_000, seed=RANDOM_SEED):
 """Descriptive row bootstrap for MAE(A)-MAE(B)."""
 y_true = np.asarray(y_true)
 prediction_a = np.asarray(prediction_a)
 prediction_b = np.asarray(prediction_b)
 rng = np.random.default_rng(seed)
 n = y_true.size
 differences = np.empty(repeats)

 for repeat in range(repeats):
 index = rng.integers(0, n, size=n)
 mae_a = np.abs(y_true[index] - prediction_a[index]).mean()
 mae_b = np.abs(y_true[index] - prediction_b[index]).mean()
 differences[repeat] = mae_a - mae_b

 return {
 "estimate": float(
 np.abs(y_true - prediction_a).mean()
 - np.abs(y_true - prediction_b).mean()
),
 "interval_95": np.quantile(differences, [0.025, 0.975]).tolist(),
 }

def main():
 data = make_mhp_practitioner_data(n=520, seed=3030)
 development = data[data["start_year"] < TEST_START_YEAR].copy()
 locked_test = data[data["start_year"] >= TEST_START_YEAR].copy()

 assert development["start_year"].max() < locked_test["start_year"].min()
 assert "final_material_bill_million_pkr" not in RAW_NUMERIC
 assert "road_distance_km" not in RAW_NUMERIC

 # Track 1: select one operational point-prediction procedure by MAE.
 point_candidates = make_point_candidates()
 point_scores = expanding_window_scores(
 development,
 point_candidates,
 VALIDATION_YEARS,
 mean_absolute_error,
)
 best_point_name, point_summary = select_by_equal_year_average(point_scores)

 X_development = development.drop(columns=[TARGET])
 y_development = development[TARGET]
 X_test = locked_test.drop(columns=[TARGET])
 y_test = locked_test[TARGET]

 best_point = clone(point_candidates[best_point_name]).fit(
 X_development, y_development
)
 test_point_prediction = best_point.predict(X_test)
 test_point_metrics = regression_metrics(y_test, test_point_prediction)

```

```

A pre-specified reference supports a paired, descriptive comparison.
reference = clone(point_candidates["ols_raw"]).fit(
 X_development, y_development
)
reference_prediction = reference.predict(X_test)
paired_difference = paired_mae_bootstrap(
 y_test, test_point_prediction, reference_prediction
)

Track 2: select median and upper-quantile procedures separately.
quantile_results = {}
quantile_predictions = {}
for quantile in [0.50, 0.80]:
 candidates = make_quantile_candidates(quantile)
 scores = expanding_window_scores(
 development,
 candidates,
 VALIDATION_YEARS,
 lambda observed, predicted, q=quantile: mean_pinball_loss(
 observed, predicted, alpha=q
),
)
 best_name, summary = select_by_equal_year_average(scores)
 fitted = clone(candidates[best_name]).fit(X_development, y_development)
 prediction = fitted.predict(X_test)
 quantile_predictions[quantile] = prediction
 quantile_results[str(quantile)] = {
 "selected": best_name,
 "test_pinball_loss": float(mean_pinball_loss(
 y_test, prediction, alpha=quantile
)),
 "test_coverage": float((y_test <= prediction).mean()),
 "validation_table": summary.reset_index().to_dict("records"),
 }

test_rows = locked_test[[
 "project_id", "district", "access_mode", "start_year", TARGET
]].copy()
test_rows["point_prediction"] = test_point_prediction
test_rows["q50_prediction"] = quantile_predictions[0.50]
test_rows["q80_prediction"] = quantile_predictions[0.80]
test_rows["error"] = test_rows["point_prediction"] - test_rows[TARGET]
test_rows["absolute_error"] = test_rows["error"].abs()
test_rows["q80_covered"] = (
 test_rows[TARGET] <= test_rows["q80_prediction"]
)
test_rows["quantile_crossed"] = (
 test_rows["q50_prediction"] > test_rows["q80_prediction"]
)

subgroup = (
 test_rows.groupby("access_mode", observed=True)
 .agg(
 n=(TARGET, "size"),
 mae=("absolute_error", "mean"),
 signed_error=("error", "mean"),
 q80_coverage=("q80_covered", "mean"),
 crossing_rate=("quantile_crossed", "mean"),
)
 .reset_index()
)

metadata = {
 "seed": RANDOM_SEED,
 "python": platform.python_version(),
 "numpy": np.__version__,
 "pandas": pd.__version__,
 "scipy": scipy.__version__,
 "scikit_learn": sklearn.__version__,
}

```

```

 "development_rows": len(development),
 "test_rows": len(locked_test),
 "selected_point_model": best_point_name,
 "test_point_metrics": test_point_metrics,
 "selected_minus_raw_ols_mae": paired_difference,
 "quantiles": quantile_results,
}

output = Path("chapter3_results")
output.mkdir(exist_ok=True)
point_scores.to_csv(output / "point_validation_folds.csv", index=False)
point_summary.to_csv(output / "point_validation_summary.csv")
test_rows.to_csv(output / "locked_test_predictions.csv", index=False)
subgroup.to_csv(output / "locked_test_subgroups.csv", index=False)
(output / "run_metadata.json").write_text(
 json.dumps(metadata, indent=2), encoding="utf-8"
)

print("Selected point procedure:", best_point_name)
print("Locked-test point metrics:", test_point_metrics)
print("Selected minus raw-OLS MAE:", paired_difference)
print("\nLocked-test subgroup audit:\n", subgroup.to_string(index=False))
print("\nQuantile results:", {
 q: {k: v for k, v in result.items() if k != "validation_table"}
 for q, result in quantile_results.items()
})

if __name__ == "__main__":
 main()

```

## Why the assert statements matter

`road_distance_km` is the complete latent measurement used to generate the teaching data. It is not available when its observed version is missing. `final_material_bill_million_pkr` is known after the prediction time. Both would leak forbidden information. The assertions make that contract executable.

Assertions do not replace a data audit. They protect only the conditions they state.

## 20.7 Read the validation table before the winner's name

The mean validation summary contains:

- average MAE across years;
- standard deviation across four year scores; and
- worst-year MAE.

Ask:

1. Is the apparent winner consistently better or driven by one year?
2. Is the difference operationally meaningful?
3. Did a simple raw OLS model nearly tie a complex candidate?
4. Do any candidates fail to converge?
5. Would the pre-specified tie rule choose a more stable model?

If two candidates are practically tied, report the tie. Ranking estimates more precisely than the validation evidence permits creates false certainty.

## 20.8 Interpreting the locked test

The test result estimates performance of one selected procedure in a later time period. It does not prove permanent superiority.

Report at least:

- development and test date ranges;
- test sample size;
- the selected procedure and all candidate families;
- test MAE, RMSE, and signed error;
- the raw OLS reference difference;
- median and 80th-quantile pinball loss;
- 80th-quantile overall and subgroup coverage;
- quantile crossing rate;
- notable distribution shift; and
- every deviation from the registered plan.

Do not reopen tuning after seeing test errors. If a new idea is inspired by the test, label the old test as development data and acquire a genuinely new future test period.

## 20.9 Uncertainty for performance differences

The paired bootstrap resamples rows and recalculates the difference

$$\Delta = \text{MAE}_{\text{selected}} - \text{MAE}_{\text{raw OLS}}.$$

A negative value favours the selected procedure. Pairing matters because both models predict the same projects.

The supplied interval is explicitly descriptive. A row bootstrap assumes exchangeable independent rows. It does not honour district dependence or temporal blocks. With only three test years and five districts, a research report should:

- display year- and district-level results;
- avoid claiming precise asymptotic confidence;
- collect more independent periods or clusters when possible; and
- use a resampling design aligned to the actual sampling process.

## 20.10 Subgroup audit without leaderboard fishing

Subgroup evaluation asks whether failure is concentrated where the system matters. It becomes misleading when analysts scan dozens of tiny groups and publish only the worst one.

For every pre-specified subgroup, report:

- sample size;
- MAE and signed error;
- target prevalence or distribution, where relevant;
- quantile coverage;
- uncertainty or at least raw cases for small groups; and
- whether the analysis was pre-specified or exploratory.

A group with three projects cannot support a precise performance ranking. It can still expose a safety-relevant failure that warrants more data.

## 20.11 A model card for this study

Complete these fields before deployment.

| Field              | Required content                                                            |
|--------------------|-----------------------------------------------------------------------------|
| Intended use       | appraisal-stage cost planning for eligible MHP projects                     |
| Out-of-scope use   | causal effect estimation, other infrastructure types, unrepresented regions |
| Training period    | exact dates and inclusion rules                                             |
| Features           | source, units, availability time, missingness handling                      |
| Target             | accounting definition and inflation adjustment                              |
| Evaluation         | temporal folds, locked period, primary and subgroup metrics                 |
| Limitations        | rare access modes, five districts, simulated teaching data, extrapolation   |
| Monitoring         | feature drift, missingness, error, quantile coverage, override rate         |
| Human role         | who reviews estimates and can override them                                 |
| Retraining trigger | stated time, drift, or performance condition                                |

## 20.12 Reproducibility checklist

A second person should be able to reconstruct the result from:

- immutable raw data or a versioned generator;
- data dictionary and eligibility rules;
- source code and configuration;
- random seeds;
- exact package and Python versions;
- saved fold assignments;
- validation tables for every candidate;
- selected hyperparameters;
- locked-test predictions, not only aggregate scores;
- environment and hardware notes when relevant; and
- a record of exclusions and deviations.

Reproducibility means recovering the same computation. Replicability asks whether the conclusion persists with new data or a new implementation. A study can be reproducible and still wrong.

## 20.13 Reading guide and discussion prompts: Varma and Simon (2006)

Sudhir Varma and Richard Simon's *Bias in Error Estimation When Using Cross-Validation for Model Selection* shows that using the same cross-validation results both to choose a model and to report its error can produce optimistic estimates. They evaluate nested cross-validation as a correction.

Read with five questions.

1. **Procedure.** What exactly is selected—features, hyperparameters, or classifier?
2. **Estimand.** Is the desired error for a fixed model or for a model-selection procedure?
3. **Leak.** Why does taking the minimum among noisy cross-validation estimates create optimism?

4. **Remedy.** What occurs in the inner and outer loops of nested cross-validation?
5. **Transfer.** How should the idea change when observations are temporal or grouped rather than exchangeable?

## Replication task

Generate pure-noise predictors and an unrelated target. Compare many hyperparameter values. Record:

- the minimum inner validation error;
- an outer-fold estimate of the selected procedure; and
- performance on a large independent test set.

Increase the number of tried candidates. Observe how the most flattering inner estimate becomes more optimistic even though no candidate contains signal.

## 20.14 The capstone research report

Write a report with this structure.

1. **Abstract:** question, population, temporal design, selected procedure, test result, and main limitation.
2. **Prediction contract:** row, target, time, eligible population, and decision.
3. **Data:** provenance, units, missingness, chronology, and exclusions.
4. **Methods:** candidate procedures, temporal folds, metrics, and selection rule.
5. **Results:** all validation candidates, one locked-test result, subgroup audit, and quantile calibration.
6. **Diagnostics:** multicollinearity, influential projects, residual structure, and convergence.
7. **Sensitivity:** pre-specified alternatives and clearly labelled post-hoc analyses.
8. **Limitations:** sampling, dependence, shift, causal non-identification, and simulated-data status.
9. **Reproducibility:** code, seed, environment, and result files.
10. **Conclusion:** a bounded claim tied to the evidence.

## A defensible conclusion template

Under an expanding-window comparison on projects through 2022, procedure A had the lowest pre-specified average validation MAE. After refitting on all development projects, its MAE on projects beginning in 2023–2025 was X million PKR, compared with Y for the raw OLS reference. Performance was weakest for subgroup Z, based on n projects. The 80th-quantile model covered C% overall and C\_Z% in subgroup Z. These results support appraisal-stage prediction within the represented districts and feature ranges; they do not identify causal effects or establish performance in new regions.

Fill every placeholder. Preserve unfavourable results.

## 20.15 Day 20 build, break, and reflect

### Build

1. Write the analysis plan before running `main()`.
2. save every validation-fold score.
3. select one point and two quantile procedures without test access.
4. run the test once and complete the report template.
5. ask another learner to reproduce the result from a clean environment.

### Break

1. add a feature after viewing locked-test residuals and keep calling the test locked.
2. report the minimum validation error as unbiased final performance.
3. hide candidates that performed poorly.
4. omit group sizes from subgroup claims.
5. describe a temporal test in five fictional districts as universal external validation.

### **Reflect**

Which result would change your intended decision? If no possible result would change it, the benchmark is ceremonial rather than informative.

### **Day 20 exit check**

You have completed Chapter 3 when you can hand another researcher a frozen prediction contract, executable pipeline, complete candidate table, locked-test predictions, subgroup audit, and limitations section—and they can explain exactly where every reported number came from.

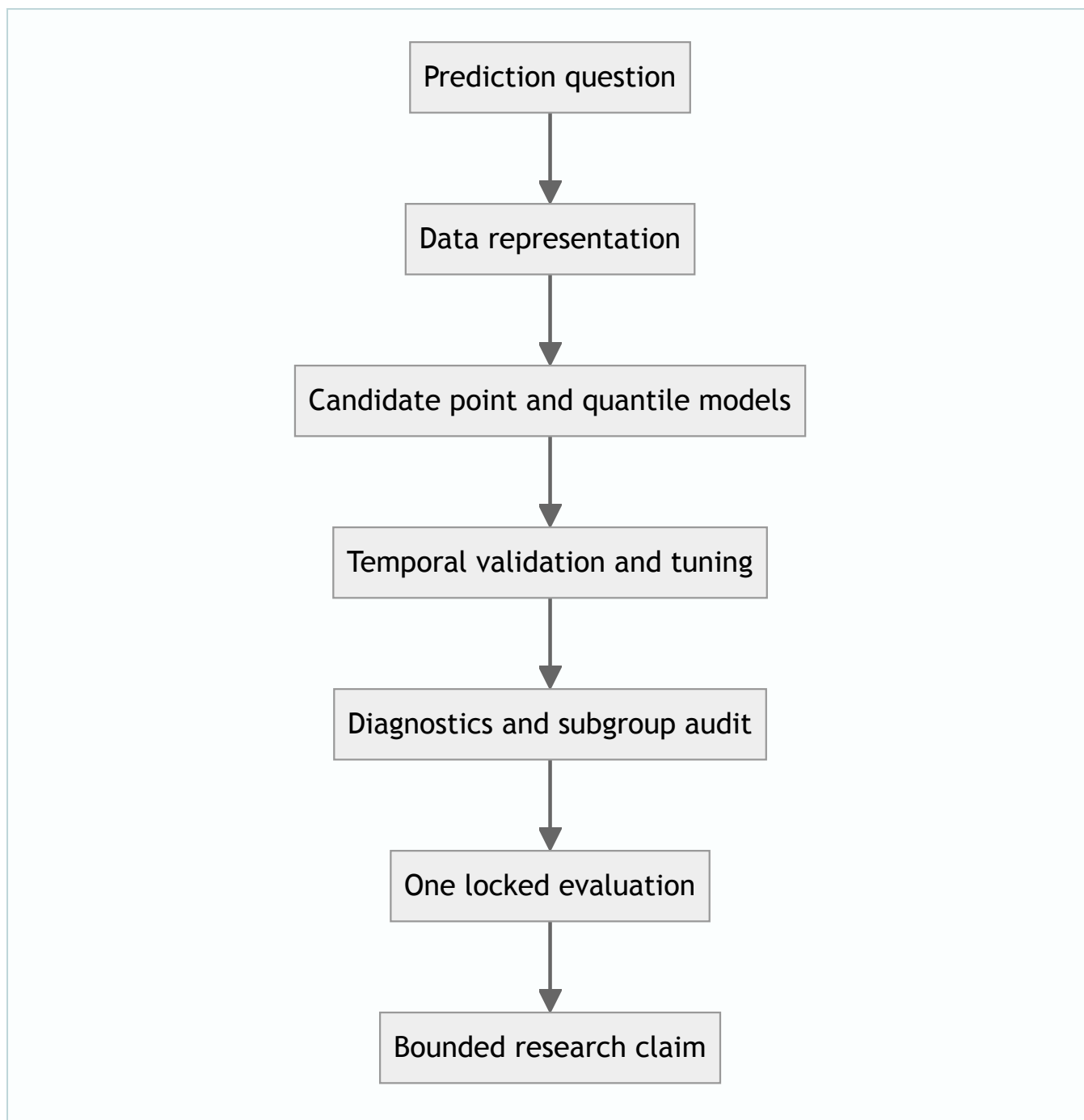
---

## **Chapter 3 Synthesis**

---

### **From technique collection to research procedure**

The chapter's methods form one system.



Preprocessing changes the information the estimator receives. Feature engineering changes the relationships it can express. Regularisation changes the fitting criterion. Diagnostics reveal where a fitted specification is weak or sample-dependent. Robust covariance changes a layer of uncertainty. Robust and quantile regression change the loss or target. Validation estimates performance of the entire procedure. A research claim is defensible only when these layers are kept distinct.

## Cumulative concept check

Answer without code.

1. Why can fitting an imputer before splitting be leakage even when the target is never used?
2. Why is target encoding especially vulnerable to leakage?
3. Can MCAR, MAR, and MNAR always be distinguished using observed data alone?
4. Why should a quadratic feature usually be accompanied by its linear term?



5. What does the derivative of a quadratic regression reveal that the sign of  $\beta_2$  alone does not?
6. Why is scaling essential to the interpretation of ridge and lasso penalties?
7. Which singular directions does ridge shrink most?
8. Why can lasso select one of two correlated features arbitrarily?
9. Does a lasso zero prove absence of a relationship?
10. What difference separates a target outlier from a high-leverage observation?
11. Why is Cook's distance not an automatic deletion rule?
12. What does VIF inflate, and relative to what reference?
13. Which part of an OLS result changes when HC3 is requested?
14. Why can five-cluster standard errors be unreliable despite hundreds of rows?
15. How does Huber loss differ from squared loss in the tails?
16. Which functional of  $\mathbf{Y} \mid \mathbf{X}$  does an 80th-quantile regression estimate?
17. Why can correct global quantile coverage hide a deployment problem?
18. What makes the minimum validation score optimistic after a large search?
19. What exactly does a locked test evaluate after tuning?
20. Why is reproducibility insufficient for scientific correctness?

## Derivation workshop

Complete each proof on paper, then verify numerically.

1. Differentiate the ridge objective and derive its normal equations.
2. Use an SVD to derive the ridge coefficient and fitted-value shrinkage factors.
3. Show that  $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$  is positive definite for  $\lambda > 0$ .
4. Derive lasso soft thresholding under  $\mathbf{X}^\top \mathbf{X} / n = \mathbf{I}$ .
5. Show that  $\sum_i h_{ii} = k$  for a full-rank OLS design.
6. Derive  $\text{Var}(\mathbf{e}_i \mid \mathbf{X}) = \sigma^2(1 - h_{ii})$ .
7. Expand the general sandwich covariance from the OLS coefficient expression.
8. Differentiate expected pinball loss and recover the quantile condition.
9. Show why the cost ratio  $c_u : c_o$  leads to  $\tau = c_u / (c_u + c_o)$ .
10. Derive the slope of a continuous predictor when it participates in both a square term and an interaction.

## Practical laboratories

### Laboratory A — missingness as information

Create two data-generating processes:

- road survey missing completely at random; and
- road survey missing more often for highly remote projects after controlling for observed access mode.

Compare complete cases, median imputation, median-plus-indicator, and a correctly specified multiple-imputation analysis. Separate predictive error from inferential coverage.

### Laboratory B — regularisation under correlation

Vary the correlation between two predictors from 0 to 0.999. Across repeated samples, plot:

- OLS coefficient variance;
- ridge coefficient variance and bias;
- lasso selection frequency; and
- elastic-net prediction error.

## Laboratory C — influence map

Create four synthetic rows:

- ordinary predictors and ordinary target;
- ordinary predictors and extreme target;
- extreme predictors lying on the fitted surface; and
- extreme predictors with an extreme residual.

Compare residual, leverage, and Cook's distance. Explain every ranking.

## Laboratory D — covariance failure

Simulate constant and increasing error variance. Estimate empirical coverage of classical, HC0, and HC3 slope intervals for several sample sizes and leverage patterns.

## Laboratory E — dependence

Generate projects nested within districts with a shared district shock. Increase projects per district while holding the number of districts fixed. Observe why nominal row count does not substitute for independent clusters.

## Laboratory F — robust target choice

Contaminate a regression outcome with increasingly large vertical shocks. Compare OLS, Huber, median regression, and ridge on mean-target and median-target losses. State why no single ranking is meaningful without a target.

## Laboratory G — quantile calibration

Fit 0.5 and 0.8 quantiles under homoskedastic and heteroskedastic data. Check pinball loss, overall coverage, subgroup coverage, and crossing.

## Laboratory H — selection optimism

Repeat the Varma–Simon replication with 5, 50, and 500 candidate settings. Compare the best inner score with an outer or independent score.

## Formula sheet

| Concept            | Formula                                                                 | Use                                                |
|--------------------|-------------------------------------------------------------------------|----------------------------------------------------|
| Standardisation    | $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$                                     | comparable penalty scale using training statistics |
| Ridge objective    | $\ y - X\beta\ _2^2 + \lambda\ \beta\ _2^2$                             | stable $L_2$ shrinkage                             |
| Ridge estimate     | $(X^\top X + \lambda I)^{-1} X^\top y$                                  | closed form for centred penalised slopes           |
| Ridge shrinkage    | $\sigma_j^2/(\sigma_j^2 + \lambda)$                                     | fitted-value retention in singular direction $j$   |
| Ridge effective df | $\text{tr}(H_\lambda)$                                                  | smooth measure of flexibility                      |
| Lasso objective    | $\ y - X\beta\ _2^2/(2n) + \alpha\ \beta\ _1$                           | sparse shrinkage                                   |
| Soft threshold     | $S(z, \alpha) = \text{sign}(z)( z  - \alpha)_+$                         | one-coordinate lasso solution                      |
| Elastic net        | $\text{loss} + \alpha\rho\ \beta\ _1 + \alpha(1 - \rho)\ \beta\ _2^2/2$ | sparsity plus correlated-feature stabilisation     |
| VIF                | $1/(1 - R_j^2)$                                                         | conditional variance inflation for slope $j$       |

| Concept              | Formula                                              | Use                                                   |
|----------------------|------------------------------------------------------|-------------------------------------------------------|
| Hat matrix           | $H = X(X^\top X)^{-1}X^\top$                         | maps outcomes to OLS fitted values                    |
| Average leverage     | $k/n$                                                | reference leverage for $k$ design columns             |
| Studentised residual | $e_i/[s\sqrt{1-h_{ii}}]$                             | residual adjusted for its model variance              |
| Cook's distance      | $e_i^2 h_{ii}/[ks^2(1-h_{ii})^2]$                    | aggregate case influence                              |
| General sandwich     | $(X^\top X)^{-1}X^\top \Omega X(X^\top X)^{-1}$      | coefficient covariance under general error covariance |
| HC0 meat             | $\sum_i e_i^2 x_i x_i^\top$                          | independent heteroskedastic errors                    |
| HC3 residual square  | $e_i^2/(1-h_{ii})^2$                                 | stronger leverage correction                          |
| Cluster meat         | $\sum_g (X_g^\top e_g)(X_g^\top e_g)^\top$           | arbitrary within-cluster covariance                   |
| Huber loss           | quadratic inside $\delta$ , linear outside           | reduce large-residual gradient                        |
| Conditional quantile | $\inf q : \Pr(Y \leq q \mid X = x) \geq \tau$        | distributional target at level $\tau$                 |
| Pinball loss         | $u(\tau - \mathbb{1}u < 0)$                          | proper loss for a conditional quantile                |
| Quantile coverage    | $m^{-1} \sum_i \mathbb{1}y_i \leq \hat{q}_\tau(x_i)$ | held-out calibration diagnostic                       |

All penalty formulas depend on the stated loss normalisation. Never treat the symbol  $\lambda$  or `alpha` as portable without checking that convention.

## Glossary

### Active set

Features whose lasso or elastic-net coefficients are nonzero at a given fit and tuning value.

### Bias–variance trade-off

The possibility that adding deliberate estimator bias can reduce sampling variance enough to improve expected prediction error.

### Complete-case analysis

An analysis restricted to rows with every required value observed.

### Conditional quantile

A quantile of the outcome distribution among observations with a specified feature vector.

### Condition number

A measure of sensitivity associated with the ratio of largest to smallest relevant singular values.

### Cook's distance

A measure of how much all fitted values change when one observation is deleted, scaled by estimated noise and model dimension.

### Cross-fitting

Constructing a learned representation for each training row using a model fitted without that row's fold.

### Elastic net

A penalised regression combining  $L_1$  and squared  $L_2$  penalties.

### Effective degrees of freedom

A smooth measure of fitted flexibility, such as  $\text{tr}(H_\lambda)$  for ridge.

**Heteroskedasticity**

Conditional error variance that changes across observations or predictor values.

**Hierarchy principle**

The practice of retaining lower-order terms that define higher-order polynomial or interaction effects.

**Huber loss**

A loss that is quadratic near zero and linear for sufficiently large absolute residuals.

**Influence**

Sensitivity of a fitted result to the presence of an observation.

**Leverage**

Unusualness of a row's predictor configuration in the geometry of a fitted design.

**Lasso**

Least-squares regression with an  $L_1$  penalty, capable of exact zero coefficients.

**MAR**

Missing at random: after conditioning on observed information in the missingness model, missingness does not depend on the missing value itself.

**MCAR**

Missing completely at random: missingness is independent of observed and missing data under the specified model.

**MNAR**

Missing not at random: missingness still depends on unobserved values after conditioning on observed information.

**Multiple imputation**

Creating several completed datasets to propagate missing-data uncertainty through an analysis and combine results under stated assumptions.

**Pinball loss**

An asymmetric absolute loss whose population minimiser is a chosen quantile.

**Post-selection inference**

Inference that accounts for the fact that a model or hypothesis was selected using observed data.

**Quantile crossing**

Predicted lower conditional quantiles exceeding predicted higher conditional quantiles for the same feature vector.

**Regularisation**

Constraining or penalising model parameters to control flexibility, improve stability, or encode prior preference.

**Ridge regression**

Least-squares regression with a squared  $L_2$  coefficient penalty.

**Sandwich covariance**

A covariance estimator with model-sensitivity matrices around an estimate of score variability.

**Shrinkage**

Pulling estimated coefficients toward a reference, usually zero.

**Soft thresholding**

Subtracting a threshold from coefficient magnitude and setting values inside the threshold to zero.

## Target encoding

Representing a category using outcome summaries, with smoothing and cross-fitting needed to control overfit and leakage.

## VIF

The factor by which collinearity inflates a slope's conditional variance relative to a corresponding orthogonal reference.

# Research-paper reading ladder

The papers are arranged by the conceptual problem they solve, not by publication date.

| Paper                    | Read after       | Central contribution                                       | Replication focus                                      |
|--------------------------|------------------|------------------------------------------------------------|--------------------------------------------------------|
| Rubin (1976)             | Day 13           | formal missing-data mechanisms and ignorability conditions | predictive and inferential consequences of missingness |
| Hoerl & Kennard (1970)   | Day 15           | biased shrinkage for nonorthogonal regression              | coefficient stability under collinearity               |
| Tibshirani (1996)        | Day 16           | $L_1$ shrinkage and selection                              | sparse prediction and correlated-feature instability   |
| Efron et al. (2004)      | Day 16 extension | efficient geometric path algorithm                         | compare LARS and coordinate paths                      |
| White (1980)             | Day 18           | heteroskedasticity-consistent covariance                   | interval coverage under changing variance              |
| Huber (1964)             | Day 19           | minimax robust location estimation                         | efficiency–contamination trade-off                     |
| Koenker & Bassett (1978) | Day 19           | linear conditional quantiles                               | varying conditional spread                             |
| Varma & Simon (2006)     | Day 20           | model-selection bias in CV error                           | nested versus non-nested estimates                     |

## A six-pass research reading method

1. **Citation pass:** record the question, year, venue, and terminology.
2. **Structure pass:** read the abstract, introduction, figures, and conclusion.
3. **Notation pass:** build a symbol table and restate every target in words.
4. **Derivation pass:** reproduce one central equation without copying.
5. **Evidence pass:** reconstruct one simulation or empirical table.
6. **Boundary pass:** list assumptions, failure modes, and what later work would need to test.

Reading a paper is not memorising its conclusion. It is reconstructing the argument well enough to find where it could fail.

# Capstone assessment rubric

| Dimension           | Emerging                 | Competent                        | Research-ready for this level                                                       |
|---------------------|--------------------------|----------------------------------|-------------------------------------------------------------------------------------|
| Prediction contract | target and row are vague | target, row, and time are stated | population, decision, availability, units, and exclusions are executable            |
| Preprocessing       | performed globally       | pipelines fit within splits      | missingness assumptions, category policy, and unknown-level behaviour are justified |

| Dimension      | Emerging                        | Competent                                  | Research-ready for this level                                              |
|----------------|---------------------------------|--------------------------------------------|----------------------------------------------------------------------------|
| Feature design | transformations are decorative  | mechanisms and hierarchy are stated        | derivatives, support, timing, and extrapolation limits are examined        |
| Regularisation | defaults used                   | penalties and scaling are tuned honestly   | objectives, paths, stability, and selection limits are explained           |
| Diagnostics    | ritual plots                    | collinearity and influence are interpreted | provenance and sensitivity lead to bounded decisions                       |
| Uncertainty    | one default standard error      | HC or clustered choice is justified        | remaining mean, dependence, and small-sample assumptions are explicit      |
| Quantiles      | fitted without target rationale | pinball loss and coverage reported         | decision ratio, subgroup calibration, and crossing are assessed            |
| Validation     | one convenient split            | temporal selection and locked test         | full procedure, search budget, tie rule, and adaptation log are preserved  |
| Reporting      | best metric only                | reference and subgroup metrics included    | predictions, uncertainty limits, failures, and deviations are reproducible |

## Instructor and self-study notes

### Suggested daily rhythm

Each day can be taught in roughly three hours.

| Segment              | Approximate time | Activity                                                   |
|----------------------|------------------|------------------------------------------------------------|
| Retrieval            | 15 minutes       | answer the previous exit check without notes               |
| Concept construction | 35 minutes       | small example and visual intuition                         |
| Mathematical proof   | 40 minutes       | derive the day's central result                            |
| Code laboratory      | 55 minutes       | implement, verify, and compare with a library              |
| Failure laboratory   | 25 minutes       | deliberately break an assumption or boundary               |
| Research reading     | 20 minutes       | one figure, theorem, or simulation from the assigned paper |
| Reflection           | 10 minutes       | write a bounded claim and remaining uncertainty            |

### Frequent misconceptions to surface

- Scaling does not make an unpenalised feature intrinsically important.
- Imputation is a model, not neutral replacement.
- A polynomial can be linear in coefficients while nonlinear in a feature.
- Ridge stability does not identify a causal variable.
- Lasso sparsity is sample- and tuning-dependent.
- High VIF need not damage in-range prediction.
- A large residual is not the definition of leverage.
- HC3 is not robust regression.
- A quantile is not a confidence interval.
- A locked test is spent once analysis adapts to it.

## Ethical and contextual discussion

The fictional Khyber Pakhtunkhwa setting is not a claim that simulated values represent actual districts. A real study must involve local technical, financial, and community knowledge.

Discuss:

- who bears harm when a remote project's cost is underestimated;
- whether historical underinvestment becomes encoded as a feature effect;
- which appraisal fields can be measured reliably across districts;
- whether a “rare” access mode is rare only because data collection excluded it;
- how procurement incentives can alter target quality; and
- who is authorised to override a model and how that decision is recorded.

Technical competence without contextual accountability is not research readiness.

## Where Chapter 4 begins

Chapter 3 remains within regression for continuous outcomes. Chapter 4 should introduce generalised linear models and classification while preserving the same research discipline:

- probability, odds, and log odds;
- Bernoulli likelihood and logistic regression;
- maximum likelihood, gradients, and convex optimisation;
- regularised classification;
- calibration and proper scoring rules;
- threshold choice as a decision problem;
- class imbalance and precision–recall analysis;
- subgroup error and fairness definitions;
- tree-based models as nonlinear benchmarks;
- external and prospective validation; and
- a locked classification study.

The learner should not leave linear models behind. Logistic regression will reuse design matrices, interactions, penalties, influence, temporal validation, and post-selection caution.

## Final reflection

Research level is not reached by accumulating model names. It is approached by making every link in an argument inspectable:

question → data → representation → estimator → validation → uncertainty → claim.

At each arrow, ask what could leak, what could shift, what assumption entered, and what evidence would change your mind.

---

# References and Further Reading

---

## Research papers discussed in the chapter

- Efron, B., Hastie, T., Johnstone, I., & Tibshirani, R. (2004). Least Angle Regression. *The Annals of Statistics*, 32(2), 407–499.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
- Huber, P. J. (1964). Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1), 73–101.
- Koenker, R., & Bassett, G. (1978). Regression Quantiles. *Econometrica*, 46(1), 33–50.
- Rubin, D. B. (1976). Inference and Missing Data. *Biometrika*, 63(3), 581–592.
- Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- Varma, S., & Simon, R. (2006). Bias in Error Estimation When Using Cross-Validation for Model Selection. *BMC Bioinformatics*, 7, 91.
- White, H. (1980). A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity. *Econometrica*, 48(4), 817–838.

## Software documentation

- scikit-learn. Linear Models User Guide.
- scikit-learn. Ridge API, Lasso API, and ElasticNet API.
- scikit-learn. HuberRegressor API and QuantileRegressor API.
- scikit-learn. Target Encoder's Internal Cross-Fitting.
- statsmodels. Regression and Linear Models, OLS Results, and Variance Inflation Factor.

## Recommended textbooks for the next pass

- Fox, J. *Applied Regression Analysis and Generalized Linear Models*.
- Harrell, F. E. *Regression Modeling Strategies*.
- Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning*.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. *An Introduction to Statistical Learning*.
- Kuhn, M., & Johnson, K. *Applied Predictive Modeling*.
- van Buuren, S. *Flexible Imputation of Missing Data*.

Use textbooks to consolidate notation and breadth. Return to primary papers to understand what was actually established and under which assumptions.



# Chapter 3 Companion Guide: From a Trustworthy Baseline to a Research-Grade Regression Study

## A beginner-first guide to preprocessing, feature design, regularisation, diagnostics, robust inference, quantiles, and auditable research

This guide accompanies `chapter3.md`. It assumes you have completed Chapters 1 and 2, but it continues to explain new Python, mathematics, and statistics from first principles.

Chapter 1 built ordinary least squares (OLS). Chapter 2 surrounded it with stable computation and honest evaluation. Chapter 3 develops the habits of a regression practitioner:

1. treat preprocessing as part of the fitted model;
2. express substantive hypotheses through features;
3. stabilise complex designs through regularisation;
4. find projects and predictor directions that dominate results;
5. match uncertainty calculations to variance and dependence;
6. change the loss or conditional target when the decision requires it; and
7. freeze the full comparison procedure before opening final evidence.

The goal is not to collect model names. It is to make every link auditable:

```
question
-> data
-> representation
-> estimator
-> validation
-> uncertainty
-> bounded claim
```

At every arrow ask: What was learned? From which rows? Which assumption entered? What could leak? What could shift? What evidence would change the decision?

## How to use this guide

For Days 13–20:

1. Read the matching guide section.
2. Reproduce the central equation on paper.
3. Type and alter the smallest code example.
4. Run the main chapter's complete laboratory.
5. Create the stated failure deliberately.
6. Write one supported conclusion and one unsupported conclusion.
7. Complete the exit check without notes.

When evaluating a technique, separate these layers:

- **representation**: which columns enter the model;
- **target**: mean, median, or another quantile;
- **fitting criterion**: squared, Huber, pinball, or penalised loss;
- **uncertainty**: classical, heteroskedasticity-consistent, clustered, bootstrap, or none;
- **evaluation**: which untouched cases simulate deployment; and
- **interpretation**: predictive, explanatory, or causal.

Methods acting on different layers are not interchangeable.

---

## 0. Prerequisite and software foundations

---

### 0.1 Retrieve these Chapter 1 and 2 ideas

Before proceeding, explain without notes:

- $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$  and  $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ ;
- the OLS objective and gradient;
- full rank, singular values, and condition number;
- why scaling and imputation fit only on training data;
- train, validation, test, grouped CV, temporal CV, and nested CV;
- error term versus residual;
- coefficient confidence interval versus project prediction interval; and
- ME, MAE, RMSE, held-out  $R^2$ , and subgroup evaluation.

Chapter 3 relies on these rather than replacing them.

### 0.2 The MHP prediction contract

One row is one fictional microhydro power project. The model predicts final project cost in constant 2025 million PKR at technical appraisal, before procurement and construction.

Chapter 3 adds:

- missing road-distance surveys;
- contractor experience counts;
- categorical access mode;
- a correlated route-length measurement;
- curved capacity costs;
- cable-by-road interactions;
- larger variability for remote projects; and
- a few extreme but legitimate logistics events.

The complete hidden `road_distance_km` remains in the synthetic generator only so learners can audit how missingness was created. In a real incomplete dataset, the true missing values are unavailable. Using the hidden column as a feature would be leakage.

## 0.3 Software setup

```
python -m pip install numpy pandas scipy matplotlib scikit-learn statsmodels
```

Scikit-learn supports predictive pipelines and regularised estimators. Statsmodels supplies mature inferential and influence diagnostics. Educational from-scratch functions reveal mechanisms; verify them against tested libraries before relying on them.

Record Python and package versions, seeds, input-data versions, and output files for every benchmark.

## 0.4 Understanding the practitioner data generator

`make_mhp_practitioner_data` extends Chapter 2's synthetic data.

```
df["contractor_experience_projects"] = rng.poisson(lam=5.0, size=n)
df["access_mode"] = np.select(conditions, choices, default="porter_or_air")
df["road_distance_observed_km"] = df["road_distance_km"].mask(missing_road)
```

- A Poisson draw creates nonnegative event counts.
- `np.select` applies the first true condition and otherwise uses a default.
- `.mask(condition)` replaces true positions with missing values.

Missingness is more likely for difficult access and early years. Extreme logistics events are added only to some remote projects. These are genuine outcomes within the simulated population, not automatic deletion candidates.

Always distinguish:

- **latent complete value:** available to the simulation author;
- **observed appraisal value:** available to the fitted procedure; and
- **post-appraisal field:** prohibited at prediction time.

## 0.5 New mathematical language

### Norms

For coefficient vector  $\beta$ :

$$\|\beta\|_1 = \sum_j |\beta_j|, \quad \|\beta\|_2^2 = \sum_j \beta_j^2.$$

The  $L_1$  norm sums magnitudes. Squared  $L_2$  sums squares. Lasso and ridge use them as penalties.

### Trace

The trace of a square matrix is the sum of diagonal elements:

$$\text{tr}(A) = \sum_j A_{jj}.$$

Trace appears in average leverage and ridge effective degrees of freedom.

## Indicator functions

$\mathbb{1}_{\text{condition}}$  equals 1 when the condition is true and 0 otherwise. It is the mathematical version of a Boolean converted to a number.

## Partial derivatives and conditional slopes

If a prediction depends on several inputs,  $\partial \hat{y} / \partial x_j$  measures the local change in prediction as  $x_j$  changes while other displayed inputs remain fixed.

## Subgradients

$|b|$  has no single derivative at zero because its left slope is -1 and right slope is +1. A subgradient uses every slope in the interval  $[-1, 1]$  at that kink. This permits the lasso optimum to remain exactly zero.

## Positive definite matrices

A symmetric matrix  $A$  is positive definite when:

$$v^T A v > 0$$

for every nonzero  $v$ . Such curvature gives a unique minimum for a quadratic problem.

## MAP estimation

Maximum a posteriori (MAP) estimation maximises a Bayesian posterior density. Under particular priors, its objective matches ridge or lasso. A MAP point is not a full posterior distribution and does not by itself quantify uncertainty.

# 0.6 New Python and scikit-learn patterns

## Custom transformers

A scikit-learn transformer implements `fit` and `transform`:

```
from sklearn.base import BaseEstimator, TransformerMixin

class ExampleTransformer(BaseEstimator, TransformerMixin):
 def fit(self, X, y=None):
 self.learned_value_ = X["column"].median()
 return self

 def transform(self, X):
 X = X.copy()
 X["centered"] = X["column"] - self.learned_value_
 return X
```

The trailing underscore marks learned state. `fit` must use only the current training fold. `transform` applies stored state to validation, test, or deployment rows.

## statsmodels

```
import statsmodels.api as sm

X_design = sm.add_constant(X, has_constant="add")
```

```
result = sm.OLS(y, X_design).fit()
print(result.summary())
```

Statsmodels generally expects you to add the intercept explicitly. Its summary is not automatically valid merely because it prints p-values; design and covariance assumptions still matter.

## Logarithmic grids

```
alphas = np.logspace(-4, 4, 100)
```

This creates penalty values from  $10^{-4}$  to  $10^4$ . Regularisation strength often needs orders-of-magnitude exploration rather than equally spaced values.

---

# Day 13 — Preprocessing is part of the model

---

## 13.1 Missingness is data

For feature  $j$  and row  $i$ :

$$R_{ij} = \begin{cases} 1, & X_{ij} \text{ observed,} \\ 0, & X_{ij} \text{ missing.} \end{cases}$$

The missingness mechanism describes how  $R$  relates to observed and unobserved values. It is not a moral label for data collectors.

## 13.2 MCAR, MAR, and MNAR

Let  $X_{obs}$  be observed data and  $X_{mis}$  missing values.

### Missing completely at random (MCAR)

$$P(R \mid X_{obs}, X_{mis}) = P(R).$$

Missingness is unrelated to observed and missing values, such as an independent random scanner failure.

### Missing at random (MAR)

$$P(R \mid X_{obs}, X_{mis}) = P(R \mid X_{obs}).$$

After conditioning on observed information, missingness no longer depends on the unseen value. “At random” does not mean “for no reason.” Early year and observed access mode might explain recording probability.

### Missing not at random (MNAR)

Even after conditioning on observed information, missingness depends on an unseen value or unobserved factor. Extremely long access distances may be omitted precisely because they are hard to measure.

Observed data generally cannot prove which mechanism holds or rule out MNAR. Sensitivity analysis must state alternative assumptions.

## 13.3 Complete-case analysis changes the population

Deleting every incomplete row can:

- reduce precision;
- change district and access-mode composition;
- remove remote projects systematically; and
- bias estimates unless deletion is justified for the analysis.

Compare retained and excluded rows on observed fields, including year, terrain, access mode, and retrospective outcomes where appropriate for an audit. This can reveal selection but cannot prove deletion safe.

Do not use the target in a deployment-time imputer merely because it was legitimate in a retrospective missingness investigation.

## 13.4 Median imputation is a learned model

$$x_{ij}^{imp} = \begin{cases} x_{ij}, & R_{ij} = 1, \\ \text{median}(X_{j,train}), & R_{ij} = 0. \end{cases}$$

Median imputation preserves rows and offers a useful predictive baseline, but it does not recover truth:

- values pile up at the median;
- variance shrinks;
- relationships with other variables weaken; and
- ordinary formulas treating imputed values as observed understate uncertainty.

The median must be learned independently inside every training fold.

## 13.5 Missingness indicators

$$M_{ij} = 1 - R_{ij}.$$

Adding  $M$  lets a model learn an offset for rows whose value was imputed. In scikit-learn:

```
SimpleImputer(strategy="median", add_indicator=True)
```

The future missing value gets the training median plus indicator 1. An observed future value remains unchanged and gets indicator 0.

An indicator can exploit stable predictive information in recording patterns. It does not make MNAR ignorable, identify a cause, or restore the missing distribution.

## 13.6 Single and multiple imputation serve different purposes

Single imputation makes one completed feature matrix. Multiple imputation creates  $m$  plausible completed datasets, runs the analysis on each, and pools estimates and uncertainty.

For estimate  $\hat{Q}_l$  and within-imputation variance  $U_l$ :

$$\bar{Q} = \frac{1}{m} \sum_{l=1}^m \hat{Q}_l,$$

$$\bar{U} = \frac{1}{m} \sum_{l=1}^m U_l,$$

$$B = \frac{1}{m-1} \sum_{l=1}^m (\hat{Q}_l - \bar{Q})^2,$$

$$T = \bar{U} + \left(1 + \frac{1}{m}\right) B.$$

$\bar{U}$  is ordinary within-completed-data uncertainty.  $B$  measures sensitivity to plausible missing values. The final factor acknowledges finite imputations.

Generating several datasets is not enough. Imputation and analysis models must support the estimand, include relevant missingness predictors, respect deployment timing, and fit separately inside evaluation folds. Prediction performance is scored on observed held-out outcomes.

## 13.7 Categorical encodings impose hypotheses

### One-hot encoding

With A as reference:

$$\hat{y} = \beta_0 + \beta_B I(B) + \beta_C I(C).$$

$\beta_B$  is B minus A conditional on other features. Changing reference changes coefficient labels but not fitted values. With an intercept, dropping one category prevents exact redundant coding.

### Ordinal encoding

Mapping low, medium, high to 0, 1, 2 imposes order. A linear coefficient additionally imposes equal gaps. Never assign numbers to districts merely because software needs numeric input.

### Frequency encoding

Replace a category with its training frequency. This says category commonness is predictive and makes equally frequent categories indistinguishable.

### Target encoding

$$TE(c) = \frac{1}{n_c} \sum_{i: C_i=c} y_i.$$

This uses outcome information. Naively encoding training rows with their own category mean leaks each row's target, especially for rare or unique categories.

## 13.8 Smoothed and cross-fitted target encoding

Smoothed category mean:

$$\widehat{TE}(c) = \frac{n_c \bar{y}_c + m \bar{y}}{n_c + m}.$$

Rare categories shrink toward the global training mean. Smoothing reduces variance but does not eliminate same-row leakage.

Cross-fitting procedure:

1. split development training data into internal folds;
2. calculate category statistics on the other folds;
3. encode the held-out fold;
4. combine out-of-fold training encodings;
5. fit a final mapping on all development rows only for future transformation; and
6. map unseen categories to a pre-specified fallback such as training global mean.

The internal splitter must also respect groups and time. Random target-encoding folds can leak district or future information while avoiding same-row leakage.

Target encoding therefore needs two protections:

- cross-fitting within development; and
- complete separation from the final test.

## 13.9 A leakproof mixed-type pipeline

Numeric branch:

```
median imputation + missing indicators -> standardisation
```

Categorical branch:

```
most-frequent imputation -> one-hot encoding
```

`ColumnTransformer` combines branches, then a model fits transformed columns. Every cross-validation fold learns its own medians, indicator availability, scales, and categories.

`handle_unknown="ignore"` prevents unseen categories from crashing. It does not demonstrate good transfer to them.

## 13.10 Research discussion: Rubin on missing data

Rubin formalised when a missingness process may be ignored for particular likelihood or sampling analyses.

Beginner lessons:

1. MAR is conditional on observed information.
2. Ignorability depends on the mechanism and inferential framework.
3. Observed values generally cannot rule out MNAR.
4. Filling blanks is not propagating missing-value uncertainty.

MHP implication: missing remote-road surveys require recording-process knowledge and sensitivity analysis, not only a median-filled column.

Replication: simulate identical complete data followed by MCAR, MAR, and MNAR deletion. Compare complete cases, median imputation, median-plus-indicator prediction, and multiple-imputation interval coverage. Keep predictive error separate from coefficient bias.



## Day 13 exit check

Explain why MAR is not “missing for no reason,” why a missing indicator does not solve MNAR, and why target encoding needs cross-fitting plus final-test separation.

---

# Day 14 — Feature engineering is model specification

---

## 14.1 Linear in parameters can be nonlinear in a feature

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$$

is curved in  $x$  but linear in coefficients. The design matrix simply contains  $\mathbf{1}, x, x^2$ . OLS or regularised linear regression can fit it using familiar machinery.

Creating a column changes the family of possible relationships. It is a mathematical hypothesis, not free accuracy.

## 14.2 Centre before powers and interactions

Let:

$$x_c = x - c,$$

where  $c$  is a meaningful reference such as the training median.

$$\hat{y} = \gamma_0 + \gamma_1 x_c + \gamma_2 x_c^2.$$

Then:

- $\gamma_0$  is prediction at  $x = c$ ;
- $\gamma_1$  is slope at  $x = c$ ;
- $x_c$  and  $x_c^2$  are often less correlated; and
- numerical interpretation improves.

Data-derived centres are learned preprocessing and fit only inside training folds.

## 14.3 Interpret quadratics through derivatives

For:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2,$$

the local slope is:

$$f'(x) = \beta_1 + 2\beta_2 x.$$

A stationary point occurs at:

$$x^* = -\frac{\beta_1}{2\beta_2},$$

when  $\beta_2 \neq 0$ .

- $\beta_2 > 0$ : convex, stationary point is a minimum.
- $\beta_2 < 0$ : concave, stationary point is a maximum.

Before interpreting, check whether  $\mathbf{x}^*$  lies inside observed and credible deployment support, whether uncertainty is acceptable, whether the specification fits, and whether a causal claim is identified. The sign of  $\beta_2$  alone proves none of these.

## 14.4 Continuous-by-continuous interactions

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2.$$

Conditional slopes:

$$\frac{\partial \hat{y}}{\partial x_1} = \beta_1 + \beta_3 x_2,$$

$$\frac{\partial \hat{y}}{\partial x_2} = \beta_2 + \beta_3 x_1.$$

$\beta_1$  is the slope of  $x_1$  only at  $x_2 = 0$ . Centring makes zero a meaningful reference. An MHP cable-by-road interaction allows cable cost slope to vary with remoteness.

## 14.5 Continuous-by-binary interactions

Let  $D = 1$  for porter/air access and 0 for reference access:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 D + \beta_3 x D.$$

For  $D = 0$ :

$$\hat{y} = \beta_0 + \beta_1 x.$$

For  $D = 1$ :

$$\hat{y} = (\beta_0 + \beta_2) + (\beta_1 + \beta_3)x.$$

$\beta_2$  changes intercept and  $\beta_3$  changes slope.

## 14.6 Hierarchy principle

When including  $\mathbf{x}^2$ , normally retain  $\mathbf{x}$ . When including  $\mathbf{x}_1 \mathbf{x}_2$ , retain both main effects.

This:

- anchors higher-order interpretation;
- avoids unintended constraints; and
- makes predictions invariant to harmless shifts of feature origin.

Hierarchy is a disciplined default, not an absolute theorem. Depart only for a defensible substantive constraint.

## 14.7 Ratios make strong assumptions

$$r = \frac{x_1}{x_2}.$$

Risks:

- denominator near zero creates instability;
- both measurements may contain error;
- separate size information is lost;
- denominator may be post-outcome and leak; and
- low ratio error may not imply low total-budget error.

“Cost per kW” changes the estimand from total cost. Converting it back requires capacity weighting and careful loss interpretation.

## 14.8 Log transformations

For positive  $x$ , use  $\log x$ . For nonnegative counts,  $\log(1+x)$  is common.

Interpretations for small changes:

- level-log:  $y = \beta_0 + \beta_1 \log x$ ; 1% more  $x$  corresponds to about  $0.01\beta_1$  target units;
- log-level:  $\log y = \beta_0 + \beta_1 x$ ; one more unit of  $x$  corresponds to about  $100\beta_1$  percent change when  $\beta_1$  is small;
- log-log: coefficient is elasticity.

Logs compress large values but do not guarantee normality, and normal features are not an OLS requirement. Exponentiating a predicted log target generally returns a conditional median under log-error assumptions, not automatically the mean. Retransformation bias requires treatment.

## 14.9 Piecewise linear hinges and splines

Hinge at knot  $k$ :

$$(x - k)_+ = \max(0, x - k).$$

Model:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 (x - k)_+.$$

- below  $k$ : slope  $\beta_1$ ;
- above  $k$ : slope  $\beta_1 + \beta_2$ .

Splines use basis functions:

$$f(x) = \sum_{m=1}^M \theta_m B_m(x).$$

Once knots and degree define the bases, coefficients remain linearly fitted. Knot count, placement, and degree control flexibility and belong inside model selection unless pre-specified.

Splines usually behave more locally than high-degree global polynomials, but no fitted curve certifies unsupported extrapolation.

## 14.10 Feature explosion

Degree-2 expansion of  $p$  inputs can create:

- $p$  main effects;
- $p$  squares; and
- $p(p-1)/2$  pairwise interactions.

This increases computation, collinearity, search opportunities, and noise fitting. Domain knowledge limits candidates; regularisation stabilises them; validation evaluates the complete procedure.

## 14.11 Read MHPFeatureEngineer

Its `fit` validates required columns and learns training medians. `transform` creates:

- centred capacity and its square;
- centred cable-by-road interaction;
- an 18 km road hinge; and
- `log1p` contractor experience.

Pipeline order matters: a feature transformer operating on missing road values must either follow imputation or handle missing values deliberately. Every generated column needs an information receipt:

- raw fields used;
- whether target was used;
- rows used to learn constants;
- units;
- mechanism represented;
- valid range; and
- availability at appraisal.

## 14.12 Extrapolation audit

Plot fitted functions inside and outside observed ranges. A quadratic and hinge can agree over 80–850 kW yet diverge severely at 1,300 kW. Validation on existing support cannot certify a region with no comparable projects.

## Day 14 exit check

Interpret a quadratic through its derivative, explain an interaction using conditional slopes, apply hierarchy, and state why feature generation repeats inside every validation training fold.

---

# Day 15 — Ridge regression: shrinkage and stability

---

## 15.1 Why accept bias?

Nearly redundant predictors can yield large opposite OLS coefficients that cancel in predictions. Small sample changes reassign their contribution. Ridge accepts deliberate bias to reduce coefficient variance and possibly improve future prediction.

## 15.2 Ridge objective

For centred target and standardised features, with intercept separated:

$$J_{\text{ridge}}(\beta) = \|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2, \quad \lambda \geq 0.$$

- $\lambda = 0$ : OLS where unique.
- small positive  $\lambda$ : mild shrinkage.
- large  $\lambda$ : penalised slopes approach zero.
- intercept is ordinarily not penalised.

Packages normalise loss differently using  $n$ , 2, or  $2n$ . Therefore `alpha` and  $\lambda$  values are not portable without checking the documented objective.

## 15.3 Derive the ridge solution

Gradient:

$$\nabla J = -2X^T y + 2X^T X\beta + 2\lambda\beta.$$

Set to zero:

$$(X^T X + \lambda I)\hat{\beta}_{\text{ridge}} = X^T y.$$

Thus:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

In code, solve the system; do not explicitly invert.

### Why a unique solution exists

For nonzero  $v$  and  $\lambda > 0$ :

$$v^T (X^T X + \lambda I)v = \|Xv\|_2^2 + \lambda\|v\|_2^2 > 0.$$

The system is positive definite even when  $X^T X$  is singular. Ridge chooses a unique coefficient vector but does not create scientific information to distinguish identical features.

## 15.4 Scaling defines the penalty

The same distance measured in metres needs a coefficient one-thousandth the per-kilometre coefficient. Penalising raw magnitudes treats arbitrary units differently.

Standardising inside each fold makes one coefficient unit correspond to one training standard deviation. It does not make features causal, equally useful, Gaussian, or outlier-free.

## 15.5 SVD explains ridge shrinkage

For  $X = U\Sigma V^T$ :

$$\hat{\beta}_{ridge} = V \text{diag} \left( \frac{\sigma_j}{\sigma_j^2 + \lambda} \right) U^T y.$$

Fitted values:

$$\hat{y}_{ridge} = U \text{diag} \left( \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \right) U^T y.$$

Direction-specific retention:

$$s_j(\lambda) = \frac{\sigma_j^2}{\sigma_j^2 + \lambda}.$$

Large singular directions retain more. Weak directions shrink more. OLS's  $1/\sigma_j$  amplification is replaced by  $\sigma_j/(\sigma_j^2 + \lambda)$ .

## 15.6 Effective degrees of freedom

Ridge smoother matrix:

$$H_\lambda = X(X^T X + \lambda I)^{-1} X^T.$$

Effective flexibility:

$$df_{eff}(\lambda) = \text{tr}(H_\lambda) = \sum_{j=1}^r \frac{\sigma_j^2}{\sigma_j^2 + \lambda}.$$

This need not be an integer. It equals rank at zero penalty for the centred component and decreases as shrinkage grows.

## 15.7 Constrained geometry

Ridge is equivalent for a corresponding  $t$  to:

$$\min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_2^2 \leq t.$$

In two dimensions the constraint is a circle. SSR contours are ellipses. Their first contact gives the ridge solution. A smooth circle generally does not touch at an axis, so ridge shrinks but rarely makes exact zeros.

## 15.8 Read RidgeFromScratch

It:

1. validates nonnegative alpha;
2. learns training means and scales;
3. centres target;
4. solves  $Z^T Z + \alpha I$ ;
5. converts standardised slopes back to raw units; and
6. reconstructs the intercept.

Constant columns receive scale 1 to avoid division by zero, but should still be investigated. Predictions should match `StandardScaler` plus scikit-learn `Ridge` under the same objective.

## 15.9 Coefficient paths and tuning

Plot standardised coefficients against log penalty. Paths reveal whether correlated slopes stabilise and how quickly weak directions shrink.

The path is development information. Select alpha with pre-specified validation, never by choosing an attractive final-test coefficient.

## 15.10 Bias–variance trade-off

Ridge increases bias. If variance falls by more, expected prediction error improves. It is not universally superior: adequate sample size, few well-conditioned features, and correct linear structure may favour OLS.

## 15.11 Bayesian MAP interpretation

Gaussian likelihood:

$$y \mid X, \beta \sim \mathcal{N}(X\beta, \sigma^2 I).$$

Gaussian prior on standardised slopes:

$$\beta \sim \mathcal{N}(0, \tau^2 I).$$

Negative log posterior is proportional to:

$$\frac{1}{2\sigma^2} \|y - X\beta\|_2^2 + \frac{1}{2\tau^2} \|\beta\|_2^2.$$

Thus MAP has ridge form with  $\lambda = \sigma^2/\tau^2$ . Smaller prior variance means stronger shrinkage. This interpretation depends on that prior and does not provide the whole posterior.

## 15.12 Ridge limitations

Ridge does not automatically provide causality, valid post-tuning p-values, leakage protection, outlier resistance, missing-data validity, or variable selection. Tuning makes selection part of the estimator; fixed-model uncertainty formulas do not automatically include it.

## 15.13 Research discussion: Hoerl and Kennard

Their proposal showed how biased estimates can lower coefficient mean squared error in nonorthogonal problems. Read separately:

- instability being addressed;
- diagonal modification of normal equations;
- coefficient MSE versus modern test prediction error;
- original ridge-constant choice versus nested/temporal validation; and
- dependence on feature units and linear assumptions.

Replication: across many samples with  $\mathbf{x}_2 = \mathbf{x}_1 + \epsilon$ , compare OLS and ridge coefficient bias, variance, coefficient MSE, and test prediction error as correlation increases.

## Day 15 exit check

Derive ridge normal equations, explain singular-direction shrinkage and effective degrees of freedom, and state why scaling defines penalty meaning.

---

# Day 16 — Lasso, elastic net, and sparse models

---

## 16.1 Lasso objective

For centred target and standardised design:

$$J_{\text{lasso}}(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1.$$

This matches scikit-learn's convention. The intercept is separate and unpenalised. Because absolute value has a kink at zero, there is no ordinary closed-form matrix inverse—and exact zero coefficients become possible.

## 16.2 Subgradient condition

For nonzero  $b$ :

$$\frac{d|b|}{db} = \text{sign}(b).$$

At zero:

$$\partial|b| = [-1, 1].$$

Optimality:

$$-\frac{1}{n} X^T (y - X\hat{\beta}) + \alpha z = 0,$$

where  $z_j = 1$  for positive coefficients,  $-1$  for negative coefficients, and any value in  $[-1, 1]$  at zero. A coefficient remains zero when its residual correlation lies within the penalty threshold.



## 16.3 Derive soft thresholding

For orthonormal standardised columns:

$$\frac{1}{n} X^T X = I, \quad z_j = \frac{1}{n} x_j^T y.$$

One-coordinate objective becomes:

$$\frac{1}{2}(\beta_j - z_j)^2 + \alpha|\beta_j|.$$

Cases:

- $z_j > \alpha$ :  $\hat{\beta}_j = z_j - \alpha$ ;
- $z_j < -\alpha$ :  $\hat{\beta}_j = z_j + \alpha$ ;
- $|z_j| \leq \alpha$ :  $\hat{\beta}_j = 0$ .

Combined:

$$S(z, \alpha) = \text{sign}(z) \max(|z| - \alpha, 0).$$

Soft thresholding both shrinks and selects.

## 16.4 Why $L_1$ geometry produces zeros

The constrained form uses a diamond:

$$\min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_1 \leq t.$$

Diamond corners lie on coordinate axes. Expanding elliptical SSR contours often first touch a corner. Ridge's circular boundary usually touches away from axes.

## 16.5 Coordinate descent

For coordinate  $j$ , calculate the partial residual excluding it:

$$r^{(j)} = y - \sum_{k \neq j} x_k \beta_k.$$

Update:

$$\beta_j \leftarrow \frac{S(x_j^T r^{(j)} / n, \alpha)}{x_j^T x_j / n}.$$

Cycle through coefficients until convergence. The from-scratch class centres and scales, caches column energy, updates one coordinate at a time, and checks maximum coefficient change.

Production solvers also examine optimality through a duality gap. Verify educational predictions against scikit-learn with identical scaling, alpha, tolerance, and objective normalisation.

## 16.6 Correlated predictors make selection unstable

When road distance and surveyed route carry overlapping information, lasso can select either one across resamples while predictions remain similar.

A zero means:

This feature was not used by this penalised fit at this penalty, sample, and representation.

It does not mean no relationship, no importance, or no causal role.

Selection-frequency bootstraps are descriptive unless embedded in a formal stability-selection method. Use transformed feature names from the fitted pipeline rather than guessing coefficient order.

## 16.7 Elastic net

$$J_{EN}(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \alpha\rho\|\beta\|_1 + \frac{\alpha(1-\rho)}{2} \|\beta\|_2^2.$$

Scikit-learn calls  $\rho$  `l1_ratio`.

- $\rho = 1$ : lasso.
- $\rho = 0$ : ridge-like, though use `Ridge` for pure `L_2`.
- between 0 and 1: sparsity plus correlated-feature stabilisation.

Both alpha and ratio belong inside tuning.

## 16.8 Bayesian lasso interpretation

With Gaussian outcome errors and independent Laplace priors:

$$p(\beta_j | b) = \frac{1}{2b} \exp\left(-\frac{|\beta_j|}{b}\right),$$

the posterior mode has lasso form after matching normalisations. A continuous Laplace prior assigns zero probability to the event  $\beta_j = 0$  exactly; a zero MAP is not posterior proof of absence.

## 16.9 Paths and the one-standard-error rule

A validation minimum may be broad and noisy. The one-standard-error rule selects the most regularised candidate whose estimated validation loss is within one estimated standard error of the minimum.

It encodes a simplicity/stability preference under near-ties; it is not a theorem of optimality. Temporal folds are dependent and heterogeneous, so report year scores and describe any standard-error approximation.

## 16.10 Post-selection inference warning

Selecting three variables from forty with lasso, then fitting ordinary OLS on those three using the same data, makes classical intervals too optimistic because they pretend selection never occurred.

Options:

- focus on honestly held-out prediction;
- confirm on a new sample;
- use valid selective/debiased methods with assumptions; or
- pre-specify a low-dimensional explanatory model.

Regularisation solves fitting and prediction problems, not inference after searching.

## 16.11 Research discussion: Tibshirani and LARS

Tibshirani's lasso combines ridge-like shrinkage with subset-like sparsity through an  $L_1$  constraint. Read its motivation, geometry, evidence, original computation, and causal/inferential boundaries.

Efron and colleagues later introduced least-angle regression (LARS), efficiently tracing a lasso path under suitable conditions. Coordinate descent and LARS illuminate different computational geometry.

Replication: generate correlated feature pairs, repeat samples, and compare lasso versus elastic net on MAE, active-set size, selection frequency, and correlation of predictions across fits.

## Day 16 exit check

Derive soft thresholding, explain the diamond geometry, and explain why prediction stability and variable-selection stability are different.

# Day 17 — Multicollinearity, leverage, and influence

## 17.1 Four diagnostic questions

| Idea              | Question                                              |
|-------------------|-------------------------------------------------------|
| Multicollinearity | Are predictor directions weakly distinguished?        |
| Target outlier    | Is the outcome surprising under this model?           |
| Leverage          | Is the predictor combination unusual?                 |
| Influence         | Would removing this row substantially change the fit? |

A row can have high leverage with a small residual, or a large residual with ordinary leverage. Influence usually combines discrepancy and leverage. Every label is relative to a particular fitted design.

## 17.2 Consequences of collinearity depend on purpose

- Exact collinearity: rank deficiency and non-unique OLS coefficients.
- Near collinearity: unique but unstable coefficients.
- In-range prediction: combined fitted values may remain stable.
- Interpretation: conditional slopes are uncertain.
- Extrapolation: cancellation between large slopes may fail.

- Computation: rounding errors can be amplified.

Under the classical model, collinearity increases variance; it does not itself bias OLS.

## 17.3 Variance inflation factor

Regress feature  $x_j$  on other predictors and obtain auxiliary  $R_j^2$ . Then:

$$VIF_j = \frac{1}{1 - R_j^2}.$$

Slope variance can be written:

$$\text{Var}(\hat{\beta}_j | X) = \frac{\sigma^2}{\sum_i (x_{ij} - \bar{x}_j)^2 (1 - R_j^2)}.$$

Relative to a corresponding orthogonal reference, variance is inflated by VIF and standard error by  $\sqrt{VIF_j}$ . VIF 9 means threefold standard-error inflation, not “90% wrong.”

There is no universal cutoff. High VIF is structurally expected with polynomials and interactions and may affect explanation more than interpolation. Do not remove a necessary confounder just to lower it.

Calculate VIF on the actual transformed design, not merely raw inputs. Singular-value diagnostics reveal multi-feature weak directions that individual VIFs may summarise incompletely.

## 17.4 Hat matrix and leverage

$$\hat{y} = X(X^T X)^{-1} X^T y = Hy,$$

$$H = X(X^T X)^{-1} X^T.$$

Row  $i$  leverage:

$$h_{ii} = x_i^T (X^T X)^{-1} x_i.$$

For a full-rank design with  $k$  columns:

$$0 \leq h_{ii} \leq 1, \quad \sum_i h_{ii} = \text{tr}(H) = k.$$

Average leverage is  $k/n$ . Rules like  $2k/n$  and  $3k/n$  are screening conventions, not automatic exclusions. Rare deployment-relevant projects may legitimately have high leverage.

## 17.5 Studentised residuals

Since  $e = (I - H)\epsilon$  under the model:

$$\text{Var}(e_i | X) = \sigma^2(1 - h_{ii}).$$

Internally studentised residual:

$$r_i = \frac{e_i}{s\sqrt{1 - h_{ii}}}, \quad s^2 = \frac{\sum_i e_i^2}{n - k}.$$

It adjusts raw residuals for their model-implied variance. External studentisation estimates variance after deleting row  $i$  and is useful in formal outlier tests, which also require multiple-testing care.

## 17.6 Cook's distance

$$D_i = \frac{e_i^2}{ks^2} \frac{h_{ii}}{(1 - h_{ii})^2}.$$

It is proportional to aggregate fitted-value change after deleting row  $i$ :

$$D_i = \frac{\sum_m (\hat{y}_m - \hat{y}_{m(i)})^2}{ks^2}.$$

$D_i > 4/n$  is an investigation flag, not a deletion rule.

## 17.7 Read the diagnostic code

The educational function computes:

1. OLS coefficients and fitted values;
2. residuals;
3. leverage with row-wise quadratic forms;
4. residual MSE;
5. studentised residuals; and
6. Cook's distance.

Use `statsmodels.get_influence()` as a reference. The explicit inverse mirrors formulas; production implementations should use stable factorizations.

## 17.8 Diagnostic quartet

1. Residual versus fitted: curvature, spread, groups.
2. Normal Q–Q: tail departure when normal inference matters.
3. Scale–location:  $\sqrt{|r_i|}$  versus fitted for changing variance.
4. Residual versus leverage with Cook contours: influential combinations.

Plots suggest mechanisms but do not select remedies. A funnel may reflect multiplicative noise, omitted groups, wrong mean structure, or data errors.

## 17.9 Influence investigation protocol

1. Verify source, units, duplicate status, and transcription.
2. Confirm target-population and prediction-time eligibility.
3. Understand why predictors are rare or outcome surprising.
4. compare pre-justified specifications.
5. report sensitivity of coefficients, validation, and conclusions.
6. preserve valid in-scope rows in the primary analysis.

Document any data-error deletion rule so it can be applied without viewing a desired result.

## 17.10 Regularised influence differs

OLS formulas apply to unpenalised OLS. Ridge uses  $H_\lambda$  and effective degrees of freedom. Lasso changes active set nonlinearly, so copying OLS Cook formulas is unsafe. Assess complex selected pipelines through case-deletion refits or carefully designed resampling.

## Day 17 exit check

Distinguish residual, leverage, and influence; derive average leverage; and explain VIF as conditional variance inflation rather than a variable-deletion command.

---

# Day 18 — Heteroskedasticity and dependent data

---

## 18.1 Coefficients and uncertainty are separate layers

If the conditional mean is correctly linear and:

$$\mathbb{E}[\varepsilon \mid X] = 0,$$

OLS can remain unbiased even when error variance changes.

Under homoskedastic independent errors:

$$\text{Var}(\varepsilon \mid X) = \sigma^2 I,$$

$$\text{Var}(\hat{\beta} \mid X) = \sigma^2 (X^T X)^{-1}.$$

When variance or dependence differs, OLS coefficients stay the same but covariance, standard errors, tests, and intervals change.

## 18.2 General sandwich covariance

For general error covariance  $\Omega$ :

$$\text{Var}(\hat{\beta} \mid X) = (X^T X)^{-1} X^T \Omega X (X^T X)^{-1}.$$

Outer matrices are bread; middle is meat. For independent heteroskedastic observations,  $\Omega$  is diagonal with different variances.

## 18.3 HC0 through HC3

HC0:

$$\widehat{\text{Var}}_{HC0}(\hat{\beta}) = (X^T X)^{-1} \left( \sum_i e_i^2 x_i x_i^T \right) (X^T X)^{-1}.$$

| Estimator | Residual contribution    | Purpose                                |
|-----------|--------------------------|----------------------------------------|
| HC0       | $e_i^2$                  | Basic asymptotic sandwich              |
| HC1       | $\frac{n}{n-k} e_i^2$    | Degrees-of-freedom scaling             |
| HC2       | $e_i^2 / (1 - h_{ii})$   | Leverage-related residual shrinkage    |
| HC3       | $e_i^2 / (1 - h_{ii})^2$ | Stronger leave-one-out-like correction |

HC3 is often reasonable in modest samples, not universally guaranteed.

## 18.4 Build and verify HC covariance

The chapter function:

1. computes bread  $(X^T X)^{-1}$ ;
2. fits OLS and residuals;
3. calculates leverage;
4. chooses the HC residual weights;
5. computes `X.T @ (omega[:, None] * X)` as meat; and
6. sandwiches the result.

Compare coefficients and covariance with `statsmodels.OLS(...).fit(cov_type="HC3")`.

## 18.5 What HC standard errors do not fix

They do not repair:

- wrong conditional mean;
- omitted variables or endogeneity;
- leakage;
- measurement-error bias;
- unsupported causal contrasts;
- row dependence when using independent-row HC; or
- out-of-sample prediction.

“Robust” always means robust to a stated violation while retaining other assumptions.

## 18.6 Heteroskedasticity tests

Breusch–Pagan relates squared residuals to predictors. White's test allows a richer variance auxiliary model.

- small samples may lack power;
- huge samples flag trivial departures;
- rejection may reflect mean misspecification; and
- non-rejection does not prove equal variance.

Use design knowledge and plots. If changing variance is expected by construction, pre-specifying HC3 can be more coherent than choosing covariance after a significance test.

## 18.7 Cluster dependence

For cluster  $g$ :

$$\mathbf{meat} = \sum_{g=1}^G (X_g^T e_g)(X_g^T e_g)^T.$$

Cluster-robust covariance permits arbitrary within-cluster covariance while relying on approximate independence across clusters. The number of independent clusters, not row count, drives asymptotic justification.

## 18.8 Five districts are too few for casual asymptotics

Five clusters can produce a software number without reliable reference distribution. Possible responses:

- gather more independent districts;
- state a hierarchical model explicitly;
- use justified small-cluster methods such as wild cluster bootstrap;
- use randomisation inference when design permits; or
- limit claims and show district sensitivity descriptively.

No technique converts five independent shocks into fifty.

## 18.9 Choose cluster level from dependence

- shared district shocks: district;
- repeated rows per project: project;
- village-level treatment: assignment unit constrains inference;
- dependence across district and year: one-way clustering may be insufficient.

Do not choose the level that makes significance attractive. Time and spatial dependence need methods with their own bandwidth and asymptotic assumptions.

## 18.10 Prediction intervals need a variance model

HC covariance improves uncertainty of estimated mean coefficients. It does not automatically make project-level interval widths adapt to remoteness.

Options include conditional scale models, quantile regression, conformal methods under stated exchangeability/shift assumptions, or hierarchical probability models.

## 18.11 Research discussion: White

White developed consistent OLS covariance estimation under unknown heteroskedasticity subject to regularity conditions.

Read:

- target asymptotic distribution;
- failure of homoskedastic covariance;
- squared-residual construction;
- sensitivity to sample size and leverage; and
- assumptions that remain beyond variance form.



Replication: under variance  $(0.5 + |x|)^2$ , compare classical and HC3 95% slope-interval coverage across repeated samples. Then omit a true quadratic term and observe that HC3 does not repair mean bias.

## Day 18 exit check

Construct the sandwich, explain HC3's leverage correction, and explain why few clusters are a design limitation.

# Day 19 — Robust and quantile regression

## 19.1 Three meanings of “robust”

| Method              | Fitted target/loss          | Changed layer        |
|---------------------|-----------------------------|----------------------|
| OLS + HC3           | OLS mean/squared loss       | covariance only      |
| Huber regression    | robust conditional location | coefficient fit/loss |
| Quantile regression | chosen conditional quantile | target and loss      |

They answer different questions.

## 19.2 Why squared loss reacts strongly

OLS uses  $\frac{1}{2}r^2$ , whose derivative with respect to residual is  $r$ . Residual magnitude 20 contributes twenty times the local gradient magnitude of residual 1.

Large residuals may be errors, omitted mechanisms, heavy tails, mixed regimes, rare genuine events, or out-of-scope rows. Changing loss reduces sensitivity but does not diagnose the cause.

## 19.3 Huber loss and score

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta(|r| - \frac{1}{2}\delta), & |r| > \delta. \end{cases}$$

Derivative:

$$\psi_{\delta}(r) = \begin{cases} r, & |r| \leq \delta, \\ \delta \text{sign}(r), & |r| > \delta. \end{cases}$$

Large gradients are clipped. Huber remains smooth at  $\pm\delta$  and is less sensitive to vertical outliers.

## 19.4 Iteratively reweighted least squares intuition

For  $r \neq 0$ :

$$w(r) = \frac{\psi_{\delta}(r)}{r} = \begin{cases} 1, & |r| \leq \delta, \\ \delta/|r|, & |r| > \delta. \end{cases}$$

Iterate:

1. fit current coefficients;
2. calculate residuals and weights;
3. solve weighted least squares; and
4. repeat.

Production Huber regression accounts for scale because a numerical threshold changes meaning when target units change.

## 19.5 Huber is not robust to every leverage point

A predictor-space outlier can pull the fitted surface toward itself and end with a modest residual. Residual clipping alone may not protect against it. Combine Huber with leverage, case deletion, provenance, and deployment-population checks.

In scikit-learn, `epsilon` sets the scaled-residual transition and `alpha` adds  $L_2$  regularisation. Both tune inside validation. Smaller epsilon is not automatically better.

## 19.6 Conditional quantiles

OLS targets:

$$\mathbb{E}[Y \mid X = x].$$

Conditional quantile:

$$Q_Y(\tau \mid x) = \inf\{q : P(Y \leq q \mid X = x) \geq \tau\}.$$

- 0.5: conditional median.
- 0.8: upper planning quantile.
- 0.95: more conservative upper quantile.

A quantile is not a confidence interval for a mean.

## 19.7 Derive pinball loss

For residual  $u = y - q$ :

$$\rho_\tau(u) = u(\tau - \mathbb{1}\{u < 0\}) = \begin{cases} \tau u, & u \geq 0, \\ (\tau - 1)u, & u < 0. \end{cases}$$

At  $\tau = 0.8$ , underprediction costs 0.8 per unit and overprediction 0.2: four times as much. Expected risk derivative is:

$$R'(q) = F_Y(q) - \tau.$$

The optimum satisfies the quantile condition. At jumps, a subgradient produces the set-valued definition.

## 19.8 Linear quantile regression

Assume:

$$Q_Y(\tau \mid X = x) = x^T \beta_\tau.$$

Fit:

$$\hat{\beta}_\tau = \arg \min_{\beta} \sum_i \rho_\tau(y_i - x_i^T \beta),$$

possibly with regularisation. Coefficients can differ across quantiles because spread and distribution shape change with features.

Scikit-learn `QuantileRegressor.alpha` controls  $L_1$  regularisation, so scaling and validation remain necessary.

## 19.9 Evaluate quantile predictions

Use mean pinball loss at the target quantile. Check held-out empirical coverage:

$$\frac{1}{m} \sum_i \mathbb{1}\{y_i \leq \hat{q}_\tau(x_i)\}.$$

For a calibrated stable 0.8 conditional-quantile model, overall coverage should be near 0.8. Also report coverage by pre-specified group and predicted-quantile bins with counts and uncertainty.

Coverage alone is insufficient: an enormous upper prediction covers everything but is useless. Pinball loss balances calibration and sharpness according to the target.

## 19.10 Quantile crossing

Separately fitted models can yield:

$$\hat{Q}(0.5 | x) > \hat{Q}(0.8 | x).$$

This violates ordering. Check within deployment-like ranges. Remedies include joint non-crossing fitting, full-distribution models, validated rearrangement, or simpler representation. Sorting predictions post hoc changes the procedure and must be validated.

## 19.11 Decision basis for an upper quantile

If underprediction costs `$c_u` per unit and overprediction costs `$c_o`:

$$\tau = \frac{c_u}{c_u + c_o}.$$

When underbudgeting costs four times as much,  $\tau = 0.8$ . This is a decision-based choice made before outcomes, not a quantile chosen for attractive test coverage.

## 19.12 Research discussions: Huber and Koenker–Bassett

Huber's robust-location work formalised an efficiency-versus-contamination trade-off, explaining linear central score and clipped tails. Its location setting does not automatically solve regression leverage.

Replication: contaminate 5% of normal observations with a much wider distribution and compare mean, median, and Huber location across repeats.

Koenker and Bassett generalised sample quantiles to regression through asymmetric absolute loss, allowing conditional effects at different distribution points.

Replication: simulate heteroskedastic  $Y = 2 + 3X + (0.5 + X)\epsilon$ , fit 0.1, 0.5, and 0.9 regressions, compare theoretical slopes, and check held-out coverage.

## Day 19 exit check

Distinguish HC3, Huber, and quantile regression; derive Huber and pinball losses; and evaluate an upper quantile overall and by subgroup.

# Day 20 — A pre-specified regression benchmark

## 20.1 Exploration, confirmation, and benchmarks

- **Exploratory:** choices adapt to observed results; findings generate hypotheses.
- **Confirmatory:** primary question and analysis are fixed in advance; deviations recorded.
- **Predictive benchmark:** candidate procedures and selection rule are fixed; locked test evaluates the selected result.

Exploration is essential. Mislabeling an adaptive result as pre-specified is the problem.

## 20.2 Registered prediction contract

Freeze:

| Component         | Choice                                    |
|-------------------|-------------------------------------------|
| Unit              | One eligible MHP project                  |
| Prediction time   | Technical appraisal                       |
| Target            | Final constant-2025 cost                  |
| Development       | Start through 2022                        |
| Locked test       | 2023–2025                                 |
| Validation        | Expanding years 2019–2022                 |
| Point metric      | Equal-year-average MAE                    |
| Secondary metrics | RMSE, signed error, year spread           |
| Planning target   | Conditional 80th percentile               |
| Planning metric   | 0.8 pinball loss; coverage diagnostic     |
| Subgroups         | Access mode and district with counts      |
| Exclusions        | Documented eligibility/source errors only |
| Test use          | Once after selection                      |

Equal year weighting gives each year equal influence. Project weighting answers a different question. Either can be justified if fixed in advance.

## 20.3 Compare complete procedures

A candidate includes:

1. imputation and indicators;
2. categorical encoding;
3. centring and engineered features;
4. standardisation;
5. estimator and loss;
6. penalty and other hyperparameters; and
7. convergence settings.

The chapter pre-specifies historical mean, raw and engineered OLS, ridge grid, elastic-net grid, Huber grid, and median/0.8 quantile grids.

Point candidates need not estimate the same scientific functional: squared loss targets mean, MAE's population optimum is median, and Huber targets a robust location. Comparing them under operational MAE is legitimate for a point decision, but do not relabel an MAE-selected robust location as expected cost.

## 20.4 Temporal boundary

For validation year  $t$ , fit every learned transformation and estimator only on years before  $t$ . Aggregate the pre-specified year losses, select one procedure, refit through 2022, and test on 2023–2025 once.

Feature design, influence decisions, and metric choice cannot use locked-test information.

## 20.5 Read the end-to-end code

### Constants and feature lists

They make target, validation years, test boundary, seed, eligible raw features, engineered features, and categories explicit. Assertions prohibit latent road distance and post-construction material bill.

### Feature engineer

Learns capacity/year centres per fold and creates capacity square, time centre, road hinge, cable-road interaction, and log experience.

### Candidate factories

Return fresh complete pipelines for point and quantile procedures. A modest theory-guided grid controls selection opportunity.

### Expanding-window scoring

Clones and refits every candidate for every year. It stores candidate, year, validation count, and loss rather than keeping only an average.

### Selection

Ranks candidates by equal-year mean and worst-year tie breaker. A broad practical tie should be reported rather than over-precisely ranked.

## Locked test

Refits winner on all development rows, predicts test once, and compares with pre-specified raw OLS reference.

## Quantile track

Selects median and 80th-quantile procedures separately by matching pinball loss, then records coverage and crossing.

## Outputs

Writes validation fold scores, summaries, row-level locked predictions, subgroup audit, and JSON metadata containing environment and selections.

Assertions protect only their stated contract. They do not replace a full audit.

## 20.6 Read validation before winner

Ask:

- Is the lead consistent or one-year-driven?
- Is it operationally meaningful?
- Does raw OLS nearly tie?
- Did candidates converge?
- Does the tie rule prefer a stable simpler model?

Minimum validation loss after a search is optimistic as a performance estimate. The locked test evaluates the chosen procedure.

## 20.7 Interpret the locked test narrowly

Report:

- date ranges and sample sizes;
- full candidate families and selected procedure;
- MAE, RMSE, signed error;
- comparison with raw OLS;
- median and 0.8 pinball loss;
- overall/subgroup 0.8 coverage;
- crossing rate;
- shift evidence; and
- plan deviations.

It estimates performance in the represented later period, not permanent universal superiority.

## 20.8 Paired bootstrap difference

$$\Delta = MAE_{selected} - MAE_{raw\ OLS}.$$

Negative favours selected. Resample projects as paired rows because both models predict the same cases. The supplied row-bootstrap interval is descriptive and assumes row exchangeability; it ignores district/time dependence. Show year/district results and avoid unjustified precision.

## 20.9 Subgroup audit without leaderboard fishing

Pre-specify groups for operational reasons. Report counts, errors, quantile coverage, and crossing. Do not search many slices until one looks dramatic, then present it as confirmed. Post-hoc findings become hypotheses for new evidence.

## 20.10 Model card and reproducibility

Model card should state:

- purpose and prohibited uses;
- prediction time and target units;
- training period/population;
- features and missingness policy;
- candidate selection and metrics;
- overall and subgroup performance;
- quantile calibration;
- limitations and monitoring;
- human override and accountability.

Reproducibility checklist includes source code, seed, versions, data generator/input hash, frozen contract, candidate grid, all fold scores, row predictions, metadata, and deviation log.

Reproducibility means another researcher can repeat the procedure. It does not prove the procedure is scientifically correct.

## 20.11 Research discussion: Varma and Simon

Their work demonstrates selection bias when cross-validation both tunes and reports performance. Nested CV or a locked independent evaluation separates tuning from assessment.

Replication: increase candidate settings from 5 to 50 to 500 and compare best inner score with outer/independent performance. Selection optimism should grow with search opportunity.

## 20.12 Research report structure

1. Question and deployment decision.
2. Data and eligibility.
3. Registered contract and deviations.
4. Complete candidate procedures.
5. Validation design and selection rule.
6. Locked results, diagnostics, influence, convergence.
7. Pre-specified sensitivity and labeled post-hoc analysis.
8. Limitations: sampling, dependence, shift, causality, synthetic status.
9. Reproducibility artifacts.
10. Bounded conclusion.

Conclusion template:

Under expanding-window comparison through 2022, procedure A had the lowest pre-specified average validation MAE. After refitting on all development projects, its 2023–2025 MAE was X million PKR versus Y for raw OLS. Performance was weakest for subgroup Z based on n projects. The 80th-quantile procedure covered C% overall and C-Z% in subgroup Z. Results support appraisal prediction within represented districts and ranges; they do not identify causal effects or establish transfer to new regions.

Fill every placeholder and retain unfavourable results.

## Day 20 exit check

You are finished when another researcher can trace every number from frozen contract through pipeline, fold scores, selected procedure, locked predictions, subgroup audit, metadata, and limitations.

# Chapter 3 synthesis and laboratories

## How the techniques form one system

- Preprocessing changes information received.
- Feature engineering changes expressible relationships.
- Regularisation changes fitting criterion.
- Diagnostics reveal weak directions and dominating cases.
- Robust covariance changes uncertainty, not coefficients.
- Huber changes loss and fitted location.
- Quantile regression changes conditional target.
- Validation evaluates the whole procedure.
- Locked evidence bounds the final claim.

## Derivation workshop

Complete on paper and verify numerically:

1. ridge normal equations;
2. ridge SVD coefficient and fitted-value factors;
3. positive definiteness of  $\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$ ;
4. lasso soft thresholding under  $\mathbf{X}^T \mathbf{X} / n = \mathbf{I}$ ;
5.  $\sum_i h_{ii} = k$ ;
6.  $\text{Var}(\mathbf{e}_i | \mathbf{X}) = \sigma^2(1 - h_{ii})$ ;
7. sandwich covariance from the OLS coefficient expression;
8. pinball expected-risk derivative;
9. cost ratio leading to  $\tau = \mathbf{c}_u / (\mathbf{c}_u + \mathbf{c}_o)$ ; and
10. slope when a predictor appears in a square and interaction.



# Practical laboratories

## A — Missingness as information

Compare MCAR and remote-dependent missingness using complete cases, median, median-plus-indicator, and multiple imputation. Separate prediction error from inferential coverage.

## B — Regularisation under correlation

Vary predictor correlation from 0 to 0.999. Across repeats report OLS/ridge coefficient variance and bias, lasso selection frequency, and elastic-net prediction.

## C — Influence map

Create ordinary, vertical-outlier, high-leverage-on-line, and high-leverage-large-residual rows. Compare residual, leverage, and Cook distance.

## D — Covariance failure

Compare classical, HC0, and HC3 coverage under equal/changing variance and several leverage patterns.

## E — Dependence

Increase projects per district while fixing district count. Observe why row count does not replace independent clusters.

## F — Robust target choice

Contaminate outcomes and compare OLS, Huber, median, and ridge under mean- and median-target losses.

## G — Quantile calibration

Fit median and 0.8 quantiles under equal/changing spread. Check loss, overall/group coverage, and crossing.

## H — Selection optimism

Compare 5, 50, and 500 candidate searches using inner versus outer performance.

# Self-assessment rubric

| Dimension           | Ready for this level when you can...                                                  |
|---------------------|---------------------------------------------------------------------------------------|
| Prediction contract | make population, decision, timing, target, units, and exclusions executable           |
| Preprocessing       | justify missingness assumptions, category policy, unknown levels, and fold boundaries |
| Feature design      | state mechanism, hierarchy, derivative, support, units, and extrapolation limits      |
| Regularisation      | derive objectives, explain paths and scaling, and tune without test access            |
| Diagnostics         | separate collinearity, residual size, leverage, and influence and audit provenance    |
| Uncertainty         | justify classical, HC, or clustered covariance and state remaining assumptions        |
| Robust targets      | distinguish mean, robust location, median, and upper quantile and use matching losses |
| Validation          | preserve temporal selection, locked test, search budget, tie rule, and adaptation log |

| Dimension       | Ready for this level when you can...                                                       |
|-----------------|--------------------------------------------------------------------------------------------|
| Reporting       | retain predictions, failed candidates, subgroup counts, uncertainty limits, and deviations |
| Reproducibility | enable another researcher to trace every claim to code and saved output                    |

## Formula sheet with plain-language readings

| Concept                    | Formula                                                      | Meaning                                             |
|----------------------------|--------------------------------------------------------------|-----------------------------------------------------|
| Missing observed indicator | $R_{ij} \in 0, 1$                                            | Whether a value is observed                         |
| Missing indicator          | $M_{ij} = 1 - R_{ij}$                                        | Whether imputation occurred                         |
| Rubin pooled estimate      | $\bar{Q} = m^{-1} \sum_l \hat{Q}_l$                          | Average across imputations                          |
| Rubin pooled variance      | $T = \bar{U} + (1 + 1/m)B$                                   | Within plus missing-value uncertainty               |
| Smoothed target encoding   | $(n_c \bar{y}_c + m \bar{y}) / (n_c + m)$                    | Shrink rare category means                          |
| Quadratic slope            | $\beta_1 + 2\beta_2 x$                                       | Local change at $x$                                 |
| Turning point              | $-\beta_1 / (2\beta_2)$                                      | Zero local quadratic slope                          |
| Interaction slope          | $\beta_1 + \beta_3 x_2$                                      | Effect of $x_1$ conditional on $x_2$                |
| Hinge                      | $(x - k)_+ = \max(0, x - k)$                                 | Add a slope change at knot $k$                      |
| Standardisation            | $(x_{ij} - \bar{x}_j) / s_j$                                 | Define comparable penalty scale                     |
| Ridge objective            | $\ y - X\beta\ ^2 + \lambda \ \beta\ _2^2$                   | Fit plus squared shrinkage                          |
| Ridge estimate             | $(X^T X + \lambda I)^{-1} X^T y$                             | Closed-form centred slopes                          |
| Ridge retention            | $\sigma_j^2 / (\sigma_j^2 + \lambda)$                        | Fitted-value signal retained in direction $j$       |
| Ridge effective df         | $\text{tr}(H_\lambda)$                                       | Smooth flexibility                                  |
| Lasso objective            | $\ y - X\beta\ ^2 / (2n) + \alpha \ \beta\ _1$               | Fit plus sparse shrinkage                           |
| Soft threshold             | $\text{sign}(z) \max( z  - \alpha, 0)$                       | $z$                                                 |
| Elastic net                | $\text{Loss} + \alpha \rho L_1 + \alpha(1 - \rho) L_2^2 / 2$ | Sparsity plus correlated-feature stability          |
| VIF                        | $1 / (1 - R_j^2)$                                            | Slope-variance inflation                            |
| Hat matrix                 | $X(X^T X)^{-1} X^T$                                          | Maps outcomes to OLS fits                           |
| Average leverage           | $k/n$                                                        | Mean design leverage                                |
| Studentised residual       | $e_i / [s \sqrt{1 - h_{ii}}]$                                | Variance-adjusted residual                          |
| Cook distance              | $e_i^2 h_{ii} / [k s^2 (1 - h_{ii})^2]$                      | Case influence on fitted values                     |
| General sandwich           | $(X^T X)^{-1} X^T \Omega X (X^T X)^{-1}$                     | Coefficient covariance for general error covariance |
| HC0 meat                   | $\sum_i e_i^2 x_i x_i^T$                                     | Independent heteroskedastic residual contribution   |
| HC3 residual square        | $e_i^2 / (1 - h_{ii})^2$                                     | Leverage-corrected contribution                     |

| Concept              | Formula                                              | Meaning                                    |
|----------------------|------------------------------------------------------|--------------------------------------------|
| Cluster meat         | $\sum_g (X_g^T e_g)(X_g^T e_g)^T$                    | Arbitrary within-cluster covariance        |
| Huber loss           | Quadratic centre, linear tails                       | Clip large-residual gradients              |
| Conditional quantile | $\inf q : P(Y \leq q \mid X = x) \geq \tau$          | Outcome distribution point at level $\tau$ |
| Pinball loss         | $u(\tau - \mathbb{1}u < 0)$                          | Proper asymmetric quantile loss            |
| Quantile coverage    | $m^{-1} \sum_i \mathbb{1}y_i \leq \hat{q}_\tau(x_i)$ | Held-out calibration diagnostic            |

Penalty values depend on loss normalisation. Never copy `alpha` or  $\lambda$  across implementations without checking the exact objective.

## Expanded glossary

**Active set:** Features with nonzero lasso or elastic-net coefficients at one tuning value and fit.

**Bias–variance trade-off:** Deliberate estimator bias may reduce sample sensitivity enough to improve prediction.

**Complete-case analysis:** Restrict analysis to rows with every required value observed.

**Conditional quantile:** Quantile of outcome distribution for a specified feature vector.

**Cook's distance:** Scaled aggregate fitted-value change caused by deleting one observation.

**Cross-fitting:** Build each training-row representation using a fit that excluded its fold.

**Elastic net:** Regression combining  $L_1$  and squared  $L_2$  penalties.

**Effective degrees of freedom:** Smooth flexibility measure such as  $\text{tr}(H_\lambda)$ .

**Frequency encoding:** Replace category with its training frequency.

**Hat matrix:** Linear map from outcomes to OLS fitted outcomes.

**HC covariance:** Heteroskedasticity-consistent coefficient covariance estimator.

**Heteroskedasticity:** Conditional error variance changes across observations.

**Hierarchy principle:** Retain lower-order terms defining polynomial or interaction effects.

**Huber loss:** Quadratic near zero and linear for large absolute residuals.

**Influence:** Sensitivity of a fitted result to an observation's presence.

**Leverage:** Unusualness of a predictor combination relative to the design.

**Lasso:** Least squares with an  $L_1$  penalty capable of exact zeros.

**MAP:** Posterior-mode estimate under a Bayesian model.

**MAR:** Missingness no longer depends on missing values after conditioning on observed information.

**MCAR:** Missingness independent of observed and missing data under the model.

**MNAR:** Missingness still depends on unobserved information after conditioning.

**Multiple imputation:** Several plausible completions used to propagate missing-value uncertainty.

**One-hot encoding:** Binary columns representing nominal categories.

**Pinball loss:** Asymmetric absolute loss whose population minimiser is a chosen quantile.

**Post-selection inference:** Inference accounting for data-driven model or hypothesis selection.

**Quantile crossing:** Predicted lower quantile exceeds a higher quantile at the same features.

**Regularisation:** Penalty or constraint controlling flexibility or encoding prior preference.

**Ridge:** Least squares with squared  $L_2$  penalty.

**Sandwich covariance:** Bread matrices surrounding estimated score/error covariance meat.

**Shrinkage:** Pull coefficients toward a reference, usually zero.

**Soft thresholding:** Reduce magnitude by a threshold and set smaller values to zero.

**Spline:** Curve represented by weighted local basis functions.

**Studentised residual:** Residual divided by its estimated model-based standard deviation.

**Target encoding:** Category represented by smoothed, cross-fitted outcome summaries.

**VIF:** Conditional slope-variance inflation relative to an orthogonal reference.

---

## Research-paper reading ladder

---

| Paper                    | Problem                                       | Replication focus                           |
|--------------------------|-----------------------------------------------|---------------------------------------------|
| Rubin (1976)             | Missing-data mechanisms and ignorability      | Prediction and coverage under MCAR/MAR/MNAR |
| Hoerl & Kennard (1970)   | Biased shrinkage for nonorthogonal regression | Stability under collinearity                |
| Tibshirani (1996)        | $L_1$ shrinkage and selection                 | Sparse prediction and unstable selection    |
| Efron et al. (2004)      | Efficient geometric lasso paths               | LARS versus coordinate paths                |
| White (1980)             | Heteroskedasticity-consistent covariance      | Interval coverage under changing variance   |
| Huber (1964)             | Robust location under contamination           | Efficiency versus contamination             |
| Koenker & Bassett (1978) | Regression quantiles                          | Conditional spread and coverage             |
| Varma & Simon (2006)     | Selection bias in CV estimates                | Nested versus non-nested evaluation         |

Six reading passes:

1. record citation, question, venue, and vocabulary;
2. scan abstract, introduction, figures, and conclusion;
3. build a symbol table;
4. reproduce one central derivation;
5. reconstruct one simulation or result; and

6. list assumptions, failure modes, and transfer limits.

Reading means reconstructing an argument well enough to identify where it could fail.

---

## Suggested eight-day study rhythm

---

| Segment                         | Minutes    |
|---------------------------------|------------|
| Retrieve previous exit check    | 15         |
| Construct concept and intuition | 35         |
| Mathematical derivation         | 40         |
| Code laboratory                 | 55         |
| Failure laboratory              | 25         |
| Research reading                | 20         |
| Bounded reflection              | 10         |
| <b>Total</b>                    | <b>200</b> |

Regularisation and covariance days may require two sessions. Slow reconstruction is preferable to treating formulas as incantations.

---

## Common beginner misconceptions

---

| Misconception                                    | Correction                                                   |
|--------------------------------------------------|--------------------------------------------------------------|
| Imputation neutrally fills blanks                | It is a learned representation with assumptions              |
| MAR means no reason for missingness              | MAR allows dependence on observed information                |
| Smoothing target encoding prevents leakage       | Cross-fitting is still required                              |
| Polynomial regression is nonlinear estimation    | It is linear in coefficients after feature expansion         |
| Negative quadratic coefficient proves an optimum | Derivative, support, uncertainty, and causality still matter |
| Scaling makes raw features important             | It defines conditioning and penalty units                    |
| Ridge discovers the true correlated feature      | It stabilises a combined direction, not causal identity      |
| Lasso zero proves no relationship                | Zero is sample-, penalty-, and representation-dependent      |
| VIF above 5 requires deletion                    | Cutoffs are conventions; respond to consequences             |
| Large residual means high leverage               | Residual concerns outcome; leverage concerns predictors      |
| Cook flag means delete                           | It triggers provenance and sensitivity investigation         |
| HC3 is robust regression                         | HC3 changes covariance, not fitted OLS coefficients          |

| Misconception                                     | Correction                                        |
|---------------------------------------------------|---------------------------------------------------|
| Many rows fix five-cluster inference              | Independent cluster count remains five            |
| Quantile is a confidence interval                 | It is a conditional outcome-distribution target   |
| Global 80% coverage proves calibration everywhere | Subgroups can severely under-cover                |
| Reproducible means scientifically correct         | Reproducibility repeats both good and bad designs |

## Ethical and contextual questions

The KP setting is fictional and cannot evaluate actual districts. A real study requires local technical, financial, administrative, and community knowledge.

Ask:

- Who bears harm when remote cost is underpredicted?
- Does historical underinvestment become encoded as a district effect?
- Are appraisal fields measured consistently across locations?
- Is an access mode “rare” because data collection excluded it?
- Are missing surveys evidence of structural access barriers?
- Could contractor or district coefficients create unfair rankings?
- Are extreme projects genuine obligations rather than disposable outliers?
- Who may override the model, for which reasons, and with what record?
- Which groups need minimum calibration or error safeguards?
- Would deployment redirect resources away from poorly represented communities?

Technical competence without contextual accountability is not research readiness.

## Where Chapter 4 begins

Chapter 4 can introduce classification and generalised linear models while reusing everything learned here:

- Bernoulli likelihood, probability, odds, and log odds;
- logistic regression gradients and convex optimisation;
- regularised classification;
- calibration and proper scoring rules;
- decision thresholds and class imbalance;
- precision–recall analysis;
- subgroup error and fairness definitions;
- nonlinear tree benchmarks;
- external/prospective validation; and
- a locked classification study.

The practitioner does not leave linear-model thinking behind. Design matrices, interactions, penalties, influence, temporal validation, and post-selection caution all remain.

Research readiness begins when another person can inspect the complete argument, reproduce every number, identify every assumption, and understand exactly how strongly the evidence supports the final decision.

# Chapter 4 — From Probabilities to Accountable Decisions

---

## Level 4 Advanced Modeler: nine days of classification, ensembles, explanation, and external scrutiny

**Central promise.** Chapter 3 taught you to design and audit a research-grade regression study. By the end of Chapter 4, you will be able to define a binary outcome without leaking the future, derive logistic regression from a Bernoulli likelihood, distinguish probability estimation from label assignment, evaluate discrimination and calibration with proper metrics, choose a decision threshold from consequences, handle class imbalance without corrupting evaluation, construct nonlinear and ensemble classifiers, tune them inside a valid resampling design, interrogate their predictions without making causal claims, audit group-level harms, and complete a locked temporal classification study.

The learner is still treated as a beginner. Classification introduces familiar-looking words—positive, accuracy, risk, importance—that can conceal precise mathematical choices. We will therefore construct each idea before calling a library.

The advanced-modeler mindset is not:

“Use the model that wins the leaderboard.”

It is:

- define the event and observation window before fitting;
- estimate probabilities before turning them into actions;
- evaluate the probability, ranking, and decision layers separately;
- make consequences and capacity constraints explicit;
- treat tuning, calibration, and threshold selection as learned operations;
- explain the model that was actually fitted, on a stated output scale;
- report where errors and interventions fall across groups; and
- limit the claim to populations, periods, and mechanisms that were tested.

---

## Why the short advanced-regression draft needed a new role

The original draft contained useful introductions to splines, random forests, boosting, nearest neighbours, support-vector methods, tuning, and explanation. But it repeated continuous-outcome regression immediately after Chapter 3 and conflicted with Chapter 3's explicit handoff to classification.

It also compressed several technical distinctions into slogans.

1. **A classifier and a decision rule are separated.** A model can estimate a 0.23 event probability; a policy decides whether 0.23 triggers review.



2. **Accuracy is not a universal target.** It depends on prevalence and a threshold and can conceal costly minority-class failures.
3. **Gradient boosting classification does not simply fit ordinary residuals.** It follows negative gradients of a classification loss in function space.
4. **Tree support is implementation-specific.** Trees do not require scaling, but categorical and missing-value handling depend on the software and estimator.
5. **Calibration gets its own evidence boundary.** A calibrator trained on the same fitted probabilities it corrects can overfit.
6. **PDP does not literally hold every other feature fixed at one value.** It replaces a feature across rows and averages predictions, potentially creating implausible combinations.
7. **SHAP is not causal attribution.** Its baseline, background distribution, feature-dependence treatment, and output scale are part of the explanation.
8. **Fairness is not one metric.** Different definitions encode different moral and operational commitments and can conflict when base rates differ.

The chapter retains the draft's strongest advanced-modeling material, but uses it to answer the next coherent question: how do we estimate event risk and convert it into an accountable action?

## Prerequisite checkpoint

Before starting, retrieve these Chapter 2 and Chapter 3 ideas without notes:

- conditional expectation and residual;
- likelihood, log likelihood, gradient, and Hessian;
- train, validation, test, temporal CV, and nested CV;
- pipelines and leakage boundaries;
- ridge and lasso objectives;
- interactions, splines, and extrapolation;
- bootstrap uncertainty and subgroup evaluation;
- robust covariance versus robust fitting; and
- the difference between a prediction target, loss, metric, and scientific claim.

If the terms are recognisable but not explainable, return to the earlier exit checks. Classification reuses all of them.

## Learning outcomes

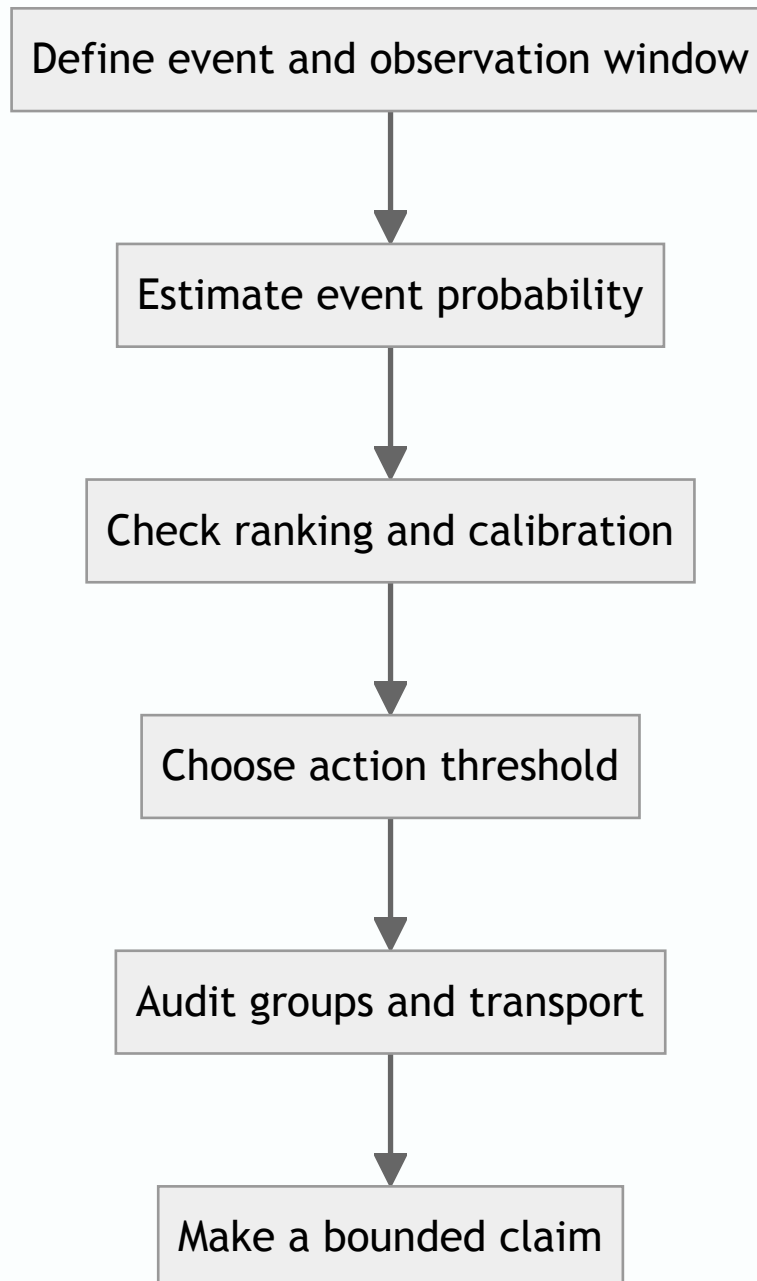
At the end of Chapter 4, you should be able to:

- define a binary event, index time, prediction time, observation window, and eligibility rule;
- explain why unknown or censored outcomes are not automatically negative labels;
- calculate prevalence and state why it is population- and time-dependent;
- move between probability, odds, and log odds;
- derive the Bernoulli likelihood and binary cross-entropy loss;
- derive the logistic-regression gradient and Hessian;
- implement regularised logistic regression with gradient descent;
- interpret a logistic coefficient as a conditional log-odds change and its exponential as an odds ratio;
- recognise complete and quasi-complete separation;
- distinguish discrimination, calibration, and decision utility;
- prove that Brier score and log loss are proper for a binary probability;
- construct a reliability table and interpret calibration intercept and slope;
- calibrate a classifier without same-data leakage;
- calculate every cell and common rate in a confusion matrix;

- derive a cost-based threshold from calibrated probabilities;
- explain ROC AUC as a ranking probability and precision–recall metrics as prevalence-sensitive;
- distinguish class weighting, resampling, threshold movement, and probability correction;
- formulate spline-logistic, K-nearest-neighbour, kernel SVM, softmax, and multilabel models;
- derive Gini impurity and weighted impurity reduction;
- explain bagging, feature subsampling, out-of-bag prediction, and forest correlation;
- derive pseudo-residuals for logistic gradient boosting;
- design early stopping and hyperparameter tuning inside temporal or grouped boundaries;
- distinguish permutation importance, PDP, ICE, ALE, and SHAP by their target and assumptions;
- calculate a Shapley value in a small feature game;
- distinguish demographic parity, equal opportunity, equalised odds, predictive parity, and group calibration;
- explain why a metric parity is neither a complete ethical theory nor evidence of causal fairness;
- separate internal, temporal, geographic, and prospective validation; and
- execute and report a locked classification benchmark with probability, ranking, threshold, calibration, and subgroup results.

## The nine-day route

| Day    | Central idea                         | Problem resolved                                               |
|--------|--------------------------------------|----------------------------------------------------------------|
| Day 21 | Labels and probability               | A binary column can hide timing, censoring, and policy choices |
| Day 22 | Logistic likelihood and optimisation | A linear score must become a valid probability                 |
| Day 23 | Probability quality                  | Correct labels can coexist with misleading confidence          |
| Day 24 | Decisions and consequences           | A probability is not yet an action                             |
| Day 25 | Alternative decision surfaces        | Linear log odds may omit curved or local structure             |
| Day 26 | Recursive partitioning and bagging   | Interactions can be learned without enumerating them           |
| Day 27 | Functional gradients and search      | Powerful ensembles invite adaptive overfitting                 |
| Day 28 | Global and local explanation         | Opaque prediction needs bounded, assumption-aware inspection   |
| Day 29 | Accountability and transport         | Average internal performance is not deployment evidence        |



---

## Running case: major cost-overrun review

We continue the fictional microhydro power (MHP) programme. At technical appraisal, a review team has limited capacity to inspect projects at high risk of a **major cost overrun**.

For this chapter:

- one row is one eligible project;
- the index time is technical appraisal;
- all predictors must be known at appraisal;
- the event is final constant-price cost exceeding 115% of the approved appraisal budget;
- the outcome window ends when final accounts are closed; and
- a positive prediction can trigger senior engineering review, not automatic project rejection.

The model estimates:

$$p(x) = P(Y = 1 \mid X = x),$$

where  $Y = 1$  denotes a major overrun. A later policy maps  $p(x)$  to review or no review.

## Classification data extension

Save the Chapter 3 generator as `chapter3_data.py`, then save this file as `chapter4_data.py`.

```
import numpy as np
import pandas as pd

from chapter3_data import make_mhp_practitioner_data

def make_mhp_overrun_data(n=900, seed=4040):
 """Create a fictional appraisal-time binary prediction study."""
 df = make_mhp_practitioner_data(n=n, seed=seed)
 rng = np.random.default_rng(seed + 1)

 # This budget is created from appraisal-time information only. It is not
 # derived from final cost, even though the final outcome is stored nearby.
 district_budget_adjustment = {
 "Chitral": 3.0,
 "Dir": 1.0,
 "Swat": 0.5,
 "Shangla": 2.0,
 "Kohistan": 3.5,
 }

 district_term = df["district"].map(district_budget_adjustment).to_numpy()

 appraisal_budget = (
 11.0
 + 0.032 * df["planned_capacity_kw"]
 + 0.000018 * df["planned_capacity_kw"] ** 2
 + 1.05 * df["estimated_cable_km"]
 + 0.68 * df["surveyed_route_km"]
 + 2.10 * df["terrain_index"]
 + district_term
 + 0.55 * (df["start_year"] - 2016)
 + rng.normal(0.0, 2.5, size=n)
)
 df["appraisal_budget_2025_million_pkr"] = appraisal_budget

 df["major_cost_overrun"] = (
 df["actual_cost_2025_million_pkr"] > 1.15 * appraisal_budget
).astype(int)

 # Final-cost fields remain for teaching audits but are prohibited features.
 return df.sort_values(["start_year", "project_id"]).reset_index(drop=True)

if __name__ == "__main__":
 projects = make_mhp_overrun_data()
 projects.to_csv("mhp_overrun_chapter4.csv", index=False)
 print(projects.groupby("start_year")["major_cost_overrun"].agg(["size", "mean"]))
```

This is simulated teaching data, not evidence about actual projects or districts. A real label audit would require accounting expertise, consistent inflation adjustment, close-out dates, contract changes, and dispute-resolution rules.

## Minimal software setup

```
python -m pip install numpy pandas scipy matplotlib scikit-learn
```

Optional interpretation laboratories use:

```
python -m pip install shap
```

The executable capstone depends only on the first installation line. Record exact versions, seeds, and operating environment.

# Day 21 — The Outcome, Probability, and Prediction Contract

**Today's central idea:** The hardest classification error can occur before modelling—when a convenient binary column is mistaken for a well-defined future event.

## 21.1 Classification has at least three layers

Suppose a model returns  $p = 0.27$  for one project.

1. **Outcome layer:** What event does  $Y = 1$  mean, and when is it observed?
2. **Probability layer:** How credible is the estimate  $P(Y = 1 \mid X = x) = 0.27$ ?
3. **Decision layer:** Under current costs and capacity, does 0.27 trigger review?

The same probability can support different actions in different settings. Collapsing these layers makes it impossible to tell whether a failure came from the label, model, or policy.

## 21.2 Binary does not mean simple

A defensible binary target specifies:

- **event:** final cost exceeds 115% of approved appraisal budget;
- **index time:** technical appraisal;
- **time horizon:** final account close or a stated maximum follow-up;
- **eligibility:** projects for which that event can be meaningfully assessed;
- **positive class:** coded 1;
- **negative class:** coded 0 only after adequate observation; and
- **competing outcomes:** cancellation, merger, or scope change handled explicitly.

If a project has not reached final accounting, its event status may be unknown. Coding every incomplete record as 0 gives older projects more opportunity to become positive and creates label leakage through follow-up time.

When event timing and censoring are central, survival analysis is more appropriate than forcing a fixed binary label. A later chapter should develop that framework.

## 21.3 Label construction is a measurement model

The event definition contains choices.

- Why 15% rather than 10%?
- Which budget revision counts as the denominator?
- Are inflation and currency changes removed consistently?
- Does approved scope expansion count as an overrun?
- Who finalises the account, and can that process differ by district?

Sensitivity to alternate definitions should be studied, but the primary label must be fixed before model comparison. Trying several definitions and retaining the easiest one to predict changes the research question.

## 21.4 Probability, odds, and log odds

For an event probability  $0 < p < 1$ , the odds are

$$\text{odds} = \frac{p}{1-p}.$$

If  $p = 0.20$ , the odds are  $0.20/0.80 = 0.25$ , or 1 to 4. Odds are not probabilities.

Recover probability from odds  $o$ :

$$p = \frac{o}{1+o}.$$

Log odds, or the logit, are

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right).$$

Log odds range across all real numbers even though probabilities remain between 0 and 1. This makes them a useful scale for a linear predictor.

| Probability | Odds   | Log odds |
|-------------|--------|----------|
| 0.01        | 0.0101 | -4.595   |
| 0.20        | 0.25   | -1.386   |
| 0.50        | 1.00   | 0.000    |
| 0.80        | 4.00   | 1.386    |
| 0.99        | 99.00  | 4.595    |

## 21.5 Bernoulli outcomes

A Bernoulli random variable has

$$Y \sim \text{Bernoulli}(p), \quad Y \in \{0, 1\}.$$

Its probability mass function is

$$P(Y = y) = p^y(1 - p)^{1-y}.$$

Check both possibilities:

- if  $y = 1$ , the expression is  $p$ ;
- if  $y = 0$ , the expression is  $1 - p$ .

The conditional mean and variance are

$$\mathbb{E}[Y \mid X = x] = p(x),$$

$$\text{Var}(Y \mid X = x) = p(x)(1 - p(x)).$$

Unlike homoskedastic linear regression, Bernoulli variance is determined by the conditional mean and is largest at  $p = 0.5$ .

## 21.6 Prevalence is a property of a population and window

Sample prevalence is

$$\hat{\pi} = \frac{1}{n} \sum_{i=1}^n y_i.$$

It is the event fraction in a stated sample. It can change with:

- project mix;
- event threshold;
- time period;
- follow-up completeness;
- inclusion rules; and
- deliberate oversampling.

```
from chapter4_data import make_mhp_overrun_data

df = make_mhp_overrun_data()
print("Overall prevalence:", df["major_cost_overrun"].mean())
print(
 df.groupby("start_year")["major_cost_overrun"]
 .agg(n="size", prevalence="mean")
)
print(
 df.groupby("access_mode")["major_cost_overrun"]
 .agg(n="size", prevalence="mean")
)
```

A case-control sample may intentionally contain 50% events even if deployment prevalence is 5%. Such a sample can support some association or ranking analyses, but its raw fitted probabilities and precision do not automatically transport to the population.

## 21.7 The probability baseline

A no-feature probability model predicts the training prevalence for every row:

$$\hat{p}_i = \hat{\pi}_{train}.$$

This is the maximum-likelihood constant Bernoulli model. It is the classification analogue of a mean baseline under squared loss.

For a later temporal fold, use the prevalence in earlier training years only. A full-dataset prevalence leaks future event frequency.

```
import numpy as np

def prevalence_baseline(y_train, n_future):
 probability = float(np.mean(y_train))
 return np.full(n_future, probability)
```

Every probability model should beat a relevant constant baseline under the pre-specified proper score. A 90% accurate classifier in a 10% positive population may merely predict 0 for everyone.

## 21.8 Selection into the labelled sample

Let  $S = 1$  mean that a project appears with a resolved label. The model is trained on

$$P(Y = 1 \mid X, S = 1),$$

but deployment may require

$$P(Y = 1 \mid X).$$

These differ if label resolution depends on outcome or predictors. Fast-closing, well-documented projects may be overrepresented. Before model fitting, compare eligible labelled and unlabelled projects on appraisal-time variables and follow-up.

This comparison cannot identify every selection mechanism, but it prevents a complete table from being mistaken for a representative one.

## 21.9 Day 21 build, break, and reflect

### Build

1. Write the event, index time, horizon, eligibility, and unresolved-outcome policy.
2. calculate prevalence by year, district, and access mode with sample sizes.
3. construct a training-only prevalence baseline for each forward fold.
4. create a label sensitivity table at 10%, 15%, and 20% overrun without changing the primary definition.

### Break

1. code unfinished projects as non-events.
2. define the event after comparing which label is easiest to predict.
3. use the post-construction material bill as a predictor.
4. report sample prevalence as a timeless project property.
5. call a 90% all-negative rule a strong model in a 10% event sample.

### Reflect

Who can be harmed by a false positive review, a false negative missed overrun, or an unresolved outcome coded as negative? The label is already part of the intervention.



## Day 21 exit check

You are ready for logistic regression when you can move between probability and log odds, explain Bernoulli mean and variance, and defend every time boundary in the label.

# Day 22 — Logistic Regression from Bernoulli Likelihood

**Today's central idea:** Logistic regression is a linear model for log odds, estimated by maximising a Bernoulli likelihood—not OLS applied to zeros and ones.

## 22.1 Why not fit a straight probability line?

The linear probability model

$$P(Y = 1 \mid X = x) = x^\top \beta$$

can predict below 0 or above 1 and imposes a constant probability change for a one-unit feature change. It can still be useful in some inferential settings, but it is not a naturally bounded probability model.

Logistic regression makes log odds linear:

$$\log \left( \frac{p(x)}{1 - p(x)} \right) = x^\top \beta.$$

Apply the inverse logit:

$$p(x) = \sigma(x^\top \beta) = \frac{1}{1 + e^{-x^\top \beta}}$$

where  $\sigma$  is the sigmoid function.

## 22.2 Sigmoid geometry

Let  $z = x^\top \beta$ . Then

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

The derivative is largest at  $z = 0$ , where  $p = 0.5$ , and approaches zero in the tails. A fixed log-odds change produces the largest absolute probability change near 0.5.

```
import numpy as np
import matplotlib.pyplot as plt

def stable_sigmoid(z):
 z = np.asarray(z, dtype=float)
 result = np.empty_like(z)
```

```

positive = z >= 0
result[positive] = 1.0 / (1.0 + np.exp(-z[positive]))
exp_z = np.exp(z[-positive])
result[-positive] = exp_z / (1.0 + exp_z)
return result

z = np.linspace(-8.0, 8.0, 400)
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(z, stable_sigmoid(z))
ax.axhline(0.5, color="black", linewidth=1, alpha=0.5)
ax.axvline(0.0, color="black", linewidth=1, alpha=0.5)
ax.set(xlabel="linear score z", ylabel="probability",
 title="The sigmoid maps every real score into (0, 1)")
ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()

```

The branch implementation avoids overflow for very large positive or negative scores.

## 22.3 Constructing the likelihood

For independent observations with  $p_i = \sigma(\mathbf{x}_i^\top \beta)$ ,

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Take logs:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Maximising log likelihood is equivalent to minimising average negative log likelihood, also called binary cross-entropy or log loss:

$$J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

A confidently wrong probability receives a large penalty. If  $y = 1$ , predicting 0.9 costs  $-\log(0.9) \approx 0.105$ , while predicting 0.001 costs about 6.908.

## 22.4 Deriving the gradient

For one observation, write  $\mathbf{z}_i = \mathbf{x}_i^\top \beta$  and  $p_i = \sigma(\mathbf{z}_i)$ . The derivative of negative log likelihood with respect to  $\mathbf{z}_i$  is

$$\frac{\partial J_i}{\partial \mathbf{z}_i} = p_i - y_i.$$

Because  $\partial \mathbf{z}_i / \partial \beta = \mathbf{x}_i$ ,

$$\nabla_{\beta} J = \frac{1}{n} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}).$$

This resembles the OLS gradient, but residual-like probability errors are weighted through the nonlinear link.

With an  $L_2$  penalty on slopes but not the intercept,

$$J_{\lambda}(\beta) = J(\beta) + \frac{\lambda}{2} \sum_{j=1}^p \beta_j^2,$$

and

$$\nabla J_{\lambda} = \frac{1}{n} X^{\top} (p - y) + \lambda \begin{bmatrix} 0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix}.$$

Software packages use different loss normalisations and inverse-penalty parameters. In `scikit-learn`, smaller `C` means stronger regularisation. Do not copy `C` or  $\lambda$  across packages without checking the objective.

## 22.5 Hessian, convexity, and IRLS

Define

$$W = \text{diag}(p_i(1 - p_i)).$$

The unpenalised Hessian is

$$\nabla_{\beta}^2 J = \frac{1}{n} X^{\top} W X.$$

For any vector  $v$ ,

$$v^{\top} X^{\top} W X v = (Xv)^{\top} W (Xv) \geq 0,$$

so the negative log likelihood is convex. A full-rank, nonseparated problem has a unique optimum. An  $L_2$  penalty adds positive curvature to penalised directions.

Newton's update is

$$\beta_{new} = \beta_{old} - H^{-1} \nabla J.$$

For a generalized linear model this can be expressed as **iteratively reweighted least squares** (IRLS): calculate current probabilities and weights, form a working response, solve a weighted least-squares problem, and repeat. This is a computational connection, not a claim that a binary outcome has Gaussian errors.

## 22.6 Logistic regression from scratch

```
import numpy as np

class LogisticRegressionFromScratch:
 def __init__(self, learning_rate=0.1, l2=0.0,
 max_iter=50_000, tol=1e-9):
 self.learning_rate = float(learning_rate)
 self.l2 = float(l2)
 self.max_iter = int(max_iter)
 self.tol = float(tol)

 @staticmethod
 def _sigmoid(z):
 z = np.asarray(z, dtype=float)
 result = np.empty_like(z)
 positive = z >= 0
```

```

 result[positive] = 1.0 / (1.0 + np.exp(-z[positive]))
 exp_z = np.exp(z[~positive])
 result[~positive] = exp_z / (1.0 + exp_z)
 return result

def fit(self, X, y):
 X = np.asarray(X, dtype=float)
 y = np.asarray(y, dtype=float)
 if set(np.unique(y)) - {0.0, 1.0}:
 raise ValueError("y must contain only 0 and 1")

 self.x_mean_ = X.mean(axis=0)
 self.x_scale_ = X.std(axis=0, ddof=0)
 self.x_scale_[self.x_scale_ == 0] = 1.0
 Z = (X - self.x_mean_) / self.x_scale_
 design = np.column_stack([np.ones(len(Z)), Z])

 beta = np.zeros(design.shape[1])
 previous_loss = np.inf

 for iteration in range(self.max_iter):
 probability = self._sigmoid(design @ beta)
 clipped = np.clip(probability, 1e-15, 1.0 - 1e-15)
 loss = -np.mean(
 y * np.log(clipped) + (1.0 - y) * np.log(1.0 - clipped)
) + 0.5 * self.l2 * np.sum(beta[1:] ** 2)

 penalty_gradient = np.r_[0.0, self.l2 * beta[1:]]
 gradient = design.T @ (probability - y) / len(y)
 gradient += penalty_gradient
 beta -= self.learning_rate * gradient

 if abs(previous_loss - loss) < self.tol:
 break
 previous_loss = loss

 self.n_iter_ = iteration + 1
 self.coef_standardized_ = beta[1:]
 self.coef_ = beta[1:] / self.x_scale_
 self.intercept_ = beta[0] - self.x_mean_ @ self.coef_
 self.loss_ = loss
 return self

def predict_proba(self, X):
 X = np.asarray(X, dtype=float)
 positive = self._sigmoid(self.intercept_ + X @ self.coef_)
 return np.column_stack([1.0 - positive, positive])

def predict(self, X, threshold=0.5):
 return (self.predict_proba(X)[:, 1] >= threshold).astype(int)

```

Verify the unpenalised implementation on a small, nonseparated dataset.

```

import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(422)
X_demo = rng.normal(size=(600, 3))
true_beta = np.array([0.8, -1.2, 0.5])
true_probability = stable_sigmoid(-0.4 + X_demo @ true_beta)
y_demo = rng.binomial(1, true_probability)

manual = LogisticRegressionFromScratch(
 learning_rate=0.2, l2=0.0, max_iter=100_000, tol=1e-12
).fit(X_demo, y_demo)
reference = LogisticRegression(C=1e12, max_iter=10_000).fit(X_demo, y_demo)

manual_probability = manual.predict_proba(X_demo)[:, 1]

```

```
reference_probability = reference.predict_proba(X_demo)[:, 1]
assert np.allclose(manual_probability, reference_probability, atol=5e-4)
print("iterations:", manual.n_iter_)
print("manual slopes:", manual.coef_)
print("library slopes:", reference.coef_[0])
```

For production work, use stable, tested solvers and inspect convergence warnings.

## 22.7 Interpreting coefficients and odds ratios

For

$$\text{logit}(p) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p,$$

a one-unit increase in  $x_j$ , holding represented covariates fixed, changes log odds by  $\beta_j$  and multiplies odds by

$$\exp(\beta_j).$$

If  $\beta_j = 0.4$ , the conditional odds ratio is  $e^{0.4} \approx 1.49$ . This is a 49% increase in odds, not a 49-percentage-point increase in probability and not necessarily a causal effect.

The probability change depends on the starting risk:

$$\frac{\partial p}{\partial x_j} = \beta_j p(1 - p).$$

Interactions and nonlinear terms make the derivative conditional on other feature values, just as in Chapter 3.

## 22.8 Complete separation

Suppose every project with road distance above 25 km is positive and every other project negative. A coefficient can grow without bound while the unpenalised likelihood keeps improving. This is **complete separation**. Quasi-complete separation leaves some boundary ties.

Symptoms include:

- huge coefficients or standard errors;
- failure to converge;
- fitted probabilities extremely near 0 and 1; and
- estimates that change dramatically under small perturbations.

Responses include:

- verify the data and feature timing;
- simplify or combine sparse categories when substantively justified;
- apply pre-specified regularisation;
- use a bias-reduced estimator for inference; and
- collect observations in the separated region.

Do not celebrate perfect training accuracy before investigating separation and leakage.

## 22.9 Regularisation and rare events

Logistic ridge, lasso, and elastic net have the same scaling and selection cautions as their regression counterparts. Regularisation can stabilise sparse categories and correlated features, but:

- a zero coefficient is not proof of irrelevance;
- ordinary post-selection intervals ignore the search;
- class weighting changes the fitted objective; and
- rare-event probability calibration still requires representative validation.

An intercept correction may be needed when training prevalence differs from deployment prevalence because of sampling. Simply assigning class weights and interpreting the resulting outputs as population probabilities is unsafe.

## 22.10 Research paper study: Nelder and Wedderburn (1972)

John Nelder and Robert Wedderburn's *Generalized Linear Models* unified several outcome distributions, variance functions, and link functions within one modelling framework and developed iterative weighted fitting.

Read with five questions.

1. **Unification.** How do random component, systematic component, and link function fit together?
2. **Binary case.** Which distribution and links are available for proportions or binary outcomes?
3. **Computation.** Why does iterative weighted least squares arise?
4. **Evidence.** How is deviance used to compare fitted structures?
5. **Boundary.** Which independence, mean-structure, and asymptotic assumptions remain?

### Replication task

Implement Newton–Raphson or IRLS for a two-feature logistic model. At every iteration record log likelihood, gradient norm, and smallest Hessian eigenvalue. Repeat under well-overlapped data, near separation, and  $L_2$  regularisation.

## 22.11 Day 22 build, break, and reflect

### Build

1. Derive the gradient without consulting the displayed result.
2. fit the from-scratch model and compare probabilities with `scikit-learn`.
3. convert three slopes into odds ratios and probability changes at risks 0.1, 0.5, and 0.9.
4. create a separated toy dataset and observe coefficient growth with and without regularisation.

### Break

1. interpret an odds ratio of 1.5 as a 50-point risk increase.
2. ignore a convergence warning because training accuracy is perfect.
3. penalise unscaled predictors and compare coefficient magnitudes as importance.
4. use a final-account variable to achieve separation.
5. call an associational odds ratio causal without a causal design.

### Reflect

Why might a stable, well-calibrated logistic model be preferable to a more accurate but opaque classifier for a high-stakes review process? State the decision context rather than appealing to simplicity alone.

### Day 22 exit check

You are ready for calibration when you can derive binary cross-entropy, its gradient, and its Hessian; interpret an odds ratio; and recognise separation.

---

# Day 23 — Proper Scoring and Calibration

**Today's central idea:** A useful risk model must say how often events occur among cases assigned each probability—not merely place many cases on the correct side of 0.5.

## 23.1 Three evaluation targets

Consider two projects. One receives probability 0.51 and the other 0.99; both later overrun.

At threshold 0.5, both labels are correct. Yet the probability claims differ greatly. Classification evaluation should separate:

1. **Discrimination:** are higher-risk cases ranked above lower-risk cases?
2. **Calibration:** do stated probabilities match observed frequencies in a relevant population?
3. **Decision value:** do actions based on those probabilities improve outcomes under stated consequences?

No single metric fully answers all three.

## 23.2 What calibration means

Let  $P = \hat{p}(X)$  be a model's predicted probability. Perfect calibration means

$$P(Y = 1 \mid P = p) = p$$

for predicted values with support.

Among many deployment-like projects assigned risk near 0.20, about 20% should experience the event. Calibration is population-specific. A model can be calibrated in one time period and miscalibrated after prevalence or conditional relationships shift.

Calibration is also weaker than correctness for an individual. An event either occurs or does not; the validity of a 0.20 forecast is assessed across comparable forecasts or through a proper scoring rule.

## 23.3 Brier score

For binary outcomes,

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2.$$

Lower is better. It measures squared probability error and combines calibration and discrimination-related resolution.

### Why Brier score is proper

Suppose the true event probability is  $q$  but a forecaster reports  $p$ . Expected squared loss is

$$\mathbb{E}[(p - Y)^2] = q(p - 1)^2 + (1 - q)p^2.$$

Expand:

$$= p^2 - 2pq + q.$$

Differentiate with respect to  $p$ :

$$\frac{d}{dp} = 2(p - q).$$

The unique minimum is  $p = q$ . Truthful probability reporting minimises expected loss. This is what **strictly proper** means in the binary case.

## 23.4 Log loss

Average log loss is

$$\text{LogLoss} = -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

It is also strictly proper. If the true event probability is  $q$ , expected loss for report  $p$  is

$$-q \log p - (1 - q) \log(1 - p).$$

Its derivative is

$$-\frac{q}{p} + \frac{1 - q}{1 - p} = \frac{p - q}{p(1 - p)},$$

which is zero at  $p = q$ . The second derivative is positive on  $(0, 1)$ .

Log loss penalises extreme mistakes more strongly than Brier score. That can be appropriate when unjustified certainty is dangerous, but it also makes label errors and tiny samples influential.

## 23.5 Compare probability forecasts directly

```
import numpy as np
from sklearn.metrics import brier_score_loss, log_loss

y = np.array([1, 0, 1, 0])
moderate = np.array([0.70, 0.30, 0.65, 0.40])
overconfident = np.array([0.99, 0.01, 0.01, 0.99])

for name, probability in {
 "moderate": moderate,
 "overconfident": overconfident,
}.items():
 print(name, {
 "brier": brier_score_loss(y, probability),
 "log_loss": log_loss(y, probability),
 "accuracy_at_0.5": np.mean((probability >= 0.5) == y),
 })
```

The extreme wrong forecasts dominate log loss. Accuracy discards how confident the model was.

## 23.6 Reliability diagrams

A reliability diagram groups predicted probabilities and compares average prediction with event frequency in each group.



```

import numpy as np
import pandas as pd

def reliability_table(y_true, probability, bins=10):
 frame = pd.DataFrame({
 "outcome": np.asarray(y_true, dtype=int),
 "probability": np.asarray(probability, dtype=float),
 })
 frame["bin"] = pd.cut(
 frame["probability"],
 bins=np.linspace(0.0, 1.0, bins + 1),
 include_lowest=True,
)
 return (
 frame.groupby("bin", observed=True)
 .agg(
 n=("outcome", "size"),
 mean_probability=("probability", "mean"),
 event_rate=("outcome", "mean"),
)
 .reset_index()
)

```

Interpret with care.

- Empty bins contain no evidence.
- Small bins have wide uncertainty.
- Equal-width and equal-count bins answer slightly different descriptive questions.
- A line near the diagonal can change with binning.
- A reliability curve does not measure ranking quality.

Plot raw case distributions along the horizontal axis and include bin counts or uncertainty intervals.

## 23.7 Brier decomposition

If forecasts take  $K$  distinct values—or if every forecast in bin  $k$  is deliberately replaced by the bin mean  $\bar{p}_k$ —let:

- $n_k$  be bin size;
- $\bar{p}_k$  be mean forecast in bin  $k$ ;
- $\bar{y}_k$  be event rate in bin  $k$ ; and
- $\bar{y}$  be overall event rate.

the resulting binned Brier score decomposes as

$$\text{Brier} = \underbrace{\frac{1}{n} \sum_k n_k (\bar{p}_k - \bar{y}_k)^2}_{\text{reliability}} - \underbrace{\frac{1}{n} \sum_k n_k (\bar{y}_k - \bar{y})^2}_{\text{resolution}} + \underbrace{\bar{y}(1 - \bar{y})}_{\text{uncertainty}}.$$

- **Reliability:** penalty for forecast–frequency disagreement.
- **Resolution:** reward for separating groups with event rates different from the overall rate.
- **Uncertainty:** irreducible variability associated with prevalence.

The original score of continuously varying  $p_i$  is not generally identical to this binned score; within-bin forecast variation and association with outcomes can contribute extra terms. The numerical decomposition therefore depends on the grouping scheme. It is a diagnostic lens, not three invariant properties of a finite dataset.

## 23.8 Calibration intercept and slope

On an untouched validation sample, calculate the model logit

$$z_i = \log \left( \frac{\hat{p}_i}{1 - \hat{p}_i} \right).$$

Fit the recalibration model

$$\text{logit } P(Y_i = 1) = a + bz_i.$$

Ideal values are:

- calibration intercept  $a = 0$ ; and
- calibration slope  $b = 1$ .

Broad interpretations:

- $a < 0$  often indicates probabilities are too high on average;
- $a > 0$  often indicates they are too low on average;
- $b < 1$  often indicates predictions are too extreme; and
- $b > 1$  often indicates an under-dispersed risk range.

These estimates have uncertainty and can be unstable with few events. Fitting them on the training predictions is optimistically biased.

```
import numpy as np
from sklearn.linear_model import LogisticRegression

def calibration_intercept_slope(y_true, probability):
 probability = np.clip(np.asarray(probability), 1e-6, 1.0 - 1e-6)
 logit = np.log(probability / (1.0 - probability)).reshape(-1, 1)
 # A very large C approximates an unpenalised calibration regression.
 model = LogisticRegression(C=1e12, max_iter=10_000)
 model.fit(logit, y_true)
 return float(model.intercept_[0]), float(model.coef_[0, 0])
```

Use this for validation diagnostics. Do not fit a recalibration map on the locked test and then report the recalibrated score on that same test.

## 23.9 Calibration methods

Two common post-hoc methods are:

### Sigmoid calibration

Fit a logistic mapping from an uncalibrated score  $s$  to probability:

$$P(Y = 1 \mid s) = \sigma(as + b).$$

This is low-dimensional and relatively stable, but the sigmoid form can be wrong.

### Isotonic calibration

Fit a nondecreasing piecewise-constant mapping from score to observed event rate. It is more flexible and can overfit when calibration data contain few observations or events.

Both require scores for observations not used to fit the underlying model. Valid patterns include:

- a dedicated calibration period after model-training data;
- cross-fitted predictions within development data; or
- a nested calibration procedure inside every outer validation fold.

The final calibrated procedure must then be evaluated on untouched data.

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold

inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=423)
calibrated_forest = CalibratedClassifierCV(
 estimator=RandomForestClassifier(
 n_estimators=500,
 min_samples_leaf=10,
 random_state=423,
 n_jobs=-1,
),
 method="sigmoid",
 cv=inner_cv,
)
```

For temporal deployment, replace random stratified folds with an order-respecting calibration design. The code above demonstrates the estimator interface, not the MHP time split.

## 23.10 Expected calibration error is not enough

Expected calibration error (ECE) averages absolute bin gaps:

$$\text{ECE} = \sum_k \frac{n_k}{n} |\bar{p}_k - \bar{y}_k|.$$

It is easy to communicate but depends heavily on bin boundaries, may conceal local miscalibration, and is not a strictly proper score for comparing probability forecasts. Report a proper score, a reliability display, and uncertainty rather than optimising ECE alone.

## 23.11 Research paper study: Niculescu-Mizil and Caruana (2005)

Alexandru Niculescu-Mizil and Rich Caruana's *Predicting Good Probabilities with Supervised Learning* empirically compared probability predictions from several learning algorithms and studied sigmoid and isotonic calibration.

Read with five questions.

1. **Distinction.** How do ranking performance and probability quality differ?
2. **Distortion.** Which characteristic probability distortions appear for margin methods, boosted trees, and naive Bayes?
3. **Calibration design.** Which data fit the base model and which fit the calibrator?
4. **Metric.** How are squared error, cross-entropy, and ROC performance used?
5. **Transfer.** Would the paper's random-split findings automatically hold under temporal prevalence shift?

## Replication task

Simulate a known nonlinear probability surface. Fit logistic regression, a decision tree, a random forest, and a boosted classifier. Compare log loss, Brier score, ROC AUC, and reliability plots before and after cross-fitted sigmoid and isotonic calibration. Repeat at sample sizes 300, 3,000, and 30,000.

## 23.12 Day 23 build, break, and reflect

### Build

1. Prove Brier score and log loss are proper for a binary probability.
2. build reliability tables using equal-width and equal-count bins.
3. estimate calibration intercept and slope on each temporal validation year.
4. compare uncalibrated, sigmoid, and isotonic procedures inside development data.

### Break

1. evaluate probabilities using accuracy alone.
2. fit and assess a calibrator on the same predictions.
3. declare perfect calibration from three tiny bins.
4. optimise ECE while omitting a proper score.
5. transport calibration across periods without checking prevalence or covariates.

### Reflect

A model can have better ROC AUC and worse log loss than another. Which one should be used for risk communication? The answer depends on whether ranking or credible probabilities drive the decision.

## Day 23 exit check

You are ready for threshold decisions when you can distinguish discrimination from calibration, prove propriety of two scores, and design a leakage-free calibration step.

---

# Day 24 — Thresholds, Ranking, and Class Imbalance

---

**Today's central idea:** A threshold is a policy parameter chosen from consequences, resources, and probability quality—not a natural property of a classifier.

## 24.1 From probability to action

Let  $a = 1$  mean “send to senior review.” A simple rule is

$$a_i = \mathbb{1}\{p_i \geq t\}.$$

Changing  $t$  changes who is reviewed without retraining the probability model. The default  $t = 0.5$  is rarely justified by operational consequences.

Keep three objects separate:

- fitted probability model;
- chosen threshold or allocation rule; and
- realised decision outcomes.

## 24.2 Confusion matrix

For positive event  $Y = 1$  and positive action  $\hat{Y} = 1$ :

|                                 | Event $Y = 1$       | No event $Y = 0$    |
|---------------------------------|---------------------|---------------------|
| Predict/review $\hat{Y} = 1$    | true positive (TP)  | false positive (FP) |
| Predict/no review $\hat{Y} = 0$ | false negative (FN) | true negative (TN)  |

Common rates are:

$$\text{sensitivity} = \text{recall} = \text{TPR} = \frac{TP}{TP + FN},$$

$$\text{specificity} = \text{TNR} = \frac{TN}{TN + FP},$$

$$\text{precision} = \text{PPV} = \frac{TP}{TP + FP},$$

$$\text{FPR} = \frac{FP}{FP + TN}, \quad \text{FNR} = \frac{FN}{FN + TP}.$$

Accuracy is

$$\frac{TP + TN}{n},$$

and balanced accuracy is

$$\frac{1}{2}(\text{TPR} + \text{TNR}).$$

Precision depends on prevalence as well as the score distributions. Sensitivity and specificity also need not transport if conditional distributions change.

## 24.3 Deriving a cost-based threshold

Assume calibrated event probability  $p$  and two actions.

- Review costs  $C_{FP}$  when no event would occur.
- Failing to review costs  $C_{FN}$  when an event occurs.
- Correct-action costs are set to zero for this simplified derivation.

Expected cost of review is

$$C_{\text{review}}(p) = C_{FP}(1 - p).$$

Expected cost of no review is

$$C_{no\ review}(p) = C_{FN}p.$$

Review when

$$C_{FP}(1 - p) \leq C_{FN}p,$$

which gives

$$p \geq \frac{C_{FP}}{C_{FP} + C_{FN}}$$

If a missed major overrun costs five times as much as an unnecessary review, the simplified threshold is  $1/(1 + 5) \approx 0.167$ .

Real decisions can include review benefit, event severity, limited capacity, delay, project-specific costs, and downstream human behaviour. Then use an explicit expected-utility or constrained-allocation calculation rather than forcing everything into one constant threshold.

## 24.4 Threshold tables

```
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix

def safe_ratio(numerator, denominator):
 return numerator / denominator if denominator else np.nan

def threshold_table(y_true, probability, thresholds, fp_cost=1.0, fn_cost=5.0):
 y_true = np.asarray(y_true, dtype=int)
 probability = np.asarray(probability, dtype=float)
 rows = []

 for threshold in thresholds:
 prediction = (probability >= threshold).astype(int)
 tn, fp, fn, tp = confusion_matrix(
 y_true, prediction, labels=[0, 1]
).ravel()
 rows.append({
 "threshold": threshold,
 "review_rate": prediction.mean(),
 "sensitivity": safe_ratio(tp, tp + fn),
 "specificity": safe_ratio(tn, tn + fp),
 "precision": safe_ratio(tp, tp + fp),
 "false_positive_rate": safe_ratio(fp, fp + tn),
 "cost_per_project": (fp_cost * fp + fn_cost * fn) / len(y_true),
 })
 return pd.DataFrame(rows)
```

Calculate this table on validation data. Selecting a threshold on the locked test spends the test.

## 24.5 $F_1$ and why it is not a cost function

The  $F_1$  score is the harmonic mean of precision and recall:

$$F_1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

It ignores true negatives and encodes a particular symmetric combination of precision and recall. It does not directly represent monetary cost, review capacity, calibrated probability quality, or event severity.

Use it only when that trade-off is substantively appropriate. “The data are imbalanced” is not enough justification.

## 24.6 ROC curves and AUC

For every threshold, plot:

- false-positive rate on the horizontal axis; and
- true-positive rate on the vertical axis.

The ROC curve describes ranking trade-offs across thresholds. Under standard treatment of ties, ROC AUC can be interpreted as the probability that a randomly chosen positive receives a higher score than a randomly chosen negative, plus half the tie probability.

ROC AUC:

- is threshold-free but not decision-free;
- measures ranking, not calibration;
- weights parts of the FPR range that may be operationally irrelevant; and
- can appear strong while precision is poor in a rare-event population.

A partial AUC or a decision-specific region may be more relevant when only low false-positive rates are feasible, but the region must be pre-specified.

## 24.7 Precision–recall curves

A precision–recall (PR) curve plots precision against recall as the threshold moves. Its no-skill precision baseline is approximately event prevalence.

PR curves focus attention on positive predictions and are often informative for rare events. But they are sensitive to prevalence: a model can have unchanged class-conditional ranking and lower precision in a population with rarer events.

Average precision summarises the curve using a weighted average of precision values across recall increments. It is not identical to every trapezoidal “PR AUC” implementation. State the calculation used.

```
import matplotlib.pyplot as plt
from sklearn.metrics import (
 PrecisionRecallDisplay,
 RocCurveDisplay,
 average_precision_score,
 roc_auc_score,
)

print("ROC AUC:", roc_auc_score(y_valid, probability_valid))
print("Average precision:", average_precision_score(y_valid, probability_valid))

fig, axes = plt.subplots(1, 2, figsize=(10, 4.3))
RocCurveDisplay.from_predictions(y_valid, probability_valid, ax=axes[0])
PrecisionRecallDisplay.from_predictions(y_valid, probability_valid, ax=axes[1])
plt.tight_layout()
plt.show()
```

## 24.8 What class imbalance changes

Class imbalance is not itself a defect. It is a description of event frequency. It creates practical challenges:

- accuracy can be dominated by the majority class;
- probability and calibration estimates for rare events have high variance;
- random folds may contain too few events;
- extreme resampling can distort probability estimation; and
- decision costs may be asymmetric.

Responses must match the goal.

### Class weighting

Weighted loss gives some observations larger influence during fitting. This can improve ranking or minority recall, but it changes the fitted target. Raw outputs may no longer estimate population probabilities without correction and representative calibration.

### Oversampling and undersampling

Resampling changes the training distribution. Perform it inside each training fold only. Never oversample before splitting, because duplicates or synthetic points can contaminate validation.

### Synthetic minority oversampling

Methods such as SMOTE create synthetic minority points between neighbours. They can be implausible for mixed, constrained, temporal, or high-dimensional features. They do not create new independent event information.

### Threshold movement

If ranking and calibration are adequate, changing the decision threshold may address an asymmetric action problem without changing the probability model.

These methods are not interchangeable.

## 24.9 Capacity-constrained review

Suppose the team can review only 30 projects. A rank-based policy reviews the 30 largest probabilities. Its effective threshold changes with each cohort.

Evaluate:

- recall among the top  $k$ ;
- event yield among reviewed projects;
- stability of the selected set;
- subgroup allocation; and
- whether probabilities remain useful for communicating risk.

Top- $k$  allocation is a policy. Do not report its performance as though a fixed threshold had been deployed.



## 24.10 Research paper study: Davis and Goadrich (2006)

Jesse Davis and Mark Goadrich's *The Relationship Between Precision-Recall and ROC Curves* develops the mathematical relationship between the two spaces and explains why visual impressions can differ under class imbalance.

Read with five questions.

1. **Mapping.** Which confusion-matrix quantities connect ROC and PR points?
2. **Dominance.** Under what condition does one curve dominate another in both spaces?
3. **Interpolation.** Why is linear interpolation in PR space generally inappropriate?
4. **Prevalence.** Where does the positive-class proportion enter the mapping?
5. **Decision.** Why does curve dominance still not select an operating threshold without consequences?

### Replication task

Generate fixed positive and negative score distributions. Change only the number of negative cases. Plot ROC and PR curves and calculate ROC AUC, average precision, and precision at a fixed recall. Explain which quantities change and why.

## 24.11 Day 24 build, break, and reflect

### Build

1. Derive the simplified cost threshold.
2. produce a validation threshold table from 0.05 to 0.80.
3. compare accuracy, balanced accuracy, ROC AUC, average precision, and cost.
4. simulate a prevalence shift while holding class-conditional scores fixed.
5. formulate a top- $k$  review rule and audit who receives the scarce reviews.

### Break

1. use 0.5 because it is the software default.
2. tune a threshold on the final test.
3. call ROC AUC a calibration metric.
4. oversample before splitting.
5. interpret weighted-classifier outputs as population probabilities without checking calibration.
6. optimise  $F_1$  despite known asymmetric costs and capacity.

### Reflect

Who should assign  $C_{FP}$  and  $C_{FN}$ ? An analyst can calculate consequences, but legitimate values require the people who operate, fund, and experience the decision.

### Day 24 exit check

You are ready for nonlinear classifiers when you can calculate every confusion-matrix rate, derive a decision threshold, and explain why ROC and PR summaries answer different questions.

---

# Day 25 — Nonlinear, Kernel, and Multiple-Class Models

**Today's central idea:** Nonlinear classification changes the shape of the score surface. It does not remove the need for probability evaluation, scaling, validation, or a defensible decision rule.

## 25.1 Linear log odds can still produce nonlinear probability

Logistic regression with

$$\text{logit}(p) = \beta_0 + \beta_1 x$$

already produces an S-shaped probability curve in  $x$ . “Nonlinear model” can therefore mean several different things:

- nonlinear probability induced by a link;
- nonlinear predictor terms such as  $x^2$  or splines;
- nonlinear interactions among predictors; or
- an algorithm whose decision function is not linear in an explicit design.

Name the layer that is nonlinear.

## 25.2 Polynomial and interaction logistic regression

Add a centred square:

$$\text{logit}(p) = \beta_0 + \beta_1 x_c + \beta_2 x_c^2.$$

The derivative of probability is

$$\frac{\partial p}{\partial x} = p(1-p)(\beta_1 + 2\beta_2 x_c).$$

The stationary locations on the probability scale occur where the log-odds derivative is zero, provided  $0 < p < 1$ . Interpretation still requires support, uncertainty, hierarchy, and a noncausal warning.

For an interaction,

$$\text{logit}(p) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2.$$

Then

$$\frac{\partial p}{\partial x_1} = p(1-p)(\beta_1 + \beta_3 x_2).$$

An interaction on log odds also produces probability-scale interaction patterns that depend on baseline risk. Do not infer absence of probability interaction merely because  $\beta_3 = 0$ , or causal effect modification merely because it is nonzero.

## 25.3 Spline logistic models and GAMs

A spline-logistic model uses basis functions:

$$\text{logit}(p) = \beta_0 + \sum_{m=1}^M \theta_m B_m(x).$$

A generalized additive model (GAM) extends this to

$$g(\mathbb{E}[Y | X]) = \beta_0 + f_1(x_1) + \cdots + f_p(x_p),$$

where  $g$  is a link and each  $f_j$  is a smooth function. For a binary logistic GAM,  $g$  is the logit.

GAMs provide flexible marginal shapes while retaining additive structure. They do not automatically learn interactions; selected tensor-product or interaction smooths must be introduced and regularised deliberately.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, SplineTransformer, StandardScaler

smooth_columns = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 "appraisal_budget_2025_million_pkr",
]
linear_columns = [
 "surveyed_route_km",
 "terrain_index",
 "contractor_experience_projects",
 "start_year",
]
categorical_columns = ["district", "access_mode"]

smooth_pipeline = Pipeline([
 ("impute", SimpleImputer(strategy="median")),
 ("spline", SplineTransformer(
 n_knots=5, degree=3, include_bias=False
)),
 ("scale", StandardScaler()),
])
linear_pipeline = Pipeline([
 ("impute", SimpleImputer(strategy="median", add_indicator=True)),
 ("scale", StandardScaler()),
])
categorical_pipeline = Pipeline([
 ("impute", SimpleImputer(strategy="most_frequent")),
 ("one_hot", OneHotEncoder(handle_unknown="ignore", drop="first")),
])

spline_preprocessor = ColumnTransformer([
 ("smooth", smooth_pipeline, smooth_columns),
 ("linear", linear_pipeline, linear_columns),
 ("categorical", categorical_pipeline, categorical_columns),
])

spline_logistic = Pipeline([
 ("preprocess", spline_preprocessor),
 ("model", LogisticRegression(C=1.0, max_iter=10_000)),
])
```

`SplineTransformer(n_knots=5, degree=3, include_bias=False)` places five knots across each fitted training feature (quantiles by default) and evaluates overlapping cubic basis functions around them. One original number therefore becomes several basis-column values; with these settings, each smooth input produces `n_knots + degree - 2 = 6` output columns. Four smooth inputs produce 24 spline columns before the linear and categorical blocks are added. Inspect this rather than guessing:

```
spline_only = SplineTransformer(n_knots=5, degree=3, include_bias=False)
demo_basis = spline_only.fit_transform(df[smooth_columns])
print("input shape:", df[smooth_columns].shape)
print("spline-basis shape:", demo_basis.shape)
print("learned knot arrays:", [bs.t for bs in spline_only.bsplines_])
assert demo_basis.shape[1] == 6 * len(smooth_columns)
```

The model learns coefficients for basis columns, not one global slope per original feature. Knot locations must therefore be fitted inside each training fold.

Knot count, degree, penalty, and selected smooth features form a candidate procedure and must be tuned within development data. Curves should be plotted with data support and uncertainty. Splines can extrapolate poorly beyond boundary knots; “smooth” does not mean “scientifically safe.”

## 25.4 K-nearest-neighbour classification

For a query  $x$ , find its  $k$  closest training points and estimate

$$\hat{p}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i,$$

or use distance weights.

KNN makes local predictions without fitting global coefficients. Its assumptions are hidden in the distance:

- which features are included;
- how they are scaled;
- how categorical mismatches are represented;
- whether missingness is meaningful; and
- whether Euclidean closeness corresponds to comparable projects.

Small  $k$  gives flexible, high-variance probabilities in increments of  $1/k$ . Large  $k$  smooths toward the global prevalence.

### Curse of dimensionality

In high dimensions, data become sparse. To capture a fixed fraction of a unit hypercube's volume, a neighbourhood must extend far along each dimension. Distances can concentrate, making “nearest” and “far” less distinct.

KNN therefore requires scaling, feature discipline, adequate local density, and validation. It has no meaningful feature extrapolation beyond observed neighbours.

```
from sklearn.neighbors import KNeighborsClassifier

knn = Pipeline([
 ("preprocess", spline_preprocessor),
 ("model", KNeighborsClassifier(n_neighbors=25, weights="distance")),
])
```

Using spline-expanded features for KNN is only illustrative; it changes the distance geometry. A dedicated KNN preprocessor would usually use scaled, carefully selected original features.

## 25.5 Support-vector classification

For labels  $y_i \in -1, +1$  and score  $f(x) = w^\top x + b$ , a hard-margin support-vector machine seeks a separating hyperplane with large geometric margin. With overlap, soft-margin classification uses hinge loss:

$$L_{\text{hinge}}(y, f) = \max(0, 1 - yf).$$

A common primal objective is

$$\frac{1}{2} \|w\|_2^2 + C \sum_i \max(0, 1 - y_i f(x_i)).$$

- Small  $C$  allows more margin violations and stronger regularisation.
- Large  $C$  penalises violations heavily and can create a more complex boundary.

Only observations on or inside the margin contribute directly to the hinge-loss solution; these are support vectors.

## 25.6 The kernel idea

Why does a kernel appear? Introduce one nonnegative multiplier  $\alpha_i$  for each margin constraint. Minimising the SVM's primal expression over  $w$  gives

$$w = \sum_i \alpha_i y_i x_i.$$

Substituting this representation back into the optimisation removes explicit coordinates of  $w$ . The resulting dual objective contains training observations only through pairwise inner products:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i^\top x_j,$$

subject to bounds on  $\alpha_i$  and  $\sum_i \alpha_i y_i = 0$ . Predictions have the same structure,  $f(x) = \sum_i \alpha_i y_i x_i^\top x + b$ ; only support vectors have nonzero multipliers.

Now the kernel trick is a literal substitution: replace every  $x_i^\top x_j$  with  $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$ . The algorithm behaves as if it constructed possibly many nonlinear features  $\phi(x)$ , while computing only similarities. This buys nonlinear boundaries without explicitly materialising the enlarged feature matrix; it does not remove the need for scaling, tuning, or adequate local support.

The radial-basis-function kernel is

$$K(x, x') = \exp(-\gamma \|x - x'\|_2^2).$$

- Large  $\gamma$  makes similarity decay rapidly and permits local, wiggly boundaries.
- Small  $\gamma$  makes similarity decay slowly and produces smoother boundaries.

Because distance drives the kernel, scaling is essential. Both  $C$  and  $\gamma$  require nested tuning. Kernel SVM training can become expensive as sample size grows.

SVM decision scores are margins, not probabilities. `SVC(probability=True)` fits an additional probability mapping through internal cross-validation and adds computational and calibration assumptions.

```

from sklearn.svm import SVC

rbf_svc = Pipeline([
 ("preprocess", spline_preprocessor),
 ("model", SVC(
 kernel="rbf",
 C=2.0,
 gamma="scale",
 probability=True,
 random_state=425,
)),
])

```

Again, use a purpose-built scaled representation rather than automatically feeding a spline expansion into an RBF kernel. This object merely shows where the estimator belongs.

## 25.7 Multiclass outcomes

Binary classification has two mutually exclusive classes. A multiclass outcome has  $K > 2$  mutually exclusive classes, such as final disposition:

- completed within tolerance;
- completed with major overrun;
- cancelled; or
- unresolved dispute.

Do not manufacture multiclass labels merely to avoid modelling time or competing risks. Each class must have a coherent observation process.

### Softmax regression

For class scores  $z_k = \mathbf{x}^\top \beta_k$ , softmax probabilities are

$$P(Y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}.$$

For numerical stability, subtract  $\max_j z_j$  before exponentiating. Multiclass cross-entropy is

$$-\sum_i \sum_{k=1}^K y_{ik} \log p_{ik}.$$

One class score must be constrained or treated as a reference because adding the same constant to every score leaves probabilities unchanged.

### Numeric three-class example

Suppose one project receives scores  $\mathbf{z} = (2, 1, 0)$ . Subtracting the maximum gives  $(0, -1, -2)$  without changing probabilities:

$$\mathbf{p} = \frac{(e^0, e^{-1}, e^{-2})}{e^0 + e^{-1} + e^{-2}} \approx (0.665, 0.245, 0.090).$$

If the true class is the second class, its one-hot vector is  $\mathbf{y} = (0, 1, 0)$  and the cross-entropy for this row is

$$-\sum_{k=1}^3 y_k \log p_k = -\log(0.245) \approx 1.408.$$

```
import numpy as np

scores = np.array([2.0, 1.0, 0.0])
shifted = scores - scores.max()
probabilities = np.exp(shifted) / np.exp(shifted).sum()
one_hot = np.array([0.0, 1.0, 0.0])
cross_entropy = -(one_hot * np.log(probabilities)).sum()

print(probabilities, cross_entropy)
assert np.isclose(probabilities.sum(), 1.0)
assert np.isclose(cross_entropy, -np.log(probabilities[1]))
```

Raising the true class's score raises its probability and lowers this loss; raising a competing score does the opposite.

Alternative decompositions include one-versus-rest and one-versus-one. Their probability outputs and calibration behaviour differ from joint multinomial softmax.

## 25.8 Macro, micro, and weighted summaries

For multiple classes:

- **macro average:** calculate a metric per class and average classes equally;
- **weighted macro average:** weight class metrics by support;
- **micro average:** pool decisions across all class–case pairs before calculating the metric.

These answer different questions. A micro score can conceal failure on a rare class; a macro score can let a class with ten examples contribute as much as one with ten thousand. Always report per-class support and results.

## 25.9 Multilabel outcomes and classifier chains

In multilabel classification, several labels can be true for one row—for example, geological risk, procurement delay, and community-access risk.

Independent one-versus-rest models ignore label dependence. A classifier chain predicts labels sequentially and uses earlier predicted labels as later features.

During training, feeding true upstream labels to downstream models can create train–deployment mismatch. Use out-of-fold upstream predictions when training later stages, and at deployment feed only available predictions. Chain order is a modelling choice and may change results.

This is the classification analogue of the draft's multi-output regression warning: never feed a true future outcome as though it were a prediction-time feature.

## 25.10 Comparing nonlinear candidates

No algorithm is “advanced” in isolation. Compare complete procedures on:

- proper probability score;
- calibration and ranking;
- threshold utility;
- subgroup stability;

- training and prediction cost;
- sensitivity to scaling and hyperparameters;
- feature support and extrapolation; and
- ease of monitoring and revision.

A spline-logistic model can outperform an RBF SVM when the signal is smooth and data are modest. KNN can be competitive in a low-dimensional, densely sampled region. Complex kernels can fail under shift. Empirical comparison needs a valid design, not a model hierarchy slogan.

## 25.11 Day 25 build, break, and reflect

### Build

1. Fit linear and spline logistic models and plot supported probability curves.
2. compare KNN across  $k$  and feature dimensions after training-only scaling.
3. draw the margin and support vectors for a two-dimensional SVM.
4. tune  $C$  and  $\gamma$  inside temporal validation.
5. calculate classwise, macro, and micro results for a multiclass simulation.

### Break

1. call logistic probability nonlinear evidence of a nonlinear log-odds predictor.
2. fit KNN on kilometres, rupees, and a 1–5 index without scaling.
3. treat an SVM margin as a calibrated probability.
4. select a kernel using the locked test.
5. train a classifier chain with true future labels as downstream features.

### Reflect

Which model assumption is easier to challenge with domain experts: an additive spline, a Euclidean neighbourhood, or an RBF similarity? Interpretability begins with the representation, not only with a post-hoc plot.

### Day 25 exit check

You are ready for trees when you can state the geometry imposed by splines, KNN, and RBF kernels and explain why their probability quality still needs separate validation.

## Day 26 — Classification Trees and Random Forests

**Today's central idea:** Trees create locally constant probability estimates by recursively partitioning feature space. Forests stabilise those partitions through averaging and feature randomness.



## 26.1 A classification tree

At a node containing observations  $S$ , choose a feature and split point that produce child sets  $S_L$  and  $S_R$ . Continue recursively until a stopping rule is reached.

In a binary leaf  $L$ , the unregularised event-probability estimate is

$$\hat{p}_L = \frac{1}{|L|} \sum_{i \in L} y_i.$$

Every query falling in that leaf receives the same estimate. Small leaves can produce extreme, high-variance probabilities.

## 26.2 Gini impurity

For class proportions  $p_1, \dots, p_K$  in a node,

$$I_G = 1 - \sum_{k=1}^K p_k^2.$$

For binary event proportion  $p$ ,

$$I_G = 1 - p^2 - (1 - p)^2 = 2p(1 - p).$$

It is zero in a pure node and largest at  $p = 0.5$ .

One interpretation: if a row's class and a predicted class are independently drawn from the node distribution, Gini impurity is their mismatch probability.

## 26.3 Entropy and log loss

Entropy impurity is

$$I_H = - \sum_{k=1}^K p_k \log p_k,$$

with  $0 \log 0$  defined as zero. It is also zero in a pure node and largest for a uniform class distribution.

Gini and entropy often choose similar splits but are not identical. The criterion is one hyperparameter among depth, minimum leaf size, feature availability, and pruning.

## 26.4 Weighted impurity decrease

For parent set  $S$  and children  $S_L, S_R$ , split improvement is

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R).$$

Choose the candidate split with largest decrease, subject to constraints.

### Hand calculation

Suppose a parent has 10 projects: 4 events and 6 non-events. Its Gini impurity is

$$1 - 0.4^2 - 0.6^2 = 0.48.$$

A split produces:

- left: 4 projects, 3 events;
- right: 6 projects, 1 event.

Then

$$I_L = 1 - 0.75^2 - 0.25^2 = 0.375,$$

$$I_R = 1 - (1/6)^2 - (5/6)^2 \approx 0.278.$$

The weighted child impurity is

$$0.4(0.375) + 0.6(0.278) \approx 0.317,$$

so the decrease is about  $0.48 - 0.317 = 0.163$ .

## 26.5 A decision stump from scratch

This one-dimensional implementation searches all midpoint splits and returns leaf probabilities.

```
import numpy as np

def gini(y):
 if len(y) == 0:
 return 0.0
 p = np.mean(y)
 return 2.0 * p * (1.0 - p)

class OneDimensionalStump:
 def fit(self, x, y):
 x = np.asarray(x, dtype=float)
 y = np.asarray(y, dtype=int)
 unique = np.unique(x)
 if len(unique) < 2:
 raise ValueError("x needs at least two distinct values")

 thresholds = (unique[:-1] + unique[1:]) / 2.0
 best = None
 for threshold in thresholds:
 left = x <= threshold
 right = ~left
 weighted = (
 left.mean() * gini(y[left])
 + right.mean() * gini(y[right])
)
 if best is None or weighted < best[0]:
 best = (weighted, threshold, left)

 , self.threshold, left = best
 self.left_probability_ = y[left].mean()
 self.right_probability_ = y[~left].mean()
 return self

 def predict_proba(self, x):
 x = np.asarray(x, dtype=float)
 positive = np.where(
 x <= self.threshold_,
 self.left_probability_,
 self.right_probability_,
```

```
)
return np.column_stack([1.0 - positive, positive])
```

Test it against the hand example and inspect every candidate threshold rather than treating recursive partitioning as magic.

## 26.6 Tree complexity

Important controls include:

- `max_depth`: longest root-to-leaf path;
- `min_samples_leaf`: minimum observations in a leaf;
- `min_samples_split`: minimum observations required to consider splitting;
- `max_leaf_nodes`: direct leaf-count limit; and
- `ccp_alpha`: cost-complexity pruning strength.

A fully grown tree can memorise idiosyncrasies. Larger leaves often improve probability stability even if accuracy changes little. Select complexity inside validation and inspect leaf event counts.

## 26.7 What trees do and do not handle automatically

Trees are invariant to strictly monotone transformations of an individual numeric feature because split order is preserved. They do not need standardisation for that reason.

However:

- categorical support differs across libraries;
- one-hot encoding can change how a multi-level category is split;
- missing-value routing is estimator-specific;
- high-cardinality identifiers invite memorisation;
- rare categories create tiny leaves; and
- arbitrary integer coding can impose unintended order.

“Trees handle everything” is not a data contract.

## 26.8 Feature-space extrapolation

A tree predicts using the leaf reached by comparisons learned in the training range. Moving farther beyond the largest observed capacity often leaves the query in the same outer leaf, so the prediction becomes flat.

For regression trees, averaged leaf targets cannot exceed the training-target extrema. In binary classification, probabilities are averages of zero/one labels in leaves or ensembles, so the more relevant limitation is that risk cannot evolve through an unobserved feature region except by the already learned partition.

This is not automatically safer than linear extrapolation. It is a different unsupported behaviour.

## 26.9 Bagging

A deep tree has low bias but high sample variance. Bootstrap aggregating—bagging—fits  $M$  trees on bootstrap samples and averages their probabilities:

$$\hat{p}_{bag}(x) = \frac{1}{M} \sum_{m=1}^M \hat{p}_m(x).$$

If individual predictions have variance  $\sigma^2$  and pairwise correlation  $\rho$ , an idealised average has variance

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2.$$

Adding trees reduces the second term, but not the correlated component. Diversity matters.

## 26.10 Random forests

A random forest adds feature subsampling: at each split, only a random subset of predictors is considered. Strong predictors cannot dominate every tree, reducing correlation and allowing alternative structures to contribute.

Important parameters include:

- number of trees;
- number or fraction of candidate features per split;
- minimum leaf size;
- maximum depth;
- bootstrap sample size; and
- class weighting.

More trees mainly reduce Monte Carlo variation; they do not guarantee correction of bias, leakage, or miscalibration.

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(
 n_estimators=600,
 max_features="sqrt",
 min_samples_leaf=12,
 bootstrap=True,
 oob_score=True,
 random_state=426,
 n_jobs=-1,
)
```

## 26.11 Out-of-bag prediction

A bootstrap sample of size  $n$  contains about  $1 - e^{-1} \approx 63.2\%$  distinct training observations on average. The rest are out of bag for that tree.

For each training row, average predictions only from trees whose bootstrap samples excluded it. These out-of-bag predictions can estimate internal performance and support diagnostics without a separate random validation split.

But OOB evaluation:

- assumes an exchangeability structure similar to ordinary bootstrap sampling;
- does not respect time automatically;
- can leak groups across trees; and
- does not replace an external or future test.

For MHP temporal deployment, forward validation remains primary.

## 26.12 Forest probability calibration

Forest probabilities average leaf proportions across trees. Minimum leaf size, class weighting, bootstrap design, and averaging all affect their spread. Good ranking does not guarantee calibrated risks.

Check log loss, Brier score, reliability, and calibration slope on untouched folds. If calibration is needed, include the calibrator and its data split in the candidate procedure.

## 26.13 Impurity importance is not enough

The sum of impurity decreases credited to a feature is fast but can favour variables with many potential split points or high cardinality. Correlated features can divide or mask credit.

Use held-out permutation importance and conditional reasoning in Day 28. No importance score establishes causality.

## 26.14 Research paper study: Breiman (2001)

Leo Breiman's *Random Forests* formalised forests of randomised tree predictors and related generalisation error to individual-tree strength and correlation.

Read with five questions.

1. **Definition.** What randomness generates a forest, and over what is prediction averaged?
2. **Bound.** How are strength and correlation connected to classification error?
3. **OOB evidence.** How are out-of-bag cases used for error and importance?
4. **Experiments.** Which baselines, datasets, and tuning choices are compared?
5. **Modern boundary.** Which probability-calibration, shift, fairness, and selection questions are outside the paper's central scope?

### Replication task

Generate noisy copies of one strong predictor. Compare forests with all features available at each split versus `max_features="sqrt"`. Across repeated samples, measure tree correlation, individual-tree accuracy, ensemble log loss, and ROC AUC.

## 26.15 Day 26 build, break, and reflect

### Build

1. Calculate every candidate Gini split for a ten-row dataset.
2. compare a deep tree with increasing minimum leaf sizes.
3. plot validation log loss against number of forest trees and leaf size.
4. compare OOB estimates with forward validation and explain the mismatch.
5. audit the forest's reliability by year and access mode.

### Break

1. allow leaves with one event and report 0 or 1 as certain risk.
2. assume every tree implementation handles strings and missing values natively.
3. rely on OOB performance for a future-year claim.
4. interpret impurity importance as causal influence.
5. add thousands of trees to repair systematic bias.

## Reflect

When a forest outperforms logistic regression, which new structure did it exploit: thresholds, interactions, local effects, or leakage? Performance alone does not answer.

## Day 26 exit check

You are ready for boosting when you can derive impurity reduction, explain forest variance through tree correlation, and state why OOB evidence may not match deployment.

# Day 27 — Gradient Boosting and Honest Tuning

**Today's central idea:** Gradient boosting performs stagewise optimisation in function space. Its flexibility makes the validation and search procedure part of the model.

## 27.1 Bagging and boosting solve different problems

Bagging fits unstable learners largely in parallel and averages them to reduce variance. Gradient boosting builds an additive model sequentially:

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^M h_m(x),$$

where:

- $F_m$  is a raw score function;
- $h_m$  is a weak learner, often a shallow tree;
- $M$  is number of boosting stages; and
- $\eta$  is the learning rate.

Each stage approximates a descent direction for the chosen loss.

## 27.2 Gradient descent in function space

Ordinary gradient descent changes a finite parameter vector. Gradient boosting changes a function.

Given loss  $L(y, F(x))$ , calculate the negative gradient at current predictions:

$$r_{im} = - \left[ \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}.$$

Fit weak learner  $h_m(x)$  to these pseudo-residuals, then update:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x),$$

possibly with a stage-specific line-search multiplier.

For squared-error regression, negative gradients are ordinary residuals. The draft's “fit residuals” description is therefore a special case, not the definition of boosting.

## 27.3 Logistic boosting pseudo-residuals

For binary classification, let  $F(x)$  be log odds and

$$p(x) = \sigma(F(x)).$$

One-observation logistic loss is

$$L(y, F) = -y \log p - (1 - y) \log(1 - p).$$

As in Day 22,

$$\frac{\partial L}{\partial F} = p - y.$$

The negative gradient is therefore

$$\boxed{r = y - p}.$$

At each stage, a regression tree approximates the current probability errors as a function of  $\mathbf{X}$ . Implementations can use second-order information and specialised leaf updates, but  $y - p$  is the essential first-order direction.

## 27.4 Initial score

The constant score minimising binary log loss is the training log odds:

$$F_0 = \log \left( \frac{\hat{\pi}}{1 - \hat{\pi}} \right).$$

Its sigmoid is the training prevalence. Boosting therefore begins at the constant probability baseline and adds structure stage by stage.

```
import numpy as np

def initial_log_odds(y):
 prevalence = np.clip(np.mean(y), 1e-12, 1.0 - 1e-12)
 return np.log(prevalence / (1.0 - prevalence))

def logistic_pseudo_residual(y, raw_score):
 probability = stable_sigmoid(raw_score)
 return np.asarray(y) - probability
```

## 27.5 Learning rate, depth, and stages interact

Key controls include:

- **learning rate  $\eta$** : contribution of each new tree;
- **number of iterations  $M$** : length of the additive expansion;
- **leaf count or depth**: interaction order and local complexity per tree;
- **minimum leaf size**: local sample support;
- **row subsampling**: stochastic-gradient diversity; and

- **feature subsampling or regularisation:** implementation-specific restraint.

A smaller learning rate usually requires more stages. Deep trees can learn high-order interactions quickly and overfit. There is no universally optimal range such as 0.01–0.1; ranges are starting hypotheses tied to a package, dataset, and budget.

## 27.6 Histogram gradient boosting

Histogram methods bin continuous feature values and search splits over bins rather than every unique value. This can greatly accelerate training and reduce memory.

`HistGradientBoostingClassifier` supports missing numeric values natively, but raw strings still require an appropriate categorical representation unless categorical support is configured with compatible encoded data. Read current software documentation for the exact version.

```
from sklearn.ensemble import HistGradientBoostingClassifier

boosted = HistGradientBoostingClassifier(
 learning_rate=0.05,
 max_iter=400,
 max_leaf_nodes=15,
 min_samples_leaf=20,
 l2_regularization=1.0,
 early_stopping=False,
 random_state=427,
)
```

Libraries such as XGBoost, LightGBM, and CatBoost implement related but not identical algorithms, objectives, missing-value rules, categorical strategies, and regularisation. Brand names are not interchangeable scientific methods.

## 27.7 Early stopping is another validation operation

Early stopping chooses the effective number of boosting stages based on validation loss. If the estimator silently takes a random validation fraction, a temporal study can train on later rows and stop using earlier rows, violating chronology.

Valid options include:

- supply an order-respecting validation period when the API permits it;
- tune `max_iter` as a hyperparameter in outer temporal validation with internal early stopping disabled; or
- implement an expanding-window inner selection.

After selecting iteration count, refit according to a pre-specified rule. Reusing the locked test for early stopping makes it training data.

## 27.8 Grid, random, and sequential search

### Grid search

Evaluate every point in a specified Cartesian product. It is transparent but spends many trials varying unimportant dimensions and gives an illusion of coverage between grid points.



## Random search

Sample configurations from stated distributions. It often explores more distinct values in influential dimensions for the same budget. Log-uniform distributions are appropriate for many scale parameters.

## Bayesian or sequential optimisation

Build a surrogate of configuration performance and select promising future trials using an acquisition rule. It can use trial budget efficiently, but:

- the surrogate and acquisition function add assumptions;
- parallelism and noise matter;
- conditional search spaces complicate comparison;
- it can still overfit validation; and
- “Bayesian” does not make the final error estimate Bayesian or unbiased.

Record the search space, sampling distributions, seed, number of trials, failures, stopping rule, and every score—not only the winner.

## 27.9 Search distributions matter

Sampling  $C$  uniformly between 0.0001 and 1000 places almost all mass near large values. Sample its logarithm instead:

$$\log_{10} C \sim \text{Uniform}(-4, 3).$$

For tree depth, a small discrete set may be sensible. For learning rate, a log-uniform range is often more meaningful than a linear range. Domain and computational limits should bound the search.

```
from scipy.stats import loguniform, randint, uniform
from sklearn.model_selection import RandomizedSearchCV, TimeSeriesSplit

parameter_distributions = {
 "model__learning_rate": loguniform(0.01, 0.3),
 "model__max_leaf_nodes": randint(4, 32),
 "model__min_samples_leaf": randint(10, 80),
 "model__l2_regularization": loguniform(1e-3, 30.0),
 "model__max_iter": randint(100, 700),
}

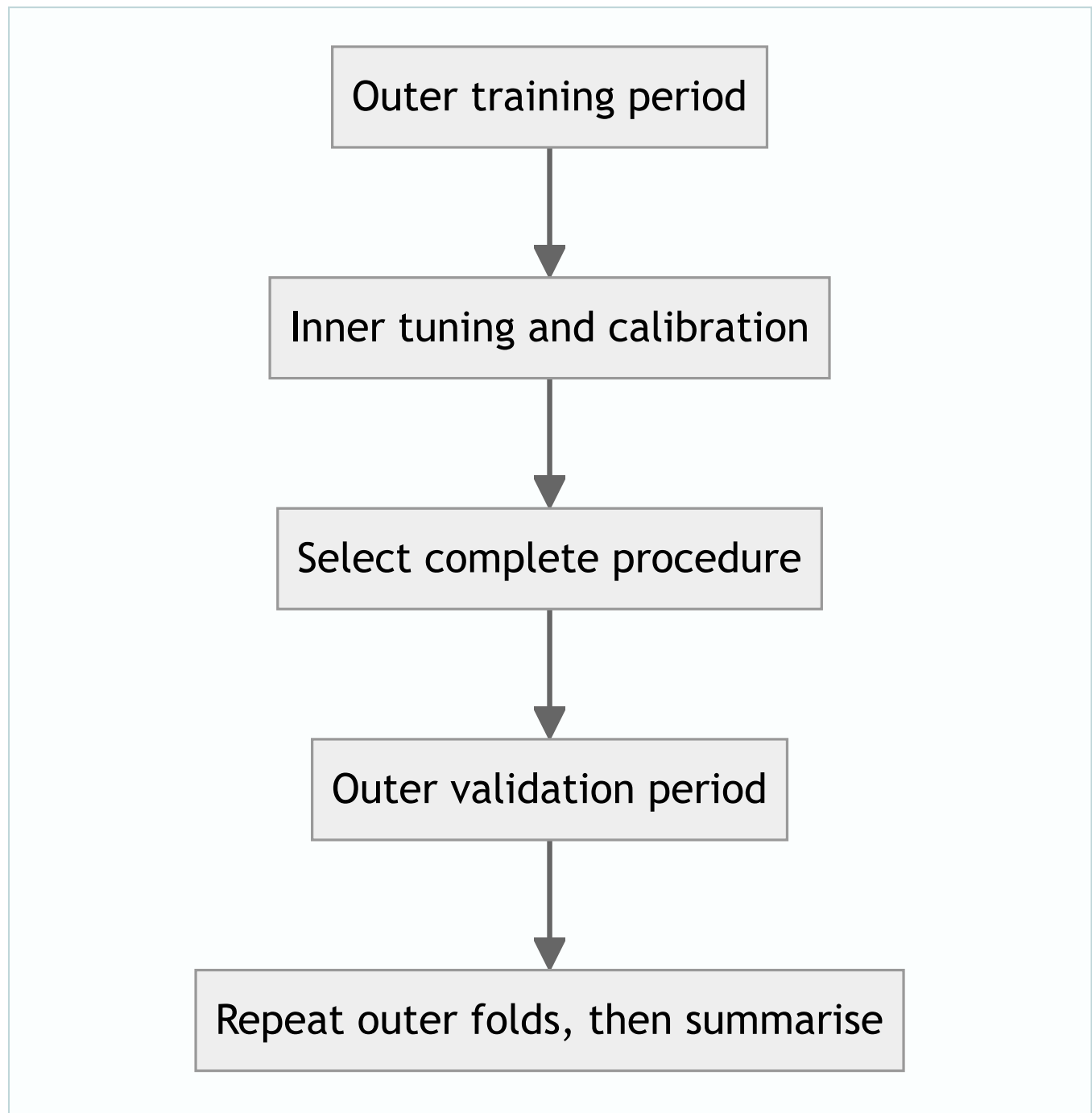
temporal_inner = TimeSeriesSplit(n_splits=4)
search = RandomizedSearchCV(
 estimator=boosting_pipeline,
 param_distributions=parameter_distributions,
 n_iter=40,
 scoring="neg_log_loss",
 cv=temporal_inner,
 random_state=427,
 n_jobs=-1,
 refit=True,
)
```

`TimeSeriesSplit` assumes rows are correctly ordered and uses row positions. For projects sharing years, a custom year-based splitter is safer so one calendar year is not divided between fit and validation.

The imported `uniform` is not used above; it is shown because linear distributions remain appropriate for some bounded fractions. Remove unused imports in production code.

## 27.10 Nested evaluation

When no separate locked test is available, nested validation estimates the performance of a selection procedure.



Every adaptive choice belongs inside the inner loop:

- preprocessing;
- feature selection;
- resampling;
- hyperparameters;
- early stopping;
- calibration method; and
- decision threshold, if threshold performance is evaluated.

The outer folds must match deployment structure. Random nested CV does not cure temporal or group mismatch.

## 27.11 Tuning budget is a source of unfair comparison

Comparing a default logistic model with a boosted model receiving 1,000 trials does not isolate algorithm quality. Search effort is part of the procedure.

Fair benchmarks report:

- candidate families;
- search spaces;
- number of trials;
- time and hardware budget;
- early-termination rules;
- failed configurations; and
- whether knowledge from other datasets shaped the search.

Compute-aware comparison can report both best performance and performance as a function of search time.

## 27.12 Statistical uncertainty after search

The winning validation score is the minimum of noisy estimates and is optimistically selected. A test or outer loop evaluates the selected procedure, but comparisons can still be uncertain.

Report:

- all outer-fold scores;
- paired differences on the same folds or test rows;
- uncertainty aligned with the sampling unit;
- number of tried procedures; and
- practical rather than merely numerical differences.

Do not run a significance test across hundreds of inner trials and treat the best-versus-second-best result as confirmatory.

## 27.13 Research paper study: Friedman (2001)

Jerome Friedman's *Greedy Function Approximation: A Gradient Boosting Machine* views stagewise additive modelling as numerical optimisation in function space and develops gradient boosting for several losses.

Read with five questions.

1. **Abstraction.** How does a parameter vector become a function estimate?
2. **Direction.** What role do negative gradients play for arbitrary differentiable loss?
3. **Base learner.** Why do small regression trees create useful interaction structures?
4. **Regularisation.** How do shrinkage and subsampling affect performance?
5. **Evidence.** Which simulated and empirical settings support the claims, and how was tuning performed?

### Replication task

Implement logistic boosting with one-dimensional stumps. Start at log prevalence, calculate  $\mathbf{y} - \mathbf{p}$ , fit a stump to pseudo-residuals, and update raw scores. Plot training and validation log loss over 500 stages for several learning rates.

## 27.14 Companion paper: Grinsztajn, Oyallon, and Varoquaux (2022)

*Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?* reports a large benchmark over medium-sized tabular datasets and investigates inductive biases such as robustness to uninformative features and irregular target functions.

Do not retain only the slogan “trees win tabular data.” Audit:

- dataset inclusion and size range;
- tuning budget and baselines;
- preprocessing;
- task and metric aggregation;
- compute accounting; and
- domains not represented by the benchmark.

### Replication task

Choose five open tabular classification datasets before seeing results. Give logistic regression, forest, boosting, and a small neural network equal tuning budgets. Publish every dataset-level result and a critical account of preprocessing and failed trials.

## 27.15 Day 27 build, break, and reflect

### Build

1. Derive logistic pseudo-residuals.
2. implement a stump-based boosting loop.
3. compare learning-rate/iteration pairs under forward validation.
4. define log-scale and discrete search distributions before running them.
5. draw the exact inner and outer time boundaries for tuning, calibration, and thresholding.

### Break

1. describe classification boosting as fitting ordinary target residuals without naming the loss.
2. let an estimator create a random early-stopping split in a temporal study.
3. tune hundreds of models and report the minimum inner score as final error.
4. compare algorithms using radically different search budgets without disclosure.
5. use the locked test to decide when boosting should stop.

### Reflect

At what point does additional search cease to be a better model and become adaptation to one validation sample? A locked outer evaluation estimates the consequence; it does not erase the cost of a broad search.

### Day 27 exit check

You are ready for interpretation when you can derive  $y - p$  as a negative gradient, place early stopping inside the validation boundary, and describe the complete search procedure rather than only its winning parameters.

---

# Day 28 — Interpreting Fitted Models Without Inventing Causes

**Today's central idea:** An explanation method describes a fitted prediction under a chosen perturbation or reference. It does not reveal nature's causal mechanism.

## 28.1 Begin with the explanation question

“Explain the model” is underspecified. Ask:

- **Object:** one prediction, average model behaviour, or performance dependence?
- **Audience:** developer, project reviewer, policymaker, or affected community?
- **Scale:** probability, log odds, margin, loss, or action?
- **Reference:** compared with which observation or background distribution?
- **Scope:** this fitted model, this dataset, or the real-world outcome mechanism?
- **Use:** debugging, scientific understanding, contestability, or justification?

Different methods answer different questions. A local additive explanation cannot replace a global error audit, and neither establishes causality.

## 28.2 Intrinsic and post-hoc interpretability

An interpretable-by-design model exposes a constrained form, such as:

- logistic coefficients with pre-specified features;
- a sparse scorecard;
- a shallow tree;
- an additive shape model; or
- a monotonic constrained model.

Post-hoc methods interrogate a fitted model after training. They can be valuable, but their approximation and perturbation choices add another model layer.

Intrinsic form is not automatically truthful. A simple misspecified model can be easy to read and wrong. A post-hoc explanation is not automatically deceptive. Match fidelity, complexity, and claim to the use.

## 28.3 Coefficients are conditional model parameters

For a logistic model,  $e^{\beta_j}$  is a conditional odds ratio per unit of  $x_j$  under the represented specification. It is affected by:

- scaling and units;
- reference coding;
- interactions and nonlinear terms;
- regularisation;
- omitted variables;
- measurement error; and
- sampling design.

Coefficient magnitude is not a general feature-importance measure. A one-unit change in terrain and one million PKR change in budget are incomparable without a defined perturbation.

## 28.4 Permutation importance

Let  $S(f, D)$  be a score for fitted model  $f$  on held-out dataset  $D$ . Permute feature  $j$  across rows to create  $D_j^{perm}$ . For a higher-is-better score,

$$I_j^{perm} = S(f, D) - S(f, D_j^{perm}).$$

For a loss, reverse the sign or report loss increase.

The question is:

How much does this fitted model's held-out performance deteriorate when the information in this column is disrupted by this permutation scheme?

It is not:

How much does this feature cause the outcome?

```
import pandas as pd
from sklearn.inspection import permutation_importance

result = permutation_importance(
 fitted_model,
 X_validation,
 y_validation,
 scoring="neg_log_loss",
 n_repeats=40,
 random_state=428,
 n_jobs=-1,
)

importance = pd.DataFrame({
 "feature": X_validation.columns,
 "mean_score_decrease": result.importances_mean,
 "sd": result.importances_std,
}).sort_values("mean_score_decrease", ascending=False)
print(importance)
```

This direct feature-name alignment works when the fitted pipeline accepts those original columns. If importance is calculated after transformation, obtain transformed names from the fitted preprocessor.

## 28.5 Correlated-feature problem

If two features carry redundant information, permuting one may barely hurt because the other remains. Both can appear unimportant even though the pair is essential.

Unconditional permutation can also create impossible combinations, such as short surveyed route paired with extreme access mode. Options include:

- permuting correlated groups together;
- conditional permutation within plausible strata;
- reporting feature clusters rather than individual columns; and
- comparing performance after retraining without a feature, while recognising that this answers a different question.

Conditional permutation requires modelling a feature's conditional distribution and introduces additional assumptions.

## 28.6 Partial dependence

Partition features into focal  $X_S$  and complement  $X_C$ . The partial dependence function is

$$PD_S(x_S) = \mathbb{E}_{X_C}[f(x_S, X_C)].$$

Empirically,

$$\widehat{PD}_S(x_S) = \frac{1}{n} \sum_{i=1}^n f(x_S, x_{iC}).$$

For each grid value, replace the focal feature for every row, predict, and average. Other features retain their row-specific values; they are not all “held constant” at one common value.

```
import matplotlib.pyplot as plt
from sklearn.inspection import PartialDependenceDisplay

PartialDependenceDisplay.from_estimator(
 fitted_model,
 X_validation,
 features=["planned_capacity_kw", "road_distance_observed_km"],
 kind="average",
 grid_resolution=40,
)
plt.tight_layout()
plt.show()
```

Specify whether the plotted response is class probability, decision function, or another output.

## 28.7 PDP and unsupported combinations

If planned capacity and appraisal budget are strongly related, replacing capacity with 850 kW while retaining a small-project budget creates off-support rows. The average then describes model behaviour on synthetic combinations that may never occur.

Before interpreting PDP:

- inspect joint support;
- restrict grids to observed ranges or meaningful quantiles;
- mark data density;
- compare with conditional methods; and
- avoid causal language.

A smooth plot can be a precise summary of an implausible intervention.

## 28.8 Individual conditional expectation

An ICE curve for row  $i$  is

$$ICE_i(x_S) = f(x_S, x_{iC}).$$

PDP averages ICE curves. Plotting ICE reveals heterogeneity that the average can hide. Diverging curves suggest model interactions with other features.

```
PartialDependenceDisplay.from_estimator(
 fitted_model,
 X_validation.sample(min(120, len(X_validation)), random_state=428),
 features=["planned_capacity_kw"],
 kind="both",
 subsample=120,
 random_state=428,
)
plt.tight_layout()
plt.show()
```

ICE is still a model perturbation. It does not make a within-row causal experiment.

## 28.9 Accumulated local effects

Accumulated local effects (ALE) divide a feature range into intervals, estimate local prediction differences while staying nearer observed joint combinations, then accumulate and centre those differences.

Conceptually, for a continuous feature  $x_j$ :

1. find rows whose observed  $x_j$  lies in interval  $k$ ;
2. predict each row at the interval's lower and upper boundaries;
3. average those local differences;
4. accumulate across intervals; and
5. centre the curve.

ALE often behaves better than PDP with correlated features, but depends on interval choice, local support, and a meaningful perturbation. Sparse regions remain uncertain.

## 28.10 Local surrogate explanations

A local surrogate samples or perturbs points around a query, obtains black-box predictions, weights points by proximity, and fits a simple model.

Its explanation depends on:

- neighbourhood definition;
- feature scaling;
- perturbation distribution;
- proximity kernel;
- surrogate family; and
- random seed.

Report local fidelity. A simple line that poorly approximates the model is not an explanation merely because it is readable.

## 28.11 Shapley values

In cooperative game theory, players form coalitions and receive a value  $v(S)$ . The Shapley value for feature  $j$  is



$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(M - |S| - 1)!}{M!} [v(S \cup \{j\}) - v(S)],$$

where  $M$  is the number of features. It averages feature  $j$ 's marginal contribution over all orders in which features could join.

For additive SHAP explanations,

$$f(x) = \phi_0 + \sum_{j=1}^M \phi_j$$

on the explainer's output scale, where  $\phi_0$  is an expected or reference output determined by the background and formulation.

## 28.12 A two-feature Shapley calculation

Let players be budget  $B$  and road access  $R$ . Define:

$$v(\emptyset) = 0.10, \quad v(\{B\}) = 0.18, \quad v(\{R\}) = 0.25, \quad v(\{B, R\}) = 0.40.$$

Budget's marginal contribution is:

- $0.18 - 0.10 = 0.08$  when it enters first;
- $0.40 - 0.25 = 0.15$  when it enters after road access.

With two players, average the two orders:

$$\phi_B = \frac{1}{2}(0.08 + 0.15) = 0.115.$$

For road access:

$$\phi_R = \frac{1}{2}[(0.25 - 0.10) + (0.40 - 0.18)] = 0.185.$$

Then

$$0.10 + 0.115 + 0.185 = 0.40.$$

The arithmetic is exact for the stated game. The difficult modelling question is how  $v(S)$  is defined when a feature is "absent."

## 28.13 Background and dependence choices

For tabular prediction, omitted-feature values must be integrated or conditioned over a background distribution. With dependent features, different choices answer different questions.

- **Interventional-style:** break some dependencies and ask how the model behaves under feature replacement.
- **Conditional-style:** average over feature values compatible with observed dependencies.

Neither is automatically causal. Conditional explanations can assign credit to a proxy because it predicts an omitted feature. Interventional explanations can create implausible combinations.

State:

- explainer algorithm;

- background sample and population;
- feature-dependence assumption;
- output scale;
- class explained; and
- approximation settings.

## 28.14 Probability versus log-odds explanation

Tree SHAP and other explainers may produce additive values on raw margin or log-odds scale rather than probability. On log odds,

$$F(x) = \phi_0 + \sum_j \phi_j, \quad p(x) = \sigma(F(x)).$$

Individual log-odds contributions do not convert into independent probability-point contributions because the sigmoid is nonlinear. Always label the scale.

## 28.15 Explanation stability

An explanation can change under:

- a new random seed;
- a near-tied model;
- correlated-feature substitution;
- another background sample;
- a small row perturbation; or
- a later training period.

Measure stability when explanations affect decisions. Compare explanations across temporal folds, bootstrap fits, and plausible background definitions. Instability may be the most important explanation result.

## 28.16 Explanations can legitimise a bad system

A colourful local plot can distract from:

- an invalid label;
- excluded populations;
- poor calibration;
- discriminatory allocation;
- leakage;
- lack of recourse; or
- no evidence that the intervention helps.

Model explanation belongs after data, evaluation, and decision audits—not in place of them.

## 28.17 Research paper study: Lundberg and Lee (2017)

Scott Lundberg and Su-In Lee's *A Unified Approach to Interpreting Model Predictions* defines a class of additive feature-attribution methods and connects several explanation approaches through Shapley-based properties.

Read with five questions.

1. **Class.** What form defines an additive feature-attribution method?
2. **Properties.** How are local accuracy, missingness, and consistency stated?
3. **Uniqueness.** Under which formulation is the Shapley solution unique?
4. **Algorithms.** Which model-specific approximations improve computation?
5. **Boundary.** Which conclusions concern attribution axioms rather than causal truth, fairness, or human understanding?

## Replication task

Fit two equally accurate models that use correlated copies of the same signal differently. Compute permutation importance, PDP, and SHAP under two background samples. Report which conclusions are stable and which are artifacts of fitted representation or reference choice.

## 28.18 Day 28 build, break, and reflect

### Build

1. write the object, audience, scale, reference, and use for an explanation request.
2. compare individual and grouped permutation importance for correlated features.
3. overlay PDP, ICE, data density, and an ALE curve.
4. calculate the two-feature Shapley example by both feature orders.
5. compare SHAP values on log-odds and probability outputs where supported.
6. repeat explanations across temporal folds.

### Break

1. call permutation importance causal effect.
2. interpret a PDP through unsupported feature combinations.
3. describe SHAP baseline as “the global average” without specifying background or scale.
4. hide low local-surrogate fidelity.
5. use an explanation dashboard to excuse poor calibration or subgroup harm.

### Reflect

What should an affected project team be allowed to contest: the input record, event definition, probability, threshold, explanation, or all five? Interpretability without contestability may provide comfort only to the model owner.

## Day 28 exit check

You are ready for the accountability study when you can define the perturbation behind each explanation, calculate a Shapley value, and state why predictive attribution is not causal attribution.

---

# Day 29 — Fairness, External Validation, and the Locked Study

**Today's central idea:** A technically strong classifier is not deployment-ready until its errors, interventions, and evidence boundaries are examined across people, places, and time.

## 29.1 Fairness begins before a parity formula

Ask first:

- Is the prediction task itself legitimate?
- Is the event label measured consistently across groups?
- Does a positive action provide support or impose burden?
- Who is absent because labels or features are unavailable?
- Can a person or project contest the record and decision?
- Does the intervention improve the outcome?
- Which institution is accountable for errors?

A parity metric can diagnose one distributional property. It cannot answer these questions alone.

## 29.2 Protected and operational groups

Let  $A$  denote a group attribute. It might be legally protected, ethically salient, geographically operational, or a proxy for unequal conditions.

Do not infer that dropping  $A$  creates fairness. Other variables can proxy it, and historical labels can encode past allocation. Conversely, retaining  $A$  for auditing does not require using it for prediction.

Document separately:

- features used by the model;
- attributes retained for auditing;
- attributes used by the decision policy; and
- legal or governance constraints on each.

## 29.3 Demographic parity

Demographic parity asks for equal positive-action rates:

$$P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b).$$

It can be relevant when the action is a scarce benefit or burden. It does not condition on the observed outcome and can conflict with other goals when event rates differ.

In the MHP case, equal review rates may or may not be appropriate. If review is supportive and risk differs because remote projects face structural barriers, equal rates could under-support one group. The policy meaning matters.

## 29.4 Equal opportunity and equalised odds

Equal opportunity asks for equal true-positive rates:

$$P(\hat{Y} = 1 \mid Y = 1, A = a) = P(\hat{Y} = 1 \mid Y = 1, A = b).$$

Among projects that truly overrun, groups should have equal chance of receiving review.

Equalised odds additionally asks for equal false-positive rates:

$$\hat{Y} \text{ is independent of } A \text{ given } Y.$$

That means both TPR and FPR are equal across groups. These are properties of a thresholded action, not probability calibration.

## 29.5 Predictive parity and group calibration

Predictive parity asks for equal precision:

$$P(Y = 1 \mid \hat{Y} = 1, A = a) = P(Y = 1 \mid \hat{Y} = 1, A = b).$$

Group calibration asks, roughly, that within each group, cases assigned risk  $p$  experience events at rate  $p$ :

$$P(Y = 1 \mid P = p, A = a) = p.$$

These properties concern different conditionals. When base rates differ and prediction is imperfect, several desirable parity conditions cannot generally all hold simultaneously. Choosing a definition is a normative and operational decision, not metric shopping.

## 29.6 Measurement can manufacture a disparity

Suppose remote projects close accounts more slowly. If unresolved cases are coded negative, measured TPR and prevalence can differ even when true event processes do not. Fairness auditing therefore requires:

- label completeness by group;
- feature missingness by group;
- follow-up duration by group;
- measurement error and appeal outcomes; and
- subgroup sample sizes.

An algorithmic parity adjustment applied to a biased label can preserve or intensify the underlying harm.

## 29.7 Intersectional and small-group uncertainty

Average results for `access_mode` can hide variation within district, year, or contractor context. Intersectional audits are important, but slicing creates tiny samples and multiple-comparison risk.

Report:

- raw counts and confusion cells;
- uncertainty intervals or posterior distributions where justified;
- pre-specified versus exploratory groups;
- suppression rules for privacy; and
- a data-collection plan for underrepresented groups.

“Insufficient evidence” is not evidence of equality.

## 29.8 Internal, temporal, geographic, and prospective validation

| Validation type     | What changes?                          | Claim supported                                                |
|---------------------|----------------------------------------|----------------------------------------------------------------|
| Internal random     | sampled rows within one source         | performance under approximate exchangeability                  |
| Internal grouped    | whole known groups held out            | transfer to comparable unseen groups under the grouping design |
| Temporal            | later time period                      | performance after observed time shift in the same broad system |
| Geographic/external | independent sites or institutions      | transport to represented external settings                     |
| Prospective         | model fixed before future cases accrue | future workflow performance under actual data collection       |
| Impact evaluation   | decisions/interventions compared       | effect of using the system, under a causal design              |

A future-year test in the same five fictional districts is temporal validation, not broad external validation. A prospective prediction study still does not establish that acting on the model improves outcomes.

## 29.9 Dataset shift

Let training distribution be  $P_{train}(X, Y)$  and deployment distribution  $P_{deploy}(X, Y)$ .

Common decompositions include:

- **covariate shift:**  $P(X)$  changes while  $P(Y | X)$  is assumed stable;
- **label shift:**  $P(Y)$  changes while  $P(X | Y)$  is assumed stable; and
- **concept or conditional shift:**  $P(Y | X)$  changes.

These are assumptions, not labels a drift dashboard can prove from unlabeled deployment features alone. Monitor:

- feature distributions and missingness;
- predicted-risk distribution;
- event prevalence after labels mature;
- calibration and decision rates;
- subgroup metrics;
- data pipeline changes; and
- human override and appeal patterns.

## 29.10 The pre-specified Chapter 4 study

Freeze the following protocol before fitting candidates.

| Component            | Choice                                                |
|----------------------|-------------------------------------------------------|
| Unit                 | one eligible fictional MHP project                    |
| Prediction time      | technical appraisal                                   |
| Event                | final cost > <b>115%</b> of approved appraisal budget |
| Development          | start years through 2022                              |
| Locked temporal test | start years 2023–2025                                 |

| Component                    | Choice                                                           |
|------------------------------|------------------------------------------------------------------|
| Forward validation           | validation years 2019–2022                                       |
| Primary metric               | log loss, averaged equally across years                          |
| Secondary probability metric | Brier score                                                      |
| Ranking metrics              | ROC AUC and average precision                                    |
| Action                       | senior review if $p \geq 1/6$                                    |
| Simplified costs             | FP cost 1; FN cost 5                                             |
| Groups                       | access mode and district, with raw counts                        |
| Candidate families           | prior, logistic, engineered logistic, forest, histogram boosting |
| Test use                     | one pre-specified evaluation after selection                     |

The  $1/6$  threshold follows the simplified cost ratio. It is a teaching policy, not a real economic valuation.

## 29.11 End-to-end locked benchmark

### How to read the capstone pipeline

The long block below combines familiar pieces under one frozen protocol. Read it in this order before executing it:

1. **Constants** freeze the target, years, costs, threshold, and seed.
2. **ClassificationFeatureEngineer** follows scikit-learn's transformer contract: `fit` learns training-only centres and returns `self`; `transform` applies those saved values to any later frame.
3. **ColumnTransformer** sends named column groups through different preprocessing paths and concatenates their outputs.
4. **Candidate pipelines** bind preprocessing to each estimator so validation folds cannot leak fitted transformations.
5. **clone(candidate)** creates a fresh unfitted object with the same settings for every fold; fitted state is never carried between folds.
6. **The locked test** is evaluated once only after the development procedure selects a candidate.

After executing the class definition in the block below, pause before the benchmark loop and verify the custom transformer's basic contract:

```
engineer = ClassificationFeatureEngineer()
small_train = make_mhp_ouerrun_data().iloc[:20]
engineer.fit(small_train)
transformed = engineer.transform(small_train.iloc[:3])
assert len(transformed) == 3
assert engineer.capacity_center_ == small_train["planned_capacity_kw"].median()
```

If you cannot yet explain the difference between `fit` and `transform`, revisit Chapter 2's pipeline boundary before attempting the full block.

The code assumes `chapter2_data.py`, `chapter3_data.py`, and `chapter4_data.py` have been saved from the three chapters.

```
from pathlib import Path
import json
import platform
```

```

import numpy as np
import pandas as pd
import scipy
import sklearn
from sklearn.base import BaseEstimator, TransformerMixin, clone
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import (
 HistGradientBoostingClassifier,
 RandomForestClassifier,
)
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
 average_precision_score,
 brier_score_loss,
 confusion_matrix,
 log_loss,
 roc_auc_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from chapter4_data import make_mhp_overrun_data

TARGET = "major_cost_overrun"
VALIDATION_YEARS = [2019, 2020, 2021, 2022]
TEST_START_YEAR = 2023
FP_COST = 1.0
FN_COST = 5.0
REVIEW_THRESHOLD = FP_COST / (FP_COST + FN_COST)
RANDOM_SEED = 429

class ClassificationFeatureEngineer(BaseEstimator, TransformerMixin):
 """Appraisal-time nonlinear features with training-fold centres."""

 def fit(self, X, y=None):
 X = X.copy()
 self.capacity_center_ = float(X["planned_capacity_kw"].median())
 self.year_center_ = float(X["start_year"].median())
 return self

 def transform(self, X):
 X = X.copy()
 capacity_c = X["planned_capacity_kw"] - self.capacity_center_
 X["capacity_centered"] = capacity_c
 X["capacity_centered_squared"] = capacity_c**2
 X["year_centered"] = X["start_year"] - self.year_center_
 X["road_hinge_above_18_km"] = (
 X["road_distance_observed_km"] - 18.0
).clip(lower=0.0)
 X["cable_by_road"] = (
 X["estimated_cable_km"]
 * X["road_distance_observed_km"]
)
 X["log_contractor_experience"] = np.log1p(
 X["contractor_experience_projects"]
)
 X["budget_per_planned_kw"] = (
 X["appraisal_budget_2025_million_pkr"]
 / X["planned_capacity_kw"]
)
 return X

RAW_NUMERIC = [
 "planned_capacity_kw",
 "estimated_cable_km",

```



```

"road_distance_observed_km",
"surveyed_route_km",
"terrain_index",
"contractor_experience_projects",
"appraisal_budget_2025_million_pkr",
"start_year",
]
ENGINEERED_NUMERIC = [
 "planned_capacity_kw",
 "capacity_centered",
 "capacity_centered_squared",
 "estimated_cable_km",
 "road_distance_observed_km",
 "road_hinge_above_18_km",
 "cable_by_road",
 "surveyed_route_km",
 "terrain_index",
 "log_contractor_experience",
 "appraisal_budget_2025_million_pkr",
 "budget_per_planned_kw",
 "year_centered",
]
CATEGORICAL = ["district", "access_mode"]

def make_preprocessor(numeric_columns):
 numeric = Pipeline([
 ("impute", SimpleImputer(strategy="median", add_indicator=True)),
 ("scale", StandardScaler()),
])
 categorical = Pipeline([
 ("impute", SimpleImputer(strategy="most_frequent")),
 ("one_hot", OneHotEncoder(
 handle_unknown="ignore",
 drop="first",
 sparse_output=False,
)),
])
 return ColumnTransformer([
 ("numeric", numeric, numeric_columns),
 ("categorical", categorical, CATEGORICAL),
])

def raw_pipeline(model):
 return Pipeline([
 ("preprocess", make_preprocessor(RAW_NUMERIC)),
 ("model", model),
])

def engineered_pipeline(model):
 return Pipeline([
 ("features", ClassificationFeatureEngineer()),
 ("preprocess", make_preprocessor(ENGINEERED_NUMERIC)),
 ("model", model),
])

def make_candidates():
 candidates = {
 "historical_prior": Pipeline([
 ("model", DummyClassifier(strategy="prior")),
]),
 }

 for c_value in [0.1, 1.0, 10.0]:
 candidates[f"logistic_raw_C_{c_value:g}"] = raw_pipeline(
 LogisticRegression(C=c_value, max_iter=10_000)
)

```

```

for c_value in [0.1, 1.0]:
 candidates[f"logistic_engineered_C_{c_value:g}"] = engineered_pipeline(
 LogisticRegression(C=c_value, max_iter=10_000)
)

for leaf_size in [5, 15, 30]:
 candidates[f"forest_leaf_{leaf_size}"] = raw_pipeline(
 RandomForestClassifier(
 n_estimators=500,
 max_features="sqrt",
 min_samples_leaf=leaf_size,
 random_state=RANDOM_SEED,
 n_jobs=-1,
)
)

for leaf_nodes, l2 in [(7, 1.0), (15, 1.0), (15, 10.0)]:
 name = f"histboost_leaves_{leaf_nodes}_l2_{l2:g}"
 candidates[name] = raw_pipeline(
 HistGradientBoostingClassifier(
 learning_rate=0.05,
 max_iter=350,
 max_leaf_nodes=leaf_nodes,
 min_samples_leaf=20,
 l2_regularization=l2,
 early_stopping=False,
 random_state=RANDOM_SEED,
)
)
return candidates

def probability_metrics(y_true, probability):
 probability = np.clip(np.asarray(probability), 1e-12, 1.0 - 1e-12)
 return {
 "log_loss": float(log_loss(y_true, probability, labels=[0, 1])),
 "brier": float(brier_score_loss(y_true, probability)),
 "roc_auc": float(roc_auc_score(y_true, probability)),
 "average_precision": float(average_precision_score(y_true, probability)),
 }

def action_metrics(y_true, probability, threshold=REVIEW_THRESHOLD):
 y_true = np.asarray(y_true, dtype=int)
 action = (np.asarray(probability) >= threshold).astype(int)
 tn, fp, fn, tp = confusion_matrix(
 y_true, action, labels=[0, 1]
).ravel()

def ratio(a, b):
 return float(a / b) if b else float("nan")

return {
 "threshold": float(threshold),
 "n": int(len(y_true)),
 "tn": int(tn),
 "fp": int(fp),
 "fn": int(fn),
 "tp": int(tp),
 "review_rate": float(action.mean()),
 "sensitivity": ratio(tp, tp + fn),
 "specificity": ratio(tn, tn + fp),
 "precision": ratio(tp, tp + fp),
 "cost_per_project": float((FP_COST * fp + FN_COST * fn) / len(y_true)),
}

def expanding_year_scores(data, candidates):
 rows = []

```

```

X_all = data.drop(columns=[TARGET])
y_all = data[TARGET]

for year in VALIDATION_YEARS:
 train = data["start_year"] < year
 validation = data["start_year"] == year
 if y_all.loc[validation].nunique() != 2:
 raise ValueError(f"Validation year {year} lacks both classes")

 for name, procedure in candidates.items():
 fitted = clone(procedure).fit(X_all.loc[train], y_all.loc[train])
 probability = fitted.predict_proba(X_all.loc[validation])[:, 1]
 metrics = probability_metrics(y_all.loc[validation], probability)
 rows.append({
 "candidate": name,
 "validation_year": year,
 "n_validation": int(validation.sum()),
 "events": int(y_all.loc[validation].sum()),
 **metrics,
 "decision_cost": action_metrics(
 y_all.loc[validation], probability
)["cost_per_project"],
 })
 return pd.DataFrame(rows)

def select_candidate(scores):
 summary = (
 scores.groupby("candidate")
 .agg(
 mean_log_loss=("log_loss", "mean"),
 sd_log_loss=("log_loss", "std"),
 worst_log_loss=("log_loss", "max"),
 mean_brier=("brier", "mean"),
 mean_roc_auc=("roc_auc", "mean"),
 mean_average_precision=("average_precision", "mean"),
 mean_decision_cost=("decision_cost", "mean"),
)
 .sort_values(["mean_log_loss", "worst_log_loss"])
)
 return summary.index[0], summary

def subgroup_audit(frame, group_column):
 rows = []
 for group, part in frame.groupby(group_column, observed=True):
 probability = part["probability"].to_numpy()
 outcome = part[TARGET].to_numpy()
 rows.append({
 group_column: group,
 "prevalence": float(outcome.mean()),
 "mean_probability": float(probability.mean()),
 "brier": float(brier_score_loss(outcome, probability)),
 **action_metrics(outcome, probability),
 })
 return pd.DataFrame(rows)

def reliability_table(y_true, probability, bins=8):
 frame = pd.DataFrame({
 "outcome": np.asarray(y_true, dtype=int),
 "probability": np.asarray(probability, dtype=float),
 })
 frame["bin"] = pd.qcut(
 frame["probability"], q=bins, duplicates="drop"
)
 return (
 frame.groupby("bin", observed=True)
 .agg(
 n=("outcome", "size"),

```

```

 events=("outcome", "sum"),
 mean_probability=("probability", "mean"),
 event_rate=("outcome", "mean"),
)
 .reset_index()
 .assign(bin=lambda x: x["bin"].astype(str))
)

def paired_log_loss_bootstrap(y_true, probability_a, probability_b,
 repeats=5_000, seed=RANDOM_SEED):
 y_true = np.asarray(y_true, dtype=int)
 a = np.clip(np.asarray(probability_a), 1e-12, 1.0 - 1e-12)
 b = np.clip(np.asarray(probability_b), 1e-12, 1.0 - 1e-12)
 row_loss_a = -(y_true * np.log(a) + (1 - y_true) * np.log(1 - a))
 row_loss_b = -(y_true * np.log(b) + (1 - y_true) * np.log(1 - b))
 row_difference = row_loss_a - row_loss_b

 rng = np.random.default_rng(seed)
 samples = np.empty(repeats)
 for repeat in range(repeats):
 index = rng.integers(0, len(y_true), size=len(y_true))
 samples[repeat] = row_difference[index].mean()

 return {
 "estimate": float(row_difference.mean()),
 "interval_95": np.quantile(samples, [0.025, 0.975]).tolist(),
 }

def main():
 data = make_mhp_overrun_data(n=900, seed=4040)
 development = data[data["start_year"] < TEST_START_YEAR].copy()
 locked_test = data[data["start_year"] >= TEST_START_YEAR].copy()

 assert development["start_year"].max() < locked_test["start_year"].min()
 prohibited = {
 "actual_cost_2025_million_pkr",
 "final_material_bill_million_pkr",
 "road_distance_km",
 "extreme_logistics_event",
 }
 assert prohibited.isdisjoint(RAW_NUMERIC + CATEGORICAL)

 candidates = make_candidates()
 fold_scores = expanding_year_scores(development, candidates)
 selected_name, validation_summary = select_candidate(fold_scores)

 X_development = development.drop(columns=[TARGET])
 y_development = development[TARGET]
 X_test = locked_test.drop(columns=[TARGET])
 y_test = locked_test[TARGET]

 selected = clone(candidates[selected_name]).fit(X_development, y_development)
 test_probability = selected.predict_proba(X_test)[: , 1]
 test_probability_metrics = probability_metrics(y_test, test_probability)
 test_action_metrics = action_metrics(y_test, test_probability)

 reference_name = "logistic_raw_C_1"
 reference = clone(candidates[reference_name]).fit(X_development, y_development)
 reference_probability = reference.predict_proba(X_test)[: , 1]
 paired_difference = paired_log_loss_bootstrap(
 y_test, test_probability, reference_probability
)

 predictions = locked_test[[
 "project_id",
 "start_year",
 "district",
 "access_mode",

```

```

 TARGET,
]).copy()
 predictions["probability"] = test_probability
 predictions["review"] = (test_probability >= REVIEW_THRESHOLD).astype(int)

 access_audit = subgroup_audit(predictions, "access_mode")
 district_audit = subgroup_audit(predictions, "district")
 calibration = reliability_table(y_test, test_probability)

 metadata = {
 "seed": RANDOM_SEED,
 "python": platform.python_version(),
 "numpy": np.__version__,
 "pandas": pd.__version__,
 "scipy": scipy.__version__,
 "scikit_learn": sklearn.__version__,
 "event_definition": "actual cost > 1.15 * appraisal budget",
 "development_rows": len(development),
 "test_rows": len(locked_test),
 "selected_candidate": selected_name,
 "reference_candidate": reference_name,
 "review_threshold": REVIEW_THRESHOLD,
 "test_probability_metrics": test_probability_metrics,
 "test_action_metrics": test_action_metrics,
 "selected_minus_reference_log_loss": paired_difference,
 }

 output = Path("chapter4_results")
 output.mkdir(exist_ok=True)
 fold_scores.to_csv(output / "validation_folds.csv", index=False)
 validation_summary.to_csv(output / "validation_summary.csv")
 predictions.to_csv(output / "locked_test_predictions.csv", index=False)
 access_audit.to_csv(output / "locked_test_access_audit.csv", index=False)
 district_audit.to_csv(output / "locked_test_district_audit.csv", index=False)
 calibration.to_csv(output / "locked_test_reliability.csv", index=False)
 (output / "run_metadata.json").write_text(
 json.dumps(metadata, indent=2), encoding="utf-8"
)

 print("Selected candidate:", selected_name)
 print("Locked-test probability metrics:", test_probability_metrics)
 print("Locked-test action metrics:", test_action_metrics)
 print("Selected minus reference log loss:", paired_difference)
 print("\nAccess-mode audit:\n", access_audit.to_string(index=False))
 print("\nReliability table:\n", calibration.to_string(index=False))

if __name__ == "__main__":
 main()

```

## 29.12 Why this benchmark does not tune the threshold

The threshold is fixed from a pre-specified cost ratio. The primary model is selected by log loss, not by threshold accuracy. This preserves the distinction between probability estimation and action.

If costs were uncertain, the registered study could report a decision curve or sensitivity analysis across plausible ratios. It should not choose the ratio that makes the locked test look best.

## 29.13 Reading the subgroup table

For each group, compare:

- sample size and events;
- prevalence and mean predicted probability;

- Brier score;
- review rate;
- TPR and FPR;
- precision; and
- simplified cost.

Never compare rates without raw confusion counts. A group with two events can have TPR 0%, 50%, or 100% after one case changes.

Differences are diagnostic. They can reflect sample variation, prevalence, measurement, conditional model error, threshold effects, or structural inequity. A metric does not identify which mechanism produced it.

## 29.14 Performance-difference uncertainty

The paired bootstrap estimates the selected model's test log-loss difference from the raw logistic reference. Negative favours the selected model.

It resamples rows as if they were exchangeable. Projects may share year and district shocks, so the interval is descriptive. A research study should align resampling with its sampling units, collect more independent periods and locations, and display period-level differences.

## 29.15 Prospective deployment protocol

Before live use:

1. freeze code, environment, feature definitions, and threshold;
2. shadow-run without influencing decisions;
3. verify feature availability and latency;
4. record predictions before outcomes occur;
5. preserve human actions, overrides, and reasons;
6. wait for complete outcome windows;
7. assess probability, action, and subgroup metrics;
8. compare intended with actual workflow; and
9. decide whether an impact evaluation is ethically and operationally appropriate.

If decisions change outcomes, later labels reflect the deployed policy. Monitoring must distinguish model drift from intervention effects.

## 29.16 Research paper study: Hardt, Price, and Srebro (2016)

Moritz Hardt, Eric Price, and Nati Srebro's *Equality of Opportunity in Supervised Learning* formalises equalised-odds and equal-opportunity criteria and studies post-processing of learned predictors.

Read with five questions.

1. **Definition.** Which conditional independence expresses equalised odds?
2. **Relaxation.** How does equal opportunity differ?
3. **Construction.** How can group-specific randomised thresholds satisfy constraints?
4. **Trade-off.** What happens to predictive performance under the constraint?
5. **Boundary.** Which questions about label validity, structural injustice, individual fairness, and intervention legitimacy remain outside the criterion?

## Replication task

Simulate two groups with different base rates and overlapping score distributions. Compare one common threshold, group-specific thresholds for equal TPR, and a post-processed equalised-odds rule. Report calibration, precision, TPR, FPR, utility, and individual decision changes.

## 29.17 The final research report

Use this structure.

1. **Abstract:** event, population, prediction time, selection design, locked result, and main limitation.
2. **Label study:** source, observation window, unresolved outcomes, and group measurement audit.
3. **Prediction contract:** intended use, prohibited uses, feature timing, and action.
4. **Methods:** candidate procedures, folds, search budget, scores, threshold, and uncertainty.
5. **Validation results:** every candidate and year, not only the winner.
6. **Locked results:** probability, ranking, calibration, and action metrics.
7. **Subgroup audit:** counts, rates, uncertainty limits, and exploratory labels.
8. **Interpretation:** pre-specified global and local methods, scale, background, and stability.
9. **Transport:** temporal, geographic, workflow, and outcome-maturity limits.
10. **Governance:** human review, contestability, monitoring, incident response, and retraining rules.
11. **Reproducibility:** data version, code, environment, seeds, predictions, and deviations.
12. **Conclusion:** a bounded claim that does not turn prediction into cause or deployment into benefit.

## Bounded conclusion template

Among eligible fictional MHP projects available through 2022, candidate A had the lowest pre-specified mean forward-validation log loss. After refitting, its 2023–2025 temporal-test log loss was L, Brier score B, ROC AUC R, and average precision P. At the registered review threshold T, sensitivity was S and review rate Q. Performance and action rates differed across access modes, with small group counts limiting precision. This supports further prospective evaluation within the represented workflow; it does not establish causal risk factors, fairness, benefit from review, or transport to new regions.

## 29.18 Day 29 build, break, and reflect

### Build

1. state which fairness concept, if any, follows from the review policy.
2. audit label completeness and feature missingness before outcome rates.
3. run the benchmark once and preserve every prediction.
4. complete group tables with confusion counts and uncertainty.
5. write the prospective shadow protocol and contest process.

### Break

1. drop group attributes and declare the model fair.
2. search fairness definitions and publish the one with the smallest disparity.
3. call temporal validation external validation across regions.
4. recalibrate or retune after viewing the locked test.
5. claim that predictive performance proves the review intervention helps.
6. hide small or poorly performing groups.

**Reflect**

If the model accurately predicts that structurally disadvantaged projects overrun, should the system deny them, review them, fund them differently, or change the process that creates overruns? Prediction cannot answer the policy question by itself.

**Day 29 exit check**

You have completed Chapter 4 when you can defend the label, probability model, threshold, interpretation, subgroup effects, temporal evidence, and governance process as separate but connected parts of one study.

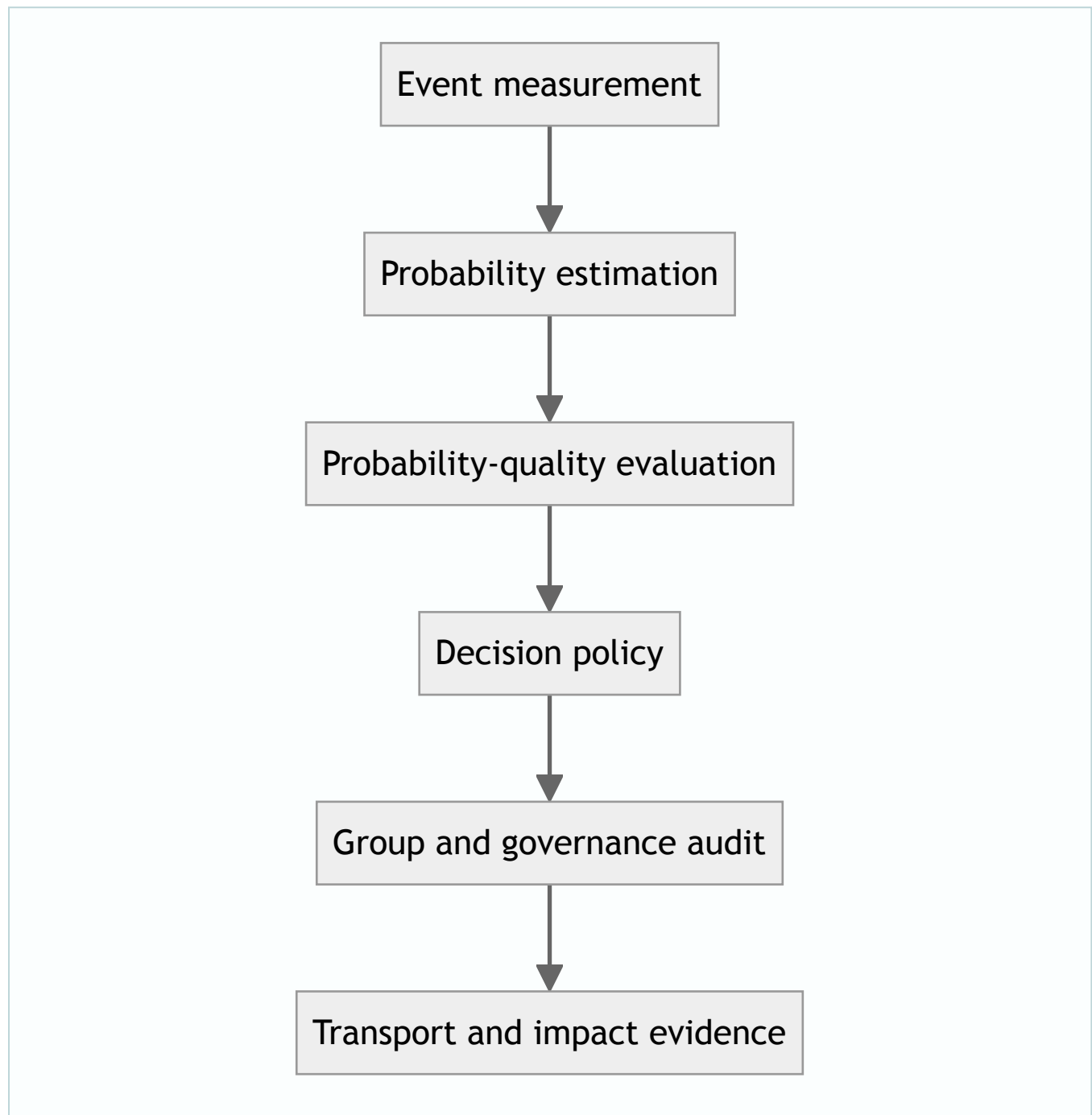
---



# Chapter 4 Synthesis

---

## One system, six evidence layers



An error at an earlier layer propagates forward. Perfect threshold optimisation cannot repair a biased event label. Beautiful calibration cannot legitimise a harmful action. Equal TPR on an internal test cannot prove transport to a new region. Research maturity means knowing which layer each result supports.

## Cumulative concept check

Answer without code.

1. Why is an unresolved outcome not automatically a negative class?
2. What changes when the event threshold moves from 15% to 10% over budget?
3. Convert probability 0.2 into odds and log odds.
4. Why is Bernoulli variance not constant?
5. Derive binary cross-entropy from the likelihood.
6. Why is logistic negative log likelihood convex?
7. What does  $e^{\beta_j}$  mean, and what does it not mean?
8. How does complete separation affect maximum likelihood?
9. Can a model have excellent ROC AUC and poor calibration?
10. Why are Brier score and log loss proper?
11. Why can an ECE value change under different bin boundaries?
12. Which data may fit a probability calibrator?
13. Derive the threshold  $C_{FP}/(C_{FP} + C_{FN})$ .
14. Why does precision change with prevalence?
15. What ranking interpretation does ROC AUC have?
16. Why can class weighting damage raw probability interpretation?
17. Which distance assumptions does KNN make?
18. Why are SVM margins not automatically probabilities?
19. Derive Gini impurity for a binary node.
20. How does random feature selection reduce forest correlation?
21. Why are OOB estimates unsuitable for some temporal claims?
22. Derive  $\mathbf{y} - \mathbf{p}$  as the logistic boosting pseudo-residual.
23. Why is early stopping a hyperparameter-selection operation?
24. What question does permutation importance answer?
25. Why can PDP create unsupported feature combinations?
26. Which choices define a SHAP game for dependent features?
27. Why is equal opportunity different from demographic parity?
28. Why can calibrated scores conflict with equalised odds when base rates differ?
29. What makes a validation study prospective?
30. Why does prospective prediction still not prove intervention benefit?

## Derivation workshop

Complete each derivation by hand, then verify with a small simulation.

1. Recover probability from log odds.
2. Derive Bernoulli mean and variance.
3. Derive logistic gradient  $\mathbf{X}^\top (\mathbf{p} - \mathbf{y})/\mathbf{n}$ .
4. Derive Hessian  $\mathbf{X}^\top \mathbf{W} \mathbf{X}/\mathbf{n}$  and prove positive semidefiniteness.
5. Derive the probability-scale derivative of a logistic coefficient.
6. Prove Brier score propriety.
7. Prove binary log-loss propriety.
8. Derive the binned Brier decomposition.
9. Derive the simplified cost threshold.
10. Express precision using prevalence, sensitivity, and specificity through Bayes' rule.
11. Derive binary Gini impurity  $2p(1 - p)$ .
12. Derive the variance of an average of equicorrelated tree predictions.
13. Derive logistic boosting pseudo-residuals.
14. Calculate every coalition weight for a three-feature Shapley game.
15. Express equalised odds as conditional independence.

# Practical laboratories

## Laboratory A — label maturity

Simulate projects with varying close-out times. Compare a naive label that codes unfinished cases negative with a label observed after a fixed window. Plot prevalence and calibration by project age.

## Laboratory B — logistic optimisation

Compare gradient descent, Newton–Raphson, and a library solver under scaling, collinearity, and near separation. Record objective, gradient norm, and runtime.

## Laboratory C — probability quality

Construct two models with equal accuracy but different confidence. Compare Brier score, log loss, calibration intercept, calibration slope, and reliability curves.

## Laboratory D — prevalence shift

Hold  $P(S \mid Y)$  fixed for score  $S$  and alter prevalence. Recalculate precision, PR curve, calibration, and the cost-optimal action under stated assumptions.

## Laboratory E — nonlinear support

Fit spline logistic, KNN, RBF SVM, forest, and boosting to a two-dimensional probability surface. Evaluate inside and outside the training support.

## Laboratory F — forest correlation

Add redundant strong predictors and vary `max_features`. Measure individual-tree strength, pairwise prediction correlation, ensemble Brier score, and stability.

## Laboratory G — adaptive tuning

Tune 5, 50, and 500 configurations on pure-noise features. Compare best inner log loss, outer log loss, and independent-test log loss.

## Laboratory H — explanation disagreement

Fit near-tied models that use correlated features differently. Compare coefficients, impurity importance, permutation importance, PDP, ICE, ALE, and SHAP.

## Laboratory I — fairness trade-offs

Simulate groups with different prevalence. Apply one threshold, equal-TPR thresholds, equalised-odds post-processing, and equal review rates. Report every metric and decision change.

## Laboratory J — prospective shadow run

Design a mock registry that stores feature timestamp, model version, probability, threshold, action, override, outcome-maturity date, and final event. Demonstrate why reconstructing these fields later is unreliable.

# Formula sheet

| Concept                  | Formula                                              | Purpose                                       |
|--------------------------|------------------------------------------------------|-----------------------------------------------|
| Odds                     | $p/(1-p)$                                            | event-to-non-event ratio                      |
| Logit                    | $\log[p/(1-p)]$                                      | maps probability to real line                 |
| Sigmoid                  | $1/(1+e^{-z})$                                       | maps score to probability                     |
| Bernoulli mass           | $p^y(1-p)^{1-y}$                                     | binary likelihood contribution                |
| Bernoulli variance       | $p(1-p)$                                             | conditional outcome variance                  |
| Binary log loss          | $-[y \log p + (1-y) \log(1-p)]$                      | proper probability loss                       |
| Logistic gradient        | $X^\top(p-y)/n$                                      | first-order optimisation direction            |
| Logistic Hessian         | $X^\top W X/n$                                       | curvature, $W_{ii} = p_i(1-p_i)$              |
| Odds ratio               | $e^{\beta_j}$                                        | conditional multiplicative odds change        |
| Brier score              | $n^{-1} \sum_i (p_i - y_i)^2$                        | squared probability error                     |
| Calibration              | $P(Y=1 \mid P=p) = p$                                | frequency validity of probabilities           |
| Sensitivity              | $TP/(TP+FN)$                                         | positive-event capture rate                   |
| Specificity              | $TN/(TN+FP)$                                         | non-event rejection rate                      |
| Precision                | $TP/(TP+FP)$                                         | event fraction among positive actions         |
| Balanced accuracy        | $(TPR+TNR)/2$                                        | equal-class average recall                    |
| $F_1$                    | $2PR/(P+R)$                                          | harmonic precision–recall summary             |
| Cost threshold           | $C_{FP}/(C_{FP}+C_{FN})$                             | simplified calibrated-probability action rule |
| Gini impurity            | $1 - \sum_k p_k^2$                                   | classification-node impurity                  |
| Split gain               | $I_P - (n_L I_L + n_R I_R)/n_P$                      | impurity reduction                            |
| Bagged probability       | $M^{-1} \sum_m \hat{p}_m(x)$                         | average across bootstrap learners             |
| Average variance         | $\rho\sigma^2 + (1-\rho)\sigma^2/M$                  | role of learner correlation                   |
| RBF kernel               | $\exp(-\gamma\ x-x'\ _2^2)$                          | local similarity in implicit feature space    |
| Hinge loss               | $\max(0, 1-yf)$                                      | margin violation penalty                      |
| Softmax                  | $e^{z_k} / \sum_j e^{z_j}$                           | joint multiclass probabilities                |
| Boosting update          | $F_m = F_{m-1} + \eta h_m$                           | stagewise additive fit                        |
| Logistic pseudo-residual | $y-p$                                                | negative gradient in score space              |
| Permutation importance   | $S(D) - S(D_j^{perm})$                               | score dependence on disrupted feature         |
| PDP                      | $\mathbb{E}_{X_C}[f(x_S, X_C)]$                      | average prediction under feature replacement  |
| Shapley value            | weighted average of marginal coalition contributions | additive local attribution                    |
| Demographic parity       | $\hat{Y}$ independent of $A$                         | equal positive-action rate                    |
| Equalised odds           | $\hat{Y}$ independent of $A$ given $Y$               | equal TPR and FPR                             |

Metric symbols such as  $P$  and  $R$  in the  $F_1$  row mean precision and recall, not probability and a missingness indicator. Define notation locally in research writing.

## Glossary

### Action threshold

A rule converting a score or probability into a discrete action.

### Average precision

A weighted summary of precision across recall increments, sensitive to event prevalence.

### Bagging

Fitting learners to bootstrap samples and averaging their outputs.

### Balanced accuracy

The average of sensitivity and specificity in binary classification.

### Brier score

Mean squared error of binary probability forecasts.

### Calibration

Agreement between predicted probabilities and event frequencies in a stated population.

### Classifier chain

A multilabel procedure using earlier label predictions as later model features.

### Complete separation

A predictor combination that perfectly separates binary classes, causing unpenalised logistic estimates to diverge.

### Concept shift

A change in the conditional relationship  $P(Y | X)$ .

### Confusion matrix

Counts of true and false positive and negative threshold decisions.

### Covariate shift

A change in  $P(X)$  under an assumption that  $P(Y | X)$  remains stable.

### Demographic parity

Equality of positive-action rates across groups.

### Discrimination

The ability of a score to rank events above non-events.

### Early stopping

Choosing training length according to performance on validation data.

### Equal opportunity

Equality of true-positive rates across groups.

### Equalised odds

Equality of true-positive and false-positive rates across groups.

### Event horizon

The follow-up period during which a binary event is assessed.

**Gini impurity**

A node-mixing measure equal to  $1 - \sum_k p_k^2$ .

**Gradient boosting**

Stagewise additive fitting along estimated negative loss gradients.

**ICE**

Individual conditional expectation: a fitted prediction curve for one row as a focal feature is varied.

**Impact evaluation**

A design estimating the causal effect of using an intervention or decision system.

**Index time**

The moment at which eligibility, predictors, and prediction are anchored.

**Isotonic calibration**

A flexible nondecreasing mapping from scores to probabilities.

**Label shift**

A change in class prevalence under an assumption that  $P(X | Y)$  remains stable.

**Log loss**

Negative log likelihood of reported class probabilities.

**Log odds**

The logarithm of event odds, ranging across the real line.

**Macro average**

An equal-weight average of class-specific metrics.

**Multiclass**

A task with more than two mutually exclusive outcome classes.

**Multilabel**

A task in which several binary labels may be true for one row.

**Out of bag**

Training rows excluded from a particular bootstrap sample and available for that learner's internal evaluation.

**Partial dependence**

Average fitted response after setting focal features and averaging over complement-feature rows.

**Permutation importance**

Held-out score deterioration after disrupting a feature through permutation.

**Precision**

Event fraction among positive predictions or actions.

**Prevalence**

Event fraction in a stated population, sample, and observation window.

**Proper scoring rule**

A loss whose expected value is optimised by reporting the true probability distribution.

**Prospective validation**

Evaluation on future cases after fixing the model and protocol in advance.

**Pseudo-residual**

The negative loss gradient used as the target for a boosting stage.

**Random forest**

An ensemble of bootstrap trees with random feature availability at splits.

**Recall**

Another name for sensitivity or true-positive rate.

**Reliability diagram**

A binned comparison of mean predicted probability with event frequency.

**ROC AUC**

A threshold-free ranking summary over false-positive and true-positive rates.

**Sensitivity**

The fraction of events assigned a positive action.

**SHAP**

A family of additive feature-attribution methods based on Shapley-value ideas.

**Sigmoid calibration**

A logistic mapping from an uncalibrated score to probability.

**Specificity**

The fraction of non-events assigned a negative action.

**Support vector**

An observation on or within the margin that contributes to an SVM solution.

## Research-paper reading ladder

| Paper                            | Read after       | Central contribution                                     | Replication focus                         |
|----------------------------------|------------------|----------------------------------------------------------|-------------------------------------------|
| Nelder & Wedderburn (1972)       | Day 22           | generalized linear model framework and iterative fitting | IRLS and separation                       |
| Brier (1950)                     | Day 23           | evaluation of probability forecasts                      | proper squared probability score          |
| Niculescu-Mizil & Caruana (2005) | Day 23           | empirical comparison and post-hoc calibration            | algorithm-specific probability distortion |
| Davis & Goadrich (2006)          | Day 24           | mathematical ROC–PR relationship                         | prevalence change                         |
| Breiman (2001)                   | Day 26           | random forests, strength, correlation, and OOB analysis  | correlated-tree averaging                 |
| Friedman (2001)                  | Day 27           | gradient boosting as function-space optimisation         | logistic pseudo-residuals                 |
| Grinsztajn et al. (2022)         | Day 27 extension | modern tabular benchmark and inductive-bias study        | compute-equal model comparison            |
| Lundberg & Lee (2017)            | Day 28           | additive feature attribution and SHAP                    | background and dependence sensitivity     |
| Hardt et al. (2016)              | Day 29           | equal opportunity and equalised odds                     | parity–utility trade-offs                 |

## Seven-pass advanced reading method

1. **Question:** state the problem in one falsifiable sentence.
2. **Estimand:** identify the mathematical object being estimated or bounded.
3. **Algorithm:** reconstruct the procedure as pseudocode.
4. **Assumptions:** list sampling, model, optimisation, and measurement conditions.
5. **Evidence:** reproduce one theorem, simulation, or table.
6. **Stress test:** create a setting in which the method should fail.
7. **Transfer:** state which new data would be needed for the intended application.

## Capstone assessment rubric

| Dimension         | Emerging               | Competent                          | Research-ready for this level                                                             |
|-------------------|------------------------|------------------------------------|-------------------------------------------------------------------------------------------|
| Event definition  | binary column accepted | event and positive class stated    | index time, horizon, eligibility, censoring, and sensitivity are audited                  |
| Probability model | labels predicted       | likelihood and pipeline understood | scoring, calibration, shift, and uncertainty are separated                                |
| Decision policy   | threshold 0.5          | threshold validated                | consequences, capacity, human action, and policy sensitivity are registered               |
| Imbalance         | accuracy reported      | PR and class counts reported       | weighting/resampling target changes and prevalence transport are explicit                 |
| Ensembles         | defaults compared      | complexity tuned honestly          | loss gradients, learner correlation, support, and compute budget are analysed             |
| Interpretation    | importance plot shown  | method target and scale stated     | dependence, background, stability, fidelity, and noncausal scope are tested               |
| Fairness          | group averages shown   | confusion rates and counts shown   | measurement, metric conflict, uncertainty, intervention, and contestability are addressed |
| Validation        | random split           | temporal or grouped design         | tuning, calibration, threshold, locked test, and external limits form one protocol        |
| Reproducibility   | seed listed            | code and predictions saved         | data/version provenance, all trials, deviations, and governance artifacts are preserved   |

## Instructor and self-study notes

### Suggested daily rhythm

| Segment                    | Approximate time | Activity                         |
|----------------------------|------------------|----------------------------------|
| Retrieval                  | 15 minutes       | answer the previous exit check   |
| Event or decision scenario | 20 minutes       | clarify what the model is for    |
| Concept construction       | 35 minutes       | small numerical example          |
| Derivation                 | 45 minutes       | prove the day's central equation |
| Code laboratory            | 55 minutes       | implement and compare            |
| Failure laboratory         | 25 minutes       | break a boundary or assumption   |
| Paper reading              | 25 minutes       | reconstruct one claim            |



| Segment    | Approximate time | Activity                   |
|------------|------------------|----------------------------|
| Reflection | 10 minutes       | write a bounded conclusion |

## Frequent misconceptions to surface

- A binary target is not automatically fully observed.
- Odds are not probabilities.
- Logistic regression is linear on log odds, not probability.
- Accuracy is not probability quality.
- ROC AUC is not calibration.
- A 0.5 threshold is not neutral.
- Class weights do not create population probabilities by default.
- Trees need no scaling, but preprocessing still matters.
- Classification boosting follows loss gradients, not always ordinary residuals.
- More tuning increases selection opportunity.
- Permutation importance is not causal effect.
- PDP does not make a real intervention.
- SHAP values depend on background, dependence, and output scale.
- Dropping a protected attribute does not prove fairness.
- Temporal validation is not necessarily geographic validation.
- Predictive accuracy does not prove beneficial impact.

## Ethical and contextual discussion

In the fictional MHP case, review can be helpful, burdensome, or both. A false positive might delay approval; a false negative might leave a community facing an underfunded project. Historical overrun labels may reflect procurement systems, remoteness, political attention, or inconsistent accounting—not intrinsic project worth.

Discuss:

- whether review resources should follow risk, need, equal allocation, or structural disadvantage;
- whether a predicted overrun should increase funding rather than restrict approval;
- who may correct appraisal records;
- how local expertise enters or overrides the system;
- whether overrides are audited without punishing staff for appropriate judgment;
- how long an outcome remains unresolved; and
- who can stop deployment after a safety or fairness incident.

## Where Chapter 5 begins

Chapter 4 forced a fixed event window. Many research questions cannot be reduced safely to “event by date  $T$ .” Chapter 5 should develop time-to-event analysis:

- event time, censoring, truncation, and competing risks;
- risk sets and the Kaplan–Meier estimator;
- hazard, survival, and cumulative incidence;
- Cox proportional-hazards regression;
- time-varying covariates and landmark prediction;
- proportional-hazards diagnostics;
- survival forests and boosting;
- time-dependent discrimination and calibration;
- dynamic decision thresholds;

- external and prospective validation; and
- a registered censored-outcome study.

That chapter should also clarify when a causal question—“does review prevent overruns?”—requires a different design from predicting time to overrun.

## Final reflection

Classification is not the act of assigning zeros and ones. It is the construction of a chain:

event → probability → evidence → action → consequence.

Every arrow contains assumptions and institutional choices. The advanced modeller's job is to make those choices visible enough to test, challenge, and revise.

# References and Further Reading

## Research papers discussed

- Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5–32.
- Brier, G. W. (1950). Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review*, 78(1), 1–3.
- Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*, 233–240.
- Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5), 1189–1232.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?. *Advances in Neural Information Processing Systems*, 35.
- Hardt, M., Price, E., & Srebro, N. (2016). Equality of Opportunity in Supervised Learning. *Advances in Neural Information Processing Systems*, 29.
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.
- Nelder, J. A., & Wedderburn, R. W. M. (1972). Generalized Linear Models. *Journal of the Royal Statistical Society: Series A*, 135(3), 370–384.
- Niculescu-Mizil, A., & Caruana, R. (2005). Predicting Good Probabilities with Supervised Learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625–632.

## Additional primary reading

- Apley, D. W., & Zhu, J. (2020). Visualizing the Effects of Predictor Variables in Black Box Supervised Learning Models. *Journal of the Royal Statistical Society: Series B*, 82(4), 1059–1086. Introduces accumulated local effects.
- Cox, D. R. (1958). The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 215–242.
- Fawcett, T. (2006). An Introduction to ROC Analysis. *Pattern Recognition Letters*, 27(8), 861–874.

## Official software documentation

- [scikit-learn. Generalized Linear Models and Logistic Regression.](#)
- [scikit-learn. Probability Calibration.](#)
- [scikit-learn. Classification Metrics.](#)
- [scikit-learn. Tuning the Decision Threshold.](#)
- [scikit-learn. Nearest Neighbours and Support Vector Machines.](#)
- [scikit-learn. Ensembles: Forests and Gradient Boosting.](#)
- [scikit-learn. Permutation Feature Importance.](#)
- [scikit-learn. Partial Dependence and ICE.](#)
- [SHAP. Documentation.](#)

## Recommended textbooks

- Barocas, S., Hardt, M., & Narayanan, A. *Fairness and Machine Learning.*
- Bishop, C. M., & Bishop, H. *Deep Learning: Foundations and Concepts.*
- Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning.*
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. *An Introduction to Statistical Learning.*
- Molnar, C. *Interpretable Machine Learning.*
- Murphy, K. P. *Probabilistic Machine Learning: An Introduction.*
- Steyerberg, E. W. *Clinical Prediction Models.*

Use software documentation for current interfaces, textbooks for connected exposition, and primary papers for the claims and assumptions that originated the methods.

# Chapter 4 Companion Guide: From Probabilities to Accountable Decisions

---

## A beginner-first guide to classification, probability, decisions, ensembles, explanations, fairness, and validation

This guide accompanies `chapter4.md`. It assumes no fluency in Python, mathematics, or statistics. When Chapter 4 uses a compact formula or a library object, this guide slows down, names every part, and connects it to a small example.

Chapter 4 is about much more than predicting `0` or `1`. It builds an accountable chain:

```
define the event
 -> estimate a probability
 -> test probability quality
 -> choose an action rule
 -> examine who receives errors and actions
 -> test where the evidence transports
```

Each arrow can fail independently. A sophisticated model cannot repair a badly defined label. A well-calibrated probability does not choose an ethical policy. A fairness table cannot prove that an intervention helps.

---

## How to use this guide

For each day:

1. Read the companion section before the matching chapter section.
2. Work through the tiny numerical example with paper or a calculator.
3. Type the code rather than only reading it.
4. change one value and predict what will happen before rerunning it.
5. Complete the chapter's larger build exercise.
6. Deliberately create one of the listed failures.
7. Explain the exit check in ordinary language.

Keep these five objects separate throughout the chapter:

- **label:** what later counts as an event;
  - **features:** what is known when the prediction is made;
  - **score or probability:** what the fitted model returns;
  - **action rule:** how a person or policy uses the output; and
  - **claim:** what the study's evidence actually supports.
-

# 0. Foundations for a complete beginner

## 0.1 The running example

One row represents one fictional microhydro power project. At technical appraisal, the model estimates the chance that final inflation-adjusted cost will exceed 115% of the approved appraisal budget.

The positive label is:

```
major_cost_overrun = 1
```

The negative label is:

```
major_cost_overrun = 0
```

A high probability may send a project to senior engineering review. It must not automatically reject the project. Prediction and action are different stages.

The data are simulated teaching data. District or access-mode patterns in them are not facts about real places.

## 0.2 Essential Python vocabulary

Python evaluates expressions and stores results in names:

```
probability = 0.20
threshold = 0.15
send_to_review = probability >= threshold
print(send_to_review) # True
```

- `=` assigns a value.
- `>=` compares two values.
- `True` and `False` are Boolean values.
- `#` starts a comment that Python ignores.
- a function packages reusable instructions.

```
def odds(p):
 return p / (1 - p)

print(odds(0.20))
```

The indented line belongs to the function. `return` sends a result back to the caller.

## 0.3 Arrays, tables, and labels

NumPy arrays store many values of the same general kind:

```
import numpy as np

y = np.array([1, 0, 1, 0])
p = np.array([0.80, 0.30, 0.60, 0.10])
```

Here `y[i]` is an observed label and `p[i]` is a predicted probability for the same row.

Pandas DataFrames are labelled tables:

```
import pandas as pd

projects = pd.DataFrame({
 "project_id": ["P1", "P2", "P3"],
 "probability": [0.72, 0.18, 0.41],
})

print(projects["probability"].mean())
```

Important patterns used in Chapter 4:

- `frame["column"]` selects one column;
- `.mean()` calculates an average;
- `.groupby("district")` forms groups;
- `.fit(X, y)` learns from features `X` and labels `y`;
- `.predict_proba(X)[: , 1]` selects the probability of class 1;
- `random_state=...` makes randomized code reproducible.

## 0.4 Mathematical symbols used in the chapter

- $Y$  is a random binary outcome;  $y_i$  is the observed label for row  $i$ .
- $X$  is the complete feature table;  $x_i$  is one row.
- $n$  is the number of rows.
- *sum* means “add these terms.”
- *prod* means “multiply these terms.”
- *log* is the natural logarithm.
- $e^z$  is the exponential function.
- $\mathbb{E}$  means an average over a probability distribution.
- $P(A)$  means the probability of event  $A$ .
- $P(A | B)$  means the probability of  $A$  among cases for which  $B$  is true.
- $\mathbb{1}_{condition}$  is 1 when the condition is true and 0 otherwise.
- $\partial$  denotes a derivative when a function has several inputs.
- $X^T$  is the transpose of a matrix: rows and columns exchange roles.

## 0.5 Four different meanings of “positive”

Classification language can be misleading. “Positive” need not mean good.

- **actual positive**: the event occurred,  $Y = 1$ ;
- **predicted positive**: the action rule marked the case,  $\hat{Y} = 1$ ;
- **true positive**: the event occurred and the rule marked it;
- **false positive**: the rule marked it but the event did not occur.

In this chapter, a positive event is a major cost overrun, while a positive action is senior review.

## 0.6 Training, validation, calibration, and test data

- **Training data** fit model parameters.
- **Validation data** compare models and tune settings.
- **Calibration data** can learn a mapping from model scores to probabilities.
- **Test data** estimate final performance after choices are frozen.

One row may sometimes receive different roles through carefully designed cross-validation, but information from its evaluation role must not leak into its training role.

## 0.7 The prediction-time rule

Imagine stopping the project clock at technical appraisal. A feature is allowed only if it is available then. Final material bills, final costs, and events learned during construction are prohibited, even if they make prediction easy.

This simple question catches much leakage:

Could a real reviewer know this value at the exact moment the prediction must be issued?

## 0.8 Installing the software

From a terminal in the project directory, install the core packages:

```
python -m pip install numpy pandas scipy matplotlib scikit-learn
```

- NumPy supplies arrays and numerical functions.
- pandas supplies labelled tables.
- SciPy supplies probability distributions and optimization utilities.
- Matplotlib draws diagnostic plots.
- scikit-learn supplies preprocessing, models, metrics, and validation tools.

SHAP is optional and needed only for its interpretation laboratories:

```
python -m pip install shap
```

An installation command changes the current Python environment. Record Python, operating-system, and package versions so another learner can reproduce the run. A seed reproduces pseudo-random choices under the same software; it does not make a scientific result universally stable.

## 0.9 Understanding chapter4\_data.py

The Chapter 4 generator begins with Chapter 3's fictional project table, then creates an appraisal budget from appraisal-time variables:

```
df = make_mhp_practitioner_data(n=n, seed=seed)
rng = np.random.default_rng(seed + 1)
```

`n` is the number of rows. A separate random generator produces reproducible budget noise without silently reusing Chapter 3's random stream.

A dictionary maps district labels to simulated budget adjustments:

```
district_term = df["district"].map(district_budget_adjustment).to_numpy()
```

`.map(...)` looks up one adjustment per row. `.to_numpy()` converts the pandas Series into a NumPy array for arithmetic.

The appraisal-budget expression combines linear, squared, categorical, time, and random terms. It deliberately uses only information available at appraisal. The binary event is constructed by comparison:

```
df["major_cost_ouerrun"] = (
 df["actual_cost_2025_million_pkr"]
 > 1.15 * df["appraisal_budget_2025_million_pkr"]
).astype(int)
```

The comparison first creates `True` or `False`; `.astype(int)` converts them to 1 and 0. Final cost is legitimate for constructing the later outcome but prohibited as a prediction feature.

Finally:

```
df.sort_values(["start_year", "project_id"]).reset_index(drop=True)
```

puts earlier years first and gives rows a fresh numeric index. This ordering is essential for time-based splitting, but sorting alone does not make every generic cross-validator time-aware.

The generator retains final-cost fields for teaching audits. Their presence in the same DataFrame does not authorize using them. Real data would need accounting review, inflation rules, close-out dates, contract-change treatment, and dispute policies.

## 0.10 Pipelines and column transformations

A scikit-learn `Pipeline` runs steps in order:

```
from sklearn.pipeline import Pipeline

procedure = Pipeline([
 ("preprocess", preprocessor),
 ("model", classifier),
])
```

When cross-validation calls `fit`, preprocessing learns only from that fold's training rows. Validation rows receive the stored transformation. This is why a pipeline is part of the scientific procedure, not merely convenient syntax.

A `ColumnTransformer` can send numeric and categorical columns through different paths. Numeric columns may receive median imputation and standardization. Categorical columns may receive most-frequent imputation and one-hot encoding. `handle_unknown="ignore"` prevents a previously unseen validation category from crashing prediction, but it does not prove that the new category is represented adequately.

For classifiers, the fitted procedure usually exposes:

```
procedure.predict_proba(X_new)[:, 1]
```

Use the class-1 probability for probability metrics and registered action rules. `.predict(...)` normally applies an estimator default threshold and can discard information needed for calibration and decision analysis.

---



# Day 21 — The outcome, probability, and prediction contract

---

## 21.1 Begin with time, not an algorithm

A binary column hides a story. Define:

- **unit**: what one row represents;
- **eligibility**: which rows belong in the study;
- **index time**: when prediction occurs;
- **event**: what makes the outcome 1;
- **horizon**: how long outcomes are observed;
- **negative rule**: when 0 is justified;
- **unresolved rule**: what happens before the outcome matures; and
- **competing outcomes**: how cancellation, merger, or scope change are handled.

An unfinished project is unknown, not automatically 0. Coding it negative gives older projects more time to become positive and contaminates the meaning of the label.

If event timing and incomplete follow-up are central, a time-to-event model may be more honest than a fixed binary outcome.

## 21.2 Label construction is measurement

“Overrun above 15%” is not a natural fact until accounting rules are specified. Ask:

- Is the original or revised budget the denominator?
- Are inflation and exchange rates handled consistently?
- Does approved scope expansion count?
- When is the final account considered closed?
- Can accounting practice differ across locations?

The primary definition must be fixed before model comparison. Alternate 10%, 15%, and 20% definitions are useful sensitivity analyses, not invitations to choose the easiest result afterward.

## 21.3 Probability, odds, and log odds

A probability  $p$  lies from 0 to 1. Odds compare the event probability with the non-event probability:

$$o = \frac{p}{1 - p}.$$

For  $p = 0.20$ :

$$o = \frac{0.20}{0.80} = 0.25.$$

This is “one event for four non-events.” It is not a 25% probability.

Convert odds back to probability:

$$p = \frac{o}{1 + o}.$$

Log odds are:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right).$$

Probabilities are bounded, but log odds range from negative infinity to positive infinity. At  $p = 0.5$ , odds are 1 and log odds are 0.

```
import numpy as np

def probability_to_log_odds(p):
 if not 0 < p < 1:
 raise ValueError("p must be strictly between 0 and 1")
 return np.log(p / (1 - p))

def log_odds_to_probability(z):
 return 1 / (1 + np.exp(-z))
```

## 21.4 Bernoulli outcomes

A Bernoulli variable has two possible values:

$$Y \sim \text{Bernoulli}(p), \quad Y \in \{0, 1\}.$$

Its compact probability formula is:

$$P(Y = y) = p^y(1-p)^{1-y}.$$

Substitute both values:

- for  $y = 1$ , it becomes  $p$ ;
- for  $y = 0$ , it becomes  $1 - p$ .

Its mean and variance are:

$$\mathbb{E}[Y] = p, \quad \text{Var}(Y) = p(1-p).$$

Variance is greatest at  $p = 0.5$  and small near 0 or 1. Binary data therefore do not have constant conditional variance.

## 21.5 Prevalence and the constant baseline

Prevalence is the event fraction in a named sample and time window:

$$\hat{\pi} = \frac{\text{number of events}}{\text{number of eligible resolved rows}}.$$

It can change by period, group, label definition, follow-up, and sampling plan.

A no-feature probability model predicts training prevalence for every future row. If 18 of 100 training projects overrun, it predicts 0.18 for each validation project. This is a serious baseline for Brier score and log loss.

Never calculate a fold's baseline using the full dataset; that reveals future prevalence.

## 21.6 Selection into the labelled sample

Let  $S = 1$  mean a resolved label is available. Training estimates:

$$P(Y = 1 \mid X, S = 1),$$

while deployment may need  $P(Y = 1 \mid X)$ . These differ if fast-closing or well-documented projects are overrepresented.

Compare labelled and unresolved eligible projects using appraisal-time features, year, group, and follow-up length. This does not prove selection is harmless, but it exposes obvious differences.

## 21.7 Beginner checklist

Before Day 22, be able to:

- describe the event without referring to a dataset column name;
- distinguish 0 from unknown;
- convert 0.20 to odds and log odds;
- explain why prevalence is not timeless;
- construct a training-only constant probability baseline; and
- name one way labelled projects could be unrepresentative.

---

# Day 22 — Logistic regression from Bernoulli likelihood

---

## 22.1 Why a straight probability line is awkward

A line  $p = \beta_0 + \beta_1 x$  can predict  $p < 0$  or  $p > 1$ . Logistic regression instead makes log odds linear:

$$\log\left(\frac{p}{1-p}\right) = x^\top \beta.$$

The sigmoid converts any real score  $z = x^\top \beta$  into a valid probability:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

“Linear logistic regression” means linear on the log-odds scale. The probability curve is S-shaped.

## 22.2 Vectors and the linear score

With two features and an intercept:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2.$$

The shorthand  $x^\top \beta$  means multiply matching feature and coefficient values and add them. The intercept is represented by a constant feature equal to 1.

The sigmoid derivative is:

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)] = p(1 - p).$$

It is largest at  $p = 0.5$ . The same log-odds change has a larger probability effect near 0.5 than near 0 or 1.

## 22.3 Likelihood and log loss

Likelihood asks: under coefficients  $\beta$ , how plausible are all observed labels?

$$L(\beta) = \prod_i p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Products of many small probabilities are numerically awkward, so take logarithms. Logs turn products into sums:

$$\ell(\beta) = \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Maximizing log likelihood is equivalent to minimizing negative average log likelihood:

$$J(\beta) = -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

This is binary cross-entropy or log loss. A confident wrong answer has very large loss. Code clips probabilities away from exact 0 and 1 before taking logs.

## 22.4 Gradient and Hessian without mystery

A derivative says how fast a quantity changes. A gradient collects one derivative for each coefficient:

$$\nabla J = \frac{1}{n} X^\top (p - y).$$

The vector  $p - y$  contains probability errors. Multiplication by  $X^\top$  accumulates how those errors align with each feature.

The Hessian records curvature:

$$H = \frac{1}{n} X^\top W X, \quad W_{ii} = p_i(1 - p_i).$$

Because every diagonal weight is nonnegative,  $H$  is positive semidefinite. The logistic negative log likelihood is convex: it has no deceptive local valleys, although separation or redundant columns can prevent a finite unique solution.

## 22.5 Gradient descent, Newton's method, and IRLS

Gradient descent repeatedly moves opposite the gradient:

$$\beta_{new} = \beta_{old} - \eta \nabla J.$$

$\eta$  is the learning rate. Too large can overshoot; too small can be slow.

Newton's method also uses curvature:

$$\beta_{new} = \beta_{old} - H^{-1} \nabla J.$$

For generalized linear models, the Newton calculation can be written as iteratively reweighted least squares (IRLS). It repeatedly updates probabilities, weights, and a working least-squares problem. This is a computational device, not a claim that binary errors are Gaussian.

## 22.6 Regularisation

An  $L_2$  penalty adds:

$$\frac{\lambda}{2} \sum_{j=1}^p \beta_j^2.$$

It discourages very large slopes and stabilizes correlated or sparse designs. The intercept is usually not penalized. Scaling matters because the same numeric coefficient represents different changes in metres, kilometres, or rupees.

In scikit-learn, `C` is inverse penalty strength:

- smaller `C` means stronger regularisation;
- larger `C` means weaker regularisation.

Packages normalize loss differently, so identical numeric settings need not mean identical objectives.

## 22.7 Coefficients and odds ratios

A slope  $\beta_j$  is the conditional log-odds change for a one-unit feature increase, holding represented covariates fixed. Exponentiating gives an odds ratio:

$$OR = e^{\beta_j}.$$

If  $\beta_j = 0.4$ , then  $OR \approx 1.49$ : odds multiply by 1.49. This is not a 49 percentage-point probability increase.

The local probability slope is:

$$\frac{\partial p}{\partial x_j} = \beta_j p(1 - p).$$

Therefore the probability change depends on starting risk. Coefficients are associational unless a separate causal design justifies a causal interpretation.

## 22.8 Complete and quasi-complete separation

Complete separation occurs when a feature rule perfectly divides every 0 from every 1. Unpenalised coefficients can grow without bound while likelihood keeps improving. Quasi-separation leaves a few boundary ties.

Warning signs:

- convergence warnings;
- huge coefficients or standard errors;
- probabilities nearly exactly 0 or 1;
- perfect training accuracy; and
- extreme sensitivity to a few rows.

Check leakage and data errors first. Then consider justified category pooling, regularisation, bias-reduced inference, or more observations in sparse regions. Never celebrate perfect accuracy before this audit.

## 22.9 Minimal library workflow

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(C=1.0, max_iter=10_000)
model.fit(X_train, y_train)
probability = model.predict_proba(X_valid)[: , 1]
```

- `fit` learns from training rows.
- `predict_proba` returns one column per class.
- `[:, 1]` selects class-1 probabilities.
- a convergence warning is evidence to investigate, not text to suppress.

# Day 23 — Proper scoring and calibration

## 23.1 Three different evaluation questions

- **Discrimination:** are event cases generally ranked higher?
- **Calibration:** do stated probabilities match event frequencies?
- **Decision value:** do resulting actions perform well under stated consequences?

A model can rank perfectly but call a 20% event a 90% risk. It can be calibrated overall while barely separating cases. One number cannot represent all three goals.

## 23.2 Calibration in ordinary language

Perfect calibration means:

$$P(Y = 1 \mid P = p) = p.$$

Among many comparable cases receiving 0.20, roughly 20% should experience the event. This validates repeated forecasts, not an individual claim that a particular case is “20% true.”

Calibration belongs to a population and time. It can break after prevalence, workflow, or conditional relationships change.

## 23.3 Brier score and propriety

The Brier score is mean squared probability error:

$$\text{Brier} = \frac{1}{n} \sum_i (p_i - y_i)^2.$$

Lower is better. Suppose true probability is  $q$  but someone reports  $p$ . Expected squared loss is:

$$q(p - 1)^2 + (1 - q)p^2 = p^2 - 2pq + q.$$

Its derivative is  $2(p - q)$ , which is zero only at  $p = q$ . Truthful reporting minimizes expected loss. That makes Brier score strictly proper.

## 23.4 Log loss and propriety

For one row:

$$-y \log p - (1 - y) \log(1 - p).$$

If true probability is  $q$ , expected loss is:

$$-q \log p - (1 - q) \log(1 - p).$$

Its derivative is  $(p - q)/[p(1 - p)]$ , giving the minimum at  $p = q$ . Log loss punishes extreme wrong confidence more strongly than Brier score.

## 23.5 Reliability tables and diagrams

A reliability table groups similar probabilities, then reports:

- number of cases;
- mean predicted probability; and
- observed event fraction.

If a bin has mean prediction 0.31 and event rate 0.29, it is close in that bin. But binning is descriptive:

- tiny bins are uncertain;
- empty regions supply no evidence;
- equal-width and equal-count bins differ;
- results change with boundaries; and
- a diagonal-looking graph says nothing about ranking.

Always show counts or data density and preferably uncertainty.

## 23.6 Brier decomposition

For deliberately binned forecasts, Brier score can be written as:

$$\text{reliability} - \text{resolution} + \text{uncertainty}.$$

- **Reliability** penalizes mean forecast/event-rate disagreement within bins.
- **Resolution** rewards groups whose event rates differ from overall prevalence.
- **Uncertainty** is  $\bar{y}(1 - \bar{y})$ , the variability associated with prevalence.

With continuously varying forecasts, a binned decomposition depends on the grouping and can omit within-bin structure. Treat it as a diagnostic, not an invariant fingerprint.

## 23.7 Calibration intercept and slope

On untouched validation data, form predicted log odds  $z$  and fit:

$$\text{logit } P(Y = 1) = a + bz.$$

Ideally  $a = 0$  and  $b = 1$ .

- $a < 0$  often means average risk is too high.
- $a > 0$  often means average risk is too low.
- $b < 1$  often means predictions are too extreme.

- $b > 1$  often means the risk range is too narrow.

These are estimates with uncertainty. Training predictions make the assessment optimistic.

## 23.8 Sigmoid and isotonic calibration

Sigmoid calibration fits a logistic mapping from score to probability. It is relatively stable but assumes an S-shaped correction.

Isotonic calibration fits a flexible nondecreasing staircase. It assumes higher scores should never map to lower risks, but it can overfit small calibration sets.

Valid calibration patterns use:

- a dedicated later calibration period;
- cross-fitted development predictions; or
- calibration entirely inside every outer validation fold.

The final calibrated procedure must be judged on untouched data. Fitting and assessing a calibrator on the same probabilities is leakage.

## 23.9 Why ECE is insufficient

Expected calibration error averages absolute bin gaps. It changes with bins, can hide local failures, and is not a strictly proper scoring rule for choosing forecasts. Report a proper score, a reliability display, sample support, and uncertainty.

---

# Day 24 — Thresholds, ranking, and class imbalance

---

## 24.1 A probability is not an action

A threshold rule is:

$$a_i = \mathbb{1}\{p_i \geq t\}.$$

Changing  $t$  changes actions without changing the probability model. A default 0.5 has no special ethical or economic status.

## 24.2 Confusion matrix from four counts

|           | Event | No event |
|-----------|-------|----------|
| Review    | TP    | FP       |
| No review | FN    | TN       |

From these counts:



$$\begin{aligned}\text{sensitivity} &= \frac{TP}{TP + FN}, & \text{specificity} &= \frac{TN}{TN + FP}, \\ \text{precision} &= \frac{TP}{TP + FP}, & \text{accuracy} &= \frac{TP + TN}{n}.\end{aligned}$$

Sensitivity asks what fraction of events were reviewed. Precision asks what fraction of reviewed projects actually had events. Their denominators differ.

Balanced accuracy averages sensitivity and specificity, giving the two classes equal weight. Always report raw counts because rates can be unstable in small groups.

## 24.3 Deriving a cost threshold

Suppose unnecessary review costs  $C_{FP}$  and missed overrun costs  $C_{FN}$ . With calibrated risk  $p$ :

$$C_{\text{review}} = C_{FP}(1 - p),$$

$$C_{\text{no review}} = C_{FN}p.$$

Review when the first is no larger than the second. Rearranging gives:

$$t = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

If a miss costs five times an unnecessary review,  $t = 1/6 \approx 0.167$ .

Real systems can have limited capacity, varying event severity, delayed reviews, human behavior, and project-specific costs. Then use explicit expected utility or constrained allocation rather than pretending one threshold captures everything.

## 24.4 Threshold tables and $F_1$

A validation threshold table should list review rate, sensitivity, specificity, precision, confusion counts, and cost for many plausible thresholds. Select according to the registered goal, never on the locked test.

$F_1$  is the harmonic mean of precision and recall:

$$F_1 = 2 \frac{PR}{P + R}.$$

It omits true negatives and is not a monetary cost, probability score, capacity rule, or severity model. Use it only when its trade-off matches the real task.

## 24.5 ROC curves and AUC

An ROC curve plots true-positive rate against false-positive rate over thresholds. ROC AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative, with half credit for ties.

ROC AUC measures ranking, not calibration. It averages threshold regions that operations may never use. A partial ROC region can be useful only if selected in advance.

## 24.6 Precision–recall curves

A PR curve plots precision against recall. Its no-skill precision baseline is prevalence, so it makes rare-event difficulty visible.

Precision changes when prevalence changes, even if class-conditional score distributions remain fixed. Average precision is a particular weighted summary across recall increments and is not identical to every trapezoidal “PR AUC.” Name the implementation.

## 24.7 What imbalance does and does not mean

Class imbalance merely describes unequal class frequency. It can make accuracy misleading, rare-event estimates noisy, and random folds deficient in events.

Possible responses answer different questions:

- **class weighting** changes how fitting errors are valued;
- **over/undersampling** changes the training distribution;
- **SMOTE** synthesizes points between minority neighbors;
- **threshold movement** changes actions while retaining the probability model.

Resampling must occur inside each training fold. Synthetic cases are not new independent event evidence and may violate mixed, constrained, geographic, or temporal structure. Weighted or resampled outputs may require probability correction and representative calibration.

## 24.8 Capacity-constrained review

If only 30 cases can be reviewed, a top-30 policy ranks each cohort. Its effective threshold changes with the cohort.

Evaluate event yield, recall among the top  $k$ , selection stability, subgroup allocation, and the usefulness of actual probabilities. Describe it as a rank allocation policy, not a fixed-threshold classifier.

---

# Day 25 — Nonlinear, kernel, and multiple-class models

---

## 25.1 Several meanings of nonlinear

Logistic regression has an S-shaped probability even when log odds are linear. Nonlinearity may instead enter through:

- polynomial terms such as  $x^2$ ;
- interactions such as  $x_1x_2$ ;
- spline basis functions;
- neighborhoods or kernels; or
- a complex learned decision surface.

Always name the scale on which a relationship is linear or nonlinear.

## 25.2 Polynomial and interaction logistic models

For a centered quadratic:

$$\text{logit}(p) = \beta_0 + \beta_1 x_c + \beta_2 x_c^2.$$

Centering means subtracting a meaningful reference, which makes coefficients and computation easier to interpret. The probability derivative is:

$$\frac{\partial p}{\partial x} = p(1 - p)(\beta_1 + 2\beta_2 x_c).$$

For an interaction:

$$\text{logit}(p) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2.$$

The slope for  $x_1$  depends on  $x_2$ . Log-odds interaction and probability interaction are not the same object, and neither proves causal effect modification.

## 25.3 Splines and generalized additive models

A spline represents a smooth curve using several basis columns  $B_m(x)$ :

$$\text{logit}(p) = \beta_0 + \sum_m \theta_m B_m(x).$$

A generalized additive model combines smooth feature functions:

$$g(\mathbb{E}[Y | X]) = \beta_0 + f_1(x_1) + \dots + f_p(x_p).$$

For binary logistic GAMs,  $g$  is the logit. Additivity improves readability but does not automatically learn interactions. Knot count, smoothness penalty, and chosen features belong inside validation. Plot curves with data density and uncertainty; smooth extrapolation beyond support is still unsupported.

## 25.4 K-nearest-neighbor classification

KNN finds the  $k$  closest training points and averages their labels:

$$\hat{p}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i.$$

Small  $k$  is flexible and noisy. Large  $k$  is smoother and drifts toward prevalence.

Distance silently encodes assumptions about scaling, categories, missingness, and feature relevance. Without scaling, a variable measured in thousands can dominate one measured from 1 to 5.

In high dimensions, points become sparse and distances often become similar: the curse of dimensionality. KNN needs disciplined features, adequate local data, and no claims beyond observed neighborhoods.

## 25.5 Support-vector machines

For labels encoded  $-1, +1$ , an SVM finds a separating score  $f(x)$  with a wide margin. Soft-margin SVM uses hinge loss:

$$\max(0, 1 - yf(x)).$$

The objective balances a small weight norm with margin violations. In the common `C` parameterization:

- small `C` allows more violations and stronger regularisation;
- large `C` emphasizes fitting violations.

Support vectors are points on or inside the margin that directly shape the solution.

## 25.6 Kernels and the RBF kernel

A kernel replaces an explicit inner product with a similarity function. The radial-basis-function kernel is:

$$K(x, x') = \exp[-\gamma \|x - x'\|_2^2].$$

Large  $\gamma$  makes similarity very local and permits wiggly boundaries. Small  $\gamma$  creates broader, smoother similarity. Scaling is essential, and `C` plus  $\gamma$  need honest nested tuning.

An SVM margin is not a probability. `probability=True` adds a calibration-like step and extra computation; its probability quality still requires untouched evaluation.

## 25.7 Multiclass classification and softmax

Multiclass outcomes have more than two mutually exclusive categories. Softmax turns class scores  $z_k$  into probabilities summing to one:

$$P(Y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Subtract the largest score before exponentiating for numerical stability. Cross-entropy sums the negative log probability assigned to each observed class.

One-versus-rest and one-versus-one decompose the problem differently and can have different calibration behavior. Do not invent classes to hide censoring or competing-event problems.

## 25.8 Macro, micro, and weighted metrics

- **Macro:** calculate each class metric, then average classes equally.
- **Weighted macro:** weight class metrics by class count.
- **Micro:** pool all class decisions before calculating the metric.

Macro can give ten examples the same influence as ten thousand; micro can conceal a rare-class failure. Report class support and per-class results.

## 25.9 Multilabel classification and chains

Multilabel means several labels can be true for one row. Independent models ignore dependence. A classifier chain predicts labels sequentially and uses earlier predictions as later inputs.

During training, true upstream future labels would leak information unavailable at deployment. Train downstream stages using out-of-fold upstream predictions, and feed only predictions at deployment. Chain order is itself a modeling choice.

---

# Day 26 — Classification trees and random forests

---

## 26.1 How a tree makes probabilities

A tree repeatedly splits feature space. Each terminal leaf predicts its event fraction:

$$\hat{p}_L = \frac{\text{events in leaf } L}{\text{rows in leaf } L}.$$

Every case in the leaf receives the same probability. A leaf with one event and no non-events reports 1.0 but has almost no support.

## 26.2 Gini impurity and entropy

For class proportions  $p_k$ :

$$I_G = 1 - \sum_k p_k^2.$$

For two classes,  $I_G = 2p(1 - p)$ . It is 0 for a pure node and largest at an even split.

Entropy is:

$$I_H = - \sum_k p_k \log p_k,$$

where  $0 \log 0$  is defined as 0. Both measure mixing, but they need not choose the same split.

## 26.3 Weighted split improvement

A split is judged by parent impurity minus size-weighted child impurity:

$$\Delta I = I_P - \frac{n_L}{n_P} I_L - \frac{n_R}{n_P} I_R.$$

Weighting prevents a tiny pure child from receiving all the credit while leaving most cases mixed. The tree greedily chooses the best available split now; it does not search every possible future tree.

## 26.4 Controlling tree complexity

- `max_depth` limits path length.
- `min_samples_leaf` protects leaf support.
- `min_samples_split` controls when splitting is considered.
- `max_leaf_nodes` caps leaves.
- `ccp_alpha` controls cost-complexity pruning.

Large leaves often improve probability stability. Select complexity inside validation and inspect leaf event counts, not only accuracy.

## 26.5 Preprocessing and extrapolation

Numeric trees do not need standardization because feature order is unchanged by a strictly monotone transformation. But categorical handling, missing values, rare categories, and high-cardinality IDs remain implementation-specific.

Beyond the training range, queries often fall into the same outer leaf, producing flat predictions. This is not evidence of safe extrapolation; it is one unsupported behavior.

## 26.6 Bagging and why correlation matters

Bagging fits trees on bootstrap samples and averages probabilities:

$$\hat{p}_{bag}(x) = \frac{1}{M} \sum_m \hat{p}_m(x).$$

If tree predictions have variance  $\sigma^2$  and pairwise correlation  $\rho$ , the average has idealized variance:

$$\rho\sigma^2 + \frac{1-\rho}{M}\sigma^2.$$

More trees shrink the independent portion but cannot remove shared error. Diversity matters.

## 26.7 Random forests

A random forest also offers each split only a random feature subset. Strong variables cannot dominate every tree, which can reduce correlation and let alternative patterns contribute.

Important settings include tree count, candidate features per split, leaf size, depth, bootstrap size, and class weights. More trees reduce Monte Carlo noise; they do not repair leakage, bias, or miscalibration.

## 26.8 Out-of-bag prediction

A bootstrap sample of size  $n$  contains about 63.2% distinct rows. For each training row, an OOB prediction averages only trees that did not include that row.

OOB estimates are useful internal evidence under bootstrap-like exchangeability, but ordinary bootstraps mix years and groups. They do not replace forward, grouped, geographic, or external validation.

## 26.9 Forest probability quality and importance

Forest probabilities average leaf proportions. Leaf size, class weighting, and sampling alter their spread, so good ROC AUC does not guarantee calibration.

Impurity importance credits accumulated split gains. It can favor continuous or high-cardinality features and split credit strangely among correlated variables. Held-out permutation methods help, but no importance score is a causal effect.

---

# Day 27 — Gradient boosting and honest tuning

---

## 27.1 Bagging versus boosting

Bagging fits many learners mostly independently and averages them. Boosting builds sequentially:

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^M h_m(x).$$

Each new weak learner tries to reduce the current loss.  $F$  is a raw score,  $h_m$  is often a shallow tree,  $\eta$  is learning rate, and  $M$  is the number of stages.

## 27.2 Gradient descent in function space

Boosting changes a function rather than one fixed coefficient vector. At stage  $m$ , calculate negative loss gradients:

$$r_{im} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}.$$

Fit a weak learner to these pseudo-residuals, then add a small version of it. For squared error they equal ordinary residuals. For other losses they do not.

## 27.3 Logistic pseudo-residuals and initial score

For logistic loss,  $p = \sigma(F)$  and:

$$\frac{\partial L}{\partial F} = p - y.$$

Therefore the negative gradient is:

$$r = y - p.$$

Boosting begins at training log prevalence:

$$F_0 = \log \left( \frac{\hat{\pi}}{1 - \hat{\pi}} \right).$$

Its sigmoid is the constant prevalence baseline. Each tree then adds structure.

## 27.4 Interacting complexity controls

- smaller learning rate usually needs more stages;
- deeper or larger trees learn higher-order interactions;
- minimum leaf size protects local support;
- row and feature subsampling add randomness;
- penalties and leaf rules depend on the implementation.

There is no universal best range. A parameter setting only has meaning with a package version, dataset, preprocessing procedure, and search budget.

## 27.5 Histogram boosting

Histogram methods bin numeric feature values and search over bins, saving time and memory. Implementations differ in missing-value routing, categorical support, objectives, and regularisation. XGBoost, LightGBM, CatBoost, and scikit-learn estimators are related, not interchangeable.

Read the documentation for the exact version. Raw strings are not automatically accepted merely because numeric missing values are.

## 27.6 Early stopping is model selection

Early stopping chooses the training length using validation loss. A hidden random validation fraction can violate a temporal design.

Use an order-respecting validation period, tune iteration count inside temporal folds, or implement expanding-window selection. The locked test must never decide when to stop.

## 27.7 Grid, random, and sequential search

- **Grid search** evaluates a fixed Cartesian grid; transparent but wasteful in many dimensions.
- **Random search** samples from stated distributions; often explores influential dimensions better for a fixed budget.
- **Bayesian/sequential search** uses prior trials to choose promising next trials; efficient but still adaptive and capable of overfitting validation.

Scale parameters often need log-uniform sampling. Uniformly sampling  $C$  from 0.0001 to 1000 spends almost all draws at the large end; uniformly sampling  $\log_{10} C$  spreads trials across orders of magnitude.

Record spaces, distributions, seed, trial count, failures, stopping, hardware, and every result.

## 27.8 Nested evaluation

The inner loop tunes preprocessing, features, hyperparameters, calibration, resampling, early stopping, and possibly threshold. The outer loop evaluates the resulting selection procedure.

Outer splits must still match deployment. Random nested CV does not cure temporal or group mismatch.

## 27.9 Fair search budgets and uncertainty

A default logistic model versus 1,000 boosted-model trials is a comparison of complete procedures and budgets, not pure algorithms. Report performance as well as computation.

The best inner score is the minimum of many noisy estimates and is optimistically selected. Use a locked test or outer folds, report all fold scores and paired differences, align uncertainty with the sampling unit, and focus on practical differences.

---



# Day 28 — Interpreting fitted models without inventing causes

---

## 28.1 Specify the explanation request

Before choosing a method, state:

- object: one prediction, global behavior, or performance dependence;
- audience: developer, reviewer, policymaker, or affected party;
- scale: probability, log odds, margin, loss, or action;
- reference: compared with what background;
- scope: fitted model or real-world mechanism; and
- use: debugging, contesting, understanding, or justifying.

No explanation method turns prediction into causation.

## 28.2 Intrinsic and post-hoc interpretation

Logistic coefficients, sparse scorecards, shallow trees, additive shape models, and monotonic models expose constrained structures. Post-hoc methods probe a fitted model afterward.

A simple model can be readable and wrong. A post-hoc method can be useful but adds perturbation, approximation, and reference assumptions. Match fidelity and complexity to the use.

## 28.3 Coefficients are not generic importance

Logistic coefficients depend on units, coding, correlated variables, interactions, regularisation, omissions, measurement error, and sampling. Comparing a one-kilometre coefficient with a one-million-rupee coefficient is meaningless without a defined perturbation.

## 28.4 Permutation importance

On held-out data, shuffle feature  $j$  and measure score deterioration:

$$I_j = S(f, D) - S(f, D_j^{perm}).$$

It asks how much this fitted model's held-out performance relies on information disrupted by this permutation. It does not ask how much the feature causes the outcome.

Correlated substitutes can make each feature look weak. Unconditional shuffling can create impossible combinations. Consider grouped or conditional permutations and clearly state the added assumptions. Retraining without a feature asks a different question.

## 28.5 Partial dependence and ICE

Partial dependence replaces a focal feature across rows and averages predictions:

$$PD_S(x_S) = \mathbb{E}_{X_C}[f(x_S, X_C)].$$

Other features retain their row-specific values; they are not all fixed at one shared value. Correlated features can create unsupported synthetic combinations.

ICE shows the corresponding curve for one row:

$$ICE_i(x_S) = f(x_S, x_{iC}).$$

Diverging ICE curves reveal model heterogeneity hidden by an average. Both remain model perturbations, not causal experiments. Show support and output scale.

## 28.6 Accumulated local effects

ALE estimates local prediction changes inside observed feature intervals, averages them, accumulates them across the range, and centers the result. It often stays closer to observed joint support than PDP under correlation.

It still depends on bins, density, and meaningful local perturbations. Sparse regions remain uncertain.

## 28.7 Local surrogate explanations

A local surrogate perturbs points near one case, queries the black box, weights by proximity, and fits a simple approximation. Results depend on neighborhood, scaling, perturbation distribution, kernel, surrogate family, and seed.

Always report local fidelity. A readable but inaccurate surrogate does not explain the original model.

## 28.8 Shapley values and SHAP

Shapley values average a feature's marginal contribution over every order in which features can join a coalition:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(M - |S| - 1)!}{M!} [v(S \cup \{j\}) - v(S)].$$

For two features, each can enter first or second, so average its contribution across the two orders. Additive SHAP explanations satisfy:

$$f(x) = \phi_0 + \sum_j \phi_j$$

on the explainer's stated scale.

The hard question is what  $v(S)$  means when features are “missing.” Interventional-style explanations can break dependencies; conditional-style explanations preserve modeled dependencies and can credit proxies. Neither is automatically causal.

State the algorithm, background population, dependence treatment, output scale, link, approximation, and version. Log-odds contributions cannot simply be read as probability-point contributions because the sigmoid is nonlinear.

## 28.9 Stability and legitimacy

Repeat explanations across seeds, temporal folds, bootstrap fits, correlated substitutions, and plausible backgrounds. Instability may be the most decision-relevant finding.

Explanations cannot repair an invalid label, excluded population, leakage, poor calibration, unfair allocation, lack of recourse, or an ineffective intervention. Explanation follows system validity; it does not substitute for it.

---

# Day 29 — Fairness, external validation, and the locked study

---

## 29.1 Begin before metrics

Ask whether the task is legitimate, the label is equally measured, the action helps or burdens, important populations are absent, records and decisions can be contested, and an accountable institution can respond to harm.

A parity number diagnoses one distributional property. It is not a complete ethical theory.

## 29.2 Group attributes have different roles

Separate:

- attributes used as model features;
- attributes retained only for auditing;
- attributes used in decision policy; and
- governance or legal restrictions.

Dropping a group attribute does not prevent proxies or historical labels from reproducing disparities. Retaining an attribute for audit does not require using it for prediction.

## 29.3 Demographic parity

Demographic parity asks for equal positive-action rates:

$$P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b).$$

It may matter for a scarce benefit or burden, but it ignores observed outcomes. If review is supportive and structural barriers create different risks, equal review rates could under-support a group. Meaning depends on the action.

## 29.4 Equal opportunity and equalized odds

Equal opportunity asks for equal TPR among actual events:

$$P(\hat{Y} = 1 \mid Y = 1, A = a) = P(\hat{Y} = 1 \mid Y = 1, A = b).$$

Equalized odds requires both TPR and FPR equality, equivalently  $\hat{Y}$  independent of  $A$  given  $Y$ . These describe thresholded actions, not probability calibration.

## 29.5 Predictive parity and group calibration

Predictive parity asks for equal precision among positive actions. Group calibration asks that within each group, cases assigned probability  $p$  experience events at rate  $p$ .

These condition on different quantities. With different base rates and imperfect prediction, several desirable parity properties generally cannot all hold. Choosing among them is a normative and operational decision, not metric shopping.

## 29.6 Measurement disparity and small groups

If remote projects close accounts later and unresolved cases become 0, the dataset can manufacture group differences. Audit label completeness, follow-up, missing features, measurement errors, and appeals by group before adjusting algorithms.

Intersectional groups can reveal hidden failures but create small samples and repeated comparisons. Report counts, confusion cells, uncertainty, pre-specified versus exploratory status, privacy rules, and collection plans. Lack of evidence is not evidence of equality.

## 29.7 Types of validation

- **Internal random:** similar sampled rows from one source.
- **Internal grouped:** whole known groups held out.
- **Temporal:** later periods in the broad same system.
- **Geographic/external:** independent represented locations or institutions.
- **Prospective:** model frozen before future cases accrue.
- **Impact evaluation:** causal comparison of using versus not using the intervention.

A future-year test in the same districts is temporal, not broad geographic validation. Prospective prediction performance still does not prove that review improves outcomes.

## 29.8 Dataset shift

Training and deployment distributions may differ:

- **covariate shift:**  $P(X)$  changes while  $P(Y | X)$  is assumed stable;
- **label shift:**  $P(Y)$  changes while  $P(X | Y)$  is assumed stable;
- **concept/conditional shift:**  $P(Y | X)$  changes.

These are assumptions, not labels that an unlabeled dashboard proves. Monitor feature distributions, missingness, risk distribution, mature prevalence, calibration, actions, subgroup metrics, pipeline changes, overrides, and appeals.

## 29.9 The locked Chapter 4 protocol

The chapter freezes:

- projects through 2022 for development;
- 2019–2022 as forward validation years;
- 2023–2025 as a locked temporal test;
- log loss as the primary metric, equally averaged across validation years;
- Brier, ROC AUC, and average precision as secondary views;
- a  $1/6$  action threshold from FP cost 1 and FN cost 5;
- access mode and district audits with raw counts; and
- prior, logistic, engineered logistic, forest, and histogram-boosting candidates.

The test is opened once after selection. Threshold, calibration, preprocessing, and candidates are part of the frozen procedure.

## 29.10 Reading the benchmark code

The code:

1. creates and time-sorts simulated projects;
2. separates development from locked test;
3. asserts prohibited future fields are not features;
4. builds all candidate pipelines;
5. fits each candidate on years before each validation year;
6. selects by mean validation log loss;
7. refits the selected procedure on all development rows;
8. predicts the locked test once;
9. compares with a fixed logistic reference using paired row losses;
10. produces subgroup and reliability tables; and
11. saves scores, predictions, audits, and environment metadata.

The custom transformer learns centers only in `fit`, then reuses them in `transform`. `clone` creates a fresh unfitted copy. Pipelines ensure imputation, scaling, encoding, feature engineering, and modeling obey the same fold boundary.

## 29.11 Why the threshold is not tuned

The threshold comes from a registered simplified cost ratio; the model is selected for probability quality. Tuning the threshold on the locked test would spend the test and blur probability estimation with action optimization.

If costs are uncertain, pre-register sensitivity or decision-curve analyses over plausible costs rather than selecting whichever makes test results attractive.

## 29.12 Subgroups and uncertainty

For every group report sample size, events, prevalence, mean probability, Brier score, review rate, TP, FP, FN, TN, sensitivity, specificity, precision, and cost.

Differences can arise from chance, prevalence, measurement, model error, threshold effects, or structural inequity. A rate alone cannot identify the mechanism.

The chapter's paired bootstrap resamples rows. Shared year or district shocks violate simple row exchangeability, so its interval is descriptive. Better research aligns resampling with independent sampling units and collects more periods and sites.

## 29.13 Prospective shadow deployment

Before influencing real decisions:

1. freeze code, environment, feature definitions, and policy;
2. issue timestamped predictions without changing decisions;
3. verify feature availability and latency;
4. record human actions, overrides, and reasons;
5. wait until outcomes mature;
6. assess probability, action, subgroup, and workflow results;
7. establish contest, incident, pause, and retraining rules; and
8. consider a separate ethical impact evaluation.

Once a deployed system changes decisions, later labels reflect both the world and the intervention. Drift monitoring must account for that feedback.

## 29.14 The bounded conclusion

A defensible conclusion names the represented projects and period, selected procedure, locked probability and action results, subgroup limitations, and next validation step. It explicitly does not claim causal risk factors, fairness, intervention benefit, or transport to untested regions.

## 29.15 Structure of the final research report

The report should let a reader reconstruct decisions, not only admire the winning model:

1. **Abstract:** event, population, prediction time, design, locked result, and main limitation.
2. **Label study:** data source, window, unresolved cases, and group measurement audit.
3. **Prediction contract:** intended and prohibited uses, feature timing, and action.
4. **Methods:** complete candidates, folds, search budget, scores, threshold, and uncertainty.
5. **Validation:** every candidate and year, including failures and near ties.
6. **Locked results:** probability, ranking, calibration, and action evidence.
7. **Subgroup audit:** raw counts, rates, uncertainty, and exploratory labels.
8. **Interpretation:** method, output scale, background, support, fidelity, and stability.
9. **Transport:** temporal, geographic, workflow, and outcome-maturity limits.
10. **Governance:** human review, contestability, monitoring, incidents, and retraining.
11. **Reproducibility:** data and code versions, environment, seeds, predictions, and deviations.
12. **Conclusion:** a bounded statement separating prediction from cause and benefit.

Save every locked-test probability, not only aggregate metrics. Predictions permit paired comparisons, reliability reconstruction, subgroup audit, and later error review. Preserve deviations from the protocol rather than quietly rewriting the plan.

---

# Chapter 4 synthesis and practical laboratories

---

## S.1 The six evidence layers

1. **Event measurement:** Is the label valid and mature?
2. **Probability estimation:** Does the fitted procedure estimate supported risks?
3. **Probability evaluation:** Are ranking and calibration adequate on untouched data?
4. **Decision policy:** Are consequences, capacity, and human roles explicit?
5. **Group and governance audit:** Who receives errors, actions, recourse, and accountability?
6. **Transport and impact:** Does performance persist elsewhere, and does use help?

Later layers cannot repair earlier ones.

## S.2 Derivations to complete by hand

1. Convert probability to odds, log odds, and back.
2. Derive Bernoulli mean and variance from the two outcomes.

3. Derive binary log loss from the likelihood.
4. Derive  $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})/n$  and  $\mathbf{X}^\top \mathbf{W} \mathbf{X}/n$ .
5. Show why the Hessian is positive semidefinite.
6. Derive  $\partial \mathbf{p} / \partial x_j = \beta_j \mathbf{p}(1 - \mathbf{p})$ .
7. Prove Brier and log-loss propriety.
8. Derive the binned Brier decomposition and state its bin dependence.
9. Derive the cost threshold.
10. Use Bayes' rule to express precision through prevalence, sensitivity, and specificity.
11. Derive binary Gini  $2\mathbf{p}(1 - \mathbf{p})$ .
12. Derive variance of an average of equally correlated trees.
13. Derive the logistic pseudo-residual  $\mathbf{y} - \mathbf{p}$ .
14. Calculate all coalitions for a three-feature Shapley game.
15. Express equalized odds as conditional independence.

## S.3 Ten laboratories

### Laboratory A: label maturity

Simulate different close-out times. Compare coding incomplete cases as 0 with waiting for a fixed outcome window. Plot prevalence and calibration by project age.

### Laboratory B: logistic optimization

Compare gradient descent, Newton/IRLS, and a library solver under scaling, collinearity, and near separation. Record loss, gradient norm, iterations, and runtime.

### Laboratory C: probability quality

Build two forecasts with equal 0.5-threshold accuracy but different confidence. Compare Brier, log loss, reliability, calibration intercept, and slope.

### Laboratory D: prevalence shift

Keep positive and negative score distributions fixed while changing class frequency. Observe ROC stability and changes in precision, PR curves, probability calibration, and decision cost.

### Laboratory E: nonlinear support

Fit spline logistic, KNN, RBF SVM, forest, and boosting on a known two-dimensional probability surface. Evaluate within support and beyond it.

### Laboratory F: forest correlation

Add redundant strong predictors and vary feature subsampling. Measure individual-tree performance, tree correlation, ensemble Brier score, and stability.

### Laboratory G: adaptive tuning

Search 5, 50, and 500 configurations on noise. Compare best inner validation loss with outer and independent-test loss.

## Laboratory H: explanation disagreement

Fit near-tied models using correlated features differently. Compare coefficients, impurity and permutation importance, PDP, ICE, ALE, and SHAP under multiple backgrounds.

## Laboratory I: fairness trade-offs

Simulate two groups with different prevalence. Compare one threshold, equal-TPR thresholds, equalized-odds post-processing, and equal review rates. Preserve every individual decision change.

## Laboratory J: prospective registry

Store feature timestamps, data version, model version, probability, threshold, action, override, outcome-maturity date, and final event. Try reconstructing omitted fields later to see why prospective logging matters.

## S.4 Model-family comparison questions

For every candidate ask:

- What representation does it assume?
- Does it need scaling or encoding?
- What probability target does fitting optimize?
- Which settings were learned from validation data?
- How does it behave outside feature support?
- Does it return probabilities or only scores?
- How will it be calibrated?
- What computation and monitoring does it require?
- How stable are its predictions and explanations?
- What claim does the evaluation design permit?

## Formula sheet with plain-language readings

| Formula                         | Plain-language reading                   |
|---------------------------------|------------------------------------------|
| $p/(1-p)$                       | event odds                               |
| $\log[p/(1-p)]$                 | log odds on the real line                |
| $1/(1+e^{-z})$                  | turn a real score into probability       |
| $p^y(1-p)^{1-y}$                | probability of one binary observation    |
| $p(1-p)$                        | Bernoulli variance                       |
| $-[y \log p + (1-y) \log(1-p)]$ | penalty for one probability forecast     |
| $X^\top(p-y)/n$                 | logistic-loss gradient                   |
| $X^\top W X/n$                  | logistic curvature                       |
| $e^{\beta_j}$                   | conditional odds multiplier for one unit |
| $n^{-1} \sum (p_i - y_i)^2$     | Brier score                              |



| Formula                               | Plain-language reading                            |
|---------------------------------------|---------------------------------------------------|
| $P(Y = 1 \mid P = p) = p$             | calibration definition                            |
| $TP/(TP + FN)$                        | sensitivity or recall                             |
| $TN/(TN + FP)$                        | specificity                                       |
| $TP/(TP + FP)$                        | precision                                         |
| $(TPR + TNR)/2$                       | balanced accuracy                                 |
| $2PR/(P + R)$                         | $F_1$ summary                                     |
| $C_{FP}/(C_{FP} + C_{FN})$            | simplified cost threshold                         |
| $1 - \sum p_k^2$                      | Gini impurity                                     |
| $I_P - (n_L I_L + n_R I_R)/n_P$       | split improvement                                 |
| $M^{-1} \sum_m \hat{p}_m(x)$          | bagged probability                                |
| $\rho\sigma^2 + (1 - \rho)\sigma^2/M$ | variance left after averaging correlated learners |
| $\exp[-\gamma\ x - x'\ ^2]$           | RBF similarity                                    |
| $\max(0, 1 - yf)$                     | hinge loss                                        |
| $e^{z_k} / \sum_j e^{z_j}$            | softmax class probability                         |
| $F_m = F_{m-1} + \eta h_m$            | one boosting update                               |
| $y - p$                               | logistic negative gradient                        |
| $S(D) - S(D_j^{perm})$                | permutation score decrease                        |
| $\mathbb{E}[f(x_S, X_C)]$             | partial dependence                                |
| $f(x) = \phi_0 + \sum_j \phi_j$       | additive SHAP explanation                         |

## Expanded glossary

**Action threshold** — A rule converting probability or score into an action.

**Average precision** — A prevalence-sensitive weighted summary of precision over recall increments.

**Bagging** — Fitting learners on bootstrap samples and averaging their outputs.

**Baseline** — A simple reference procedure a candidate should improve upon.

**Bernoulli distribution** — A probability model for one outcome coded 0 or 1.

**Brier score** — Mean squared error of binary probability forecasts.

**Calibration** — Agreement between forecast probabilities and event frequencies in a stated population.

**Classifier chain** — A multilabel method that feeds earlier label predictions into later models.

**Complete separation** — Perfect feature-based class division that makes unpenalised logistic estimates diverge.

**Concept shift** — A change in  $P(Y | X)$ .

**Confusion matrix** — TP, FP, FN, and TN action counts.

**Covariate shift** — A change in  $P(X)$  under an assumed stable  $P(Y | X)$ .

**Cross-entropy** — Negative log likelihood used to assess or fit class probabilities.

**Demographic parity** — Equality of positive-action rates across groups.

**Discrimination** — Ability to rank events above non-events.

**Early stopping** — Choosing training length using validation performance.

**Equal opportunity** — Equality of true-positive rates across groups.

**Equalized odds** — Equality of both true-positive and false-positive rates across groups.

**Event horizon** — The period during which a future event is assessed.

**Generalized additive model** — A linked outcome mean represented by a sum of smooth feature functions.

**Gini impurity** — A measure of class mixing within a tree node.

**Gradient** — The vector of first derivatives of an objective.

**Gradient boosting** — Sequential additive fitting along negative loss gradients.

**Hessian** — A matrix of second derivatives describing local curvature.

**ICE** — One row's fitted prediction curve as a focal feature is varied.

**Impact evaluation** — A causal design estimating the effect of using an intervention.

**Index time** — The moment at which eligibility, features, and prediction are anchored.

**Isotonic calibration** — A flexible nondecreasing mapping from score to probability.

**Kernel** — A function representing similarity or an implicit feature-space inner product.

**Label shift** — A prevalence change under assumed stable  $P(X | Y)$ .

**Likelihood** — Plausibility of observed data as a function of model parameters.

**Log loss** — Negative log probability assigned to observed classes.

**Log odds** — Logarithm of event odds.

**Macro average** — Equal-weight mean of class-specific metrics.

**Multiclass** — More than two mutually exclusive classes.

**Multilabel** — Several binary labels may be true for one row.

**Nested validation** — Inner model selection surrounded by outer procedure evaluation.

**Odds ratio** — Multiplicative change in conditional odds.

**Out of bag** — Rows omitted from a tree's bootstrap sample and usable for that tree's internal assessment.

**Partial dependence** — Average fitted output after replacing focal feature values across rows.

**Permutation importance** — Held-out score deterioration after shuffling feature information.

**Precision** — Event fraction among positive actions.

**Prevalence** — Event fraction in a specified population, sample, and window.

**Proper scoring rule** — A score whose expected optimum is truthful probability reporting.

**Prospective validation** — Evaluation on future cases after the procedure is frozen.

**Pseudo-residual** — Negative loss gradient used as a boosting-stage target.

**Random forest** — Bootstrap trees with random feature availability at splits.

**Recall** — Sensitivity or true-positive rate.

**Reliability diagram** — Binned mean probabilities compared with event rates.

**ROC AUC** — Ranking summary over true- and false-positive rates.

**Sensitivity** — Event fraction assigned positive action.

**SHAP** — Additive feature-attribution methods based on Shapley ideas.

**Sigmoid** — The function mapping a real score into a value between 0 and 1.

**Specificity** — Non-event fraction assigned negative action.

**Support vector** — A point on or within the SVM margin that shapes the solution.

---

## Research-paper reading ladder

---

| Paper                            | Main question to reconstruct                                                   | Beginner replication                                         |
|----------------------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------|
| Nelder & Wedderburn (1972)       | How do distribution, linear predictor, link, and iterative fitting form a GLM? | Implement IRLS and create separation                         |
| Brier (1950)                     | Why evaluate probability forecasts directly?                                   | Verify squared-loss propriety by simulation                  |
| Niculescu-Mizil & Caruana (2005) | Why can ranking and calibration differ by algorithm?                           | Calibrate four model families without leakage                |
| Davis & Goadrich (2006)          | How are ROC and PR spaces related?                                             | Alter prevalence while keeping score distributions fixed     |
| Breiman (2001)                   | How do forest strength and tree correlation affect error?                      | Measure correlation while changing <code>max_features</code> |
| Friedman (2001)                  | How is boosting function-space optimization?                                   | Fit stumps to $y - p$ stage by stage                         |
| Grinsztajn et al. (2022)         | Under which benchmark design do tree methods excel on tabular data?            | Give four families equal search budgets                      |
| Lundberg & Lee (2017)            | Which axioms define additive feature attribution?                              | Change background and dependence assumptions                 |
| Hardt, Price & Srebro (2016)     | What do equal opportunity and equalized odds require?                          | Compare parity rules, utility, and changed decisions         |

Read every paper in seven passes:

1. State its question in one falsifiable sentence.
2. Identify the mathematical object estimated or bounded.
3. Rewrite the algorithm as pseudocode.
4. List sampling, measurement, model, and optimization assumptions.
5. Reproduce one derivation, table, or simulation.
6. Construct a setting where the method should fail.
7. State what new evidence application to MHP projects would require.

---

## Suggested nine-day study rhythm

---

Use roughly three hours per day:

1. 15 minutes: retrieve yesterday's exit check.
2. 20 minutes: define the event or decision scenario.
3. 35 minutes: work a tiny numerical example.
4. 45 minutes: derive the central equation.
5. 55 minutes: type, run, and alter code.
6. 25 minutes: create one deliberate failure.
7. 20 minutes: reconstruct one paper claim.
8. 10 minutes: write one bounded conclusion.

Do not compress all days into a model-comparison weekend. The label, probability, action, explanation, and governance distinctions need repeated retrieval.

---

## Common beginner misconceptions

---

- Binary does not mean fully observed.
- Unknown is not automatically 0.
- Odds are not probabilities.
- Logistic regression is linear on log odds, not probability.
- An odds ratio is not a percentage-point risk change.
- A convergence warning is not harmless formatting.
- Accuracy is not probability quality.
- ROC AUC is not calibration.
- A 0.5 threshold is not neutral.
- $F_1$  is not a general cost function.
- Class weighting and threshold movement are not interchangeable.
- SMOTE does not create independent event evidence.
- KNN and RBF kernels require meaningful scaling.
- SVM margins are not probabilities.
- Trees avoid numeric scaling but not preprocessing decisions.
- OOB evidence is not automatically temporal evidence.
- Classification boosting does not always fit ordinary residuals.

- Early stopping uses validation information.
  - More tuning creates more selection opportunity.
  - Coefficients and importance values are not causal effects.
  - PDP changes a feature across rows; it does not hold all other features at one common value.
  - SHAP depends on background, dependence treatment, and output scale.
  - Dropping group attributes does not prove fairness.
  - One parity metric does not define justice.
  - Temporal validation is not broad external validation.
  - Prospective prediction does not prove intervention benefit.
- 

## Ethical and governance questions

---

Discuss these with model developers, reviewers, funders, affected communities, and domain experts:

- Who defines a major overrun and can challenge that definition?
  - Is review supportive, burdensome, or both?
  - Should high predicted risk trigger more resources instead of restriction?
  - Who supplies false-positive and false-negative costs?
  - How is limited review capacity allocated?
  - Who may correct a feature record or appeal an action?
  - How are overrides recorded without punishing sound judgment?
  - Which groups have incomplete labels or missing features?
  - What disparities require investigation, mitigation, or stopping?
  - Who can pause the system after an incident?
  - When does a changed workflow require recalibration or full revalidation?
  - How will an impact study determine whether review actually helps?
- 

## Final readiness checklist

---

You are ready to leave Chapter 4 when you can:

- defend unit, eligibility, event, time boundaries, and unresolved-outcome policy;
- move among probability, odds, and log odds;
- derive Bernoulli likelihood, log loss, logistic gradient, and Hessian;
- interpret coefficients without confusing odds and risk;
- recognize separation and respond safely;
- distinguish discrimination, calibration, and utility;
- prove Brier and log loss are proper;
- design calibration without leakage;
- calculate every confusion-matrix rate;
- derive and challenge a cost threshold;
- explain ROC and PR behavior under prevalence change;
- distinguish weighting, resampling, calibration, and thresholding;
- describe spline, KNN, SVM, softmax, and multilabel assumptions;
- derive Gini gain and forest averaging behavior;

- derive logistic boosting pseudo-residuals;
- place tuning and early stopping inside honest validation;
- define the perturbation and scale behind every explanation;
- calculate a small Shapley example;
- compare fairness definitions and explain their conflicts;
- distinguish internal, temporal, geographic, prospective, and impact evidence;
- run the locked benchmark without revising it after test exposure; and
- write a conclusion limited to the data, period, workflow, and action actually studied.

The central lesson is simple even though the machinery is rich:

A classifier is not only a model. It is an event definition, a probability procedure, an evaluation design, an action policy, and an accountable institution.

# Interlude — From a Single Moment to a History Through Time

---

## Two short days between Chapter 4 and Chapter 5

**Central promise.** Chapter 4 asked "did the event happen by month 12?" — a single yes/no, with a single probability attached. Chapter 5 asks "*when* does the event happen?" — and answering that means working with a probability that's spread out continuously over time, plus two pieces of mathematics this book hasn't used yet: integrals and formal limits. By the end of this interlude, a sentence like "the hazard is the instantaneous event rate among those still event-free" will read as an idea you can compute, not a wall of symbols.

## Why this exists

Look at what Chapter 5, Day 31 does in its opening section. In about thirty lines it introduces a cumulative distribution function, a probability density, a hazard defined as a limit of a conditional probability divided by a shrinking interval, an integral linking hazard to cumulative hazard, and the identity  $S(t) = \exp[-H(t)]$ . Chapter 4 prepared you for the *conditional probability* and the *exp/log* parts — logistic regression uses both constantly. It did not prepare you for **continuous** random variables, for **integrals**, or for a **formal limit**. Those three ideas arrive on Day 31 with no runway, and everything from Kaplan–Meier to the Cox model is built on top of them.

This interlude is short on purpose — you're not starting from zero the way Chapter 0 did. You already know discrete probability, likelihood, and derivatives cold. We're extending three things you know into a fourth thing you don't, not starting over.

Same rules as always: code proves every claim, you build something small, you break it, you don't move on until the exit check passes.

---

## Interlude Day A — From a Single Probability to a Curve Over Time

---

**Today's central idea:** When an outcome can land anywhere on a continuous scale instead of just "yes" or "no," you can't ask "what's the probability of exactly this value?" — the honest question becomes "what's the probability of landing in this small window?"

### A.1 What Chapter 4 gave you, and where it stops

Chapter 4, Day 21 built everything on a Bernoulli outcome: a project either triggers a cost-overrun warning by month 12, or it doesn't. One probability,  $p$ , describes the whole thing.

```
import numpy as np

Chapter 4's world: did it happen by month 12? One number answers everything.
p_warning_by_12 = 0.30
```

But a project that triggers a warning at month 3 and one that triggers a warning at month 11 both get coded as `1` in that world. Chapter 5's whole reason for existing is that this throws away real information — *when* it happened, and for the ones that haven't happened yet, *how long they've already survived without it*. To keep that information, the outcome can no longer be a single 0-or-1; it has to be a number that can land anywhere on the timeline: 3.2 months, 11.87 months, 40.001 months. That's a **continuous random variable**, and it needs new machinery.

## A.2 Why you can't just ask "what's the probability of exactly month 11.870000...?"

Here's the uncomfortable fact first: for a continuous outcome, the probability of hitting any *exact* value is zero. There are infinitely many possible values between month 11 and month 12 — 11.1, 11.01, 11.001, forever — so no single one of them can carry a positive share of the total probability. This isn't a technicality to wave away; it's the reason the whole toolkit looks different from Chapter 4's.

**Code proof — watch the probability of "close to 11.87" shrink as "close" gets stricter:**

```
rng = np.random.default_rng(0)

Simulate 200,000 synthetic warning times (months), just to look at the shape.
(This is illustrative data, not the real KP mechanism Chapter 5 will use.)
warning_times = rng.exponential(scale=20.0, size=200_000)

for window in [2.0, 0.5, 0.05, 0.005]:
 hits = np.sum(np.abs(warning_times - 11.87) < window)
 prob_estimate = hits / len(warning_times)
 print(f"window={window:<6} estimated P ≈ {prob_estimate:.5f}")
```

As the window shrinks, the estimated probability keeps shrinking toward zero — but the probability *per unit width* of that window settles down to a stable number. That stable number — probability per unit width, not probability itself — is exactly what a **density** is.

## A.3 The density: probability, rescaled by width

If  $f(t)$  is the density of event time  $T$ , then the probability of landing in a small window of width  $\Delta t$  around  $t$  is approximately  $f(t) \cdot \Delta t$  — density times width, the same "rate times interval" idea you'll recognise from Day 0E's slope work, just for probability instead of cost.

```
Estimate the density near t=11.87 directly: (probability of a window) / (window width)
window = 0.5
hits = np.sum(np.abs(warning_times - 11.87) < window)
prob_in_window = hits / len(warning_times)
density_estimate = prob_in_window / (2 * window) # window extends both directions
print(f"estimated density near t=11.87: {density_estimate:.5f}")
```

A density is **not** a probability — it can be larger than 1, because it's a rate, not a share. Only density multiplied by a width gives you back an actual probability. This is exactly analogous to Chapter 5's warning about hazard later: *a rate is not a probability, even though it's built from one*.



## A.4 The cumulative distribution function and the survival function

You've actually already met the discrete cousin of this in Chapter 4:  $P(T \leq t)$ . For a continuous  $T$ , this accumulated probability is called the **cumulative distribution function**:

$$F(t) = P(T \leq t)$$

Chapter 5 will spend far more time on its complement, the **survival function** — directly meaningful in the KP running case as "the probability a project has *not yet* triggered a warning by month  $t$ ":

$$S(t) = P(T > t) = 1 - F(t)$$

**Code proof — build both from raw simulated data, no library shortcuts:**

```
sample = np.sort(warning_times)[:1000] # a manageable sample, sorted

def empirical_cdf(sample, t):
 return np.mean(sample <= t)

def empirical_survival(sample, t):
 return np.mean(sample > t)

for t in [5, 10, 20, 40]:
 print(f"t={t:>3} F(t)≈{empirical_cdf(sample, t):.3f} "
 f"S(t)≈{empirical_survival(sample, t):.3f} "
 f"sum={empirical_cdf(sample, t) + empirical_survival(sample, t):.3f}")
```

Notice  $F(t) + S(t)$  always comes out to (approximately) 1 — they're complements by construction, the same way "probability of rain" and "probability of no rain" always summed to 1 back in Chapter 4. This crude "count how many are past  $t$ " estimator is exactly the starting intuition for Kaplan–Meier — the only thing Day 31 adds is a correction for the projects whose exact time you never got to see because they were censored.

## A.5 Build: a tiny empirical survival curve

```
def survival_curve(sample, time_grid):
 return np.array([empirical_survival(sample, t) for t in time_grid])

time_grid = np.linspace(0, 60, 13)
curve = survival_curve(sample, time_grid)

for t, s in zip(time_grid, curve):
 bar = "#" * int(s * 40)
 print(f"t={t:5.1f} S(t)={s:.3f} {bar}")
```

Run this and watch the curve fall from 1.0 toward 0.0 as time passes — every project eventually triggers a warning in this synthetic world, so survival decays to zero. This bar chart *is* the shape of a Kaplan–Meier curve, minus the censoring correction Day 31 adds.

## A.6 Break it deliberately

```
A tempting but wrong shortcut: treat the density as if it were a probability.
window = 0.01
hits = np.sum(np.abs(warning_times - 11.87) < window)
density_estimate = (hits / len(warning_times)) / (2 * window)
print(density_estimate) # this number can easily exceed 1.0
```

If this prints something bigger than 1, that's not a bug — a density genuinely can exceed 1 (it's a *rate per unit time*, and if your unit of time is coarse relative to how fast probability accumulates, the rate number can be large). The mistake would be reporting this number as "the probability of triggering a warning around month 11.87." It isn't one. Only density  $\times$  width is a probability. Keep this straight now, because Day 31 makes the identical distinction for hazard, and conflating the two is the single most common survival-analysis mistake beginners make.

## Interlude Day A exit check

You should be able to:

- explain, in one sentence, why the probability of a continuous outcome hitting one exact value is zero;
- explain what a density is, in terms of "probability per unit width," and why it isn't itself a probability;
- build an empirical CDF and survival function from a raw sample using nothing but comparisons and `.mean()`;
- state, without notes, why  $F(t) + S(t) = 1$ .

# Interlude Day B — Rates, Limits, and Integrals

**Today's central idea:** An integral is a sum, the same way  $\sum$  is a sum — just added over infinitely many infinitely thin slices instead of a finite list. Once you see it that way,  $H(t) = \int_0^t h(u) du$  stops being unfamiliar notation and becomes "add up the hazard rate over every instant from 0 to  $t$ ."

## B.1 You've already used a limit — now we name it properly

Back in Day 0E, you approximated a derivative like this:

```
def numerical_slope(f, x, h):
 return (f(x + h) - f(x)) / h
```

and watched the estimate settle down as `h` shrank toward zero. That *is* a limit — you were computing  $\lim_{h \rightarrow 0} \frac{f(x+h)-f(x)}{h}$  without the formal notation. Chapter 5's hazard function is defined the same way, just wrapped around a conditional probability instead of a plain function:

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t}$$

Read it exactly the way you read the derivative: "take the probability of the event landing in a shrinking window just after  $t$ , given that it hasn't happened yet by  $t$ ; divide by the width of that window; watch what happens as the window shrinks to nothing."  $\lim_{\Delta t \rightarrow 0}$  means precisely what `h` shrinking toward `0` meant in your derivative code.

**Code proof — estimate a hazard numerically, the same way you estimated a derivative numerically:**

```
def numerical_hazard(sample, t, window):
 at_risk = sample[sample >= t] # everyone still event-free at t
 if len(at_risk) == 0:
 return np.nan
 events_in_window = np.sum((at_risk >= t) & (at_risk < t + window))
 conditional_prob = events_in_window / len(at_risk)
 return conditional_prob / window
```

```
for window in [5.0, 1.0, 0.1, 0.01]:
 print(f"window={window:<6} hazard estimate at t=10: {numerical_hazard(sample, 10, window):.4f}")
```

As `window` shrinks, this settles toward a stable number — the true instantaneous hazard at  $t = 10$  — exactly the way your derivative estimate settled toward  $2x$  back in Day 0E. Same limiting process, applied to a conditional probability instead of a plain function.

## B.2 The integral: summation notation for infinitely thin slices

Recall  $\sum_{i=1}^n x_i$  from Day 0C — add up a formula over a finite list, indexed by whole numbers. An integral does the same *adding-up*, but over a continuous stretch of a number line instead of a finite list of items, and it's written  $\int$ .

The bridge between the two is a **Riemann sum**: chop the interval into many thin rectangles, add up their areas, and watch the total settle down as the rectangles get thinner.

```
def area_under(func, a, b, n_slices):
 xs = np.linspace(a, b, n_slices, endpoint=False)
 width = (b - a) / n_slices
 return np.sum(func(xs) * width) # a literal sum of thin rectangle areas

def constant_hazard(t, lam=0.03):
 return np.full_like(t, lam)

for n in [10, 100, 10_000, 1_000_000]:
 approx = area_under(constant_hazard, 0, 12, n)
 print(f"n_slices={n:<10} approx integral = {approx:.6f}")
```

For a constant hazard  $\lambda = 0.03$ , the true value of  $\int_0^{12} \lambda dt$  is exactly  $\lambda \times 12 = 0.36$ , and you should see the approximation converge to that as `n_slices` grows. This is the entire idea behind  $\int$ : it's `np.sum(values * width)` in the limit of infinitely many, infinitely thin slices — the continuous sibling of the  $\sum$  you already know.

## B.3 Why cumulative hazard needs an integral, and hazard alone doesn't

Chapter 5 defines cumulative hazard as

$$H(t) = \int_0^t h(u) du$$

which is exactly the pattern in §B.2: add up the instantaneous rate  $h(u)$  over every instant from 0 to  $t$ , in infinitely thin slices. This mirrors something you already did in Chapter 4: an overall risk over a *fixed* window is built by accumulating a rate; here, instead of one fixed window, you're accumulating continuously across the whole timeline up to  $t$ .

## B.4 Closing the loop: hazard, cumulative hazard, and survival

Three quantities, one identity, and you now have the pieces for all of it:

$$H(t) = -\log S(t) \quad \Longleftrightarrow \quad S(t) = \exp[-H(t)]$$

This uses exactly the `exp/log` machinery Chapter 4's logistic regression already put in your hands — **log** turns a product into a sum (useful because survival to time  $t$  is a *product* of surviving each small interval along the way), and **exp** undoes it.

**Code proof** — build a survival curve two independent ways and check they agree:

```

lam = 0.03
time_grid = np.linspace(0, 60, 13)

Route 1: the closed-form constant-hazard survival function
survival_direct = np.exp(-lam * time_grid)

Route 2: integrate the hazard numerically, then apply S(t) = exp(-H(t))
def cumulative_hazard_numeric(t, lam, n_slices=10_000):
 return area_under(lambda u: np.full_like(u, lam), 0, t, n_slices)

H_values = np.array([cumulative_hazard_numeric(t, lam) if t > 0 else 0.0 for t in time_grid])
survival_via_integral = np.exp(-H_values)

for t, s1, s2 in zip(time_grid, survival_direct, survival_via_integral):
 print(f"t={t:5.1f} direct S(t)={s1:.4f} via integral S(t)={s2:.4f}")

```

The two routes should agree to several decimal places. This is the single identity that Chapter 5, Day 31 onward leans on constantly: model the *rate* (hazard), integrate it, and you get back an *absolute probability* (survival) — the same "rate → total" logic as converting a speed into a distance travelled.

## B.5 Build: a hazard-to-survival mini pipeline

```

def hazard_to_survival(hazard_fn, time_grid, n_slices=5_000):
 H = np.array([
 area_under(hazard_fn, 0, t, n_slices) if t > 0 else 0.0
 for t in time_grid
])
 return np.exp(-H)

def rising_hazard(t):
 # A hazard that increases over time – riskier the longer a project runs.
 return 0.01 + 0.002 * t

time_grid = np.linspace(0, 40, 9)
survival = hazard_to_survival(rising_hazard, time_grid)

for t, s in zip(time_grid, survival):
 print(f"t={t:5.1f} S(t)={s:.4f}")

```

Unlike the constant-hazard case, this survival curve falls faster as time goes on, because the risk itself is rising — exactly the kind of hazard shape Chapter 5, Day 33 will teach you to detect and model when the constant-risk assumption fails.

## B.6 Break it deliberately

```

A believable but wrong claim: "the hazard at t=10 is 0.4, so there's a 40% chance
of the event happening in the next month."
window = 12.0 # a whole year, not an instant
hazard_at_10 = numerical_hazard(sample, 10, window)
print(hazard_at_10)
print("naive probability guess:", hazard_at_10 * window) # can exceed 1

```

If `hazard_at_10 * window` comes out above 1, you've just produced an impossible "probability." The bug isn't in the arithmetic — it's in forgetting that  $h(t)\Delta t$  is only a good *approximation* to a probability when  $\Delta t$  is small. Stretch the window out and the approximation breaks, the same way a straight-line approximation to a curve breaks if you extend it too far past the point you built it at.

## Interlude Day B exit check

You should be able to:

- explain a hazard as a limit, in the same terms you used to explain a derivative in Day 0E;
- explain an integral as "a sum over infinitely thin slices," and connect it to  $\sum$  from Day 0C;
- compute a Riemann-sum approximation of an integral in code and watch it converge as slices get finer;
- derive a survival curve from a hazard function two independent ways ( $\exp(-\lambda t)$  directly, and numerically integrating first) and get matching answers;
- explain why  $h(t) \cdot \Delta t$  stops being a good probability approximation once  $\Delta t$  is large.

## Interlude Capstone

Using `rising_hazard` from §B.5:

1. Compute  $S(24)$  two ways — direct numeric integration, and by first tabulating  $H(t)$  at a fine grid and reading off  $\exp(-H(24))$  — and confirm they match.
2. Estimate the density  $f(t)$  at  $t = 20$  by using  $f(t) = h(t) \cdot S(t)$ , then cross-check it against a direct "windowed probability ÷ width" estimate the way you did in §A.3.
3. Deliberately pick a `window` too large in `numerical_hazard` (say, 20 months) and show the "hazard × window" arithmetic producing a value the size of a real probability shouldn't reach. Write one sentence explaining why.
4. In your own words (no formulas), explain to an imaginary colleague the difference between "the probability a project triggers a warning by month 12" (Chapter 4's question) and "the hazard of triggering a warning at month 12" (Chapter 5's question).

If you can do all four without reopening this interlude, Chapter 5, Day 31 will read as a continuation of something you already understand, not a new subject.

## Glossary (new symbols for this interlude)

| Symbol                          | Name                             | Meaning                                                                                                   |
|---------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------|
| $f(t)$                          | density                          | probability per unit time near $t$ ; not itself a probability                                             |
| $F(t)$                          | cumulative distribution function | $P(T \leq t)$                                                                                             |
| $S(t)$                          | survival function                | $P(T > t) = 1 - F(t)$                                                                                     |
| $h(t)$                          | hazard                           | instantaneous event rate among those still event-free at $t$                                              |
| $H(t)$                          | cumulative hazard                | $\int_0^t h(u) du$ ; accumulated rate from 0 to $t$                                                       |
| $\lim_{\Delta t \rightarrow 0}$ | limit                            | what a quantity settles toward as $\Delta t$ shrinks to nothing — same idea as Day 0E's derivative limit  |
| $\int_a^b \dots du$             | integral                         | a sum over infinitely many, infinitely thin slices between $a$ and $b$ — the continuous sibling of $\sum$ |

## Where Chapter 5 continues

Day 30 picks up right where Chapter 4 left off — the censoring, risk-set, and truncation content there is already built from scratch and doesn't need a bridge. Day 31 is where this interlude pays off: §31.1's density, survival, and hazard definitions, and §31.2's  $H(t) = -\log S(t)$  identity, are now the exact ideas you just built by hand, wearing their formal notation for the first time.

# Chapter 5 — From Event Labels to Event Histories

---

## Level 5 Research Apprentice: nine days of survival analysis, dynamic prediction, and registered evidence

**Central promise.** Chapter 4 estimated whether an event would occur inside a fixed window. By the end of Chapter 5, you will be able to preserve the time information that a binary label discards: define a time origin and risk set, distinguish censoring from truncation and competing events, derive Kaplan–Meier and Cox estimators, diagnose proportional hazards, update predictions with newly available information, estimate cumulative incidence, evaluate censored predictions at clinically or operationally meaningful horizons, compare nonlinear survival models honestly, and execute a registered temporal study whose uncertainty and causal limits are explicit.

The learner is still treated as a beginner. Survival analysis has intimidating vocabulary because it was developed across medicine, engineering, demography, economics, and reliability theory. Underneath the vocabulary is a simple observation:

For some subjects, we know that the event had not happened **up to** a time; we do not know that it would never happen.

Ordinary regression discards that partial information. Ordinary classification often mislabels it. Time-to-event methods use it.

The research-apprentice mindset is not:

“Run a Cox model and report hazard ratios.”

It is:

- define the event, clock, eligibility rule, and estimand before opening the data;
  - preserve who was genuinely at risk at every event time;
  - explain why follow-up ended and whether that mechanism is ignorable;
  - separate a cause-specific rate from an absolute event probability;
  - check model assumptions rather than treating them as software defaults;
  - evaluate predictions at decision-relevant horizons with censoring-aware metrics;
  - update risk only with information available at the update time;
  - validate across calendar time, geography, and workflow;
  - distinguish prediction from intervention effects; and
  - register the analysis before the locked data are inspected.
-

## Why the supplied draft needed a new role

The supplied Chapter 5 draft contained valuable research themes: parameter uncertainty, robust inference, causal diagrams, multilevel structure, Bayesian reasoning, and conformal prediction. Each deserves a full treatment. But placing them here as a short catalogue would break the book's progression. Chapter 4 ended with an explicit unresolved problem: projects have unequal follow-up, some have not yet failed, and cancellation can prevent the event of interest.

This revision therefore makes time-to-event analysis the chapter's spine. It retains the draft's intent in a more disciplined form:

1. **Uncertainty** appears in Greenwood intervals, coefficient covariance, bootstrap optimism, and prediction intervals.
2. **Causality** appears where prediction most often overreaches: a hazard association is not the effect of an intervention.
3. **Hierarchical structure** appears in clustered projects, shared frailty, and transport across districts.
4. **Bayesian reasoning** appears as an extension for regularisation and full predictive uncertainty, not as a one-paragraph slogan.
5. **Distribution-free prediction** is discussed with the additional assumptions censoring creates; ordinary split conformal cannot simply be copied onto unobserved event times.

The result is narrower in topic and deeper in competence. That is what a research apprenticeship requires.

## Mastery boundary: build versus read and recognise

This is deliberately a large chapter, so it has two depths. Sections marked by executable builds and exit-check requirements are **build it yourself** material: event contracts, risk sets, Kaplan–Meier/Nelson–Aalen, the educational Cox fit, time-varying/landmark data, cumulative incidence, censoring-aware metrics, and the locked study. Advanced extensions—Fine–Gray modelling, full multi-state transition systems, frailty/Bayesian survival models, and specialised forest software—are **read and recognise** on a first pass. For those extensions, mastery means identifying the estimand, inputs, assumptions, and failure modes, not reproducing production software from scratch. Any exercise asking you to implement one is an optional stretch task unless a worked template is supplied.

Do not interpret “read and recognise” as unimportant. It is an honest boundary between concepts this chapter proves computationally and methods it equips you to discuss, choose, and study next.

## Prerequisite checkpoint

Before starting, retrieve these ideas from Chapters 2–4 without notes:

- random variable, conditional probability, density, expectation, and variance;
- likelihood, log likelihood, score, Hessian, and numerical optimisation;
- a binary event probability and a proper scoring rule;
- train, validation, locked test, temporal splitting, and leakage;
- regression coefficient versus causal effect;
- one-hot encoding, scaling, regularisation, interactions, and splines;
- bootstrap uncertainty and subgroup evaluation;
- calibration versus discrimination versus decision utility; and
- the difference between an outcome, model, metric, threshold, and claim.

If a term is recognisable but not explainable in your own words, revisit the relevant exit check. This chapter composes those ideas rather than replacing them.



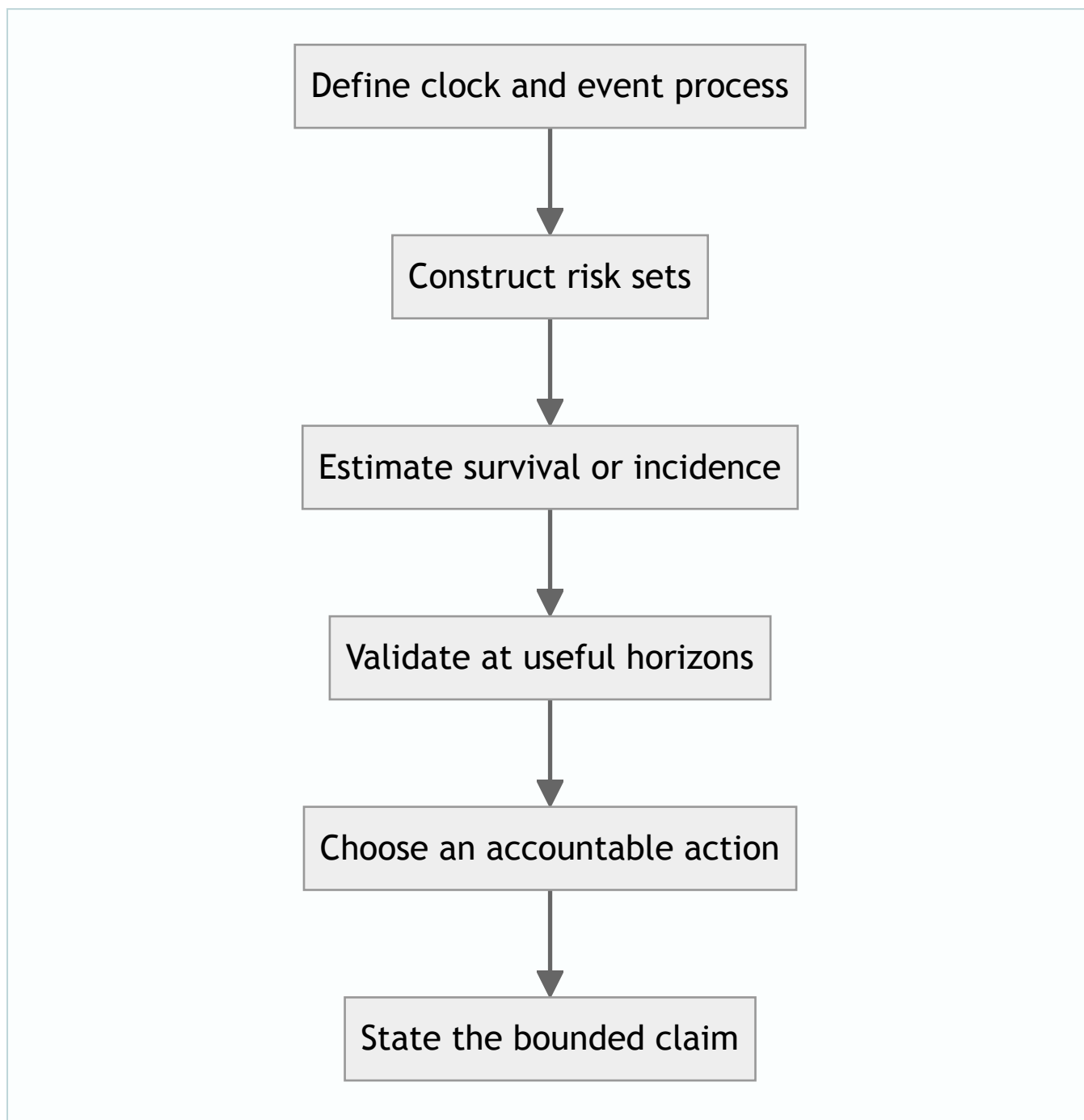
## Learning outcomes

At the end of Chapter 5, you should be able to:

- write a time-to-event protocol specifying unit, origin, entry, event, competing events, horizon, and end of follow-up;
- distinguish right censoring, left censoring, interval censoring, and left truncation;
- state a conditional independent-censoring assumption and explain why it cannot be proven from observed data alone;
- construct a risk set by hand;
- move among density, survival, hazard, and cumulative hazard functions;
- derive the Kaplan–Meier product-limit estimator and Greenwood standard error;
- calculate a Nelson–Aalen cumulative hazard and a two-sample log-rank statistic;
- derive the Cox partial likelihood, score, and observed information;
- implement a penalised Cox model with Breslow handling of tied times;
- interpret a hazard ratio without translating it into a probability ratio or causal effect;
- estimate a baseline cumulative hazard and subject-specific survival curve;
- diagnose non-proportional hazards with plots, interactions with time, and Schoenfeld residuals;
- distinguish proportional-hazards and accelerated-failure-time interpretations;
- represent time-varying covariates in start–stop form without using future values;
- build a landmark prediction dataset and explain immortal-time bias;
- distinguish a cause-specific hazard, subdistribution hazard, cumulative incidence function, and multi-state transition probability;
- explain why censoring a competing event in Kaplan–Meier overstates its real-world cumulative incidence;
- compare survival trees, random survival forests, and boosting with regularised Cox models;
- calculate Harrell concordance and describe its censoring limitation;
- calculate an inverse-probability-of-censoring-weighted Brier score at a fixed horizon;
- construct a censoring-aware calibration table;
- separate internal, temporal, geographic, and prospective validation;
- choose a dynamic action threshold from horizon-specific consequences and capacity;
- write a registration for a censored-outcome prediction study; and
- explain why “does senior review prevent overruns?” requires a causal design rather than a prognostic Cox model.

## The nine-day route

| Day    | Central idea                           | Problem resolved                                                   |
|--------|----------------------------------------|--------------------------------------------------------------------|
| Day 30 | Event history and observation          | A zero may mean “not yet observed,” not “never”                    |
| Day 31 | Risk sets and nonparametric estimation | Unequal follow-up can still contribute evidence                    |
| Day 32 | Relative hazards                       | Covariates can alter event rates without specifying baseline shape |
| Day 33 | Model criticism                        | A hazard ratio need not remain constant over time                  |
| Day 34 | Updated information                    | Baseline prediction becomes stale during follow-up                 |
| Day 35 | Multiple event types                   | Another event can prevent the event of interest                    |
| Day 36 | Flexible structure                     | Linear log hazards may omit interactions and group dependence      |
| Day 37 | Prediction quality                     | Ordinary classification metrics mishandle unknown outcomes         |
| Day 38 | Research discipline                    | Adaptive analysis and causal overclaiming can invalidate a study   |



## Running case: when will a project first trigger an overrun warning?

We continue the fictional microhydro power (MHP) programme. Chapter 4 predicted whether final constant-price cost would exceed 115% of the appraisal budget. That required completed final accounts. Chapter 5 asks a different operational question at technical appraisal:

What is the probability that a project will first trigger a formally documented major-overrun warning by 12, 24, or 36 months?

For this chapter:

- one unit is one eligible project;
- time zero is approval at technical appraisal;
- the event of interest is the first formal forecast that completion cost will exceed 115% of the approved appraisal budget;
- project cancellation is a competing event because a cancelled project cannot later trigger the warning under this definition;
- administrative database closure and loss to follow-up are censoring events;
- baseline predictors must be available at appraisal;
- a 12-month progress measure may update a prediction only among projects still under observation and event-free at month 12; and
- a high predicted incidence may trigger senior review, not automatic cancellation.

The estimand must name both event type and horizon. “Risk” alone is incomplete.

## Event-history data extension

Save the Chapter 4 generator as `chapter4_data.py`, then save this extension as `chapter5_data.py`.

```
import numpy as np
import pandas as pd

from chapter4_data import make_mhp_overrun_data

def make_mhp_survival_data(n=1200, seed=5050):
 """Create a fictional censored, competing-risks project cohort.

 Times are months from appraisal. The relationships are educational and
 must not be interpreted as evidence about real projects or districts.
 """
 df = make_mhp_overrun_data(n=n, seed=seed)
 rng = np.random.default_rng(seed + 1)

 def z(series):
 values = np.asarray(series, dtype=float)
 return (values - values.mean()) / values.std(ddof=0)

 remote = (df["access_mode"] == "porter_or_air").astype(float).to_numpy()
 mixed = (df["access_mode"] == "mixed").astype(float).to_numpy()

 # Cause 1: first formal major-overrun warning. A Weibull proportional-
 # hazards mechanism makes the simulated Cox laboratory interpretable.
 overrun_lp = (
 0.45 * z(df["terrain_index"])
 + 0.50 * z(df["estimated_cable_km"])
 + 0.38 * z(df["appraisal_budget_2025_million_pkr"])
 - 0.32 * z(np.log1p(df["contractor_experience_projects"]))
 + 0.42 * remote
 + 0.15 * mixed
)
 overrun_shape = 1.35
 overrun_scale = 58.0
 u_overrun = rng.uniform(size=n)
 overrun_time = overrun_scale * (
 -np.log(u_overrun) / np.exp(overrun_lp)
) ** (1.0 / overrun_shape)

 # Cause 2: cancellation. It has a different predictor mechanism.
 cancel_lp = (
 0.40 * z(df["road_distance_km"])
 + 0.28 * z(df["terrain_index"])
 - 0.22 * z(df["planned_capacity_kw"])
 + 0.25 * remote
)
```

```

u_cancel = rng.uniform(size=n)
cancel_time = 115.0 * (-np.log(u_cancel)) / np.exp(cancel_lp)

Database closure is administrative. Later cohorts have less potential
follow-up. A separate loss-to-follow-up time adds random censoring.
approval_month = rng.integers(0, 12, size=n)
administrative_time = 12.0 * (2030 - df["start_year"].to_numpy()) - approval_month
loss_time = rng.exponential(scale=220.0, size=n)
censor_time = np.minimum(administrative_time, loss_time)

all_times = np.column_stack([overrun_time, cancel_time, censor_time])
first_type = np.argmin(all_times, axis=1) # 0 overrun, 1 cancel, 2 censor
observed_time = all_times[np.arange(n), first_type]
np.select is vectorised if/elif/else: choose 1 for overrun, 2 for
cancellation, and default to 0 for censoring, row by row.
event_type = np.select(
 [first_type == 0, first_type == 1],
 [1, 2],
 default=0,
).astype(int)

df["follow_up_months"] = observed_time
df["event_type"] = event_type
df["overrun_warning_event"] = (event_type == 1).astype(int)
df["cancellation_event"] = (event_type == 2).astype(int)
df["censor_reason"] = np.where(
 event_type != 0,
 "event_observed",
 np.where(administrative_time <= loss_time, "database_close", "lost_follow_up"),
)

Some legacy registries enrolled projects after appraisal. This field is
for the left-truncation laboratory; the primary inception cohort starts
every project at appraisal and therefore uses entry time zero.
registry_entry = rng.uniform(0.0, 10.0, size=n)
df["legacy_registry_entry_months"] = registry_entry
df["legacy_registry_included"] = (observed_time > registry_entry).astype(int)

This is future information for a baseline model. It becomes legitimate
only in a month-12 landmark analysis among projects still at risk then.
progress_gap = (
 0.12 * df["terrain_index"].to_numpy()
 + 0.06 * df["road_distance_km"].to_numpy()
 + 0.85 * remote
 + rng.normal(0.0, 0.9, size=n)
)
df["month12_progress_gap_sd"] = np.where(
 observed_time > 12.0,
 progress_gap,
 np.nan,
)

An observational intervention field is deliberately present so the
final causal laboratory can expose confounding by indication.
review_score = overrun_lp + rng.normal(0.0, 0.7, size=n)
df["senior_review_by_month6"] = (
 (review_score > np.quantile(review_score, 0.65))
 & (observed_time > 6.0)
).astype(int)

return df.sort_values(["start_year", "project_id"]).reset_index(drop=True)

if __name__ == "__main__":
 projects = make_mhp_survival_data()
 projects.to_csv("mhp_event_history_chapter5.csv", index=False)
 print(projects["event_type"].value_counts().sort_index())
 print(projects.groupby("start_year")["follow_up_months"].median())

```

This generator exposes latent times only inside the function. The returned dataset contains the observable record: a follow-up time and an event type. Real event-time construction would require dated warning records, a stable warning definition, adjudication of back-dated entries, an audit of cancellation reasons, and a verified database-close date.

## Minimal software setup

The core chapter and locked benchmark use only the existing scientific Python stack:

```
python -m pip install numpy pandas scipy matplotlib scikit-learn
```

Optional laboratories can use specialised survival libraries:

```
python -m pip install lifelines scikit-survival
```

The executable capstone does not require the optional line. That is deliberate: implementing a small Cox model and censoring-aware metrics makes the estimators inspectable. For applied research, use a tested survival package, record its version, and verify its tie handling, variance convention, and prediction target.

# Day 30 — The Event-History Contract

**Today's central idea:** A survival dataset records a partially observed process. Before fitting a model, specify what could happen, when observation begins, and why observation ends.

## 30.1 The hidden pair of times

Let  $T_i$  be project  $i$ 's event time and  $C_i$  its censoring time. We usually observe neither pair completely. We observe

$$Y_i = \min(T_i, C_i)$$

and

$$\Delta_i = \mathbb{1}(T_i \leq C_i).$$

If  $\Delta_i = 1$ , the event occurred at  $Y_i = T_i$ . If  $\Delta_i = 0$ , we know only that  $T_i > Y_i = C_i$ . A censored observation is therefore an inequality, not a guessed event time.

Suppose project A triggered a warning at 18 months, while project B had no warning when the database closed at 18 months:

| Project | Observed time | Event indicator | What is known? |
|---------|---------------|-----------------|----------------|
| A       | 18            | 1               | $T_A = 18$     |
| B       | 18            | 0               | $T_B > 18$     |

Coding B as “no event” in an unrestricted binary analysis silently changes  $T_B > 18$  into  $T_B = \infty$ . Coding B's event time as 18 invents an event. Dropping B discards 18 event-free months. All three are wrong for the stated process.

## 30.2 The seven-part protocol

Write these fields before examining outcome associations.

| Protocol field  | Running-case definition             | Failure if omitted                                      |
|-----------------|-------------------------------------|---------------------------------------------------------|
| Unit            | one eligible approved project       | repeated rows may be treated as independent projects    |
| Time origin     | technical-appraisal approval        | durations do not share a scientific zero                |
| Entry           | approval for the inception cohort   | survivors selected into a later registry are mishandled |
| Event           | first formal $> 115\%$ cost warning | informal concern and formal warning are mixed           |
| Competing event | cancellation                        | impossible future warnings remain in the risk set       |
| Censoring       | database close or loss to follow-up | end of observation is confused with outcome             |
| Horizons        | 12, 24, and 36 months               | a model is judged at arbitrary or unsupported times     |

Calendar time and analysis time are different. Two projects can both be at month 12 of follow-up in different calendar years. Calendar change may still matter as a predictor or transport concern.

## 30.3 Four incomplete-observation mechanisms

**Right censoring** means the event occurs, if at all, after the last observed time. Database closure is the common example.

**Left censoring** means the event occurred before a known time but its exact time is unknown. A project first inspected at month 8 may already have crossed the warning threshold.

**Interval censoring** means the event occurred between two inspections. If the month-6 audit was clear and the month-9 audit found a warning, then  $6 < T \leq 9$ . Replacing this by month 9 or the midpoint is an approximation, not the observed truth.

**Left truncation**, also called delayed entry, is a sampling mechanism. A project enters a registry at month 8 only if it has remained observable and event-free until month 8. Projects failing earlier never appear. The correct risk set includes that project only after month 8.

Left censoring and left truncation are not synonyms. Censoring hides a time for an observed unit; truncation can remove a unit from the sample entirely.

## 30.4 Risk sets

At an event time  $t$ , the risk set is

$$R(t) = \{i : L_i \leq t \leq Y_i\},$$

where  $L_i$  is entry time. Under an inception cohort,  $L_i = 0$  for everyone. Under delayed entry, a subject contributes only after entering.

Consider four projects:

| Project | Entry | Exit | Event? |
|---------|-------|------|--------|
| A       | 0     | 3    | 1      |
| B       | 0     | 5    | 0      |
| C       | 2     | 6    | 1      |
| D       | 4     | 7    | 0      |

Immediately before the event at month 3,  $R(3) = A, B, C$ . D has not entered. Immediately before month 6, only C and D remain. A failed; B was censored; C and D are under observation.

```
import pandas as pd

toy = pd.DataFrame({
 "project": ["A", "B", "C", "D"],
 "entry": [0, 0, 2, 4],
 "exit": [3, 5, 6, 7],
 "event": [1, 0, 1, 0],
})

def risk_set(frame, time):
 mask = (frame["entry"] <= time) & (frame["exit"] >= time)
 return frame.loc[mask, "project"].tolist()

assert risk_set(toy, 3) == ["A", "B", "C"]
assert risk_set(toy, 6) == ["C", "D"]
```

## 30.5 Independent censoring is an assumption

A common working condition is

**$T$  is independent of  $C$  given  $X$ ,**

meaning that, conditional on recorded covariates  $X$ , the censoring time carries no additional information about the event time. This does **not** require every project to have the same censoring distribution. Administrative follow-up can differ by approval year if calendar year is represented appropriately.

The assumption fails when struggling projects disappear from the reporting system for unrecorded reasons related to imminent overruns. No plot can prove the absence of that hidden mechanism. Researchers should:

- document every reason follow-up ended;
- compare censoring across measured groups and periods;
- model censoring conditional on relevant variables when needed;
- use sensitivity analyses for plausible informative censoring; and
- avoid the phrase “non-informative censoring” as an unexamined ritual.

Independent censoring is relative to an estimand and information set. A mechanism may be independent conditional on district and access mode but not marginally.

## 30.6 Censoring is not a competing risk

A censoring event ends observation; conceptually, the event could still occur later. A competing event prevents or fundamentally changes the event of interest. Database closure is censoring. Cancellation is competing because, under the chapter's event definition, a cancelled project cannot later trigger a construction-stage overrun warning.

The distinction is scientific, not stored in a status code. If cancelled projects remain under a revised programme and can still trigger the same warning, cancellation may be a state transition rather than an absorbing competing event.

## 30.7 Data audit before estimation

```
import numpy as np
from chapter5_data import make_mhp_survival_data

projects = make_mhp_survival_data()

assert projects["project_id"].is_unique
assert projects["follow_up_months"].gt(0).all()
assert projects["event_type"].isin([0, 1, 2]).all()
assert np.array_equal(
 projects["overrun_warning_event"].to_numpy(),
 (projects["event_type"] == 1).astype(int).to_numpy(),
)

audit = (
 projects.groupby(["start_year", "event_type"])
 .size()
 .unstack(fill_value=0)
)
print(audit)
print(projects.groupby("censor_reason")["follow_up_months"].agg(["size", "median"]))
```

Also audit impossible date order, duplicate units, negative duration, events after recorded database closure, changes in event coding, and whether follow-up opportunity differs sharply across the proposed train/test split.

## 30.8 Build, break, and reflect

### Build

1. Write the seven-part protocol for employee attrition, machine failure, or loan default.
2. Create five toy subjects with entry, exit, and status.
3. List the risk set at each event time by hand and in code.
4. Explain every censored row as an inequality.

### Break

1. Recode all censored projects as event-free forever.
2. Give every legacy-registry subject entry time zero.
3. Treat cancellation as ordinary censoring without stating the resulting estimand.
4. Add `month12_progress_gap_sd` to an appraisal-time model.

For each action, name the information or population it corrupts.

### Reflect

Why can a study with 1,000 rows contain less information about 36-month risk than a study with 400 rows? Your answer must mention event counts, censoring, follow-up distribution, and effective risk sets.



## Day 30 exit check

You are ready for Day 31 when you can explain:

1. why a censored row is not a negative label;
2. why left truncation changes membership in a risk set;
3. why independent censoring is conditional and untestable in full; and
4. why a competing event is part of the outcome process rather than missingness.

# Day 31 — Survival, Hazard, and Kaplan–Meier

**Today's central idea:** Survival and hazard describe the same event-time distribution from different angles; Kaplan–Meier estimates survival by multiplying conditional survival across observed risk sets.

## 31.1 Distribution, density, survival, and hazard

For a non-negative continuous event time  $T$ , the cumulative distribution function is

$$F(t) = P(T \leq t).$$

The survival function is

$$S(t) = P(T > t) = 1 - F(t).$$

If a density exists,

$$f(t) = \frac{dF(t)}{dt} = -\frac{dS(t)}{dt}.$$

The hazard is the instantaneous event rate among those still event-free:

$$h(t) = \lim_{\Delta t \downarrow 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t} = \frac{f(t)}{S(t)}.$$

A hazard is not a probability. It is a rate per unit time and can exceed 1. The approximate probability of an event in a short interval of width  $\Delta t$ , conditional on survival to its start, is  $h(t)\Delta t$ .

## 31.2 Cumulative hazard links rate and probability

Define

$$H(t) = \int_0^t h(u) du.$$

Because  $h(t) = -S'(t)/S(t)$ ,

$$H(t) = -\log S(t),$$

and therefore

$$S(t) = \exp[-H(t)].$$

This identity is central. Regression models can describe how covariates multiply a hazard, then integrate the hazard to recover an absolute survival probability.

### Constant-hazard example

If  $h(t) = \lambda$ , then  $H(t) = \lambda t$  and

$$S(t) = e^{-\lambda t}.$$

For  $\lambda = 0.03$  per month, survival to month 12 is  $e^{-0.36} \approx 0.698$ . The event probability by month 12 is about 0.302. The median solves  $S(t) = 0.5$ , giving  $t_{0.5} = \log(2)/\lambda \approx 23.1$  months.

Constant hazard is convenient, not universal. Construction risk may rise as procurement begins and later fall after difficult tasks are completed.

## 31.3 Deriving the product-limit estimator

Let  $t_1 < \dots < t_m$  be distinct event times. At  $t_j$ :

- $n_j$  subjects are at risk immediately before the time; and
- $d_j$  events occur.

Conditional on reaching  $t_j$ , the observed fraction surviving that event time is

$$1 - \frac{d_j}{n_j} = \frac{n_j - d_j}{n_j}.$$

Survival through  $t$  is the product of those conditional survivals:

$$\hat{S}(t) = \prod_{t_j \leq t} \left(1 - \frac{d_j}{n_j}\right).$$

This is the Kaplan–Meier product-limit estimator. Censoring changes later risk-set sizes but does not cause a downward step. An event causes the step.

## 31.4 Hand calculation

Suppose the observed records are:

| Time | Status   |
|------|----------|
| 2    | event    |
| 3    | censored |
| 5    | event    |
| 5    | event    |
| 7    | censored |

At month 2,  $n_1 = 5, d_1 = 1$ , so survival becomes  $4/5 = 0.8$ . One project is censored at month 3, leaving three at risk just before month 5. At month 5,  $n_2 = 3, d_2 = 2$ , so

$$\hat{S}(5) = \frac{4}{5} \times \frac{1}{3} = \frac{4}{15} \approx 0.267.$$

The order of an event and censoring recorded at exactly the same nominal time requires a convention justified by timestamp precision. Do not let row order decide it accidentally.

## 31.5 Kaplan–Meier from scratch

```
import numpy as np
import pandas as pd

def kaplan_meier(time, event, entry=None):
 """Product-limit estimate with optional delayed entry."""
 time = np.asarray(time, dtype=float)
 event = np.asarray(event, dtype=int)
 entry = np.zeros_like(time) if entry is None else np.asarray(entry, dtype=float)

 if not (len(time) == len(event) == len(entry)):
 raise ValueError("time, event, and entry must have equal length")
 if np.any(entry > time):
 raise ValueError("entry cannot follow exit")

 survival = 1.0
 greenwood_sum = 0.0
 rows = []

 for t in np.sort(np.unique(time[event == 1])):
 at_risk = np.sum((entry <= t) & (time >= t))
 events = np.sum((time == t) & (event == 1))
 survival *= 1.0 - events / at_risk

 if at_risk > events:
 greenwood_sum += events / (at_risk * (at_risk - events))
 se = survival * np.sqrt(greenwood_sum)
 else:
 se = np.nan

 rows.append({
 "time": t,
 "at_risk": int(at_risk),
 "events": int(events),
 "survival": survival,
 "greenwood_se": se,
 })
 return pd.DataFrame(rows)

toy_time = np.array([2, 3, 5, 5, 7], dtype=float)
toy_event = np.array([1, 0, 1, 1, 0], dtype=int)
km = kaplan_meier(toy_time, toy_event)
print(km)
assert np.isclose(km.iloc[-1]["survival"], 4 / 15)
```

The ordinary standard-error approximation can produce confidence limits outside  $[0, 1]$ . Applied packages often transform survival—for example with a complementary log-log transform—construct an interval on that scale, and transform back.

## 31.6 Greenwood variance

Greenwood's approximation is

$$\widehat{\text{Var}}\{\widehat{S}(t)\} \approx \widehat{S}(t)^2 \sum_{t_j \leq t} \frac{d_j}{n_j(n_j - d_j)}.$$

It quantifies sampling uncertainty under the estimator's assumptions. It does not account for event-definition error, informative censoring, unmeasured clustering, or model selection.

Wide late intervals are not a plotting defect. They reveal that few subjects remain at risk. Always display numbers at risk below a survival plot.

## 31.7 Nelson–Aalen cumulative hazard

The Nelson–Aalen estimator is

$$\widehat{H}(t) = \sum_{t_j \leq t} \frac{d_j}{n_j}.$$

Then  $\exp[-\widehat{H}(t)]$  approximates survival. It is not algebraically identical to Kaplan–Meier because

$$-\log(1 - d_j/n_j) \neq d_j/n_j$$

except approximately for small event fractions. The cumulative-hazard scale is useful for diagnostics and forms the basis of baseline-hazard estimation after Cox regression.

## 31.8 Comparing groups with the log-rank test

At each pooled event time  $t_j$ , suppose group 1 has  $n_{1j}$  of the  $n_j$  at-risk subjects. Under equal hazards, its expected event count is

$$e_{1j} = d_j \frac{n_{1j}}{n_j}.$$

Summing observed minus expected counts yields

$$U = \sum_j (d_{1j} - e_{1j}).$$

Why “hypergeometric”? Conditional on  $d_j$  pooled events occurring at time  $t_j$ , the null model treats those events like  $d_j$  draws without replacement from a risk set containing  $n_{1j}$  group-1 members among  $n_j$  total members. Therefore

$$V_j = d_j \frac{n_{1j}}{n_j} \left(1 - \frac{n_{1j}}{n_j}\right) \frac{n_j - d_j}{n_j - 1},$$

where the final factor is the finite-population correction. Add the event-time variances,  $V = \sum_j V_j$ . Then  $U^2/V$  is compared with a  $\chi_1^2$  reference distribution. This derivation assumes the usual pooled risk-set conditioning; tie conventions and weighting variants can change details.

The log-rank test is most sensitive to a roughly proportional separation across time. Crossing survival curves can produce a small statistic even when groups differ meaningfully at particular horizons. A test of equality also does not measure effect size or establish causality.

## 31.9 Research paper study: Kaplan and Meier (1958)

Read Kaplan and Meier's "Nonparametric Estimation from Incomplete Observations". The paper's durable contribution is not merely a staircase plot. It constructs a nonparametric maximum-likelihood estimator from incomplete lifetimes, develops variance reasoning, and confronts tied observations and difficult boundary cases.

Read with four questions:

1. What information does a censored observation contribute?
2. Why is the estimator a product of conditional terms?
3. Which uncertainty arguments are exact and which are approximations?
4. Which data complications discussed in the paper are hidden by a modern one-line function call?

**Replication task.** Reproduce a small table from the paper or construct an equivalent five-subject example. Calculate every risk set and product term, then compare your result with `kaplan_meier`. Change one event to censoring and explain every altered step.

## 31.10 Build, break, and reflect

### Build

1. Estimate overrun-warning-free survival while treating cancellation as censoring; label this a **cause-specific** descriptive curve.
2. Plot the curve with numbers at risk at 0, 12, 24, 36, and 48 months.
3. Calculate the Nelson–Aalen estimate at the same horizons.
4. Compare access modes with a log-rank test.

### Break

1. Remove every censored row before estimation.
2. Count censoring as an event.
3. Keep delayed entrants in risk sets before their entry.
4. Report  $1 - \hat{S}(t)$  as real-world overrun incidence while ignoring cancellation.

The fourth error is subtle. Day 35 will repair it.

### Reflect

Two districts have identical Kaplan–Meier curves through month 24, but one has only five projects still at risk and the other has 80. What is identical, and what is not?

## Day 31 exit check

You are ready for Day 32 when you can:

1. derive  $S(t) = \exp[-H(t)]$ ;
  2. compute a Kaplan–Meier step from  $n_j$  and  $d_j$ ;
  3. explain why censoring changes risk sets but not survival directly; and
  4. state why a log-rank  $p$ -value is neither an effect size nor a causal conclusion.
-

# Day 32 — Cox Regression from Partial Likelihood

**Today's central idea:** Cox regression compares covariate values inside each event-time risk set. It estimates relative hazards without requiring a parametric formula for the baseline hazard.

## 32.1 The proportional-hazards model

For covariate vector  $\mathbf{x}_i$ , the Cox model specifies

$$h(t | \mathbf{x}_i) = h_0(t) \exp(\mathbf{x}_i^\top \boldsymbol{\beta}).$$

The model has two components:

- $h_0(t)$  is an unspecified baseline hazard shared by subjects; and
- $\exp(\mathbf{x}_i^\top \boldsymbol{\beta})$  multiplies that hazard according to covariates.

Take logs:

$$\log h(t | \mathbf{x}_i) = \log h_0(t) + \mathbf{x}_i^\top \boldsymbol{\beta}.$$

The baseline can change freely over time. The covariate score is linear and time-constant unless the model is extended.

For subjects  $\mathbf{a}$  and  $\mathbf{b}$ ,

$$\frac{h(t | \mathbf{x}_a)}{h(t | \mathbf{x}_b)} = \exp\{(\mathbf{x}_a - \mathbf{x}_b)^\top \boldsymbol{\beta}\}.$$

The baseline cancels, so the hazard ratio does not depend on  $t$ . This is the proportional-hazards assumption.

## 32.2 What a hazard ratio does not say

If a one-unit increase in terrain index has coefficient  $\hat{\beta} = 0.30$ , its fitted hazard ratio is

$$e^{0.30} \approx 1.35.$$

Conditional on the model covariates, the instantaneous warning rate among projects still at risk is estimated to be 35% higher. It does **not** mean:

- event probability is 35 percentage points higher;
- median event time is 35% shorter;
- 35% more projects will eventually experience the event;
- terrain causes the event; or
- every project's risk is multiplied by 1.35 at every horizon when competing events differ.

Hazard conditions on survival to  $t$ . The members of that surviving group can change over time. Hazard ratios are therefore less intuitive than probability contrasts and should usually be accompanied by absolute survival or cumulative-incidence predictions.

## 32.3 Deriving one partial-likelihood term

Suppose exactly one event occurs at time  $t_j$ , and subject  $i(j)$  experiences it. Conditional on one member of risk set  $R(t_j)$  failing at that time, the Cox model assigns subject  $i(j)$  probability

$$\frac{h_0(t_j) \exp(x_{i(j)}^\top \beta)}{\sum_{k \in R(t_j)} h_0(t_j) \exp(x_k^\top \beta)} = \frac{\exp(x_{i(j)}^\top \beta)}{\sum_{k \in R(t_j)} \exp(x_k^\top \beta)}.$$

The unknown baseline hazard cancels. Multiplying over event times gives the partial likelihood

$$L_p(\beta) = \prod_{j=1}^m \frac{\exp(x_{i(j)}^\top \beta)}{\sum_{k \in R(t_j)} \exp(x_k^\top \beta)}.$$

It is called *partial* because it uses the conditional identity of the failing subject to estimate  $\beta$  without simultaneously parameterising  $h_0(t)$ .

## 32.4 Log partial likelihood and score

Let  $\eta_i = x_i^\top \beta$ . With one event at each distinct time,

$$\ell_p(\beta) = \sum_{j=1}^m \left[ x_{i(j)}^\top \beta - \log \sum_{k \in R(t_j)} e^{x_k^\top \beta} \right].$$

Define the risk-set weighted mean

$$\bar{x}(\beta, t) = \frac{\sum_{k \in R(t)} x_k e^{x_k^\top \beta}}{\sum_{k \in R(t)} e^{x_k^\top \beta}}.$$

Differentiate:

$$U(\beta) = \frac{\partial \ell_p}{\partial \beta} = \sum_{j=1}^m [x_{i(j)} - \bar{x}(\beta, t_j)].$$

At the optimum, observed covariates of event subjects balance their model-weighted risk-set expectations. This “observed minus expected” structure will reappear in Schoenfeld residuals.

The negative Hessian is a sum of weighted risk-set covariance matrices. It supplies curvature for optimisation and, under regularity assumptions, an approximate covariance matrix for  $\hat{\beta}$ .

## 32.5 Tied event times

Real datasets often record time in days or months, so several events share a timestamp. The simple derivation assumed one event per time. Common approaches are:

- **exact partial likelihood:** averages over possible event orderings; principled but potentially expensive;
- **Efron approximation:** progressively removes fractions of the tied event score; often accurate for moderate ties; and
- **Breslow approximation:** treats the tied group against the same full risk denominator; simpler, but can be less accurate with many ties.

Software defaults differ. Record the method. The teaching implementation below uses Breslow ties and says so explicitly.

## 32.6 A penalised Cox model from scratch

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import logsumexp

class CoxPHFromScratch:
 """Right-censored Cox model using Breslow ties and optional L2 penalty."""

 def __init__(self, l2=0.0, max_iter=500):
 self.l2 = float(l2)
 self.max_iter = int(max_iter)

 def _objective(self, beta, X, time, event):
 eta = X @ beta
 log_likelihood = 0.0
 score = np.zeros_like(beta)

 for t in np.sort(np.unique(time[event == 1])):
 deaths = (time == t) & (event == 1)
 risk = time >= t
 d = int(deaths.sum())

 log_likelihood += eta[deaths].sum() - d * logsumexp(eta[risk])
 weights = np.exp(eta[risk] - logsumexp(eta[risk]))
 weighted_mean = weights @ X[risk]
 score += X[deaths].sum(axis=0) - d * weighted_mean

 penalised = -log_likelihood + 0.5 * self.l2 * np.dot(beta, beta)
 gradient = -score + self.l2 * beta
 return penalised, gradient

 def fit(self, X, time, event):
 X = np.asarray(X, dtype=float)
 time = np.asarray(time, dtype=float)
 event = np.asarray(event, dtype=int)
 if X.ndim != 2 or len(time) != X.shape[0]:
 raise ValueError("X must be 2D and aligned with time")
 if event.sum() == 0:
 raise ValueError("at least one event is required")

 # L-BFGS-B is a quasi-Newton optimiser: it uses the supplied gradient
 # and a compact history of recent steps to approximate inverse
 # curvature, avoiding storage of a full p-by-p Hessian. "B" means it
 # can also enforce bounds, although this fit supplies none.
 result = minimize(
 fun=lambda b: self._objective(b, X, time, event),
 x0=np.zeros(X.shape[1]),
 jac=True,
 method="L-BFGS-B",
 options={"maxiter": self.max_iter},
)
 if not result.success:
 raise RuntimeError(result.message)

 self.coef_ = result.x
 self.optimisation_result_ = result
 self._fit_baseline(X, time, event)
 return self

 def _fit_baseline(self, X, time, event):
 eta = X @ self.coef_
 rows = []
```



```

 cumulative = 0.0
 for t in np.sort(np.unique(time[event == 1])):
 d = np.sum((time == t) & (event == 1))
 denominator = np.exp(eta[time >= t]).sum()
 increment = d / denominator
 cumulative += increment
 rows.append((t, increment, cumulative))
 self.event_times_ = np.array([row[0] for row in rows])
 self.baseline_increments_ = np.array([row[1] for row in rows])
 self.baseline_cumulative_hazard_ = np.array([row[2] for row in rows])

 def predict_log_partial_hazard(self, X):
 return np.asarray(X, dtype=float) @ self.coef_

 def predict_survival(self, X, horizons):
 X = np.asarray(X, dtype=float)
 horizons = np.asarray(horizons, dtype=float)
 # searchsorted finds each horizon's insertion point in sorted event
 # times; side="right" includes an event occurring exactly at horizon.
 positions = np.searchsorted(self.event_times_, horizons, side="right") - 1
 baseline = np.where(
 positions >= 0,
 self.baseline_cumulative_hazard_[np.maximum(positions, 0)],
 0.0,
)
 relative_hazard = np.exp(self.predict_log_partial_hazard(X))
 return np.exp(-np.outer(relative_hazard, baseline))

```

The implementation is educational, not production-ready. It does not include Efron ties, delayed entry, stratification, sampling weights, robust covariance, or sparse matrices. Its purpose is to make risk-set optimisation and baseline recovery visible.

## 32.7 A controlled recovery experiment

```

rng = np.random.default_rng(32)
n = 700
X = rng.normal(size=(n, 2))
beta_true = np.array([0.7, -0.45])

Exponential event times satisfy proportional hazards.
event_time = rng.exponential(scale=1 / (0.035 * np.exp(X @ beta_true)))
censor_time = rng.exponential(scale=45.0, size=n)
time = np.minimum(event_time, censor_time)
event = (event_time <= censor_time).astype(int)

model = CoxPHFromScratch(l2=0.01).fit(X, time, event)
print("true coefficients: ", beta_true)
print("estimated coefficients:", model.coef_)
print("events:", event.sum())

assert np.all(np.sign(model.coef_) == np.sign(beta_true))
assert np.max(np.abs(model.coef_ - beta_true)) < 0.18

```

Repeated samples will not recover the exact generating coefficients. Recovery should be assessed across simulations, censoring levels, sample sizes, tie patterns, and misspecification—not by celebrating one seed.

## 32.8 From relative hazard to absolute survival

After estimating  $\hat{\beta}$ , the Breslow baseline cumulative hazard is

$$\hat{H}_0(t) = \sum_{t_j \leq t} \frac{d_j}{\sum_{k \in R(t_j)} \exp(\mathbf{x}_k^\top \hat{\beta})}.$$

Under the Cox model,

$$\hat{S}(t | \mathbf{x}) = \exp \left[ -\hat{H}_0(t) \exp(\mathbf{x}^\top \hat{\beta}) \right] = \hat{S}_0(t) \exp(\mathbf{x}^\top \hat{\beta}).$$

This step is essential for decision support. A linear predictor ranks relative hazard; it is not itself a probability. Absolute prediction also depends on the estimated baseline and the target population's event process.

## 32.9 Penalisation and effective event information

Ridge-penalised Cox regression maximises

$$\ell_p(\beta) - \frac{\lambda}{2} \sum_{j=1}^p \beta_j^2.$$

Lasso uses  $\lambda \sum_j |\beta_j|$ , and elastic net combines both. As in earlier chapters, scaling and tuning belong inside resampling. The amount of information is driven more by event counts and risk-set comparisons than by row count alone. Hundreds of covariates with 35 events invite instability even if there are thousands of censored rows.

Penalisation controls variance; it does not repair an ill-defined time origin, dependent censoring, a missing competing event, or leakage.

## 32.10 Inference, clustering, and strata

The inverse observed information gives a model-based covariance approximation. To see the sandwich construction, let  $I(\hat{\beta})$  be the observed information—the curvature or “bread”—and let  $\mathbf{u}_g(\hat{\beta})$  be the sum of score contributions for cluster  $g$ . Independent clusters may have arbitrary dependence within them, so the empirical score variance or “meat” is  $\sum_g \mathbf{u}_g \mathbf{u}_g^T$ . The cluster-robust covariance is

$$I(\hat{\beta})^{-1} \left( \sum_{g=1}^G \mathbf{u}_g(\hat{\beta}) \mathbf{u}_g(\hat{\beta})^T \right) I(\hat{\beta})^{-1}.$$

The estimating equation—and therefore  $\hat{\beta}$ —does not change; only its estimated sampling covariance changes. Small numbers of clusters require corrections or design-specific methods because the large- $G$  approximation can be poor. Projects within a district may share procurement rules, inspectors, or shocks. Options answer different questions:

- a **cluster-robust sandwich covariance** changes standard errors while leaving coefficients unchanged;
- a **shared frailty** adds latent group heterogeneity and changes the model;
- **district fixed effects** estimate conditional contrasts for represented districts;
- a **stratified Cox model** gives each district its own baseline hazard while constraining covariate hazard ratios to be common.

Do not choose among them merely by which produces the smallest  $p$ -value. State the sampling unit, dependence mechanism, prediction destination, and estimand.

## 32.11 Research paper study: Cox (1972)

Read Cox's "Regression Models and Life-Tables". The paper proposes a model that leaves the time component largely unspecified while estimating regression effects from ordered failures. It also discusses extensions and model checking; the modern ritual of reporting one hazard-ratio table is much narrower than the paper.

Read with these questions:

1. What is gained by leaving the baseline hazard unspecified?
2. Which conditioning step makes partial likelihood possible?
3. How does Cox distinguish an explanatory analysis from prediction?
4. Which issues are raised in the published discussion that software summaries omit?

**Replication task.** Simulate exponential and Weibull proportional-hazards data with the same  $\beta$ . Fit the scratch model to each. Coefficients should recover the common relative-hazard structure even though baseline shapes differ. Then simulate a coefficient that changes sign after month 20 and show why a single estimate is an average compromise.

## 32.12 Build, break, and reflect

### Build

1. Encode appraisal-time predictors using training-only medians, scales, and category levels.
2. Fit the scratch Cox model for the overrun cause, treating cancellations as censoring for the **cause-specific hazard**.
3. Report coefficient, hazard ratio, and a 36-month survival contrast for two realistic profiles.
4. Repeat with three ridge penalties chosen inside temporal validation.

### Break

1. Add final cost and month-12 progress to the appraisal-time matrix.
2. Standardise on the locked test period.
3. Interpret  $e^{\hat{\beta}}$  as a risk ratio.
4. Estimate absolute survival without a baseline hazard.

### Reflect

Why can two populations share exactly the same Cox coefficients but have different 36-month event probabilities?

## Day 32 exit check

You are ready for Day 33 when you can:

1. derive a partial-likelihood contribution from a risk set;
  2. explain why the baseline cancels during coefficient estimation;
  3. distinguish a linear predictor, hazard ratio, survival probability, and cumulative incidence; and
  4. name the tie method used by an analysis.
-

# Day 33 — Assumptions, Diagnostics, and Alternative Time Scales

**Today's central idea:** Model checking asks where the fitted representation fails—not whether one omnibus test grants permission to stop thinking.

## 33.1 What proportional hazards means

For two profiles, proportional hazards requires

$$\log h(t \mid x_a) - \log h(t \mid x_b) = (x_a - x_b)^\top \beta,$$

a constant difference over analysis time. It does not require the hazard itself to be constant. Both hazards may rise and fall as long as their ratio is fixed.

Violations are plausible. Difficult terrain may matter most during early access works; contractor experience may matter later during commissioning. Compressing those processes into constant coefficients can hide sign changes or declining effects.

## 33.2 Complementary log-log plots

Under proportional hazards,

$$S(t \mid x) = S_0(t)^{\exp(x^\top \beta)}.$$

Taking logs twice gives

$$\log[-\log S(t \mid x)] = \log[-\log S_0(t)] + x^\top \beta.$$

Therefore group curves of  $\log[-\log \hat{S}(t)]$  against  $\log t$  should be roughly parallel if a categorical group follows proportional hazards. This is a rough diagnostic: sparse tails, arbitrary grouping of continuous predictors, and competing events can distort it.

## 33.3 Schoenfeld residuals

For an event subject at  $t_j$ , the Schoenfeld residual is

$$r_j = x_{i(j)} - \bar{x}(\hat{\beta}, t_j).$$

It is the observed event-subject covariate minus its fitted risk-set expectation. Under a correctly specified proportional effect, residuals should not show systematic association with event time. Scaled Schoenfeld residual plots can reveal when and how a coefficient changes.

```
def schoenfeld_residuals(X, time, event, beta):
 X = np.asarray(X, dtype=float)
 time = np.asarray(time, dtype=float)
 event = np.asarray(event, dtype=int)
 eta = X @ np.asarray(beta, dtype=float)
```

```

rows = []

for t in np.sort(np.unique(time[event == 1])):
 deaths = np.flatnonzero((time == t) & (event == 1))
 risk = time >= t
 weights = np.exp(eta[risk] - logsumexp(eta[risk]))
 expected = weights @ X[risk]
 for index in deaths:
 rows.append((t, *list(X[index] - expected)))
return np.asarray(rows)

```

A small global test does not prove proportionality. A large study can flag negligible departures; a small study can miss important ones. Inspect effect plots, uncertainty, and decision consequences.

## 33.4 Model a time-varying coefficient

One extension is

$$h(t | x) = h_0(t) \exp\{x\beta + xg(t)\gamma\}.$$

If  $g(t) = \log t$ , the log hazard ratio changes linearly with log time:

$$\log HR(t) = \beta + \gamma \log t.$$

This can be fitted by constructing the appropriate time interaction in specialised software. A step function before and after a prespecified operational milestone is often easier to communicate. Choosing a cut point after inspecting the most dramatic curve exaggerates evidence.

If the effect is of no scientific interest but its non-proportionality is a nuisance, stratification can give groups separate baseline hazards. A stratified variable receives no single coefficient.

## 33.5 Functional form is a separate assumption

Proportional hazards says nothing about whether terrain enters linearly. A continuous predictor modelled as  $x\beta$  assumes each one-unit increase multiplies the hazard by the same factor. Diagnose nonlinearity with:

- prespecified transformations grounded in subject knowledge;
- restricted cubic splines fitted within the Cox model;
- plots of estimated effect against the original scale; and
- out-of-sample comparison using identical temporal folds.

Categorising a continuous predictor at a sample-chosen threshold loses information, creates artificial discontinuity, and adds multiplicity.

## 33.6 Martingale, deviance, and influence residuals

For subject  $i$ , a martingale residual has the form

$$M_i = \Delta_i - \hat{H}_i(Y_i).$$

It compares the observed event count, zero or one, with fitted cumulative hazard. Martingale residuals are asymmetric and can help diagnose functional form. Deviance residuals transform them to be more symmetric. Score residuals and DFBETA-like quantities assess how subjects influence coefficients.

Residuals are diagnostic views, not deletion commands. An influential remote project may be the only observation representing the deployment population. Investigate measurement, support, and model sensitivity before excluding it.

## 33.7 Accelerated failure time models

An accelerated failure time (AFT) model describes log time directly:

$$\log T = \mathbf{x}^\top \boldsymbol{\beta} + \sigma \varepsilon.$$

For two profiles differing by one unit in  $\mathbf{x}_j$ , the event-time ratio is

$$\frac{T(\mathbf{x}_j + 1)}{T(\mathbf{x}_j)} = e^{\beta_j}$$

under the model's distributional interpretation. If  $e^{\beta_j} = 1.20$ , the event-time distribution is stretched by 20%, conditional on other covariates. This is often easier to explain than a hazard ratio.

The error distribution determines the survival family:

| Error/distribution choice | Implied time family | Typical hazard shapes             |
|---------------------------|---------------------|-----------------------------------|
| extreme value             | Weibull             | monotone increasing or decreasing |
| normal                    | log-normal          | may rise then fall                |
| logistic                  | log-logistic        | may rise then fall; heavier tail  |

An AFT model gains direct time interpretation but imposes more baseline structure than Cox regression. Inspect probability plots, residuals, tail support, and extrapolation. A lower AIC is not permission to predict far beyond observed follow-up.

## 33.8 Restricted mean survival time

The restricted mean event-free time through horizon  $\tau$  is

$$\text{RMST}(\tau) = E[\min(T, \tau)] = \int_0^\tau S(t) dt.$$

It is the area under the survival curve through a prespecified horizon. The difference in RMST between groups is expressed in months of event-free time and does not require proportional hazards.

Choose  $\tau$  where follow-up supports estimation in all compared groups. Moving  $\tau$  until significance appears is outcome-driven analysis.

```
def restricted_mean(km_table, tau):
 """Right-continuous KM area from time zero through tau."""
 times = np.r_[0.0, km_table["time"].to_numpy()]
 survival = np.r_[1.0, km_table["survival"].to_numpy()]
 grid = np.unique(np.r_[times[times < tau], tau])
 area = 0.0
 for left, right in zip(grid[:-1], grid[1:]):
 position = np.searchsorted(times, left, side="right") - 1
 area += survival[position] * (right - left)
 return float(area)
```

The ordinary mean event time is not identifiable if the largest observed times are censored and the survival tail has not reached zero. RMST is intentionally bounded.

## 33.9 Uncertainty after diagnostics and selection

Standard coefficient intervals usually condition on the chosen variables, transformations, interactions, and time scale. If those were selected after extensive inspection, naive intervals omit selection uncertainty. Research-grade options include:

- prespecifying the primary model;
- separating exploratory and confirmatory analyses;
- bootstrapping the **entire** selection procedure for prediction performance;
- using shrinkage rather than unstable stepwise selection; and
- reporting sensitivity across a small, justified model set.

Robust standard errors address certain covariance misspecifications. They do not make the conditional log-hazard relation correct, remove selection bias, or establish causality.

## 33.10 Bayesian survival extension

A Bayesian parametric or semiparametric survival model combines a likelihood for event and censoring observations with priors on coefficients and baseline components:

$$p(\beta, h_0 \mid \text{data}) \propto p(\text{data} \mid \beta, h_0)p(\beta, h_0).$$

Posterior draws can propagate uncertainty into survival curves, RMST, and decisions. Hierarchical priors can partially pool district effects. But a posterior is conditional on the event definition, likelihood, prior, censoring assumption, and computation. Prior predictive checks, posterior predictive checks, convergence diagnostics, and sensitivity to plausible priors are part of the analysis.

“Bayesian” does not automatically mean causal, robust, or calibrated.

## 33.11 Research paper study: Schoenfeld (1982)

Read Schoenfeld's “Partial Residuals for the Proportional Hazards Regression Model”. The short paper connects residual contributions to partial-likelihood derivatives and shows how they can assess fit. The enduring lesson is structural: the same observed-minus-risk-set-expected quantity that estimates a coefficient can diagnose its time stability.

**Replication task.** Simulate one proportional coefficient and one effect that fades with time. Fit constant-coefficient Cox models. Plot each Schoenfeld residual against event time with a smooth trend. Repeat over seeds and record power and false alarms rather than presenting one attractive plot.

## 33.12 Build, break, and reflect

### Build

1. Plot complementary log-log curves for access mode.
2. Calculate Schoenfeld residuals for a fitted scratch model.
3. Compare a constant terrain effect with a prespecified terrain-by-log-time effect in appropriate software.
4. Report 36-month RMST for two prespecified profiles.

### Break

1. Declare proportional hazards true because a global test has  $p > 0.05$ .
2. Search 30 time cut points and report only the strongest interaction.

3. Remove influential projects without auditing them.
4. Extrapolate a fitted Weibull curve to 15 years with 3 years of support.

### Reflect

When would an AFT time ratio be more decision-relevant than a Cox hazard ratio? When would its distributional assumptions be less defensible?

## Day 33 exit check

You are ready for Day 34 when you can:

1. state proportional hazards as a time-constant hazard ratio;
2. explain a Schoenfeld residual as observed minus risk-set expected;
3. distinguish non-proportionality from nonlinear covariate form; and
4. interpret RMST and an AFT time ratio.

# Day 34 — Time-Varying Information and Dynamic Prediction

**Today's central idea:** Information collected during follow-up can update a prediction only at a time when it is available and only among subjects who have survived and remained observable to that time.

## 34.1 Baseline versus time-varying covariates

A baseline covariate is fixed or measured at time zero: appraisal capacity, district, or planned cable length. A time-varying covariate  $X_i(t)$  can change during follow-up: cumulative expenditure, latest schedule gap, or inspection status.

An extended Cox model writes

$$h_i(t) = h_0(t) \exp\{\beta^\top X_i(t)\}.$$

At each event time, the model uses the covariate value known immediately before that time. Substituting a subject's eventual maximum delay into all earlier risk sets leaks the future.

## 34.2 Start–stop representation

Suppose a project's schedule-gap status changes at month 10 and it triggers an event at month 17. Represent it as:

| Project | Start | Stop | Gap status | Event at stop? |
|---------|-------|------|------------|----------------|
| P1      | 0     | 10   | 0          | 0              |
| P1      | 10    | 17   | 1          | 1              |



Each row describes an interval during which the covariate is constant. The risk set at  $t$  includes intervals satisfying  $\text{start} < t \leq \text{stop}$  under a stated convention. Multiple rows are not independent subjects; inference and resampling must respect project identity.

```
import pandas as pd

updates = pd.DataFrame({
 "project_id": ["P1", "P1", "P2"],
 "start": [0.0, 10.0, 0.0],
 "stop": [10.0, 17.0, 14.0],
 "schedule_gap": [0, 1, 0],
 "event": [0, 1, 0],
})

assert (updates["start"] < updates["stop"]).all()
assert updates.groupby("project_id")["event"].sum().le(1).all()
```

Audit overlaps, gaps, duplicated events, updates recorded after the stop time, and whether measurements occur in response to latent deterioration. Observation frequency can itself be informative.

## 34.3 Internal and external time-varying covariates

An **external** covariate evolves independently of the subject's event process, such as a regional market price series. An **internal** covariate is generated by the subject's evolving state, such as accumulated cost or inspection result.

Internal covariates can be highly predictive, but interpretation is delicate. Conditioning on a process shaped by earlier health, management, or intervention can induce time-dependent confounding and selection. A time-varying Cox coefficient remains associational unless a causal design justifies more.

## 34.4 The landmark principle

Choose a landmark time  $s$ , such as month 12. Construct a new prediction problem:

- include only projects event-free and observable at  $s$ ;
- use information available by  $s$ ;
- reset analysis time or condition explicitly on survival to  $s$ ; and
- predict an event in  $(s, s + \tau]$ .

The target is

$$P(T \leq s + \tau, J = 1 \mid T > s, \mathcal{H}(s)),$$

where  $\mathcal{H}(s)$  is the information history available at the landmark and  $J = 1$  identifies the overrun event.

This is a different estimand from baseline 36-month risk. Its population contains survivors to month 12, and its prediction time is month 12.

## 34.5 Constructing the month-12 landmark dataset

```
from chapter5_data import make_mhp_survival_data

projects = make_mhp_survival_data()
landmark = 12.0
horizon_after_landmark = 24.0

At risk and still observed strictly after the landmark.
```

```

eligible = projects["follow_up_months"] > landmark
month12 = projects.loc[eligible].copy()

month12["landmark_follow_up"] = np.minimum(
 month12["follow_up_months"] - landmark,
 horizon_after_landmark,
)
month12["landmark_ouerrun_event"] = (
 (month12["event_type"] == 1)
 & (month12["follow_up_months"] <= landmark + horizon_after_landmark)
).astype(int)

assert month12["month12_progress_gap_sd"].notna().all()
assert month12["landmark_follow_up"].gt(0).all()

```

If a project was censored at exactly month 12, it is not known to be observable after the landmark. Timestamp conventions must be fixed before filtering.

## 34.6 Immortal-time bias

Suppose researchers label projects as “reviewed by month 6” and compare survival from appraisal between ever-reviewed and never-reviewed groups. To enter the reviewed group, a project must remain event-free and observed until its review. The pre-review period is “immortal” with respect to group assignment: an event there prevents classification as reviewed.

This can make review look protective even if it has no effect. Defensible options depend on the question:

- treat review as a time-varying exposure in a correctly specified causal analysis;
- emulate a target trial with assignment strategies and time zero aligned;
- perform a month-6 landmark analysis among eligible survivors; or
- for prediction, use review status only in a prediction made at or after month 6.

Merely adding `senior_review_by_month6` to a baseline Cox model is not a fix.

## 34.7 Recurrent events

Some outcomes can recur: repeated warning notices, outages, or hospitalisations. Reducing them to first event may answer a valid question, but discards burden after that event.

The Andersen–Gill counting-process extension represents multiple start–stop intervals and models an event intensity. It must address within-subject dependence, often using subject-clustered robust covariance. Other models distinguish event order, gap time, or subject-specific frailty. Choose according to whether the scientific target is first event, total event rate, time between events, or a latent propensity.

## 34.8 Joint models

Landmarking summarises history at selected times. A joint model instead links a longitudinal measurement model and a time-to-event model through shared latent structure. It can account for measurement error and informative observation/dropout under its assumptions.

Joint models are powerful but demanding: the longitudinal trajectory, association structure, event submodel, missingness, and priors or random effects all require checking. Use them when repeated noisy measurements and the event process are scientifically inseparable, not merely because a package offers them.

## 34.9 Research paper study: Andersen and Gill (1982)

Read Andersen and Gill's "Cox's Regression Model for Counting Processes: A Large Sample Study". The counting-process formulation places risk indicators, covariate histories, and event increments into a martingale framework. It is more than a data-format trick: it clarifies what information may be used just before each event and supports time-dependent and recurrent-event asymptotics.

Read van Houwelingen's "Dynamic Prediction by Landmarking in Event History Analysis" as a companion. Landmarking refits or updates prediction among those still at risk at selected times, offering a pragmatic alternative to a fully specified multi-state or joint model.

**Replication task.** Create month-6, month-12, and month-18 landmark cohorts. At each landmark, record sample size, event count in the next 18 months, predictor availability, and censoring. Compare a baseline-only model with one using the current progress gap. Do not pool their metrics without stating how changing landmark populations are weighted.

## 34.10 Build, break, and reflect

### Build

1. Convert a three-update project history into start–stop form.
2. Construct the month-12 cohort and verify every feature timestamp.
3. Compare baseline and updated 24-month predictions among month-12 survivors.
4. Draw a timeline showing appraisal, review eligibility, landmark, event, and censoring.

### Break

1. Backfill the latest progress gap to time zero.
2. split start–stop rows randomly across train and test.
3. Classify a subject as reviewed throughout follow-up because review happened eventually.
4. Compare different landmark populations as if they were the same cohort.

### Reflect

Why can a month-12 model be better calibrated than the baseline model yet be useless at appraisal? State its prediction time and eligible population.

## Day 34 exit check

You are ready for Day 35 when you can:

1. represent a changing covariate without future leakage;
  2. define a landmark estimand;
  3. explain immortal time as guaranteed event-free time created by exposure classification; and
  4. distinguish first-event, recurrent-event, and dynamic-prediction questions.
-

# Day 35 — Competing Risks and Multi-State Processes

**Today's central idea:** When another event prevents the event of interest, its occurrence is known outcome information. Treating it as ordinary censoring answers a different question and can overstate absolute risk.

## 35.1 Event type belongs beside event time

Let  $T$  be time to the first event of any type and  $J \in 1, \dots, K$  identify its cause. In the running case:

- $J = 1$ : first formal overrun warning;
- $J = 2$ : project cancellation; and
- $J = 0$ : no event type observed because follow-up was censored.

The cause-specific cumulative incidence function (CIF) is

$$F_k(t) = P(T \leq t, J = k).$$

For overrun warning,  $F_1(36)$  is the probability that a project experiences an overrun warning by month 36 **before cancellation**, under the observed event process.

The all-cause survival function is

$$S(t) = P(T > t),$$

meaning no event of any type has occurred by  $t$ .

## 35.2 Cause-specific hazards

The cause-specific hazard for type  $k$  is

$$\lambda_k(t) = \lim_{\Delta t \downarrow 0} \frac{P(t \leq T < t + \Delta t, J = k \mid T \geq t)}{\Delta t}.$$

It is the instantaneous rate of cause  $k$  among subjects still free of **all** event types. The overall hazard is

$$\lambda(t) = \sum_{k=1}^K \lambda_k(t).$$

Consequently,

$$S(t) = \exp \left[ - \sum_{k=1}^K \int_0^t \lambda_k(u) du \right].$$

The cumulative incidence is

$$F_k(t) = \int_0^t S(u-) \lambda_k(u) du.$$

The formula expresses a necessary sequence: remain free of every event until just before  $u$ , then experience cause  $k$  at  $u$ .

### 35.3 Why $1 - \hat{S}_{KM,1}$ is wrong for real-world cause probability

If cancellations are treated as censored and Kaplan–Meier estimates an overrun survival curve,  $1 - \hat{S}_{KM,1}(t)$  imagines that cancelled projects remain capable of later overruns. It estimates a net-risk quantity in a hypothetical setting where the competing cause is removed under strong assumptions. It is not the observed-world overrun probability.

As cancellation incidence rises, this overstatement can become large. The competing event is not an incomplete label; it is a known outcome that changes what can happen next.

### 35.4 Aalen–Johansen cumulative incidence by hand

At distinct event time  $t_j$ , let:

- $n_j$  be the all-event-free risk set;
- $d_{kj}$  be events from cause  $k$ ; and
- $d_j = \sum_k d_{kj}$ .

Just before the time, estimated all-cause survival is  $\hat{S}(t_{j-})$ . The CIF increment is

$$\Delta \hat{F}_k(t_j) = \hat{S}(t_{j-}) \frac{d_{kj}}{n_j}.$$

Then survival updates by

$$\hat{S}(t_j) = \hat{S}(t_{j-}) \left(1 - \frac{d_j}{n_j}\right).$$

Suppose five projects begin, one overrun occurs at month 2, one cancellation at month 3, and one overrun at month 5 after one censoring at month 4.

- Month 2:  $\Delta \hat{F}_1 = 1 \times 1/5 = 0.20$ ; survival becomes **0.80**.
- Month 3:  $\Delta \hat{F}_2 = 0.80 \times 1/4 = 0.20$ ; survival becomes **0.60**.
- Month 4 censoring changes the later risk set but no probability directly.
- Month 5: two remain at risk, so  $\Delta \hat{F}_1 = 0.60 \times 1/2 = 0.30$ .

Thus  $\hat{F}_1(5) = 0.50$ ,  $\hat{F}_2(5) = 0.20$ , and  $\hat{S}(5) = 0.30$ . Their sum is 1.

### 35.5 Aalen–Johansen from scratch

```
import numpy as np
import pandas as pd

def cumulative_incidence(time, event_type, causes=None):
 """Aalen-Johansen CIF for one initial state and absorbing causes."""
 time = np.asarray(time, dtype=float)
 event_type = np.asarray(event_type, dtype=int)
 if causes is None:
 causes = np.sort(np.unique(event_type[event_type > 0]))
```

```

survival = 1.0
cif = {int(cause): 0.0 for cause in causes}
rows = []

for t in np.sort(np.unique(time[event_type > 0])):
 at_risk = int(np.sum(time >= t))
 counts = {
 int(cause): int(np.sum((time == t) & (event_type == cause)))
 for cause in causes
 }
 total_events = sum(counts.values())
 survival_before = survival

 for cause, count in counts.items():
 cif[cause] += survival_before * count / at_risk
 survival *= 1.0 - total_events / at_risk

 row = {
 "time": t,
 "at_risk": at_risk,
 "survival": survival,
 }
 row.update({f"cif_{cause}": cif[cause] for cause in causes})
 rows.append(row)
return pd.DataFrame(rows)

toy_time = [2, 3, 4, 5, 8]
toy_type = [1, 2, 0, 1, 0]
aj = cumulative_incidence(toy_time, toy_type, causes=[1, 2])
print(aj)
last = aj.iloc[-1]
assert np.isclose(last["cif_1"] + last["cif_2"] + last["survival"], 1.0)

```

This is the single-starting-state form of the Aalen–Johansen estimator. In a general multi-state model, a matrix product integral estimates transition probabilities among several states.

## 35.6 Cause-specific Cox regression

To model cause  $k$ , fit a Cox model in which cause- $k$  events have event indicator 1 and other causes leave the cause- $k$  risk set at their event times. The model is

$$\lambda_k(t \mid x) = \lambda_{0k}(t) \exp(x^\top \beta_k).$$

The coefficient  $\beta_k$  describes a covariate's association with the instantaneous cause- $k$  rate among subjects still free of every cause. Fit one cause-specific model per event type, then combine their estimated hazards to obtain absolute CIFs.

A variable can increase the cause-1 hazard yet decrease cause-1 cumulative incidence if it increases a competing cause even more. Hazards and probabilities answer different questions.

## 35.7 The Fine–Gray subdistribution hazard

Fine and Gray define a subdistribution hazard connected directly to the CIF. Its risk-set construction retains, with weighting conventions, subjects who experienced a competing event. A proportional subdistribution-hazards model can describe covariate association with cumulative incidence through a subdistribution hazard ratio.

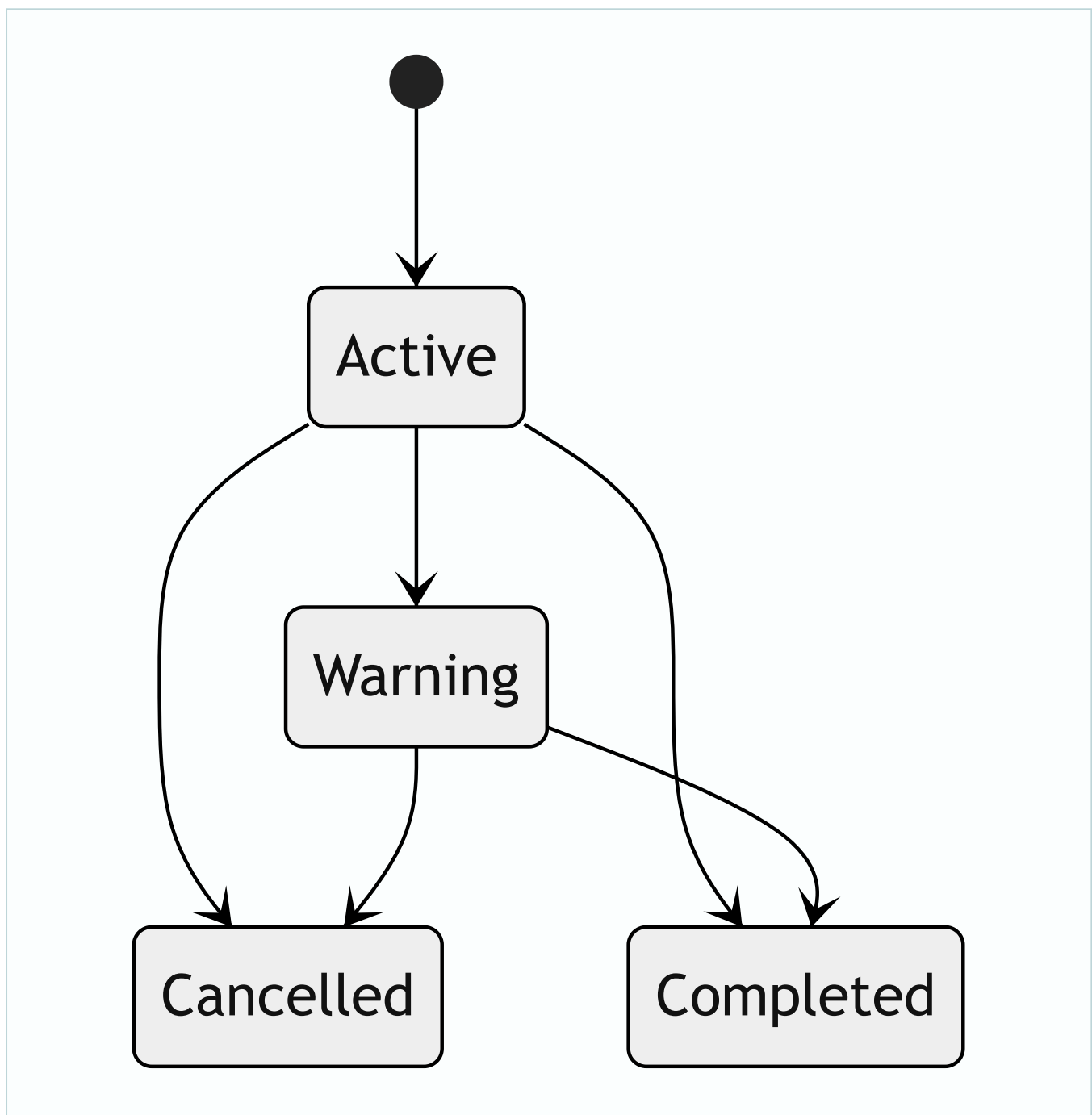
That ratio is not a cause-specific hazard ratio and should not be described as the instantaneous event rate among currently event-free subjects. Fine–Gray is useful when the regression target is a particular CIF, but fitting separate Fine–Gray models for all causes does not automatically produce coherent probabilities that sum to at most one.

Choose by estimand:

| Question                                                                                         | Natural target               |
|--------------------------------------------------------------------------------------------------|------------------------------|
| What is the instantaneous overrun rate among active, warning-free projects?                      | cause-specific hazard        |
| What proportion will receive an overrun warning before cancellation by 36 months?                | cumulative incidence         |
| How are covariates associated directly with that CIF under a regression model?                   | subdistribution hazard       |
| What are probabilities of moving among active, delayed, warned, cancelled, and completed states? | multi-state transition model |

## 35.8 Multi-state models

Some “competing” events are better represented as intermediate states. A project could move through:



A multi-state model estimates transition-specific hazards and state-occupation probabilities. It can answer questions that a first-event analysis cannot: probability of being active and warning-free at month 24, probability of completing after a warning, or expected time spent in delayed status.

The Markov assumption says future transitions depend on current state and covariates, not the full past. A semi-Markov model can incorporate time since entering the current state. State definitions and transition timestamps are measurement choices, not merely model inputs.

## 35.9 Research papers: Aalen–Johansen and Fine–Gray

Read Aalen and Johansen's "An Empirical Transition Matrix for Non-homogeneous Markov Chains Based on Censored Observations". Its product-integral view generalises product-limit survival to transition matrices.

Then read Fine and Gray's "A Proportional Hazards Model for the Subdistribution of a Competing Risk". The paper targets covariate effects on a subdistribution connected to the CIF. Do not read the phrase "proportional hazards" and assume it is the Cox cause-specific risk set.

**Replication task.** Simulate a covariate that strongly increases cancellation but has no direct effect on overrun hazard. Plot cause-specific overrun hazards and overrun CIFs by covariate group. Explain why the CIFs separate even though the direct cause-1 mechanism does not.

## 35.10 Build, break, and reflect

### Build

1. Estimate overrun and cancellation CIFs with the scratch function.
2. Verify  $\hat{S}(t) + \hat{F}_1(t) + \hat{F}_2(t) = 1$  after every event time.
3. Compare  $\hat{F}_1(36)$  with one minus a Kaplan–Meier curve that censors cancellations.
4. Fit separate cause-specific Cox models and combine their hazards into profile-specific CIFs.

### Break

1. Recode cancellations as event-free at database close.
2. Call a subdistribution hazard ratio a probability ratio.
3. Fit independent binary 36-month models for mutually exclusive causes and ignore predictions summing above one.
4. Use a first-event model to claim what happens after a warning.

### Reflect

Can preventing cancellations increase the observed number of overrun warnings even if it does not change the overrun mechanism? Explain using time at risk and competing events.

## Day 35 exit check

You are ready for Day 36 when you can:

1. derive a CIF increment as survival-before times a cause-specific hazard increment;
  2. explain why a competing event is not ordinary censoring for absolute risk;
  3. distinguish cause-specific and subdistribution hazard ratios; and
  4. identify when a multi-state process is more informative than first event.
-



# Day 36 — Nonlinear and Hierarchical Survival Models

---

**Today's central idea:** Flexible survival learners change how covariates map to hazards or survival curves; they do not remove censoring assumptions, time horizons, or validation boundaries.

## 36.1 Why move beyond a linear Cox score?

The Cox model can already include splines and interactions. Move beyond it when evidence and domain knowledge suggest:

- complex interactions that are hard to enumerate;
- thresholds or local effects;
- many predictors with regularisation needs;
- time-varying effects poorly represented by a small extension; or
- prediction as the primary goal and sufficient event information to support flexibility.

Always retain a strong regularised Cox reference. A flexible model that barely improves validated prediction but is harder to calibrate, audit, and maintain may not be the better research product.

## 36.2 Survival trees

A survival tree recursively partitions subjects. A common split statistic is based on separation of child-node event processes, such as a log-rank score. Each terminal node estimates a survival curve or cumulative hazard from the subjects reaching it.

Unlike a regression tree, it cannot minimise ordinary squared error on observed times: a censored time is not the true target. Valid split criteria and terminal estimators must respect risk sets.

Tree choices include:

- candidate split variables and thresholds;
- minimum terminal-node event count, not just row count;
- tree depth or pruning;
- handling of ties and competing events; and
- whether terminal predictions are survival, cause-specific hazard, or CIF.

A tiny leaf with 30 rows but two events does not contain 30 complete event-time observations.

## 36.3 Random survival forests

A random survival forest usually:

1. draws bootstrap samples of subjects;
2. grows an unpruned or deeply grown survival tree;
3. considers a random feature subset at each split;
4. estimates a terminal-node cumulative hazard or survival curve; and

5. averages predictions across trees.

Feature subsampling and bootstrap variation reduce correlation among trees. Out-of-bag subjects can estimate performance without an additional inner holdout, but OOB evaluation is not external or temporal validation and must preserve clusters when units are clustered.

Hyperparameters include number of trees, features per split, minimum terminal events, split rule, and sampled fraction. Tune against a censoring-aware metric at prespecified horizons.

## 36.4 Log-rank splitting is not universal

A log-rank split favours proportional separation. It may overlook crossing hazards or differences confined to an important horizon. Alternative split rules target survival differences, conservation of events, or competing-risk CIFs. The split target should align with the intended prediction.

No tree method handles causal confounding, informative censoring, left truncation, or measurement error “automatically.” Support depends on the implementation.

## 36.5 Survival boosting

Gradient boosting can optimise the negative Cox partial log likelihood. If  $f(\mathbf{x})$  is the current log-risk score, the loss couples subjects through event-time risk sets:

$$\mathcal{L}(f) = - \sum_{i:\Delta_i=1} \left[ f(\mathbf{x}_i) - \log \sum_{k \in R(T_i)} e^{f(\mathbf{x}_k)} \right].$$

Small trees or splines are added in directions that reduce this loss. Other boosting methods optimise parametric AFT, discrete-time likelihood, or ranking objectives.

As in Chapter 4, learning rate, tree complexity, stages, and early stopping interact. Early stopping uses validation evidence and therefore belongs inside the development procedure.

## 36.6 Discrete-time survival as repeated classification

Partition follow-up into intervals. For each subject-interval still at risk, model the conditional event probability

$$q_{it} = P(T \text{ occurs in interval } t \mid T \text{ has not occurred earlier}, \mathbf{X}_i).$$

Then

$$S_i(t) = \prod_{u \leq t} (1 - q_{iu}).$$

Logistic or complementary log-log models, boosted trees, and neural networks can estimate  $q_{it}$ . But interval rows from one subject are dependent, time basis matters, and random row splitting leaks subjects and later history. Competing causes require a coherent multinomial transition probability at each interval.

## 36.7 Neural survival models

Neural models may parameterise a Cox score, discrete hazards, an event-time distribution, or multiple causes. Their names do not identify their estimand. Ask:

- Is the output a relative score, hazard, survival curve, density, or CIF?

- How are censored observations represented in the loss?
- Are predicted survival curves monotone and probabilities coherent?
- How is time represented?
- How are hyperparameters selected with limited events?
- Does improvement persist under temporal and external validation?

Deep learning is most plausible with large event counts, rich inputs such as images or sequences, and a validation design matching deployment. Typical tabular cohorts often reward simpler regularised models.

## 36.8 Shared frailty and partial pooling

For project  $i$  in district  $j$ , a shared-frailty Cox model can write

$$h_{ij}(t) = h_0(t) \exp(\mathbf{x}_{ij}^\top \boldsymbol{\beta} + u_j),$$

where  $u_j$  is a latent district effect. Districts with few events are partially pooled toward the population distribution. Frailty induces dependence and changes survival among the selected risk set over time.

Frailty variance does not identify a specific unmeasured cause. A large estimated variance says districts differ beyond recorded predictors under the model; it does not say why.

For prediction in a new district, a conditional prediction using an estimated existing-district frailty is unavailable. The deployment target determines whether to integrate over the frailty distribution, estimate a new effect from local data, or use observed district features.

## 36.9 Cluster-aware resampling

If several projects share a contractor, watershed, district office, or procurement batch, randomly splitting individual projects can put near-dependent units on both sides. Choose the unit of generalisation:

- new project within known district: group structure may be represented in both sets;
- new district: hold out entire districts;
- future calendar period: split on approval time;
- future district and time: use nested geographic-temporal evaluation if data permit.

A model can pass one target and fail another. “Generalises” must name a destination.

## 36.10 Explanation for survival models

Permutation importance should use a survival metric and preserve relevant grouping. Partial dependence can target survival  $S(t \mid \mathbf{x})$  or CIF  $F_k(t \mid \mathbf{x})$  at a stated horizon; importance can change across horizons. SHAP-style explanations require an output scale and background distribution.

An explanation of a 36-month CIF is not an explanation of a cause-specific hazard score. Neither is a causal decomposition. Correlated features, unsupported perturbations, and selection into the risk set remain concerns.

## 36.11 Research paper study: Ishwaran and colleagues (2008)

Read “Random Survival Forests”. The paper introduces splitting rules for right-censored outcomes, ensemble cumulative-hazard ideas, an event-conservation principle, and OOB assessment. Study the survival-specific construction rather than summarising it as “random forest plus censoring.”

**Replication task.** In a survival library, compare regularised Cox, a spline Cox model, and a random survival forest under the same temporal folds. Predefine 24- and 36-month IPCW Brier score as selection metrics. Record events per fold, tune equal budgets, and examine whether nonlinear gains survive the locked period.

## 36.12 Build, break, and reflect

### Build

1. Fit a shallow survival tree and inspect event counts in every leaf.
2. Fit a random survival forest with subject-level bootstrap sampling.
3. Compare OOB and forward-year performance.
4. Plot horizon-specific permutation importance at 12 and 36 months.

### Break

1. Fit ordinary random-forest regression to observed follow-up time.
2. Tune leaf size using row count while ignoring terminal events.
3. Split interval rows independently.
4. Explain a hazard score as if it were a 36-month probability.

### Reflect

If a forest improves concordance but worsens 36-month calibration and net benefit, is it better? Answer for a ranking use case and for a capacity-limited review policy.

## Day 36 exit check

You are ready for Day 37 when you can:

1. explain how a survival tree's split and leaf differ from ordinary regression;
2. identify the output target of a survival ensemble;
3. state why OOB performance is not evidence of temporal transport; and
4. design resampling around the intended unit of generalisation.

---

# Day 37 — Censoring-Aware Evaluation and Decisions

---

**Today's central idea:** A survival model must be evaluated at supported horizons using methods that recognise unknown outcome status; ranking alone is not probability accuracy or decision value.

## 37.1 Begin with the prediction contract

For every reported metric, state:

- prediction time: appraisal or month-12 landmark;
- eligible population at that time;

- event type;
- horizon;
- competing-event treatment;
- censoring adjustment;
- validation destination; and
- whether the metric assesses ranking, probability, or action.

“C-index 0.74” is incomplete. A useful statement is: “In projects approved during 2023–2025, the baseline model's cause-specific Harrell concordance for overrun warnings was 0.74, with cancellation treated as a competing event in absolute-risk evaluation and as censoring for this cause-specific ranking summary.”

## 37.2 Harrell's concordance

For a pair  $i, j$ , suppose  $i$  has the earlier observed time and experiences the event of interest then. The pair is comparable. A risk score is concordant if it assigns higher risk to  $i$ .

$$\hat{C} = \frac{\text{concordant} + 0.5 \times \text{risk-score ties}}{\text{comparable pairs}}.$$

```
def harrell_c_index(time, event, risk_score):
 time = np.asarray(time, dtype=float)
 event = np.asarray(event, dtype=int)
 risk_score = np.asarray(risk_score, dtype=float)
 concordant = 0.0
 comparable = 0

 for i in range(len(time)):
 if event[i] != 1:
 continue
 later = time > time[i]
 comparable += int(later.sum())
 concordant += np.sum(risk_score[i] > risk_score[later])
 concordant += 0.5 * np.sum(risk_score[i] == risk_score[later])

 return concordant / comparable if comparable else np.nan
```

Harrell's C can depend on the censoring distribution and usually excludes ambiguous pairs. It is a global rank summary across observed event times, not horizon-specific calibration. Uno's C uses inverse censoring weights to target a concordance measure less dependent on censoring under its assumptions.

## 37.3 Time-dependent discrimination

At horizon  $\tau$ , define cases as subjects with the event by  $\tau$  and controls according to a chosen incident/dynamic or cumulative/dynamic definition. Time-dependent AUC asks whether predicted risk at  $\tau$  ranks cases above eligible controls, with censoring adjustment.

Different definitions answer different questions. Report the definition and competing-risk handling. A model may rank early events well but late events poorly, so plot AUC across a supported time range rather than selecting its highest point.

## 37.4 Brier score with incomplete status

Without censoring, the horizon-specific event Brier score is

$$BS(\tau) = \frac{1}{n} \sum_{i=1}^n \left\{ I(T_i \leq \tau, J_i = 1) - \hat{F}_{1i}(\tau) \right\}^2.$$

If a subject is censored before  $\tau$ , its event status at  $\tau$  is unknown. Inverse probability of censoring weighting (IPCW) upweights comparable observed subjects by the inverse probability of remaining uncensored.

Let  $G(t) = P(C > t)$  and  $\hat{G}$  be estimated from training/reference data, treating censoring as the event in a reverse Kaplan–Meier calculation. For competing risks, event type 2 before  $\tau$  is known not to be type 1 by  $\tau$  and remains an observed outcome, not a censoring.

## 37.5 IPCW Brier score from scratch

```
def step_value(table, query, column="survival", left_limit=False):
 """Evaluate a right-continuous step curve; optionally just before query."""
 times = table["time"].to_numpy(dtype=float)
 values = table[column].to_numpy(dtype=float)
 side = "left" if left_limit else "right"
 position = np.searchsorted(times, query, side=side) - 1
 if np.isscalar(query):
 return 1.0 if position < 0 else float(values[position])
 return np.where(position < 0, 1.0, values[np.maximum(position, 0)])

def fit_censoring_curve(reference_time, reference_type):
 censor_event = (np.asarray(reference_type) == 0).astype(int)
 return kaplan_meier(reference_time, censor_event)

def ipcw_brier_competing(time, event_type, predicted_cif, horizon,
 censoring_curve):
 """IPCW Brier score for cause 1 with cause 2 as a known competing event."""
 time = np.asarray(time, dtype=float)
 event_type = np.asarray(event_type, dtype=int)
 predicted_cif = np.asarray(predicted_cif, dtype=float)
 squared = np.zeros_like(time)
 weight = np.zeros_like(time)

 event_before = (time <= horizon) & (event_type > 0)
 at_risk_after = time > horizon

 if event_before.any():
 g_before = step_value(
 censoring_curve,
 time[event_before],
 left_limit=True,
)
 outcome = (event_type[event_before] == 1).astype(float)
 weight[event_before] = 1.0 / np.clip(g_before, 1e-8, None)
 squared[event_before] = (outcome - predicted_cif[event_before]) ** 2

 if at_risk_after.any():
 g_horizon = step_value(censoring_curve, horizon)
 weight[at_risk_after] = 1.0 / max(g_horizon, 1e-8)
 squared[at_risk_after] = predicted_cif[at_risk_after] ** 2

 # Subjects censored on or before the horizon contribute zero weight.
 return float(np.mean(weight * squared))
```

This implementation assumes censoring independent of the event process under the chosen reference information. If censoring depends on covariates, estimate  $G(t \mid \mathbf{X})$  appropriately and validate that model. Extremely small  $\hat{G}$  creates unstable weights; truncate the evaluation horizon or prespecify defensible weight truncation and sensitivity analyses.

The integrated Brier score averages  $BS(t)$  over a stated time interval. Integration weights and maximum time must be reported.

## 37.6 Censoring-aware calibration

At horizon  $\tau$ , calibration compares predicted CIF with observed CIF:

1. calculate each subject's  $\hat{F}_1(\tau)$  without using validation outcomes;
2. create prespecified or quantile-based groups using validation predictions;
3. report mean prediction in each group;
4. estimate observed cause-1 CIF with Aalen–Johansen inside each group; and
5. show uncertainty and numbers at risk.

Do not use the raw proportion of observed cause-1 events when censoring hides status for some subjects. Avoid declaring calibration from a 45-degree-looking plot with six subjects per bin.

Calibration-in-the-large and slope can be extended to survival predictions, but implementation depends on the target model and horizon. Flexible recalibration needs its own development evidence and must not be fitted on the final test.

## 37.7 Prediction error versus coefficient inference

A narrow confidence interval for  $\beta_j$  does not imply accurate individual prediction. Conversely, a useful predictor may have unstable individual coefficients under collinearity. Evaluate:

- coefficient uncertainty for parameter claims;
- calibration for probability accuracy;
- discrimination for ranking;
- prediction intervals or posterior uncertainty for individual forecasts where defined; and
- decision analysis for consequences.

For resampled performance, repeat the entire modelling pipeline: preprocessing, feature selection, hyperparameter tuning, baseline-hazard estimation, and any recalibration.

## 37.8 Split conformal is not plug-and-play under censoring

Ordinary split conformal regression assumes observed calibration outcomes from an exchangeable sample. A censored event time is not an observed target, so absolute residuals  $|T_i - \hat{T}_i|$  are unavailable for censored subjects. Dropping them destroys exchangeability and favours short observed times.

Conformal survival methods introduce censoring assumptions, weighting, imputation bounds, or alternative prediction sets. For example, Candès and colleagues' "Conformalized Survival Analysis" distinguishes completely and conditionally independent censoring conditions. The lesson is not that conformal guarantees vanish, but that the guarantee is conditional on a censored-data construction. State the target—lower bound, interval, or survival-probability band—and its coverage population.

## 37.9 Validation has destinations

| Validation type | Main question                                              | Typical split                 |
|-----------------|------------------------------------------------------------|-------------------------------|
| Apparent        | How well does the fitted model describe its training data? | none; optimistic              |
| Internal        | How much optimism arises within this source?               | bootstrap or cross-validation |

| Validation type | Main question                                      | Typical split                                 |
|-----------------|----------------------------------------------------|-----------------------------------------------|
| Temporal        | Does it transport to later practice?               | earlier approvals train, later approvals test |
| Geographic      | Does it transport to new places?                   | entire districts/regions held out             |
| External        | Does it work in an independently collected source? | separate organisation or registry             |
| Prospective     | Does the frozen system work in live workflow?      | forward collection after lock                 |

External validation is not merely a random 20% holdout. It changes source, time, setting, or all three. Dataset shift should be described in predictors, censoring, competing-event rates, follow-up opportunity, measurement, and intervention practices.

## 37.10 Horizon-specific decisions

Suppose review by month 12 can avert or mitigate losses expected before month 36. Let  $p_i = \hat{F}_{1i}(36)$ ,  $C_{FP}$  be the burden of reviewing a project that would not experience the event, and  $C_{FN}$  the preventable loss from not reviewing a project that would.

Under simplified symmetric action effectiveness, review when

$$p_i > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

This derivation assumes calibrated probabilities, common costs, an action that is available now, and that review changes consequences as encoded. Real decisions may include limited capacity, time-dependent review effectiveness, competing cancellation, equity constraints, and harms from delay.

If only 10% of projects can be reviewed, ranking determines allocation, but probability calibration still informs expected yield and planning. Freeze the threshold or capacity rule using development data.

## 37.11 Dynamic thresholds and monitoring

At appraisal, a 36-month probability can trigger preventive design review. At month 12, the remaining horizon is shorter and the intervention differs. A dynamic policy should specify at each landmark:

- eligible projects;
- prediction horizon;
- model version and inputs;
- threshold or capacity rule;
- intervention offered;
- override and appeal process; and
- re-evaluation schedule.

Repeated alerts impose burden. Evaluate alert frequency, first-alert timing, warning lead time, repeated false alarms, and whether actions alter the future data distribution.

## 37.12 Subgroup evaluation

For each important group, report counts, event types, censoring, follow-up, horizon-specific CIF, calibration, discrimination where estimable, review rate, and uncertainty. A subgroup with four cause-1 events cannot support a stable AUC merely because software returns a number.



Differences can reflect sampling, measurement, baseline incidence, competing risks, intervention access, or model misspecification. Metric parity is not proof of fairness; disparity is not by itself proof of discrimination. Tie evaluation to the concrete decision and institutional process.

## 37.13 Research papers: Graf and Uno

Graf and colleagues' "Assessment and Comparison of Prognostic Classification Schemes for Survival Data" develops prediction-error assessment with censored outcomes, including weighted Brier-style evaluation over time. Read it to see why validation subjects with incomplete status need principled weighting.

Uno and colleagues' "On the C-Statistics for Evaluating Overall Adequacy of Risk Prediction Procedures with Censored Survival Data" shows a limitation of the commonly used concordance estimator under censoring and proposes an IPCW alternative.

**Replication task.** Hold the data-generating event model fixed while changing only the censoring distribution. Compare Harrell and IPCW concordance across 200 simulations. Then change the prediction horizon and show that a global concordance ranking need not match horizon-specific Brier ranking.

## 37.14 Build, break, and reflect

### Build

1. Calculate Harrell concordance for a hand-worked six-project example.
2. Fit the censoring curve on development data and compute 12-, 24-, and 36-month IPCW Brier scores on validation data.
3. Build an eight-bin cause-1 calibration table using Aalen–Johansen observed incidence.
4. Compare a cost-derived threshold with a 10% capacity rule.

### Break

1. Treat censored-before-horizon subjects as non-events.
2. Fit the censoring distribution on the locked test and tune weight truncation for the best score.
3. Report only concordance.
4. Construct calibration bins from observed outcomes.

### Reflect

Why might a lower 36-month Brier score matter more than a higher global C-index for budgeting review teams? When would the opposite be true?

## Day 37 exit check

You are ready for Day 38 when you can:

1. construct comparable pairs for concordance;
  2. explain IPCW as recovery of information under a censoring assumption;
  3. compare predicted CIF with Aalen–Johansen incidence; and
  4. separate internal, temporal, external, and prospective evidence.
-

# Day 38 — The Registered Locked Study and Causal Boundary

**Today's central idea:** A research result is a frozen chain from protocol to claim. Prediction of an event history does not identify what would happen under an intervention.

## 38.1 Registration before contact with the lockbox

A useful registration is specific enough that another researcher can identify deviations. It should include:

- scientific and operational question;
- unit, eligibility, time origin, entry, event types, censoring, and horizons;
- prediction times and permitted feature timestamps;
- estimands for cause-specific rate and absolute incidence;
- development, validation, locked-test, and external-validation sources;
- preprocessing fitted inside each fold;
- complete candidate families and tuning ranges;
- primary and secondary metrics;
- censoring model and weight rules;
- competing-risk method;
- subgroup analyses and minimum reporting counts;
- missing-data strategy;
- uncertainty procedure;
- decision rule and cost/capacity assumptions;
- diagnostic and sensitivity analyses;
- stopping rules and model-update policy; and
- exact claim language allowed by the design.

Registration does not forbid exploration. It labels exploration honestly and prevents it from masquerading as confirmation.

## 38.2 The pre-specified Chapter 5 study

| Component       | Frozen choice                                                      |
|-----------------|--------------------------------------------------------------------|
| Unit            | one eligible fictional MHP project                                 |
| Prediction time | technical appraisal                                                |
| Time origin     | approval date                                                      |
| Cause 1         | first formal projected-cost warning above 115% of appraisal budget |
| Cause 2         | cancellation before cause 1                                        |
| Censoring       | database close or loss to follow-up                                |
| Horizons        | 12, 24, and 36 months                                              |
| Development     | approval years through 2022                                        |

| Component                 | Frozen choice                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------|
| Forward validation        | validation years 2020, 2021, and 2022                                                           |
| Locked temporal test      | approval years 2023–2025                                                                        |
| Primary selection metric  | mean cause-1 IPCW Brier across the three horizons, averaged equally across validation years     |
| Secondary metrics         | horizon-specific Brier, cause-specific Harrell C, cause-1 calibration                           |
| Candidates                | historical CIF; raw and engineered cause-specific Cox systems with prespecified ridge penalties |
| Competing-risk prediction | combine the two fitted cause-specific hazards into coherent CIFs                                |
| Groups                    | access mode and district, with counts and event types                                           |
| Uncertainty               | paired row bootstrap for locked Brier difference, interpreted conditionally on the frozen study |
| Test use                  | one evaluation after candidate selection                                                        |

The historical-CIF candidate is essential. A complicated model should outperform the population event-history baseline, not merely another complicated model.

### 38.3 Complete executable benchmark

The code assumes `chapter2_data.py`, `chapter3_data.py`, `chapter4_data.py`, and `chapter5_data.py` have been saved from the four chapters. It also assumes the `CoxPHFromScratch`, `kaplan_meier`, `cumulative_incidence`, `step_value`, `fit_censoring_curve`, `ipcw_brier_competing`, and `harrell_c_index` definitions from this chapter are in scope. To make a standalone script, copy those definitions above this block.

```

from pathlib import Path
import json
import platform

import numpy as np
import pandas as pd
import scipy

from chapter5_data import make_mhp_survival_data

HORIZONS = np.array([12.0, 24.0, 36.0])
VALIDATION_YEARS = [2020, 2021, 2022]
TEST_START_YEAR = 2023
RANDOM_SEED = 538

RAW_NUMERIC = [
 "planned_capacity_kw",
 "estimated_cable_km",
 "road_distance_observed_km",
 "surveyed_route_km",
 "terrain_index",
 "contractor_experience_projects",
 "appraisal_budget_2025_million_pkr",
 "start_year",
]
CATEGORICAL = ["district", "access_mode"]

class DesignBuilder:
 """Training-only imputation, feature construction, scaling, and encoding."""

 def __init__(self, engineered=False):
 self.engineered = bool(engineered)

```

```

def _numeric_frame(self, frame):
 out = frame[RAW_NUMERIC].copy()
 if self.engineered:
 out["capacity_squared"] = frame["planned_capacity_kw"] ** 2
 out["road_hinge_above_18"] = (
 frame["road_distance_observed_km"] - 18.0
).clip(lower=0.0)
 out["cable_by_road"] = (
 frame["estimated_cable_km"]
 * frame["road_distance_observed_km"]
)
 out["log_contractor_experience"] = np.log1p(
 frame["contractor_experience_projects"]
)
 out["budget_per_planned_kw"] = (
 frame["appraisal_budget_2025_million_pkr"]
 / frame["planned_capacity_kw"]
)
 return out.astype(float)

def fit(self, frame):
 numeric = self._numeric_frame(frame)
 self.numeric_columns_ = numeric.columns.tolist()
 self.medians_ = numeric.median()
 imputed = numeric.fillna(self.medians_)
 self.means_ = imputed.mean()
 self.scales_ = imputed.std(ddof=0).replace(0.0, 1.0)
 self.levels_ = {
 column: sorted(frame[column].dropna().astype(str).unique().tolist())
 for column in CATEGORICAL
 }
 self.feature_names_ = self.numeric_columns_.copy()
 for column in CATEGORICAL:
 self.feature_names_.extend(
 [f"{column}={level}" for level in self.levels_[column][1:]]
)
 return self

def transform(self, frame):
 numeric = self._numeric_frame(frame)[self.numeric_columns_]
 numeric = numeric.fillna(self.medians_)
 blocks = [(numeric - self.means_) / self.scales_].to_numpy()

 for column in CATEGORICAL:
 values = frame[column].astype(str).to_numpy()
 levels = self.levels_[column]
 if len(levels) > 1:
 encoded = np.column_stack(
 [(values == level).astype(float) for level in levels[1:]]
)
 blocks.append(encoded)
 return np.column_stack(blocks)

def fit_transform(self, frame):
 return self.fit(frame).transform(frame)

def cif_from_cause_specific_models(model1, model2, X, horizons):
 """Combine cause-specific Breslow hazards with piecewise-exponential jumps."""
 X = np.asarray(X, dtype=float)
 horizons = np.sort(np.asarray(horizons, dtype=float))
 # union1d creates one sorted, duplicate-free event-time grid.
 event_times = np.union1d(model1.event_times_, model2.event_times_)

 # zip pairs each event time with its hazard increment; dict makes the
 # increment retrievable by time, and .get(t, 0) below handles absent jumps.
 increment1 = dict(zip(model1.event_times_, model1.baseline_increments_))
 increment2 = dict(zip(model2.event_times_, model2.baseline_increments_))
 relative1 = np.exp(model1.predict_log_partial_hazard(X))

```

```

relative2 = np.exp(model2.predict_log_partial_hazard(X))

survival = np.ones(X.shape[0])
cif1 = np.zeros(X.shape[0])
cif2 = np.zeros(X.shape[0])
output1 = np.zeros((X.shape[0], len(horizons)))
output2 = np.zeros((X.shape[0], len(horizons)))
cursor = 0

for column, horizon in enumerate(horizons):
 while cursor < len(event_times) and event_times[cursor] <= horizon:
 t = event_times[cursor]
 hazard1 = relative1 * increment1.get(t, 0.0)
 hazard2 = relative2 * increment2.get(t, 0.0)
 total = hazard1 + hazard2
 event_probability = 1.0 - np.exp(-total)
 fraction1 = np.divide(
 hazard1,
 total,
 out=np.zeros_like(total),
 where=total > 0,
)
 fraction2 = np.divide(
 hazard2,
 total,
 out=np.zeros_like(total),
 where=total > 0,
)
 cif1 += survival * event_probability * fraction1
 cif2 += survival * event_probability * fraction2
 survival *= np.exp(-total)
 cursor += 1

 output1[:, column] = cif1
 output2[:, column] = cif2
return output1, output2

```

```

class CauseSpecificCoxSystem:
 def __init__(self, l2, engineered=False):
 self.l2 = float(l2)
 self.engineered = bool(engineered)

 def fit(self, frame):
 self.design_ = DesignBuilder(self.engineered)
 X = self.design_.fit_transform(frame)
 time = frame["follow_up_months"].to_numpy()
 event_type = frame["event_type"].to_numpy()
 self.overrun_model_ = CoxPHFromScratch(l2=self.l2).fit(
 X, time, event_type == 1
)
 self.cancel_model_ = CoxPHFromScratch(l2=self.l2).fit(
 X, time, event_type == 2
)
 return self

 def predict(self, frame, horizons=HORIZONS):
 X = self.design_.transform(frame)
 cif1, cif2 = cif_from_cause_specific_models(
 self.overrun_model_, self.cancel_model_, X, horizons
)
 risk_score = self.overrun_model_.predict_log_partial_hazard(X)
 return cif1, cif2, risk_score

def aj_value_at(time, event_type, cause, horizon):
 table = cumulative_incidence(time, event_type, causes=[1, 2])
 if table.empty:
 return 0.0
 return float(step_value(table, horizon, column=f"cif_{cause}"))

```

```

class HistoricalCIF:
 """Feature-free cumulative-incidence reference."""

 def fit(self, frame):
 self.time_ = frame["follow_up_months"].to_numpy()
 self.event_type_ = frame["event_type"].to_numpy()
 return self

 def predict(self, frame, horizons=HORIZONS):
 base1 = np.array([
 aj_value_at(self.time_, self.event_type_, 1, h) for h in horizons
])
 base2 = np.array([
 aj_value_at(self.time_, self.event_type_, 2, h) for h in horizons
])
 cif1 = np.tile(base1, (len(frame), 1))
 cif2 = np.tile(base2, (len(frame), 1))
 return cif1, cif2, np.zeros(len(frame))

def make_candidates():
 candidates = {"historical_cif": lambda: HistoricalCIF()}
 for l2 in [0.1, 1.0, 10.0]:
 # Default-argument capture freezes the current loop value. Without
 # penalty=l2, every lambda would look up the final l2 only when called.
 candidates[f"cox_raw_l2_{l2:g}"] = (
 lambda penalty=l2: CauseSpecificCoxSystem(penalty, engineered=False)
)
 for l2 in [0.1, 1.0]:
 candidates[f"cox_engineered_l2_{l2:g}"] = (
 lambda penalty=l2: CauseSpecificCoxSystem(penalty, engineered=True)
)
 return candidates

def evaluate_prediction(frame, cif1, risk_score, censoring_curve):
 rows = {}
 for column, horizon in enumerate(HORIZONS):
 rows[f"brier_{int(horizon)}"] = ipcw_brier_competing(
 frame["follow_up_months"],
 frame["event_type"],
 cif1[:, column],
 horizon,
 censoring_curve,
)
 rows["mean_brier"] = float(np.mean([
 rows[f"brier_{int(h)}"] for h in HORIZONS
]))
 rows["cause_specific_c"] = harrell_c_index(
 frame["follow_up_months"],
 (frame["event_type"] == 1).astype(int),
 risk_score,
)
 return rows

def expanding_year_scores(development, candidates):
 rows = []
 for year in VALIDATION_YEARS:
 train = development[development["start_year"] < year].copy()
 validation = development[development["start_year"] == year].copy()
 censoring_curve = fit_censoring_curve(
 train["follow_up_months"], train["event_type"]
)

 for name, factory in candidates.items():
 fitted = factory().fit(train)
 cif1, cif2, risk = fitted.predict(validation)

```

```

 assert np.all((cif1 >= 0) & (cif2 >= 0))
 assert np.all(cif1 + cif2 <= 1.0 + 1e-10)
 rows.append({
 "candidate": name,
 "validation_year": year,
 "n": len(validation),
 "overrun_events": int((validation["event_type"] == 1).sum()),
 "cancellations": int((validation["event_type"] == 2).sum()),
 **evaluate_prediction(validation, cif1, risk, censoring_curve),
 })
return pd.DataFrame(rows)

```

```

def calibration_table(frame, probability, horizon=36.0, bins=8):
 work = frame[["follow_up_months", "event_type"]].copy()
 work["probability"] = probability
 # qcut forms quantile bins with roughly equal row counts; duplicate edges
 # are dropped when repeated probability values prevent distinct cutoffs.
 work["bin"] = pd.qcut(probability, q=bins, duplicates="drop")
 rows = []
 for label, part in work.groupby("bin", observed=True):
 rows.append({
 "bin": str(label),
 "n": len(part),
 "overrun_events": int((part["event_type"] == 1).sum()),
 "cancellations": int((part["event_type"] == 2).sum()),
 "mean_prediction": float(part["probability"].mean()),
 "observed_cif": aj_value_at(
 part["follow_up_months"].to_numpy(),
 part["event_type"].to_numpy(),
 cause=1,
 horizon=horizon,
),
 })
 return pd.DataFrame(rows)

```

```

def subgroup_audit(frame, probability, censoring_curve, group_column,
 horizon=36.0):
 work = frame.copy()
 work["probability"] = probability
 rows = []
 for group, part in work.groupby(group_column, observed=True):
 rows.append({
 group_column: group,
 "n": len(part),
 "overrun_events": int((part["event_type"] == 1).sum()),
 "cancellations": int((part["event_type"] == 2).sum()),
 "censored": int((part["event_type"] == 0).sum()),
 "median_follow_up": float(part["follow_up_months"].median()),
 "mean_prediction": float(part["probability"].mean()),
 "observed_cif": aj_value_at(
 part["follow_up_months"].to_numpy(),
 part["event_type"].to_numpy(),
 cause=1,
 horizon=horizon,
),
 "ipcw_brier": ipcw_brier_competing(
 part["follow_up_months"],
 part["event_type"],
 part["probability"],
 horizon,
 censoring_curve,
),
 })
 return pd.DataFrame(rows)

```

```

def paired_brier_bootstrap(frame, probability_a, probability_b,
 censoring_curve, horizon=36.0,

```

```

 repeats=2000, seed=RANDOM_SEED):
 rng = np.random.default_rng(seed)
 estimates = np.empty(repeats)
 for repeat in range(repeats):
 index = rng.integers(0, len(frame), size=len(frame))
 sample = frame.iloc[index]
 score_a = ipcw_brier_competing(
 sample["follow_up_months"], sample["event_type"],
 probability_a[index], horizon, censoring_curve,
)
 score_b = ipcw_brier_competing(
 sample["follow_up_months"], sample["event_type"],
 probability_b[index], horizon, censoring_curve,
)
 estimates[repeat] = score_a - score_b
 return {
 "estimate": float(ipcw_brier_competing(
 frame["follow_up_months"], frame["event_type"],
 probability_a, horizon, censoring_curve,
) - ipcw_brier_competing(
 frame["follow_up_months"], frame["event_type"],
 probability_b, horizon, censoring_curve,
)),
 "interval_95": np.quantile(estimates, [0.025, 0.975]).tolist(),
 }

def main():
 data = make_mhp_survival_data(n=1200, seed=5050)
 development = data[data["start_year"] < TEST_START_YEAR].copy()
 locked_test = data[data["start_year"] >= TEST_START_YEAR].copy()
 assert development["start_year"].max() < locked_test["start_year"].min()

 prohibited = {
 "actual_cost_2025_million_pkr",
 "final_material_bill_million_pkr",
 "major_cost_outrun",
 "month12_progress_gap_sd",
 "senior_review_by_month6",
 "follow_up_months",
 "event_type",
 }
 used = set(RAW_NUMERIC + CATEGORICAL)
 assert prohibited.isdisjoint(used)

 candidates = make_candidates()
 fold_scores = expanding_year_scores(development, candidates)
 summary = (
 fold_scores.groupby("candidate")
 .agg(
 mean_validation_brier=("mean_brier", "mean"),
 sd_validation_brier=("mean_brier", "std"),
 worst_validation_brier=("mean_brier", "max"),
 mean_c=("cause_specific_c", "mean"),
)
 .sort_values(["mean_validation_brier", "worst_validation_brier"])
)
 selected_name = summary.index[0]

 # The lockbox is touched only after the candidate is fixed.
 selected = candidates[selected_name]().fit(development)
 selected_cif1, selected_cif2, selected_risk = selected.predict(locked_test)
 baseline = HistoricalCIF().fit(development)
 baseline_cif1, baseline_cif2, baseline_risk = baseline.predict(locked_test)

 censoring_curve = fit_censoring_curve(
 development["follow_up_months"], development["event_type"]
)
 selected_metrics = evaluate_prediction(
 locked_test, selected_cif1, selected_risk, censoring_curve

```



```

)
baseline_metrics = evaluate_prediction(
 locked_test, baseline_cif1, baseline_risk, censoring_curve
)

assert np.all(selected_cif1 + selected_cif2 <= 1.0 + 1e-10)
horizon_index = int(np.flatnonzero(HORIZONS == 36.0)[0])
selected_36 = selected_cif1[:, horizon_index]
baseline_36 = baseline_cif1[:, horizon_index]

calibration = calibration_table(locked_test, selected_36)
access_audit = subgroup_audit(
 locked_test, selected_36, censoring_curve, "access_mode"
)
district_audit = subgroup_audit(
 locked_test, selected_36, censoring_curve, "district"
)
paired_difference = paired_brier_bootstrap(
 locked_test, selected_36, baseline_36, censoring_curve
)

predictions = locked_test[[
 "project_id", "start_year", "district", "access_mode",
 "follow_up_months", "event_type",
]].copy()
for column, horizon in enumerate(HORIZONS.astype(int)):
 predictions[f"overrun_cif_{horizon}"] = selected_cif1[:, column]
 predictions[f"cancel_cif_{horizon}"] = selected_cif2[:, column]

metadata = {
 "seed": RANDOM_SEED,
 "python": platform.python_version(),
 "numpy": np.__version__,
 "pandas": pd.__version__,
 "scipy": scipy.__version__,
 "development_rows": len(development),
 "locked_test_rows": len(locked_test),
 "selected_candidate": selected_name,
 "primary_metric": "equal-horizon mean IPCW Brier for cause 1",
 "horizons_months": HORIZONS.tolist(),
 "selected_test_metrics": selected_metrics,
 "baseline_test_metrics": baseline_metrics,
 "selected_minus_baseline_brier_36": paired_difference,
 "censoring_reference": "development reverse Kaplan-Meier",
 "tie_method": "Breslow",
}

output = Path("chapter5_results")
output.mkdir(exist_ok=True)
fold_scores.to_csv(output / "forward_validation_folds.csv", index=False)
summary.to_csv(output / "candidate_summary.csv")
predictions.to_csv(output / "locked_predictions.csv", index=False)
calibration.to_csv(output / "calibration_36_months.csv", index=False)
access_audit.to_csv(output / "access_mode_audit.csv", index=False)
district_audit.to_csv(output / "district_audit.csv", index=False)
with open(output / "metadata.json", "w", encoding="utf-8") as file:
 json.dump(metadata, file, indent=2)

print(summary)
print(json.dumps(metadata, indent=2))
print(calibration)

if __name__ == "__main__":
 main()

```

## 38.4 What the benchmark does correctly

The code makes several research choices executable:

- final cost, final material bill, Chapter 4's final binary label, future progress, and future review are prohibited baseline features;
- medians, means, scales, and category levels are learned inside each training fold;
- calendar years move forward rather than being randomly mixed;
- cause-1 and cause-2 hazards are modelled separately;
- predicted cause probabilities remain non-negative and sum with event-free probability to at most one;
- model selection uses development years only;
- the locked period is evaluated once after selection;
- Brier weighting uses a development censoring reference;
- the baseline is a nonparametric cumulative-incidence model;
- calibration uses observed cumulative incidence rather than raw event fractions; and
- outputs include metadata, fold results, individual predictions, calibration, and subgroup audits.

The code is not a universal template. Real data may require conditional censoring weights, delayed entry, Efron ties, cluster resampling, interval-censoring methods, missingness models, external validation, or a different event process.

## 38.5 Combining cause-specific hazards

The benchmark treats each Breslow increment as integrated hazard in a short piecewise-exponential interval. For subject  $i$ , if increments are  $dH_{1i}$  and  $dH_{2i}$ , the probability of any event across that interval is

$$1 - \exp\{-(dH_{1i} + dH_{2i})\}.$$

It allocates that probability between causes in proportion to their increments, then updates event-free survival. This produces coherent profile-specific CIFs. Specialised libraries may use product-integral conventions or finer grids; verify definitions and numerical agreement on controlled examples.

## 38.6 Interpreting the paired bootstrap

The paired bootstrap resamples locked-test projects and recalculates the score difference while keeping model fits and the censoring curve fixed. It quantifies validation-sample uncertainty conditional on the fitted systems and frozen reference curve. It does not include:

- development-sample variability;
- candidate-selection uncertainty;
- censoring-model uncertainty;
- district clustering;
- event-adjudication error; or
- transport to another programme.

To estimate end-to-end procedure uncertainty, bootstrap at the correct sampling unit and repeat preprocessing, forward validation, selection, fitting, and evaluation. That is computationally expensive because it answers a larger question.

## 38.7 Sensitivity analyses that change one assumption at a time

Prespecify a small set:

1. horizons 24 and 36 months separately rather than equally averaged;
2. Efron rather than Breslow ties in a specialised package;

3. censoring conditional on start year and access mode;
4. cancellation included in a composite “project-control event”;
5. exclusion of records with ambiguous warning dates;
6. district-cluster bootstrap;
7. spline terms for terrain and cable length; and
8. a month-12 landmark model using progress information.

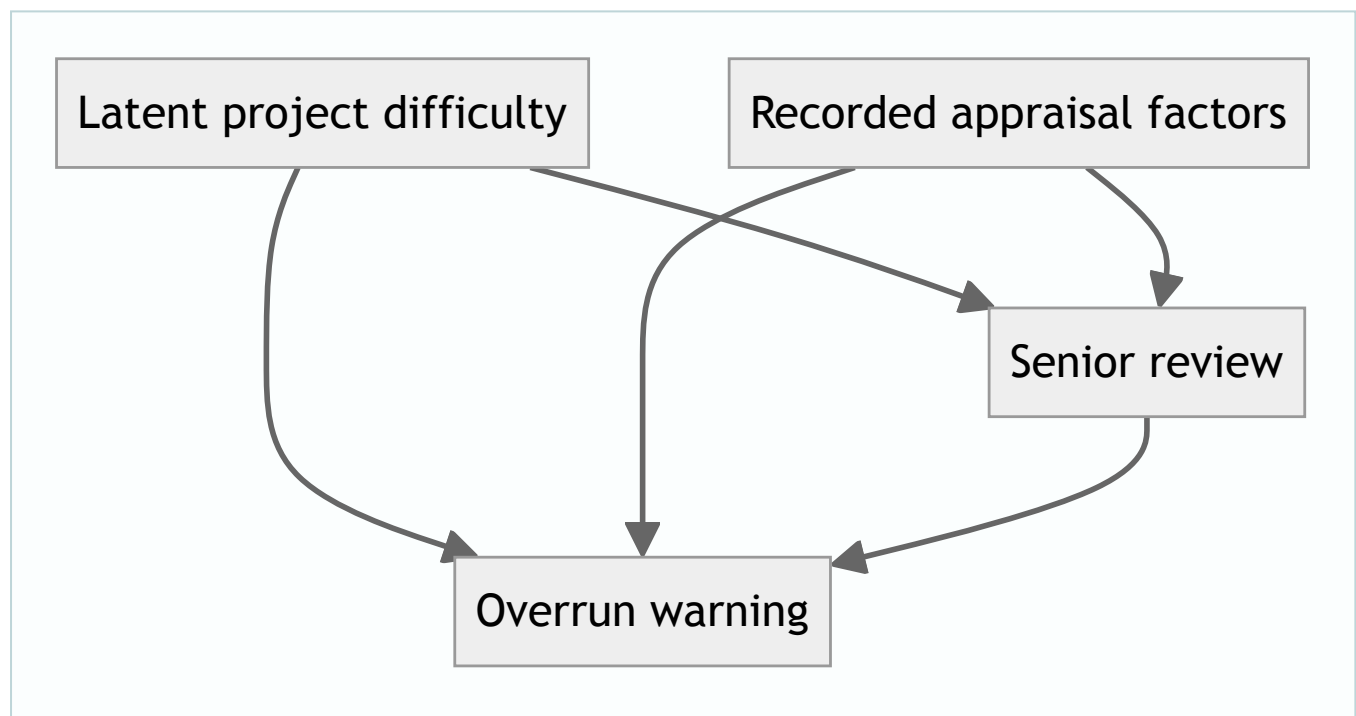
A sensitivity analysis should reveal dependence on a choice. It should not become a menu from which the most favourable answer is selected.

## 38.8 Prediction is not an intervention effect

The fitted model can estimate which projects are more likely to trigger warnings. It cannot answer:

Does senior review by month 6 prevent overrun warnings?

Review is preferentially assigned to concerning projects. Latent risk affects both review and outcome:



The arrow  $R \rightarrow Y$  is the causal effect of interest. The back-door path  $R \leftarrow U \rightarrow Y$  creates confounding by indication. In addition, classifying projects by eventual review can create immortal-time bias, and time-varying project status can both influence review and be influenced by earlier review.

Adjusting a Cox model for every available variable is not automatically sufficient. Mediators, colliders, time-dependent confounders, and measurement timing matter.

## 38.9 Emulate a target trial

A causal study should specify the hypothetical experiment:

| Target-trial component | Example                                                                                  |
|------------------------|------------------------------------------------------------------------------------------|
| Eligibility            | active, warning-free projects at month 6                                                 |
| Treatment strategies   | immediate senior review versus no review during a defined grace period                   |
| Assignment             | hypothetical random assignment; observational emulation adjusts measured confounding     |
| Time zero              | month 6 for both strategies                                                              |
| Follow-up              | month 6 through month 36, cancellation, or database close                                |
| Outcome                | first formal overrun warning; competing cancellation handled by a stated causal estimand |
| Causal contrast        | 30-month cumulative-incidence difference under the two strategies                        |
| Analysis               | method aligned with baseline/time-varying confounding and censoring                      |

Possible observational methods include standardisation, inverse-probability weighting, the parametric g-formula, marginal structural models, and doubly robust estimators. Each requires assumptions such as consistency, positivity, exchangeability conditional on measured history, and correct nuisance estimation.

This is where the original draft's causal diagrams and regression adjustment properly begin. They require their own chapter because the design—not the regression family—creates causal identification.

## 38.10 Research integrity during and after the study

Maintain a deviation log. For every change after registration, record:

- date and reason;
- whether locked outcomes were visible;
- affected analysis components;
- whether the result is primary, sensitivity, or exploratory; and
- who approved the change.

Freeze code with tests and environment information. Preserve raw data read-only; create derived data through scripted transformations. Use a data dictionary and timestamp audit. Report null, unstable, and subgroup findings, not only favourable ones.

If the selected model loses to the historical CIF on the locked period, that is a result. Do not reopen candidate search and continue calling the period “test.” It has become development evidence for a future model, which needs a new prospective test.

## 38.11 A paper-reading workflow for research apprentices

For every methods paper, write a one-page evidence map:

1. **Problem:** What failure of an earlier method motivates the work?
2. **Estimand:** What exact mathematical object is targeted?
3. **Observed data:** What variables and censoring/truncation structure are assumed?
4. **Assumptions:** Which conditions identify or consistently estimate the target?
5. **Estimator:** What objective, recursion, or algorithm is proposed?
6. **Uncertainty:** How are variance and intervals obtained?
7. **Evidence:** Theory, simulation, empirical example, or all three?
8. **Failure modes:** What happens when assumptions fail?
9. **Software gap:** What choices does the implementation add?

10. **Replication:** What smallest example would falsify your understanding?

Do not begin with citation count or an online summary. Read the abstract, estimand, core derivation, simulation design, limitations, and discussion. Reproduce a small result before applying the method to a consequential dataset.

## 38.12 Day 38 build, break, and reflect

### Build

1. Save all chapter data generators and estimator functions.
2. Run the locked benchmark once.
3. Inspect invariants, candidate summary, calibration, subgroup audits, and metadata.
4. Write a 250-word results section that states the estimand, period, events, censoring, selected procedure, primary score, uncertainty, and bounded claim.
5. Write a target-trial table for the separate review-effect question.

### Break

1. Choose horizons after seeing test performance.
2. Add month-12 progress to the appraisal model.
3. Tune on 2023–2025 and still call it locked.
4. Interpret the review coefficient causally.
5. Hide a worse-than-baseline result.

### Reflect

What would you need to change—in design, data, analysis, and claim—to move from “predicts time to warning” to “review reduces warnings”?

## Day 38 exit check

You have completed Chapter 5 when you can:

1. reproduce the benchmark from a clean environment;
2. explain every risk-set, censoring, competing-risk, and validation choice;
3. distinguish cause-specific association, cumulative-incidence prediction, and causal effect; and
4. state what new data could falsify the study's deployment claim.

---

# Chapter 5 Synthesis

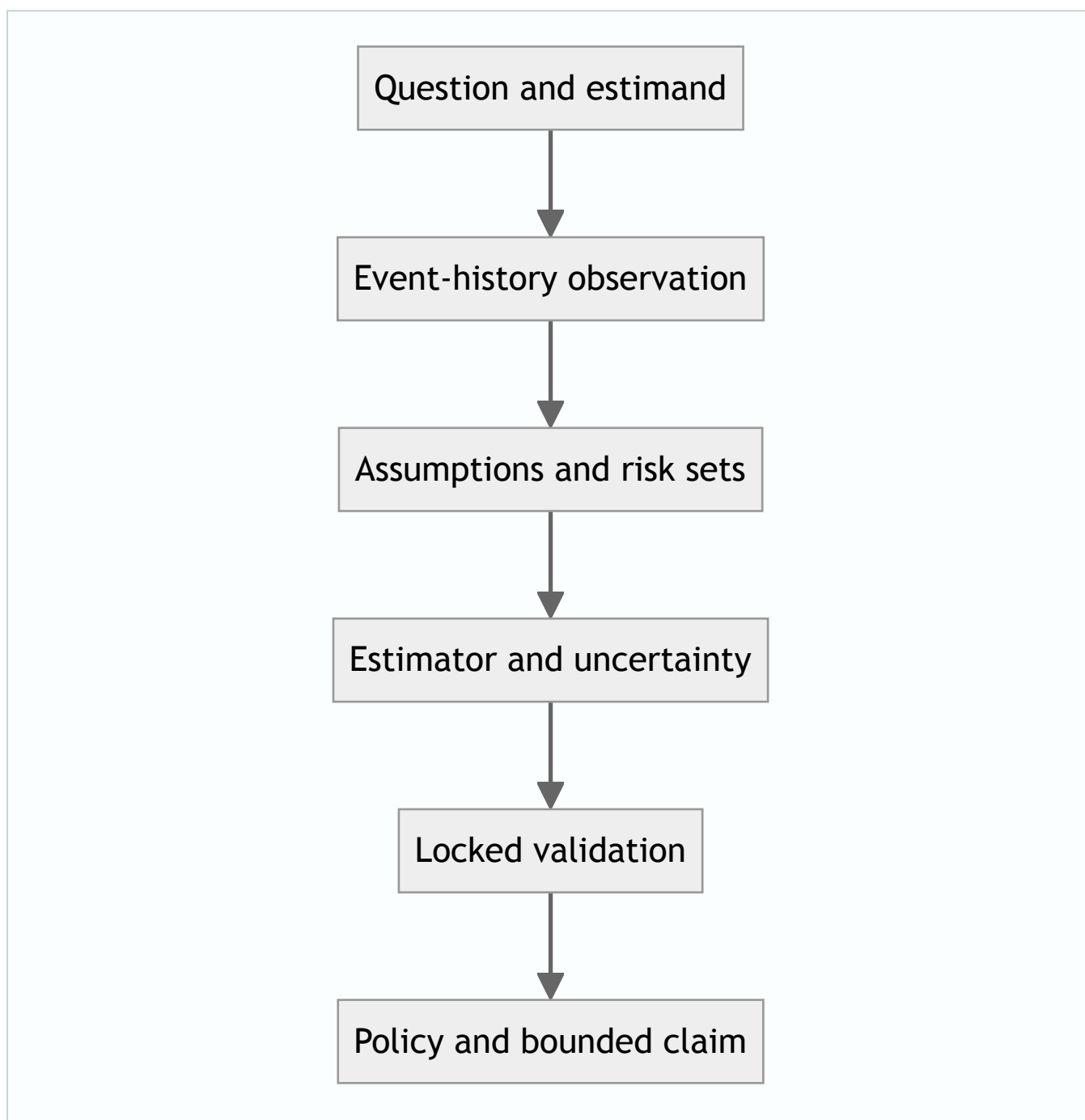
---

## One event history, several valid questions

| Question                                             | Target                     | Typical method             | Common misstatement               |
|------------------------------------------------------|----------------------------|----------------------------|-----------------------------------|
| How long until any event?                            | all-cause $S(t)$ or $H(t)$ | Kaplan–Meier, Nelson–Aalen | censored means event-free forever |
| How does a covariate relate to current cause-1 rate? | $\lambda_1(t \mid x)$      | cause-specific Cox         | hazard ratio equals risk ratio    |

| Question                                        | Target                           | Typical method                                 | Common misstatement                           |
|-------------------------------------------------|----------------------------------|------------------------------------------------|-----------------------------------------------|
| What is real-world cause-1 probability by $t$ ? | $F_1(t   \mathbf{x})$            | Aalen–Johansen; combined cause-specific models | <b>1</b> —KM while censoring competing events |
| How does information at month 12 update risk?   | conditional landmark CIF         | landmark model                                 | use month-12 value at baseline                |
| Can nonlinear structure improve prediction?     | horizon-specific survival/CIF    | spline Cox, forest, boosting                   | ordinary regression on observed time          |
| Does review prevent the event?                  | causal contrast under strategies | target-trial emulation or experiment           | adjusted prediction coefficient is causal     |

## The complete research chain



A failure upstream cannot be repaired downstream. A perfect optimiser cannot correct a wrong time origin. A flexible forest cannot identify informative censoring. A strong C-index cannot establish calibration. An adjusted Cox model cannot turn treatment selection into random assignment.

## A compact analysis checklist

Before fitting:

- ☐ Unit, origin, entry, event, competing event, censoring, and horizon are frozen.
- ☐ Every feature has a prediction-time timestamp.
- ☐ The causal or predictive aim is explicit.
- ☐ Development and lockbox boundaries match deployment.
- ☐ Candidate families, metrics, and censoring method are registered.

Before interpretation:

- ☐ Event counts and numbers at risk support the requested horizon.
- ☐ Ties, delayed entry, clustering, and missingness are handled explicitly.
- ☐ Proportionality and functional form have been examined.
- ☐ Absolute survival or CIF accompanies relative scores.
- ☐ Competing-risk estimands are named.

Before deployment:

- ☐ Temporal and relevant geographic evidence are reported.
- ☐ Calibration, discrimination, and decision value are separated.
- ☐ Groups include raw counts, censoring, and event types.
- ☐ Threshold, capacity, action, override, and monitoring are specified.
- ☐ Model updates trigger a new validation plan.

---

## Chapter 5 Exercises

---

### Foundation exercises

#### Exercise 5.1 — Decode incomplete observations

For eight subjects, create latent event and censoring times, then calculate observed time and status. For each censored row, write the inequality you know. Change the database-close date and identify which records change.

#### Exercise 5.2 — Risk sets with delayed entry

Given entry times  $[0, 0, 2, 4, 4]$ , exit times  $[3, 5, 6, 6, 9]$ , and event indicators  $[1, 0, 1, 1, 0]$ :

1. list distinct event times;
2. construct every risk set;
3. calculate the delayed-entry Kaplan–Meier curve; and
4. recalculate incorrectly using entry zero, then explain the direction of error.

### Exercise 5.3 — Hazard is not probability

For hazards 0.02, 0.08, and 1.20 per month, approximate event probability over one day, one month, and six months. Identify where  $h\Delta t$  is a poor approximation and use  $1 - e^{-h\Delta t}$  under a constant hazard.

### Exercise 5.4 — Product-limit calculation

Construct a ten-project event table with a tied event and tied censoring. Calculate Kaplan–Meier, Nelson–Aalen, Greenwood standard error, median survival if reached, and 24-month RMST.

## Modelling exercises

### Exercise 5.5 — Partial likelihood by hand

Use three subjects with one covariate and event order A, B, then censor C. Write each partial-likelihood factor, log likelihood, score at  $\beta = 0$ , and the direction in which the likelihood initially increases.

### Exercise 5.6 — Tie methods

Create a dataset with increasing proportions of tied events. Compare Breslow, Efron, and exact estimates in a survival library. Explain when differences matter and document the library default.

### Exercise 5.7 — Proportional-hazards failure

Simulate a binary covariate whose effect is harmful before month 12 and protective afterward. Compare:

1. a constant-coefficient Cox model;
2. a prespecified time interaction;
3. 24-month RMST difference; and
4. horizon-specific CIF differences.

Explain why the single hazard ratio is a poor summary.

### Exercise 5.8 — AFT versus Cox

Simulate Weibull and log-normal event times under censoring. Fit Cox, Weibull AFT, and log-normal AFT models. Compare coefficient interpretation, calibration within support, and tail extrapolation.

## Competing-risk and dynamic exercises

### Exercise 5.9 — Kaplan–Meier overstatement

Simulate increasing cancellation rates while holding the overrun cause-specific hazard fixed. Plot  $1 - \text{KM}$  and Aalen–Johansen overrun incidence. Explain the growing gap.

### Exercise 5.10 — Cause-specific prediction system

Fit a Cox model for each cause. Combine baseline increments into subject-specific CIFs. Verify non-negativity, monotonicity over time, and  $F_1(t) + F_2(t) + S(t) = 1$  within numerical tolerance.



## Exercise 5.11 — Landmark prediction

Create month-6, month-12, and month-18 cohorts. At each landmark:

1. define eligibility;
2. name the information history;
3. predict the next 18-month cause-1 CIF;
4. evaluate IPCW Brier; and
5. explain why scores describe different survivor populations.

## Exercise 5.12 — Immortal-time trap

Compare projects ever reviewed by month 6 with never-reviewed projects from time zero. Then realign time zero at month 6 among eligible survivors. Draw both timelines and explain why the estimates differ even before confounding adjustment.

## Research exercises

### Exercise 5.13 — Censoring sensitivity

Create a censoring mechanism related to an unrecorded risk factor. Compare marginal IPCW, measured-covariate IPCW, and a sensitivity analysis that varies the hidden dependence. State which bias cannot be diagnosed from observed data alone.

### Exercise 5.14 — External validation report

Treat one district as an external site. Report feature shift, missingness, event types, censoring, follow-up support, calibration, Brier score, concordance, and threshold consequences. Decide whether recalibration, model revision, or rejection is warranted.

### Exercise 5.15 — Paper replication

Choose Kaplan–Meier (1958), Cox (1972), Fine–Gray (1999), or Ishwaran et al. (2008). Produce:

- a one-page evidence map;
- a minimal mathematical derivation;
- a simulation reproducing one qualitative claim;
- a failure-case simulation; and
- a 300-word comparison between the paper and one software implementation.

### Exercise 5.16 — Registered mini-study

Before fitting, register a study on machine failure, customer churn, loan default, or patient follow-up. Include all fields from Section 38.1. Lock a temporal or grouped test set, run once, preserve deviations, and write a results section that does not exceed the evidence.

---

# Capstone Rubric

| Dimension       | Beginning                           | Competent                              | Research-ready                                                       |
|-----------------|-------------------------------------|----------------------------------------|----------------------------------------------------------------------|
| Outcome process | binary label with unclear follow-up | event, origin, censoring stated        | event types, entry, measurement, and estimand audited                |
| Mathematics     | reports software output             | explains survival, hazard, KM, and Cox | derives risk-set estimators and links hazards to CIFs                |
| Leakage control | random split with future fields     | timestamped baseline features          | dynamic features aligned to landmarks/start–stop intervals           |
| Competing risks | ignored or censored silently        | CIF reported                           | cause-specific and absolute targets separated and justified          |
| Diagnostics     | one global test                     | PH and functional form inspected       | deviations modelled with prespecified sensitivity analyses           |
| Validation      | apparent or random holdout only     | temporal/grouped validation            | locked, external/prospective destinations with procedure uncertainty |
| Metrics         | C-index only                        | Brier, calibration, and C-index        | censoring-aware metrics aligned to decisions and horizons            |
| Flexibility     | complex model assumed better        | strong Cox reference                   | equal-budget comparison with event-aware tuning and transport checks |
| Uncertainty     | no intervals                        | standard or bootstrap intervals        | uncertainty target and omitted sources explicitly bounded            |
| Causal language | association called effect           | prediction and effect separated        | target trial and identification assumptions specified                |
| Reproducibility | notebook state                      | seeded script and outputs              | registration, tests, environment, lockbox, and deviation log         |

A research-ready submission can conclude that no candidate transports or that the data cannot identify the desired horizon. Refusing an unsupported claim is a success.

## Glossary

**Aalen–Johansen estimator** — Product-integral estimator of transition probabilities; in simple competing risks, it estimates cause-specific cumulative incidence.

**Accelerated failure time model** — Regression model in which covariates multiply the event-time scale, commonly expressed through a linear model for log time.

**Administrative censoring** — Follow-up ending at a planned study or database-close date.

**At risk** — Eligible, under observation, and not yet in an absorbing event state immediately before a time.

**Baseline hazard** — Time-varying hazard for a reference covariate profile in a proportional-hazards model.

**Calibration at horizon  $t$**  — Agreement between predicted and observed event probability by  $t$  in the target population.

**Cause-specific hazard** — Instantaneous rate of one cause among subjects currently free of every event cause.

**Censoring** — Incomplete observation of an event time; right censoring establishes that the event time exceeds the censoring time.

**Concordance** — Probability-like ranking measure comparing risk scores for usable subject pairs.

**Competing event** — Event that prevents or fundamentally changes occurrence of the event of interest.

**Cumulative hazard** — Integral of hazard over time; for one event type without competing risks,  $H(t) = -\log S(t)$ .

**Cumulative incidence function** — Probability of experiencing a particular event cause by a time in the presence of competing causes.

**Delayed entry** — See left truncation.

**Dynamic prediction** — Prediction updated during follow-up using history available at an update time.

**Event-free survival** — Probability that no event in the defined event set has occurred by a time.

**Fine–Gray model** — Regression model for a competing cause's subdistribution hazard, linked to its cumulative incidence.

**Frailty** — Latent random effect representing unobserved heterogeneity or shared dependence in a survival model.

**Greenwood formula** — Approximate variance estimator for Kaplan–Meier survival.

**Hazard** — Instantaneous event rate conditional on remaining at risk; not a probability.

**Hazard ratio** — Ratio of two hazards at a time; constant over time under proportional hazards.

**Independent censoring** — Assumption that censoring adds no event-time information conditional on the variables used to justify the analysis.

**Interval censoring** — Event known to occur inside an interval but not at an exact time.

**IPCW** — Inverse probability of censoring weighting, which reweights observed outcome information under a censoring model.

**Kaplan–Meier estimator** — Product-limit estimator of survival using event-time risk sets.

**Landmark analysis** — Dynamic prediction analysis restricted to subjects at risk at a chosen update time, using information available by then.

**Left censoring** — Event known to have occurred before an observed bound.

**Left truncation** — Sampling only subjects whose event time exceeds their entry time; requires delayed risk-set entry.

**Log-rank test** — Risk-set comparison of observed and expected events between groups, most sensitive to proportional separation.

**Nelson–Aalen estimator** — Sum of event count divided by risk-set size, estimating cumulative hazard.

**Partial likelihood** — Cox likelihood component estimating relative-hazard coefficients without parameterising the baseline hazard.

**Proportional hazards** — Assumption that covariate hazard ratios are constant over analysis time.

**Restricted mean survival time** — Area under a survival curve through a finite horizon; expected event-free time restricted to that horizon.

**Risk set** — Subjects under observation and event-free immediately before an event time, after respecting entry.

**Schoenfeld residual** — Event-subject covariate minus its risk-set weighted expectation; used to assess time stability of effects.

**Subdistribution hazard** — Hazard-like quantity whose risk set retains competing-event subjects under a defined construction and whose regression is linked to a CIF.

**Time-varying covariate** — Predictor whose value can change during follow-up and must be aligned with the time it was known.

---

## Where Chapter 6 Begins

---

Chapter 5 ends at the boundary between prognosis and intervention. Chapter 6 should develop causal inference as a design problem:

- counterfactual outcomes and causal estimands;
- consistency, exchangeability, positivity, and interference;
- causal diagrams, confounders, mediators, colliders, and selection;
- randomised experiments and noncompliance;
- regression standardisation, matching, weighting, and doubly robust estimation;
- target-trial emulation with time-varying treatment;
- marginal structural models and the g-formula;
- instrumental variables, regression discontinuity, and difference-in-differences;
- sensitivity to unmeasured confounding;
- heterogeneous treatment effects without data dredging; and
- registered causal studies whose claims follow from identification assumptions.

It should revisit the question “does senior review prevent overruns?” with a design capable of answering it.

## Final Reflection

---

Time-to-event research is the construction of a chain:

clock → risk set → event process → probability → decision → claim.

Every arrow contains assumptions. Censoring asks why observation ended. Competing risks ask what else could happen. Dynamic prediction asks when information became available. Validation asks where the model is expected to travel. Causal inference asks what would change under an intervention.

A research apprentice becomes research-ready not by memorising more estimators, but by making those assumptions visible enough to challenge, test where possible, and revise.

---

# References and Further Reading

---

## Research papers discussed

- Aalen, O. O., & Johansen, S. (1978). An Empirical Transition Matrix for Non-homogeneous Markov Chains Based on Censored Observations. *Scandinavian Journal of Statistics*, 5, 141–150.
- Andersen, P. K., & Gill, R. D. (1982). Cox's Regression Model for Counting Processes: A Large Sample Study. *The Annals of Statistics*, 10(4), 1100–1120.
- Candès, E., Lei, L., & Ren, Z. (2023). Conformalized Survival Analysis. *Journal of the Royal Statistical Society: Series B*, 85(1), 24–45.
- Cox, D. R. (1972). Regression Models and Life-Tables. *Journal of the Royal Statistical Society: Series B*, 34(2), 187–220.
- Fine, J. P., & Gray, R. J. (1999). A Proportional Hazards Model for the Subdistribution of a Competing Risk. *Journal of the American Statistical Association*, 94(446), 496–509.
- Graf, E., Schmoor, C., Sauerbrei, W., & Schumacher, M. (1999). Assessment and Comparison of Prognostic Classification Schemes for Survival Data. *Statistics in Medicine*, 18(17–18), 2529–2545.
- Ishwaran, H., Kogalur, U. B., Blackstone, E. H., & Lauer, M. S. (2008). Random Survival Forests. *The Annals of Applied Statistics*, 2(3), 841–860.
- Kaplan, E. L., & Meier, P. (1958). Nonparametric Estimation from Incomplete Observations. *Journal of the American Statistical Association*, 53(282), 457–481.
- Schoenfeld, D. (1982). Partial Residuals for the Proportional Hazards Regression Model. *Biometrika*, 69(1), 239–241.
- Uno, H., Cai, T., Pencina, M. J., D'Agostino, R. B., & Wei, L. J. (2011). On the C-Statistics for Evaluating Overall Adequacy of Risk Prediction Procedures with Censored Survival Data. *Statistics in Medicine*, 30(10), 1105–1117.
- van Houwelingen, H. C. (2007). Dynamic Prediction by Landmarking in Event History Analysis. *Scandinavian Journal of Statistics*, 34(1), 70–85.

## Recommended textbooks

- Andersen, P. K., Borgan, Ø., Gill, R. D., & Keiding, N. *Statistical Models Based on Counting Processes*.
- Harrell, F. E. *Regression Modeling Strategies*.
- Hernán, M. A., & Robins, J. M. *Causal Inference: What If*.
- Hosmer, D. W., Lemeshow, S., & May, S. *Applied Survival Analysis*.
- Kalbfleisch, J. D., & Prentice, R. L. *The Statistical Analysis of Failure Time Data*.
- Klein, J. P., & Moeschberger, M. L. *Survival Analysis: Techniques for Censored and Truncated Data*.
- Steyerberg, E. W. *Clinical Prediction Models*.
- van Houwelingen, H. C., & Putter, H. *Dynamic Prediction in Clinical Survival Analysis*.

## Software documentation to consult during implementation

- lifelines: Survival Regression and Time-Varying Survival Regression.
- scikit-survival: User Guide, including evaluation metrics and competing risks.
- R `survival`: Package documentation.
- R `riskRegression`: Package documentation.

Use primary papers for estimands and derivations, specialised texts for connected theory, and current software documentation for interfaces and defaults.

# Chapter 5 Companion Guide: From Event Labels to Event Histories

## A beginner-first guide to survival analysis, censoring, competing risks, dynamic prediction, and registered evidence

This guide accompanies `chapter5.md`. It assumes you are still learning Python, mathematics, and statistics. It explains the vocabulary and symbols before using them, develops the main estimators from small risk sets, and repeatedly separates prediction from causal claims.

Chapter 4 asked whether an event occurs inside a fixed window. Chapter 5 preserves more information:

```
when observation begins
 -> how long a subject remains event-free
 -> which event happens first
 -> why observation ends
 -> what can be predicted at a stated horizon
```

The central fact is:

If a project has no observed warning when follow-up ends, we often know “not yet,” not “never.”

Survival analysis is also called time-to-event or event-history analysis. It applies to project warnings, machine failures, customer churn, loan defaults, hospital outcomes, and many other processes—not only death.

## How to use this guide

For each of Days 30–38:

1. Draw the timeline before reading the formula.
2. List who is genuinely at risk immediately before each event.
3. Work the smallest numerical example by hand.
4. Type the short code and alter one record.
5. Run the main chapter's full laboratory.
6. Deliberately create one listed failure.
7. State the estimand, assumptions, and allowed conclusion in ordinary language.

Keep these objects separate:

- **event process:** what can happen and in what order;
- **observation process:** when and why records become unavailable;
- **estimand:** the exact survival, hazard, incidence, or causal quantity wanted;
- **estimator:** the calculation used under assumptions;
- **prediction:** a future probability or score at a stated time and horizon;

- **decision:** an action based on consequences and capacity; and
  - **claim:** what the design actually supports.
- 

## 0. Foundations for a complete beginner

### 0.1 The running project question

One row begins as one fictional microhydro project approved at technical appraisal. The event of interest is the first formal warning that projected final cost exceeds 115% of the approved budget.

- Time zero is appraisal approval.
- A warning is event type 1.
- Cancellation before warning is event type 2.
- Database closure or loss to follow-up is censoring.
- Predictions concern 12, 24, or 36 months.
- Month-12 progress can only enter a prediction made at month 12 or later.

A high predicted incidence may trigger review, not automatic cancellation. The data are simulated and say nothing factual about real projects or districts.

### 0.2 Python language used in this chapter

```
import numpy as np
import pandas as pd

time = np.array([2.0, 3.0, 5.0])
event = np.array([1, 0, 1])
```

- an array stores several values;
- `time[i]` and `event[i]` refer to the same subject;
- `event == 1` produces a Boolean mask;
- `time[event == 1]` keeps event times;
- `np.sum(mask)` counts `True` values;
- `np.unique(...)` finds distinct values;
- `np.sort(...)` orders them;
- `&` means elementwise “and”; and
- `|` means elementwise “or.”

Pandas provides labelled tables:

```
history = pd.DataFrame({
 "project": ["A", "B", "C"],
 "time": [2.0, 3.0, 5.0],
 "event": [1, 0, 1],
})
```

Parentheses matter in masks:

```
at_risk = (history["entry"] <= t) & (history["exit"] >= t)
```

## 0.3 Mathematical vocabulary

- $T$  is a latent event time: the time that would be observed with sufficient follow-up.
- $C$  is censoring time: when observation stops without the event.
- $Y = \min(T, C)$  is observed follow-up time.
- $\Delta$  is 1 if the event was observed and 0 if censored.
- $t$  is a particular time;  $\tau$  is often a chosen prediction horizon.
- $P(A)$  is the probability of  $A$ .
- $P(A | B)$  is probability of  $A$  among cases satisfying  $B$ .
- $f(t)$  is a probability density.
- $\mathbb{E}$  is an average over a distribution.
- $\sum$  means add;  $\prod$  means multiply.
- $\int$  accumulates infinitely small contributions over a continuous range.
- $\exp(z) = e^z$  and  $\log$  is its inverse.
- $x^\top \beta$  means multiply matching features and coefficients, then add.
- $u-$  means the instant just before time  $u$ .

## 0.4 A timeline contains more information than a binary label

Suppose A has a warning at month 18 and B is event-free when the database closes at month 18:

```
A: approval ----- warning
B: approval ----- database closes
 month 18
```

For A,  $T_A = 18$ . For B,  $T_B > 18$ . Calling B a permanent non-event changes an inequality into a claim about infinity. Calling month 18 its event invents an event. Dropping B discards 18 known event-free months.

## 0.5 Installing software

Core code uses:

```
python -m pip install numpy pandas scipy matplotlib scikit-learn
```

Optional applied laboratories can use:

```
python -m pip install lifelines scikit-survival
```

Record Python and package versions. Survival packages can differ in tie handling, variance estimates, censoring conventions, and prediction outputs. The same function name does not guarantee the same estimand.

## 0.6 Understanding chapter5\_data.py

The generator creates three hidden times per project:

- time to overrun warning;
- time to cancellation; and
- time to censoring.

The first one determines the observable record:

```
all_times = np.column_stack([overrun_time, cancel_time, censor_time])
first_type = np.argmin(all_times, axis=1)
```



```
observed_time = all_times[np.arange(n), first_type]
```

`column_stack` creates a three-column matrix. `argmin(..., axis=1)` returns the column holding the smallest time for each row.

Event type is coded:

```
0 = censored
1 = overrun warning
2 = cancellation
```

Later cohorts receive less possible administrative follow-up because the database closes at one calendar date. A separate exponential time simulates loss to follow-up.

`legacy_registry_entry_months` supports delayed-entry exercises. The primary inception cohort starts everyone at approval, so entry is zero. `month12_progress_gap_sd` is future information for the appraisal model but legitimate among observable, event-free month-12 survivors. `senior_review_by_month6` is deliberately confounded and exists to demonstrate why an observational treatment coefficient is not a causal effect.

The hidden potential times are not returned as usable columns. Real construction would need dated warnings, adjudication of back-dated records, stable event definitions, cancellation reasons, and a verified close date.

## 0.7 Baseline, dynamic, and causal questions

- **Baseline prediction:** using appraisal information, what is cause-1 incidence by month 36?
- **Dynamic prediction:** among projects still observable and event-free at month 12, what is incidence during the next 24 months using history through month 12?
- **Causal question:** what would incidence have been under review versus no review?

They have different populations, time zeros, features, assumptions, and designs. A better month-12 model cannot be used at appraisal. A prognostic model cannot identify the effect of review merely by adding a review column.

---

# Day 30 — The event-history contract

---

## 30.1 Observed time and status

We observe:

$$Y_i = \min(T_i, C_i), \quad \Delta_i = \mathbb{1}(T_i \leq C_i).$$

If  $\Delta = 1$ , then  $T = Y$ . If  $\Delta = 0$ , then  $T > Y$ . A censored row contributes genuine partial information.

With competing events, store event type beside time rather than one binary indicator. Cancellation is known outcome information, while database closure leaves the next event unknown.

## 30.2 The seven-part protocol

Freeze before association analysis:

1. **Unit:** one eligible project.

2. **Origin:** approval at appraisal.
3. **Entry:** when it begins contributing to risk sets.
4. **Event:** first formal warning under a stable rule.
5. **Competing events:** cancellation or other events that prevent warning.
6. **Censoring:** why observation ends without a recorded event.
7. **Horizons:** 12, 24, and 36 months.

Analysis time and calendar time differ. Two projects can both be at follow-up month 12 in different years; calendar change can still affect transport.

### 30.3 Four incomplete-observation mechanisms

**Right censoring:** event, if it happens, is after last observation. We know  $T > C$ .

**Left censoring:** event happened before a bound, but exact time is unknown. We know  $T \leq L$ .

**Interval censoring:** event happened between inspections:  $L < T \leq R$ .

**Left truncation/delayed entry:** a subject appears only after surviving event-free to entry time. Earlier failures are absent from the sample.

Left censoring concerns an unknown time for an observed subject. Left truncation changes which subjects are observed. Replacing interval-censored times by endpoints or midpoints is an approximation that needs sensitivity analysis.

### 30.4 Risk sets

Immediately before time  $t$ :

$$R(t) = \{i : L_i \leq t \leq Y_i\}.$$

A subject must have entered, remain observed, and remain event-free. Under a start–stop convention this may be written  $\text{start} < t \leq \text{stop}$ ; state the convention.

For A (0,3,event), B (0,5,censor), C (2,6,event), and D (4,7,censor):

- at month 3: A, B, C are at risk;
- at month 6: C and D are at risk.

D has not entered at month 3. A failed and B was censored before month 6.

### 30.5 Independent censoring

A common assumption is:

$$T \perp C \mid X.$$

Conditional on recorded  $X$ , censoring provides no extra information about event time. This can fail when deteriorating projects disappear for unrecorded reasons.

Observed data cannot prove the absence of hidden informative censoring. Document reasons, compare censoring across periods and groups, include justified predictors in censoring models, and run sensitivity analyses.

Administrative censoring can differ by approval year without automatically being informative, provided calendar time and the mechanism are handled appropriately.

## 30.6 Censoring versus competing risk

Censoring means the future event is unknown after observation ends. A competing event means a known event has changed or prevented the future event of interest. Treating cancellation as censoring can be useful for estimating a cause-specific hazard, but it does not make cancellation missing information for absolute incidence.

## 30.7 Audit before estimation

Check:

- unique project IDs;
- origin, entry, exit, and event ordering;
- positive follow-up and entry no later than exit;
- impossible or duplicate event types;
- date precision and ties;
- censoring reasons by year and group;
- event counts and numbers at risk at every intended horizon;
- missing features and measurements recorded after prediction time; and
- whether event definitions changed over calendar time.

Do not begin with a survival curve. Begin with whether the timeline is coherent.

---

# Day 31 — Survival, hazard, and Kaplan–Meier

---

## 31.1 Distribution, density, survival, and hazard

For event time  $T$ :

$$F(t) = P(T \leq t)$$

is cumulative event probability, and:

$$S(t) = P(T > t) = 1 - F(t)$$

is survival or event-free probability.

For continuous time, density  $f(t)$  describes probability concentration near  $t$ . Hazard is:

$$f(t) = \frac{dF(t)}{dt} = -\frac{dS(t)}{dt}, \quad h(t) = \frac{f(t)}{S(t)}.$$

The first identity says density is the rate at which cumulative event probability rises, equivalently the rate at which survival falls. Dividing by  $S(t)$  conditions the density on having survived to  $t$ .

$$h(t) = \lim_{\Delta t \downarrow 0} \frac{P(t \leq T < t + \Delta t \mid T \geq t)}{\Delta t}.$$

Hazard is an instantaneous rate among current survivors. It can exceed 1 because it is per time unit, not a probability.

For a short interval, event probability is approximately  $h(t)\Delta t$ . Under constant hazard  $h$ , exact interval probability is  $1 - e^{-h\Delta t}$ .

## 31.2 Cumulative hazard

$$H(t) = \int_0^t h(u) du.$$

For a single event process:

$$S(t) = e^{-H(t)}, \quad H(t) = -\log S(t).$$

Derivation intuition: multiply survival across tiny intervals, take logs to turn the product into a sum, then let interval width shrink.

If hazard is constant  $\lambda$ , then  $H(t) = \lambda t$  and  $S(t) = e^{-\lambda t}$ . The median solves  $S(t) = 0.5$ , giving  $\log 2/\lambda$ .

## 31.3 Kaplan–Meier product limit

At event time  $t_j$ , let  $n_j$  be at risk just before and  $d_j$  the number of events. Conditional survival through that time is:

$$1 - \frac{d_j}{n_j}.$$

Multiply these conditional survivals:

$$\hat{S}(t) = \prod_{t_j \leq t} \left(1 - \frac{d_j}{n_j}\right).$$

Events create downward steps. Censoring changes later risk-set denominators but does not create a survival step.

For records (2, event), (3, censor), (5, event), (5, event), (7, censor):

- at 2:  $n = 5, d = 1$ , so survival is  $4/5$ ;
- censoring at 3 leaves three at risk before 5;
- at 5:  $n = 3, d = 2$ , so survival becomes  $(4/5)(1/3) = 4/15$ .

Tied event/censor timestamps need an explicit precision convention. File row order must not decide risk sets.

## 31.4 Greenwood uncertainty

$$\widehat{Var}\{\hat{S}(t)\} \approx \hat{S}(t)^2 \sum_{t_j \leq t} \frac{d_j}{n_j(n_j - d_j)}.$$

Applied software often transforms the curve before forming intervals so limits remain between 0 and 1. Greenwood uncertainty does not include event-definition error, informative censoring, clustering, or selection.

Late intervals widen when few subjects remain. Always show numbers at risk; a smooth-looking long tail with three projects is weak evidence.

## 31.5 Nelson–Aalen cumulative hazard

$$\hat{H}(t) = \sum_{t_j \leq t} \frac{d_j}{n_j}.$$

$e^{-\hat{H}(t)}$  approximates survival but is not exactly Kaplan–Meier because  $-\log(1 - d/n)$  is only approximately  $d/n$  for small event fractions.

Nelson–Aalen is useful for hazard-scale diagnostics and resembles the baseline cumulative-hazard estimator used after Cox fitting.

## 31.6 Log-rank comparison

At pooled event time  $j$ , if group 1 has  $n_{1j}$  of  $n_j$  at-risk subjects and  $d_j$  events, its expected count under equal hazards is:

$$e_{1j} = d_j \frac{n_{1j}}{n_j}.$$

The test accumulates observed minus expected events and scales by a hypergeometric variance. It is most sensitive to roughly proportional separation.

Crossing curves can differ at important horizons yet have a small log-rank statistic. A  $p$ -value is not an effect size, probability forecast, or causal result.

## 31.7 Cause-specific warning curve

If cancellation is treated as censoring, the resulting Kaplan–Meier curve describes a cause-specific/net-risk construction. Label it clearly. One minus that curve is not real-world warning incidence in the presence of cancellation; Day 35 supplies Aalen–Johansen.

# Day 32 — Cox regression from partial likelihood

## 32.1 The proportional-hazards model

$$h(t | x) = h_0(t)e^{x^\top \beta}.$$

$h_0(t)$  is an unspecified baseline hazard. The exponential covariate score multiplies it. For profiles  $a, b$ :

$$\frac{h(t | x_a)}{h(t | x_b)} = e^{(x_a - x_b)^\top \beta}.$$

The baseline cancels, so the ratio is constant over time under proportional hazards. The hazards themselves may rise or fall.

## 32.2 Hazard-ratio interpretation

If  $\beta = 0.30$ , then  $e^{0.30} \approx 1.35$ . Among projects still event-free and observed, the fitted instantaneous rate is 35% higher for a one-unit difference, conditional on modeled covariates.

It is not a 35-point probability increase, risk ratio, time ratio, eventual-event ratio, individual guarantee, or causal effect. Accompany hazard ratios with absolute survival or cumulative incidence at relevant horizons.

## 32.3 Partial likelihood

If subject  $i(j)$  is the sole event at  $t_j$ , conditional probability that this risk-set member is the one who fails is:

$$\frac{e^{x_{i(j)}^\top \beta}}{\sum_{k \in R(t_j)} e^{x_k^\top \beta}}.$$

$h_0(t_j)$  cancels from numerator and denominator. Multiply across event times to obtain partial likelihood. It is “partial” because it estimates relative-hazard coefficients without simultaneously choosing a baseline-hazard formula.

## 32.4 Log partial likelihood, score, and information

Taking logs converts products to sums:

$$\ell_p(\beta) = \sum_j \left[ x_{i(j)}^\top \beta - \log \sum_{k \in R(t_j)} e^{x_k^\top \beta} \right].$$

The score is:

$$U(\beta) = \sum_j [x_{i(j)} - \bar{x}(\beta, t_j)],$$

where  $\bar{x}$  is the risk-score-weighted feature mean in the risk set. The optimum balances observed event-subject covariates against expected risk-set covariates.

The negative Hessian is a sum of weighted risk-set covariance matrices. It drives optimization and model-based covariance estimates.

## 32.5 Tied event times

- **Exact:** averages possible event orders; expensive.
- **Efron:** progressively removes fractions of tied-event score; often accurate.
- **Breslow:** uses the same full denominator for a tied group; simple but less accurate with many ties.

Record the package default. The chapter's educational Cox implementation uses Breslow ties.

## 32.6 Reading the scratch Cox class

For each event time, `_objective`:

1. identifies deaths and the risk set;
2. adds event linear scores;
3. subtracts tied-event count times `logsumexp` of risk scores;
4. calculates weighted risk-set feature means; and
5. adds an optional  $L_2$  penalty.

`logsumexp` avoids overflow when exponentiated scores are large. `L-BFGS-B` numerically minimizes negative partial log likelihood. The class then estimates Breslow baseline increments.

It is educational, not production-ready: no Efron ties, delayed entry, strata, robust covariance, sampling weights, or sparse matrices.

## 32.7 Absolute survival after fitting

The Breslow baseline cumulative hazard is:

$$\hat{H}_0(t) = \sum_{t_j \leq t} \frac{d_j}{\sum_{k \in R(t_j)} e^{x_k^\top \hat{\beta}}}.$$

Then:

$$\hat{S}(t | x) = \exp[-\hat{H}_0(t) e^{x^\top \hat{\beta}}].$$

A Cox linear predictor is a relative-risk score, not a probability. Absolute prediction needs the baseline event process and target population.

## 32.8 Penalization and event information

Ridge subtracts  $\lambda \sum \beta_j^2 / 2$  from partial log likelihood. Lasso and elastic net use  $L_1$  or mixed penalties. Scaling and penalty selection must occur inside resampling.

Information depends strongly on event counts and risk-set comparisons, not simply total rows. Thirty-five events cannot reliably support hundreds of flexible coefficients merely because thousands of censored rows exist.

Penalization cannot repair wrong time zero, dependent censoring, missing competing events, or leakage.

## 32.9 Clustering, frailty, fixed effects, and strata

- Cluster-robust covariance changes standard errors, not coefficients.
- Shared frailty adds latent group heterogeneity and changes the model.
- District fixed effects estimate represented-district conditional contrasts.
- Stratified Cox gives strata separate baseline hazards while constraining other hazard ratios to be common.

Choose based on sampling unit, dependence, estimand, and prediction destination—not the smallest  $p$ -value.

---

# Day 33 — Assumptions, diagnostics, and alternative time scales

---

## 33.1 What proportional hazards requires

For two profiles, the log-hazard difference must remain constant over analysis time. It does not require a constant baseline hazard.

Terrain might matter during early access works but less during commissioning. A single coefficient can average a fading or sign-changing effect and obscure the operational story.

## 33.2 Complementary log-log plots

Under PH:

$$\log[-\log S(t | x)] = \log[-\log S_0(t)] + x^\top \beta.$$

Grouped curves of  $\log[-\log \hat{S}(t)]$  against  $\log t$  should be roughly parallel. This is a rough visual diagnostic; sparse tails, arbitrary grouping, and competing events can mislead.

### 33.3 Schoenfeld residuals

At an event time:

$$r_j = x_{i(j)} - \bar{x}(\hat{\beta}, t_j).$$

This is observed event-subject feature minus fitted risk-set expectation. Under a stable effect, residuals should not systematically trend with event time. Scaled residuals and smooth curves help reveal changing coefficients.

A non-significant global test does not prove PH. Large samples flag trivial deviations; small samples miss important ones. Examine size, shape, uncertainty, and decision consequences.

### 33.4 Time-varying coefficients and stratification

One extension is:

$$h(t | x) = h_0(t) \exp\{x\beta + xg(t)\gamma\}.$$

With  $g(t) = \log t$ ,  $\log HR(t) = \beta + \gamma \log t$ . A prespecified before/after milestone may communicate better. Searching many cut points and keeping the strongest exaggerates evidence.

If a variable is a nuisance with nonproportional effect, stratification can give it separate baseline hazards; then it receives no single coefficient.

### 33.5 Functional form is separate

PH concerns stability over time. Linearity concerns how a continuous predictor enters the log hazard. Diagnose curvature with justified transformations, restricted cubic splines, supported effect plots, and out-of-sample comparison.

Cutting a continuous feature at a data-chosen value loses information, creates a false jump, and adds selection multiplicity.

### 33.6 Other residuals and influence

Martingale residual:

$$M_i = \Delta_i - \hat{H}_i(Y_i).$$

It compares observed event count with fitted cumulative hazard and helps diagnose functional form. Deviance residuals symmetrize it. Score residuals and DFBETA-like measures examine coefficient influence.

Residuals are investigation tools, not automatic deletion rules. A rare influential project can be precisely the population the model must serve.

### 33.7 Accelerated failure time models

$$\log T = x^\top \beta + \sigma \varepsilon.$$



$e^{\beta_j}$  is a modeled time ratio. A value 1.20 stretches the event-time distribution by 20%, conditional on covariates.

The error distribution determines baseline shape: extreme-value gives Weibull, normal gives log-normal, logistic gives log-logistic. AFT interpretation is often intuitive but imposes more distributional structure than Cox. Check tails and never extrapolate far beyond supported follow-up because AIC is lower.

### 33.8 Restricted mean survival time

$$RMST(\tau) = \mathbb{E}[\min(T, \tau)] = \int_0^\tau S(t) dt.$$

It is area under the survival curve through a chosen horizon and is expressed in event-free time, such as months. RMST differences do not require PH.

Choose  $\tau$  before outcome inspection and within follow-up support for every compared group. The unrestricted mean is not identifiable when the tail remains censored; RMST is deliberately bounded.

### 33.9 Selection uncertainty and Bayesian extension

Standard intervals condition on the selected variables, splines, interactions, and time scale. Extensive diagnostic-driven selection makes them too narrow for the full procedure. Prespecify a primary model, label exploration, use shrinkage, and bootstrap the entire selection process for prediction performance when appropriate.

A Bayesian survival model combines event/censoring likelihood with priors on coefficients and baseline components. Posterior draws can propagate uncertainty into curves, RMST, and decisions; hierarchical priors can partially pool districts.

Results remain conditional on event definition, likelihood, censoring, priors, and computation. Perform prior and posterior predictive checks, convergence diagnostics, and prior sensitivity. “Bayesian” does not automatically mean causal, robust, or calibrated.

---

## Day 34 — Time-varying information and dynamic prediction

---

### 34.1 Baseline versus changing covariates

Baseline values are known at time zero. A time-varying value  $X_i(t)$  changes during follow-up:

$$h_i(t) = h_0(t)e^{\beta^\top X_i(t)}.$$

At event time  $t$ , use only the value known immediately before  $t$ . Backfilling a future maximum delay into early risk sets leaks the future.

### 34.2 Start–stop data

If schedule status changes at month 10 and warning occurs at 17:

| Project | Start | Stop | Status | Event at stop |
|---------|-------|------|--------|---------------|
| P1      | 0     | 10   | 0      | 0             |
| P1      | 10    | 17   | 1      | 1             |

Each row is an interval of constant information. Multiple rows are not independent subjects. Audit overlaps, gaps, duplicated events, update timestamps, and observation frequency. Split and resample by subject, never interval row.

### 34.3 Internal and external covariates

An external covariate evolves independently of subject state, such as a regional price index. An internal covariate, such as accumulated cost, is generated by the evolving project.

Internal values may predict strongly but can be shaped by earlier condition and intervention, creating time-dependent confounding and selection. Their Cox coefficients are associational without a causal design.

### 34.4 Landmark prediction

At landmark  $s$ :

- retain subjects observable and event-free at  $s$ ;
- use history available by  $s$ ;
- condition explicitly on survival to  $s$ ; and
- predict in  $(s, s + \tau]$ .

The target is:

$$P(T \leq s + \tau, J = 1 \mid T > s, \mathcal{H}(s)).$$

This population consists of survivors to  $s$ , not the baseline cohort. A project censored exactly at the landmark is not known observable afterward unless a timestamp convention justifies inclusion.

### 34.5 Immortal-time bias

Calling projects “ever reviewed by month 6” from appraisal guarantees that reviewed projects survived event-free and observable long enough to receive review. Their pre-review time is immortal with respect to that classification.

Defensible choices depend on the question:

- model review as a correctly aligned time-varying exposure;
- align both strategies at month 6 in a target-trial emulation;
- use a month-6 landmark among survivors; or
- use review only in predictions made at or after it is known.

Adding eventual review to a baseline Cox model does not fix the bias.

### 34.6 Recurrent events

First-event analysis discards later warnings or outages. Andersen–Gill represents repeated start–stop intervals and models event intensity, often with subject-clustered covariance.

Other models target event order, gap time, or latent frailty. Decide whether the scientific quantity is first event, total burden, time between events, or individual propensity.

## 34.7 Joint models

Landmarking summarizes history at selected times. Joint models connect a longitudinal measurement process with event time through shared latent structure, potentially handling noisy repeated values and informative observation/dropout.

They require assumptions for trajectory, association, event process, missingness, and random effects or priors. Use them when longitudinal and event processes are scientifically inseparable, not simply because they are sophisticated.

# Day 35 — Competing risks and multi-state processes

## 35.1 Event type and cumulative incidence

Let  $T$  be time to the first event of any type and  $J$  identify its cause. Cause- $k$  cumulative incidence is:

$$F_k(t) = P(T \leq t, J = k).$$

$F_1(36)$  is real-world probability of a warning by month 36 before cancellation. All-cause survival  $S(t)$  means no event type has occurred.

## 35.2 Cause-specific hazards

$$\lambda_k(t) = \lim_{\Delta t \downarrow 0} \frac{P(t \leq T < t + \Delta t, J = k \mid T \geq t)}{\Delta t}.$$

Overall hazard is  $\sum_k \lambda_k(t)$ , and:

$$S(t) = \exp \left[ - \sum_k \int_0^t \lambda_k(u) du \right].$$

Cause- $k$  incidence is:

$$F_k(t) = \int_0^t S(u-) \lambda_k(u) du.$$

First remain free of every event until just before  $u$ , then experience cause  $k$ .

## 35.3 Why one minus Kaplan–Meier overstates incidence

If cancellation is censored,  $1 - \hat{S}_{KM}$  imagines cancelled projects remain capable of later warnings. This is a hypothetical net-risk quantity, not observed-world warning probability. The discrepancy grows as competing incidence grows.

## 35.4 Aalen–Johansen by hand

At event time  $t_j$ :

$$\Delta \hat{F}_k(t_j) = \hat{S}(t_{j-}) \frac{d_{kj}}{n_j},$$
$$\hat{S}(t_j) = \hat{S}(t_{j-}) \left( 1 - \frac{\sum_k d_{kj}}{n_j} \right).$$

In the chapter's five-project example, warning increments are 0.20 at month 2 and 0.30 at month 5; cancellation incidence is 0.20 and remaining survival is 0.30. The three quantities sum to 1.

The scratch function loops over all event times, uses one shared all-event-free risk set, adds each cause's incidence increment using survival-before, and then updates survival. Censoring only changes later risk sets.

## 35.5 Cause-specific Cox and coherent predictions

Fit one Cox model per cause:

$$\lambda_k(t \mid x) = \lambda_{0k}(t) e^{x^\top \beta_k}.$$

Other causes leave the cause- $k$  risk set at their event time. Combine all fitted cause-specific hazards to produce coherent CIFs.

A covariate can raise cause-1 hazard yet lower cause-1 incidence by increasing a competing cause even more. Rate and probability are different targets.

## 35.6 Fine–Gray subdistribution hazard

Fine–Gray uses a special risk set that retains competing-event subjects under weighting conventions and models a subdistribution hazard linked to one CIF.

A subdistribution hazard ratio is not an instantaneous rate ratio among currently event-free subjects, not a probability ratio, and not the same as a cause-specific hazard ratio. Separate Fine–Gray models for every cause do not automatically create probabilities that sum coherently.

Choose the model from the estimand, not a familiar output table.

## 35.7 Multi-state models

A first-event model stops after warning or cancellation. A multi-state system can represent Active, Warning, Cancelled, and Completed, with transition-specific hazards and state-occupation probabilities.

A Markov model assumes future transitions depend on current state and covariates, not full history. A semi-Markov model can use time since state entry. State definitions, allowed transitions, and timestamps are measurement choices.

Use multi-state analysis when questions include completion after warning, time spent delayed, or probability of occupying a state—not just the first event.

---

# Day 36 — Nonlinear and hierarchical survival models

---

## 36.1 When flexibility is justified

Cox can already contain splines and interactions. Consider other learners for complex thresholds, local effects, many regulated predictors, or difficult time-varying effects when event information is sufficient.

Keep a strong regularized Cox reference. A tiny predictive gain may not justify worse calibration, auditability, computation, or maintenance.

## 36.2 Survival trees

A survival tree splits subjects using event-process criteria such as log-rank separation. Each terminal node estimates a survival curve or cumulative hazard.

It cannot regress observed follow-up time with ordinary squared error because censored time is not the event time. Control terminal **event counts**, not only rows. Specify tie handling, competing-risk target, depth, pruning, and leaf output.

Log-rank splitting favors proportional separation and may miss crossing hazards or differences at one operational horizon. Split criteria must match the prediction target.

## 36.3 Random survival forests

Typically:

1. bootstrap subjects;
2. grow deep survival trees;
3. offer random feature subsets at splits;
4. estimate terminal cumulative hazards or survival curves; and
5. average across trees.

Feature and bootstrap randomness reduce tree correlation. OOB prediction is internal evidence, not temporal, geographic, or external transport. Clustered data require cluster-aware sampling.

Tune tree count, feature subset, minimum terminal events, sampling, and split rule using censoring-aware metrics at prespecified horizons.

## 36.4 Survival boosting

For Cox boosting:

$$\mathcal{L}(f) = - \sum_{i: \Delta_i=1} \left[ f(x_i) - \log \sum_{k \in R(T_i)} e^{f(x_k)} \right].$$

Weak learners reduce a loss coupled through risk sets. Other methods optimize AFT likelihood, discrete hazards, or ranking. Name the objective.

Learning rate, tree complexity, stages, and early stopping interact. Early stopping consumes validation evidence and belongs inside development folds.

## 36.5 Discrete-time survival

Create one row per subject-interval while still at risk and estimate conditional interval probability:

$$q_{it} = P(T \text{ in interval } t \mid T \text{ not earlier}, X_i).$$

Then:

$$S_i(t) = \prod_{u \leq t} (1 - q_{iu}).$$

Logistic, complementary log-log, boosted, or neural models can estimate these probabilities. Interval rows from one subject are dependent. Random row splitting leaks identities and later history. Multiple causes need coherent multinomial transitions.

## 36.6 Neural survival models

A neural model may output a Cox score, discrete hazard, distribution, survival curve, or cause-specific incidence. The brand name does not reveal the estimand.

Check censoring loss, monotonic curves, probability coherence, time representation, event count, tuning, and temporal/external validation. Deep learning is most plausible with large event counts and rich inputs; ordinary tabular cohorts often favor simpler regularized models.

## 36.7 Shared frailty and partial pooling

$$h_{ij}(t) = h_0(t) e^{x_{ij}^\top \beta + u_j}.$$

$u_j$  is a latent group effect. Sparse districts partially pool toward the population distribution. Frailty induces within-group dependence and risk-set selection over time.

Frailty variance does not identify why districts differ. For a new district, an existing-district random effect is unavailable; prediction may integrate over the frailty distribution or require local data.

## 36.8 Resampling destinations and explanation

Hold out according to the generalization target: future project, new district, future period, or both. “Generalizes” is incomplete without a destination.

Survival permutation importance needs a survival metric. PDP or SHAP must name whether output is  $S(t)$ , hazard score, or  $F_k(t)$  and at which horizon. Importance may change with time. These are fitted-model explanations, not causal decompositions.

---

# Day 37 — Censoring-aware evaluation and decisions

---

## 37.1 Complete prediction contract

Every metric must name prediction time, eligible population, event type, horizon, competing-event handling, censoring adjustment, validation destination, and whether it assesses ranking, probability, or action.

“C-index 0.74” omits most of the scientific question.

## 37.2 Harrell concordance

If subject  $i$  has the earlier observed time and experiences cause 1 then, its pair with a later subject is comparable. A score is concordant if it ranks  $i$  higher.

$$\hat{C} = \frac{\text{concordant} + 0.5 \times \text{score ties}}{\text{comparable pairs}}.$$

Harrell C excludes ambiguous pairs and can depend on censoring. It is a global ranking measure, not horizon-specific probability accuracy. Uno C uses inverse censoring weights under additional assumptions.

## 37.3 Time-dependent AUC

At  $\tau$ , define cases and controls using a stated incident/dynamic or cumulative/dynamic convention, then adjust for censoring. Different conventions answer different questions.

Plot supported horizons. Selecting the time where AUC peaks is outcome-driven. A model can rank early events well and later events poorly.

## 37.4 IPCW Brier score

Without censoring, cause-1 Brier at  $\tau$  is:

$$\frac{1}{n} \sum_i [\mathbb{1}(T_i \leq \tau, J_i = 1) - \hat{F}_{1i}(\tau)]^2.$$

Status is unknown for subjects censored before  $\tau$ . IPCW upweights subjects whose status is observed by the inverse probability of remaining uncensored.

Estimate  $G(t) = P(C > t)$  with a reverse Kaplan–Meier or justified conditional censoring model, using training/reference data. A competing event before  $\tau$  is known not to be cause 1 and remains observed; it is not censored.

Small  $\hat{G}$  produces unstable weights. Restrict horizons or prespecify weight truncation and sensitivity analyses. Integrated Brier averages over a stated time range whose weights and maximum time must be reported.

## 37.5 Reading the scratch IPCW function

- Subjects with any observed event by the horizon receive inverse  $G$  just before their event.

- Cause-1 events have outcome 1; competing events have outcome 0.
- Subjects still at risk after the horizon receive inverse  $G(\tau)$  and outcome 0.
- Subjects censored on or before the horizon receive zero weight because status is unknown.

This recovers information only under the censoring assumption. If censoring depends on features, model  $G(t | X)$  appropriately and validate it.

## 37.6 Calibration at a horizon

For each prediction group:

1. calculate predictions without validation outcomes;
2. report mean predicted CIF;
3. estimate observed CIF with Aalen–Johansen;
4. show events, competing events, censoring, numbers at risk, and uncertainty.

Raw observed cause-1 proportion mishandles censored status. Tiny bins cannot establish calibration. Any flexible recalibration needs separate development evidence.

## 37.7 Inference, prediction, and conformal methods

A narrow coefficient interval does not imply accurate individual probabilities. Coefficient inference, ranking, calibration, forecast uncertainty, and decisions require different evidence.

Ordinary split conformal cannot use  $|T - \hat{T}|$  for censored rows because  $T$  is unknown. Dropping censored rows breaks exchangeability. Conformal survival methods add censoring assumptions, weighting, bounds, or alternative targets. State whether the output is a lower bound, interval, or probability band and the population receiving coverage.

## 37.8 Validation destinations

- Apparent performance uses training data and is optimistic.
- Internal validation estimates optimism within one source.
- Temporal validation uses later approvals.
- Geographic validation holds out places.
- External validation uses an independently collected source.
- Prospective validation freezes the system before live future cases.

Describe shifts in predictors, censoring, competing events, follow-up, measurement, and interventions. A random 20% holdout is not external validation.

## 37.9 Horizon-specific decisions

For calibrated incidence  $p$  and simplified costs:

$$p > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

This assumes a currently available action, common costs, encoded intervention effectiveness, and an appropriate horizon. Capacity limits can instead select the top 10%, but calibration still informs expected yield.



A dynamic policy must specify at each landmark its population, remaining horizon, model, inputs, action, threshold/capacity, override, appeal, and next evaluation. Monitor repeated alerts, lead time, false alarms, burden, and feedback from actions.

## 37.10 Subgroup evaluation

Report group counts, event types, censoring, follow-up, CIF, calibration, discrimination when estimable, actions, and uncertainty. A software AUC based on four events is not stable evidence.

Differences may reflect sampling, measurement, baseline incidence, competing risks, access to intervention, or model failure. Tie fairness analysis to the concrete action and institution.

---

# Day 38 — The registered locked study and causal boundary

---

## 38.1 Registration

Before touching the lockbox, freeze the question, cohort, time zero, entry, causes, censoring, horizons, prediction timestamps, estimands, splits, preprocessing, candidates, tuning, metrics, censoring weights, competing-risk method, groups, missing data, uncertainty, decision policy, sensitivity analyses, stopping, updates, and allowed claim language.

Registration permits exploration; it labels it honestly and makes deviations visible.

## 38.2 The Chapter 5 protocol

The chapter uses projects through 2022 for development, forward validation years 2020–2022, and 2023–2025 as one locked temporal test. It selects by cause-1 IPCW Brier averaged equally over 12, 24, and 36 months and validation years.

Candidates include historical Aalen–Johansen CIF and raw/engineered cause-specific Cox systems with fixed ridge choices. Separate cause models are combined into coherent incidence. The study also reports concordance, calibration, access-mode and district audits, and paired locked-test uncertainty.

The feature-free historical CIF is essential: complexity must beat a credible event-history baseline.

## 38.3 Reading the benchmark

`DesignBuilder` learns medians, means, scales, and category levels only from its training fold. It optionally constructs prespecified nonlinear features.

`CauseSpecificCoxSystem` fits one Cox model for warning and one for cancellation.

`cif_from_cause_specific_models` combines their hazard increments:

1. union all cause-specific event times;
2. calculate each subject's cause increments;
3. convert total integrated hazard to event probability  $1 - e^{-dH}$ ;
4. allocate it among causes in proportion to increments;

5. add incidence using prior event-free survival; and
6. update survival.

The result is nonnegative and obeys  $F_1 + F_2 + S = 1$  within numerical precision.

`HistoricalCIF` predicts the training cohort's Aalen–Johansen values for everyone. Forward folds train only on earlier years. The lockbox is opened after candidate selection. The development reverse-KM curve supplies censoring weights.

The script saves every fold score, candidate summary, individual locked prediction, calibration table, subgroup audits, and metadata. Assertions prohibit final cost, future progress, future review, outcome time, and event type as baseline features.

## 38.4 What it does not solve

Real applications may need conditional censoring weights, delayed entry, Efron ties, clustered resampling, interval-censoring models, better missingness handling, external validation, and different state definitions. Executable does not mean universally valid.

## 38.5 Paired bootstrap scope

The benchmark resamples locked projects and recomputes the Brier difference while fits and censoring curve remain fixed. It measures validation-sample variation conditional on those frozen objects.

It omits development variability, selection uncertainty, censoring-model uncertainty, clustering, adjudication error, and transport. End-to-end procedure uncertainty would repeat preprocessing, validation, selection, fitting, and evaluation at the correct sampling unit.

## 38.6 Sensitivity analysis

Prespecified analyses change one assumption at a time: horizons, Efron ties, conditional censoring, composite events, ambiguous-date exclusion, district bootstrap, spline terms, or a month-12 landmark.

Sensitivity shows dependence on choices. It must not become a menu from which the best-looking answer is selected.

## 38.7 Prediction is not treatment effect

Projects with latent difficulty are more likely to receive review and warnings. The back-door path:

```
review <- latent difficulty -> warning
```

creates confounding by indication. Event-free survival needed to receive review can add immortal-time bias. Time-varying condition may influence review and be influenced by earlier review.

Adding every available variable to Cox does not guarantee control: mediators, colliders, unmeasured confounders, positivity, and timing matter.

## 38.8 Target-trial emulation

A hypothetical experiment specifies:

- eligibility: active warning-free projects at month 6;

- strategies: immediate review versus no review in a grace period;
- common time zero: month 6;
- follow-up: through month 36, cancellation, or censoring;
- outcome and competing-risk estimand;
- causal contrast: incidence difference under strategies; and
- analysis aligned with baseline/time-varying confounding.

Observational methods include standardization, inverse-probability weighting, g-formula, marginal structural models, and doubly robust estimation. They require consistency, positivity, conditional exchangeability, and correct nuisance estimation. Design and identification—not a regression name—support causal claims.

## 38.9 Research integrity and paper reading

Maintain a deviation log with date, reason, lockbox visibility, affected components, analysis status, and approval. Freeze tested code and environment; keep raw data read-only; script derived data; preserve a dictionary and timestamp audit.

A selected model losing to historical CIF is a valid result. Reopening search turns the old test into development evidence and requires a new future test.

For every paper map its problem, estimand, observed data, assumptions, estimator, uncertainty, evidence, failures, software additions, and smallest replication. Read core derivation and limitations, not only summaries.

---

# Capstone companion: exercises, reporting, and assessment

---

## C.1 How to approach the exercises

For every exercise, write four lines before code:

1. **Observed record:** what time, status, entry, cause, and features are available?
2. **Target:** survival, hazard, CIF, dynamic risk, performance, or causal contrast?
3. **Assumptions:** censoring, truncation, PH, distribution, exchangeability, or transport?
4. **Check:** which hand calculation, invariant, simulation, or independent implementation could reveal an error?

Foundation exercises build inequalities, delayed-entry risk sets, hazard-to-probability conversion, KM, Nelson–Aalen, Greenwood, and RMST. Modeling exercises cover partial likelihood, ties, non-PH, and AFT alternatives.

Dynamic/competing exercises cover KM overstatement, coherent cause predictions, landmark populations, and immortal time. Research exercises stress censoring sensitivity, external validation, paper replication, and registration.

## C.2 Essential invariants

Automated checks should verify:

- entry never follows exit;
- interval start is less than stop;
- event type belongs to the allowed set;
- at most one first event occurs per project;
- risk-set counts match a hand example;

- survival is nonincreasing and between 0 and 1;
- each CIF is nondecreasing and between 0 and 1;
- $S + \sum_k F_k = 1$  in the simple competing-risk system;
- predictions use no post-prediction features;
- train years precede validation/test years;
- preprocessing learns only from training rows; and
- locked predictions are saved before aggregate reporting.

## C.3 Research-ready report

1. Abstract: cohort, time zero, causes, horizons, design, locked result, limitation.
2. Event-history construction: sources, timestamps, adjudication, entry, censoring.
3. Estimands and prediction contract.
4. Candidate procedures, folds, tuning, censoring, ties, competing-risk method.
5. Every validation fold and event count.
6. Locked discrimination, Brier, calibration, and incidence.
7. Groups with counts, causes, censoring, follow-up, and uncertainty.
8. Diagnostics and prespecified sensitivity analyses.
9. Transport and supported follow-up limits.
10. Decision rule, workflow, overrides, monitoring, and contestability.
11. Reproducibility artifacts and deviations.
12. Bounded predictive conclusion and separate causal question.

A research-ready study can conclude that no candidate transports or that follow-up cannot support the horizon. Refusing an unsupported claim is success.

# Chapter 5 synthesis and practical laboratories

## S.1 One history, several targets

| Question                                      | Target                           | Typical method                           |
|-----------------------------------------------|----------------------------------|------------------------------------------|
| Time until any first event                    | $S(t)$ or $H(t)$                 | Kaplan–Meier, Nelson–Aalen               |
| Feature association with current cause-1 rate | $\lambda_1(t \mid x)$            | cause-specific Cox                       |
| Real-world cause-1 probability                | $F_1(t \mid x)$                  | Aalen–Johansen or combined cause hazards |
| Updated risk at month 12                      | conditional landmark CIF         | landmark model                           |
| Flexible prediction                           | horizon survival/CIF             | spline Cox, forest, boosting             |
| Effect of review                              | causal contrast under strategies | experiment or target-trial emulation     |

## S.2 Derivations to complete by hand

1. Derive what is known under  $Y = \min(T, C)$  and  $\Delta$ .
2. Construct risk sets with delayed entry.
3. Derive  $S(t) = e^{-H(t)}$ .
4. Derive KM as a product of conditional survivals.

5. Calculate Greenwood variance and Nelson–Aalen hazard.
6. Derive log-rank observed-minus-expected counts.
7. Derive one Cox partial-likelihood factor and score.
8. Recover Breslow baseline survival.
9. Derive complementary-log-log parallelism under PH.
10. Calculate RMST as a staircase area.
11. Construct start–stop intervals and a landmark estimand.
12. Derive Aalen–Johansen CIF increments.
13. Combine cause-specific hazard increments coherently.
14. Enumerate concordance-comparable pairs.
15. Derive IPCW Brier contributions for event, competitor, survivor, and censoring.
16. Derive a horizon-specific cost threshold.
17. Draw the confounding and immortal-time paths for review.

## S.3 Practical laboratories

### Laboratory A: incomplete records

Generate latent event and censoring times, reveal only observed time/status, move the database-close date, and state every censored inequality.

### Laboratory B: product-limit estimation

Calculate delayed-entry KM, Greenwood, Nelson–Aalen, median survival if reached, and RMST with ties.

### Laboratory C: Cox recovery

Simulate exponential and Weibull PH data, vary censoring and ties, compare Breslow/Efron, then generate a sign-changing coefficient.

### Laboratory D: diagnostics and alternative scale

Compare constant Cox, prespecified time interaction, spline form, AFT, and RMST under crossing hazards.

### Laboratory E: dynamic information

Create month-6, 12, and 18 landmark cohorts. Compare baseline and current-history prediction without pooling unlike survivor populations.

### Laboratory F: competing risks

Hold warning hazard fixed while increasing cancellation. Compare 1—KM, Aalen–Johansen, cause-specific hazards, and Fine–Gray interpretation.

### Laboratory G: flexible survival

Give regularized Cox, spline Cox, survival forest, and boosting equal temporal folds and tuning budgets. Select using supported-horizon IPCW Brier.

Laboratory H: censoring shift

Hold event process fixed while changing censoring. Compare Harrell C, Uno C, IPCW Brier, calibration, and weight stability.

Laboratory I: external destination

Hold out a district. Report feature, missingness, cause, censoring, and follow-up shifts before model metrics.

Laboratory J: causal boundary

Demonstrate confounding by indication and immortal time, then write a target-trial protocol whose time zero and strategies align.

Formula sheet with plain-language readings

| Formula                                   | Meaning                                               |
|-------------------------------------------|-------------------------------------------------------|
| $Y = \min(T, C)$                          | observed follow-up ends at event or censoring         |
| $\Delta = \mathbb{1}(T \leq C)$           | whether the event was observed                        |
| $S(t) = P(T > t)$                         | event-free probability beyond time $t$                |
| $F(t) = 1 - S(t)$                         | cumulative event probability without competing causes |
| $h(t)$                                    | instantaneous rate among those still at risk          |
| $H(t) = \int_0^t h(u)du$                  | cumulative hazard                                     |
| $S(t) = e^{-H(t)}$                        | link from cumulative rate to survival                 |
| $\prod(1 - d_j/n_j)$                      | Kaplan–Meier survival                                 |
| $S^2 \sum d/[n(n - d)]$                   | Greenwood variance                                    |
| $\sum d_j/n_j$                            | Nelson–Aalen cumulative hazard                        |
| $d_j n_{1j}/n_j$                          | expected group-1 events in log-rank calculation       |
| $h_0(t)e^{x^\top \beta}$                  | Cox proportional hazard                               |
| $e^{\beta_j}$                             | conditional hazard ratio for one unit                 |
| $\sum [x_{event} - \bar{x}_R]$            | Cox score                                             |
| $e^{-\hat{H}_0(t)e^{x^\top \beta}}$       | Cox absolute survival                                 |
| $\log T = x^\top \beta + \sigma \epsilon$ | AFT model                                             |
| $\int_0^\tau S(t)dt$                      | RMST through $\tau$                                   |
| $F_k(t) = P(T \leq t, J = k)$             | cause- $k$ cumulative incidence                       |
| $S(t-)d_{kj}/n_j$                         | Aalen–Johansen CIF increment                          |
| $S_i(t) = \prod_{u \leq t}(1 - q_{iu})$   | discrete-time survival                                |

| Formula                                                        | Meaning                       |
|----------------------------------------------------------------|-------------------------------|
| $(\textit{concordant} + 0.5\textit{ties})/\textit{comparable}$ | Harrell concordance           |
| $1/G(t)$                                                       | inverse censoring weight      |
| $C_{FP}/(C_{FP} + C_{FN})$                                     | simplified decision threshold |

## Expanded glossary

**Aalen–Johansen estimator** — Product-integral estimator of transition probabilities and competing-risk CIFs.

**Accelerated failure time model** — Model in which covariates multiply the event-time scale through log time.

**Administrative censoring** — Planned follow-up end at database or study closure.

**At risk** — Entered, observable, and free of the defined absorbing event immediately before a time.

**Baseline hazard** — Time-varying reference hazard in a PH model.

**Calibration at a horizon** — Agreement between predicted and observed event probability by that time.

**Cause-specific hazard** — Instantaneous rate of one cause among subjects free of all causes.

**Censoring** — Incomplete event-time observation; right censoring tells us event time exceeds censor time.

**Concordance** — Ranking agreement across usable event-time pairs.

**Competing event** — An event that prevents or fundamentally changes the event of interest.

**Cumulative hazard** — Hazard accumulated over time.

**Cumulative incidence function** — Probability of one particular cause by time in the presence of competitors.

**Delayed entry** — Risk-set entry after time zero because of left truncation.

**Dynamic prediction** — Updated prediction using history available at a later prediction time.

**Fine–Gray model** — Regression model for a cause's subdistribution hazard linked to its CIF.

**Frailty** — Latent random effect representing unobserved heterogeneity or shared dependence.

**Greenwood formula** — Approximate KM variance estimator.

**Hazard** — Instantaneous event rate conditional on remaining at risk; not a probability.

**Hazard ratio** — Ratio of hazards; time-constant under PH.

**Independent censoring** — Conditional assumption that censoring adds no further event-time information.

**Interval censoring** — Event known only to lie within an interval.

**IPCW** — Inverse probability of censoring weighting.

**Kaplan–Meier estimator** — Product-limit survival estimator based on event-time risk sets.

**Landmark analysis** — Prediction among survivors at a chosen update time using information then available.

- Left censoring** — Event occurred before a known bound.
- Left truncation** — Subjects are sampled only after surviving to an entry time.
- Log-rank test** — Risk-set observed-versus-expected group comparison.
- Multi-state model** — Model for transitions among several states rather than only first event.
- Nelson–Aalen estimator** — Sum of events divided by risk-set size, estimating cumulative hazard.
- Partial likelihood** — Cox likelihood component estimating relative hazard without specifying baseline shape.
- Proportional hazards** — Covariate hazard ratios remain constant over analysis time.
- Recurrent event** — Event type that may occur multiple times for one subject.
- Restricted mean survival time** — Expected event-free time capped at a finite horizon.
- Risk set** — Subjects eligible, entered, observed, and event-free immediately before a time.
- Schoenfeld residual** — Event-subject covariate minus its risk-set-weighted expectation.
- Subdistribution hazard** — Hazard-like quantity constructed to model a competing-risk CIF.
- Time-varying covariate** — Feature changing during follow-up and aligned to when known.
- Target trial** — Explicit hypothetical randomized experiment an observational causal analysis seeks to emulate.

# Research-paper reading ladder

| Paper                   | Central idea                                                  | Beginner replication                                        |
|-------------------------|---------------------------------------------------------------|-------------------------------------------------------------|
| Kaplan & Meier (1958)   | nonparametric estimation from incomplete lifetimes            | hand-calculate every product term                           |
| Cox (1972)              | relative-hazard regression through risk-set conditioning      | compare two baseline hazard shapes                          |
| Schoenfeld (1982)       | partial-likelihood residuals for model checking               | simulate stable and fading effects                          |
| Andersen & Gill (1982)  | counting-process framework for histories and recurrent events | build valid start–stop risk sets                            |
| van Houwelingen (2007)  | dynamic prediction by landmarking                             | compare several landmark populations                        |
| Aalen & Johansen (1978) | product-integral transition probabilities                     | verify CIFs plus survival sum to one                        |
| Fine & Gray (1999)      | regression tied to a competing-risk CIF                       | separate cause-specific and subdistribution interpretations |
| Ishwaran et al. (2008)  | survival-specific forest construction                         | compare equal-budget temporal validation                    |
| Graf et al. (1999)      | prediction error with censoring                               | implement horizon IPCW Brier                                |
| Uno et al. (2011)       | censoring limitation of common concordance                    | vary only censoring across simulations                      |
| Candès et al. (2023)    | conformal prediction under survival censoring                 | state target and coverage assumptions                       |



For every methods paper, map problem, estimand, observed record, assumptions, estimator, uncertainty, evidence, failures, software gap, and a minimal falsifying replication.

---

## Suggested nine-day study rhythm

---

Use roughly three hours daily:

1. 15 minutes: retrieve the prior exit check.
2. 20 minutes: draw event and observation timelines.
3. 35 minutes: construct a tiny risk set.
4. 45 minutes: derive the central estimator.
5. 55 minutes: type and alter code.
6. 25 minutes: create a failure case.
7. 20 minutes: reconstruct a paper claim.
8. 10 minutes: write an estimand, assumption, and bounded conclusion.

Survival skill grows by repeatedly rebuilding risk sets. Do not reduce the chapter to memorizing package calls.

---

## Common beginner misconceptions

---

- Censored does not mean event-free forever.
- Left censoring is not left truncation.
- A competing event is not missing follow-up.
- Hazard is a rate, not a probability.
- Hazard ratio is not a risk ratio, probability difference, time ratio, or causal effect.
- Censoring changes later risk sets, not KM directly.
- One minus KM is not a cause-specific real-world incidence with competitors.
- A log-rank  $p$ -value is not effect size or causality.
- PH does not require constant hazard.
- PH and linear functional form are separate assumptions.
- A non-significant diagnostic does not prove an assumption.
- A Cox risk score is not an absolute probability.
- Rows do not equal information; event counts matter.
- Future progress cannot enter a baseline model.
- Start–stop rows from one subject are dependent.
- Ever-treated grouping can create immortal-time bias.
- Different landmarks contain different survivor populations.
- Fine–Gray and cause-specific hazard ratios differ.
- Ordinary regression on observed follow-up time mishandles censoring.
- OOB performance is not temporal transport.
- A C-index is not calibration or decision value.
- Subjects censored before a horizon do not have known negative status.
- A random holdout is not external validation.
- Split conformal is not automatically valid with unknown event times.
- A predictive treatment coefficient is not an intervention effect.

- Reopening a lockbox makes it development data.
- 

## Ethical, operational, and governance questions

---

- Who defines and adjudicates a formal warning?
  - Could warning practices differ by district or period?
  - Why does follow-up end, and who becomes invisible?
  - Does cancellation represent failure, protection, choice, or several processes?
  - Is review supportive, burdensome, or both?
  - What horizon allows meaningful preventive action?
  - Who sets false-alert and missed-warning consequences?
  - Can teams correct dates, features, states, and decisions?
  - How are repeated alerts and override burden monitored?
  - Which groups lack enough follow-up for a promised horizon?
  - Who can pause the system when calibration or censoring changes?
  - How do actions alter later warnings, cancellations, and recorded data?
  - What new causal design is needed before claiming review benefit?
- 

## Final readiness checklist

---

You are ready to leave Chapter 5 when you can:

- define unit, origin, entry, event types, censoring, and horizon;
- distinguish four incomplete-observation mechanisms;
- construct delayed-entry risk sets;
- state independent censoring as an unprovable conditional assumption;
- move among density, survival, hazard, and cumulative hazard;
- derive KM, Greenwood, Nelson–Aalen, and log-rank calculations;
- derive Cox partial likelihood, score, and baseline survival;
- interpret hazard ratios without probability or causal overreach;
- document tie handling and event-information limits;
- diagnose PH separately from functional form;
- interpret AFT time ratios and RMST;
- align time-varying information using start–stop or landmarks;
- explain recurrent events, joint models, and immortal time;
- derive Aalen–Johansen incidence and distinguish Fine–Gray;
- explain survival trees, forests, boosting, discrete time, and frailty;
- design resampling for the destination;
- calculate concordance and IPCW Brier contributions;
- build censoring-aware calibration at supported horizons;
- separate ranking, probability, decision, coefficient, and forecast uncertainty;
- execute the registered temporal benchmark once;
- preserve predictions, metadata, deviations, and unfavorable results; and
- distinguish prognostic association from a target-trial causal effect.

The entire chapter can be remembered as:

clock → risk set → event process → probability → decision → bounded claim.

# Chapter 6 — From Prediction to Intervention

---

## Level 6 Research Practitioner: ten days of causal inference, reproducible evidence, and responsible deployment

**Central promise.** Earlier chapters taught you to describe data, predict outcomes, quantify uncertainty, and model event histories. This final chapter asks the harder question: **what would change if we intervened?** By the end, you will be able to define causal effects with counterfactuals, defend an identification strategy, estimate effects with experiments or observational data, diagnose failure of overlap and balance, study longitudinal and quasi-experimental designs, investigate heterogeneity without data dredging, compare models honestly, package a reproducible study, and complete a registered capstone whose claim does not outrun its design.

This is the final chapter of the book. It does not make you an all-knowing researcher; no chapter could. It gives you something more useful: a disciplined route from a vague policy question to a claim whose assumptions, evidence, uncertainty, and limits can be inspected by another person.

The supplied draft correctly identified several summit skills: paired model comparison, ablation, model cards, experiment tracking, production consistency, drift monitoring, and capstone work. Those topics remain in Day 47 and Day 48. But Chapter 5 ended with a more urgent unresolved question:

Does senior review prevent major project overruns?

A predictive model cannot answer it. Projects sent for review may be precisely the projects that were already in trouble. If reviewed projects fail more often, review may be ineffective—or it may have helped a very high-risk group. This chapter therefore places causal design before research operations. Reproducibility preserves an analysis; it cannot rescue the wrong estimand or an unidentified effect.

---

## Prerequisite checkpoint

Before beginning, you should be able to explain, without relying on software output:

- a population, sample, random variable, expectation, variance, and conditional probability;
- a confidence interval and why it is not a probability that a fixed parameter lies in one realised interval;
- correlation versus causation;
- train, validation, test, temporal split, leakage, calibration, and decision threshold;
- linear and logistic regression, regularisation, interactions, splines, and how a fitted logistic probability is produced and evaluated;
- bootstrap resampling and why the resampling unit matters;
- survival, hazard, censoring, competing risks, and a target prediction horizon;
- the difference between association, prediction, decision utility, and intervention effect; and
- why a registered analysis protects against changing the question after seeing the answer.

If several items feel uncertain, revisit the earlier exit checks. Research-level work is mostly the careful composition of fundamentals.

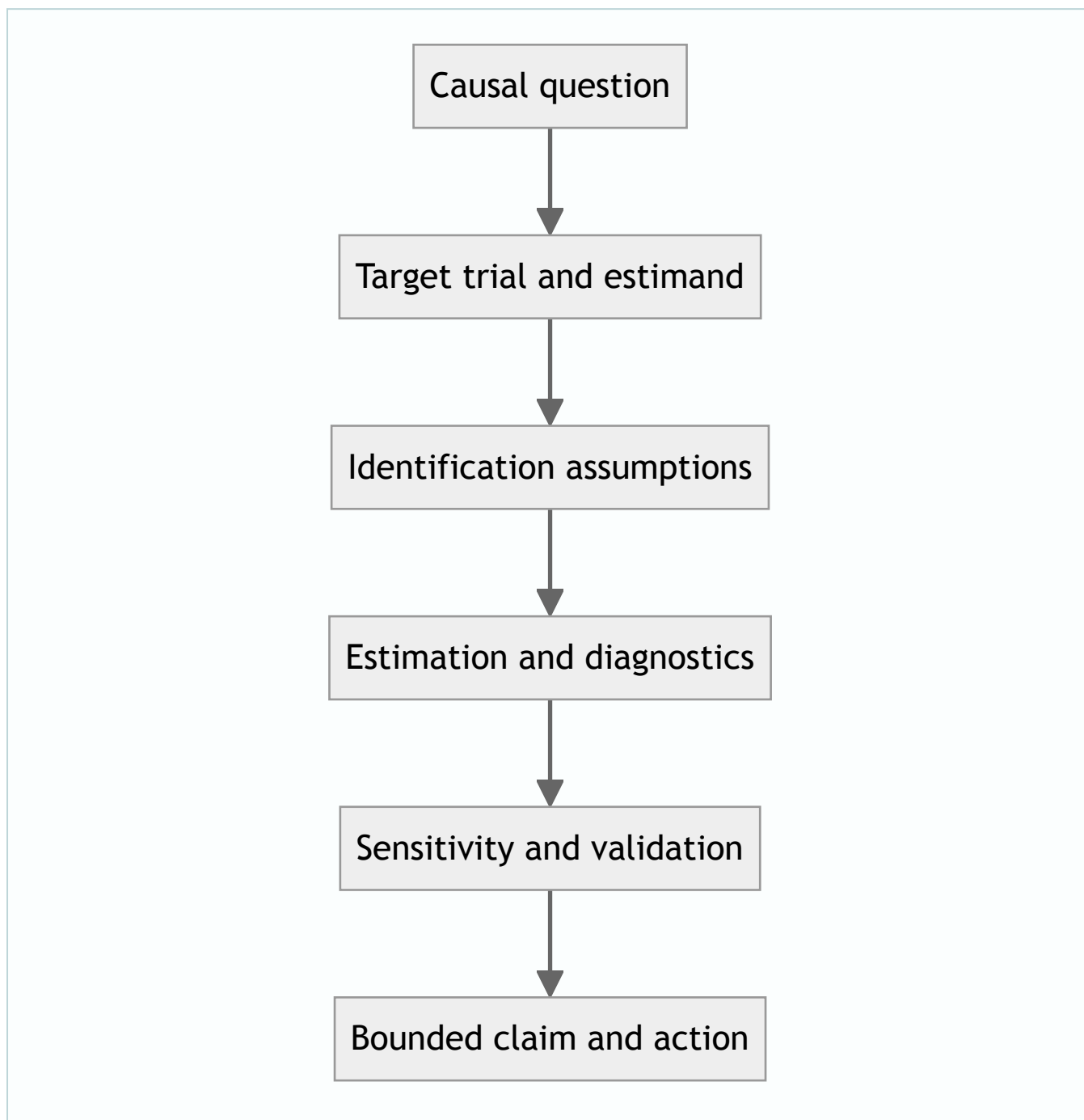
## Learning outcomes

At the end of Chapter 6, you should be able to:

- translate a policy question into a treatment, outcome, population, time zero, follow-up, contrast, and causal estimand;
- define potential outcomes and explain the fundamental problem of causal inference;
- distinguish ATE, ATT, conditional effects, risk difference, risk ratio, odds ratio, and policy value;
- state consistency, exchangeability, positivity, and non-interference assumptions in plain language and notation;
- draw and criticise a directed acyclic graph containing confounders, mediators, colliders, selection variables, instruments, and proxies;
- explain why adjusting for every available variable can increase bias;
- analyse randomised experiments using intention-to-treat reasoning, including blocked and cluster designs;
- distinguish assignment, receipt, adherence, per-protocol effects, and local effects under noncompliance;
- identify effects by regression standardisation, matching, stratification, inverse weighting, and augmented inverse weighting;
- inspect covariate balance, overlap, influential weights, and effective sample size;
- use cross-fitting to separate nuisance-model training from effect-score evaluation;
- explain what “doubly robust” does and does not guarantee;
- emulate the protocol of a target trial and recognise immortal-time and time-zero bias;
- describe the parametric g-formula and marginal structural models for time-varying treatment;
- distinguish the identifying assumptions and target populations of difference-in-differences, regression discontinuity, and instrumental variables;
- investigate treatment-effect heterogeneity with pre-specified modifiers, causal trees, forests, and policy evaluation;
- perform sensitivity analyses that expose the dependence of a conclusion on unmeasured confounding and modelling choices;
- compare predictive procedures with paired evidence while respecting dependence induced by resampling;
- produce a data sheet, model card, environment record, analysis manifest, and deviation log;
- plan monitoring for drift, delayed outcomes, feedback loops, safety, rollback, and retirement; and
- execute and report a registered causal study whose conclusion is bounded by its design.

## The ten-day route

| Day | Central idea                  | Question resolved                                                   |
|-----|-------------------------------|---------------------------------------------------------------------|
| 39  | Counterfactuals and estimands | What effect are we trying to learn?                                 |
| 40  | Causal structure              | Which variables should—and should not—be adjusted for?              |
| 41  | Designed assignment           | What does randomisation identify?                                   |
| 42  | Observational adjustment      | How can measured confounding be addressed?                          |
| 43  | Orthogonal estimation         | How do flexible nuisance models and uncertainty fit together?       |
| 44  | Longitudinal causality        | How do we align eligibility, assignment, follow-up, and analysis?   |
| 45  | Natural assignment mechanisms | What can be learned when treatment is not randomised?               |
| 46  | Effect variation              | For whom might an intervention help, and how should that be tested? |
| 47  | Research operations           | Can others reproduce, challenge, and safely use the result?         |
| 48  | Integrated capstone           | Can the entire design-to-claim chain survive a locked analysis?     |



The ordering matters. Selecting an estimator before defining the causal question is like choosing a measuring instrument before deciding what quantity exists.

## Mastery boundary for the final chapter

- **Build it yourself:** Days 39–43 and 47–48 derive estimands and estimators, execute code, diagnose failures, and culminate in a registered analysis.
- **Read and recognise:** Days 44–46 survey longitudinal, quasi-experimental, and heterogeneous-effect frameworks. On a first pass, mastery means specifying the design and estimand, tracing assumptions, interpreting diagnostics, and implementing only the supplied subgroup-score example—not independently fitting production g-formula, MSM, RD, DiD, IV, synthetic-control, DR-learner, or causal-forest systems.

Stretch implementations in Days 44–46 require a methods text, current software documentation, and usually expert review. They are explicitly optional and are not prerequisites for the final capstone.

## Running case: should a project receive senior review at month 6?

We continue the fictional microhydro power programme. At month 6 after approval, programme managers can assign a project to a structured senior review. The review may revise procurement, add engineering support, or change milestones. The outcome is a formally documented major-overrun warning between month 6 and month 36.

Our first causal question is:

Among projects active and warning-free at month 6, what would be the difference in 36-month warning risk if every eligible project received senior review at month 6 versus if none did?

That sentence fixes important design elements:

| Element           | Definition in the running study                                |
|-------------------|----------------------------------------------------------------|
| Unit              | An eligible project                                            |
| Eligibility       | Active, observable, and warning-free through month 6           |
| Time zero         | Month 6 after approval                                         |
| Treatment         | Assignment to the defined senior-review package at time zero   |
| Comparator        | No senior-review package at time zero                          |
| Outcome           | Major-overrun warning from just after month 6 through month 36 |
| Follow-up         | 30 months                                                      |
| Causal contrast   | Assign everyone versus assign no one                           |
| Primary scale     | Risk difference                                                |
| Target population | Projects meeting the month-6 eligibility rule                  |

## A causal laboratory

The next generator is fictional. It gives review more often to projects with difficult terrain, long cable routes, and inexperienced contractors. Those same factors increase warning risk, so the raw reviewed-versus-unreviewed comparison is confounded. The generator internally creates two potential outcomes but hides them unless explicitly asked to reveal the simulation truth.

Save this block as `chapter6_data.py` if you want to reuse it outside the notebook.

```
import numpy as np
import pandas as pd
from scipy.special import expit, logit

def make_mhp_causal_data(n=4000, seed=6060, reveal_counterfactuals=False):
 """Create a fictional month-6 senior-review target-trial cohort.

 The relationships are educational, not evidence about real projects,
 places, contractors, or review programmes.
 """
 rng = np.random.default_rng(seed)
```

```

district = rng.choice(
 ["North", "Central", "South", "Frontier", "Valley"],
 size=n,
 p=[0.18, 0.25, 0.20, 0.15, 0.22],
)
district_risk = pd.Series(district).map(
 {"North": 0.15, "Central": -0.10, "South": 0.05,
 "Frontier": 0.28, "Valley": -0.08}
).to_numpy()

terrain = np.clip(rng.normal(0.0, 1.0, n) + district_risk, -2.5, 2.8)
cable_km = np.clip(rng.gamma(2.4, 1.7, n) + 0.8 * terrain, 0.2, None)
experience = rng.poisson(np.clip(7.5 - 1.0 * terrain, 1.5, None)) + 1
budget = np.exp(rng.normal(4.0 + 0.12 * terrain, 0.45, n))
remote_probability = expit(-0.7 + 0.8 * terrain)
remote_access = rng.binomial(1, remote_probability, n)
approval_year = rng.integers(2016, 2026, n)

z_budget = (np.log(budget) - np.log(budget).mean()) / np.log(budget).std()
z_cable = (cable_km - cable_km.mean()) / cable_km.std()
z_experience = (np.log1p(experience) - np.log1p(experience).mean()) / np.log1p(experience).std()

Review assignment: difficult projects are preferentially reviewed.
propensity = expit(
 -0.35 + 0.78 * terrain + 0.42 * z_cable
 + 0.30 * z_budget - 0.45 * z_experience
 + 0.35 * remote_access + 0.08 * district_risk
)
senior_review = rng.binomial(1, propensity, n)

Untreated warning probability. Review lowers log odds, with a somewhat
larger benefit in difficult terrain. This heterogeneity is hidden during
ordinary analysis and revealed only for simulation validation.
untreated_logit = (
 -1.45 + 0.58 * terrain + 0.40 * z_cable
 + 0.28 * z_budget - 0.38 * z_experience
 + 0.34 * remote_access + district_risk
)
p_y0 = expit(untreated_logit)
treatment_log_odds_effect = -0.62 - 0.16 * (terrain > 0.75)
p_y1 = expit(untreated_logit + treatment_log_odds_effect)

A common latent draw makes Y(1) <= Y(0) in this teaching simulation.
Monotonic benefit is NOT required by the adjustment estimators below.
latent_u = rng.uniform(size=n)
y0 = (latent_u < p_y0).astype(int)
y1 = (latent_u < p_y1).astype(int)
observed_y = np.where(senior_review == 1, y1, y0)

data = pd.DataFrame({
 "district": district,
 "approval_year": approval_year,
 "terrain_index": terrain,
 "estimated_cable_km": cable_km,
 "contractor_experience_projects": experience,
 "appraisal_budget_million_pkr": budget,
 "remote_access": remote_access,
 "senior_review_month6": senior_review,
 "warning_by_month36": observed_y,
})

if reveal_counterfactuals:
 data["y0"] = y0
 data["y1"] = y1
 data["true_propensity"] = propensity
 data["true_p_y0"] = p_y0
 data["true_p_y1"] = p_y1
return data

```



```
df = make_mhp_causal_data()
print(df.shape)
print(df[["senior_review_month6", "warning_by_month36"]].mean())
```

In real data, the `y0`, `y1`, and true probabilities do not exist as observable columns. Revealing them later is a simulation check, not a workflow that empirical researchers can copy.

## Day 39 — Counterfactuals, Estimands, and Identification

### 39.1 Prediction and causation answer different questions

Let  $A$  be senior review,  $Y$  the later warning, and  $X$  information available before review.

A predictive question is:

$$P(Y = 1 \mid A, X).$$

It asks what outcomes tend to occur among projects with particular observed attributes and actions. A causal question is:

$$P\{Y(a) = 1\},$$

where  $Y(a)$  is the outcome the project would have under an intervention setting review to  $a$ . The conditioning bar and intervention are not interchangeable. If managers send risky projects to review, then  $P(Y = 1 \mid A = 1)$  can exceed  $P(Y = 1 \mid A = 0)$  even when review prevents warnings.

Prediction can be excellent without identifying an intervention effect. Causal estimation can be unbiased while predicting individual outcomes poorly. Neither task is superior; they serve different decisions.

### 39.2 Potential outcomes

For each project  $i$ , define:

- $Y_i(1)$ : the warning outcome if project  $i$  were assigned senior review;
- $Y_i(0)$ : the warning outcome if project  $i$  were not assigned senior review.

The individual causal effect on the risk-difference scale is

$$Y_i(1) - Y_i(0).$$

Only one potential outcome is observed:

$$Y_i = A_i Y_i(1) + (1 - A_i) Y_i(0).$$

The other is counterfactual. This is the fundamental problem: the same unit cannot be observed at the same time under both assignments. Causal inference therefore targets averages and uses design or assumptions to make groups comparable.

## 39.3 Choose an estimand before an estimator

For binary outcomes, common estimands include:

| Estimand                       | Definition                | Question                                                  |
|--------------------------------|---------------------------|-----------------------------------------------------------|
| Average treatment effect       | $EY(1) - Y(0)$            | What if everyone versus no one were reviewed?             |
| Average effect among treated   | $EY(1) - Y(0) \mid A = 1$ | What was the average effect for reviewed projects?        |
| Average effect among untreated | $EY(1) - Y(0) \mid A = 0$ | What might review do for currently unreviewed projects?   |
| Conditional average effect     | $EY(1) - Y(0) \mid X = x$ | How does the average effect vary with baseline profile?   |
| Policy value                   | $E[Yd(X)]$                | What outcome would follow a rule $d(X)$ assigning review? |

An ATE, ATT, and policy value can differ because they average over different people and assignments. “The treatment effect” is incomplete.

The effect scale also matters. Let  $p_1 = EY(1)$  and  $p_0 = EY(0)$ .

$$\text{Risk difference} = p_1 - p_0, \quad \text{risk ratio} = \frac{p_1}{p_0},$$

$$\text{odds ratio} = \frac{p_1/(1 - p_1)}{p_0/(1 - p_0)}.$$

Risk differences are often easiest for capacity planning: a difference of  $-0.06$  means about six fewer warnings per 100 projects under the stated intervention and population. Ratios transport differently and may be scientifically useful. Odds ratios are non-collapsible: a conditional odds ratio can differ from a marginal odds ratio even without confounding. Report the scale you registered; do not switch because another looks more dramatic.

For time-to-event outcomes, replace the binary risk by  $PT(a) \leq t$ , survival  $PT(a) > t$ , or restricted mean event-free time. Always name the horizon and competing-event strategy.

## 39.4 Four core identification assumptions

An estimand is a property of an imagined intervention. Identification connects it to the observed-data distribution.

### Consistency

If a project actually receives treatment  $A = a$ , its observed outcome equals  $Y(a)$ :

$$A_i = a \implies Y_i = Y_i(a).$$

This demands a well-defined treatment. “Senior review” cannot silently range from a five-minute signature to a six-week engineering intervention. Versions can be allowed, but the estimand must say how they are assigned or averaged.

### Conditional exchangeability

After conditioning on adequate pre-treatment covariates  $X$ ,

**Notation:** The symbol  $\perp$  means "is statistically independent of." You already used the underlying idea in Chapter 2, Day 8, when the Gaussian linear model assumed errors were independent of one another — this is the same concept, just written with its own symbol instead of a sentence.  $U \perp V \mid X$  reads as "U and V are independent of each other, once you already know X": knowing V (in addition to X) would tell you nothing more about U than X alone already does.

$$\{Y(1), Y(0)\} \perp A \mid X.$$

Informally, within covariate strata, treatment behaves as if assigned independently of the potential outcomes. In an ideal randomised trial it follows from assignment by design. In observational data it is an untestable no-unmeasured-confounding assumption supported—or undermined—by domain knowledge, measurement, timing, and design.

## Positivity

For covariate profiles in the target population,

$$0 < P(A = 1 \mid X = x) < 1.$$

If every frontier project is reviewed, the data contain no untreated frontier comparison. A flexible algorithm cannot invent it. Near violations create extreme weights and make results depend on extrapolation. Remedies change the question: restrict the target population, alter the estimand, collect data, or compare realistic treatment rules.

## No interference

One project's treatment should not alter another project's outcome under the simple potential-outcome notation. Shared engineers violate this if reviewing one project diverts capacity from another. Network, cluster, and spillover estimands can replace the assumption; ignoring interference cannot.

These assumptions are sometimes grouped with treatment-variation requirements as SUTVA, but naming them separately makes criticism easier.

## 39.5 Identification by the g-formula

Under consistency, conditional exchangeability, and positivity,

$$E\{Y(a)\} = E_X[E(Y \mid A = a, X)].$$

Derivation:

$$\begin{aligned} E\{Y(a)\} &= E_X[E\{Y(a) \mid X\}] \\ &= E_X[E\{Y(a) \mid A = a, X\}] && \text{exchangeability} \\ &= E_X[E(Y \mid A = a, X)] && \text{consistency.} \end{aligned}$$

Positivity ensures the final conditional mean can be learned where it is needed. This derivation—not a preference for a regression package—is the justification for standardisation.

## 39.6 Identification, estimation, and inference are different

Keep three questions separate:

1. **Identification:** If the observed-data distribution were known exactly, would it determine the causal estimand under stated assumptions?

2. **Estimation:** With a finite sample, how will the required probabilities or expectations be approximated?
3. **Inference:** How will sampling and procedure uncertainty be quantified?

More data can reduce estimation error. It cannot fix a collider-conditioned design, undefined intervention, or unmeasured confounder. A confidence interval quantifies uncertainty under the model and identification assumptions; it does not place those assumptions on trial.

## 39.7 The naive answer in the running case

```
naive = (
 df.loc[df["senior_review_month6"] == 1, "warning_by_month36"].mean()
 - df.loc[df["senior_review_month6"] == 0, "warning_by_month36"].mean()
)
print(f"Naive reviewed-minus-unreviewed risk difference: {naive:.3f}")
```

This is a valid description of observed groups. It is not automatically an ATE. Before adjusting, explain the assignment mechanism and draw the causal structure.

## 39.8 Research-paper study: Holland and Rubin

Read Peter Holland's 1986 essay, *Statistics and Causal Inference*, alongside Donald Rubin's 1974 paper on causal effects in randomised and nonrandomised studies. Do not read only for formulas. Ask:

- What object is defined before data are observed?
- Why is causation attached to effects of causes rather than causes of effects?
- Which role does assignment play?
- Which problems are scientific, and which are statistical?

The durable lesson is that causal language requires a comparison of interventions, not merely a coefficient attached to a variable.

## 39.9 Build, break, and reflect

**Build.** Write a one-page estimand table for a familiar intervention. Include eligibility, treatment versions, comparator, time zero, follow-up, outcome, competing events, population, contrast, and scale.

**Break.** Change the comparator from “no review” to “usual review practice.” Which potential outcomes change? Change the target from all eligible projects to reviewed projects. Is the estimand still the ATE?

**Reflect.** Name one version-of-treatment problem, one possible spillover, and one population with no treatment overlap.

## Day 39 exit check

You are ready for Day 40 when you can:

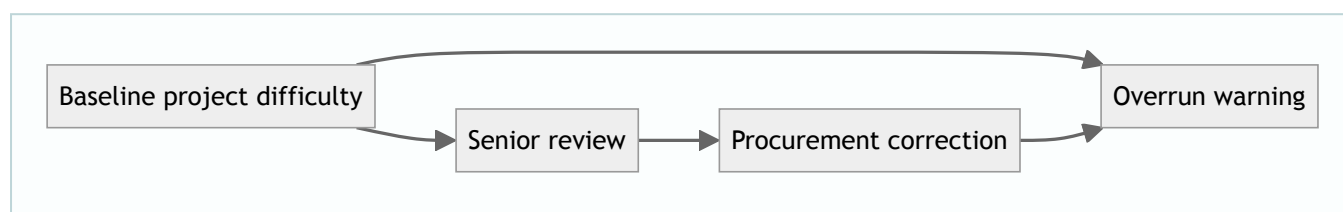
- express the primary question using  $Y(1)$  and  $Y(0)$ ;
  - distinguish an observed association from a causal contrast;
  - state all four assumptions in words and notation;
  - explain why positivity depends on the target population; and
  - separate identification failure from estimator error.
-

# Day 40 — Causal Diagrams and Adjustment

## 40.1 A DAG is a compact causal argument

A directed acyclic graph, or DAG, contains nodes and one-way arrows with no directed cycles. An arrow  $X \rightarrow Y$  asserts that  $X$  may causally influence  $Y$  under the system being represented. Missing arrows are also claims.

For the running study, a first graph might be:



The path  $A \leftarrow X \rightarrow Y$  is a backdoor path. It produces association between  $A$  and  $Y$  that is not caused by the review. Conditioning on a sufficient set of pre-treatment common causes blocks such paths.

A DAG does not discover causality from the dataset. It records the team's scientific assumptions so that missing variables, questionable timing, and competing explanations can be challenged before modelling.

## 40.2 Five roles a variable can play

| Role       | Basic pattern                                                     | Typical action for a total effect                 |
|------------|-------------------------------------------------------------------|---------------------------------------------------|
| Confounder | $A \leftarrow C \rightarrow Y$                                    | Adjust if measured adequately                     |
| Mediator   | $A \rightarrow M \rightarrow Y$                                   | Do not adjust when targeting total effect         |
| Collider   | $A \rightarrow K \leftarrow Y$                                    | Do not condition without a design-specific reason |
| Instrument | $Z \rightarrow A \rightarrow Y$ , no other $Z \rightarrow Y$ path | Useful for an IV design; usually not a confounder |
| Proxy      | $C \rightarrow P$ , with $C$ partly unmeasured                    | May reduce but need not eliminate confounding     |

Variable roles belong to a causal question and time order. The same measurement can be a confounder for one treatment and a mediator for another.

## 40.3 Confounders are not selected by p-values

A pre-treatment cause of both assignment and outcome can confound even if its sample association is not “significant.” Conversely, a strongly predictive variable may not be a confounder. Adjustment-set selection should use causal knowledge, then diagnostics—not stepwise outcome regression.

It can be useful to include strong pre-treatment outcome predictors for precision even when they are not confounders. But that statistical choice follows identification and must respect sample size and model stability.

## 40.4 Mediators change the estimand

Suppose review changes procurement, which changes warning risk:

$$A \rightarrow M \rightarrow Y.$$

Adjusting for post-review procurement blocks part of the total effect. The coefficient for  $A$  in a model containing  $M$  is not automatically a direct effect: mediator-outcome confounding, treatment-induced confounding, and scale issues complicate mediation. Begin with a total effect unless the scientific question genuinely concerns pathways, then define natural, controlled, or interventional direct and indirect effects precisely.

## 40.5 Colliders create association

Suppose both senior review and latent political pressure increase the probability that a project appears in a special audit dataset:

$$A \rightarrow S \leftarrow U \rightarrow Y.$$

In the full population,  $A$  and  $U$  need not be associated. Restricting to  $S = 1$  makes them associated because either can explain selection. This opens the path from review to outcome through  $U$ . Common collider traps include:

- analysing only admitted patients when exposure and illness affect admission;
- analysing only loan approvals when credit score and unmeasured judgement affect approval;
- controlling for a post-treatment variable caused by treatment and prognosis;
- complete-case analysis when missingness is affected by exposure and outcome causes; and
- selecting only projects that reached a milestone influenced by review.

“Control for everything” is therefore unsafe.

## 40.6 Descendants, instruments, and bias amplification

Conditioning on a collider's descendant can also open a path. Conditioning on a strong instrument—something that shifts treatment but has no direct outcome path—can worsen finite-sample overlap and amplify bias from an unmeasured confounder in some settings. Neither result means instruments are bad. It means an adjustment model and an instrumental-variable design use variables for different purposes.

## 40.7 Backdoor adjustment

An adjustment set  $X$  identifies the total effect through the backdoor criterion when it:

1. contains no descendant of treatment; and
2. blocks every path from  $A$  to  $Y$  that enters  $A$  through an arrow pointing into  $A$ .

There can be several valid sets. More variables are not necessarily better. Prefer variables measured before assignment, justify their causal roles, and consider whether measurement error leaves residual confounding.

For the simulated review study, a defensible baseline set includes district, approval year, terrain, cable length, contractor experience, appraisal budget, and remoteness. We do not adjust for procurement corrections measured after review.

## 40.8 Selection diagrams and transport

Even if an effect is identified inside the study, applying it elsewhere is a separate problem. Let  $S$  indicate participation or site. If terrain changes treatment effect and the terrain distribution differs between study and target populations, transport requires measuring and standardising over terrain. Ask:

- How did units enter the study?
- Which causes of treatment effect differ at the destination?

- Are treatment versions and outcome measurement comparable?
- Is there overlap between study and target populations?

External validity is not granted by a large sample.

## 40.9 DAG workflow

Use this order:

1. State one treatment, outcome, population, and time zero.
2. Add causes of treatment and causes of outcome, including unmeasured variables.
3. Add variables created after treatment only when scientifically relevant.
4. Mark selection and measurement mechanisms.
5. Identify open backdoor paths and candidate adjustment sets.
6. Compare the graph with actual timestamps and data provenance.
7. Record contested arrows and run analyses under plausible alternatives.

Do not erase an unmeasured node because the dataset lacks its column. That absence is a limitation, not a graph-editing instruction.

## 40.10 Research-paper study: diagrams as epidemiologic reasoning

Read Greenland, Pearl, and Robins' 1999 article on causal diagrams. Reconstruct one example with three variables, then explain why conditioning blocks a chain or fork but opens a collider. The important research skill is not drawing attractive graphs; it is tracing which statistical associations an analysis creates or removes under a scientific model.

## 40.11 Build, break, and reflect

**Build.** Draw a DAG for review, warning, terrain, budget, contractor experience, procurement correction, and inclusion in an audit sample. Label each variable's timing.

**Break.** Add a post-review "project health score" affected by review and latent engineering quality. What happens if eligibility requires a high score?

**Reflect.** List the smallest plausible sufficient adjustment set. Then list one unmeasured common cause and how its omission might move the effect estimate.

## Day 40 exit check

You are ready for Day 41 when you can:

- distinguish a confounder, mediator, collider, instrument, and proxy by causal role;
- explain collider bias without using the phrase "because the software says so";
- defend an adjustment set using paths and timing; and
- state why a DAG cannot prove its own missing arrows.

# Day 41 — Randomised Experiments and Noncompliance

---

## 41.1 Randomisation is an assignment mechanism

In a randomised experiment, investigators control a known probability of assignment. For a simple two-arm trial,

$$A_i \sim \text{Bernoulli}(p),$$

independently of potential outcomes. This creates exchangeability **in expectation**. A realised finite sample can still have imbalances, so report important baseline summaries and improve precision with pre-specified adjustment. Do not use significance tests of baseline differences to decide whether randomisation “worked”; the randomisation mechanism is known by design.

Randomisation does not automatically solve:

- an ambiguous treatment or outcome;
- nonadherence after assignment;
- differential loss to follow-up;
- interference between units;
- measurement altered by treatment;
- a mismatch between study and policy populations; or
- selective reporting after many outcomes and subgroups are examined.

It addresses confounding of **assignment**. The rest of the protocol still matters.

## 41.2 Intention-to-treat effect

Let  $Z$  be assignment to offer senior review, whether or not the review is completed. The intention-to-treat, or ITT, estimand is

$$E\{Y(Z = 1) - Y(Z = 0)\}.$$

The difference in observed group means is unbiased under simple randomisation:

$$\hat{\tau}_{\text{ITT}} = \bar{Y}_{Z=1} - \bar{Y}_{Z=0}.$$

A familiar large-sample standard error is

$$\widehat{SE}(\hat{\tau}) = \sqrt{\frac{s_1^2}{n_1} + \frac{s_0^2}{n_0}}.$$

Randomisation-based inference treats the potential outcomes as fixed and assignment as random. Model-based regression can improve precision, but the analysis should respect the design. With binary outcomes, report arm risks and an absolute contrast, not only an odds ratio.

## 41.3 Blocking, stratification, and covariate adjustment

If terrain and district strongly predict outcomes, randomise within pre-specified blocks. The estimator combines within-block contrasts using target-population weights. Blocking prevents severe imbalance on chosen factors and often improves precision.



Pre-specified regression adjustment can further improve precision. Include the block variables used in randomisation. When treatment effects may vary, interactions between assignment and centred baseline covariates protect the marginal contrast from restrictive constant-effect modelling. Standardise fitted potential outcomes over the trial population rather than interpreting a nonlinear-model treatment coefficient as the marginal effect.

## 41.4 Cluster randomisation

If a review team changes how an entire district operates, randomise teams or districts rather than projects. Outcomes within clusters are correlated. The effective information is closer to the number of independent clusters than the number of projects.

For roughly equal cluster size  $m$  and intraclass correlation  $\rho$ , a planning approximation is the design effect

$$1 + (m - 1)\rho.$$

Analyse at the level supported by the assignment: cluster-level summaries, randomisation inference over clusters, or cluster-robust methods with enough clusters. Five districts do not become a large trial because each contains hundreds of rows.

## 41.5 Interference and spillovers

If reviewed projects consume scarce engineers, one project's assignment can affect another's outcome. Define assignment at cluster level, or define exposure mappings such as own review plus the fraction of neighbouring projects reviewed. Relevant estimands might include:

- a direct effect holding cluster coverage fixed;
- a spillover effect among unreviewed projects;
- a total effect of changing district-level coverage; or
- an allocation-policy effect under a fixed capacity.

The ordinary two-potential-outcome notation is too small for this problem.

## 41.6 Noncompliance: assignment is not receipt

Let  $Z$  be random assignment and  $A$  actual receipt. Four latent compliance types are defined by  $A(1)$  and  $A(0)$ :

| Type         | $A(1)$ | $A(0)$ |
|--------------|--------|--------|
| Always-taker | 1      | 1      |
| Complier     | 1      | 0      |
| Never-taker  | 0      | 0      |
| Defier       | 0      | 1      |

Comparing recipients with nonrecipients discards randomisation because receipt may depend on prognosis. ITT preserves the assignment comparison and answers the effect of offering or assigning the programme.

Under four strong instrumental-variable assumptions—random assignment, relevance, exclusion, and monotonicity—the Wald ratio identifies a local average treatment effect among compliers:

$$\hat{\tau}_{\text{LATE}} = \frac{E(Y \mid Z = 1) - E(Y \mid Z = 0)}{E(A \mid Z = 1) - E(A \mid Z = 0)}.$$

Exclusion says assignment affects the outcome only through receipt; monotonicity rules out defiers. LATE is not automatically the ATE. If assignment itself changes behaviour before receipt, exclusion can fail.

## 41.7 Per-protocol effects require more assumptions

A per-protocol question asks about sustained adherence to a defined strategy. Randomisation alone no longer identifies this effect when post-assignment prognostic factors influence adherence. Longitudinal standardisation or inverse-probability methods may be needed, with sequential exchangeability and positivity assumptions. “As treated” is an analysis label, not an identification strategy.

## 41.8 Attrition and missing outcomes

Compare follow-up by arm, document reasons, and distinguish administrative closure from outcome-related loss. Complete-case analysis can break exchangeability. Depending on the estimand and missingness structure, use:

- active outcome ascertainment;
- inverse-probability-of-observation weights;
- multiple imputation compatible with treatment and outcome models;
- bounds under extreme missing outcomes; and
- tipping-point analyses.

A precise estimate among the easily observed is not necessarily the trial effect.

## 41.9 A minimal randomised laboratory

The following experiment uses the hidden untreated and treated risks only to generate a fresh randomised outcome. It does not reveal both potential outcomes to the estimator.

```
trial_source = make_mhp_causal_data(n=1400, seed=6141, reveal_counterfactuals=True)
rng_trial = np.random.default_rng(41)
trial_source["assignment"] = rng_trial.binomial(1, 0.5, len(trial_source))
trial_source["trial_outcome"] = np.where(
 trial_source["assignment"] == 1,
 trial_source["y1"],
 trial_source["y0"],
)

arm = trial_source.groupby("assignment")["trial_outcome"].agg(["mean", "var", "count"])
trial_rd = arm.loc[1, "mean"] - arm.loc[0, "mean"]
trial_se = np.sqrt(arm.loc[1, "var"] / arm.loc[1, "count"] + arm.loc[0, "var"] / arm.loc[0, "count"])
print(f"Randomised risk difference: {trial_rd:.3f}")
print(f"Approximate 95% CI: [{trial_rd - 1.96*trial_se:.3f}, {trial_rd + 1.96*trial_se:.3f}]"
```

## 41.10 Build, break, and reflect

**Build.** Design a blocked trial of senior review. Specify assignment probability, blocks, unit, primary estimand, sample-size inputs, follow-up, missing-data strategy, analysis, and stopping rule.

**Break.** Allow reviewed projects to share one engineering team. Then allow 30% of assigned projects to decline. Which estimands remain identified by the simple difference in assignment means?

**Reflect.** Explain in two sentences why ITT can be small when treatment receipt has a large effect. Then explain why dividing by compliance changes the target population.

## Day 41 exit check

You are ready for Day 42 when you can:

- distinguish assignment, receipt, adherence, ITT, per-protocol effect, and LATE;
- choose project- versus cluster-level randomisation from the interference mechanism;
- explain why recipient comparisons abandon randomisation; and
- name assumptions needed beyond randomisation when outcomes are missing.

---

# Day 42 — Standardisation, Matching, and Weighting

---

When randomisation is unavailable, measured baseline covariates can support an adjusted comparison. The methods in this day are not alternative ways to make assumptions disappear. They are different computational expressions of identification under consistency, exchangeability, and positivity.

## 42.1 Three views of the same target

For the ATE, identification can be written as:

### Outcome standardisation

$$\psi = E_X\{m_1(X) - m_0(X)\}, \quad m_a(X) = E(Y \mid A = a, X).$$

### Inverse-probability weighting

With  $e(X) = P(A = 1 \mid X)$ ,

$$\psi = E \left\{ \frac{AY}{e(X)} - \frac{(1-A)Y}{1-e(X)} \right\}.$$

### Covariate balancing or matching

Construct treated and untreated comparisons with similar distributions of the confounders, then average an outcome contrast over the intended target population.

Standardisation models outcomes. Weighting models assignment. Matching designs a comparable subset or weighted comparison. Their agreement can be reassuring, but agreement does not test unmeasured exchangeability.

## 42.2 Regression standardisation, step by step

1. Fit  $m(A, X) = E(Y \mid A, X)$ .
2. Copy every row and set  $A = 1$ ; predict  $\hat{m}(1, X_i)$ .
3. Copy again and set  $A = 0$ ; predict  $\hat{m}(0, X_i)$ .
4. Average predictions in each intervention world.
5. Subtract the averages.

This produces a marginal risk difference even when the working model is logistic. The treatment coefficient alone is a conditional log odds ratio, not the ATE.

## 42.3 Propensity scores

Rosenbaum and Rubin defined the propensity score as

$$e(X) = P(A = 1 \mid X).$$

Under conditional exchangeability, treatment is exchangeable within levels of the true propensity score. Its purpose is **design and balance**, not maximum treatment-prediction accuracy. A propensity model that separates treatment perfectly creates unusable comparisons.

Common uses include:

- matching treated and untreated units on the score or a distance metric;
- stratifying by score quantiles;
- inverse-probability weighting;
- overlap weighting; and
- covariate-balancing estimation.

Never select the propensity model by treatment AUC alone. Inspect balance and overlap for the variables required by the causal design.

## 42.4 Matching changes the analysed population

Nearest-neighbour matching pairs similar units, often within a caliper on the logit propensity score. Matching with replacement can improve similarity but reuse controls. Without replacement, results can depend on match order. Discarding unmatched units commonly moves the estimand away from the original ATE.

A matching report should state:

- target estimand and eligible population;
- distance, caliper, ratio, replacement, and exact-match constraints;
- counts discarded and why;
- pre- and post-match balance;
- overlap and unmatched profiles;
- outcome analysis that respects pairs or weights; and
- sensitivity to reasonable design choices.

Do not inspect outcomes while repeatedly tuning a match to obtain the desired effect.

## 42.5 Weighting creates a pseudo-population

ATE weights are

$$w_i = \frac{A_i}{e(X_i)} + \frac{1 - A_i}{1 - e(X_i)}.$$

ATT weights keep treated units at weight 1 and weight controls by  $e(X)/(1 - e(X))$ . Stabilised weights multiply numerators by marginal treatment probabilities to reduce variance without changing the large-sample target under correct specification.

Extreme weights signal weak overlap, model misspecification, or both. Truncation may reduce variance but introduces bias and changes the procedure. Register thresholds or show a sensitivity curve rather than selecting a cutoff after seeing the effect.

Overlap weights, proportional to  $1 - e(X)$  for treated and  $e(X)$  for controls, emphasise profiles with assignment equiprobability. They can be stable, but estimate an overlap-population effect rather than the full ATE.

## 42.6 Balance, not propensity-model fit

For a continuous covariate  $X_j$ , a standardised mean difference is

$$\text{SMD}_j = \frac{\bar{X}_{1j} - \bar{X}_{0j}}{\sqrt{(s_{1j}^2 + s_{0j}^2)/2}}.$$

Compute weighted analogues after weighting. Inspect categorical levels, nonlinear terms, and important interactions too. A conventional absolute threshold such as 0.1 is a diagnostic convention, not proof of no confounding.

The effective sample size of nonnegative weights is

$$ESS = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

Two thousand rows with a few dominating weights may contain far less information than two thousand balanced observations.

## 42.7 Executable adjustment analysis

```
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

treatment = "senior_review_month6"
outcome = "warning_by_month36"
numeric = [
 "approval_year",
 "terrain_index",
 "estimated_cable_km",
 "contractor_experience_projects",
 "appraisal_budget_million_pkr",
 "remote_access",
]
categorical = ["district"]
covariates = numeric + categorical

def make_logistic_pipeline(numeric_columns, categorical_columns):
 transform = ColumnTransformer([
 ("num", StandardScaler(), numeric_columns),
 ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_columns),
])
 return Pipeline([
 ("transform", transform),
 ("model", LogisticRegression(max_iter=2000, C=1.0)),
])

Outcome regression and standardisation.
g_model = make_logistic_pipeline(numeric + [treatment], categorical)
g_model.fit(df[covariates + [treatment]], df[outcome])
```

```

x1 = df[covariates + [treatment]].copy()
x0 = x1.copy()
x1[treatment] = 1
x0[treatment] = 0
m1 = g_model.predict_proba(x1)[: , 1]
m0 = g_model.predict_proba(x0)[: , 1]
gcomp_ate = np.mean(m1 - m0)

Propensity model and Horvitz-Thompson IPW estimate.
e_model = make_logistic_pipeline(numeric, categorical)
e_model.fit(df[covariates], df[treatment])
e_hat = np.clip(e_model.predict_proba(df[covariates])[: , 1], 0.02, 0.98)
a = df[treatment].to_numpy()
y = df[outcome].to_numpy()
ipw_ate = np.mean(a * y / e_hat - (1 - a) * y / (1 - e_hat))

print(f"Naive risk difference: {naive:.3f}")
print(f"Standardised risk difference: {gcomp_ate:.3f}")
print(f"IPW risk difference: {ipw_ate:.3f}")
print(f"Estimated propensity range: [{e_hat.min():.3f}, {e_hat.max():.3f}]")

```

The logistic outcome model imposes linearity on the log-odds scale for continuous covariates. The propensity model likewise imposes structure. Add transformations and interactions based on the causal and measurement model, then evaluate diagnostics. Avoid choosing specification solely by which estimate is preferred.

## 42.8 Balance and weight diagnostics in code

```

def weighted_mean_variance(x, w):
 w = np.asarray(w, dtype=float)
 x = np.asarray(x, dtype=float)
 mean = np.sum(w * x) / np.sum(w)
 variance = np.sum(w * (x - mean) ** 2) / np.sum(w)
 return mean, variance

def smd_table(frame, treatment_values, weights=None):
 design = pd.get_dummies(frame, drop_first=False, dtype=float)
 weights = np.ones(len(design)) if weights is None else np.asarray(weights)
 rows = []
 for column in design.columns:
 x = design[column].to_numpy()
 m1_, v1_ = weighted_mean_variance(x[treatment_values == 1], weights[treatment_values == 1])
 m0_, v0_ = weighted_mean_variance(x[treatment_values == 0], weights[treatment_values == 0])
 denominator = np.sqrt((v1_ + v0_) / 2)
 smd = 0.0 if denominator < 1e-12 else (m1_ - m0_) / denominator
 rows.append((column, smd))
 return pd.DataFrame(rows, columns=["covariate", "smd"])

ate_weights = a / e_hat + (1 - a) / (1 - e_hat)
raw_balance = smd_table(df[covariates], a)
weighted_balance = smd_table(df[covariates], a, ate_weights)
balance = raw_balance.merge(weighted_balance, on="covariate", suffixes=("_raw", "_weighted"))
ess = ate_weights.sum() ** 2 / np.sum(ate_weights ** 2)

print(balance.assign(abs_weighted=lambda z: z.smd_weighted.abs())
 .sort_values("abs_weighted", ascending=False).head(8).to_string(index=False))
print(f"Weight range: [{ate_weights.min():.2f}, {ate_weights.max():.2f}]")
print(f"Effective sample size: {ess:.0f} of {len(df)}")

```

Also plot propensity distributions by treatment, inspect the profiles receiving the largest weights, and compare the weighted target population with the policy population. A single balance number cannot reveal every failure.

## 42.9 Research-paper study: Rosenbaum and Rubin (1983)

Read the original propensity-score paper with three questions:

1. What balancing result follows when treatment assignment is strongly ignorable given  $\mathbf{X}$ ?
2. Which assumptions enter before the propensity score is estimated?
3. Why does dimension reduction not make unmeasured confounding vanish?

Then reproduce a small simulation in which treatment depends on five covariates. Compare raw, matched, stratified, and weighted balance before examining outcomes.

## 42.10 Build, break, and reflect

**Build.** Estimate the review ATE by standardisation, stratification, matching, and weighting. Keep one registered covariate set.

**Break.** Remove terrain from every adjustment method. Force positivity failure by assigning review to every remote project. Add an unnecessary post-treatment mediator. Diagnose how each failure appears—or fails to appear—in balance and estimates.

**Reflect.** If matching discards 35% of reviewed projects, precisely describe the population to which the result applies.

### Day 42 exit check

You are ready for Day 43 when you can:

- derive standardisation and IPW from the same identification assumptions;
- distinguish ATE, ATT, and overlap weights;
- calculate an SMD and effective sample size;
- explain why propensity AUC is not the design objective; and
- treat extreme weights as a scientific warning, not only a numerical inconvenience.

---

# Day 43 — Doubly Robust Estimation and Sensitivity

---

## 43.1 Augmented inverse-probability weighting

Outcome regression can fail if  $m_a(\mathbf{X})$  is misspecified. Weighting can fail if  $e(\mathbf{X})$  is misspecified and becomes unstable near 0 or 1. Augmented inverse-probability weighting, or AIPW, combines them:

$$\hat{\psi}_{\text{AIPW}} = \frac{1}{n} \sum_{i=1}^n \left[ \hat{m}_1(X_i) - \hat{m}_0(X_i) + \frac{A_i\{Y_i - \hat{m}_1(X_i)\}}{\hat{e}(X_i)} - \frac{(1 - A_i)\{Y_i - \hat{m}_0(X_i)\}}{1 - \hat{e}(X_i)} \right].$$

The first term standardises outcome predictions. The residual terms correct those predictions using observed outcomes, scaled by assignment probability.

## 43.2 What “doubly robust” means

Under the causal assumptions and regularity conditions, AIPW is consistent if either:

- the propensity model is correct; or
- the outcome models are correct.

It does **not** mean:

- two wrong models cancel each other;
- unmeasured confounding is repaired;
- positivity no longer matters;
- treatment and outcome can be measured badly;
- standard errors are automatically valid; or
- trying many nuisance learners is free from researcher degrees of freedom.

The robustness is about two statistical nuisance functions inside an already identified problem.

## 43.3 Influence-function view

Define the estimated score

$$\hat{\phi}_i = \hat{m}_1(X_i) - \hat{m}_0(X_i) + \frac{A_i\{Y_i - \hat{m}_1(X_i)\}}{\hat{e}(X_i)} - \frac{(1 - A_i)\{Y_i - \hat{m}_0(X_i)\}}{1 - \hat{e}(X_i)}.$$

Then  $\hat{\psi} = n^{-1} \sum_i \hat{\phi}_i$ . With suitable nuisance estimation, an approximate standard error is

$$\widehat{SE}(\hat{\psi}) = \frac{SD(\hat{\phi}_i)}{\sqrt{n}}.$$

This expression reveals influential observations: residuals receive large multipliers where estimated treatment probabilities are small.

## 43.4 Why cross-fitting helps

Flexible models can overfit the same outcomes later used inside the effect score. In  $K$ -fold cross-fitting:

1. split observations into  $K$  folds;
2. fit nuisance models outside fold  $k$ ;
3. predict propensity and potential-outcome means for fold  $k$ ;
4. repeat until every observation has out-of-fold nuisance predictions;
5. compute the effect score once from those predictions.

Cross-fitting weakens empirical-process restrictions and reduces own-observation overfitting. It does not prevent leakage from preprocessing before the split, tune itself, or repair causal assumptions. All learned transformations belong inside each training fold.

## 43.5 Cross-fitted AIPW in code

```
from sklearn.model_selection import KFold

def cross_fitted_aipw(data, folds=5, seed=43):
 n = len(data)
```



```

a_ = data[treatment].to_numpy()
y_ = data[outcome].to_numpy()
e_ = np.empty(n)
m1_ = np.empty(n)
m0_ = np.empty(n)

splitter = KFold(n_splits=folds, shuffle=True, random_state=seed)
for train_idx, test_idx in splitter.split(data):
 train = data.iloc[train_idx]
 test = data.iloc[test_idx]

 propensity_model = make_logistic_pipeline(numeric, categorical)
 propensity_model.fit(train[covariates], train[treatment])
 e_[test_idx] = propensity_model.predict_proba(test[covariates])[:, 1]

 treated_model = make_logistic_pipeline(numeric, categorical)
 control_model = make_logistic_pipeline(numeric, categorical)
 treated_train = train[train[treatment] == 1]
 control_train = train[train[treatment] == 0]
 treated_model.fit(treated_train[covariates], treated_train[outcome])
 control_model.fit(control_train[covariates], control_train[outcome])
 m1_[test_idx] = treated_model.predict_proba(test[covariates])[:, 1]
 m0_[test_idx] = control_model.predict_proba(test[covariates])[:, 1]

e_ = np.clip(e_, 0.02, 0.98)
score = (
 m1_ - m0_
 + a_ * (y_ - m1_) / e_
 - (1 - a_) * (y_ - m0_) / (1 - e_)
)
estimate = score.mean()
standard_error = score.std(ddof=1) / np.sqrt(n)
return {
 "estimate": estimate,
 "standard_error": standard_error,
 "ci_low": estimate - 1.96 * standard_error,
 "ci_high": estimate + 1.96 * standard_error,
 "score": score,
 "propensity": e_,
 "m1": m1_,
 "m0": m0_,
}

aipw = cross_fitted_aipw(df)
print(
 f"Cross-fitted AIPW risk difference: {aipw['estimate']:.3f} "
 f"(95% CI {aipw['ci_low']:.3f} to {aipw['ci_high']:.3f})"
)

```

For clustered data, replace the individual-level variance or bootstrap with a method respecting independent clusters. With few clusters, asymptotic cluster-robust intervals can be unreliable. If nuisance learners are tuned, tuning belongs inside the training portion of each cross-fit split or inside a clearly separated design phase.

## 43.6 Targeted maximum likelihood and double machine learning

Targeted maximum likelihood estimation (TMLE) begins with initial outcome estimates  $\hat{m}_1(X)$  and  $\hat{m}_0(X)$  plus a propensity estimate  $\hat{e}(X)$ . For a binary outcome, a common update is:

1. clip initial probabilities away from 0 and 1;
2. construct the **clever covariate**  $H(A, X) = A/\hat{e}(X) - (1 - A)/(1 - \hat{e}(X))$ ;
3. fit a one-parameter logistic fluctuation of the observed-outcome prediction, using **logit**  $\hat{m}_A(X)$  as an offset and  $H$  as the covariate;
4. use the fitted fluctuation coefficient to update both counterfactual predictions; and

5. average updated  $\hat{m}_1(X) - \hat{m}_0(X)$ .

The targeting step chooses the update so the empirical influence-curve equation for the requested estimand is approximately solved. It can keep binary-outcome predictions inside  $[0,1]$ ; it does not validate the causal assumptions or rescue lack of overlap.

Double/debiased machine learning (DML) is a broader recipe:

1. write a score whose expectation identifies the target parameter;
2. choose nuisance functions needed by that score;
3. estimate nuisances on training folds and evaluate the score on held-out folds;
4. choose an **orthogonal** score whose first derivative with respect to small nuisance errors is zero at the truth; and
5. solve or average the cross-fitted score and estimate uncertainty at the independent sampling-unit level.

Orthogonality reduces the *first-order* effect of nuisance-estimation error; cross-fitting prevents each observation from being used to both train its nuisance prediction and evaluate its score. Neither says arbitrary machine learners are safe. The AIPW score implemented above is a canonical DML score for an ATE, so you have already executed the central mechanics even though no library branded “DML” was called.

These are frameworks, not magic brands. State the score, nuisance functions, folds, tuning, truncation, variance method, and target parameter.

## 43.7 Bootstrap the whole procedure

A bootstrap for procedure uncertainty should resample the independent unit, refit every nuisance model, redo cross-fitting and tuning according to the registered algorithm, and recompute the estimate. Bootstrapping only the final vector of predictions understates uncertainty from model fitting. Use a cluster or block bootstrap when independence lies at a higher level.

The influence-function interval above is fast and useful for this simulation. Compare it with a full-procedure bootstrap in the exercise. Large disagreement is a diagnostic, not an invitation to report whichever interval excludes zero.

## 43.8 Sensitivity to unmeasured confounding

No observed-data diagnostic proves exchangeability. A serious report asks how strong an omitted common cause would need to be to change the conclusion.

Useful approaches include:

- **bias parameters:** posit associations between an unmeasured confounder, treatment, and outcome; subtract the implied bias over a credible grid;
- **tipping points:** vary the prevalence and outcome risk of a missing factor until the estimate crosses a decision threshold;
- **E-values for ratio measures:** summarise the minimum confounder associations, on a risk-ratio scale under specific conditions, needed to explain an observed association;
- **Rosenbaum bounds:** assess how hidden assignment bias could alter randomisation-based conclusions in matched studies;
- **negative controls:** use an outcome not plausibly caused by treatment or an exposure not plausibly causing the outcome to detect some residual biases;
- **quantitative bias analysis:** propagate distributions over misclassification, selection, and confounding parameters; and
- **partial identification:** report bounds when point identification is indefensible.

An E-value is not a universal certificate. Negative controls can fail silently if they do not share the relevant bias mechanism. Sensitivity parameters should be calibrated against measured covariates and domain evidence, not selected for convenience.

## 43.9 Specification and overlap sensitivity

Run analyses that change one decision at a time:

| Sensitivity                  | Question                                               |
|------------------------------|--------------------------------------------------------|
| Alternative nonlinear terms  | Does functional-form choice drive the estimate?        |
| Pre-specified covariate sets | Does a disputed confounder materially change it?       |
| Weight truncation grid       | Is the conclusion carried by extreme weights?          |
| Overlap-population effect    | What is learned where treatment choice is realistic?   |
| Outcome definition           | Does a stricter warning definition reverse the result? |
| Missing-data scenarios       | How dependent is the result on unobserved outcomes?    |
| Negative-control analysis    | Is a shared residual bias detectable?                  |

Do not call a conclusion “robust” because every model using the same flawed identification strategy agrees.

## 43.10 Research papers: Bang–Robins and double machine learning

Bang and Robins (2005) explain doubly robust estimation across missing-data and causal settings. Chernozhukov and colleagues (2018) develop orthogonal scores and cross-fitting for high-dimensional nuisance estimation. For each paper, answer:

- What is the target parameter?
- Which nuisance functions are estimated?
- What product of estimation errors must shrink?
- What sampling structure supports the variance?
- Which causal assumptions are taken as given?

Then implement one deliberate failure: omit a nonlinear confounder from the outcome model but include it in the propensity model; reverse the misspecification; then misspecify both.

## 43.11 Build, break, and reflect

**Build.** Implement AIPW twice: once with logistic nuisance models and once with tree-based nuisance models. Preserve out-of-fold predictions and effect scores.

**Break.** Compute in-sample nuisance predictions, remove weight clipping, and omit terrain from both models. Which diagnostics react? Which causal failure remains invisible?

**Reflect.** Write a results paragraph containing the estimate, interval, estimand, identification assumptions, overlap population, and one quantitative sensitivity statement.

### Day 43 exit check

You are ready for Day 44 when you can:

- derive and code the AIPW score;
- define double robustness without overclaiming;
- explain the purpose and limits of cross-fitting;
- distinguish influence-function and full-procedure uncertainty; and
- design a sensitivity analysis whose parameters have substantive meaning.

## Day 44 — Target Trials and Time-Varying Treatment

**Mode: read and recognise.** Complete the protocol table, time-zero diagnosis, and strategy comparison. The g-formula, marginal structural model, and clone–censor–weight sections are design templates, not from-scratch software training.

### 44.1 Why observational studies need an imaginary protocol

A target trial is the randomised experiment we would conduct if it were ethical, feasible, and timely. Target-trial emulation makes an observational analysis imitate that protocol as closely as the data permit. It prevents a common error: comparing vaguely defined groups whose eligibility, treatment assignment, and follow-up begin at different times.

The protocol should specify:

| Component            | Question                                                                        |
|----------------------|---------------------------------------------------------------------------------|
| Eligibility          | Who could enter, and when are all criteria assessed?                            |
| Treatment strategies | What actions, timing, grace periods, and versions are compared?                 |
| Assignment           | What randomisation would be emulated; what covariates support exchangeability?  |
| Time zero            | When do eligibility, strategy assignment, and outcome follow-up align?          |
| Outcome              | What event, measurement, horizon, and competing-event rule apply?               |
| Causal contrast      | ITT-like assignment effect or per-protocol strategy effect?                     |
| Analysis             | How are confounding, adherence, censoring, clustering, and uncertainty handled? |

The month-6 review study aligns these pieces: eligibility is assessed at month 6, review is assigned at month 6, and warning follow-up begins immediately afterward. Calling review “baseline” while selecting projects using month-12 survival would not align them.

### 44.2 Time-zero bias

Suppose reviewed projects are classified as treated if they receive review anytime in the first year, but follow-up starts at approval. To enter the treated group, a project must remain active and warning-free until its eventual review. The pre-review time is immortal with respect to the classification: an early warning would prevent the project from being labelled treated. Assigning that guaranteed event-free time to treatment can make review look protective.

Solutions depend on the question:

- set time zero when eligibility and treatment are both defined;
- treat review as time-varying;
- use sequential trials at repeated eligibility times;
- clone units into compatible strategies and censor when they deviate; or
- specify a grace period and analyse it with methods that avoid immortal-time assignment.

Simply adding “time to review” as a baseline covariate does not repair the design.

### 44.3 Longitudinal notation

At visits  $k = 0, 1, \dots, K$ , let:

- $L_k$  be covariates observed before the treatment decision at visit  $k$ ;
- $A_k$  be treatment during interval  $k$ ;
- $Y$  be the final outcome;
- $\bar{L}_k = (L_0, \dots, L_k)$  and  $\bar{A}_k = (A_0, \dots, A_k)$  be histories.

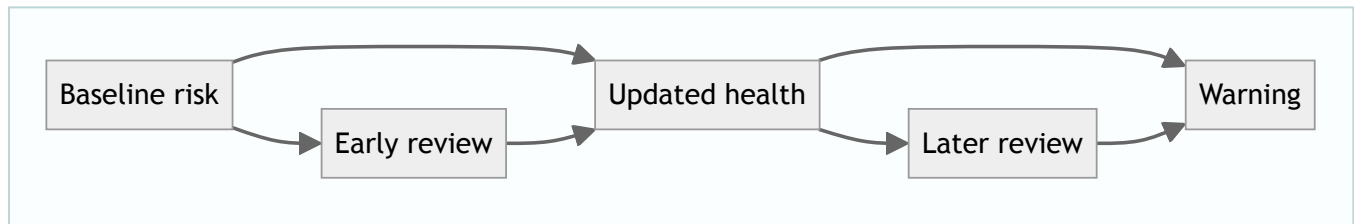
A dynamic strategy  $d$  maps history to treatment, for example:

assign senior review when the current forecast ratio exceeds 1.10 and engineering capacity is available.

The potential outcome  $Y^d$  is the outcome under sustained adherence to rule  $d$ . A causal contrast might be  $E(Y^{d_1} - Y^{d_0})$ .

### 44.4 Treatment-confounder feedback

Consider monthly project health  $L_1$ . Earlier review  $A_0$  affects health, and health affects later review  $A_1$  and the outcome:



If we adjust conventionally for  $L_1$ , we block part of  $A_0$ 's effect. If we do not adjust,  $L_1$  confounds  $A_1$ . This treatment-confounder feedback motivates the longitudinal g-formula and marginal structural models.

### 44.5 Sequential identification assumptions

For every visit and treatment history, longitudinal identification requires versions of:

#### Sequential exchangeability

$$Y^{\bar{a}} \perp A_k \mid \bar{L}_k, \bar{A}_{k-1}.$$

Given measured history, the current decision is independent of future potential outcomes.

## Sequential positivity

Every treatment required by the strategies has positive probability within histories occurring in the target population.

## Longitudinal consistency

Observed outcomes under the observed treatment history equal the corresponding potential outcomes, and strategies are well defined.

Interference and informative censoring also need attention. These assumptions grow harder—not easier—as history becomes richer.

## 44.6 Parametric g-formula

The g-formula simulates the outcome distribution under an intervention strategy:

1. Fit models for each time-varying covariate  $L_{k+1}$  given past history.
2. Fit an outcome or event model given treatment and covariate history.
3. Sample baseline units from the target population.
4. Set treatment according to strategy  $d$  at visit 0.
5. Simulate the next covariates from their fitted conditional distributions.
6. Set the next treatment according to  $d$ .
7. Continue through follow-up and simulate outcomes.
8. Average outcomes; repeat for each strategy and contrast them.

This is standardisation across entire longitudinal histories. It requires correct models for relevant covariate and outcome processes. Simulation Monte Carlo error should be made negligible relative to sampling uncertainty.

## 44.7 Marginal structural models

Inverse-probability treatment weights create a pseudo-population in which observed treatment history is independent of measured time-varying confounder history. A stabilised weight through visit  $K$  is conceptually

$$SW_i = \prod_{k=0}^K \frac{P(A_{ik} \mid \bar{A}_{i,k-1}, L_{i0})}{P(A_{ik} \mid \bar{A}_{i,k-1}, \bar{L}_{ik})}.$$

Additional inverse-probability-of-censoring factors address measured informative loss. Fit a marginal structural model in the weighted data, such as a pooled logistic model for discrete-time risk under treatment history.

Diagnostics include:

- numerator and denominator probabilities by visit;
- weight distribution, truncation sensitivity, and effective sample size;
- balance of histories after weighting;
- strategy support over time;
- model calibration for treatment and censoring; and
- whether a few rare histories dominate late follow-up.

Weights can multiply into instability. A large raw cohort may support only a short-horizon or restricted strategy contrast.

## 44.8 Clone–censor–weight for sustained strategies

When a unit initially remains compatible with several strategies, create one clone per strategy. Artificially censor a clone when its observed treatment history deviates from that strategy. Weight uncensored clones by the inverse probability of remaining compatible, conditional on measured history. Then compare weighted strategy-specific outcomes.

This approach makes protocol deviations explicit but does not make artificial censoring ignorable without the censoring model. Standard errors must account for clones originating from the same unit.

## 44.9 Repeated target trials

If monthly records are available, emulate a sequence of trials:

1. At each month, find projects meeting eligibility.
2. Classify assignment at that month's time zero.
3. Start a new follow-up record.
4. Pool trials with trial indicators or time trends.
5. Use project-clustered inference because one project may enter multiple trials.

This increases information and makes eligibility transparent. It can also mix changing policy periods and treatment versions, so calendar-time adjustment and transport questions remain.

## 44.10 Censoring and competing events

For a causal risk by month 36, decide what cancellation means:

- **Total effect on real-world warning incidence:** cancellation remains a competing event; the outcome is warning before cancellation by the horizon.
- **Hypothetical effect eliminating cancellation:** requires identification of a world in which cancellation is prevented and stronger assumptions.
- **Composite effect:** combine warning and cancellation when both represent policy failure.
- **Separable effects:** decompose treatment components under specialised assumptions.

Calling cancellation “censoring” quietly changes the causal question and may introduce informative censoring. Chapter 5's event-process discipline still applies.

## 44.11 Research papers: target trials and marginal structural models

Hernán and Robins (2016) show how explicitly emulating a target trial can prevent avoidable design errors in observational work. Robins, Hernán, and Brumback (2000) develop marginal structural models for time-dependent treatment and confounding. Read them together and ask:

- Where is time zero?
- Which variables are measured before each decision?
- What would make a strategy impossible for some histories?
- Is the estimand assignment-like or adherence-like?
- Which data expansion and weighting choices are part of the procedure?

## 44.12 Build, break, and reflect

**Build.** Write a target-trial table comparing “review by month 7” with “no review through month 12” among projects eligible at month 6. Specify the grace-period analysis.

**Break.** Define treated projects using any review through month 18 while starting outcomes at approval. Explain every guaranteed-survival interval and selection event.

**Reflect.** Decide whether cancellation is competing, composite, censoring, or hypothetical for your policy question. Defend the choice.

## Day 44 exit check

You are ready for Day 45 when you can:

- align eligibility, assignment, and follow-up at time zero;
- identify immortal-time bias from group definitions;
- explain treatment-confounder feedback;
- compare the longitudinal g-formula with marginal structural models; and
- state a longitudinal estimand with a realistic strategy and competing-event rule.

---

# Day 45 — Quasi-Experimental Designs

---

**Mode: read and recognise.** Derive the small two-period DiD and binary-instrument contrasts and audit each design's assumptions. Production estimators for staggered DiD, robust RD, synthetic control, and serially correlated panels require dedicated follow-up study.

Quasi-experimental designs exploit assignment mechanisms or discontinuities that can be more credible than adjustment for all measured confounders. Each identifies a particular effect for a particular population. The design, not the estimator's sophistication, carries the causal argument.

## 45.1 Difference-in-differences

Suppose some districts introduce mandatory senior review in 2023 and others do not. With one pre and one post period, the difference-in-differences contrast is

$$\hat{\tau}_{\text{DiD}} = (\bar{Y}_{T,\text{post}} - \bar{Y}_{T,\text{pre}}) - (\bar{Y}_{C,\text{post}} - \bar{Y}_{C,\text{pre}}).$$

It subtracts the control group's time change from the treated group's time change. Identification requires a parallel-trends assumption: absent the policy, the treated group's average untreated outcome trend would have matched the comparison group's trend on the chosen scale.

Parallel trends concerns counterfactual post-policy outcomes and cannot be proven by pre-trends. Multiple stable pre-periods can reveal obvious violations and help model seasonality, but a nonsignificant pre-trend test is weak reassurance.

Threats include:

- anticipation before formal implementation;
- composition changes in eligible projects;
- district-specific shocks concurrent with policy;
- outcome measurement changing with review;
- spillovers into comparison districts;



- functional-form dependence; and
- serial correlation with incorrectly small standard errors.

## 45.2 Event studies and staggered adoption

An event study estimates effects relative to treatment timing, with a pre-policy reference period. It can display dynamics and pre-period differences. In staggered adoption, ordinary two-way fixed-effects regression may combine comparisons with inappropriate or negative weights when effects vary across cohorts and time.

Use estimators designed for group-time average treatment effects, define never-treated or not-yet-treated comparisons, aggregate transparently, and report support by cohort and event time. Callaway and Sant'Anna's framework is one principled approach.

## 45.3 Regression discontinuity

Suppose projects with a pre-review risk score  $R \geq c$  must receive senior review. A sharp regression-discontinuity design compares projects just above and below cutoff  $c$ :

$$\tau_{RD} = \lim_{r \downarrow c} E(Y \mid R = r) - \lim_{r \uparrow c} E(Y \mid R = r).$$

The effect is local to the cutoff. Identification requires potential outcomes to vary continuously through the threshold in the absence of treatment and no precise manipulation of the running variable around the cutoff.

Good practice includes:

- plot raw outcome means and treatment probability against the running variable;
- verify the score and cutoff were determined before outcomes;
- inspect density for suspicious sorting near the threshold;
- inspect continuity of pre-treatment covariates as falsification evidence;
- fit local polynomials, usually low order, on each side;
- use data-driven but defensible bandwidths with robust bias-corrected inference;
- report sensitivity across reasonable bandwidths and kernels; and
- avoid high-order global polynomials.

In a fuzzy RD, crossing the cutoff changes treatment probability rather than determining treatment. The discontinuity in outcome divided by the discontinuity in treatment identifies a local complier effect under IV-like assumptions.

## 45.4 Instrumental variables

An instrument  $Z$  shifts treatment  $A$  but is otherwise excluded from the outcome process. For a binary instrument, the Wald estimand is

$$\frac{E(Y \mid Z = 1) - E(Y \mid Z = 0)}{E(A \mid Z = 1) - E(A \mid Z = 0)}.$$

Core assumptions are:

1. **Relevance:**  $Z$  changes treatment probability.
2. **Independence:**  $Z$  is as-if randomly assigned relative to potential outcomes.
3. **Exclusion:**  $Z$  affects the outcome only through treatment.
4. **Monotonicity:** the instrument does not make some units move systematically opposite its encouragement.

Under heterogeneous effects these support LATE among instrument-induced compliers, not a population ATE. A weak first stage creates unstable estimates and nonstandard finite-sample behaviour. A strong statistical association between instrument and treatment does not defend exclusion.

Potential instruments such as distance to a review office are often questionable: distance can affect procurement and outcomes directly. Write and attack the DAG before calculating a two-stage regression.

## 45.5 Synthetic controls and interrupted time series

When one region adopts a policy, a synthetic control combines untreated regions to reproduce its pre-policy trajectory and predictors. Credibility depends on a long, stable pre-period, unaffected donors, good pre-treatment fit, and transparent placebo analyses.

An interrupted time series estimates a level or slope change at intervention time. It needs enough pre and post observations, control of seasonality and autocorrelation, a stable measurement process, and no concurrent intervention that explains the break. Adding a comparison series strengthens the design.

## 45.6 Design comparison

| Design               | Assignment leverage                    | Typical estimand                       | Central vulnerability                            |
|----------------------|----------------------------------------|----------------------------------------|--------------------------------------------------|
| Covariate adjustment | Measured baseline causes               | ATE/ATT/overlap effect                 | Unmeasured confounding                           |
| DiD                  | Change over time plus comparison group | Effect for treated groups/periods      | Nonparallel counterfactual trends                |
| RD                   | Threshold rule                         | Local effect at cutoff                 | Manipulation or discontinuous potential outcomes |
| IV                   | Exogenous encouragement                | Local complier effect                  | Exclusion, independence, weak first stage        |
| Synthetic control    | Weighted untreated trajectory          | Effect for treated unit after adoption | Poor donor fit, concurrent shocks                |
| Randomised trial     | Known assignment                       | Assignment effect in trial population  | Nonadherence, attrition, interference, transport |

Never describe DiD, RD, or IV as “natural randomisation” without defending exactly what is as-if random and for whom.

## 45.7 Placebos and falsification

Useful design-specific checks include:

- fake policy dates in DiD or interrupted time series;
- outcomes that the intervention should not affect;
- pre-treatment “effects” in an event study;
- fake RD cutoffs away from the assignment threshold;
- continuity of baseline covariates at the RD cutoff;
- instrument associations with pre-treatment outcomes; and
- leave-one-donor-out synthetic controls.

Passing falsification tests raises credibility but does not prove the identifying assumption. Failing a well-chosen test demands investigation.

## 45.8 Research papers: IV, RD, and modern DiD

Study three papers as a design trilogy:

- Angrist, Imbens, and Rubin (1996) connect instruments to potential outcomes and local effects.
- Hahn, Todd, and van der Klaauw (2001) formalise identification in regression-discontinuity designs.
- Callaway and Sant'Anna (2021) identify and aggregate group-time effects with multiple treatment periods.

For each, write one paragraph containing the estimand, comparison population, assignment leverage, identifying assumption, diagnostic evidence, and failure mode. If the paragraph cannot name the population, the design has not been understood.

## 45.9 Build, break, and reflect

**Build.** Design three hypothetical senior-review studies: threshold assignment, staggered district mandate, and lottery encouragement. For each, define the estimand and analysis before discussing software.

**Break.** Allow managers to manipulate the threshold score, introduce a district funding reform in the same year as review, and let lottery notification motivate projects even without review. Match each contamination to the assumption it threatens.

**Reflect.** Which design estimates the most policy-relevant effect? Which makes the most defensible assumptions? These need not be the same.

### Day 45 exit check

You are ready for Day 46 when you can:

- derive the two-period DiD and binary-instrument Wald contrasts;
- state why pre-trends do not prove parallel trends;
- interpret RD as local to a cutoff;
- distinguish LATE from ATE; and
- choose diagnostics that follow from a design's actual failure modes.

---

# Day 46 — Heterogeneous Effects and Policy Learning

---

**Mode: guided build plus survey.** Build the supplied pre-specified subgroup analysis. Meta-learners and causal forests are recognition-level extensions unless you deliberately undertake the optional software lab in §46.12.

Average effects can hide meaningful variation. Yet subgroup exploration is one of the easiest places to manufacture findings. Research-level heterogeneity analysis begins with an estimand and validation plan, not a colourful tree.

## 46.1 Conditional effects are still averages

The conditional average treatment effect is

$$\tau(x) = E\{Y(1) - Y(0) \mid X = x\}.$$

It is an average among units with profile  $x$ , not an observed individual causal effect. Even in a randomised trial, both potential outcomes for a person remain unobserved. Avoid statements such as “the model knows this project will benefit.”

## 46.2 Confirmatory versus exploratory heterogeneity

A confirmatory analysis should pre-specify:

- a small set of scientifically credible effect modifiers;
- their coding and functional form;
- the effect scale;
- the interaction contrast;
- multiplicity control or hierarchical testing;
- subgroup sample-size and overlap requirements; and
- how missing modifiers are handled.

Exploratory discovery is valuable when labelled as hypothesis generation and validated in independent data. A significant effect in one subgroup and nonsignificant effect in another does not establish interaction; test their difference directly.

## 46.3 Interaction on one scale need not exist on another

Suppose baseline risk is higher in remote projects. A constant risk ratio can yield a larger absolute risk reduction in that subgroup. A constant risk difference can yield different ratios. Policy capacity often depends on absolute benefit, while mechanisms may be studied on another scale. Register the scale and report baseline risks alongside effects.

## 46.4 Pre-specified subgroup estimation with AIPW scores

The AIPW score has conditional expectation related to the conditional effect when nuisance functions are valid. For the pre-specified terrain modifier in this simulation, compare mean scores by baseline group and test the interaction contrast.

```
high_terrain = (df["terrain_index"] > 0.75).to_numpy()

def mean_and_se(values):
 values = np.asarray(values)
 return values.mean(), values.std(ddof=1) / np.sqrt(len(values))

low_effect, low_se = mean_and_se(aipw["score"][~high_terrain])
high_effect, high_se = mean_and_se(aipw["score"][high_terrain])
interaction = high_effect - low_effect
interaction_se = np.sqrt(low_se ** 2 + high_se ** 2)

print(f"Lower-terrain ATE: {low_effect:.3f} (SE {low_se:.3f})")
print(f"High-terrain ATE: {high_effect:.3f} (SE {high_se:.3f})")
print(
 f"Interaction contrast: {interaction:.3f} "
 f"(95% CI {interaction - 1.96*interaction_se:.3f} to "
 f"{interaction + 1.96*interaction_se:.3f})"
)
```

This variance calculation treats the two groups as independent and the cross-fitted score approximation as adequate. A clustered or multi-stage study needs a variance procedure matching its design. One simulated finding is not confirmation.

## 46.5 Meta-learners

Common machine-learning constructions include:

- **S-learner:** one outcome model  $m(A, X)$ , with treatment as a feature;
- **T-learner:** separate outcome models  $m_1(X)$  and  $m_0(X)$ ;
- **X-learner:** impute treatment effects and combine treatment-specific effect models, often helpful with imbalanced assignment;
- **R-learner:** residualise outcome and treatment using nuisance models, then learn heterogeneity from an orthogonal loss; and
- **DR-learner:** regress a doubly robust pseudo-outcome on effect modifiers.

Their performance depends on smoothness, sample size by arm, overlap, tuning, and the true effect structure. Predictive cross-validation loss does not automatically select the best CATE estimator because individual effects have no observed label.

## 46.6 Causal trees and forests

Athey and Imbens introduced honest causal trees that use separate observations to choose partitions and estimate leaf effects. Honesty reduces adaptive bias at the cost of some sample efficiency. Causal forests aggregate many such trees and can provide asymptotically grounded conditional-effect estimates under conditions.

Inspect:

- treatment and control counts within leaves;
- overlap across predicted-benefit regions;
- calibration of CATE predictions in held-out data using group summaries;
- stability across seeds and sample splits;
- variable importance under correlated features;
- uncertainty and multiplicity; and
- whether discovered groups correspond to actionable pre-treatment information.

A complex forest cannot identify effects in regions where only one treatment occurs.

## 46.7 Evaluating heterogeneity without individual-effect labels

Possible held-out diagnostics include:

- rank units by predicted effect, then estimate observed causal contrasts in pre-specified quantile groups using a valid score;
- test whether predicted CATEs calibrate against doubly robust scores;
- estimate the value of a learned policy using out-of-fold or external data;
- compare against simple constant-effect and pre-specified interaction baselines; and
- repeat the full discovery procedure across sites or calendar periods.

Using the same outcomes to discover a subgroup and claim its effect exaggerates evidence. Honest sample splitting, nested resampling, or external confirmation is part of the estimand-to-procedure contract.

## 46.8 From CATE to a treatment policy

If treatment has cost  $c(X)$  and outcome benefit is valued at  $b$ , a simple rule might treat when

$$-b\hat{\tau}(X) > c(X),$$

for an adverse binary outcome where negative  $\tau$  is beneficial. With limited capacity, assign the highest expected net-benefit projects subject to fairness, geography, and operational constraints.

The value of policy  $d$  is

$$V(d) = E[Y\{d(X)\}]$$

or a utility-transformed version. Estimate value on data not used to optimise the rule, using randomisation probabilities or doubly robust off-policy estimators. Compare against realistic policies—usual practice, treat all, treat none, or a transparent score—not a straw man.

## 46.9 Fairness, capacity, and treatment effects

Predictive parity metrics do not directly answer whether an intervention is distributed fairly. Ask:

- Is the treatment beneficial, burdensome, or both?
- Which groups were represented with treatment overlap?
- Does the policy allocate scarce benefit or intrusive scrutiny?
- Are protected attributes effect modifiers, proxies for structural conditions, or constraints on allocation?
- Will action change future labels and access to treatment?
- Who bears errors, delays, and monitoring burdens?

Do not remove a protected attribute automatically: omission can hide disparities or worsen confounding. Its use requires legal, ethical, causal, and governance analysis appropriate to the setting.

## 46.10 Transporting heterogeneous effects

To move an effect across populations, measure variables that both modify treatment effect and differ between source and target. Standardise source conditional effects over the target covariate distribution when support exists. If rural terrain values in the target never occurred in the study, this is extrapolation, not transport evidence.

Report the target population's covariate support, treatment versions, outcome measurement, implementation fidelity, and baseline risk. A model that transports predictions need not transport causal effects, and vice versa.

## 46.11 Research papers: honest trees and causal forests

Read Athey and Imbens (2016) on recursive partitioning for heterogeneous causal effects, then Wager and Athey (2018) on causal forests. Trace exactly where honesty, subsampling, nuisance adjustment, and asymptotic arguments enter. Reproduce a constant-effect simulation: a useful heterogeneity method should not invent stable subgroups when none exist.

## 46.12 Build, break, and reflect

**Build.** Register terrain and contractor experience as two effect modifiers. Extend the supplied AIPW-score analysis to report both modifiers separately, their pre-specified interaction contrasts, group counts, overlap summaries, and confidence intervals on held-out folds.

**Optional software lab—not required for chapter mastery.** After consulting the current official installation and API documentation for one dedicated package such as EconML or R `grf`, compare one DR-learner or causal forest with the simple registered interaction baseline. Record the exact package version and use the package's documented inference assumptions. Do not install a causal library merely to complete the required exercise.

**Break.** Search 100 random subgroups and report the smallest p-value. Then repeat on a new sample. Quantify the discovery's instability.

**Reflect.** Write a policy rule with a capacity constraint and a no-treatment option. State who is outside its evidence base.

## Day 46 exit check

You are ready for Day 47 when you can:

- distinguish an individual effect from a CATE;
- test an interaction rather than compare two significance labels;
- explain honesty in causal trees;
- evaluate a learned policy out of sample; and
- separate effect transport from prediction transport.

# Day 47 — Comparison, Reproducibility, and Responsible Production

---

A causal estimate can be well identified yet irreproducible. A predictive model can be reproducible yet useless or harmful. Research operations preserve the full chain from question to action so that another person can audit both.

## 47.1 What exactly is being compared?

Before a statistical test, define the comparison unit and loss:

- Are the candidates fixed algorithms, or the complete tuning procedures?
- Do they receive the same training observations and feature information?
- Is preprocessing learned inside each training split?
- Is the metric calculated per observation, district, time block, or dataset?
- Is the scientific target average performance, worst-group performance, calibration, policy value, compute cost, or a noninferiority margin?
- Is the destination internal, temporal, geographic, or prospective?

Comparing one tuned model's best run with another model's default run is not an algorithm comparison. It is a comparison of unequal procedures.

## 47.2 Paired evaluation

If two procedures are evaluated on the same held-out observations or resampling splits, compare paired losses. For split  $r$ ,

$$d_r = L_{A,r} - L_{B,r}.$$

The sign and uncertainty of  $\bar{d}$  matter more than two separate metric intervals. Pairing removes variation shared by the same difficult test cases.

Whenever possible, retain observation-level loss differences on a final untouched test set. Bootstrap the independent sampling unit—project, patient, district, or time block—and refit models if the target is procedure performance rather than conditional performance of already fitted artifacts.

## 47.3 Why ordinary tests on cross-validation folds fail

Training sets in cross-validation overlap. Fold differences are therefore dependent. A one-sample t-test treating  $K$  fold differences as independent is too optimistic. A Wilcoxon signed-rank test changes the distributional target but does **not** remove this dependence merely because it is nonparametric.

Nadeau and Bengio proposed corrected variance approximations for resampled comparisons. For  $r$  repeated train/test evaluations with test-to-train size ratio  $q$ , a commonly used form is

$$\widehat{SE}_{corr}(\bar{d}) = \sqrt{\left(\frac{1}{r} + q\right) s_d^2}.$$

The approximation depends on the resampling design and does not create many independent datasets. With one benchmark dataset, conclusions about broad algorithm superiority should remain modest. Prefer multiple independent datasets for general algorithm claims and a locked destination set for a specific deployment claim.

```
from scipy.stats import t as student_t

def corrected_resampled_interval(differences, test_train_ratio, confidence=0.95):
 differences = np.asarray(differences, dtype=float)
 r = len(differences)
 mean_difference = differences.mean()
 corrected_se = np.sqrt((1 / r + test_train_ratio) * differences.var(ddof=1))
 critical = student_t.ppf((1 + confidence) / 2, df=r - 1)
 return mean_difference, corrected_se, (
 mean_difference - critical * corrected_se,
 mean_difference + critical * corrected_se,
)

example_differences = np.array([0.21, 0.08, -0.03, 0.17, 0.11,
 0.05, 0.19, -0.02, 0.07, 0.13])
mean_d, se_d, ci_d = corrected_resampled_interval(
 example_differences, test_train_ratio=0.25
)
print(f"Mean paired loss difference: {mean_d:.3f}")
print(f"Corrected SE: {se_d:.3f}; 95% CI: [{ci_d[0]:.3f}, {ci_d[1]:.3f}]"
```

This calculation is a teaching approximation, not a universal test. Report the split generator, repetition count, correction, metric, seeds, and all paired differences.

## 47.4 Practical equivalence and multiplicity

Statistical significance is not practical importance. Register a smallest relevant difference  $\Delta$ . A noninferiority question may ask whether a simpler model is no worse than a complex one by more than  $\Delta$ , while offering lower latency or better interpretability.

If many models, outcomes, subgroups, horizons, and metrics are compared, the chance of a favourable result rises. Pre-specify one primary comparison, control multiplicity for confirmatory secondary claims, and label the rest exploratory. A leaderboard is a multiple-testing machine when repeatedly consulted during development.



## 47.5 Ablations answer a procedural question

An ablation removes or changes one component of a pipeline:

| Run                    | Change                             | What can be learned                                          |
|------------------------|------------------------------------|--------------------------------------------------------------|
| Full procedure         | None                               | Reference performance                                        |
| No engineered features | Remove registered feature block    | Incremental predictive value of that block in this procedure |
| No log transformation  | Use original target scale          | Dependence on target transformation                          |
| No calibration         | Retain raw probabilities           | Effect of the calibration stage                              |
| No cross-fitting       | Use in-sample nuisance predictions | Sensitivity to own-observation fitting                       |

Hold every other element—including data split, tuning budget, and seed schedule—constant. Repeat enough splits to quantify interaction with sampling.

An ablation is not automatically a causal effect of a component. Removing engineered features may change optimisation, regularisation, and downstream hyperparameters. Write “the registered procedure performed worse without the feature block,” not “the feature block caused a 2.6-point improvement in every context.”

## 47.6 Reproducibility has levels

Terminology varies across fields, but distinguish at least:

- **computational repeatability:** the same team, code, data, and environment regenerate the result;
- **computational reproducibility:** another team regenerates it from shared artifacts;
- **robustness:** reasonable analytic choices do not overturn the bounded conclusion;
- **replication:** new data and an independent implementation address the same claim; and
- **generalisation or transport:** the conclusion applies at a new destination.

A fixed random seed helps repeatability. It does not prove robustness or replication.

## 47.7 The research bundle

Every final study should preserve:

1. **Protocol or registration** with version and timestamp.
2. **Data provenance:** sources, inclusion flow, extraction query, units, timestamps, schema, licence, and known measurement changes.
3. **Immutable data manifest** with row counts, date range, schema version, and cryptographic hashes where permitted.
4. **Executable code** from raw inputs to tables and figures.
5. **Environment lock:** interpreter, operating system, package versions, and hardware-relevant settings.
6. **Configuration** containing seeds, folds, outcomes, estimands, thresholds, and hyperparameter spaces.
7. **Results manifest** mapping every reported number to an output artifact.
8. **Tests** for cohort construction, time alignment, leakage, estimator invariants, and expected dimensions.
9. **Deviation log** recording every departure from the protocol before its result is interpreted.
10. **Data sheet, model card, and causal-study card** describing intended use and identification limits.

Do not rely on notebook execution order or an analyst's memory.

## 47.8 Seeds and determinism

Record seeds for data splitting, model initialisation, simulation, and bootstrap. Some parallel numerical libraries remain nondeterministic because reduction order or hardware kernels vary. Report whether exact bitwise equality is expected or only tolerance-level numerical agreement.

Seed sensitivity is a diagnostic. If the headline conclusion appears only for seed 42, the procedure is unstable.

## 47.9 Hashes, environments, and manifests

```
import hashlib
import json
import platform
import sklearn
import scipy

def sha256_bytes(payload):
 return hashlib.sha256(payload).hexdigest()

analysis_config = {
 "chapter": 6,
 "n": len(df),
 "seed": 6060,
 "estimand": "ATE risk difference by month 36",
 "treatment": treatment,
 "outcome": outcome,
 "cross_fit_folds": 5,
 "propensity_clip": [0.02, 0.98],
}
config_bytes = json.dumps(analysis_config, sort_keys=True).encode("utf-8")
manifest = {
 "config_sha256": sha256_bytes(config_bytes),
 "python": platform.python_version(),
 "numpy": np.__version__,
 "pandas": pd.__version__,
 "scipy": scipy.__version__,
 "scikit_learn": sklearn.__version__,
}
print(json.dumps(manifest, indent=2, sort_keys=True))
```

For sensitive data, publish hashes and synthetic fixtures without publishing protected records. A hash confirms byte identity, not data quality.

## 47.10 Data sheets, model cards, and causal-study cards

Geburu and colleagues proposed datasheets for documenting datasets; Mitchell and colleagues proposed model cards for model reporting. Use both, then add a causal-study card.

### Data sheet

- motivation, creators, funding, and collection process;
- population, sampling, inclusion, exclusion, and consent or authority;
- variables, units, timing, missingness, labels, and measurement error;
- transformations, linkage, deduplication, and update schedule;
- sensitive attributes and foreseeable harms;
- licences, access controls, retention, and deletion; and
- known shifts, gaps, and prohibited uses.

## Model card

- model version, owners, dependencies, and training date;
- intended users, decisions, population, and out-of-scope uses;
- training and evaluation destinations;
- discrimination, calibration, uncertainty, subgroup, and decision metrics;
- thresholds, latency, resource use, and fallback;
- ethical considerations, limitations, and monitoring; and
- change history and retirement conditions.

## Causal-study card

- target trial, estimand, treatment versions, and outcome scale;
- DAG and adjustment set;
- identification assumptions and their evidence;
- positivity, balance, missingness, and interference diagnostics;
- estimator, nuisance models, cross-fitting, and variance;
- sensitivity analyses and negative controls;
- target population and transport limits; and
- claims the design does **not** support.

## 47.11 Training–serving consistency

Production must execute the same transformations learned during training. Persist the complete pipeline, not only final coefficients. Validate:

- schema names, types, units, ranges, and category handling;
- missing-value conventions and timestamps;
- feature availability at decision time;
- transformation parameters learned only from training data;
- probability calibration and threshold version;
- dependency and model artifact hashes; and
- parity between batch, online, and shadow predictions on identical fixtures.

Serialising an untrusted artifact can execute code in some ecosystems. Use approved formats, signatures, access controls, and isolated loading appropriate to the threat model.

## 47.12 Monitor a decision system, not only a score

Monitoring should cover:

| Layer           | Examples                                                            |
|-----------------|---------------------------------------------------------------------|
| Service         | latency, errors, missing features, schema failures                  |
| Input           | ranges, categories, missingness, joint support, data freshness      |
| Prediction      | score distribution, uncertainty, calibration when labels arrive     |
| Decision        | action rate, overrides, queue length, capacity, subgroup allocation |
| Outcome         | delayed performance, censoring, competing events, utility, harms    |
| Causal feedback | treatment changes labels, selective labels, strategic behaviour     |

| Layer      | Examples                                                         |
|------------|------------------------------------------------------------------|
| Governance | incidents, access, appeals, rollback readiness, audit completion |

A Kolmogorov–Smirnov test or population stability index can flag marginal feature shifts. Neither proves model failure. Large samples make tiny shifts statistically significant; small samples miss important changes; univariate checks miss joint drift. Connect alerts to plausible failure modes and actions.

## 47.13 Feedback loops

Once a risk score triggers senior review, later warnings are outcomes under the policy influenced by the score. Retraining on those labels without treatment information can teach that high-risk projects do well and suppress future intervention. This is performative prediction and selective-label feedback.

Log score, action, treatment version, eligibility, override, outcome-observation process, and timestamps. Use randomised exploration where ethical, phased rollouts, or causal adjustment to evaluate policy changes. A production model may need an impact trial, not just another retrospective AUC.

## 47.14 Retraining, rollback, and retirement

Do not retrain automatically because one drift statistic crosses a threshold. A retraining gate should ask:

1. Is the alert a data incident, population change, policy change, or ordinary variation?
2. Are outcomes sufficiently mature to evaluate the current model?
3. Does a challenger improve registered metrics on a locked recent destination?
4. Are calibration, subgroups, capacity, safety, latency, and interpretability acceptable?
5. Can the new version be shadowed, canaried, rolled back, and audited?

Retire a system when the decision no longer exists, input provenance degrades, treatment versions change beyond evidence, monitoring cannot observe harm, a simpler rule dominates, or continued use is no longer justified.

## 47.15 Research papers: dependence, model cards, and datasheets

Read Nadeau and Bengio (2003) to understand why resampled performance estimates are correlated. Read Mitchell and colleagues (2019) and Gebru and colleagues (2021) as governance interventions: both make hidden context inspectable. For each reporting field, ask who needs it to reproduce, contest, operate, or retire the system.

## 47.16 Build, break, and reflect

**Build.** Package the Chapter 4 or 5 model with a deterministic fixture, schema validator, environment lock, model card, and drift dashboard specification.

**Break.** Change a unit from million PKR to PKR, introduce an unseen category, delay labels selectively, and change the review policy. Which monitor catches each failure? Which remains invisible?

**Reflect.** Define one automatic alert, one human investigation, one rollback trigger, and one retirement criterion.

### Day 47 exit check

You are ready for the final study when you can:

- define the independent unit and procedure in a model comparison;

- explain why paired folds are not independent replicates;
- distinguish an ablation from a causal component effect;
- produce a reproducibility bundle from a clean environment; and
- monitor inputs, actions, outcomes, feedback, and governance together.

## Day 48 — The Registered Final Study

The final day combines causal design, estimation, diagnostics, sensitivity, reproducibility, and bounded communication. The goal is not a favourable p-value. It is a study another researcher can rerun and criticise.

### 48.1 Registration before analysis

For the running simulation, freeze this protocol before revealing counterfactual truth:

| Field             | Registered choice                                                                                          |
|-------------------|------------------------------------------------------------------------------------------------------------|
| Question          | Does month-6 senior review reduce warning risk by month 36?                                                |
| Population        | Active, observable, warning-free projects eligible at month 6                                              |
| Treatment         | Defined senior-review package assigned at month 6                                                          |
| Comparator        | No package at month 6                                                                                      |
| Outcome           | First major-overrun warning from month 6 through 36                                                        |
| Estimand          | ATE, marginal risk difference, assign all versus assign none                                               |
| Confounders       | District, approval year, terrain, cable length, contractor experience, appraisal budget, remote access     |
| Primary estimator | Five-fold cross-fitted AIPW with logistic nuisance models                                                  |
| Positivity rule   | Report score range, weight distribution, ESS; clip probabilities at 0.02/0.98 as part of procedure         |
| Uncertainty       | Influence-function interval; full row bootstrap as sensitivity                                             |
| Heterogeneity     | One pre-specified interaction: terrain index above 0.75                                                    |
| Sensitivity       | Outcome standardisation, IPW, overlap restriction, omitted-confounder tipping analysis                     |
| Claim rule        | Causal language only conditional on consistency, exchangeability, positivity, and no relevant interference |

In a real study, add cohort extraction version, missing-data handling, competing-event estimand, cluster structure, sample-size justification, negative controls, correction for multiple secondary analyses, and a dated public or access-controlled registration.

### 48.2 Lockbox discipline

Separate the work into zones:

- **Design data:** schema, timestamps, treatment prevalence, missingness, and covariate distributions; outcomes hidden where feasible.
- **Development data or simulation:** code tests, nuisance-model tuning, estimator recovery, and failure injections.
- **Locked analysis data:** opened once the protocol, code, tables, and stopping rules pass review.

- **External or prospective replication:** a new destination used to test transport and implementation.

If an error appears after unlocking, preserve the original result, describe the error, correct it, and label the rerun. Quietly replacing the protocol is not reproducibility.

## 48.3 Executable primary analysis

The earlier code already generated the cohort, estimated the naive contrast, standardised outcome regression, IPW, balance diagnostics, and cross-fitted AIPW. The following block assembles a one-row primary report, then reveals the simulation truth **only after** estimation.

```
analysis_truth = make_mhp_causal_data(
 n=4000, seed=6060, reveal_counterfactuals=True
)
true_ate = np.mean(analysis_truth["y1"] - analysis_truth["y0"])

primary_report = pd.DataFrame([
 "n": len(df),
 "treated_fraction": a.mean(),
 "outcome_fraction": y.mean(),
 "naive_risk_difference": naive,
 "gcomp_ate": gcomp_ate,
 "ipw_ate": ipw_ate,
 "aipw_ate": aipw["estimate"],
 "aipw_ci_low": aipw["ci_low"],
 "aipw_ci_high": aipw["ci_high"],
 "effective_sample_size": ess,
 "max_abs_weighted_smd": balance["smd_weighted"].abs().max(),
 "simulation_true_ate": true_ate,
 "absolute_estimation_error": abs(aipw["estimate"] - true_ate),
])

with pd.option_context("display.max_columns", None, "display.width", 180):
 print(primary_report.round(3).to_string(index=False))
```

The truth is the finite simulated cohort's average  $Y(1) - Y(0)$ , not the difference in average latent probabilities. Either can be a valid simulation target if registered; do not mix them.

## 48.4 Integrity tests before interpreting the number

```
assert len(df) == 4000
assert df[treatment].isin([0, 1]).all()
assert df[outcome].isin([0, 1]).all()
assert set(covariates).issubset(df.columns)
assert np.isfinite(aipw["score"]).all()
assert np.all((aipw["propensity"] >= 0.02) & (aipw["propensity"] <= 0.98))
assert 0 < ess <= len(df) + 1e-9
assert np.isclose(df[treatment].mean(), analysis_truth[treatment].mean())
assert np.isclose(df[outcome].mean(), analysis_truth[outcome].mean())
print("All registered integrity checks passed.")
```

Add tests for no post-treatment columns in the design matrix, timestamps preceding treatment, one row per eligible unit, missingness rules, fold separation, and identical output from a clean environment.

## 48.5 How to write the result

A defensible paragraph has five layers:

1. **Description:** number eligible, treatment prevalence, outcome frequency, missingness, overlap, and balance.
2. **Estimate:** population, treatment versions, horizon, scale, point estimate, and interval.
3. **Identification:** assumptions connecting the adjusted contrast to the causal estimand.
4. **Sensitivity:** how estimates changed under registered alternative models, overlap targets, missingness, and unmeasured-confounding scenarios.
5. **Decision:** absolute consequences per 100 or 1,000 projects, capacity, uncertainty, burden, and evidence gaps.

Example structure:

Among projects meeting the month-6 eligibility rule, the registered cross-fitted AIPW analysis estimated the change in 36-month warning risk under assignment of the defined senior-review package to all rather than none. The estimate and interval are reported on the risk-difference scale. Interpreting this as causal requires adequate measurement of the registered common causes, treatment-version consistency, treatment overlap, and absence of materially relevant spillovers. Results were compared with standardisation and weighting, accompanied by balance, weight, and sensitivity diagnostics. Because the data are simulated, the analysis demonstrates estimator behaviour rather than evidence about a real programme.

Insert numbers only from generated results. Do not write “proved,” “no effect,” or “works for every project.” If an interval includes both useful benefit and harm, say the study remains decision-uncertain.

## 48.6 Three graduation capstones

Choose one capstone and complete every required artifact. A null, harmful, unstable, or nonidentified result can pass if the reasoning is rigorous.

### Capstone A — Senior Review Target-Trial Emulation

**Question.** What is the effect of assigning senior review at a clearly aligned eligibility time on a pre-specified project outcome?

**Required work.** Define treatment versions and time zero; draw a DAG; audit timestamps; estimate by standardisation, weighting, and cross-fitted AIPW; diagnose overlap and balance; address censoring and competing events; quantify unmeasured-confounding sensitivity; examine one pre-specified modifier; and propose a prospective randomised or phased replication.

### Capstone B — Threshold or Policy-Rollout Evaluation

**Question.** Did a real threshold rule or staggered policy change an operational outcome?

**Required work.** Justify RD or modern staggered DiD from the assignment process; register the local or group-time estimand; inspect manipulation, pre-period dynamics, anticipation, spillovers, and concurrent shocks; run falsification and robustness analyses; and state the narrow population and time period supported.

### Capstone C — Prospective Model-Impact Trial

**Question.** Does using a prediction system improve decisions and outcomes compared with current practice?

**Required work.** Freeze the model and card; choose individual, cluster, stepped-wedge, or encouragement randomisation; define ITT and adherence estimands; pre-specify utility, safety, subgroup, capacity, and process outcomes; monitor feedback and missing outcomes; compare decision policies rather than AUC alone; and publish a reproducible impact report with rollback criteria.

Each report should contain an abstract, protocol, causal diagram, estimand table, data flow, methods, diagnostics, primary and sensitivity results, limitations, ethical and operational discussion, reproducibility statement, and a plain-language summary.

## 48.7 Research-practitioner certification rubric

| Dimension           | Beginning                            | Competent                                           | Research-ready                                                                |
|---------------------|--------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------------|
| Question            | asks whether variables are “related” | names treatment and outcome                         | specifies target trial, population, contrast, horizon, and scale              |
| Causal structure    | controls for all columns             | distinguishes baseline and post-treatment variables | defends DAG, adjustment set, selection, measurement, and interference         |
| Identification      | estimator treated as proof           | states core assumptions                             | connects every assumption to design evidence and sensitivity                  |
| Positivity          | ignored                              | plots propensity distributions                      | revises population/estimand when support is inadequate                        |
| Estimation          | one regression coefficient           | standardisation or weighting                        | cross-fitted robust estimator with registered nuisance procedure              |
| Diagnostics         | reports model fit only               | balance and weights checked                         | overlap, influence, missingness, negative controls, and failure injections    |
| Longitudinal design | future treatment called baseline     | aligns time zero                                    | emulates strategies with feedback, adherence, censoring, and competing events |
| Quasi-experiment    | method chosen from menu              | names design assumption                             | establishes assignment mechanism and design-specific falsification            |
| Heterogeneity       | mines subgroups                      | tests pre-specified interaction                     | honest discovery, external confirmation, policy value, multiplicity control   |
| Uncertainty         | p-value only                         | interval for primary estimate                       | procedure-, cluster-, and identification-sensitivity uncertainty separated    |
| Reproducibility     | working notebook                     | seeded script and environment                       | immutable manifests, tests, registration, deviation log, clean rerun          |
| Deployment          | tracks accuracy                      | monitors drift and outcomes                         | monitors decisions, feedback, harms, rollback, and retirement                 |
| Communication       | causal overclaim                     | cautious association language                       | claim precisely bounded by estimand, assumptions, population, and time        |

Research-ready does not mean every assumption is satisfied. It means weaknesses are visible enough to change the claim or stop the study.

## 48.8 The final oral defence

Before certification, answer these questions without slides:

1. What exact counterfactual contrast did you estimate?
2. Why is the treatment sufficiently well defined?
3. How did units become treated, untreated, observed, and missing?
4. Which open backdoor paths does your adjustment set block?
5. Which important paths could remain open?



6. Where is positivity weak, and which population is unsupported?
7. What does your interval include and omit?
8. Which result changed most across sensitivity analyses?
9. Why is the conclusion not merely a model coefficient?
10. What new evidence would most change the decision?

If the honest answer to Question 5 is “an unmeasured cause likely remains,” revise the conclusion. If Question 6 identifies a region with no comparison, restrict the target. Intellectual honesty is part of the method.

## Day 48 exit check

You have completed the chapter when you can:

- defend the design before describing the estimator;
- regenerate every table from raw inputs in a clean environment;
- show that the primary result was specified before the lockbox opened;
- explain every causal noun in the conclusion using an assumption or design feature;
- distinguish uncertainty about sampling, modelling, measurement, identification, and transport; and
- propose the next study that addresses the largest remaining uncertainty.

---

# Chapter 6 Synthesis

---

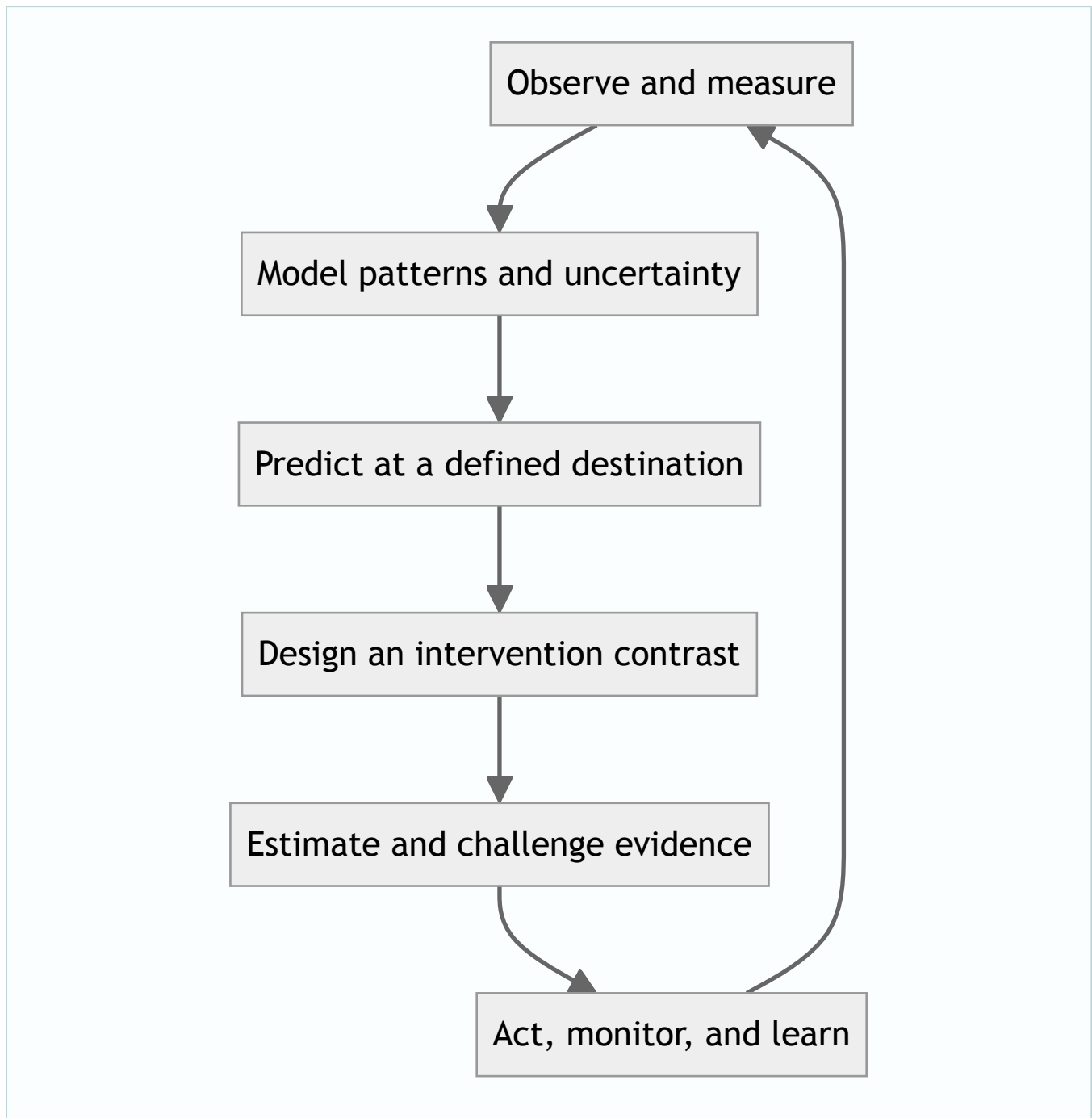
## One question, six layers

The question “does senior review prevent overruns?” becomes answerable only after six layers are separated:

1. **Intervention:** What review package, compared with what alternative?
2. **Estimand:** For whom, over what horizon, and on what effect scale?
3. **Identification:** Which design and assumptions connect observed data to counterfactual outcomes?
4. **Estimation:** Which outcome, assignment, censoring, and effect models approximate the identified functional?
5. **Validation:** Do balance, overlap, falsification, sensitivity, replication, and transport evidence support the claim?
6. **Decision:** What action follows under capacity, costs, benefits, uncertainty, fairness, and governance?

Collapsing these layers creates familiar errors: a coefficient becomes an effect, a narrow local effect becomes a universal claim, a predictive score becomes a treatment rule, and a reproducible pipeline preserves a biased design.

## The complete book in one research loop



The chapters built this loop progressively:

- **Chapter 1** established mathematical and programming literacy.
- **Chapter 2** turned data into estimable regression problems with honest validation.
- **Chapter 3** developed practical modelling, preprocessing, tuning, and diagnostics.
- **Chapter 4** treated classification as probability estimation and decision design.
- **Chapter 5** preserved clocks, censoring, competing events, and dynamic risk.
- **Chapter 6** separated prognosis from intervention and connected research to accountable use.

Research is not the last box. Monitoring creates new data, interventions change the system, and every deployment changes the next question.

# A compact causal-study checklist

Before data analysis:

- ☐ Write the target trial and estimand.
- ☐ Define treatment versions, comparator, time zero, outcome, horizon, and population.
- ☐ Draw a DAG with unmeasured and selection variables.
- ☐ Choose adjustment or quasi-experimental design from the assignment mechanism.
- ☐ Register primary analysis, diagnostics, sensitivity, and claim rule.

Before estimation:

- ☐ Audit timestamps, duplicates, units, missingness, eligibility, and treatment provenance.
- ☐ Exclude post-treatment leakage from baseline adjustment.
- ☐ Inspect treatment support without fishing through outcomes.
- ☐ Validate code on synthetic data with known failure cases.

Before interpretation:

- ☐ Report balance, overlap, weights, ESS, and influence.
- ☐ Separate primary, secondary, sensitivity, and exploratory analyses.
- ☐ Quantify sampling and procedural uncertainty.
- ☐ Challenge unmeasured confounding, selection, measurement, and transport.
- ☐ Translate effects to absolute consequences and capacity.

Before deployment:

- ☐ Freeze artifacts, environment, cards, and manifests.
- ☐ Validate training-serving parity and decision-time feature availability.
- ☐ Monitor service, inputs, actions, outcomes, feedback, and subgroup burden.
- ☐ Establish human review, appeals, incident response, rollback, and retirement.
- ☐ Plan prospective impact evaluation.

---

## Chapter 6 Exercises

---

### Counterfactual foundations

#### Exercise 6.1 — Estimand translation

Translate “Does tutoring help students?” into two distinct target trials: offer of tutoring and sustained attendance. For each, state eligibility, time zero, treatment versions, comparator, outcome, horizon, population, and scale.

#### Exercise 6.2 — Potential-outcome arithmetic

For a six-person fictional population whose  $(Y(1), Y(0))$  pairs are known only for simulation, calculate individual effects, ATE, ATT under a supplied assignment vector, treated and untreated observed risks, and confounding bias. Then hide one potential outcome per person and explain what was lost.

## Exercise 6.3 — Assumption audit

For senior review, propose one violation each of consistency, exchangeability, positivity, and no interference. For each, state whether more sample size would solve it.

## Diagrams and experiments

### Exercise 6.4 — Adjustment-set challenge

Draw a DAG containing treatment, outcome, two confounders, a mediator, a collider, a selection variable, and an instrument. Find two valid total-effect adjustment sets and three invalid sets. Trace every opened or blocked path.

### Exercise 6.5 — Trial analysis

Simulate a blocked randomised trial with a binary outcome. Estimate the ITT by block-weighted differences and regression standardisation. Repeat 1,000 times to check bias and confidence-interval coverage.

### Exercise 6.6 — Noncompliance

Simulate compliers, always-takers, and never-takers with no defiers. Calculate ITT, first stage, and Wald LATE. Violate exclusion by giving assignment a direct motivational effect and show what the ratio estimates no longer means.

## Observational estimation

### Exercise 6.7 — Propensity design without outcomes

Hide the outcome column. Build matching and weighting designs using only treatment and baseline covariates. Compare balance, overlap, discards, weight range, and ESS. Freeze the design before revealing outcomes.

### Exercise 6.8 — Standardisation by hand

Fit a small logistic outcome model with treatment, one continuous confounder, and an interaction. Calculate fitted  $m_1(X_i)$  and  $m_0(X_i)$  for five rows by hand, then standardise.

### Exercise 6.9 — Doubly robust failure matrix

Use a nonlinear simulation to create four cases: both nuisance models correct, only outcome correct, only propensity correct, neither correct. Compare g-computation, IPW, and cross-fitted AIPW over repeated samples.

### Exercise 6.10 — Whole-procedure bootstrap

Bootstrap the running AIPW study at least 500 times. Redo folds and nuisance fitting inside every resample. Compare percentile, normal, and influence-function intervals; report computation time and failures.

### Exercise 6.11 — Hidden-confounding tipping point

Create an unmeasured binary factor affecting review and warning. Analyse without it. Vary its prevalence and associations, graph bias, and identify the smallest substantively credible scenario that changes the decision.

## Longitudinal and quasi-experimental work

### Exercise 6.12 — Immortal-time demonstration

Simulate treatment initiated during follow-up. Compare an incorrect ever-treated baseline analysis, a time-varying analysis, and a properly aligned landmark target trial. Explain the direction of bias.

### Exercise 6.13 — Marginal structural model

Generate three visits with a covariate affected by prior treatment that predicts later treatment and outcome. Fit naive adjustment and a weighted marginal structural model. Inspect weights at every visit and compare estimates with simulation truth.

### Exercise 6.14 — DiD under staggered adoption

Simulate three adoption cohorts with dynamic heterogeneous effects. Compare a two-way fixed-effects coefficient with group-time effects. Add anticipation and a concurrent shock; use event-study and placebo diagnostics to reveal each.

### Exercise 6.15 — RD manipulation

Generate a sharp threshold design, estimate a local linear effect, and vary bandwidths. Then allow units to sort just above the threshold based on prognosis. Inspect density, covariate continuity, and effect bias.

### Exercise 6.16 — Instrument validity

Propose an instrument in your domain. Draw at least two plausible exclusion violations. Simulate weak and strong first stages and compare Wald uncertainty. State the complier population.

## Research practice and impact

### Exercise 6.17 — Honest heterogeneity

Use one sample to discover effect modifiers and a separate sample to confirm them. Compare subgroup calibration, interaction intervals, and policy value with a constant-effect rule. Repeat under a truly constant effect.

### Exercise 6.18 — Reproducibility stress test

Give a peer only your clean research bundle. They must regenerate the cohort count, primary estimate, interval, and one figure without verbal help. Record every hidden dependency they discover and repair the bundle.

### Exercise 6.19 — Production incident

Write a runbook for a unit change, delayed outcome feed, subgroup action-rate spike, and policy-induced feedback loop. Include detection, severity, owner, containment, rollback, communication, root-cause analysis, and prevention.

### Exercise 6.20 — Final registered study

Complete one graduation capstone. Submit the registration before the primary outcome is opened. Include the full code, synthetic test fixture, data and model cards, causal-study card, deviation log, and plain-language summary.

---

# Glossary

---

**Average treatment effect (ATE)** — Average contrast  $EY(1) - Y(0)$  in a stated population.

**Average treatment effect among the treated (ATT)** — Average contrast among units actually treated.

**Backdoor path** — Path from treatment to outcome beginning with an arrow into treatment; an open one can create confounding.

**Balance** — Similarity of covariate distributions between treatment groups in the weighted, matched, or stratified comparison.

**Causal estimand** — Precisely defined counterfactual contrast, including population, interventions, outcome, horizon, and scale.

**Collider** — Variable with two arrows pointing into it; conditioning can create association between its causes.

**Conditional average treatment effect (CATE)** — Average potential-outcome contrast within a baseline covariate profile or subgroup.

**Consistency** — Observed outcome under the received treatment equals the potential outcome for that well-defined treatment version.

**Confounder** — Pre-treatment common cause of treatment and outcome in the causal structure for a specific question.

**Cross-fitting** — Estimating nuisance functions outside the observations on which their effect scores are evaluated.

**Difference-in-differences** — Change in a treated group minus contemporaneous change in a comparison group, identified under a counterfactual parallel-trends assumption.

**Directed acyclic graph (DAG)** — Causal graph with directed arrows and no directed cycle.

**Double robustness** — Consistency of certain estimators when either one of two nuisance models is correctly specified, under the identification assumptions.

**Effective sample size** — Weight-concentration summary  $(\sum w)^2 / \sum w^2$ .

**Exchangeability** — Independence of treatment and potential outcomes, marginally through design or conditionally given adequate covariates.

**Exclusion restriction** — Instrument affects outcome only through treatment under the specified intervention system.

**Fundamental problem of causal inference** — Both treatment-specific potential outcomes cannot be observed for the same unit in the same circumstance.

**G-formula** — Standardisation formula identifying intervention outcomes by averaging conditional outcome distributions over covariates or histories.

**Honesty** — Separating data used to discover model structure from data used to estimate effects within that structure.

**Instrumental variable** — Variable that shifts treatment and, under relevance, independence, exclusion, and often monotonicity, identifies a local causal contrast.

**Intention-to-treat effect** — Effect of random assignment or offer, analysed according to assigned group.

**Interference** — One unit's treatment affects another unit's outcome.

**Local average treatment effect (LATE)** — Average treatment effect among compliers whose treatment is changed by an instrument under stated assumptions.

**Marginal structural model** — Model for counterfactual outcomes under treatment histories, commonly fit using inverse-probability weights.

**Mediator** — Post-treatment variable on a causal pathway from treatment to outcome.

**Negative control** — Exposure or outcome chosen to share a bias mechanism while lacking the causal relation of interest.

**Nuisance function** — Auxiliary function, such as propensity or outcome regression, required to estimate the target parameter.

**Overlap** — Empirical presence of credible treatment comparisons across covariate profiles in the target population.

**Policy value** — Expected outcome or utility under a treatment rule.

**Positivity** — Nonzero probability of each required treatment strategy for histories in the target population.

**Potential outcome** — Outcome that would occur under a specified intervention.

**Propensity score** — Conditional probability of treatment given baseline covariates.

**Regression discontinuity** — Local design using a treatment-rule discontinuity at a running-variable cutoff.

**Sensitivity analysis** — Structured assessment of how conclusions change under plausible violations, parameters, or analytic choices.

**Standardisation** — Averaging conditional outcome predictions under each intervention over a target covariate distribution.

**Target trial** — Protocol of the ideal randomised experiment an observational study seeks to emulate.

**Time-zero bias** — Bias from misaligning eligibility, treatment classification, and the start of follow-up.

---

## Where to Go After the Book

---

Do not respond to completion by collecting more methods at random. Choose a domain and deepen four strands together:

1. **Substance:** learn how interventions are delivered, outcomes are measured, and institutions create the data.
2. **Theory:** study probability, linear algebra, optimisation, asymptotics, causal identification, and measurement more formally.
3. **Computation:** build tested packages, simulations, reproducible pipelines, and scalable estimators.
4. **Research practice:** preregister, replicate, seek criticism, communicate uncertainty, and participate in prospective studies.

A productive paper-reading routine is:

- write the estimand before reading the methods;
- redraw the study design and DAG;
- derive the simplest estimator on paper;
- reproduce one table or simulation;

- identify the strongest untestable assumption;
- create a failure case where the method misleads; and
- propose a design or measurement improvement, not merely another algorithm.

## Final reflection

The book began with numbers and ended with interventions. Between them lies a chain:

measurement → probability → prediction → counterfactual → decision → consequence.

Every arrow contains assumptions. Research-level practice is not the absence of assumptions; it is the ability to expose them, connect them to evidence, test what can be tested, vary what cannot, and stop when the data do not support the claim.

You are ready to begin research when you can be surprised by your result without changing your question, disappointed by uncertainty without hiding it, and willing to conclude that the best next step is better measurement or a better design.

**The book ends here. The research begins here.**

## References and Further Reading

### Research papers discussed

- Angrist, J. D., Imbens, G. W., & Rubin, D. B. (1996). Identification of Causal Effects Using Instrumental Variables. *Journal of the American Statistical Association*, 91(434), 444–455.  
<https://doi.org/10.1080/01621459.1996.10476902>
- Athey, S., & Imbens, G. (2016). Recursive Partitioning for Heterogeneous Causal Effects. *Proceedings of the National Academy of Sciences*, 113(27), 7353–7360.
- Bang, H., & Robins, J. M. (2005). Doubly Robust Estimation in Missing Data and Causal Inference Models. *Biometrics*, 61(4), 962–973. <https://doi.org/10.1111/j.1541-0420.2005.00377.x>
- Callaway, B., & Sant'Anna, P. H. C. (2021). Difference-in-Differences with Multiple Time Periods. *Journal of Econometrics*, 225(2), 200–230.
- Chernozhukov, V., Chetverikov, D., Demirer, M., Duflo, E., Hansen, C., Newey, W., & Robins, J. (2018). Double/Debiased Machine Learning for Treatment and Structural Parameters. *The Econometrics Journal*, 21(1), C1–C68.
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., & Crawford, K. (2021). Datasheets for Datasets. *Communications of the ACM*, 64(12), 86–92.
- Greenland, S., Pearl, J., & Robins, J. M. (1999). Causal Diagrams for Epidemiologic Research. *Epidemiology*, 10(1), 37–48.
- Hahn, J., Todd, P., & van der Klaauw, W. (2001). Identification and Estimation of Treatment Effects with a Regression-Discontinuity Design. *Econometrica*, 69(1), 201–209.
- Hernán, M. A., & Robins, J. M. (2016). Using Big Data to Emulate a Target Trial When a Randomized Trial Is Not Available. *American Journal of Epidemiology*, 183(8), 758–764.
- Holland, P. W. (1986). Statistics and Causal Inference. *Journal of the American Statistical Association*, 81(396), 945–960.



- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model Cards for Model Reporting. In *Proceedings of FAT\** (pp. 220–229).
- Nadeau, C., & Bengio, Y. (2003). Inference for the Generalization Error. *Machine Learning*, 52, 239–281.
- Robins, J. M., Hernán, M. A., & Brumback, B. (2000). Marginal Structural Models and Causal Inference in Epidemiology. *Epidemiology*, 11(5), 550–560.
- Rosenbaum, P. R., & Rubin, D. B. (1983). The Central Role of the Propensity Score in Observational Studies for Causal Effects. *Biometrika*, 70(1), 41–55.
- Rubin, D. B. (1974). Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies. *Journal of Educational Psychology*, 66(5), 688–701. <https://doi.org/10.1037/h0037350>
- van der Laan, M. J., & Rubin, D. (2006). Targeted Maximum Likelihood Learning. *The International Journal of Biostatistics*, 2(1).
- Wager, S., & Athey, S. (2018). Estimation and Inference of Heterogeneous Treatment Effects Using Random Forests. *Journal of the American Statistical Association*, 113(523), 1228–1242.

## Books and long-form resources

- Hernán, M. A., & Robins, J. M. *Causal Inference: What If*.
- Imbens, G. W., & Rubin, D. B. *Causal Inference for Statistics, Social, and Biomedical Sciences*.
- Pearl, J., Glymour, M., & Jewell, N. P. *Causal Inference in Statistics: A Primer*.
- Cunningham, S. *Causal Inference: The Mixtape*.
- Matheus Facure. *Causal Inference for the Brave and True*.
- National Academies of Sciences, Engineering, and Medicine. *Reproducibility and Replicability in Science*.

## Software documentation to consult during implementation

Interfaces change. Consult current official documentation for the exact versions in your environment:

- scikit-learn: preprocessing, pipelines, model selection, inspection, and calibration;
- statsmodels: treatment effects, regression, robust covariance, and time-series tools;
- DoWhy: explicit causal graphs, identification, estimation, and refutation workflows;
- EconML: orthogonal learners and heterogeneous treatment effects;
- DoubleML: double/debiased machine learning with cross-fitting;
- CausalML: meta-learners and uplift evaluation;
- R `MatchIt`, `WeightIt`, `cobalt`, `did`, `rdrobust`, `grf`, `tmle`, and `ltmle`; and
- Trial registration and reporting standards appropriate to the domain.

Software can implement an estimator. It cannot choose the causal question, verify an unmeasured assumption, or take responsibility for the decision.

# Chapter 6 Companion Guide: From Prediction to Intervention

---

## A beginner-first guide to causal inference, experiments, observational designs, reproducibility, and responsible deployment

This guide accompanies `chapter6.md`. It assumes you are still learning Python, mathematics, statistics, and causal language. The chapter is advanced because the questions are demanding, not because explanations should be obscure.

Earlier chapters asked what happened, what is associated, what is likely, and when an event may occur. Chapter 6 asks:

What would change if we deliberately chose one action rather than another?

That question cannot be answered by adding a treatment column to a predictive model. A causal study needs a defined intervention, counterfactual contrast, assignment story, identification assumptions, estimator, uncertainty procedure, and population.

```
policy question
-> target trial and estimand
-> causal structure and identification
-> design and estimation
-> diagnostics and sensitivity
-> reproducible result
-> monitored decision system
```

---

## How to use this guide

For Days 39–48:

1. Write the causal question without naming a model.
2. Specify population, time zero, treatment, comparator, outcome, horizon, and scale.
3. Draw the assignment mechanism or DAG.
4. State every identification assumption in plain language.
5. Derive the smallest estimator by hand.
6. Type and deliberately break the code.
7. Identify what no observed-data diagnostic can test.
8. Write one supported and one unsupported conclusion.

Keep these layers separate:

- **estimand:** the counterfactual quantity wanted;
- **identification:** assumptions connecting it to observable data;

- **estimator:** a finite-sample calculation;
- **inference:** uncertainty under the design and procedure;
- **transport:** the population or setting receiving the claim;
- **policy:** the action rule and consequences; and
- **production:** how the system is delivered, monitored, challenged, and stopped.

## 0. Foundations for a complete beginner

### 0.1 The running causal question

At month 6, eligible fictional microhydro projects may receive a structured senior-review package. The outcome is a formal major-overrun warning between just after month 6 and month 36.

The primary question is:

Among projects active, observable, and warning-free at month 6, what would be the 36-month warning-risk difference if all received review at month 6 versus if none received it?

This fixes:

- unit: eligible project;
- time zero: month 6;
- treatment: defined review package at time zero;
- comparator: no package at time zero;
- outcome: warning through month 36;
- follow-up: 30 months;
- population: month-6 eligible projects; and
- scale: risk difference.

The data are simulated. Nothing about their districts, projects, or review effects is real-world evidence.

### 0.2 Essential notation

- $A = 1$  means treatment;  $A = 0$  means comparator.
- $Y$  is the observed outcome.
- $Y(1)$  and  $Y(0)$  are potential outcomes under the two interventions.
- $X$  contains pre-treatment covariates.
- $Z$  often denotes randomized assignment or an instrument.
- $M$  often denotes a mediator.
- $S$  can denote selection into the observed sample.
- $E(Y)$  is an average over a population distribution.
- $P(A = 1 \mid X)$  is treatment probability among profile  $X$ .
- $\perp$  means statistical independence.
- $\hat{q}$  is an estimate of unknown quantity  $q$ .
- $\bar{A}_k$  means treatment history through visit  $k$ .

Subscripts index subjects or visits; superscripts such as  $Y^d$  name a potential outcome under strategy  $d$ , not exponentiation.

## 0.3 Python patterns

```
treated = df[df["senior_review_month6"] == 1]
control = df[df["senior_review_month6"] == 0]
naive_difference = (
 treated["warning_by_month36"].mean()
 - control["warning_by_month36"].mean()
)
```

This computes an observed group difference. Code correctness does not make it causal.

Other recurring patterns:

- `.copy()` prevents accidental mutation of a shared table;
- `np.where(condition, a, b)` selects rowwise values;
- `np.clip(p, .02, .98)` limits estimated probabilities but changes the procedure;
- `.predict_proba(X)[:, 1]` selects class-1 probability;
- `KFold` creates cross-fitting partitions;
- `random_state` reproduces pseudo-random choices under the same environment.

## 0.4 Understanding the simulation generator

`make_mhp_causal_data` simulates baseline covariates, then assigns review preferentially to difficult projects:

```
propensity = expit(linear_assignment_score)
senior_review = rng.binomial(1, propensity, n)
```

`expit(z)=1/(1+e-z)` converts a score into probability. Terrain, cable length, budget, experience, access, and district affect both assignment and warning risk, creating measured confounding.

The generator creates probabilities under no review and review, then potential outcomes `y0` and `y1`. Observed outcome is:

```
observed_y = np.where(senior_review == 1, y1, y0)
```

Counterfactual columns are hidden unless explicitly revealed for simulation validation. Real datasets never reveal both potential outcomes or the true propensity. Simulation truth tests whether code can work under a known world; it does not validate assumptions in empirical data.

## 0.5 Installing and recording software

```
python -m pip install numpy pandas scipy matplotlib scikit-learn statsmodels
```

Optional causal libraries include DoWhy, EconML, DoubleML, CausalML, and relevant R packages. Interfaces change, so use current official documentation and record exact versions.

A package implements an estimator. It cannot define treatment versions, justify exchangeability, choose a valid instrument, detect every collider, or accept responsibility for policy.

## 0.6 Prediction, association, and intervention

$$P(Y = 1 \mid A, X)$$

describes observed outcomes conditional on recorded action and features.

$$P\{Y(a) = 1\}$$

describes a hypothetical intervention setting action to  $a$ . The conditioning bar and intervention are not interchangeable. Riskier projects can receive more review, so reviewed projects may have worse observed outcomes even if review helps.

## Day 39 — Counterfactuals, estimands, and identification

### 39.1 Potential outcomes

For project  $i$ :

- $Y_i(1)$  is outcome under review;
- $Y_i(0)$  is outcome under no review.

Individual causal contrast is  $Y_i(1) - Y_i(0)$ , but only one is observed:

$$Y_i = A_i Y_i(1) + (1 - A_i) Y_i(0).$$

This fundamental problem cannot be solved by sample size: the same project in the same circumstance cannot simultaneously receive both actions. Designs identify average contrasts using comparisons and assumptions.

### 39.2 Choose the estimand first

Common targets:

$$ATE = E\{Y(1) - Y(0)\},$$

$$ATT = E\{Y(1) - Y(0) \mid A = 1\},$$

$$CATE(x) = E\{Y(1) - Y(0) \mid X = x\},$$

and policy value  $E[Yd(X)]$  under rule  $d$ .

ATE, ATT, untreated-population effects, overlap effects, and policy values average over different populations or assignments. “The treatment effect” is incomplete.

For binary risks  $p_1, p_0$ :

- risk difference:  $p_1 - p_0$ ;
- risk ratio:  $p_1/p_0$ ;
- odds ratio:  $[p_1/(1 - p_1)]/[p_0/(1 - p_0)]$ .

Scales differ. Conditional and marginal odds ratios can differ without confounding because odds ratios are non-collapsible. Register the scale rather than switching to the most dramatic result.

For event time, name horizon, competing-event strategy, survival, incidence, or restricted mean.

## 39.3 Four identification assumptions

### Consistency

If  $A = a$ , observed outcome equals  $Y(a)$ . Treatment must be well defined. A five-minute signature and six-week engineering review are not automatically one intervention.

### Conditional exchangeability

$$\{Y(1), Y(0)\} \perp A \mid X.$$

Given adequate pre-treatment covariates, action behaves as if independent of potential outcomes. In observational data this no-unmeasured-confounding claim is not testable from outcomes alone.

### Positivity

$$0 < P(A = 1 \mid X = x) < 1$$

for target profiles. If all remote projects are reviewed, untreated remote outcomes cannot be learned without extrapolation. Restriction, new data, or a different estimand changes the question honestly.

### No interference

One project's treatment must not change another's outcome under simple notation. Shared engineers can create spillovers. Cluster or network estimands may be needed.

## 39.4 The g-formula identification step

Under consistency, exchangeability, and positivity:

$$E\{Y(a)\} = E_X[E(Y \mid A = a, X)].$$

Exchangeability inserts observed treatment group  $A = a$  into the conditional potential-outcome mean; consistency replaces potential with observed outcome; positivity ensures relevant comparisons exist.

This derivation justifies standardization—not loyalty to one regression package.

## 39.5 Identification, estimation, and inference

- Identification asks whether an exactly known observed distribution determines the estimand under assumptions.
- Estimation approximates required quantities from finite data.
- Inference quantifies sampling and procedural uncertainty.

More data reduce some estimation error but cannot repair undefined interventions, colliders, positivity gaps, or hidden confounding. Confidence intervals are conditional on identification and model assumptions.

## Day 39 mastery assignment

### Level 1 — Vocabulary and recognition

1. For “Does a reminder reduce missed appointments?”, identify  $A$ ,  $Y$ ,  $Y(1)$ ,  $Y(0)$ ,  $X$ , population, time zero, and horizon.

2. Classify each quantity as observed, counterfactual, or sometimes observed:  $A$ ,  $Y$ ,  $Y(1)$ ,  $Y(0)$ ,  $P(Y = 1 \mid A = 1)$ , and  $E[Y(1)]$ .
3. Explain the fundamental problem of causal inference without using the word “counterfactual.”
4. Translate consistency, exchangeability, positivity, and no interference into one sentence each for senior review.

## Level 2 — Hand calculations

Given six simulated pairs  $(Y(1), Y(0))$ :  $(0, 1)$ ,  $(0, 0)$ ,  $(1, 1)$ ,  $(0, 1)$ ,  $(1, 1)$ ,  $(0, 0)$  and assignments  $1, 0, 1, 0, 1, 0$ :

1. calculate every individual effect, ATE, ATT, and average effect among untreated;
2. construct the observed  $Y$  vector;
3. calculate the naive treated-minus-control risk difference;
4. calculate causal bias as naive contrast minus ATE; and
5. explain why this calculation is possible only in a simulation.

For  $p_1 = 0.18$  and  $p_0 = 0.30$ , calculate risk difference, risk ratio, odds under each action, odds ratio, and number of warnings prevented per 1,000 projects.

## Level 3 — Applied design and code

1. Run `make_mhp_causal_data` without revealing counterfactuals. Produce a cohort table by treatment with counts, outcome risk, and baseline summaries.
2. Write three distinct target trials for the same review program: effect of offer, effect of receipt, and effect of sustained adherence.
3. Reveal simulation truth only after freezing the three estimands. Compare their populations and explain which observed columns cannot identify each without assumptions.
4. Modify the generator so every remote project is treated. Diagnose the resulting positivity failure by profile.

## Level 4 — Research challenge

Choose an intervention in education, health, infrastructure, finance, or public administration. Write a one-page estimand specification containing treatment versions, comparator, eligibility, time zero, outcome, competing events, horizon, population, scale, interference unit, and the exact counterfactual contrast. Propose one realistic violation of each identification assumption and state whether more observations would fix it.

## Probing questions

- Can an ATE be well identified when individual treatment effects cannot be observed?
- When would ATT be more useful than ATE for a policy decision?
- If the risk difference is constant across populations, must the risk ratio be constant?
- Does excellent treatment-overlap prove exchangeability?
- Which part of your intervention definition is most vulnerable to hidden treatment versions?

## Completion evidence

You have mastered Day 39 when another reader can reconstruct your target trial, reproduce every hand calculation, distinguish estimand from estimator, and identify at least one untestable assumption without prompting.

---

# Day 40 — Causal diagrams and adjustment

---

## 40.1 DAG basics

A directed acyclic graph has variables as nodes and assumed direct causal relations as arrows. “Acyclic” means following arrow directions cannot return to the starting node.

A DAG is not learned truth. It is a compact, criticizable causal argument. Include unmeasured variables when scientifically relevant; omission from the dataset does not erase them from the world.

## 40.2 Variable roles depend on the question

- Confounder: common cause  $A \leftarrow C \rightarrow Y$ .
- Mediator: causal pathway  $A \rightarrow M \rightarrow Y$ .
- Collider: common effect  $A \rightarrow K \leftarrow Y$ .
- Instrument: causes treatment but affects outcome only through treatment under strong assumptions.
- Proxy: measured stand-in related to an unmeasured construct.
- Selection variable: controls entry or observation and can induce bias.

A variable's role depends on time, intervention, outcome, and graph. “Always adjust for age” is not causal reasoning.

## 40.3 Confounders and backdoor paths

A backdoor path starts with an arrow into treatment. A valid adjustment set blocks all such paths without conditioning on forbidden descendants that open bias.

Confounders are chosen from substantive causal knowledge, not treatment/outcome  $p$ -values, predictive importance, or automated stepwise selection. Weakly associated confounders can matter; strong predictors need not confound.

## 40.4 Mediators and estimand changes

Adjusting for a mediator removes part of the pathway and no longer targets the total effect. Direct and indirect effects need precisely defined interventions and additional assumptions, especially with mediator–outcome confounding affected by treatment.

“Control for everything” silently changes the causal question.

## 40.5 Collider and selection bias

In  $A \rightarrow K \leftarrow Y$ ,  $A$  and  $Y$  can be independent before conditioning. Restricting to or adjusting for  $K$  opens association between its causes.

Examples include analyzing only approved, tested, hospitalized, completed, or fully documented cases when selection is affected by exposure and prognosis. Missingness indicators can also be colliders.



## 40.6 Descendants, instruments, and bias amplification

Treatment descendants are usually post-treatment and inappropriate for total-effect adjustment. Conditioning on an instrument-like strong treatment predictor can amplify residual bias from an unmeasured confounder without helping outcome prediction.

Covariate choice must follow the graph and estimand, not a generic baseline-variable checklist.

## 40.7 Selection and transport

A selection diagram marks mechanisms differing between study and target settings. Transport may require standardizing study effects over target covariates under assumptions that conditional causal relations are shared.

External predictive accuracy does not by itself prove causal transport. Treatment versions, adherence, effect modifiers, outcome measurement, and interference can change.

## 40.8 DAG workflow

1. Fix treatment, comparator, outcome, horizon, and population.
2. Order variables by time.
3. Add causes even when unmeasured.
4. Trace causal and backdoor paths.
5. Identify candidate adjustment sets.
6. Reject sets containing harmful post-treatment or collider variables.
7. Compare assumptions with actual timestamps and measurements.
8. Publish and challenge the graph before effect estimation.

Different plausible DAGs imply sensitivity analyses or that identification is not credible.

## Day 40 mastery assignment

### Level 1 — Path recognition

For each structure, name the variable role and state whether conditioning is normally appropriate for a total effect:

1.  $A \leftarrow C \rightarrow Y$ ;
2.  $A \rightarrow M \rightarrow Y$ ;
3.  $A \rightarrow K \leftarrow Y$ ;
4.  $Z \rightarrow A \rightarrow Y$ ; and
5.  $A \rightarrow S \leftarrow U \rightarrow Y$  with analysis restricted to  $S = 1$ .

Trace every open path from  $A$  to  $Y$  before and after conditioning.

### Level 2 — Draw and criticize

1. Draw a DAG for review and warnings containing terrain, contractor experience, review, updated month-12 progress, cancellation, and warning.
2. Add one unmeasured management-quality variable, one selection variable, one proxy, and one possible instrument.
3. Find a minimally sufficient total-effect adjustment set.
4. Construct three invalid adjustment sets: one containing a mediator, one opening a collider, and one failing to block confounding.

5. Explain in words which path each invalid set opens or leaves open.

## Level 3 — Simulation laboratory

Simulate four datasets:

1. confounding only;
2. confounding plus a mediator;
3. treatment and outcome both causing selection; and
4. treatment and an unmeasured outcome cause both causing a collider.

For each, compare unadjusted regression with graph-appropriate and graph-inappropriate adjustment. Repeat at least 500 times and plot estimator bias. Confirm that a highly predictive adjustment variable can increase causal bias.

## Level 4 — Research challenge

Interview or consult a domain process description before seeing outcomes. Produce a DAG change log documenting who proposed each arrow, what evidence supports it, which nodes are unmeasured, which alternative graphs remain plausible, and how estimates would be reported across those graphs. Include a transport/selection diagram for a second setting.

## Probing questions

- Can a variable be a confounder for one estimand and a mediator for another?
- Why can conditioning on complete records create bias?
- When might adjustment for an instrument amplify residual confounding?
- What would make two experts disagree about a valid adjustment set?
- Which arrows in your DAG are most consequential yet least measurable?

## Completion evidence

Mastery requires correctly tracing paths without software, defending covariate timing, explaining why every adjusted variable is included, and stating how a plausible alternative DAG changes identification.

---

# Day 41 — Randomised experiments and noncompliance

---

## 41.1 What randomization does

Random assignment makes  $Z$  independent of baseline potential outcomes in expectation through a known mechanism. It balances measured and unmeasured baseline causes on average, not necessarily exactly in one sample.

Analyze according to the randomization design. Allocation concealment prevents foreknowledge; blinding reduces behavior and measurement changes. Randomization does not solve attrition, nonadherence, interference, outcome mismeasurement, or transport.

## 41.2 Intention-to-treat

$$\hat{\tau}_{ITT} = \bar{Y}_{Z=1} - \bar{Y}_{Z=0}.$$

ITT estimates effect of assignment/offer in the trial population. Preserve assignment groups regardless of receipt. Comparing recipients breaks randomization because adherence can reflect prognosis.

With binary outcomes report absolute risks and differences, not only odds ratios. Use randomization-based, robust, or prespecified model-assisted uncertainty appropriate to design.

## 41.3 Blocking and covariate adjustment

Blocking randomizes within prespecified strata and should be reflected in analysis. Baseline adjustment can improve precision when prespecified and correctly handled; post-randomization adjustment can bias the total assignment effect.

Chance baseline imbalance is not evidence randomization failed. Do not choose adjustment variables based on observed imbalance significance tests.

## 41.4 Cluster randomization and interference

If offices or districts are randomized, the independent assignment unit is the cluster. Individual-level standard errors falsely treat correlated subjects as independent.

Few clusters require careful randomization inference or small-sample corrections. Analyze recruitment timing because knowledge of cluster assignment can affect who enters.

Spillovers violate no-interference assumptions. Define direct, indirect, total, or saturation effects and randomize at a level consistent with delivery.

## 41.5 Noncompliance and LATE

Assignment  $Z$  differs from treatment received  $A$ . The first stage is:

$$E(A | Z = 1) - E(A | Z = 0).$$

Under relevance, assignment independence, exclusion, and monotonicity, the Wald ratio:

$$\frac{E(Y | Z = 1) - E(Y | Z = 0)}{E(A | Z = 1) - E(A | Z = 0)}$$

identifies a local average effect for compliers whose receipt changes with assignment. It is not automatically the ATE. Exclusion can fail if assignment itself motivates action; monotonicity excludes defiers.

Weak first stages create unstable ratios and unusual inference.

## 41.6 Per-protocol, attrition, and missing outcomes

Per-protocol effects concern adherence to a strategy and require adjustment for post-assignment predictors of adherence and outcome. Simply restricting to adherers is biased.

Attrition can destroy exchangeability if related to assignment and prognosis. Report reasons and rates by arm, preserve assignment, model or weight observation under explicit assumptions, and perform sensitivity analyses for missing outcomes.

Complete-case precision does not prove missingness is harmless.

## 41.7 Minimal randomized laboratory

Simulate assignment independently, generate noncompliance and outcomes, then calculate assignment risk difference, receipt difference, first stage, and Wald ratio. Repeat many trials to distinguish one noisy realization from estimator behavior.

## Day 41 mastery assignment

### Level 1 — Trial mechanics

1. Distinguish allocation sequence generation, allocation concealment, blinding, treatment receipt, adherence, and outcome ascertainment.
2. Explain why baseline balance is expected across repeated randomizations but not guaranteed in one trial.
3. Identify the independent analysis unit in individual, classroom, clinic, and district randomization.
4. State the causal question answered by ITT.

### Level 2 — Hand calculations

For a trial with 200 assigned to review and 200 to control, 34 versus 50 warnings, and 150 versus 20 actually receiving review:

1. calculate arm risks, ITT risk difference, risk ratio, and first stage;
2. calculate the Wald estimate and name its target population;
3. compare treatment-received risks and explain why their difference is not protected by randomization; and
4. calculate cluster-level means for a supplied four-district version before attempting individual regression.

### Level 3 — Randomization simulation

1. Simulate 1,000 blocked trials and verify average ITT bias and interval coverage.
2. Add noncompliance driven by prognosis. Compare ITT, as-treated, per-protocol restriction, and Wald LATE.
3. Violate exclusion by making assignment itself motivate better reporting. Explain the changed meaning of the Wald ratio.
4. Simulate 12 cluster-randomized districts with within-district correlation. Compare naive individual standard errors with cluster-aware uncertainty.
5. Add differential attrition and implement best/worst-case outcome bounds.

### Level 4 — Research challenge

Write a randomized evaluation protocol for a realistic intervention. Include randomization unit, blocking, concealment, treatment versions, adherence measurement, spillovers, primary ITT, secondary per-protocol strategy, attrition prevention, missing-outcome sensitivity, analysis unit, stopping, harms, and transport population.

### Probing questions

- Can ITT be policy-relevant when adherence is low?
- When is cluster randomization necessary rather than convenient?
- Why does exclusion concern assignment as well as treatment?
- What population comprises compliers, and can they be identified individually?
- Could blinding be impossible yet randomized evidence remain useful?

## Completion evidence

Mastery means you can recover the effect of assignment from raw arm counts, diagnose why receipt comparisons fail, state every LATE assumption, and align uncertainty with the randomization unit.

---

# Day 42 — Standardisation, matching, and weighting

---

## 42.1 Three views of the same identified target

Under the same consistency, exchangeability, and positivity assumptions:

### Standardization

$$\psi = E_X\{m_1(X) - m_0(X)\}, \quad m_a(X) = E(Y \mid A = a, X).$$

Fit outcome means, predict every row under both actions, average, subtract.

### Inverse probability weighting

$$\psi = E \left[ \frac{AY}{e(X)} - \frac{(1-A)Y}{1-e(X)} \right].$$

Weight each observed outcome by inverse probability of receiving its action.

### Matching or balancing

Construct treated and untreated comparisons with similar confounder distributions, then average the contrast over the intended population.

Agreement cannot test unmeasured exchangeability because all methods can share the same hidden bias.

## 42.2 Regression standardization

1. Fit  $m(A, X)$ .
2. copy every row and set  $A = 1$ ; predict  $\hat{m}_1(X_i)$ .
3. Set  $A = 0$ ; predict  $\hat{m}_0(X_i)$ .
4. Average each intervention world.
5. Subtract.

Even with logistic regression this produces a marginal risk difference. The treatment coefficient alone is a conditional log-odds ratio, not the ATE.

## 42.3 Propensity scores

$$e(X) = P(A = 1 \mid X).$$

The true propensity is a balancing score under the causal assumptions. Its job is design and covariate balance, not maximum treatment AUC. Perfect separation reveals absent comparisons, not a wonderful model.

Use propensity scores for matching, strata, IPW, overlap weighting, or balancing estimation. Select specifications using causal knowledge, overlap, and balance—not the preferred effect estimate.

## 42.4 Matching changes the target population

Document distance, caliper, ratio, replacement, exact matches, order, discards, balance, overlap, and outcome analysis. Discarding unmatched units often changes ATE to an effect in matched/overlap-support units.

Design matching without repeatedly inspecting outcomes. Account for reused controls, pairs, and matching weights in inference.

## 42.5 Weighting and pseudo-populations

ATE weight:

$$w_i = \frac{A_i}{e(X_i)} + \frac{1 - A_i}{1 - e(X_i)}.$$

ATT keeps treated weight 1 and controls weight  $e/(1 - e)$ . Stabilized weights can reduce variance. Overlap weights emphasize treatment equipoise and target the overlap population.

Extreme weights reveal weak support or misspecification. Truncation trades variance for bias and defines a registered procedure. Do not tune truncation after seeing effects.

## 42.6 Balance and effective sample size

Standardized mean difference:

$$SMD_j = \frac{\bar{X}_{1j} - \bar{X}_{0j}}{\sqrt{(s_{1j}^2 + s_{0j}^2)/2}}.$$

Inspect weighted continuous distributions, categories, nonlinear terms, and interactions required by the graph. A threshold like 0.1 is a convention, not proof.

$$ESS = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

Concentrated weights can reduce thousands of rows to far less effective information. Plot treatment-specific propensities and inspect influential profiles.

## 42.7 Reading the adjustment code

The outcome pipeline includes treatment and baseline covariates, then predicts copied rows under both treatments. The propensity pipeline predicts treatment from baseline covariates. Preprocessing and model fitting must remain inside the relevant design/resampling boundary.

Clipping propensities is not innocuous. Report raw range, clipping rule, clipped count, balance, weights, ESS, and sensitivity. Flexible nuisance models still require causal covariate timing.

# Day 42 mastery assignment

## Level 1 — Method mapping

1. Match standardization, IPW, matching, stratification, ATT weighting, and overlap weighting to the population each targets.
2. Explain why a treatment-prediction AUC of 1 can be disastrous for ATE estimation.
3. Identify which components model outcome, treatment assignment, or covariate distribution.
4. Explain why propensity analysis should initially hide outcomes.

## Level 2 — Hand calculations

For five rows with supplied  $A, Y, e(X), m_1(X), m_0(X)$ :

1. calculate standardized intervention means and ATE;
2. calculate Horvitz–Thompson ATE contributions;
3. calculate ATE, ATT, stabilized, and overlap weights;
4. calculate weighted means, one SMD, and ESS; and
5. identify which row dominates and why.

Then fit a two-stratum example by hand and show that standardization and correctly weighted estimation target the same marginal contrast.

## Level 3 — Design laboratory

1. Hide outcomes and design four comparisons: nearest-neighbor matching, propensity strata, ATE IPW, and overlap weighting.
2. Produce propensity-overlap plots, love plots for SMDs, weight histograms, ESS, discarded-profile table, and target-population description.
3. Freeze designs, reveal outcomes, and estimate effects with uncertainty respecting pairs/weights.
4. Remove terrain from the propensity model and observe whether balance diagnostics detect the omission.
5. Force deterministic treatment in remote projects. Compare unrestricted ATE with restricted and overlap-population effects.

## Level 4 — Research challenge

Design an observational adjustment analysis without using outcome data. Justify the confounder set from a DAG, choose a primary target population, define acceptable overlap, register match/weight rules, specify what happens if balance fails, and write the exact sentence that will replace an ATE claim if support is inadequate.

## Probing questions

- When matching discards treated units, whose effect remains?
- Can all SMDs be small while nonlinear or joint imbalance remains?
- Is weight truncation a numerical fix or an estimand/procedure change?
- Why can standardization extrapolate silently where IPW becomes visibly unstable?
- Which method makes your target population easiest to explain?

## Completion evidence

Mastery requires reproducing weights and standardized predictions by hand, reporting balance beyond one threshold, identifying the target population after design, and refusing unsupported extrapolation.

---

# Day 43 — Doubly robust estimation and sensitivity

---

## 43.1 AIPW score

$$\hat{\phi}_i = \hat{m}_1(X_i) - \hat{m}_0(X_i) + \frac{A_i\{Y_i - \hat{m}_1(X_i)\}}{\hat{e}(X_i)} - \frac{(1 - A_i)\{Y_i - \hat{m}_0(X_i)\}}{1 - \hat{e}(X_i)}.$$

Average scores to estimate ATE. The first term standardizes predicted outcomes. Weighted residual terms correct prediction error using observed outcomes.

## 43.2 Double robustness—precise meaning

Under identification and regularity conditions, AIPW can be consistent if either the outcome models or propensity model are correctly specified.

It does not mean two wrong models cancel, hidden confounding vanishes, positivity is optional, measurement no longer matters, or standard errors are automatic. Robustness concerns nuisance functions inside an already identified problem.

## 43.3 Influence-function uncertainty

$$\hat{\psi} = \frac{1}{n} \sum_i \hat{\phi}_i, \quad \widehat{SE}(\hat{\psi}) = \frac{SD(\hat{\phi}_i)}{\sqrt{n}}.$$

Large scores reveal influential residuals and small propensities. Clustered data require cluster-level influence aggregation or appropriate bootstrap.

## 43.4 Cross-fitting

Split observations into folds. Train nuisance models outside each fold and predict  $e, m_1, m_0$  inside the held-out fold. After all rows have out-of-fold predictions, compute the effect score once.

Cross-fitting reduces own-observation overfit and enables flexible nuisance estimation under rate conditions. It does not tune itself, prevent preprocessing leakage, or repair causal assumptions. Any tuning stays within nuisance training data.

## 43.5 TMLE and double machine learning

TMLE updates initial outcome predictions along a targeted fluctuation submodel and can preserve outcome bounds. Double/debiased machine learning uses orthogonal scores and sample splitting so small nuisance errors have reduced first-order influence.

These are frameworks, not magic brands. Report estimand, score, learners, folds, tuning, truncation, variance, and failures.



## 43.6 Whole-procedure bootstrap

Resample the independent unit, redo folds, transformations, tuning, nuisance fitting, score calculation, and effect estimation. Bootstrapping only saved predictions omits fitting uncertainty.

Use cluster or block bootstrap where independence is higher-level. Compare full-procedure and influence intervals; disagreement is diagnostic, not permission to choose the favorable interval.

## 43.7 Sensitivity to hidden confounding

Observed balance cannot prove exchangeability. Methods include bias-parameter grids, tipping points, E-values for suitable ratio measures, Rosenbaum bounds for matched designs, negative controls, quantitative bias analysis, and partial-identification bounds.

Each has assumptions. Calibrate sensitivity parameters against measured covariates and domain evidence. An E-value is not a certificate; a negative control only detects biases it shares.

## 43.8 Specification and overlap sensitivity

Change one registered choice at a time: nonlinear terms, justified covariate sets, weight truncation, overlap target, outcome definition, missing-data scenario, or negative control.

Agreement among models sharing the same flawed identification strategy is not causal robustness.

## Day 43 mastery assignment

### Level 1 — AIPW anatomy

1. Label the plug-in contrast and two residual-correction terms in the AIPW score.
2. Explain what happens to a treated row's correction when  $e(X)$  approaches zero.
3. State double robustness precisely and list five things it does not guarantee.
4. Explain cross-fitting to a reader who knows train/test splitting but not influence functions.

### Level 2 — Score calculation

Using a six-row table of  $A, Y, \hat{e}, \hat{m}_1, \hat{m}_0$ :

1. compute every AIPW score;
2. average them and calculate the influence-score standard error;
3. identify the most influential row;
4. recalculate after clipping propensities; and
5. explain the bias–variance trade-off introduced by clipping.

### Level 3 — Four-cell failure matrix

Simulate a nonlinear confounded outcome and compare g-computation, IPW, in-sample AIPW, and cross-fitted AIPW when:

1. both nuisance models are adequate;
2. only the outcome model is adequate;
3. only the propensity model is adequate; and
4. neither is adequate.

Repeat across sample sizes and overlap levels. Report bias, RMSE, coverage, nuisance calibration, maximum score, and failure rate. Then add an unmeasured confounder and demonstrate that all four statistically sophisticated estimators can converge to the wrong causal answer.

## Level 4 — Uncertainty and sensitivity study

Implement a whole-procedure bootstrap that resamples the independent unit, recreates folds, retunes nuisance learners, and recomputes AIPW. Compare percentile, normal, and influence intervals. Add a quantitative bias analysis calibrated to the strongest measured confounders and identify a decision-relevant tipping point.

### Probing questions

- Why can good nuisance prediction coexist with biased effect estimation?
- Which product of nuisance errors matters for orthogonal estimation?
- What uncertainty is omitted when saved scores alone are bootstrapped?
- How should folds change for clustered or temporal data?
- What hidden-confounding strength would be plausible in your domain?

### Completion evidence

Mastery means you can calculate the score manually, implement out-of-fold nuisance prediction without leakage, demonstrate double robustness by simulation, and present a substantively calibrated sensitivity statement.

---

# Day 44 — Target trials and time-varying treatment

---

## 44.1 Write the imaginary experiment

A target trial specifies eligibility, strategies and versions, assignment, time zero, outcome/horizon, causal contrast, and analysis. Observational emulation follows that protocol as closely as data allow.

Eligibility assessment, treatment assignment, and follow-up must align at time zero. Selecting month-12 survivors while calling review at month 6 baseline creates a mismatched population.

## 44.2 Immortal-time and time-zero bias

If “treated” means receiving review any time in the first year but follow-up starts at approval, treated projects must survive warning-free until review. Assigning this guaranteed time to treatment can make it appear protective.

Solutions include aligned time zero, time-varying treatment, repeated trials, clone–censor–weight, or properly analyzed grace periods. Adding time-to-review as a baseline covariate does not repair design.

## 44.3 Longitudinal histories

At visit  $k$ ,  $L_k$  is covariate history observed before decision  $A_k$ . Bars denote history:

$$\bar{L}_k = (L_0, \dots, L_k), \quad \bar{A}_k = (A_0, \dots, A_k).$$

A dynamic strategy  $d$  maps history to action.  $Y^d$  is outcome under sustained strategy. Define realistic grace periods and adherence.

## 44.4 Treatment–confounder feedback

Earlier treatment affects later health; later health affects both future treatment and outcome. Conventional adjustment for later health blocks part of early treatment effect, while omitting it confounds later treatment.

This motivates longitudinal g-methods, not ordinary baseline regression.

## 44.5 Sequential assumptions

$$Y^{\bar{a}} \perp A_k \mid \bar{L}_k, \bar{A}_{k-1}$$

is sequential exchangeability. Strategies also require sequential positivity, longitudinal consistency, censoring assumptions, and interference treatment. Richer history makes support harder.

## 44.6 Parametric g-formula

Fit models for next covariates and outcomes given past history; sample baseline units; impose strategy; simulate covariates forward; impose later actions; simulate outcomes; average; repeat for each strategy.

It standardizes over complete histories and depends on correct longitudinal process models. Make Monte Carlo simulation error negligible relative to sampling uncertainty.

## 44.7 Marginal structural models

Stabilized longitudinal treatment weight:

$$SW_i = \prod_k \frac{P(A_{ik} \mid \bar{A}_{i,k-1}, L_{i0})}{P(A_{ik} \mid \bar{A}_{i,k-1}, \bar{L}_{ik})}.$$

Additional censoring weights handle measured informative loss. Fit a marginal outcome model in the pseudo-population.

Inspect probabilities and weights by visit, late-history support, balance, ESS, truncation sensitivity, and treatment/censoring calibration. Multiplication can make long-horizon weights unstable.

## 44.8 Clone–censor–weight and repeated trials

Clone a subject into every strategy compatible at baseline, artificially censor a clone on deviation, and weight continued compatibility. Clones share origin, so inference must account for dependence. Artificial censoring requires its own ignorability model.

Repeated target trials enroll eligible projects at successive months, align assignment and follow-up each time, then pool with trial/calendar controls and project-clustered inference. Treatment versions and policy periods may change.

## 44.9 Censoring and competing outcomes

Cancellation may remain a competing event for total real-world incidence, join a composite policy-failure outcome, be eliminated in a hypothetical estimand under stronger assumptions, or enter separable-effects analysis.

Calling it censoring changes the question and can create informative loss. Carry Chapter 5's event-process discipline into causal work.

## Day 44 mastery assignment

### Level 1 — Timeline diagnosis

1. Draw eligibility, assignment, treatment receipt, outcome follow-up, censoring, and competing-event times for five example projects.
2. Mark every immortal period created by an “ever treated” definition.
3. Rewrite a misaligned observational question so eligibility, strategy assignment, and follow-up share time zero.
4. Distinguish baseline, time-varying, and dynamic strategies.

### Level 2 — Longitudinal notation

For three visits, write  $L_0, A_0, L_1, A_1, L_2, A_2, Y$  in temporal order. Specify two static strategies and one realistic dynamic rule. State sequential exchangeability and positivity at each decision.

Given numerator and denominator treatment probabilities at each visit, calculate individual stabilized weights, final ESS, and the change after adding censoring weights.

### Level 3 — Simulation laboratory

1. Simulate treatment–confounder feedback where  $A_0$  affects  $L_1$ , which affects  $A_1$  and  $Y$ .
2. Compare naive final-treatment regression, conventional adjustment for  $L_1$ , parametric g-formula, and an MSM.
3. Plot weight growth by visit and truncate on a registered grid.
4. Implement clone–censor–weight for two sustained strategies and cluster inference by original subject.
5. Emulate repeated monthly trials and verify one project never crosses train/test or independent-unit boundaries incorrectly.

### Level 4 — Research challenge

Write a full target-trial table for a longitudinal intervention with grace period. Define eligibility, strategies, assignment, time zero, follow-up, outcome, competing events, adherence, causal contrast, covariate history, censoring, analysis, and transport. Create a data-availability matrix showing whether every required variable is measured before each decision.

### Probing questions

- Why does adjusting for  $L_1$  both help and harm in treatment–confounder feedback?
- When does a dynamic strategy violate positivity even if each single treatment is common?
- What new assumption does artificial censoring introduce?
- How should cancellation enter a total-effect versus hypothetical estimand?
- When do repeated trials improve information but worsen transport ambiguity?

### Completion evidence

Mastery requires detecting immortal time from a timeline, computing longitudinal weights, explaining g-formula versus MSM assumptions, and producing an implementable target-trial protocol whose variables exist at the needed times.

---

# Day 45 — Quasi-experimental designs

---

## 45.1 Difference-in-differences

$$\hat{\tau}_{DiD} = (\bar{Y}_{T,post} - \bar{Y}_{T,pre}) - (\bar{Y}_{C,post} - \bar{Y}_{C,pre}).$$

Identification requires treated group's untreated counterfactual trend to match comparison trend on the chosen scale. Pre-trends can reveal problems but cannot prove post-policy parallel trends.

Threats include anticipation, changing composition, concurrent shocks, measurement change, spillovers, functional form, and serial correlation. Cluster inference at policy-assignment level.

## 45.2 Event studies and staggered adoption

Event studies show effects relative to adoption and pre-period patterns. With staggered timing and heterogeneous effects, ordinary two-way fixed effects can combine inappropriate comparisons and negative weights.

Use group-time estimands with never-treated or not-yet-treated comparisons, transparent aggregation, cohort/event-time support, and modern methods such as Callaway–Sant'Anna when assumptions fit.

## 45.3 Regression discontinuity

For cutoff  $c$ :

$$\tau_{RD} = \lim_{r \downarrow c} E(Y \mid R = r) - \lim_{r \uparrow c} E(Y \mid R = r).$$

This is local to the cutoff. It requires continuity of potential outcomes absent treatment and no precise manipulation around the threshold.

Plot raw data and treatment probability, inspect running-variable density and baseline continuity, use low-order local polynomials, justified bandwidths, robust bias correction, and bandwidth/kernel sensitivity. Avoid high-order global polynomials.

Fuzzy RD divides outcome discontinuity by treatment discontinuity and identifies a local complier effect under IV-like assumptions.

## 45.4 Instrumental variables

Wald ratio:

$$\frac{E(Y \mid Z = 1) - E(Y \mid Z = 0)}{E(A \mid Z = 1) - E(A \mid Z = 0)}.$$

Requires relevance, independence, exclusion, and usually monotonicity. Under heterogeneity it targets compliers, not population ATE. Weak instruments yield instability; a strong first stage does not defend exclusion.

Distance to a review office may directly affect outcomes, violating exclusion. Attack the DAG before fitting two-stage regression.

## 45.5 Synthetic control and interrupted time series

Synthetic control weights unaffected donors to reproduce a treated unit's pre-policy trajectory. It needs stable long pre-period, unaffected donors, good fit, and placebos.

Interrupted time series estimates level/slope changes and needs enough observations, seasonality/autocorrelation handling, stable measurement, and absence of concurrent explanations. A comparison series strengthens it.

## 45.6 Placebos and falsification

Use fake dates/cutoffs, outcomes treatment cannot affect, pre-treatment effects, covariate continuity, instrument associations with baseline outcomes, and leave-one-donor-out checks.

Passing tests cannot prove identification; failing a well-chosen falsification requires investigation.

## Day 45 mastery assignment

### Level 1 — Design identification

For ten short scenarios, choose adjustment, randomized trial, DiD, RD, IV, synthetic control, or interrupted time series. For each, name the assignment leverage, target population, estimand, and central untestable assumption.

### Level 2 — Hand calculations

1. Calculate a two-period DiD from four cell means and reconstruct the missing treated counterfactual mean.
2. Calculate sharp-RD jumps from left/right limits and explain why the effect is local.
3. Calculate reduced form, first stage, and Wald IV ratio from binary assignment counts.
4. Construct a simple three-donor synthetic-control weighted pre-period trajectory.
5. Calculate an interrupted-series level and slope change from a supplied segmented regression.

### Level 3 — Failure laboratories

1. DiD: simulate parallel trends, then add anticipation, heterogeneous staggered effects, serial correlation, composition change, and a concurrent shock. Compare two-way fixed effects with group-time effects.
2. RD: simulate a sharp cutoff, estimate local linear effects over bandwidths, then add score manipulation and a discontinuous baseline covariate.
3. IV: simulate valid compliance types, then add weak relevance, direct instrument effect, and defiers.
4. Synthetic control: remove one donor at a time and run placebo interventions.
5. Interrupted series: add seasonality and autocorrelation, then show naive standard errors.

### Level 4 — Research challenge

Find a real or hypothetical policy assignment mechanism and prepare a design memo before outcomes. Include institutional history, exact estimand, comparison population, manipulation/anticipation/spillover risks, falsification tests, inference unit, data support, and reasons alternative designs are weaker or stronger.

### Probing questions

- Why can many parallel pre-periods still fail to identify DiD?
- Who are controls under staggered adoption?
- How far from an RD cutoff may the result reasonably travel?
- Can instrument relevance be strong while exclusion is implausible?

- Which placebo would most directly expose your design's likely failure?

## Completion evidence

Mastery means you can derive each basic contrast, state its local/group-time/complier population, select design-specific diagnostics, and decline “natural experiment” language without a defended assignment mechanism.

---

# Day 46 — Heterogeneous effects and policy learning

---

## 46.1 CATE is still an average

$$\tau(x) = E\{Y(1) - Y(0) \mid X = x\}.$$

It is an average for profile/subgroup, not an observed individual effect. Even randomized trials do not reveal both outcomes for one person.

## 46.2 Confirmatory and exploratory heterogeneity

Prespecify a small set of plausible modifiers, coding, functional form, scale, interaction contrast, multiplicity control, sample/overlap requirements, and missingness handling.

“Significant here, nonsignificant there” does not prove interaction; test the difference. Exploratory discoveries need independent confirmation and must be labeled hypothesis-generating.

## 46.3 Effect scale matters

Constant risk ratio can yield larger absolute benefit in high-risk groups. Constant risk difference can yield varying ratios. Report baseline risks and register the policy-relevant scale.

## 46.4 Subgroup AIPW scores

Cross-fitted AIPW scores can be averaged within prespecified groups to estimate subgroup effects, with group-appropriate uncertainty. Nuisance models can be global while effect aggregation is subgroup-specific.

Small groups, poor treatment overlap, multiple testing, and learned subgroup boundaries undermine apparent precision.

## 46.5 Meta-learners

- S-learner fits one outcome model including treatment.
- T-learner fits separate treated/control models.
- X-learner imputes pseudo-effects and can help unequal group sizes.
- R-/DR-learners residualize or use orthogonal pseudo-outcomes.

Their assumptions and tuning matter. Cross-fitting reduces overfit but does not create exchangeability.

## 46.6 Honest causal trees and forests

Honesty separates data choosing tree splits from data estimating leaf effects. Causal forests average honest trees and can estimate conditional effects with uncertainty under conditions.

Tune for treatment-effect estimation rather than outcome prediction. Require overlap and sufficient treated/control observations in leaves. Importance does not prove true effect modification.

## 46.7 Evaluating heterogeneity without labels

Individual treatment effects are unobserved, so ordinary prediction RMSE is unavailable. Use simulation, randomized validation, held-out policy value, calibration of grouped scores, best-linear-predictor tests, and stability across samples.

Ranking apparent effects can overfit noise even if average ATE is correct.

## 46.8 From CATE to policy

A policy treats when expected benefit exceeds cost subject to capacity and constraints. Estimate policy value on data not used to learn the rule, using randomized comparison or doubly robust off-policy evaluation.

Compare with simple treat-all, treat-none, and risk-based rules. Account for treatment cost, harm, capacity, uncertainty, and implementation. A high-risk subject need not benefit most.

## 46.9 Fairness and transport

Audit who is eligible, has overlap, receives action, benefits, bears burden, and is excluded. Maximizing average value can concentrate resources or harms. Protected group rules may face legal and ethical constraints.

Transporting CATE requires stable conditional effects or measured effect modifiers plus target-population data. New settings can differ in treatment version, adherence, resources, baseline risk, and interference.

## Day 46 mastery assignment

### Level 1 — Interpretation

1. Explain why CATE is not an individual treatment effect.
2. Give an example of constant risk ratio with varying risk difference.
3. Distinguish prognostic risk from treatment benefit.
4. Explain why subgroup significance versus nonsignificance is not an interaction test.

### Level 2 — Prespecified subgroup work

Using cross-fitted AIPW scores and a prespecified binary modifier:

1. calculate group-specific effects and standard errors;
2. calculate the interaction contrast;
3. report baseline risks, overlap, treated/control counts, and multiplicity plan;
4. repeat on risk-difference and risk-ratio scales; and
5. explain why conclusions differ across scales.



## Level 3 — Learner comparison

Simulate constant and heterogeneous effects. Compare S-, T-, X-, R-, and DR-learners plus honest causal forest using nested/cross-fitted nuisance estimation. Evaluate:

- ATE recovery;
- calibration of grouped predicted effects;
- policy value on untouched data;
- stability of modifier rankings;
- overlap within learned subgroups; and
- performance under a truly constant effect.

Do not evaluate individual CATE RMSE on real data where individual effects are unobserved.

## Level 4 — Policy learning challenge

Learn a capacity-constrained rule that may treat 20% of projects. Compare treat-all, treat-none, highest-risk, and highest-estimated-benefit policies using doubly robust held-out policy value. Add treatment cost, uncertainty penalty, minimum group access, and a rule-complexity constraint. Produce a policy card describing eligibility, expected value, burden, exclusions, support, appeals, and monitoring.

## Probing questions

- When can a high-risk rule outperform an estimated-benefit rule?
- How does weak overlap masquerade as large heterogeneity?
- What is gained and lost through honest sample splitting?
- Which fairness constraint belongs in policy learning rather than model fitting?
- What evidence is required to transport a modifier to another setting?

## Completion evidence

Mastery requires separating risk and benefit, testing interactions directly, evaluating learned policies out of sample, and reporting uncertainty/support for every subgroup used in a decision.

---

# Day 47 — Comparison, reproducibility, and responsible production

---

## 47.1 Define what is compared

Distinguish algorithms, tuned procedures, estimands, decision policies, and complete systems. Equalize data, folds, preprocessing, search budgets, censoring/uncertainty rules, and action definitions where the scientific comparison requires it.

## 47.2 Paired evaluation

Compare methods on the same observations or resamples and analyze paired differences. Pairing removes shared case difficulty from variance.

Repeated CV folds overlap heavily. Fold scores are not independent replicates, so ordinary paired  $t$ -tests on folds understate uncertainty. Use nested/locked evaluation, corrected resampled tests with limitations, bootstrap at the independent unit, or dataset-level comparison across genuinely independent tasks.

## 47.3 Practical equivalence and multiplicity

Predefine the smallest practically important difference and consider equivalence/noninferiority, not only “different from zero.” Multiple models, metrics, horizons, and groups create selection opportunities; control or label multiplicity and preserve all results.

## 47.4 Ablation studies

Removing a feature or component asks how this fitted procedure changes without it. It does not establish the component's causal importance. Rerun preprocessing and tuning fairly for each ablation, use paired evidence, and disclose the full ablation set.

## 47.5 Reproducibility levels

- Computational reproducibility: same data/code/environment regenerate outputs.
- Replicability: independent study obtains compatible evidence.
- Robustness: conclusions survive justified analysis choices.
- Generalizability/transport: results hold in new populations/settings.

One does not imply the others.

## 47.6 Research bundle

Include registration, README, scripted cohort construction, data dictionary, environment lock, seeds/determinism notes, tests, analysis code, all results, individual predictions/scores where allowed, figures, model objects, manifests/hashes, data sheet, model card, causal-study card, deviation log, and plain-language summary.

No hidden notebook state or manual spreadsheet step should be required.

## 47.7 Seeds, hashes, and manifests

A seed controls specified pseudo-randomness but cannot guarantee identical results across libraries, hardware, parallelism, or versions. Record all seed locations and deterministic limitations.

A cryptographic hash fingerprints an exact file; it does not prove correctness. A manifest lists paths, sizes, hashes, sources, timestamps, licenses, and roles. Hash raw and derived artifacts while respecting privacy.

## 47.8 Data sheets, model cards, and causal-study cards

Data sheet: motivation, creators, units, collection, sampling, timestamps, missingness, labels, groups, consent/governance, limitations, update history.

Model card: intended use, prohibited use, population, inputs/outputs, training, metrics, groups, thresholds, calibration, uncertainty, failures, monitoring, version.

Causal-study card: target trial, estimand, DAG, identification assumptions, assignment mechanism, overlap, adjustment, diagnostics, sensitivity, transport, deviations, allowed wording.

## 47.9 Training–serving consistency

Verify production feature definitions, units, category handling, missingness, time availability, model version, threshold, and action exactly match validation. Use schema tests, fixture predictions, shadow runs, and parity checks.

## 47.10 Monitor the decision system

Monitor service health, input/measurement drift, missingness, predicted values, treatment/action rates, capacity, overrides, subgroup burden, delayed outcomes, calibration/impact when mature, incidents, appeals, and downstream consequences.

Actions create feedback: intervention changes outcomes and future labels; selection changes who is observed; staff adapt to scores. Monitoring a score distribution alone misses system drift.

## 47.11 Retraining, rollback, and retirement

Predefine triggers, owners, validation gates, prospective/shadow periods, versioning, rollback artifacts, communication, and retirement criteria. Retraining is a new intervention and can invalidate causal interpretation. Preserve prior versions and decision logs.

## Day 47 mastery assignment

### Level 1 — Reproducibility audit

Classify each artifact as needed for computational reproducibility, replication, robustness, transport, or governance: source data hash, environment lock, seed, independent dataset, sensitivity grid, model card, causal-study card, incident runbook, and external validation.

Explain why a seed, notebook, container, or hash alone is insufficient.

### Level 2 — Paired comparison

Given two models' losses on the same 20 projects:

1. calculate rowwise differences, mean difference, standard error, and interval;
2. compare with an incorrect unpaired calculation;
3. define a smallest practically important difference and assess equivalence;
4. adjust interpretation for five metrics and four horizons; and
5. explain why ten cross-validation fold scores are not ten independent studies.

### Level 3 — Build the research bundle

From a clean checkout or directory, create a one-command workflow that regenerates cohort count, primary estimate, interval, table, and figure. Include:

- README and entry command;
- environment lock;
- data and artifact manifests with hashes;

- deterministic limitations;
- schema and synthetic fixture tests;
- full result tables, not only winners;
- data sheet, model card, and causal-study card;
- registration and deviation log; and
- plain-language summary.

Give the bundle to a peer without verbal help and record every hidden dependency.

## Level 4 — Production stress test

Design and rehearse incidents for a unit conversion, unseen category, delayed outcomes, subgroup action spike, capacity failure, corrupted feature timestamp, feedback-driven label change, and harmful override pattern. For each specify detection, severity, owner, containment, rollback, communication, root-cause analysis, and prevention.

## Probing questions

- Can an exactly reproducible result be causally invalid?
- What does a hash prove and not prove?
- Which changes require recalibration, full revalidation, or a new causal study?
- How does deployment change the future data-generating process?
- What retirement signal cannot be solved by retraining?

## Completion evidence

Mastery requires an independent peer successfully reproducing the registered outputs, a defensible paired comparison, complete cards/manifests, and a tested rollback path.

---

# Day 48 — The registered final study

---

## 48.1 Registration and lockbox

Freeze causal question, target trial, estimand, DAG, adjustment set, data rules, exclusions, missingness, primary estimator, nuisance learners/tuning, diagnostics, sensitivity, uncertainty, groups, claim rule, and code version before primary outcomes are opened.

Lockbox access should be logged and technically separated. After opening, changes are deviations or exploratory. Reusing the test to revise the procedure consumes it.

## 48.2 Executable primary analysis

A complete script regenerates cohort, validates timestamps, constructs treatment/outcome, fits the registered method, computes diagnostics and effect, produces uncertainty, saves artifacts, and exits on failed integrity assertions.

Run synthetic fixtures with known truth and known failure cases before real outcomes. Assertions should check eligibility, treatment timing, outcome window, feature timing, overlap, finite scores, reproducible counts, and output schema.

## 48.3 Integrity before interpretation

Verify cohort counts and flow, duplicates, missingness, treatment provenance, balance, overlap, weights/ESS, nuisance predictions, influential scores, seed/environment, hash manifest, and lockbox log before interpreting the estimate.

If a primary integrity test fails, stop. Do not rationalize because the estimate looks plausible.

## 48.4 Writing the result

Report population, intervention/comparator, time zero, horizon, estimand/scale, design, identification assumptions, estimator, diagnostics, primary estimate and interval, sensitivity, deviations, transport limits, and allowed decision implication.

A narrow interval does not incorporate unmeasured confounding or undefined treatment versions. Null, harmful, unstable, or unidentified results must be reported.

## 48.5 Three graduation capstones

### Capstone A: target-trial emulation

Emulate senior review using aligned eligibility and treatment, causal adjustment, overlap/balance, sensitivity, and bounded claim.

### Capstone B: threshold or rollout evaluation

Use RD, DiD, IV, or another defensible assignment mechanism; define its local/group-time/complier estimand and falsification evidence.

### Capstone C: prospective model-impact trial

Randomize or prospectively compare use of a decision system, recording actions, adherence, spillovers, harms, capacity, subgroup outcomes, and workflow impact.

Prediction performance and intervention impact are separate endpoints.

## 48.6 Oral defense

Be ready to defend why the estimand matters, how treatment is defined, where time zero lies, which paths are adjusted, what is unmeasured, where positivity fails, why estimator matches identification, how uncertainty was calculated, what sensitivity means, where results transport, and what would falsify the claim.

“The software did it” is not a research defense.

## Day 48 mastery assignment

### Level 1 — Protocol reconstruction

Read a published or fictional result paragraph and reconstruct its population, treatment, comparator, time zero, outcome, horizon, estimand, design, assumptions, estimator, uncertainty, and transport claim. Mark every missing element.

## Level 2 — Lockbox rehearsal

1. Create synthetic development and lockbox files with hashes.
2. Write integrity tests before opening lockbox outcomes.
3. Freeze configuration, code revision, environment, and allowed claim rules.
4. Simulate an integrity-test failure and practice stopping without inspecting the effect.
5. Open once, save individual scores/predictions where allowed, and create an immutable access/deviation record.

## Level 3 — Complete registered mini-study

Choose one:

1. baseline target-trial emulation with standardization/IPW/AIPW;
2. RD, DiD, or IV analysis tied to a genuine assignment mechanism; or
3. prospective randomized model-impact evaluation.

Submit registration before primary outcome access. Deliver cohort flow, DAG, estimand, primary and secondary methods, diagnostics, sensitivity, uncertainty, groups, full results, research bundle, cards, and a 300-word bounded conclusion.

## Level 4 — Adversarial oral defense

Ask a peer to challenge:

- treatment versions and time zero;
- missing confounders and collider adjustment;
- positivity and target population;
- estimator/estimand mismatch;
- multiplicity and researcher choices;
- uncertainty unit;
- sensitivity calibration;
- transport, interference, and implementation;
- unfavorable or null findings; and
- conditions that would reverse the decision.

Revise documentation, not the locked primary estimate, in response.

## Research and probing questions

- What observation would falsify the deployment claim?
- Which assumption contributes the largest uncertainty not shown in the interval?
- Would a different stakeholder choose a different estimand or effect scale?
- If the primary model loses to a simple baseline, what happens next?
- Which outcome or harm should be measured prospectively before deployment?
- What evidence would justify expanding from the study population?
- When is “we cannot identify this effect” the most useful result?

## Completion evidence

Chapter mastery is demonstrated when the study can be rerun from a clean environment, the lockbox history is auditable, every assumption is connected to evidence or sensitivity, an adversarial reader can reproduce the result, and the conclusion remains bounded after criticism.

---

# Capstone companion: exercises and certification

---

## C.1 Exercise method

Before coding each exercise, write:

1. target trial or assignment mechanism;
2. counterfactual estimand and population;
3. identification assumptions;
4. estimator and diagnostic;
5. deliberate failure case; and
6. bounded conclusion.

The exercises progress from potential-outcome arithmetic and DAG adjustment through trials, noncompliance, propensity design, standardization, doubly robust failure matrices, whole-procedure bootstrap, hidden-confounding tipping points, longitudinal treatment, DiD, RD, IV, honest heterogeneity, reproducibility, incidents, and a registered study.

## C.2 Certification dimensions

A research-ready submission:

- defines treatment versions and time zero;
- separates observed and counterfactual quantities;
- makes identification assumptions criticizable;
- matches adjustment to causal structure;
- demonstrates overlap and balance;
- propagates procedural uncertainty;
- distinguishes confirmatory and exploratory work;
- preserves unfavorable results and deviations;
- produces a clean research bundle;
- communicates transport and sensitivity limits; and
- connects deployment to monitoring, appeal, rollback, and prospective impact evaluation.

An honest conclusion that the effect is unidentified or too uncertain can pass. Unsupported certainty cannot.

---

# Chapter 6 synthesis and practical laboratories

---

## S.1 One question, six layers

1. Measurement: are treatment, outcome, time, and population valid?
2. Counterfactual target: which intervention worlds and scale?
3. Identification: which design and assumptions connect data to target?
4. Estimation/inference: which procedure and uncertainty?
5. Translation: where does result transport and what decision follows?

6. Production: how is action delivered, monitored, contested, and stopped?

Reproducibility preserves the chain; it cannot repair a wrong early link.

## S.2 Derivations to complete

1. Observed outcome from potential outcomes.
2. ATE, ATT, CATE, risk difference/ratio/odds ratio.
3. G-formula identification steps.
4. Backdoor adjustment for a small DAG.
5. ITT, first stage, and Wald LATE.
6. Standardization and IPW identification.
7. SMD and ESS.
8. AIPW score and influence standard error.
9. Longitudinal stabilized weights.
10. Two-period DiD.
11. Sharp and fuzzy RD contrasts.
12. Policy value and incremental net benefit.
13. Paired method differences and dependence explanation.

## S.3 Practical laboratories

### Laboratory A: simulation truth

Reveal both potential outcomes only after freezing analysis; compare naive, standardization, IPW, and AIPW over repeated samples.

### Laboratory B: DAG traps

Simulate a confounder, mediator, collider, selection variable, proxy, and instrument. Adjust valid and invalid sets and trace bias paths.

### Laboratory C: randomized noncompliance

Simulate compliers, always-takers, never-takers, and exclusion violation. Compare ITT, receipt contrast, first stage, and LATE.

### Laboratory D: overlap failure

Make treatment deterministic for one profile. Compare ATE, restricted, ATT, and overlap targets, weights, ESS, and extrapolation.

### Laboratory E: double robustness

Cross correct/incorrect outcome and propensity models over repetitions. Add hidden confounding to show all adjustment estimators fail together.

### Laboratory F: longitudinal feedback

Simulate three treatment decisions with confounder feedback. Compare naive adjustment, g-formula, and marginal structural model.



## Laboratory G: quasi-experiment contamination

Break DiD with concurrent shock, RD with manipulation, and IV with direct instrument effect. Test design-specific placebos.

## Laboratory H: honest heterogeneity

Use separate discovery and estimation samples; evaluate policy value on untouched data under true constant and varying effects.

## Laboratory I: reproducibility handoff

Give a peer only the bundle. Record and repair every hidden dependency needed to reproduce cohort, estimate, interval, and figure.

## Laboratory J: production incident

Simulate unit change, delayed labels, group action spike, or feedback. Exercise detection, containment, rollback, communication, and prevention.

# Formula sheet with plain-language readings

| Formula                                                                  | Meaning                          |
|--------------------------------------------------------------------------|----------------------------------|
| $Y = AY(1) + (1 - A)Y(0)$                                                | observed potential outcome       |
| $E[Y(1) - Y(0)]$                                                         | population ATE                   |
| $E[Y(1) - Y(0) \mid A = 1]$                                              | ATT                              |
| $E[Y(1) - Y(0) \mid X = x]$                                              | CATE                             |
| $Y(1), Y(0) \perp A \mid X$                                              | conditional exchangeability      |
| $0 < P(A = 1 \mid X) < 1$                                                | positivity                       |
| $E_X[E(Y \mid A = a, X)]$                                                | g-formula intervention mean      |
| $\bar{Y}_{Z=1} - \bar{Y}_{Z=0}$                                          | ITT estimate                     |
| outcome ITT / treatment first stage                                      | Wald LATE                        |
| $e(X) = P(A = 1 \mid X)$                                                 | propensity score                 |
| $A/e + (1 - A)/(1 - e)$                                                  | ATE weight                       |
| $SMD$                                                                    | standardized covariate imbalance |
| $(\sum w)^2 / \sum w^2$                                                  | effective sample size            |
| $m_1 - m_0 + A(Y - m_1)/e - (1 - A)(Y - m_0)/(1 - e)$                    | AIPW score                       |
| $SD(\phi)/\sqrt{n}$                                                      | influence-score standard error   |
| $\prod_k P(A_k \mid \text{limited past}) / P(A_k \mid \text{full past})$ | longitudinal stabilized weight   |
| $(T_{post} - T_{pre}) - (C_{post} - C_{pre})$                            | DiD contrast                     |

| Formula                       | Meaning                    |
|-------------------------------|----------------------------|
| outcome jump / treatment jump | fuzzy-RD or IV local ratio |
| $E[Yd(X)]$                    | policy value               |

## Expanded glossary

**ATE** — Average potential-outcome contrast in a stated population.

**ATT** — Average contrast among actually treated units.

**Backdoor path** — Treatment–outcome path beginning with an arrow into treatment.

**Balance** — Similarity of treatment-group covariate distributions after design.

**CATE** — Average effect conditional on baseline profile or subgroup.

**Causal estimand** — Counterfactual contrast including intervention, comparator, population, outcome, horizon, and scale.

**Collider** — Common effect; conditioning can associate its causes.

**Consistency** — Observed outcome equals potential outcome under received well-defined treatment.

**Confounder** — Pre-treatment common cause of treatment and outcome for the question.

**Cross-fitting** — Nuisance models trained outside observations receiving their predictions.

**DAG** — Directed acyclic graph encoding causal assumptions.

**Difference-in-differences** — Treated change minus comparison change under parallel counterfactual trends.

**Double robustness** — Consistency if one of two nuisance models is correct, given identification.

**Effective sample size** — Concentration summary for nonnegative weights.

**Exchangeability** — Treatment independent of potential outcomes, by design or conditionally.

**Exclusion restriction** — Instrument affects outcome only through treatment.

**G-formula** — Standardization of conditional outcome distributions over target covariates/histories.

**Honesty** — Separate data for structure discovery and effect estimation.

**Instrument** — Treatment shifter satisfying relevance, independence, exclusion, and often monotonicity.

**Interference** — One unit's treatment affects another unit's outcome.

**ITT** — Effect of randomized assignment or offer.

**LATE** — Average effect among instrument compliers under assumptions.

**Marginal structural model** — Model for counterfactual outcomes under histories, often fit with inverse weights.

**Mediator** — Post-treatment variable on a causal pathway.

**Negative control** — Variable expected to share bias but not the causal relation.

**Nuisance function** — Auxiliary propensity or outcome function needed for target estimation.

**Overlap** — Credible presence of both treatment comparisons across target profiles.

**Policy value** — Expected outcome or utility under an assignment rule.

**Positivity** — Required actions have nonzero probability in target histories.

**Potential outcome** — Outcome under a specified intervention.

**Propensity score** — Conditional treatment probability given baseline covariates.

**Regression discontinuity** — Local design exploiting an assignment cutoff.

**Sensitivity analysis** — Structured variation of assumptions or choices.

**Standardization** — Averaging conditional predictions under interventions over a target distribution.

**Target trial** — Protocol of the ideal randomized experiment being emulated.

**Time-zero bias** — Bias from misaligned eligibility, treatment classification, and follow-up.

**Transportability** — Conditions under which a causal result applies to another target setting.

---

## Research-paper reading ladder

---

| Paper                                       | Central contribution                    | Replication focus                               |
|---------------------------------------------|-----------------------------------------|-------------------------------------------------|
| Rubin (1974), Holland (1986)                | potential outcomes and causal estimands | observed versus missing counterfactuals         |
| Greenland, Pearl & Robins (1999)            | causal diagrams and adjustment          | collider and mediator failures                  |
| Rosenbaum & Rubin (1983)                    | propensity-score balancing              | balance before outcomes                         |
| Bang & Robins (2005)                        | doubly robust estimation                | four-model failure matrix                       |
| Chernozhukov et al. (2018)                  | orthogonal scores and cross-fitting     | in-sample versus cross-fit nuisance predictions |
| Hernán & Robins (2016)                      | target-trial emulation                  | align time zero                                 |
| Robins, Hernán & Brumback (2000)            | marginal structural models              | treatment-confounder feedback                   |
| Angrist, Imbens & Rubin (1996)              | IV and local effects                    | exclusion and complier population               |
| Hahn, Todd & van der Klaauw (2001)          | RD identification                       | bandwidth and manipulation                      |
| Callaway & Sant'Anna (2021)                 | staggered DiD group-time effects        | heterogeneous adoption                          |
| Athey & Imbens (2016), Wager & Athey (2018) | honest trees and causal forests         | discovery/estimation separation                 |
| Nadeau & Bengio (2003)                      | dependence in resampled comparison      | overlapping-fold uncertainty                    |
| Gebru et al. (2021), Mitchell et al. (2019) | dataset and model documentation         | complete research cards                         |

For each paper identify estimand, assignment mechanism, population, assumptions, estimator, uncertainty, evidence, failure mode, and smallest falsifying simulation.

---

## Suggested ten-day study rhythm

---

Each day:

1. 15 minutes: retrieve yesterday's assumptions.
2. 20 minutes: write target trial/assignment mechanism.
3. 30 minutes: draw DAG or timeline.
4. 40 minutes: derive central identification or estimator.
5. 55 minutes: code and validate on simulation.
6. 25 minutes: deliberately violate one assumption.
7. 20 minutes: reconstruct one paper claim.
8. 15 minutes: write diagnostics, sensitivity, and bounded result.

Do not learn causal inference as a menu of estimators. Repeatedly rebuild the path from intervention to observed comparison.

---

## Common beginner misconceptions

---

- Conditioning is not intervention.
- Both potential outcomes are never observed for one unit.
- "Treatment effect" needs population, interventions, outcome, horizon, and scale.
- More data do not fix identification.
- No measured imbalance does not prove exchangeability.
- Adjusting for every variable can create bias.
- Mediator adjustment changes the estimand.
- Collider conditioning opens paths.
- Randomization balances in expectation, not perfectly in every sample.
- Receipt comparison is not ITT.
- IV usually identifies a complier effect, not ATE.
- Propensity AUC is not the design objective.
- Extreme weights are scientific lack-of-support warnings.
- Matching discards can change the target population.
- Double robustness does not repair two wrong models or hidden confounding.
- Cross-fitting does not repair causal assumptions.
- A non-significant sensitivity or placebo test does not prove validity.
- Time-varying treatment cannot be backfilled to baseline.
- Pre-trends do not prove parallel trends.
- RD is local to a cutoff.
- High predicted risk is not high treatment benefit.
- CATE is not an individual causal label.
- Cross-validation folds are not independent datasets.
- Reproducibility is not replication or validity.

- A model card does not absolve unsafe deployment.
  - Retraining changes the intervention system.
  - A lockbox revised against is no longer a test.
- 

## Ethical and governance questions

---

- Is the intervention sufficiently defined and deliverable?
  - Who receives benefits, burdens, or opportunity costs?
  - Does treating one project reduce resources for another?
  - Who is excluded by positivity or missingness?
  - Can affected teams contest data, assignment, and outcome definitions?
  - Are local/complier effects being generalized beyond their population?
  - Are subgroup policies lawful and legitimate?
  - Who approves deviations from registration?
  - Who owns incident response, rollback, and retirement?
  - Are delayed harms and spillovers monitored?
  - Will feedback change future labels and eligibility?
  - What prospective design can test actual system impact?
- 

## Final readiness checklist

---

You are ready to complete the book when you can:

- translate policy language into a target trial and estimand;
- distinguish predictive conditioning from intervention;
- explain potential outcomes and identification assumptions;
- draw confounders, mediators, colliders, instruments, and selection;
- defend an adjustment set from paths and timing;
- analyze randomized assignment, clusters, attrition, and noncompliance;
- derive standardization, IPW, SMD, ESS, and AIPW;
- diagnose overlap and balance without outcome fishing;
- implement cross-fitting and whole-procedure uncertainty;
- quantify sensitivity to hidden confounding;
- align longitudinal treatment and time zero;
- explain g-formula, MSM, clone-censor-weight, and repeated trials;
- distinguish DiD, RD, IV, synthetic control, and interrupted time series;
- investigate heterogeneity honestly and evaluate policies out of sample;
- compare methods using paired evidence and practical equivalence;
- build data, model, and causal-study documentation;
- verify training-serving parity and monitor feedback;
- define retraining, rollback, and retirement governance;
- execute a registered lockbox analysis without outcome-driven revision; and
- state exactly what the result does not establish.

The book's final principle is:

Reproducibility preserves a causal argument; it cannot create one.